HTML & CSS Handbook

Semantic HTML · accessibility · forms · CSS cascade · box model · flexbox · grid · responsive design · animation · modern CSS

15 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — How the Web Works**
   - 1.1 The web in one picture
   - 1.2 URLs — the address of everything
   - 1.3 DNS — turning names into addresses
   - 1.4 Connections: TCP, TLS and the HTTP versions
   - 1.5 HTTP basics — requests and responses
   - 1.6 "What happens when you type a URL and press Enter?"
   - 1.7 The browser rendering pipeline
   - 1.8 Browser DevTools
   - 1.9 Your first page and file structure
   - 1.10 Networking fundamentals
   - Chapter 1 TL;DR
2. **Chapter 2 — HTML Structure and Text**
   - 2.1 Elements, tags and attributes
   - 2.2 Document structure and the doctype
   - 2.3 The `<head>`: metadata that matters
   - 2.4 Headings
   - 2.5 Paragraphs, whitespace and text blocks
   - 2.6 Text-level semantics
   - 2.7 Lists
   - 2.8 Links
   - 2.9 Character references and comments
   - 2.10 Validation
   - Chapter 2 TL;DR
3. **Chapter 3 — Semantic HTML and Accessibility**
   - 3.1 Why semantics and accessibility
   - 3.2 Landmarks
   - 3.3 Headings and the outline
   - 3.4 The accessibility tree
   - 3.5 Alt text
   - 3.6 ARIA
   - 3.7 Keyboard and focus
   - 3.8 Accessible forms
   - 3.9 WCAG 2.2
   - 3.10 Testing tools
   - Chapter 3 TL;DR
4. **Chapter 4 — Images, Media and Embeds**
   - 4.1 The `<img>` element
   - 4.2 Image formats — JPEG, PNG, GIF, WebP, AVIF, SVG
   - 4.3 Responsive images — `srcset`, `sizes` and `<picture>`
   - 4.4 Loading hints — `loading`, `decoding`, `fetchpriority`
   - 4.5 `<figure>` and `<figcaption>`
   - 4.6 Audio and video — `<audio>`, `<video>`, `<source>`, `<track>`
   - 4.7 Embedding other pages — `<iframe>`
   - 4.8 SVG basics
   - 4.9 `<canvas>` — an introduction
   - 4.10 Choosing the right element
   - Chapter interview round-up
   - Chapter TL;DR
5. **Chapter 5 — Tables and Forms**
   - 5.1 Table semantics
   - 5.2 How a form works — `action`, `method`, `name`
   - 5.3 Input types
   - 5.4 Choosing and grouping — `select`, `datalist`, `textarea`, `fieldset`, `output`, `progress`, `meter`
   - 5.5 Built-in validation and the Constraint Validation API
   - 5.6 `autocomplete` — help the browser fill the form
   - 5.7 Accessible error messages
   - Chapter TL;DR
6. **Chapter 6 — CSS Fundamentals**
   - 6.1 Syntax and adding CSS to a page
   - 6.2 Selectors
   - 6.3 Pseudo-classes and pseudo-elements
   - 6.4 Specificity
   - 6.5 The cascade and origins
   - 6.6 Inheritance
   - 6.7 Units
   - 6.8 Colours
   - 6.9 Custom properties (CSS variables)
   - 6.10 How it all fits together
   - Chapter TL;DR
7. **Chapter 7 — The Box Model and Layout Basics**
   - 7.1 The box model
   - 7.2 `box-sizing`
   - 7.3 Margin collapsing
   - 7.4 `display`
   - 7.5 Normal flow and formatting contexts
   - 7.6 Positioning
   - 7.7 Overflow
   - 7.8 `z-index` and stacking contexts
   - 7.9 Floats (legacy)
   - Chapter 7 TL;DR
8. **Chapter 8 — Flexbox**
   - 8.1 Flex container and flex items
   - 8.2 The two axes, `flex-direction` and `order`
   - 8.3 `flex-grow`, `flex-shrink` and `flex-basis`
   - 8.4 The `flex` shorthand and minimum sizes
   - 8.5 Alignment
   - 8.6 Wrapping and `gap`
   - 8.7 Common patterns
   - 8.8 Pitfalls, debugging, and interview prep
   - Chapter 8 TL;DR
9. **Chapter 9 — CSS Grid**
   - 9.1 Grid vocabulary
   - 9.2 Defining tracks

- 9.3 `repeat()`, `auto-fill` and `auto-fit`
- 9.4 Placing items
- 9.5 The implicit grid and auto-placement
- 9.6 Alignment
- 9.7 Subgrid
- 9.8 Grid vs Flexbox
- 9.9 Patterns
- 9.10 Debugging, pitfalls and interview prep
- Chapter 9 TL;DR

10. **Chapter 10 — Responsive Design**
    - 10.1 What responsive design is
    - 10.2 The viewport meta tag and viewport units
    - 10.3 Media queries
    - 10.4 Mobile-first in practice
    - 10.5 User-preference media queries
    - 10.6 Container queries
    - 10.7 Fluid typography with `clamp()`
    - 10.8 Responsive images recap (and `image-set()`)
    - 10.9 Intrinsic layouts — responsive without breakpoints
    - 10.10 Testing responsive design
    - Chapter 10 TL;DR
11. **Chapter 11 — Typography, Colour and Visual Effects**
    - 11.1 Fonts
    - 11.2 Loading fonts well: `font-display`, preload, subsetting
    - 11.3 Text properties
    - 11.4 Colour and contrast
    - 11.5 Backgrounds and gradients
    - 11.6 Borders, radius and outlines
    - 11.7 Shadows
    - 11.8 Filters, blend modes, clipping and masks
    - 11.9 `object-fit` and `object-position`
    - Chapter 11 TL;DR
12. **Chapter 12 — Transitions, Transforms and Animation**
    - 12.1 Transitions
    - 12.2 Timing functions (easing)
    - 12.3 2D transforms
    - 12.4 3D transforms
    - 12.5 Keyframe animations
    - 12.6 The Web Animations API (WAAPI)
    - 12.7 Performance: animate `transform` and `opacity`
    - 12.8 Enter and exit animations: `@starting-style` and `allow-discrete`
    - 12.9 Reduced motion and accessibility
    - 12.10 View transitions
    - 12.11 Scroll-driven animations
    - Chapter 12 TL;DR
13. **Chapter 13 — Modern CSS and Architecture**
    - 13.1 Native CSS nesting
    - 13.2 `:is()` and `:where()`
    - 13.3 `:has()` — the relational ("parent") selector
    - 13.4 Cascade layers (`@layer`)
    - 13.5 `@scope`
    - 13.6 Logical properties
    - 13.7 Why CSS needs an architecture — and BEM
    - 13.8 Utility-first CSS and Tailwind CSS v4
    - 13.9 CSS Modules
    - 13.10 CSS-in-JS
    - 13.11 Choosing an approach
    - 13.12 Preprocessors (Sass) and PostCSS
    - 13.13 Resets and normalize
    - 13.14 Theming and dark mode with custom properties
    - Chapter 13 TL;DR
14. **Chapter 14 — Performance, SEO, Interview Prep and Cheat Sheets**
    - 14.1 The critical rendering path
    - 14.2 Render-blocking resources
    - 14.3 Core Web Vitals: LCP, INP, CLS
    - 14.4 Fixing each metric (and the other basics)
    - 14.5 SEO basics
    - 14.6 Interview questions
    - 14.7 Cheat sheets
    - 14.8 Glossary
    - Chapter 14 TL;DR
15. **Chapter 15 — Interview Question Bank**
    - How to use this bank
    - HTML documents, the `<head>` and elements
    - Semantic HTML and SEO
    - Accessibility (ARIA, keyboard and focus, WCAG)
    - Forms and validation
    - Images, media and embeds
    - Browser storage and platform APIs
    - Selectors, specificity, the cascade and inheritance
    - Box model, `display` and units
    - Positioning, `z-index`, stacking contexts, floats and BFC
    - Flexbox
    - CSS Grid
    - Responsive design: media queries, container queries and fluid sizing
    - Centering techniques
    - Transitions, transforms, animations and visual effects
    - Custom properties and modern CSS
    - CSS architecture: preprocessors, BEM, utility-first and CSS-in-JS
    - Performance and the browser rendering pipeline
    - "Build this layout" — common live-coding tasks
    - Scenarios, troubleshooting and senior-level questions
    - Sources of the question lists

Chapter 1 — How the Web Works

This chapter follows one page from a server's disk to your screen: the URL that names it, the DNS lookup that finds the server, the HTTP conversation that delivers it, and the browser pipeline that turns text into pixels. It ends with a first real page, a sensible folder layout, and the networking fundamentals interviewers ask about.

In this chapter: 1.1 The web in one picture · 1.2 URLs · 1.3 DNS · 1.4 Connections: TCP, TLS, HTTP versions · 1.5 HTTP basics · 1.6 "What happens when you type a URL?" · 1.7 The rendering pipeline · 1.8 Browser DevTools · 1.9 Your first page and file structure · 1.10 Networking fundamentals.

Every example lives in `code/`. The `.mjs` files (ES-module JavaScript) run with **Node.js 24** — a program that runs JavaScript outside the browser (`node code/01_url_parts.mjs`). Scripts that need a DOM (§1.7) use **jsdom**, a library that imitates a browser document in Node; two drive real **headless Chrome** (Chrome with no window, controlled by a script). Outputs are copied from real runs.

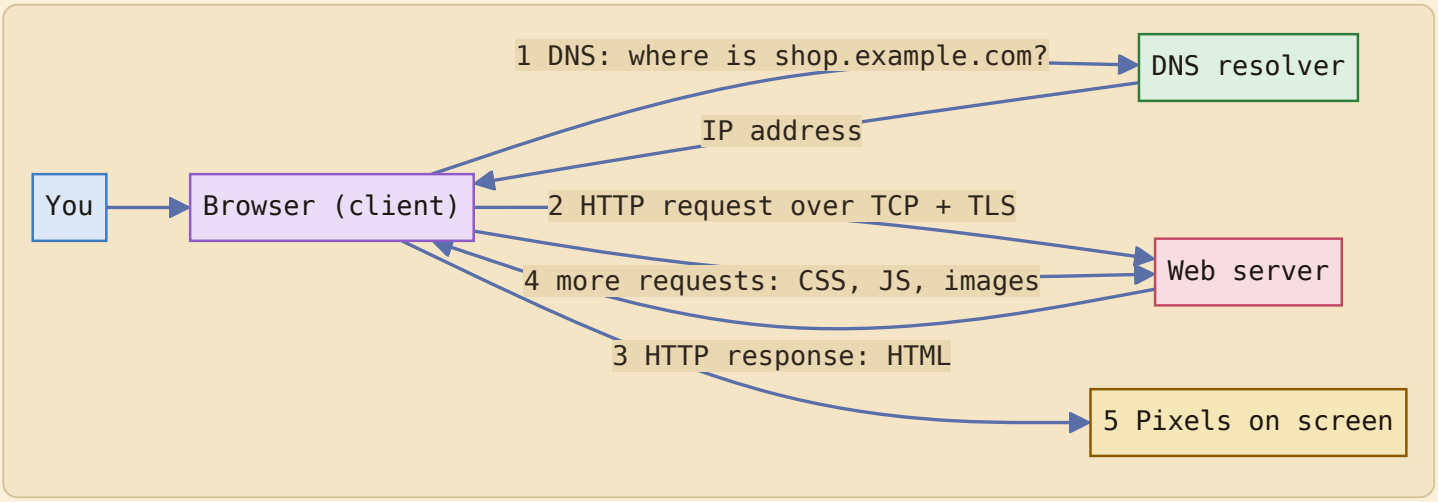
1.1 The web in one picture

What it is

The **Internet** is the global network of connected computers plus the low-level **protocols** (agreed rules for how programs talk) that move data between them: **IP** (Internet Protocol) gives every machine an address and moves small chunks called **packets**; TCP and UDP (§1.4, §1.10) carry data between programs. The **World Wide Web** is one service on top: documents and apps named by **URLs**, fetched with **HTTP**, written in **HTML** — the three ideas from Tim Berners-Lee's 1989 CERN proposal (first site live 1991). Email, games and SSH (Secure Shell, remote command-line login) use the Internet but are not "the web".

The roles

Term	Meaning	Example
Client	The program that asks for something	Your browser, a phone app, `curl` (command-line download tool)
Server	A program (and machine) that answers requests	nginx (a popular web server), a Node.js app, a CDN edge node (§1.3)
User agent	The spec's name for the program acting for the user — usually the browser	Chrome, Firefox, Safari, a screen reader (software that reads the page aloud) + browser
Resource	Anything a URL points at	An HTML page, an image, a JSON response
Browser engine	The part that parses (reads and understands) and draws pages	Blink (Chrome, Edge, Opera), Gecko (Firefox), WebKit (Safari)
JavaScript engine	The part that runs JS	V8 (Chrome, Node.js), SpiderMonkey (Firefox), JavaScriptCore (Safari)



The three languages of the front end

Language	Its job	If it were a house
HTML (HyperText Markup Language)	Structure and meaning: heading, list, button	Walls, rooms, doors
CSS (Cascading Style Sheets)	Presentation: colour, spacing, layout, animation	Paint, furniture, lighting
JavaScript	Behaviour: react to clicks, fetch data, change the page	Electricity and plumbing

Screen readers, search engines and "reader mode" use the HTML alone, so a well-structured page stays usable if CSS fails or JS is off. Building that way — working HTML first, CSS and JS layered on top — is **progressive enhancement**.

Who defines the rules

Technology	Standards body
HTML + DOM	**WHATWG** (Web Hypertext Application Technology Working Group) — a "living standard" edited continuously at html.spec.whatwg.org; no "HTML6", "HTML5" is a historical label
CSS	**W3C** CSS Working Group, as separate modules (Flexbox, Grid, Colour…)
HTTP	**IETF**, in RFCs (Requests for Comments)
JavaScript	**ECMAScript**, by Ecma TC39 (JavaScript book)
Accessibility (usability for people with disabilities)	WCAG (Web Content Accessibility Guidelines), W3C Web Accessibility Initiative (Chapter 3)

Baseline (a label on MDN — Mozilla's web reference — and web.dev) marks features that work in all core browsers (Chrome, Edge, Firefox, Safari, desktop and mobile): *newly available* when the last one ships it, *widely available* 30 months later. This book uses it whenever support matters.

1.2 URLs — the address of everything

What it is

A **URL** (Uniform Resource Locator) says *how* to reach a resource and *where* it is. Every link, image, stylesheet and API call is a URL.

How it works — the parts

`https://user@shop.example.com:8443/books/html.html?lang=en&page=2#reviews`

Part	In the example	Meaning
Scheme	`https:`	Protocol (`http`, `https`, `mailto`, `tel`, `file`, `data`…)
Userinfo	`user@`	Rare, legacy; never put passwords here
Host	`shop.example.com`	Domain name (human-readable server name) or IP address
Port	`:8443`	Which "door" on the server; omitted = scheme default (80 http, 443 https)
Path	`/books/html.html`	Which resource on that server
Query	`?lang=en&page=2`	`name=value` parameters separated by `&`
Fragment	`#reviews`	A spot *inside* the page (usually an element `id`); never sent to the server

The **origin** is scheme + host + port (`https://shop.example.com:8443`). Browsers isolate pages by origin — the **same-origin policy**: a script from one origin may not read another's data. It returns with cookies (small data a site stores in the browser), `fetch` (JS's HTTP function) and CORS (Cross-Origin Resource Sharing: how a server lets other origins read its responses) in the JavaScript book.

Node implements the same WHATWG URL Standard as browsers:

```mjs
// code/01_url_parts.mjs
const u = new URL('https://user@shop.example.com:8443/books/html.html?lang=en&page=2#reviews');
for (const k of ['href', 'protocol', 'username', 'hostname', 'port', 'origin', 'pathname', 'search', 'hash'])
  console.log(k.padEnd(9), u[k]);
console.log('lang    =', u.searchParams.get('lang'), '| page =', u.searchParams.get('page'));

// Relative URLs are resolved against the page's own URL (the "base").
const base = 'https://shop.example.com/books/html.html';
for (const rel of ['css.html', './img/cover.png', '../about.html', '/cart', '?page=3', '#top', '//cdn.example.net/a.js'])
  console.log(rel.padEnd(24), '->', new URL(rel, base).href);

// Special characters are percent-encoded; default ports disappear.
console.log(new URL('https://example.com:443/my file.html?q=café').href);
```

Output:

```
href      https://user@shop.example.com:8443/books/html.html?lang=en&page=2#reviews
protocol  https:
username  user
hostname  shop.example.com
port      8443
origin    https://shop.example.com:8443
pathname  /books/html.html
search    ?lang=en&page=2
hash      #reviews
lang    = en | page = 2
css.html                 -> https://shop.example.com/books/css.html
./img/cover.png          -> https://shop.example.com/books/img/cover.png
../about.html            -> https://shop.example.com/about.html
/cart                    -> https://shop.example.com/cart
?page=3                  -> https://shop.example.com/books/html.html?page=3
#top                     -> https://shop.example.com/books/html.html#top
//cdn.example.net/a.js    -> https://cdn.example.net/a.js
https://example.com/my%20file.html?q=caf%C3%A9
```

Absolute vs relative URLs

Kind	Looks like	Resolves to	Use for
Absolute	`https://other.site/page`	Exactly that	Other sites
Document-relative	`css.html`, `./img/a.png`, `../about.html`	The current page's folder (`.` = this folder, `..` = parent)	Files next to the page
Root-relative	`/cart`	Same origin, from the site root	Links inside a site served from a domain root
Scheme-relative	`//cdn.example.net/a.js`	Same scheme as the page	Legacy; write `https://` today
Query / fragment only	`?page=3`, `#top`	Same page, new query or jump target	Pagination, in-page links

Relative links make a site movable between your laptop, staging (a private test copy) and production (the live site).

Common mistakes

- Backslashes (`img\cat.png`) — URLs always use `/`.
- Root-relative links (`/css/style.css`) when opening files from disk: `/` becomes the root of your *disk*. Use a local server (§1.9) or document-relative paths.
- Secrets or personal data in the query string — URLs end up in logs, history and `Referer` headers (which tell a server the page the visitor came from).
- Forgetting the fragment is client-side only: the server never sees `#reviews`, and changing only the fragment makes no new request.

Self-check

1. Origin of `http://example.com/a/b?c#d`? → `http://example.com` (port 80 implied).
2. On `https://site.com/docs/intro/start.html`, where does `../api.html` go? → `https://site.com/docs/api.html`.

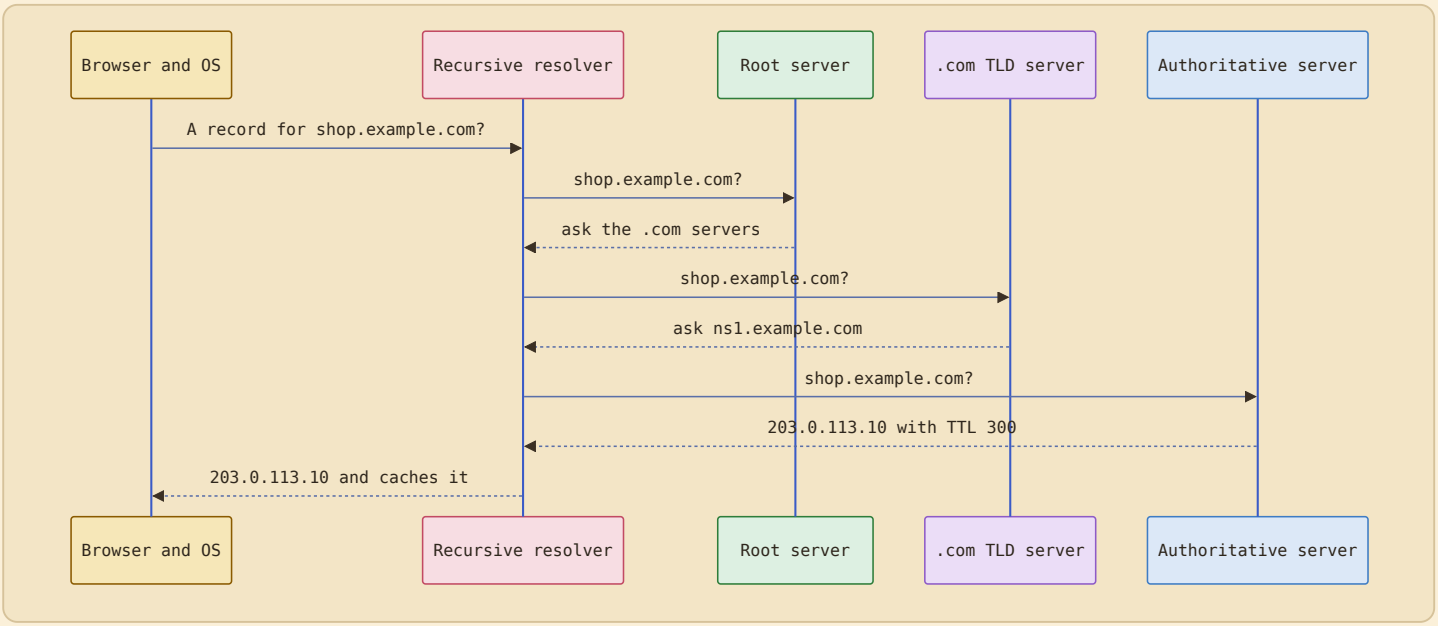
1.3 DNS — turning names into addresses

What it is

Computers route by **IP address** — e.g. `203.0.113.10` (IPv4) or `2001:db8::10` (IPv6, §1.10), both documentation ranges. The **Domain Name System (DNS)** is the Internet's distributed phone book: no single server holds it all (System Design Fundamentals).

How it works

1. The browser checks its **cache** (stored recent answers), then the OS checks its cache and the `hosts` file (`/etc/hosts` on macOS/Linux).
2. On a miss, the OS asks a **recursive resolver** — your ISP's (Internet service provider), your company's, or a public one such as 1.1.1.1 or 8.8.8.8.
3. The resolver walks the name from the right: a **root server** names the `.com` servers; the **TLD (top-level domain) server** names the authoritative server for `example.com`; the **authoritative name server** returns the record.
4. Each answer carries a **TTL** (time to live, seconds); resolver, OS and browser cache it that long, so most lookups take milliseconds.



Record types you will meet

Record	Maps	Example use
`A`	name → IPv4 address	`shop.example.com → 203.0.113.10`
`AAAA`	name → IPv6 address	same, for IPv6
`CNAME`	name → another name (alias)	`www.example.com → example.netlify.app`
`MX`	domain → mail servers	where `@example.com` email goes
`TXT`	arbitrary text	ownership checks, SPF/DKIM (email anti-spoofing)
`NS`	domain → its authoritative name servers	delegation (handing part of the name tree to other servers)

Classic DNS uses unencrypted UDP port 53; **DNS over HTTPS (DoH)** and **DNS over TLS (DoT)** encrypt lookups; most browsers can use DoH.

Real-world examples

- "Pointing a domain" at GitHub Pages, Netlify or Vercel = adding an `A` or `CNAME` record at your registrar (where you bought the name).

- A **CDN** (content delivery network: Cloudflare, Akamai) answers DNS with the IP of the nearest **edge server** (one of many copy servers close to users), so the same name reaches different machines per country (CDN notes).
- DNS changes "take time to propagate" because old answers stay cached until their TTL expires — so lower the TTL a day before moving servers.

1.4 Connections: TCP, TLS and the HTTP versions

What it is

With an IP address, the browser needs a reliable, private channel:

- **TCP** (Transmission Control Protocol): an ordered, reliable byte stream. Opening it costs one **round trip** (message out, reply back) — the *three-way handshake* SYN, SYN-ACK, ACK.
- **TLS** (Transport Layer Security): encrypts the stream and proves the server's identity with a **certificate** (a digital ID binding a domain to a public key) signed by a certificate authority (e.g. Let's Encrypt, which verifies you control the domain). TLS 1.3 (2018) adds one round trip for a new connection. `https://` = HTTP inside TLS.
- **HTTP**: the request/response language spoken inside.

Why HTTPS matters

Three guarantees: **confidentiality** (others on the Wi-Fi cannot read it), **integrity** (nobody injects ads or malware in transit), **authentication** (it really is `bank.com`). Browsers flag plain HTTP "Not secure"; service workers (background scripts that intercept requests, enabling offline use), geolocation, camera and clipboard require a **secure context** (HTTPS or `localhost`); Let's Encrypt made certificates free.

HTTP versions compared

Version	Year (current RFC)	Transport	Key idea	Practical effect
HTTP/1.1	1997 (RFC 9112, 2022)	TCP	Text messages, persistent connections	One request at a time per connection; browsers open ~6 per host
HTTP/2	2015 (RFC 9113, 2022)	TCP	Binary framing, **multiplexing** many requests on one connection, header compression	Many small files are cheap; one lost packet still stalls every stream (TCP **head-of-line blocking**)
HTTP/3	2022 (RFC 9114)	**QUIC** over UDP	Independent streams; TLS 1.3 built in	Faster set-up, better on lossy mobile networks, survives network changes

All versions share the same *semantics* — methods, status codes, headers (RFC 9110). Server and browser negotiate the version; your HTML is unchanged.

1.5 HTTP basics — requests and responses

What it is

HTTP (HyperText Transfer Protocol): one request, exactly one response. It is **stateless** — each request stands alone, so anything remembered between requests (login, cart) travels in cookies, tokens (signed strings proving identity) or URLs.

How it works — anatomy

Request: `method path version` line, headers (`Name: value`), blank line, optional body. **Response:** `version status reason` line, headers, blank line, body.

The script talks to a tiny local server over a raw TCP **socket** (one end of a network connection) to show the exact bytes; HTTP/2 and /3 carry the same information in binary frames.

```
// code/02_http_exchange.mjs (key parts)
function send(res, status, headers, body = '') {
  res.sendDate = false;                                  // no Date header -> identical output on every run
  res.writeHead(status, { ...headers, 'Content-Length': Buffer.byteLength(body) });
  res.end(body);
}
const server = http.createServer((req, res) => {
  if (req.url === '/') send(res, 200, { 'Content-Type': 'text/html; charset=utf-8', 'Cache-Control': 'max-age=60' },
                            '<!DOCTYPE html><title>Hi</title><h1>Hello</h1>');
  else if (req.url === '/old') send(res, 301, { Location: '/' });
  else send(res, 404, { 'Content-Type': 'text/plain' }, 'Not found');
});
// ... raw(path) opens a socket and writes:
//   `GET ${path} HTTP/1.1\r\nHost: localhost\r\nAccept: text/html\r\nConnection: close\r\n\r\n`
await raw('/'); await raw('/old'); await raw('/missing');
```

Output for `/` (abridged from the run):

```
>>> REQUEST
GET / HTTP/1.1
Host: localhost
Accept: text/html
Connection: close
<<< RESPONSE
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Cache-Control: max-age=60
Content-Length: 46
Connection: close

<!DOCTYPE html><title>Hi</title><h1>Hello</h1>
```

`/old` returns `HTTP/1.1 301 Moved Permanently` with `Location: /` and `Content-Length: 0`; `/missing` returns `HTTP/1.1 404 Not Found` with `Content-Type: text/plain` and body `Not found`.

- `Host` is mandatory in HTTP/1.1 — one IP can serve many sites.
- `Content-Type: text/html; charset=utf-8` tells the browser the bytes are HTML and how to decode them; the browser trusts this header, not the `.html` extension.
- `301` + `Location` is a redirect: the browser immediately requests `/`.
- The fragment (`#...`) never appears in a request line.

Methods

Method	Purpose	Safe? (no side effects)	Idempotent? (repeat = same result)	Sent by plain HTML?
`GET`	Read a resource	Yes	Yes	Links, images, forms with `method="get"`
`HEAD`	Like GET, headers only	Yes	Yes	No
`POST`	Create / submit / trigger an action	No	No	Forms with `method="post"`
`PUT`	Replace a resource	No	Yes	No (use `fetch`)
`PATCH`	Partially update	No	Not necessarily	No
`DELETE`	Remove	No	Yes	No
`OPTIONS`	Ask what is allowed (CORS preflight — a permission check before the real request)	Yes	Yes	No

HTML can only send `GET` and `POST` (links and forms, Chapter 5); `fetch` can send any method.

Status codes

Class	Meaning	Codes you should know
1xx	Informational	`103 Early Hints`
2xx	Success	`200 OK`, `201 Created`, `204 No Content`
3xx	Redirection	`301 Moved Permanently`, `302 Found`, `304 Not Modified`, `307` / `308` (redirects that keep the method)
4xx	Client error	`400 Bad Request`, `401 Unauthorized` (not logged in — "who are you?"), `403 Forbidden` (logged in, not allowed), `404 Not Found`, `429 Too Many Requests`
5xx	Server error	`500 Internal Server Error`, `502 Bad Gateway`, `503 Service Unavailable`

Headers worth recognising

Header	Direction	What it does
`Host`	request	Which site on this server
`User-Agent`	request	Identifies the browser
`Accept`, `Accept-Language`, `Accept-Encoding`	request	Preferred formats, languages, compression (gzip, br, zstd)
`Cookie` / `Set-Cookie`	request / response	Small pieces of state: server sets, browser sends back
`Content-Type`, `Content-Length`, `Content-Encoding`	both	What the body is, how big, how compressed
`Location`	response	Where a redirect points
`Cache-Control`, `ETag`, `Last-Modified`	response	Caching rules and validators (version stamps for freshness checks)
`Content-Security-Policy`, `Strict-Transport-Security`	response	Security policies (which scripts may run; always use HTTPS)

Caching in one paragraph

`Cache-Control: max-age=60` lets the browser reuse a response for 60 s without asking. After that it *revalidates*: sends `If-None-Match: <etag>` and gets a tiny `304 Not Modified` if unchanged. Sites put a content hash (a fingerprint of the file's bytes) in asset names (`app.3f9a2c.css`) and cache them for a year, while HTML is cached briefly or revalidated every time. More in Chapter 14 and the caching module.

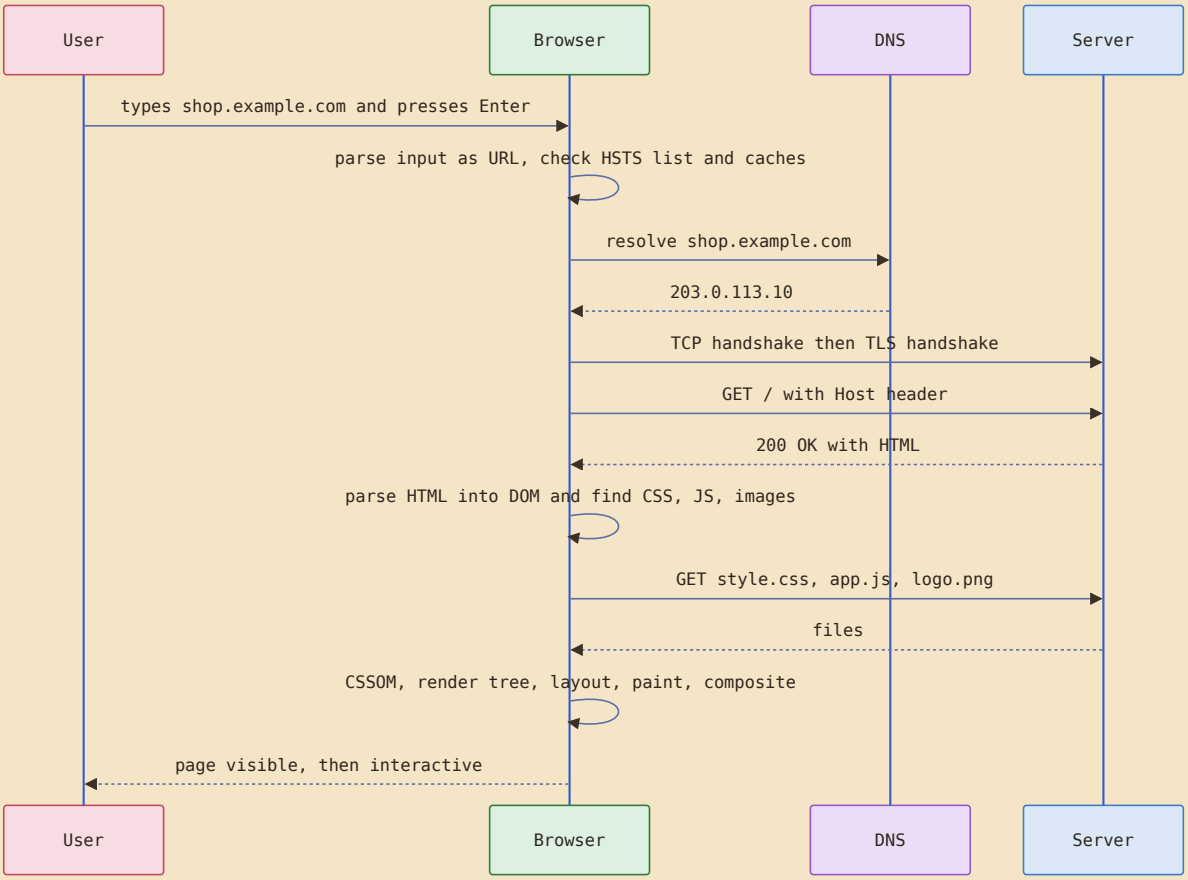
Common mistakes

- Changing data with a `GET` link (`<a href="/delete?id=5">`): crawlers (search robots) and link-prefetchers follow GETs.
- Serving HTML as `text/plain` — the browser shows the source.
- Confusing `401` with `403`.

Self-check

1. What does `304` mean and when do you see it? → Not Modified; when revalidating a cached copy that is still current.
2. Why is HTTP called stateless? → Every request is independent; state travels in cookies, tokens or URLs.

1.6 "What happens when you type a URL and press Enter?"



In the interview

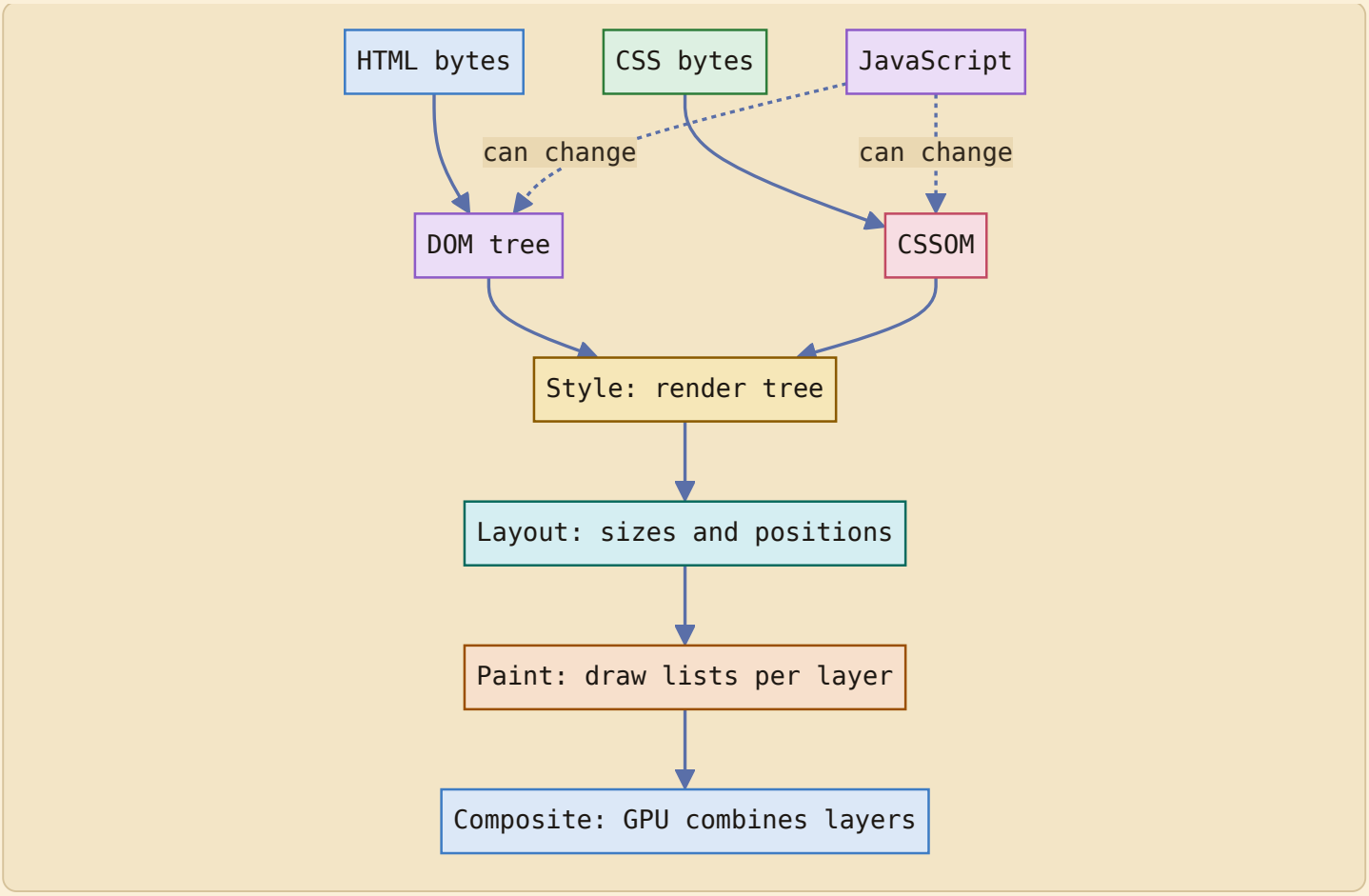
30-second answer: "The browser decides whether I typed a URL or a search, then resolves the host to an IP via its caches and DNS. It opens TCP and does a TLS handshake (or QUIC for HTTP/3), sends a GET with headers such as Host and cookies, and gets a response — maybe a redirect, ideally 200 with HTML. It parses the HTML into the DOM while fetching the CSS, JS and images it discovers, builds the CSSOM, combines them into a render tree, runs layout, paints and composites layers. Scripts then make the page interactive."

- *How could you make that faster?* → Fewer round trips (HTTP/2 or /3, a CDN near users, keep-alive — reusing one open connection — and `preconnect` to open connections early), smaller cacheable assets, no unnecessary render-blocking CSS/JS (Chapter 14).
- *What is HSTS?* → `Strict-Transport-Security` tells the browser to always use HTTPS for that site, so it upgrades `http://` itself without sending an insecure request.
- *Where can caching short-circuit this?* → DNS caches, the browser HTTP cache (no request at all under `max-age`), a service worker, a CDN edge cache.

1.7 The browser rendering pipeline

What it is

The **rendering pipeline** (the *critical rendering path* when talking about first view) is how an engine turns HTML and CSS bytes into pixels:



Step 1 — Parsing HTML into the DOM

Bytes are decoded to characters (using the `charset`), split into **tokens** (start tags, end tags, text, comments) and fed to a **tree builder**. The result is the **DOM** (Document Object Model): a tree of objects — *nodes* — that JavaScript can read and change. The WHATWG spec defines parsing exactly, including error recovery, so every modern browser builds the same tree from the same input. It never "fails".

> **Key term — tree:** each **node** has one parent and any number of children. `<html>` is the root; `<head>` and `<body>` are its children; a paragraph's text is a child of its `<p>`. "Sibling" = same parent; "descendant" = anywhere below.

jsdom implements the same algorithm, so we can watch it fix a sloppy document:

```
// code/03_parse_dom.mjs (key parts)
const body = `<title>Parsing demo</title>
<p>First paragraph
<p>Second <b>bold <i>both</b> italic?</i>
<table><tr><td>cell</td></tr></table>`;
const std = new JSDOM('<!DOCTYPE html>\n' + body).window.document;
console.log('Mode:', std.compatMode, '(CSS1Compat = standards mode)');
show(std.documentElement);                        // prints the tree, one node per line
console.log('\nSerialized back:\n' + std.body.innerHTML);
const quirks = new JSDOM(body).window.document;   // same markup, no doctype
console.log('Is the <table> inside the 2nd <p>?  standards:', !!std.querySelector('p > table'),
            '| quirks:', !!quirks.querySelector('p > table'));
```

Output:

```
Mode: CSS1Compat (CSS1Compat = standards mode)
Source had no <html>, <head>, <body>, </p> or <tbody> — the parser added them:

html
  head
    title
      "Parsing demo"
  body
    p
      "First paragraph"
    p
      "Second"
      b
        "bold"
        i
          "both"
      i
        "italic?"
    table
      tbody
        tr
          td
            "cell"

Serialized back:
<p>First paragraph
</p><p>Second <b>bold <i>both</i></b><i> italic?</i>
</p><table><tbody><tr><td>cell</td></tr></tbody></table>

Without <!DOCTYPE html> -> mode: BackCompat (BackCompat = quirks mode)
Is the <table> inside the 2nd <p>?  standards: false | quirks: true
```

What happened: `<html>`/`<head>`/`<body>` were created; a new `<p>` closed the previous one (paragraphs cannot nest); mis-nested `<b><i></b></i>` was repaired by reopening a fresh `<i>` after `</b>` (the "adoption agency algorithm"); a `<tbody>` was inserted (so `table > tr` never matches). Without `<!DOCTYPE html>` the page is in **quirks mode**, imitating 1990s browser bugs — here a table does *not* close the paragraph. Always use the doctype (Chapter 2) and validate (§2.10) rather than rely on recovery.

Scripts and parsing. A plain `<script src>` is *parser-blocking*: the parser stops, downloads and runs it (it might call `document.write`, an old function that inserts markup mid-parse), then continues.

Script tag	Downloads	Runs	Order kept?	Use for
`<script src>`	blocks parsing	immediately	yes	Rarely needed today
`<script src defer>`	in parallel	after parsing, before `DOMContentLoaded`	yes	Most page scripts
`<script src async>`	in parallel	as soon as downloaded (pauses parser briefly)	no	Independent scripts, e.g. analytics
`<script type="module">`	in parallel	deferred by default	yes	ES modules (files that `import`/`export`)

A **preload scanner** reads ahead in the raw HTML and starts downloading CSS, scripts and images early, even while the parser is blocked.

Step 2 — Parsing CSS into the CSSOM

CSS from `<link rel="stylesheet">`, `<style>` and `style=""` is parsed into the **CSSOM** (CSS Object Model): rules, their selectors (which elements a rule targets) and declarations (`property: value`). CSS is **render-blocking**: the browser won't paint until it has the stylesheets in `<head>`, to avoid a "flash of unstyled content" (FOUC). So stylesheets belong in the head and should be small.

Step 3 — Style and the render tree

The browser matches every element against the rules and computes a final value for every property (**computed style**; cascade and specificity in Chapter 6). Elements that produce boxes form the **render tree** (Chrome: *layout*

tree). Left out: `display: none` elements and their contents, and never-rendered elements (`<head>`, `<script>`, `<meta>`, `<link>`). `visibility: hidden` and `opacity: 0` elements *stay* — invisible but occupying space.

`code/04_render_tree.mjs` styles five elements with `.hidden { display: none }`, `.ghost { visibility: hidden }`, `.faded { opacity: 0 }` and `p { color: #333 }`, prints the CSSOM, then reads `getComputedStyle()` per element:

Output:

```
CSSOM rules:
  .hidden { display: none; }
  .ghost { visibility: hidden; }
  .faded { opacity: 0; }
  p { color: rgb(51, 51, 51); }

Element           display  visibility  in render tree?  painted visibly?
<h1>               block    visible     true             true
<p>                block    visible     true             true
<p .hidden>        none     visible     false            false
<p .ghost>         block    hidden      true             false
<p .faded>         block    visible     true             false
<script>           none     visible     false            false
```

The CSSOM stores parsed values (`#333` → `rgb(51, 51, 51)`). In real Chrome (`code/render-tree.html`):

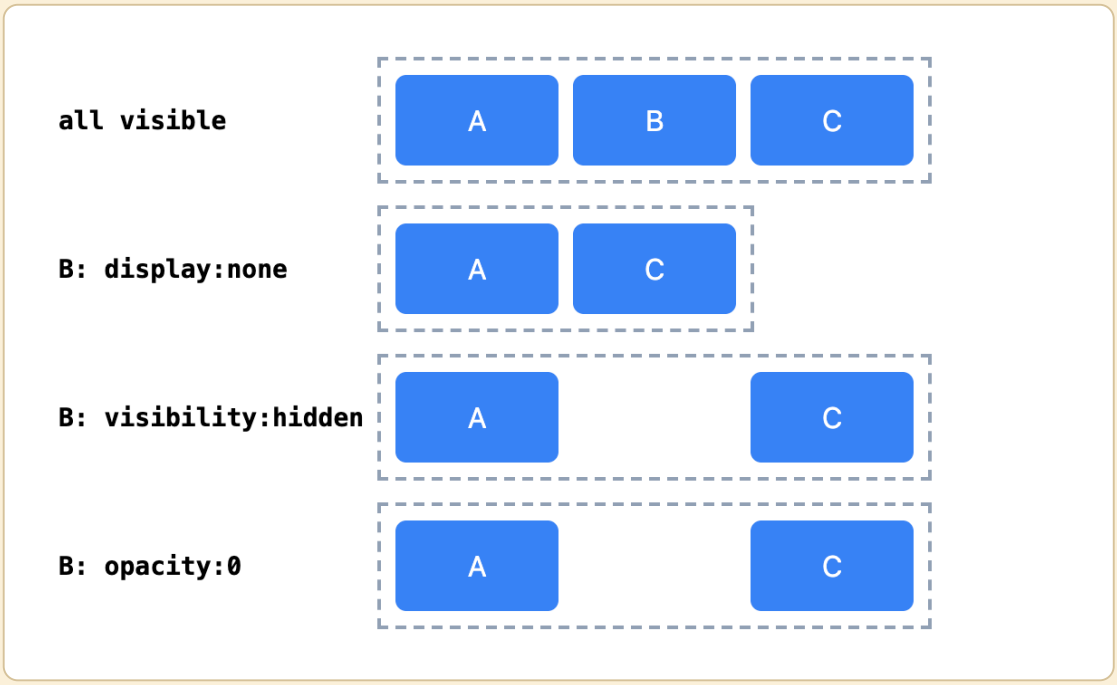


Figure 1.1 — `display: none` removes B's box entirely; `visibility: hidden` and `opacity: 0` keep its space.

Technique	Takes space?	Clickable?	Read by screen readers?
`display: none`	No	No	No
`visibility: hidden`	Yes	No	No
`opacity: 0`	Yes	**Yes**	**Yes**
`hidden` attribute	No (it applies `display: none`)	No	No

Step 4 — Layout

Layout (Firefox: *reflow*) computes each box's exact position and size in pixels, starting from the **viewport** width (the visible window area): percentages, `em`s, flex and grid become concrete numbers. Any later geometric change — width, font size, new content — can force layout again.

Step 5 — Paint

Paint turns boxes into drawing instructions — backgrounds, borders, text, shadows, images — in stacking order. The page may be split into **layers** (e.g. a fixed header, a video, `will-change: transform`), painted separately, then *rasterised* (converted to pixels), often on the GPU.

Step 6 — Composite

Compositing stacks the layers on screen. `transform` and `opacity` changes can often be handled here alone — no layout or paint — which is why they animate smoothly at 60 fps or more (Chapter 12).

What a change costs

You change	Layout	Paint	Composite	Example
`width`, `height`, `margin`, `top`, font size, content	yes	yes	yes	Expanding an accordion
`color`, `background-color`, `box-shadow`	no	yes	yes	Hover colour
`transform`, `opacity`	no	no	yes	Sliding a drawer

Events you can observe

`DOMContentLoaded` = HTML parsed and deferred scripts run (images may still load); `load` = everything, including images and iframes, loaded. First Contentful Paint (first text/image) and Largest Contentful Paint (biggest visible element) are milestones in DevTools and Lighthouse (Chapter 14).

Common mistakes

- `<script>` without `defer` in `<head>` — nothing renders until it downloads and runs.
- Hiding with `opacity: 0` and forgetting it is still clickable and read aloud.
- Reading a layout value (`offsetHeight`) right after changing styles in a loop — **layout thrashing** (recomputing geometry many times when once would do).

In the interview

> **30-second answer:** "The browser parses HTML into the DOM and CSS into the CSSOM, combines them into a render tree of visible boxes, runs layout for geometry, paints each layer, and composites layers on the GPU. CSS is render-blocking and classic scripts are parser-blocking, so I keep critical CSS small and load scripts with `defer` or as modules. For animation I prefer `transform` and `opacity` because they skip layout and paint."

- *`display:none` vs `visibility:hidden`?* → The first removes the box from the render tree and frees its space; the second keeps the space but draws nothing.
- *`defer` vs `async`?* → Both download in parallel; `defer` runs in document order after parsing, `async` runs on arrival, in any order.
- *What is a reflow?* → Recomputing layout because geometry changed; expensive on large pages.

Self-check

1. Is `<head>` in the render tree? → No.
2. What does the parser do with a `<p>` inside a `<p>`? → Closes the first paragraph.

1.8 Browser DevTools

What it is

Every desktop browser ships **developer tools** showing the DOM, styles, network traffic and performance. Open with **F12**, **Ctrl+Shift+I** (Windows/Linux), **Cmd+Option+I** (macOS), or right-click → *Inspect*.

The panels you will use (Chrome names; Firefox and Safari are similar)

Panel	What it shows	Use it to…
Elements (Firefox: Inspector)	The live DOM (not your source) + applied CSS with overridden declarations struck through, box model diagram, computed values	Find why a style isn't applied, try CSS live, see flex/grid overlays
Console	Errors, warnings, `console.log`; a JS prompt	Spot failing scripts, test expressions (`$0` = selected element)
Sources (Firefox: Debugger)	Loaded files; breakpoints (where execution pauses)	Step through JavaScript
Network	Every request: status, type, size, timing waterfall, headers	Find 404s, slow files, check caching and `Content-Type`
Performance	Timeline of parsing, scripting, style, layout, paint	Find what makes the page slow or janky (stuttering)
Lighthouse	Audits for performance, accessibility, SEO (search engine optimisation), best practices	A quick report card (Chapter 14)
Application (Firefox: Storage)	Cookies, local storage (key–value data in the browser), service workers, cache	Debug state and offline behaviour
Accessibility pane	The accessibility tree, roles, names, contrast	Chapter 3

Also: the **device toolbar** (Ctrl/Cmd+Shift+M) simulates phones and slow networks; "Rendering" offers paint flashing and emulating `prefers-color-scheme: dark` (Chapter 10).

Seeing what the Network panel sees

`code/05_what_the_browser_fetched.mjs` serves the §1.9 site, opens it in headless Chrome via Puppeteer (a Node library that remote-controls Chrome), logs every response (`page.on('response')`) like the Network list, then reads the `<h1>`'s computed colour (proving the CSS applied) and clicks the button (proving the JS runs):

Output:

```
status type       path            content-type
200  document   /index.html     text/html; charset=utf-8
200  stylesheet /css/style.css  text/css
200  script     /js/main.js     text/javascript
404  other      /favicon.ico

After load: { title: 'My First Page', elements: 31, h1Color: 'rgb(29, 78, 216)' }
After clicking "Say hi": Hi! JavaScript ran.
```

The parser discovered two more requests. The `404` for `/favicon.ico` is the browser automatically asking for a tab icon; `<link rel="icon" href="...">` (Chapter 2) controls it.

Common mistakes

- Editing styles in DevTools and forgetting they vanish on reload.
- Reading the Elements panel as your source: it shows the DOM *after* parser fixes and JS changes. "View Page Source" (Ctrl/Cmd+U) shows the original.
- Debugging caching with "Disable cache" still ticked (it applies only while DevTools is open).

Self-check

1. How do you check a file's status code and `Content-Type`? → Network panel → click the request → Headers.

1.9 Your first page and file structure

How it works

1. Create `index.html` in a folder (in a plain-text editor such as VS Code, not a word processor) — servers return `index.html` for a folder URL (`https://site.com/` → `/index.html`).
2. Open it by double-clicking (`file://` URL) or, better, a local server so URLs behave as in production: `npx serve .` (npx runs a Node package without installing it) or `python3 -m http.server 8000`, then `http://localhost:8000/` (`localhost` = your own machine).
3. Edit, save, reload; inspect with DevTools.

The complete first page (every line explained in Chapter 2):

```
<!-- code/site/index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My First Page</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header>
    <h1>Hello, web!</h1>
    <nav><a href="index.html">Home</a> · <a href="about.html">About</a></nav>
  </header>
  <main>
    <p>This site is four plain text files: <code>index.html</code>, <code>about.html</code>, <code>css/style.css</code>
and <code>js/main.js</code>.</p>
    <p>The browser read the HTML, fetched the stylesheet it links to, and drew the result.</p>
    <ul>
      <li>HTML says <strong>what</strong> things are.</li>
      <li>CSS says how they <strong>look</strong>.</li>
      <li>JavaScript says how they <strong>behave</strong>.</li>
    </ul>
    <button type="button" id="hi">Say hi</button>
    <output id="msg"></output>
  </main>
  <footer><small>Made with a text editor.</small></footer>
  <script src="js/main.js"></script>
</body>
</html>
```

```
/* code/site/css/style.css (excerpt) */
body   { font-family: system-ui, sans-serif; max-width: 40rem; margin: 2rem auto; padding: 0 1rem; line-height: 1.5;
color: #1f2933; }
header { border-bottom: 2px solid #3b82f6; margin-bottom: 1rem; }
h1     { color: #1d4ed8; margin: 0 0 .25rem; }
button { font: inherit; padding: .4rem .9rem; border-radius: 6px; border: 1px solid #1e3a8a; background: #1d4ed8; color:
white; }
```

```
// code/site/js/main.js
document.getElementById('hi').addEventListener('click', () => {
  document.getElementById('msg').textContent = 'Hi! JavaScript ran.';
});
```

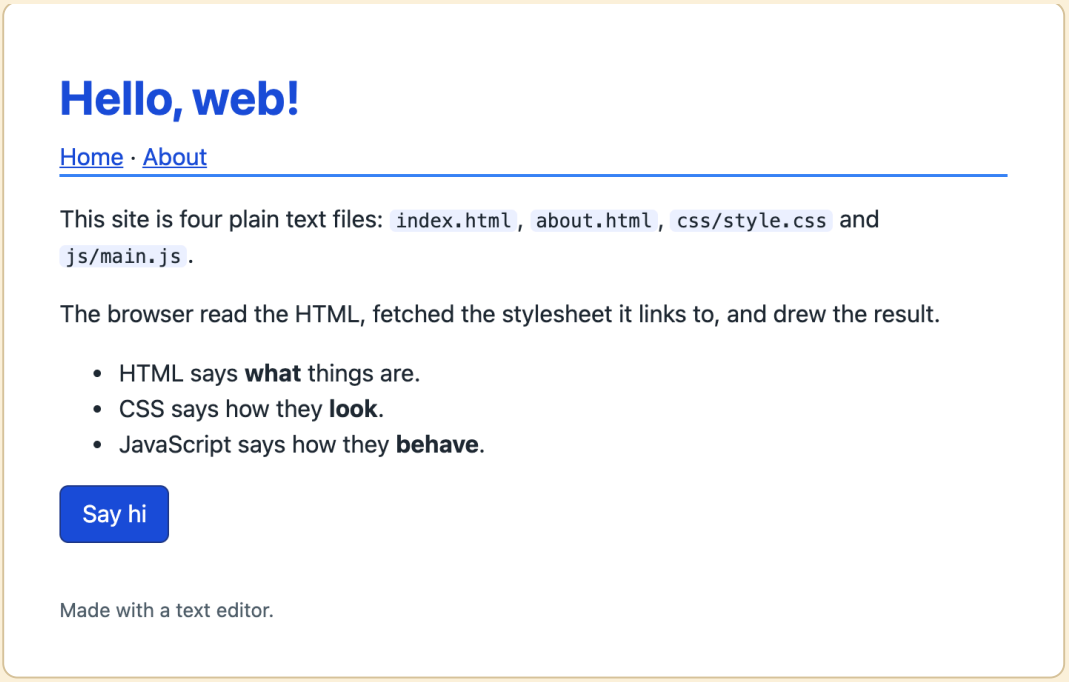


Figure 1.2 — `code/site/index.html` in Chrome.

The script sits at the end of `<body>` so the button exists when it runs; `<script src="js/main.js" defer>` in the head is equally good.

A sensible file structure

```
my-site/
├── index.html          home page  →  /
├── about.html          →  /about.html
├── blog/
│   └── index.html      →  /blog/
├── css/
│   └── style.css
├── js/
│   └── main.js
├── img/
│   ├── logo.svg
│   └── hero.avif
└── favicon.ico
```

Rule	Why
Lower-case, hyphenated names (`contact-us.html`)	URLs are case-sensitive on most servers; spaces become `%20`; hyphens are what search engines expect
One folder per asset type (`css/`, `js/`, `img/`) — or per feature in larger projects	Easy to find, easy to set cache rules
`index.html` in every folder that is a "page"	Clean URLs such as `/blog/`
Document-relative links between pages	Works from any folder or domain
Keep source and generated files apart once you add build tools (programs that transform and bundle files for production): `src/` → `dist/`	Never hand-edit generated output

Real-world examples

Static sites (ready-made files, no server code per request) are **deployed** by uploading the folder to GitHub Pages, Netlify, Cloudflare Pages or an S3 bucket behind a CDN. Frameworks (React, Next.js, Astro) generate the same HTML, CSS and JS files; the browser receives exactly what this chapter describes.

Common mistakes

- Saving as `index.html.txt` (hidden extensions on Windows/macOS).

- Linking `href="CSS/Style.css"` when the file is `css/style.css` — works on a case-insensitive laptop, breaks on a Linux server.
- Missing `<!DOCTYPE html>` or `<meta charset="utf-8">`: quirks mode and garbled accents.

Self-check

1. Why prefer a local server over double-clicking? → Root-relative URLs, `fetch`, modules and some APIs behave differently or fail on `file://`.
2. Why is the script at the end of `<body>` (or `defer`)? → Parsing isn't blocked and the elements it uses already exist.

1.10 Networking fundamentals

What it is

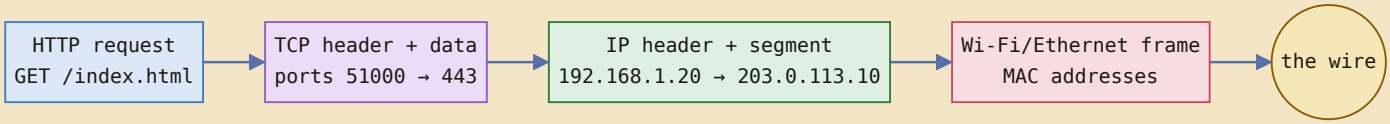
The machinery under §1.3–1.5: **layers**, transports (**TCP**, **UDP**, **QUIC**), addressing (**IPv4, IPv6, NAT**), and boxes in the path (**firewalls, proxies, VPNs**). Interviewers ask about it, and it explains why your laptop is `192.168.1.20` while websites see another address.

How it works — the layer models

Protocols are stacked in **layers**: each uses the one below and serves the one above, wrapping the data in its own **header** (labels and control info in front of the data) — **encapsulation**.

OSI model (7 layers, a teaching model, ISO 7498)	TCP/IP model (4 layers, what the Internet actually runs, RFC 1122)	Examples	Unit of data
7 Application · 6 Presentation · 5 Session	**Application**	HTTP, DNS, TLS*, SSH, SMTP (email sending)	message
4 Transport	**Transport**	TCP, UDP, QUIC	segment / datagram
3 Network	**Internet**	IP (v4, v6), ICMP (`ping`; error and test messages)	packet
2 Data link · 1 Physical	**Link**	Ethernet, Wi-Fi, fibre, radio	frame / bits

*TLS sits between transport and application — "layer 4.5", or OSI layer 6.



A "layer-4" **load balancer** (spreads requests across identical back-ends) looks only at IPs and ports; "layer 7" reads HTTP (URLs, headers, cookies). See Load Balancing and Reverse Proxy.

How it works — ports and sockets

An IP address finds the **machine**; a **port** (0–65535) finds the **program**. A **socket** = IP + port + protocol. Servers listen on **well-known ports**; clients get a random **ephemeral port** (short-lived, one connection) from the OS (`61804` in the demo below).

Port	Service	Port	Service
22	SSH	443	HTTPS (TCP), HTTP/3 (UDP)
25 / 587	SMTP (mail between servers / from clients)	3306 / 5432	MySQL / PostgreSQL
53	DNS (UDP, TCP for large answers)	6379	Redis (in-memory data store)
80	HTTP	8080 / 3000 / 5173	common dev-server choices

How it works — TCP vs UDP vs QUIC

	TCP (RFC 9293)	**UDP** (RFC 768)	**QUIC** (RFC 9000)
Connection set-up	3-way handshake (1 round trip), then TLS	none — just send	1 round trip including TLS 1.3; 0-RTT on resume
Delivery	reliable, ordered, retransmits lost data	best effort: may lose, duplicate or reorder	reliable and ordered **per stream**
Shape of data	one **byte stream**, no message boundaries	separate **datagrams**, boundaries kept	many independent streams in one connection
Flow / congestion control (not overwhelming receiver / network)	yes	no (the app must be polite)	yes
Head-of-line blocking	one lost packet stalls everything behind it	none	only the stream that lost the packet waits
Used by	HTTP/1.1, HTTP/2, SSH, databases, email	DNS, video calls, games, VoIP, DHCP (hands out IP addresses)	HTTP/3

Games and calls use UDP because late voice or game data is useless — skip it rather than stall for a retransmission. QUIC runs **on top of UDP** because routers and firewalls everywhere already pass UDP, while a new transport would be blocked; QUIC re-implements reliability and encryption itself, and its connection IDs let a download survive moving from Wi-Fi to 4G.

How it works — addresses: IPv4, IPv6, NAT

- **IPv4** (RFC 791): 32-bit addresses (~4.3 billion) written `203.0.113.10`. They ran out — the last big regional pools were exhausted between 2011 and 2019.
- **IPv6** (RFC 8200): 128-bit addresses in hex groups, `2001:db8::10` (`::` = a run of zeros) — enough for every device to have a public address. It also drops header checksums and in-network fragmentation (routers splitting oversized packets) and makes auto-configuration standard. Both run side by side (**dual stack**); browsers try IPv6 and IPv4 almost in parallel ("Happy Eyeballs", RFC 8305) and use whichever connects first.
- **CIDR notation** `10.0.0.0/8` = "first 8 bits fixed" — 2^24 addresses. Used for subnets (address blocks belonging to one network) and firewall rules.
- **Private ranges** (RFC 1918): `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16` — never routed publicly; reusable by any home or office. `127.0.0.1` / `::1` is **loopback** (this machine).
- **NAT** (network address translation) lets a private network share one public IPv4 address: the router rewrites each outgoing packet's source to its public IP and a fresh port, remembers the mapping, and rewrites replies back. Outsiders cannot start a connection inward without a mapping — which is why peer-to-peer video calls need STUN/TURN (helper servers that discover your public address or relay traffic), as used by WebRTC (the browser's real-time media API).
- **Unicast** = one host; **broadcast** = every host on the local network (IPv4 only); **multicast** = a subscribed group; **anycast** = one address announced from many places, routed to the nearest — how public DNS resolvers and CDNs answer from a nearby city.
- **LAN** = one building's network; **WAN** = networks joined across distance.

How it works — DNS records beyond the basics

Beyond §1.3's records: `SOA` (start of authority: zone's primary server and timers), `PTR` (reverse lookup IP → name, used by mail servers to check senders), `SRV` (service → host + port, e.g. SIP telephony or some databases), `CAA` (which certificate authorities may issue certificates for the domain). A `CNAME` cannot sit at the bare domain (`example.com`) alongside other records, so DNS hosts offer non-standard "ALIAS/ANAME" flattening (resolving the alias themselves and publishing the IP).

How it works — firewalls, proxies and VPNs

- A **firewall** filters by rules. A *packet filter* matches addresses, ports, protocols ("allow TCP 443 from anywhere, 22 only from 10.0.0.0/8, deny the rest"). A *stateful* firewall tracks connections, so replies to traffic you started come back automatically; cloud **security groups** are stateful. A **WAF** (web application firewall) works at layer 7, inspecting HTTP for attacks such as SQL injection.

- A **forward proxy** acts for **clients** (a company proxy: filtering, caching, logging). A **reverse proxy** acts for **servers** — nginx, a load balancer or a CDN in front of your app — doing **TLS termination** (decrypting HTTPS so the app needn't), caching and load spreading. To the browser, a reverse proxy *is* "the server".
- A **VPN** (virtual private network) tunnels your packets encrypted into another network so you behave as if plugged in there. **Remote-access VPNs** connect a laptop to the office; **site-to-site VPNs** join two networks (offices, or office and cloud). Protocols: **IPsec**, **WireGuard** (small, modern), TLS-based such as OpenVPN. A VPN hides traffic from the local Wi-Fi, not from the VPN provider.

Seeing it: `code/06_network_basics.mjs`

On loopback only: resolves `localhost`, sends three TCP writes and three UDP datagrams, checks addresses against an allow-list (`net.BlockList`), simulates a NAT table.

```
localhost -> ::1 (IPv6), 127.0.0.1 (IPv4)
isIPv4("203.0.113.10") = true | isIPv6("2001:db8::10") = true
TCP server listening on 127.0.0.1:61803
TCP server sees client 127.0.0.1:61804 (ephemeral port picked by the OS)
TCP bytes received      : "one|two|three|" (in order, as one stream)
UDP datagrams received  : ["one","two","three"] (boundaries kept; on a real network some could be lost or reordered)
firewall: 192.168.1.20  private=true  -> ALLOW (admin port 22)
firewall: 10.4.5.6      private=true  -> ALLOW (admin port 22)
firewall: 172.20.0.1    private=true  -> ALLOW (admin port 22)
firewall: 203.0.113.10  private=false -> DENY
firewall: 8.8.8.8       private=false -> DENY
NAT out: 192.168.1.10:51000 -> 203.0.113.5:40001
NAT out: 192.168.1.11:51000 -> 203.0.113.5:40002
NAT in : reply to :40001     -> 192.168.1.10:51000
NAT in : packet to :40999    -> DROP (no mapping: unsolicited inbound traffic)
all assertions passed
```

TCP delivered `one|two|three|` as one stream — the `|` separators exist because TCP keeps no message boundaries, so every protocol on top (HTTP included) must frame its messages (`Content-Length`, chunked encoding — body sent in size-prefixed pieces — and frames). UDP delivered three separate messages. Port numbers change every run.

Common mistakes

- "UDP is unreliable, so never use it" → right when late data is worthless (live media, games) or the app adds reliability (QUIC, DNS retries).
- Treating TCP reads as messages → one `send` can arrive split, or several together; always frame.
- Thinking servers see your `192.168.x.x` address → they see the router's public address after NAT.
- Opening a database port (5432, 6379) to `0.0.0.0/0` in a security group → allow only the app servers' subnet.
- Assuming a VPN makes you anonymous → the VPN operator sees what the Wi-Fi provider used to.

In the interview

30-second answer — TCP vs UDP: "TCP is connection-based: a handshake, then a reliable, ordered byte stream with flow and congestion control — used by HTTP/1.1 and /2, databases and SSH. UDP sends independent datagrams with no guarantees and no set-up cost — used by DNS, games and video calls where speed beats completeness. QUIC builds reliable, encrypted, multiplexed streams on top of UDP and powers HTTP/3."

- *OSI vs TCP/IP?* → OSI is a 7-layer teaching model; TCP/IP is the 4-layer model the Internet runs (application, transport, internet, link).
- *Why IPv6?* → IPv4's 32-bit space ran out; NAT stretched it at the cost of end-to-end reachability. IPv6's 128 bits give every device a public address.
- *Forward vs reverse proxy?* → Forward acts for clients (egress filtering — controlling outgoing traffic); reverse acts for servers (TLS termination, load balancing, caching).
- *Why does HTTP/3 use UDP rather than a new protocol?* → Middleboxes (routers, firewalls) already pass UDP; QUIC adds reliability and TLS itself.

Self-check

1. Which layer does a port belong to? → Transport (TCP/UDP).
2. What does a home router's NAT table map? → Public IP:port ↔ private IP:port per outgoing connection.

Sources: RFC 1122 (Internet host layers), RFC 9293 (TCP), RFC 768 (UDP), RFC 9000 (QUIC), RFC 791 / RFC 8200 (IPv4 / IPv6), RFC 1918 (private addresses), RFC 8305 (Happy Eyeballs), RFC 1035 (DNS), Node.js docs for `net` and `dgram`.

Chapter 1 TL;DR

- The **web** runs on the Internet: **URLs** name resources, **HTTP** fetches them, **HTML** describes them; CSS styles, JavaScript adds behaviour.
- **URL** = scheme://host:port/path?query#fragment; **origin** = scheme + host + port; relative URLs resolve against the page URL.
- **DNS** maps names to IPs through resolver → root → TLD → authoritative servers, with TTL-based caching.
- Connections use **TCP + TLS** (HTTPS) or **QUIC** (HTTP/3). HTTPS gives confidentiality, integrity and authentication.
- **HTTP** is stateless request/response: method + path + headers → status + headers + body. Know GET/POST, 2xx/3xx/4xx/5xx, `Content-Type`, `Cache-Control`.
- **Rendering**: HTML → DOM, CSS → CSSOM → render tree → layout → paint → composite. The parser always recovers from bad markup; the doctype keeps you out of quirks mode. `transform`/`opacity` changes are cheapest.
- CSS is render-blocking; classic scripts are parser-blocking; use `defer` or modules.
- A site is a folder of text files; `index.html` is the default page; lower-case, hyphenated names and relative links keep it portable.
- **Networking**: layers (application → transport → IP → link), each adding a header; **TCP** = reliable ordered stream, **UDP** = independent datagrams, **QUIC** = reliable encrypted streams over UDP (HTTP/3); IPv4 (32-bit, stretched by **NAT** and private ranges) vs IPv6 (128-bit); firewalls filter, reverse proxies front servers, VPNs tunnel.

Next: Chapter 2 — HTML Structure and Text.

Chapter 2 — HTML Structure and Text

HTML is the skeleton of every page. This chapter covers the grammar (elements, attributes, nesting), the fixed document structure, the `<head>` **metadata** (data *about* the page — title, language, character set) that browsers, search engines and social networks read, the elements for text (headings, paragraphs, inline meaning, lists, links), and finally character references, comments and validation.

In this chapter: 2.1 Elements, tags and attributes · 2.2 Document structure and the doctype · 2.3 The `<head>` · 2.4 Headings · 2.5 Paragraphs and text blocks · 2.6 Text-level semantics · 2.7 Lists · 2.8 Links · 2.9 Character references and comments · 2.10 Validation.

Examples are in `code/`. Each `.html` file is shown as a headless-Chrome screenshot; the `.mjs` scripts parse the same files with jsdom (a Node.js library that builds a browser-like DOM without a browser): `node code/01_head.mjs`.

2.1 Elements, tags and attributes

What it is

HTML (HyperText Markup Language) **marks up** plain text: you wrap content in **tags** that say what each piece *is*.

```
<a href="https://example.com" title="Opens the example site">Visit Example</a>
```

Piece	In the example
Start tag	`<a href="https://example.com" title="Opens the example site">`
Element name	`a`
Attributes (name = value)	`href` = `https://example.com`, `title` = `Opens the example site`
Content	`Visit Example`
End tag	`</a>`

- An **element** = start tag + content + end tag (`<p>Hello</p>`) — and the DOM node it becomes; a *tag* is just the `<p>` / `</p>` text. Elements nest into a tree (the DOM, Chapter 1).
- An **attribute** adds information: `name="value"`. Quote values (double quotes by convention); names are case-insensitive but write them lower-case.
- **Void elements** have no content and no end tag: `area`, `base`, `br`, `col`, `embed`, `hr`, `img`, `input`, `link`, `meta`, `source`, `track`, `wbr`. A trailing slash (`<br />`) is allowed and ignored — an XHTML habit (the stricter XML-based HTML of the 2000s).
- **Boolean attributes** are true when present, false when absent: `<input disabled>`, `<ol reversed>`. `disabled="false"` and `required="no"` still mean *on*.
- **Nesting** must be proper: `<b><i>…</i></b>`, not `<b><i>…</b></i>`.

Global attributes (usable on any element)

Attribute	Purpose
`id`	Unique identifier: link target (`#id`), label/ARIA references (ARIA = extra accessibility attributes, Chapter 3), JS hook
`class`	Space-separated names for CSS and JS; shared by many elements
`style`	Inline CSS on one element (avoid except quick tests or dynamic values)
`title`	Advisory hover tooltip; not reliably shown on touch or read by screen readers — never the only place for essential info
`lang`	Content language (`en` , `fr-CA`) — affects pronunciation, hyphenation, fonts, translation
`dir`	Text direction: `ltr` , `rtl` (e.g. Arabic), `auto`
`hidden`	Not rendered (like `display: none`)
`tabindex`	Focus order — *focus* = the element receiving keyboard input (Chapter 3)
`data-*`	Your own data for scripts: `data-product-id="42"` → `el.dataset.productId`
`inert`	Makes a subtree unfocusable and unclickable (e.g. content behind a modal dialog)
`translate` , `spellcheck` , `contenteditable`	Translation hints, spell-check, in-place editing

Block-level and inline — how elements flow

Some elements start on a new line at full width (**block**: `p` , `h1` , `ul` , `div` , `section`); others sit inside a line (**inline**: `a` , `strong` , `em` , `span` , `code`). That is only the *default CSS* `display` *value* (Chapter 7). The spec instead uses **content categories** — *flow content* (almost everything in `body`) and *phrasing content* (text and inline markup) — and each element's **content model** says what it may contain. A `<p>` takes only phrasing content, so a `<div>` or `<ul>` inside it is invalid (the parser closes the `<p>` first). An `<a>` is *transparent* (allowed content = whatever its parent allows): it may wrap blocks (a whole card) as long as it contains no other interactive element (link, button, input).

`<div>` (block) and `<span>` (inline) are generic containers with no meaning — hooks for CSS and JS when nothing meaningful fits.

Common mistakes

- Unquoted values with spaces: `class=big red` gives class `big` and a stray attribute `red` .
- Repeating an `id` — use `class` for groups.
- `<br></br>` or `<img></img>` — void elements have no end tag.
- Choosing elements for their default look ("I need bold, so `<h3>` "). Choose by meaning; CSS handles looks.

2.2 Document structure and the doctype

What it is

Every document has the same outer shape:

```
<!DOCTYPE html>
<html lang="en">
  <head>  <!-- information ABOUT the page: not displayed -->
  </head>
  <body>  <!-- the page itself: everything the user sees -->
  </body>
</html>
```

Part	Role
`<!DOCTYPE html>`	Must be first. Switches on **standards mode**; without it, **quirks mode** emulates old browser bugs (Chapter 1 showed a table ending up *inside* a paragraph). Case-insensitive; not an element.
`<html lang="en">`	The **root element**. `lang` sets screen-reader pronunciation, enables translation and correct hyphenation; required by WCAG (Web Content Accessibility Guidelines) success criterion 3.1.1 Language of Page.
`<head>`	Metadata: character set (how bytes map to letters), title, viewport, CSS, scripts, icons, social previews (the card shown when a link is shared).
`<body>`	Content.

The `html`, `head` and `body` tags are technically optional (the parser creates them), but always write them.

The full skeleton with a realistic head is `code/skeleton.html`:

```
<!DOCTYPE html>
<!-- The doctype above switches the browser to standards mode. This is an HTML comment. -->
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Trail Running Basics — Mountain Club</title>
  <meta name="description" content="A beginner's guide to trail running: shoes, pacing, safety and the five trails our club runs every week.">
  <link rel="icon" href="data:image/svg+xml,<svg ...><circle ... fill='%23f97316'/></svg>">
  <link rel="canonical" href="https://club.example.org/guides/trail-running">
  <meta name="theme-color" content="#f97316">
  <meta property="og:title" content="Trail Running Basics">
  <meta property="og:image" content="https://club.example.org/img/trail-card.jpg">
  <style> body { font-family: system-ui, sans-serif; max-width: 36rem; margin: 1.5rem auto; padding: 0 1rem; line-height: 1.5; } </style>
</head>
<body>
  <h1>Trail Running Basics</h1>
  <p>Everything in <code>&lt;head&gt;</code> is information <em>about</em> the page. Everything in <code>&lt;body&gt;</code> is the page.</p>
  <p>The title, description and icon you cannot see here show up in the browser tab, in search results and in link previews.</p>
</body>
</html>
```

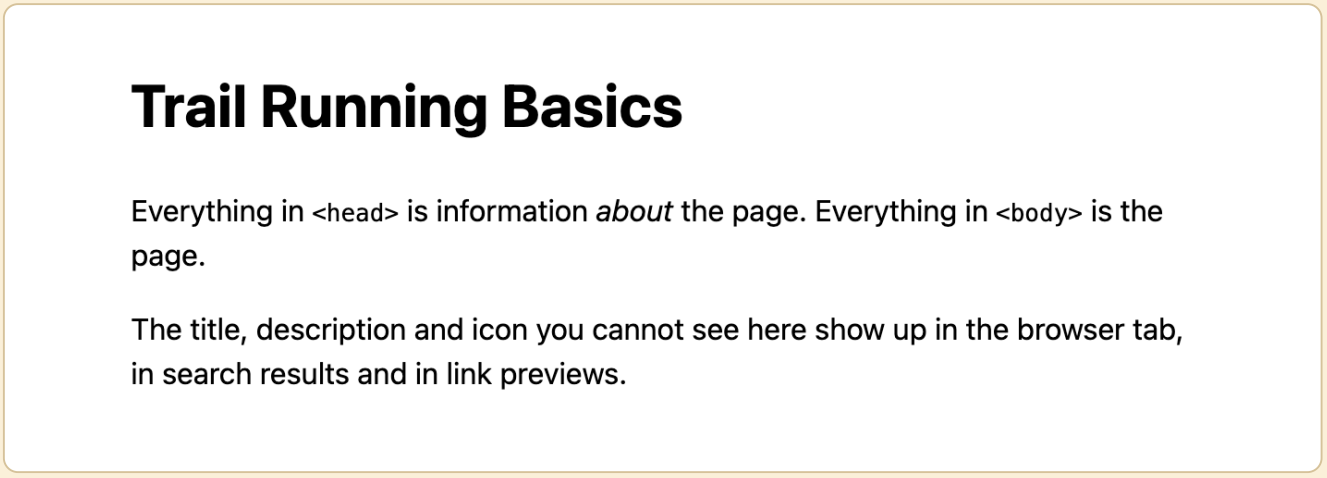


Figure 2.1 — Only the `<body>` is drawn. The head shows up elsewhere: tab title and icon, search snippets, link previews.

Old doctypes such as `<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" ...>` (DTD = Document Type Definition, the old formal grammar) mostly still trigger standards or "almost standards" mode — a few, such as HTML 4.01 Transitional without its URL, trigger quirks — so never write them today.

Self-check

1. Where does the `<title>` appear? → Tab/window title, bookmarks, history, search-result headline — not in the page body.

2.3 The `<head>`: metadata that matters

How it works

The browser reads the head top to bottom before rendering. Order matters twice: `charset` first, and render-blocking CSS (Chapter 1) early so it downloads early.

`code/01_head.mjs` parses `skeleton.html` with jsdom (as if served from `https://club.example.org/guides/trail-running`) and prints what a browser, search engine or social network learns — doctype mode, `lang`, `characterSet`, title, every `meta[name]`/`meta[property]` and `link`, comments and head children:

```
doctype   : <!DOCTYPE html> -> CSS1Compat
lang      : en
charset   : UTF-8 (declared at byte 140 — must be within the first 1024)
title     : Trail Running Basics — Mountain Club
meta      : viewport     = width=device-width, initial-scale=1
meta      : description  = A beginner's guide to trail running: shoes, pacing, safety a
meta      : theme-color  = #f97316
meta      : og:title     = Trail Running Basics
meta      : og:image     = https://club.example.org/img/trail-card.jpg
link      : icon         -> data:image/svg+xml,<svg xmlns='http://www.w3.org/2000/svg' v
link      : canonical    -> https://club.example.org/guides/trail-running
comments  : [
  'The doctype above switches the browser to standards mode. This is an HTML comment.'
]
head children: meta, meta, title, meta, link, link, meta, meta, meta, style
```

The essential head elements

Element	Example	Why
`<meta charset="utf-8">`	—	How to turn bytes into characters. Must be within the **first 1024 bytes**. Use UTF-8 (the universal encoding covering every language and emoji) only; otherwise accents turn to garbage ("cafÃ©"). An HTTP `Content-Type: …; charset=` header overrides it.
`<meta name="viewport">`	`content="width=device-width, initial-scale=1"`	Without it phones lay out at ~980 px and shrink the page. Makes CSS pixels (logical pixels, possibly several device dots each) match the device width — the foundation of responsive design (Chapter 10). Never add `maximum-scale=1` or `user-scalable=no`: blocking zoom fails WCAG 1.4.4 (Resize Text).
`<title>`	`Trail Running Basics — Mountain Club`	Required. Unique per page, most specific part first. First thing a screen reader announces; the search-result headline.
`<meta name="description">`	one or two sentences	Often the search snippet (engines may pick other text), usually cut at ~150–160 characters. Not a ranking factor — a pitch for the click. Not visible on the page.
`<link rel="stylesheet">`	`href="css/style.css"`	Loads CSS (render-blocking).
`<script src defer>` / `type="module"`	`src="js/app.js"`	Loads JS without blocking the parser (Chapter 1).
`<link rel="icon">`	`.ico`, `.png` or `.svg` (Chapter 4)	The tab icon (favicon). Add `rel="apple-touch-icon"` for iOS home screens.
`<link rel="canonical">`	full URL	The preferred ("official") URL when the same content lives at several addresses.
Open Graph (Facebook-originated) / Twitter Card metas	`property="og:title"`, `og:description`, `og:image`	The preview card on social networks and chat apps.
`<meta name="theme-color">`	`#f97316`	Tints the browser UI on some mobile browsers.
`<meta name="robots">`	`content="noindex"`	Keeps a page out of search results.
`<base href>`	`href="/docs/"`	Changes the base for *all* relative URLs. Rarely a good idea — it surprises people.
`<link rel="preload">`, `rel="preconnect"`	fonts, hero image, CDN origin	"Fetch this early" / "connect early" hints (Chapter 14).

Keep a copy-paste **template**: doctype, `lang`, charset, viewport, title, description, stylesheet, deferred script.

Common mistakes

- `<meta charset>` after a long comment or a non-ASCII `<title>` (é, emoji…) — it may fall outside the first 1024 bytes.
- The same `<title>` ("Home") on every page.
- Visible content in the head: the parser moves it into the body.
- Disabling zoom in the viewport tag.

2.4 Headings

What it is

`<h1>`…`<h6>` are section headings, `h1` most important; together they form the page's **outline**.

Why it matters

- **Screen-reader users** navigate by headings: in WebAIM's (a US accessibility non-profit) surveys, a large majority say jumping through headings is how they first find information on a long page (Chapter 3).
- **Everyone** skims headings; **search engines** use them to understand each part of the page.

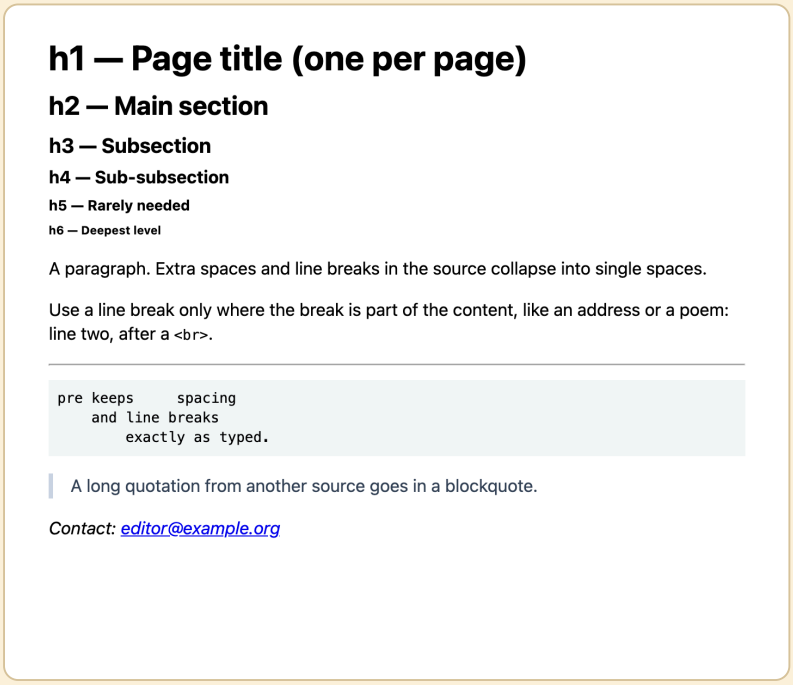


Figure 2.2 — `code/text-blocks.html`: the six heading levels at default sizes, then the block text elements from §2.5.

Rules that work

1. **One `<h1>` per page**, describing it (usually close to the `<title>`). The spec allows more; one is clear best practice.
2. **Don't skip levels going down** (`h2` → `h3`, not `h4`). Going back up any number (`h4` → `h2`) is fine — it starts a new major section.
3. **Choose the level by structure, never size.** Want a small `h2`? Use CSS.
4. **Don't use headings for non-headings** (a big slogan is a `<p>` with a class).
5. Heading + tagline: `<hgroup><h1>Title</h1><p>Tagline</p></hgroup>` — so the tagline isn't a fake heading.

There is no "document outline algorithm". Old HTML5 drafts let each `<section>` restart heading levels (so every section could use `h1`). No browser or screen reader implemented it and the WHATWG removed it in 2022: heading levels are what the numbers say, and `<section>` doesn't change them.

Common mistakes

- `<h4>` because it "looks right" — a screen reader announces "heading level 4" and users assume missing sections.
- Several `h1`s in a template (logo *and* page title).
- `<div class="title">` headings — they appear in no heading list.

2.5 Paragraphs, whitespace and text blocks

What it is

Element	Meaning	Notes
`<p>`	A paragraph	Don't use empty `<p>`s for spacing — use CSS `margin`.
` `	A line break that is part of the content	Addresses, poems, lyrics. Never for layout spacing.
`<hr>`	A thematic break (topic or scene change)	Drawn as a line; may be announced as a separator. Decorative lines → CSS border.
`<pre>`	Preformatted text: whitespace and line breaks kept, monospace font	Code blocks (`<pre><code>…</code></pre>`), ASCII art
`<blockquote>`	A long quotation from another source	Optional `cite="URL"` (not shown); visible attribution outside it or in a `<figcaption>` (Chapter 4)
`<address>`	Contact information for the nearest `<article>` or the page	Not for any postal address in text
`<div>`	Generic block container, no meaning	When nothing else fits

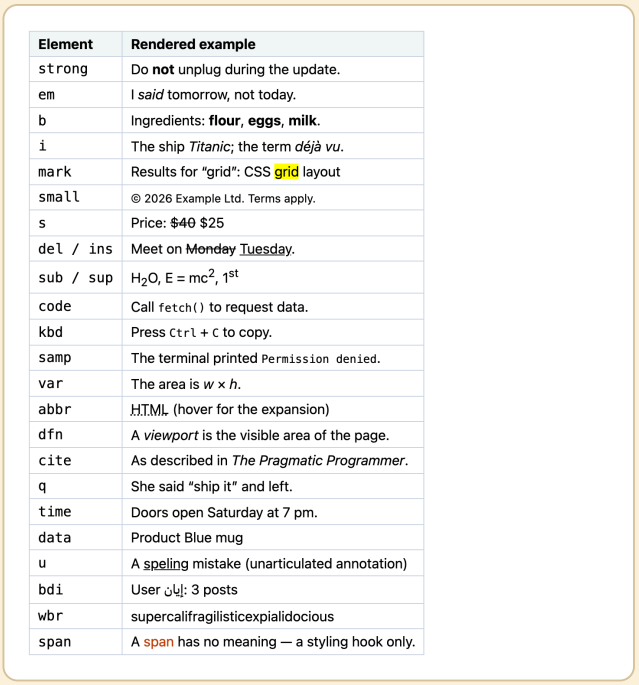
Whitespace collapsing

In normal text any run of spaces, tabs and newlines **collapses** to one space, and leading/trailing whitespace in a line is dropped — so you can indent HTML freely. To keep whitespace use `<pre>` or CSS `white-space`; for a space that never breaks a line use ` ` (§2.9).

2.6 Text-level semantics

What it is

Inline elements that say what a *phrase* is. Most have a default look, but choose by meaning, then style with CSS.



Element	Rendered example
`strong`	Do **not** unplug during the update.
`em`	I *said* tomorrow, not today.
`b`	Ingredients: **flour**, **eggs**, **milk**.
`i`	The ship *Titanic*; the term *déjà vu*.
`mark`	Results for "grid": CSS grid layout
`small`	© 2026 Example Ltd. Terms apply.
`s`	Price: ~~$40~~ $25
`del / ins`	Meet on ~~Monday~~ Tuesday.
`sub / sup`	H_2O, $E = mc^2$, 1^{st}
`code`	Call `fetch()` to request data.
`kbd`	Press `Ctrl` + `C` to copy.
`samp`	The terminal printed `Permission denied`.
`var`	The area is *w* × *h*.
`abbr`	HTML (hover for the expansion)
`dfn`	A *viewport* is the visible area of the page.
`cite`	As described in *The Pragmatic Programmer*.
`q`	She said "ship it" and left.
`time`	Doors open Saturday at 7 pm.
`data`	Product Blue mug
`u`	A speling mistake (unarticulated annotation)
`bdi`	User ؋ل: 3 posts
`wbr`	supercalifragilisticexpialidocious
`span`	A span has no meaning — a styling hook only.

Figure 2.3 — `code/inline-semantics.html`: every common text-level element with its default rendering.

Element	Means	Use for	Not for
`<strong>`	Strong importance, seriousness or urgency	Warnings, key facts	Headings, general "make it bold"
`<em>`	Stress emphasis that changes the meaning	"I *said* tomorrow"	Titles, foreign words
`<b>`	Draw attention without extra importance	Keywords, product names in a review, lead sentence	Importance (use `strong`)
`<i>`	Different voice or mood	Technical terms, foreign phrases (`<i lang="fr">`), ship names, thoughts	Emphasis (use `em`), icons
`<mark>`	Highlighted for relevance to the current context	Search matches	Style-only highlighting
`<small>`	Side comments, fine print	Copyright, legal notes	Making text small for design
`<s>`	No longer accurate or relevant	Old price	Document edits (use `del`)
`<del>` / `<ins>`	Removed / added text in an edit	Change tracking; optional `datetime`, `cite`	—
`<sub>` / `<sup>`	Subscript / superscript required by typography	H_2O, footnote markers, 1st	Pure visual offsets
`<code>`	Computer code fragment	Inline code; blocks in `<pre><code>`	—
`<kbd>`	User input (keys, commands)	Ctrl+C	—
`<samp>`	Sample program output	Error messages	—
`<var>`	A maths/programming variable	*x*, *w* × *h*	—
`<abbr>`	Abbreviation; `title` gives the expansion	First use of "HTML" — also write the expansion in text, since `title` is not reliably exposed	—
`<dfn>`	The defining instance of a term	Where a term is first defined	Later mentions
`<cite>`	Title of a creative work	Books, films, papers	People's names
`<q>`	Short inline quotation; browser adds quote marks	"She said *ship it*"	Long quotes (`blockquote`)
`<time>`	Date/time with a machine-readable `datetime`	`<time datetime="2026-10-03T19:00">Saturday at 7 pm</time>`	—
`<data>`	Content with a machine-readable `value`	Product IDs, numbers	Dates (use `time`)
`<u>`	Unarticulated, non-textual annotation	Spelling-error squiggles, Chinese proper-name marks	Normal text (looks like a link)
`<bdi>`	Isolates text of unknown direction	User names that may be Arabic/Hebrew in an LTR page	—
`<wbr>`	Where a long word *may* break	Long URLs, compound words	—
`<span>`	Nothing — styling/scripting hook	When nothing above fits	—

Why it matters

`<time datetime>` lets calendars and search engines read dates; `<code>` stays monospace in reader modes; `<abbr>`, `<dfn>` and `lang` help translation and screen readers. Most screen readers **do not** announce `<strong>`/`<em>` by default, so state critical information in words too ("Warning: …").

Common mistakes

- `<i class="icon-star">` for icons — a legacy icon-font habit; use `<span aria-hidden="true">` (hidden from screen readers) or an SVG.
- Whole paragraphs in `<strong>`: if everything is important, nothing is.
- `<cite>` around a person's name.

2.7 Lists

What it is

Element	Use when	Items
`<ul>`	Order doesn't matter (shopping list, nav links, features)	`<li>`
`<ol>`	Order matters (steps, rankings, legal clauses)	`<li>`
`<dl>`	Name–value groups (glossary, metadata, FAQ)	`<dt>` term(s) + `<dd>` description(s); groups may be wrapped in a `<div>`
`<menu>`	A toolbar of commands	Semantically the same as `<ul>`

`<ol>` attribute	Effect
`start="4"`	First number is 4
`reversed`	Counts down
`type="1	a
`<li value="10">`	Sets this item's number; later items continue from it

```
<!-- code/lists.html (excerpt) -->
<ul>
  <li>Milk</li>
  <li>Bread
    <ul><li>Sourdough</li><li>Rye</li></ul>   <!-- nested list goes INSIDE the <li> -->
  </li>
  <li>Apples</li>
</ul>
<ol start="4" type="a"> <li>fourth item is "d"</li> <li>then "e"</li> </ol>
<ol reversed> <li>Bronze</li> <li>Silver</li> <li>Gold</li> </ol>
<ol type="I"> <li>Introduction</li> <li value="10">Jumps to X</li> <li>Then XI</li> </ol>
<dl>
  <dt>HTML</dt><dd>Structure and meaning</dd>
  <dt>CSS</dt><dd>Presentation</dd>
  <dt>JS</dt><dt>JavaScript</dt><dd>Behaviour (two terms, one description)</dd>
</dl>
```

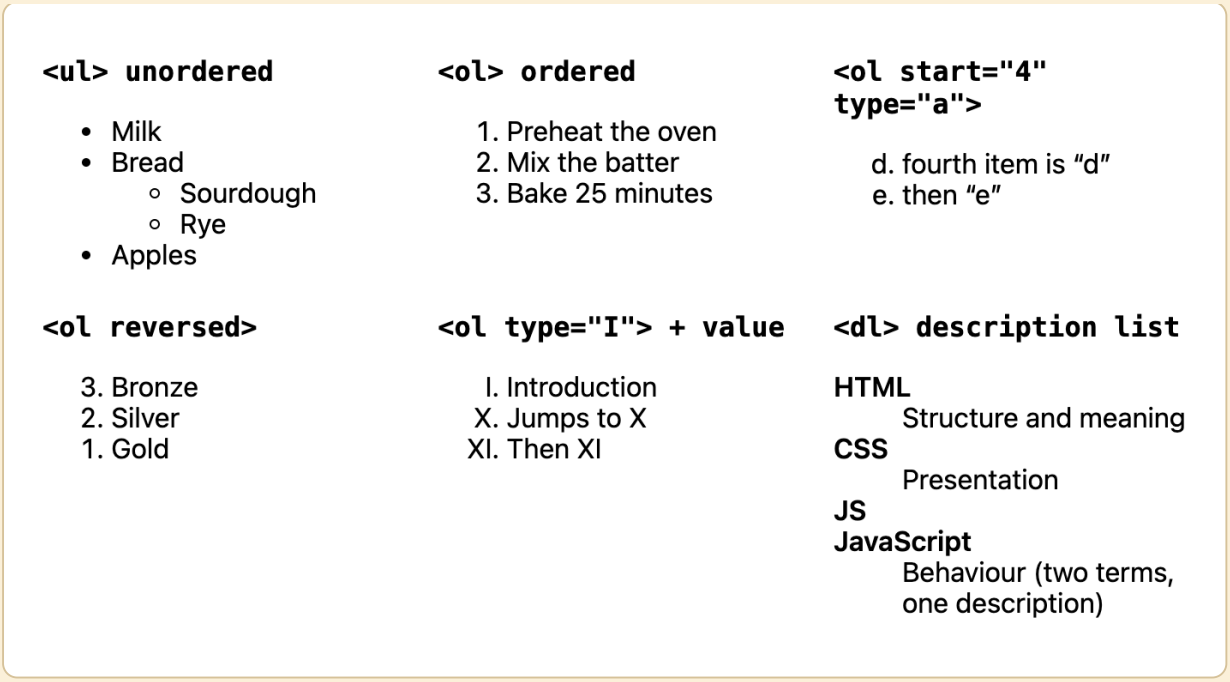


Figure 2.4 — `code/lists.html` in Chrome.

Why it matters

Screen readers announce "list, 3 items" and let users jump between lists and items. Navigation menus are usually `<nav><ul>…</ul></nav>`.

Common mistakes

- Anything other than `<li>` directly inside `<ul>`/`<ol>` (a `<p>`, `<div>`) — invalid; list semantics break.
- A sub-list as a sibling of `<li>` instead of inside one.
- Fake lists with `<br>` and "•".
- `list-style: none` — then Safari/VoiceOver (Apple's screen reader) stops announcing a list; add `role="list"` (ARIA, Chapter 3) if the count matters.

2.8 Links

What it is

`<a href="…">` — the *anchor* — is the "hyper" in hypertext. Its content (text, image, a whole card) becomes clickable and keyboard-focusable; activating it navigates to the `href`.

```
<!-- code/links.html (excerpt) -->
<a href="https://developer.mozilla.org/en-US/docs/Web/HTML">MDN HTML docs</a>        <!-- absolute -->
<a href="text-blocks.html">Headings and text</a>                                     <!-- document-relative -->
<a href="../index.html">All guides</a>                                               <!-- up one folder -->
<a href="#faq">Jump to the FAQ</a>                                                   <!-- fragment: element with id="faq"
-->
<a href="mailto:help@example.org?subject=Question">Email us</a>
<a href="tel:+15555550123">Call +1 555 555 0123</a>
<a href="skeleton.html" download="page-template.html">Download the template</a>
<a href="https://example.com/" target="_blank" rel="noopener noreferrer">Example.com (opens in new tab)</a>
<a href="https://example.com/sponsor" rel="sponsored">A paid link</a>
<a>Placeholder link (no href)</a>
...
<h2 id="faq">FAQ</h2>
```

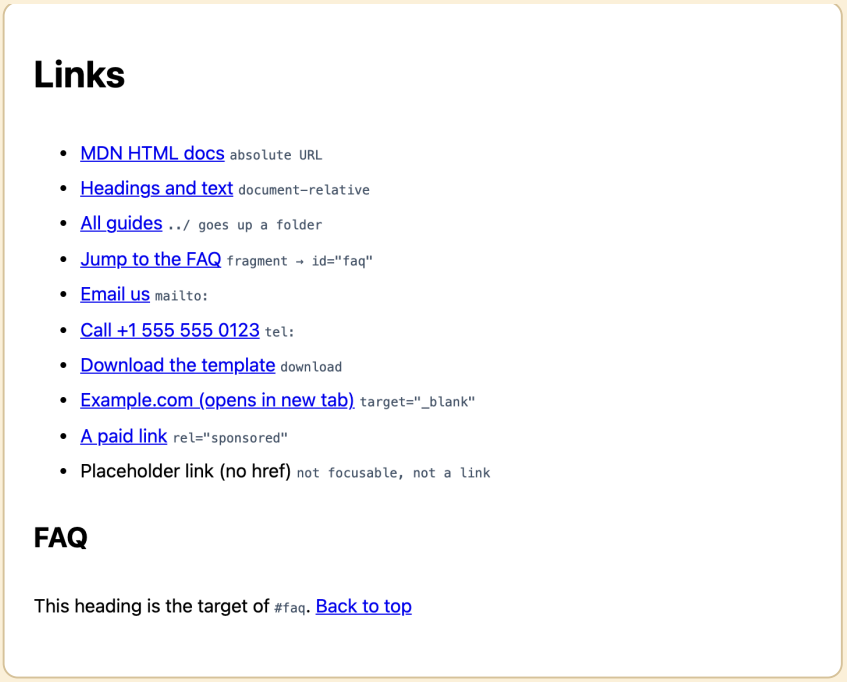


Figure 2.5 — `code/links.html`. The `<a>` without `href` is not styled or focusable as a link.

`code/02_links.mjs` loads the file as if it lived at `https://club.example.org/guides/html/links.html` and prints how each `href` resolves:

```
attribute vs property: text-blocks.html -> https://club.example.org/guides/html/text-blocks.html

MDN HTML docs              https://developer.mozilla.org/en-US/docs/Web/HTML       https:
Headings and text          https://club.example.org/guides/html/text-blocks.html   https:
All guides                 https://club.example.org/guides/index.html              https:
Jump to the FAQ            https://club.example.org/guides/html/links.html#faq     https: same-page jump to #faq
Email us                   mailto:help@example.org?subject=Question                mailto:
Call +1 555 555 0123       tel:+15555550123                                        tel:
Download the template      https://club.example.org/guides/html/skeleton.html      https: download=page-template.html
Example.com (opens in new  https://example.com/                                    https: target=_blank rel=noopener
noreferrer
A paid link                https://example.com/sponsor                             https: rel=sponsored
Placeholder link (no href) (no href: not in document.links = false)
Back to top                https://club.example.org/guides/html/links.html#top     https: same-page jump to #top
```

The `href` *attribute* holds what you typed; the `.href` *property* is the resolved URL (§1.2 rules). `href="#faq"` scrolls to `id="faq"` with no request.

Link attributes

Attribute	What it does	Notes
`href`	Destination URL	Without it `<a>` is a placeholder: not a link, not focusable
`target="_blank"`	Open in a new tab/window	Others: `_self` (default), `_parent`, `_top`, a named window
`rel`	Relationship of the target to this page	Space-separated, see below
`download`	Download instead of navigate; value = suggested file name	Only same-origin, `blob:` (in-memory file made by JS) or `data:` URLs; ignored cross-origin (navigates normally)
`hreflang`, `type`	Hints about the target's language / MIME type (format label such as `text/html`)	Advisory
`referrerpolicy`	How much of this URL to send in `Referer`	e.g. `no-referrer`
`ping`	URLs to notify when followed	Tracking; not in every browser

rel value	Meaning
`noopener`	The new page gets no `window.opener` (a JS handle back to your page), so it cannot redirect your tab to a phishing page ("reverse tabnabbing"). Modern browsers apply it automatically to `target="_blank"`; writing it is harmless.
`noreferrer`	Also don't send `Referer` (implies `noopener`)
`nofollow`	"I don't vouch for this link" — a hint to search engines
`sponsored`	Paid or affiliate link (search engines ask you to mark these)
`ugc`	User-generated content link (comments, forums)
`external`, `author`, `license`, `next`, `prev`	Descriptive relations

Opening new tabs — should you?

Mostly no: forcing one breaks Back and can disorient screen-reader and mobile users, who can open tabs themselves. Exceptions: links that would interrupt a form or playing video, help pages beside a task. Then say so ("(opens in new tab)" or an icon with a text alternative).

Links vs buttons

A **link** goes somewhere (a URL; Enter activates). A **button** does something on this page (submit, toggle, open a menu; Enter or Space). `<a href="#" onclick="…">` fakes a button: it adds `#` to the URL, jumps to the top without JS, and is announced as "link". Use `<button type="button">` (Chapter 3).

Writing good link text

Link text must make sense out of context — screen-reader users often list all links, where "Click here", "read more" and bare URLs mean nothing. Write "Download the 2026 price list (PDF, 2 MB)". This is WCAG 2.4.4 (Link Purpose (In Context)).

Real-world examples

- "Skip to main content" links are fragment links (`href="#main"`) — Chapter 3.
- GitHub and MDN give every heading an `id` for deep links; shops use `tel:` for one-tap calls.

Common mistakes

- Icon-only links with no text — no accessible name (the label a screen reader announces).
- `href="www.example.com"` without `https://` — a *relative* path (`/current/www.example.com`).
- Two adjacent links to the same place (card image + title) — noisy for keyboard and screen-reader users; wrap both in one link.

In the interview

> **30-second answer:** "An anchor with an `href` navigates to a URL — absolute, relative, fragment or another scheme like `mailto:`. I keep link text meaningful out of context, never use `href="#"` for actions because those are buttons, and use `target="_blank"` sparingly. `rel` describes the relationship: `noopener`/`noreferrer` for security and privacy, `nofollow`/`sponsored`/`ugc` for search engines."

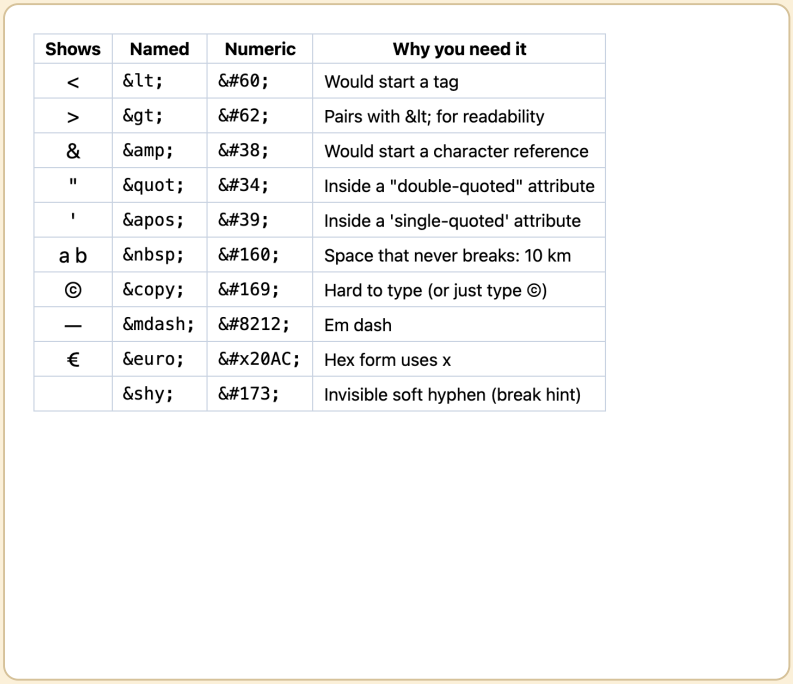
- *Why was `rel="noopener"` needed with `target="_blank"`?* → The opened page could set `window.opener.location` to swap your tab for a phishing page. Browsers now imply `noopener` for `_blank`.
- *Link or button for "Add to cart"?* → Button: an action on the current page.
- *`download` on a cross-origin URL?* → Ignored; the browser navigates normally.

2.9 Character references and comments

What it is

`<` starts a tag and `&` starts a **character reference**, so to show them as text you write a reference:

- **Named**: `<` `>` `&` `"` ` ` `©` (often called "entities"; strictly *named character references*)
- **Decimal**: `<` — the Unicode *code point* (each character's number in the Unicode catalogue)
- **Hexadecimal**: `<`



Shows	Named	Numeric	Why you need it
<	<	<	Would start a tag
>	>	>	Pairs with < for readability
&	&	&	Would start a character reference
"	"	"	Inside a "double-quoted" attribute
'	'	'	Inside a 'single-quoted' attribute
a b			Space that never breaks: 10 km
©	©	©	Hard to type (or just type ©)
—	—	—	Em dash
€	€	€	Hex form uses x
			Invisible soft hyphen (break hint)

Figure 2.6 — `code/entities.html`: what each reference renders as.

With `<meta charset="utf-8">` you can type almost any character directly (©, —, €, é, emoji). You only *need* references for:

Character	Where	Reference
`<`	In text	`<` (so the text ` ` is ` `, or set via `textContent`)
`&`	In text or attributes, when followed by something that looks like a reference	`&` (always escaping is simplest)
`"` / `'`	Inside an attribute quoted with the same character	`"` / `'`
Invisible characters	Non-breaking space, soft hyphen, zero-width characters	` `, ``, `` — visible in your source

```
// code/03_entities.mjs
import '../../../../tools_dom.mjs';
const p = document.createElement('p');
p.innerHTML = 'Fish &amp; chips &lt;b&gt; &copy; &#169; &#xA9; 10 km &mdash; café';
console.log('textContent :', JSON.stringify(p.textContent));
console.log('nbsp code   :', p.textContent.codePointAt(p.textContent.indexOf('km') - 1).toString(16));
console.log('innerHTML   :', p.innerHTML);
const q = document.createElement('p');
q.textContent = '<img src=x onerror=alert(1)> & "quotes"';     // text, not HTML
console.log('\nescaped     :', q.innerHTML);
console.log('elements    :', q.children.length, '(the tag became text, not an element)');
const r = document.createElement('p');
r.innerHTML = 'AT&T, R&D, a&b=1';
console.log('\nbare &      :', r.textContent);
const s = document.createElement('div');
s.innerHTML = '<p>price&copy=1</p><a href="?a=1&copy=2">x</a>';
console.log('in text     :', s.querySelector('p').textContent, '| in href:', s.querySelector('a').getAttribute('href'));
```

Output:

```
textContent : "Fish & chips <b> © © © 10 km — café"
nbsp code   : a0
innerHTML   : Fish &amp; chips &lt;b&gt; © © © 10 km — café

escaped     : &lt;img src=x onerror=alert(1)&gt; &amp; "quotes"
elements    : 0 (the tag became text, not an element)

bare &      : AT&T, R&D, a&b=1
in text     : price©=1 | in href: ?a=1&copy=2
```

1. The parser **decodes** references; `©`, `©` and `©` are the same `©`.
2. ` ` is the real character U+00A0 (`a0`), not a space — keeps "10 km" or "Mr. Smith" on one line.
3. When **serialising** (DOM → HTML text, e.g. reading `innerHTML`) the browser re-escapes `&`, `<`, `>` and non-breaking spaces.
4. Setting `textContent` is safe: markup becomes text. Assigning user input to `innerHTML` is how **cross-site scripting (XSS)** happens — an attacker's script runs in your page (JavaScript book).
5. A bare `&` usually survives but is fragile: ~100 legacy names such as `&copy` decode even *without* a semicolon, so `price&copy=1` shows `price©=1`. In attributes a special rule leaves `&copy=` alone, which is why most query strings survive. `&` removes all doubt.

Comments

```
<!-- This is a comment. It can span
     several lines. It is not displayed. -->
```

Comments stay in the DOM as comment nodes (`01_head.mjs` found one) and are **visible to anyone** via View Source — never put passwords, internal URLs or rude notes in them. They cannot nest; avoid `--` inside. Build tools usually strip them from production HTML.

2.10 Validation

The problem it solves

Browsers never show HTML errors — they silently repair them (Chapter 1), because the spec requires recovery. A broken page can *look* fine while behaving differently for screen readers, search engines, other browsers, or your CSS and JS.

A page with a dozen problems (`code/broken.html`):

```
<html>
<head>
<title>Broken page</title>
</head>
<body>
<h1>Welcome</h1>
<h3>Our services</h3>
<p>Great deals <b>today <i>only</b></i>
<p id="promo">Sale!</p>
<p id="promo">Another sale!</p>
<img src="data:image/svg+xml,...">
<a href="#" onclick="go()">Click here</a>
<ul><p>Not a list item</p><li>ok</li></ul>
<h1>Also welcome</h1>
</body>
</html>
```

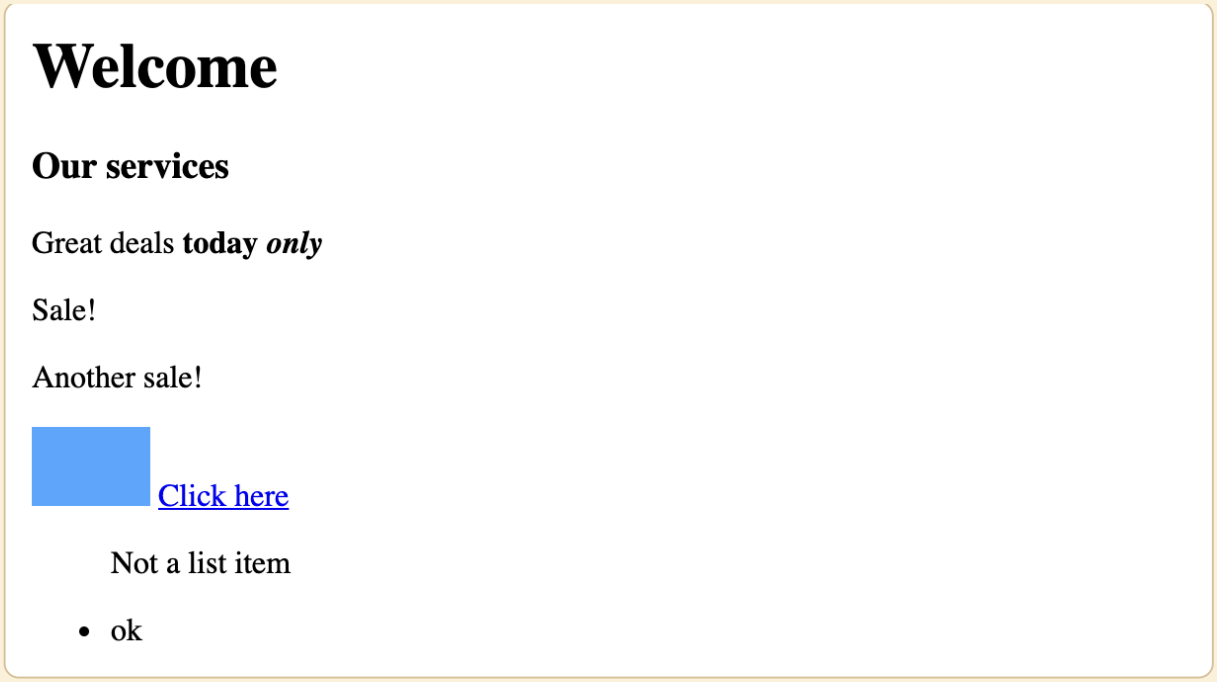


Figure 2.7 — It renders without complaint. Nothing on screen reveals the 12 problems below.

How it works — checking

1. **The W3C Nu HTML Checker** (validator.w3.org/nu) — the reference validator (paste, upload or URL). Reports **parse errors** (bad nesting, stray end tags) and **content-model errors** (`<p>` in `<ul>`, duplicate ids, missing required attributes such as `alt`).
2. **Editor and build tools**: VS Code extensions, `html-validate` or `vnu` (the Nu checker's CLI) in CI (continuous integration — automated checks on every push).
3. **Accessibility checkers** (axe, Lighthouse, WAVE; Chapter 3) catch what validators don't: empty links, poor contrast.

`code/04_validate.mjs` is a tiny jsdom checker for a dozen common problems (doctype, `lang`, charset within 1024 bytes, viewport, title, `h1` count, heading jumps, duplicate ids, `img` without `alt`, `href="#"`/`javascript:` links, vague link text, non-`li` list children, `<b><i></b>` mis-nesting):

```
skeleton.html: OK
broken.html: 12 problem(s)
  - missing <!DOCTYPE html> -> quirks mode
  - <html> has no lang attribute
  - no <meta charset> in the first 1024 bytes
  - no viewport meta (page will not scale on phones)
  - 2 <h1> elements (aim for exactly one)
  - heading jumps from h1 to h3: "Our services"
  - duplicate id="promo"
  - <img> without alt
  - fake link "Click here" (use <button>)
  - vague link text "Click here"
  - <p> directly inside <ul>
  - mis-nested tags <b><i>...</b> (parser repaired it)
```

Some are validity errors (duplicate id, `<p>` in `<ul>`, missing `alt`, missing doctype/title); others are best-practice or accessibility warnings (heading jumps, two `h1`s, vague link text). A validator plus an accessibility checker covers both.

Trade-offs

Validity isn't the goal — a valid page can still be inaccessible. But valid HTML is predictable across browsers and tools, and most errors are real bugs. Treat errors as bugs, warnings as review items.

In the interview

30-second answer: "The HTML parser repairs almost any markup, so errors are invisible. That's why I validate — the repaired DOM may not be what I meant, and assistive technology (screen readers, switch

devices), crawlers and my CSS selectors all see the repaired tree. I use the Nu HTML Checker or html-validate in CI, plus axe or Lighthouse for accessibility."

- *Name three common validation errors.* → Duplicate ids, block elements inside `<p>`, missing `alt` on `<img>` (also bad nesting, non-`li` children in lists).
- *Does invalid HTML affect SEO (search engine optimisation)?* → Not directly as a ranking factor, but broken structure can hide content, headings or links from crawlers.
- *Does a valid page guarantee accessibility?* → No — add accessibility testing.

Chapter 2 TL;DR

- **Elements** = tags + content; attributes add information; void elements have no end tag; boolean attributes are on when present. Nest properly.
- Every page: `<!DOCTYPE html>` → `<html lang>` → `<head>` (charset first, viewport, unique `<title>`, description, CSS, deferred JS, icon, canonical, social tags) → `<body>`.
- **Headings** build the outline: one `h1`, don't skip levels, choose by structure not size. There is no outline algorithm for `<section>`.
- **Text blocks**: `p` for paragraphs, `br` only for meaningful breaks, `pre` keeps whitespace, `blockquote` for long quotes; whitespace otherwise collapses.
- **Inline semantics**: `strong` (importance) vs `b` (attention), `em` (stress) vs `i` (other voice), plus `mark`, `code`, `kbd`, `time`, `abbr`, `cite`, `q`… `span`/`div` have no meaning.
- **Lists**: `ul` unordered, `ol` ordered (`start`, `reversed`, `type`, `value`), `dl` for name–value pairs; nested lists go inside `li`.
- **Links**: resolve like URLs; meaningful text; `href="#"` actions should be buttons; `target="_blank"` sparingly (implies `noopener`); `rel` for security, privacy and SEO hints.
- **References**: escape `<` and `&` (`<`, `&`), ` ` for non-breaking space; `textContent` is safe, raw `innerHTML` is an XSS risk. Comments are public.
- **Validate**: browsers silently repair errors; use the Nu checker/html-validate plus accessibility tools.

Next: Chapter 3 — Semantic HTML and Accessibility.

Chapter 3 — Semantic HTML and Accessibility

A page can look perfect and still be unusable for someone who navigates by keyboard, listens through a **screen reader** (software that reads the page aloud and lets blind users move around it with keys), zooms to 400 %, or can't tell red from green. **Accessibility** (*a11y*: "a", 11 letters, "y") means building pages that work for all of them, and the most effective tool is **semantic HTML**: using the element whose meaning matches the content. This chapter covers the *accessibility tree*, landmarks, headings, alt text, ARIA, focus management, form labels, WCAG 2.2 and testing.

In this chapter: 3.1 Why semantics and accessibility · 3.2 Landmarks · 3.3 Headings and outline · 3.4 The accessibility tree · 3.5 Alt text · 3.6 ARIA · 3.7 Keyboard and focus · 3.8 Accessible forms · 3.9 WCAG 2.2 · 3.10 Testing tools.

Code is in `code/`. Most scripts open the `.html` files in **real headless Chrome** via `puppeteer-core` (a Node.js library that remote-controls Chrome; helper `code/a11y_lib.mjs`) and print Chrome's own accessibility tree — the data a screen reader receives. `06_mini_audit.mjs` uses jsdom and `dom-accessibility-api`.

3.1 Why semantics and accessibility

What it is

Semantic HTML = choosing elements for what content *is* (`<button>` for a button, `<nav>` for navigation, `<h2>` for a section heading), not how it looks. **Accessibility** = people with disabilities can perceive, understand, navigate and interact with the page. They are linked because browsers expose your HTML's semantics to **assistive technologies** (AT): screen readers, screen magnifiers, voice control, switch devices (one or two big buttons replacing keyboard and mouse), braille displays (raised pins showing text as braille). A `<div>` styled as a button is a painted-on door.

Who benefits

Group	Examples	What they rely on
Blind and low-vision users	Screen readers (JAWS, NVDA, VoiceOver, TalkBack), magnifiers, high zoom	Roles and names, headings, alt text, contrast (text vs background brightness), reflow (content rewrapping at high zoom)
Motor impairments	Keyboard only, switch access, voice control ("click Submit")	Focusable controls, visible focus, big targets, visible labels that match names
Deaf and hard of hearing	Captions (on-screen speech text), transcripts (full text of audio/video)	Media alternatives (Chapter 4)
Cognitive and learning disabilities	Clear language, consistent layout, no time pressure	Predictable structure, clear errors, no memory tests
Everyone, sometimes ("situational")	Broken arm, bright sun, noisy train, slow network, old age	All of the above

The WHO estimates about 1.3 billion people — roughly one in six — live with a significant disability.

Why it matters

- **It is mostly not happening.** The WebAIM Million (top one million home pages, February 2026) found detectable WCAG failures on 95.9 % of them, ~56 errors per page. Six types made up 96 % of errors: low-contrast text (83.9 % of pages — the most common failure), missing alt text, missing form labels, empty links, empty buttons, missing page language. All are fixed by this chapter's basics; §3.10 builds a checker for exactly those six.
- **It is often the law.** The US ADA (Americans with Disabilities Act) and Section 508 (US government sites), the European Accessibility Act (applying since 28 June 2025) and similar laws require accessible services, usually by reference to WCAG level AA.
- **It is better engineering.** Semantic HTML gives keyboard support, focus handling and screen-reader output *for free*, and improves SEO, reader modes, voice assistants, testing (Testing Library finds elements by

role — React book) and readability.

3.2 Landmarks

What it is

Landmarks are a page's big regions — banner, navigation, main, sidebar, footer. Screen-reader users list them and jump straight to one (NVDA's `D` key; VoiceOver's **rotor**, a menu of headings, links or landmarks). HTML's **sectioning elements** create landmarks automatically; the **role** is what a screen reader announces:

Element	Landmark role (ARIA)	Rules
`<header>`	`banner` — only when *not* inside `article`, `aside`, `main`, `nav` or `section`	Site header: logo, site nav, search. Inside an article it's just that article's header.
`<nav>`	`navigation`	Major navigation blocks only. With several, label each: `aria-label="Main"`, `"Breadcrumb"` (the Home › Shop › Boots trail showing where a page sits)
`<search>`	`search`	Wraps a search/filter form. Baseline 2023, widely available since April 2026 (before: `<form role="search">`).
`<main>`	`main`	The primary content, **one visible per page**; target of the skip link
`<aside>`	`complementary`	Related but separable: sidebar, related links, pull quote
`<footer>`	`contentinfo` — only when not inside `article`, `aside`, `main`, `nav` or `section`	Site footer: copyright, contact, legal links
`<section>`	`region` — **only if it has an accessible name** (the label a screen reader reads, e.g. via `aria-labelledby` pointing at its heading)	A thematic group with its own heading
`<form>`	`form` — only if it has an accessible name	
`<article>`	`article` (a document-structure role, not a landmark)	Self-contained: blog post, product card, comment, news story

article could be syndicated (republished in a feed) or reused alone; **section** is a chunk of something bigger with its own heading (a chapter, tab panel, "Reviews"); **div** has no meaning. Unsure between `section` and `div`? Ask "would this appear in a table of contents?" If not, `div`.

Example

`code/landmarks.html`, a small news page (the yellow tags in the screenshot are `aria-hidden="true"` labels for this book):

```
<a class="skip" href="#main">Skip to main content</a>
<header>
  <a class="logo" href="/">Mountain Club</a>
  <nav aria-label="Main">
    <ul>
      <li><a href="/">Home</a></li>
      <li><a href="/news" aria-current="page">News</a></li>
      <li><a href="/events">Events</a></li>
    </ul>
  </nav>
  <search>
    <form action="/search"><label for="q">Search</label> <input id="q" name="q" type="search"></form>
  </search>
</header>
<div class="wrap">
  <main id="main">
    <h1>Trail news</h1>
    <article>
      <header>
        <h2>Ridge trail reopens</h2>
        <p>By Sam Lee · <time datetime="2026-09-20">20 September 2026</time></p>
      </header>
      <p>After three months of repairs, the ridge trail is open again.</p>
      <section aria-labelledby="h-what">
        <h3 id="h-what">What changed</h3>
        <p>New steps on the steep part and two rest benches.</p>
      </section>
      <footer><p>Filed under <a href="/tags/trails">trails</a></p></footer>
    </article>
  </main>
  <aside aria-labelledby="h-side">
    <h2 id="h-side">Upcoming</h2>
    <ul><li>Night run, Fri</li><li>Hill repeats, Sun</li></ul>
  </aside>
</div>
<footer><small>© 2026 Mountain Club</small></footer>
```

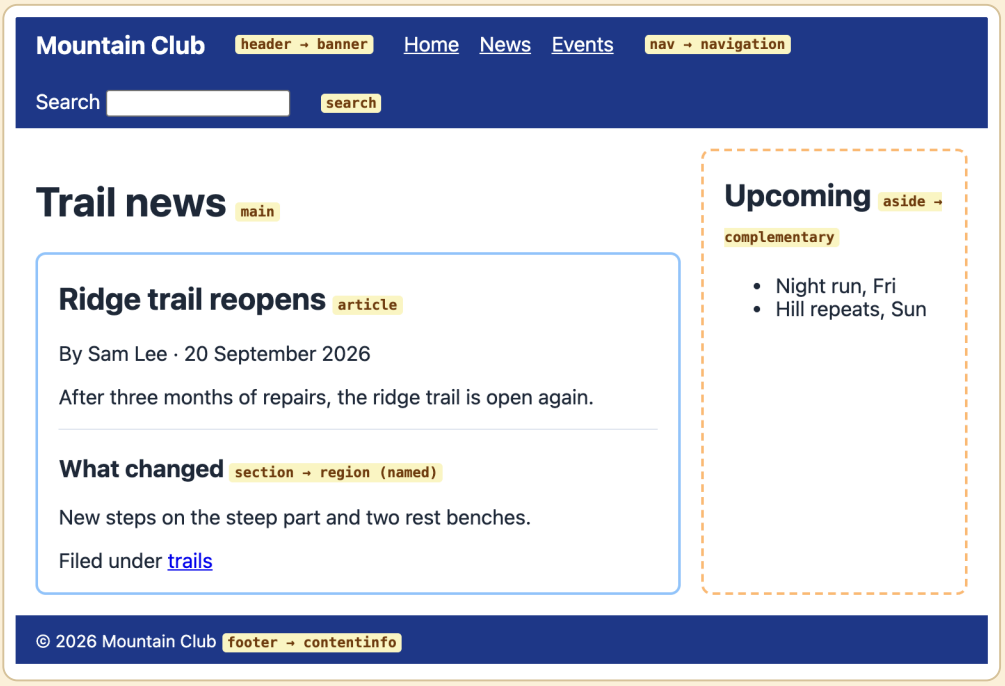


Figure 3.1 — `code/landmarks.html`. The layout is CSS; the landmarks come from the elements.

Chrome's accessibility tree and heading outline (`code/01_landmarks_tree.mjs`, text and generic nodes skipped):

```
RootWebArea "Trail news — Mountain Club"
  link "Skip to main content"
  banner
    link "Mountain Club"
    navigation "Main"
      list
        link "Home"
        link "News"
        link "Events"
    search
      form
        searchbox "Search"
  main
    heading "Trail news"  level=1
    article
      sectionheader
        heading "Ridge trail reopens"  level=2
      region "What changed"
        heading "What changed"  level=3
      sectionfooter
        link "trails"
  complementary "Upcoming"
    heading "Upcoming"  level=2
    list
  contentinfo

Heading outline:
h1 Trail news
  h2 Ridge trail reopens
    h3 What changed
  h2 Upcoming
```

1. Top-level `<header>`/`<footer>` became `banner`/`contentinfo`; the article's own header and footer did *not* (`sectionheader`/`sectionfooter` aren't landmarks).
2. `<section>` became a `region` because `aria-labelledby` named it; `<aside>` got "Upcoming" the same way.
3. The `<nav>` is "Main navigation" thanks to `aria-label`; `aria-current="page"` marks the current link.
4. Everything except the skip link sits inside a landmark — aim for that, so nothing is orphaned outside the regions users jump between.

Common mistakes

- "Div soup": `<div class="header">`, `<div class="nav">` — zero landmarks.
- Several `<main>` elements, or none.
- `<section>` everywhere "because HTML5" — sections without headings add noise.
- Several unlabelled `<nav>`s: "navigation, navigation, navigation".

GOV.UK, the BBC and GitHub use this exact skeleton (skip link → `header` with `nav` → `main` → `footer`).

3.3 Headings and the outline

Headings are the other main navigation tool: users press `H` (NVDA/JAWS) or use the rotor to jump heading to heading, or list all headings as a table of contents (the outline above). The rules of §2.4 apply: one `h1`, no skipped levels going down, levels by structure, and `<section>`/`<article>` don't reset levels (the "outline algorithm" was never implemented and was removed).

Accessibility-specific points:

- Each landmark usually starts with a heading — hence naming a `section` or `aside` with `aria-labelledby` pointing at it.
- A region with no visible heading (the main nav) gets `aria-label`, not a fake heading.

3.4 The accessibility tree

What it is

Alongside the DOM, browsers build an **accessibility tree** — only what matters to AT — exposed through the OS accessibility API (UI Automation on Windows, NSAccessibility on macOS, AT-SPI on Linux). Screen readers never read your HTML; they read this tree.

Property	Question it answers	From
Role	What is it? (button, link, heading, textbox, checkbox, navigation…)	The element's implicit role (`<button>` → button) or an ARIA `role`
Name	What is it called? ("Save", "Search", "Close")	Content, `<label>`, `alt`, `aria-label`, `aria-labelledby`…
Description	Extra help text	`aria-describedby`, sometimes `title`
State / properties	What condition? (checked, expanded, disabled, invalid, required, level 2, current page)	Native attributes (`checked`, `disabled`, `required`) or ARIA states
Value	Current value of a field, slider, progress bar	The control's value

A screen reader speaks it as: *"Email, edit text, required, invalid entry, We only use it for your receipt."*

How the accessible name is computed

The W3C *Accessible Name and Description Computation* spec; simplified, the first that yields text wins:

1. `aria-labelledby` — text of the referenced element(s), in order (several ids allowed).
2. `aria-label` — a string on the element.
3. The native label: `<label>` for fields, `alt` for images, `<caption>` for tables, `<legend>` for fieldsets, `<figcaption>` for figures.
4. The element's content, for roles with "name from content" (buttons, links, headings, cells…).
5. `title` (and for inputs `placeholder`) as a last resort.

Hidden content (`display: none`, `hidden`, `aria-hidden="true"`) is left out — unless referenced directly by `aria-labelledby`.

Chrome's computation for `code/aria.html` (§3.6), from `code/03_names_and_states.mjs` (`StaticText` nodes omitted; the visually hidden "Rated 4 out of 5" text is one of them):

```
--- aria.html
RootWebArea "ARIA patterns"
  button "Close"
  button "Shopping cart 3 items"
  textbox "Password"  description="At least 12 characters."
  button "Shipping details"  expanded=true
  navigation "Breadcrumb"
    link "Home"
    link "Shop"
    link "Boots"
  button "Add to cart"
  status

after clicking "Shipping details" and "Add to cart": {
  expanded: 'false',
  panelHidden: true,
  status: 'Added. Cart has 4 items.'
}
```

- The "×" button is named **"Close"** (`aria-label` beat the content).
- The cart button combines two elements via `aria-labelledby`: **"Shopping cart 3 items"**.
- The password field's name is its `<label>`; its **description** comes from `aria-describedby`.
- The `aria-hidden` stars are gone; only the visually hidden text remains.

Inspecting it yourself

Chrome DevTools → Elements → **Accessibility** pane (or the full-page tree view); Firefox → **Accessibility** panel; Safari → Elements → Node → Accessibility.

In the interview

30-second answer: "Browsers derive an accessibility tree from the DOM — each node has a role, an accessible name, maybe a description, states and a value — and expose it to screen readers through OS accessibility APIs. Native elements fill it in correctly; ARIA can override or add. The name comes, in order, from `aria-labelledby`, `aria-label`, native labels like `<label>` or `alt`, the content, and finally `title` or placeholder."

- *Does CSS affect the accessibility tree?* → Yes: `display: none` and `visibility: hidden` remove nodes; generated content (`::before`/`::after`, Chapter 6) can become part of names; `opacity: 0` hides nothing.
- *Is a `<div>` with a click handler a button to a screen reader?* → No: no role, no button name, not focusable (§3.7).

3.5 Alt text

What it is

`alt` is an image's **text alternative**: what a screen reader speaks, what search engines index, what shows if the image fails. It is WCAG success criterion 1.1.1 (Non-text Content, level A) and a top-six failure.

How to decide — ask "what is this image *for*?"

Ask in order	If yes →
1. Purely decorative, or nearby text already says it?	`alt=""` (empty — image ignored)
2. Content of a link or button?	Describe the **destination or action** ("Mountain Club home", "Search")
3. Complex (chart, diagram, map)?	Short `alt` naming it + the full information in nearby text or `<figcaption>`
4. An image of text?	`alt` repeats the text — better, use real text styled with CSS
5. Otherwise (a meaningful photo/illustration)	What matters about it **in this context**, briefly

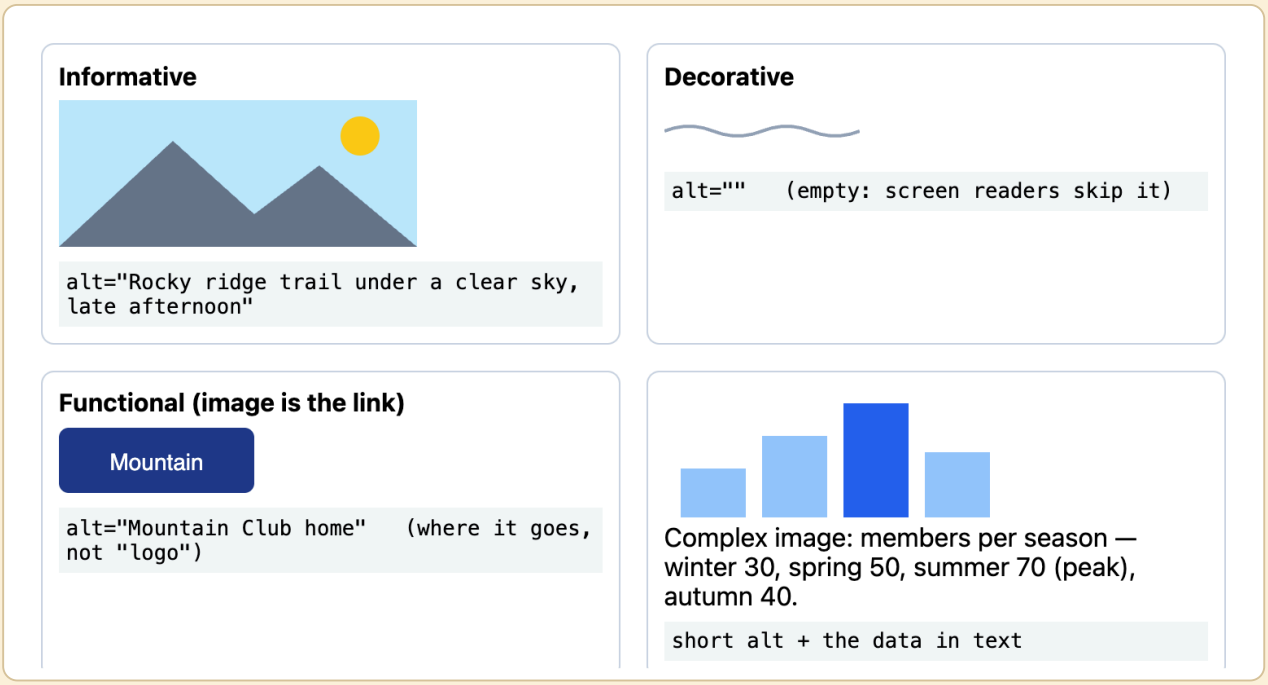


Figure 3.2 — `code/alt-text.html`: the four most common cases.

```
<img src="ridge.jpg" alt="Rocky ridge trail under a clear sky, late afternoon">       <!-- informative -->
<img src="squiggle.svg" alt="">                                                        <!-- decorative -->
<a href="/"><img src="logo.svg" alt="Mountain Club home"></a>                          <!-- functional -->
<figure>                                                                                 <!-- complex -->
  <img src="members.svg" alt="Bar chart of club members per season">
  <figcaption>Complex image: members per season — winter 30, spring 50, summer 70 (peak), autumn 40.</figcaption>
</figure>
```

Chrome's view (from `03_names_and_states.mjs`):

```
--- alt-text.html (images only)
image "Rocky ridge trail under a clear sky, late afternoon"
image "Mountain Club home"  (inside link "Mountain Club home")
image "Bar chart of club members per season"
decorative <img alt=""> in tree? false
```

The empty-alt image is gone from the tree; the linked image names the link.

Rules of thumb

- **Missing `alt` ≠ empty `alt`**: missing → screen readers may read the file name ("IMG underscore 2041 dot jpg"); empty → skipped.
- Don't start with "Image of…" — the role already says "image" (do say "Photo", "Screenshot" when the *type* matters).
- Short; what matters **in context** — the same photo is "Dr. Ana Ruiz" on a staff page, "Ana Ruiz holding the award trophy" in a news story.
- Don't repeat the caption or adjacent text; if the text already says it, `alt=""`.
- Icons beside visible text ("🗑 Delete") are decorative; icon-only buttons need a name on the button (`aria-label`).
- CSS background images have no alt: never put meaningful content in them.
- Inline `<svg>`: `role="img"` + `aria-label` (or a `<title>` child) when informative, `aria-hidden="true"` when decorative (Chapter 4).

3.6 ARIA

What it is

WAI-ARIA (Web Accessibility Initiative — Accessible Rich Internet Applications; "ARIA") is a set of attributes that change or add to what the accessibility tree says:

- **Roles** — `role="button"`, `"tablist"`, `"dialog"`, `"alert"`…
- **Properties** (mostly stable facts) — `aria-label`, `aria-labelledby`, `aria-describedby`, `aria-controls`, `aria-required`, `aria-haspopup`…
- **States** (change with interaction) — `aria-expanded`, `aria-checked`, `aria-pressed`, `aria-selected`, `aria-invalid`, `aria-disabled`, `aria-current`, `aria-hidden`…

The crucial fact: **ARIA only changes what is announced. It adds no behaviour** — no focusability, keyboard handling or styling. `role="button"` on a `<div>` makes a screen reader *say* "button"; you still must make it focusable and handle Enter and Space.

The first rule of ARIA (and the other four)

From the W3C "Using ARIA" note:

1. **Use native HTML first.** If an element/attribute with the needed meaning and behaviour exists (`<button>`, `<a href>`, `<input type="checkbox">`, `<details>`, `<dialog>`, `required`, `disabled`), use it.
2. **Don't change native semantics** unless you must (`<h2 role="tab">` is wrong; put a `<button role="tab">` inside the `<h2>`).
3. **Every interactive ARIA control (*widget* — a custom component such as tabs or a menu) must work with the keyboard.**
4. **Never put `role="presentation"`/`"none"` or `aria-hidden="true"` on a focusable element** — users would tab to something that "doesn't exist".
5. **Every interactive element must have an accessible name.**

No ARIA is better than bad ARIA: WebAIM's surveys consistently find home pages *with* ARIA have more detected errors on average than pages without — usually ARIA patching div-based widgets instead of native elements.

Useful ARIA — the patterns in `aria.html`

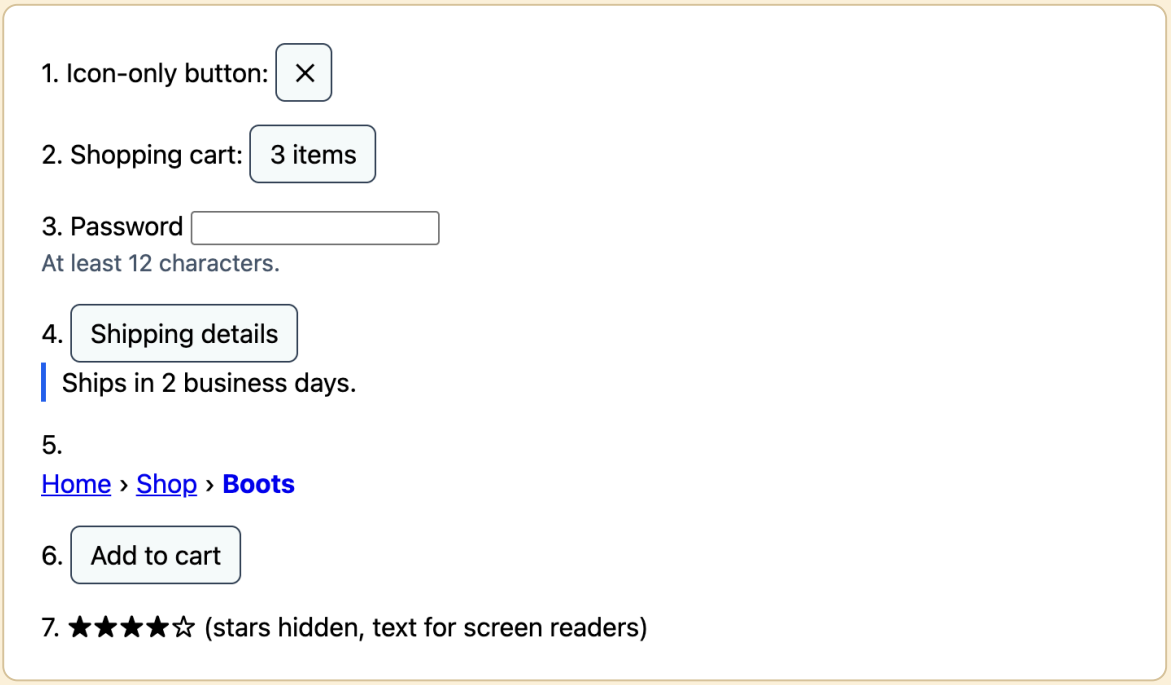


Figure 3.3 — `code/aria.html`. Visually ordinary — the differences are in the accessibility tree (§3.4).

```html
<!-- 1. Name an icon-only control -->
<button type="button" aria-label="Close">×</button>

<!-- 2. Build a name from visible text elsewhere (ids, in order) -->
<span id="cart-h">Shopping cart</span>:
<button type="button" aria-labelledby="cart-h cart-n"><span id="cart-n">3 items</span></button>

<!-- 3. Add help text as a description -->
<label for="pw">Password</label> <input id="pw" type="password" aria-describedby="pw-hint">
<div id="pw-hint">At least 12 characters.</div>

<!-- 4. Disclosure: state that JavaScript keeps in sync -->
<button type="button" aria-expanded="true" aria-controls="panel" id="toggle">Shipping details</button>
<div id="panel">Ships in 2 business days.</div>

<!-- 5. Mark the current item in a set of links -->
<nav aria-label="Breadcrumb"> … <a href="/shop/boots" aria-current="page">Boots</a></nav>

<!-- 6. Live region: announce changes without moving focus -->
<button type="button" id="add">Add to cart</button> <span role="status" id="live"></span>

<!-- 7. Hide decoration, provide a text equivalent -->
<span aria-hidden="true">★★★★☆</span><span class="visually-hidden">Rated 4 out of 5</span>
```

```js
// the only JavaScript: keep aria-expanded true to the UI, and write into the live region
t.addEventListener('click', () => { const open = t.getAttribute('aria-expanded') === 'true';
  t.setAttribute('aria-expanded', String(!open)); p.hidden = open; });
document.getElementById('add').addEventListener('click', () => { live.textContent = 'Added. Cart has 4 items.'; });
```

Attribute	Use when	Watch out
`aria-label`	No visible text (icon buttons, several `<nav>`s)	Some translation tools skip it; voice-control users can't see it — prefer visible text
`aria-labelledby`	Visible text elsewhere names it	Takes ids; overrides everything else (wins over `aria-label`)
`aria-describedby`	Hints, format rules, error messages	Read after the name, sometimes after a pause
`aria-expanded`	Buttons that show/hide something (menus, accordions, disclosures)	Your script must update it
`aria-current`	Current page/step/date in a set	`page`, `step`, `location`, `date`, `time`, `true`
`aria-hidden="true"`	Decorative or duplicated content	Removes the whole subtree from the tree; doesn't hide visually or stop focus — never on focusable things
`aria-live` / `role="status"` / `role="alert"`	Content changing without a page load (toasts, counters, errors)	Region must **exist before** the text changes; `status` = polite (waits until idle), `alert` = assertive (interrupts) — urgent errors only
`aria-invalid`, `aria-required`	Errors / required fields when native attributes don't fit	Native `required` already implies `aria-required`

> **Key term — live region:** an element whose text changes are read out automatically while focus stays put (`aria-live`, `role="status"`, `role="alert"`, changed later by JS). It's how a blind user hears "Item added" or "Saved" after an AJAX save (a background request without reload) without being moved around the page.

The **visually-hidden** pattern (`.visually-hidden` / `.sr-only`) hides text on screen but keeps it in the accessibility tree:

```
.visually-hidden { position: absolute; width: 1px; height: 1px; overflow: hidden;
                   clip-path: inset(50%); white-space: nowrap; }
```

When not to use it

- To re-create a native control (`<div role="checkbox">`).
- To "fix" headings (`<div role="heading" aria-level="2">`) when you can edit the HTML.
- `aria-label` on a role-less `<div>`/`<span>`: many screen readers ignore names on generic elements.
- `role="application"` — switches off the screen reader's normal reading keys; almost never appropriate.

In the interview

> **30-second answer:** "ARIA sets roles, states and properties in the accessibility tree when HTML can't express something — `aria-expanded` on a disclosure button, `aria-live` regions, naming an icon button. But the first rule is to use native HTML when it exists, because ARIA only changes semantics: no focus, keyboard or behaviour. A `<button>` gives me all of that free."

- *`aria-label` vs `aria-labelledby`?* → A string on the element vs a pointer to other elements' visible text (wins if both exist).
- *`role="status"` vs `role="alert"`?* → Both live regions; status is polite, alert assertive — urgent errors only.
- *What does `aria-hidden="true"` do?* → Removes the element and descendants from the accessibility tree only.

3.7 Keyboard and focus

What it is

Keyboard-only users, screen-reader users (screen readers are keyboard-driven), and switch and voice users (whose tools emulate keys) never use a mouse. **Focus** is the element receiving keyboard input: **Tab** forward,

Shift+Tab back, **Enter** activates links and buttons, **Space** activates buttons and toggles checkboxes, **arrows** move within composite widgets (radio groups, menus, tabs, selects), **Escape** closes dialogs and popups.

WCAG requires everything to work by keyboard (2.1.1), no keyboard traps (2.1.2 — places Tab can't leave), logical order (2.4.3), and a visible focus indicator (2.4.7) not hidden behind sticky headers (2.4.11, new in 2.2).

The div-button problem, measured

`code/div-vs-button.html` has three identical-looking controls:

```
<div class="btn" onclick="save()">Save (div)</div>
<button class="btn" type="button" onclick="save()">Save (button)</button>
<div class="btn" role="button" tabindex="0" onclick="save()">Save (div + role + tabindex)</div>
<p id="out" role="status"></p>
```

Figure 3.4 — Pixel-identical. The screenshot can't tell you which one works.

`code/02_div_vs_button.mjs` prints Chrome's tree, presses Tab four times, then presses Enter/Space on the `<button>` and Enter on the role=button div:

```
Accessibility tree:
RootWebArea "div vs button"
  StaticText "All three look identical. Only the middle one is a real button."
  StaticText "Save (div)"
  button "Save (button)"
  button "Save (div + role + tabindex)"
  status

Pressing Tab 4 times:
  Tab 1 -> Save (button)
  Tab 2 -> Save (div + role + tabindex)
  Tab 3 -> (body — focus left the page)
  Tab 4 -> Save (button)

button     + Enter: status = "Saved!"

button     + Space: status = "Saved!"

div + role + Enter: status = ""
```

1. The plain `<div>` is **just text** and **never reachable** by Tab.
2. The `<button>` is announced, focusable, and works with **Enter and Space** — free.
3. The `div` with `role` + `tabindex` is announced and focusable, but **Enter does nothing**: `onclick` on a div fires only for pointer clicks (mouse, touch, pen). You'd need a `keydown` handler for Enter *and* Space, disabled-state handling and form-submission behaviour — or just `<button>`.

`tabindex`

Value	Effect	Use
(none)	Natively interactive elements (`a[href]`, `button`, `input`, `select`, `textarea`, `summary`…) are focusable in DOM order	Default — keep DOM order = visual order
`tabindex="0"`	Adds a non-interactive element to the Tab order at its DOM position	Custom widgets (with role and key handlers), scrollable regions
`tabindex="-1"`	Focusable by script (`el.focus()`) or as a link target, not by Tab	Skip-link targets, headings you move focus to, items in roving-focus widgets (one item tabbable, arrows move among the rest), dialog containers
`tabindex="1"`+	Jumps ahead of everything in the Tab order	**Never** — breaks natural order, unmaintainable

Keep **DOM order** (source order) **= visual order**: CSS `order`, `flex-direction: row-reverse`, grid placement and absolute positioning move boxes but not the Tab sequence (Chapters 8–9).

Visible focus

Browsers draw a focus ring by default; the commonest way to break keyboard access is `outline: none` in a reset stylesheet. Style it with `:focus-visible` instead — it matches when the browser's heuristics say an indicator is useful (keyboard navigation), not mouse clicks on buttons; `:focus` matches any focus:

```
:focus-visible { outline: 3px solid #c2410c; outline-offset: 2px; }
.bad:focus { outline: none; }     /* never do this without a replacement */
```

The ring needs ≥ 3:1 contrast against the background (non-text contrast, 1.4.11; §3.9) — this dark orange is ~5:1 on white; a light amber would be ~2:1.

Skip links

A **skip link** — the first focusable element — jumps past the header to `<main>`, so keyboard users don't Tab through it on every page (WCAG 2.4.1 Bypass Blocks; landmarks and headings help too):

```
<a class="skip" href="#main">Skip to main content</a>
...
<main id="main" tabindex="-1">…</main>
```

```
.skip { position: absolute; left: 1rem; top: -3rem; }   /* off-screen until focused */
.skip:focus { top: 0; }
```

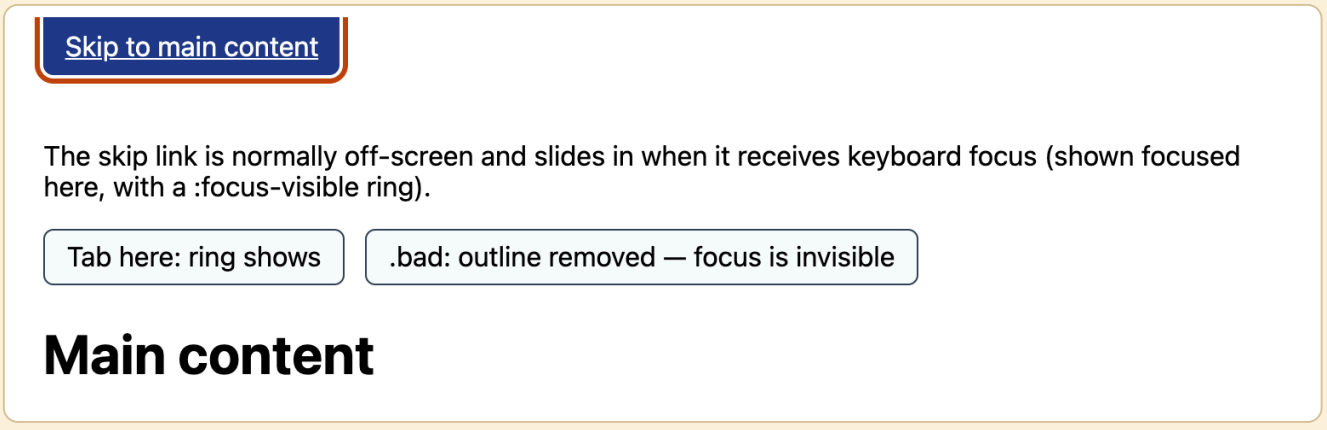


Figure 3.5 — `code/focus.html` with the skip link focused (focused on load for the screenshot).

Managing focus in dynamic UIs

- **Dialogs**: move focus in; keep Tab inside while open; Escape closes; return focus to the opener. Native `<dialog>` with `showModal()` makes the rest inert and handles Escape.
- `inert`: makes a subtree unfocusable and hidden from AT — content behind a custom modal or an off-canvas (slide-in) menu.

- **Single-page app navigation** (JS swaps content instead of loading a page): after a route change, move focus to the new `<h1>` (`tabindex="-1"`) or announce it; otherwise screen-reader users don't know anything happened (React book).
- **Deleting the focused item**: move focus to a sensible neighbour, not the page top.
- **Target size**: WCAG 2.2 2.5.8 (AA) — pointer targets ≥ **24×24 CSS px** (or enough spacing).

Common mistakes

- `outline: none` with no replacement; positive `tabindex`.
- Clickable `div`s/`span`s; links used as buttons and vice versa.
- Custom dropdowns that can't open by keyboard or trap focus; hover-only menus that never open on focus.

In the interview

> **30-second answer:** "Everything interactive must be reachable and operable by keyboard, in a logical order, with visible focus. Native elements give me that; I use `tabindex=0` only for genuine custom widgets and `-1` for programmatic focus targets, never positive values. I style focus with `:focus-visible` instead of removing outlines, add a skip link, and manage focus for dialogs and route changes."

- *Why not `tabindex="5"`?* → It jumps ahead of all `tabindex=0`/native elements, so order no longer matches the page.
- *How do you test keyboard access?* → Unplug the mouse and run the 5-minute test (§3.10).

3.8 Accessible forms

Forms are where failures cost most — a user who can't identify a field can't sign up, pay or apply. Full coverage in Chapter 5; the accessibility essentials:

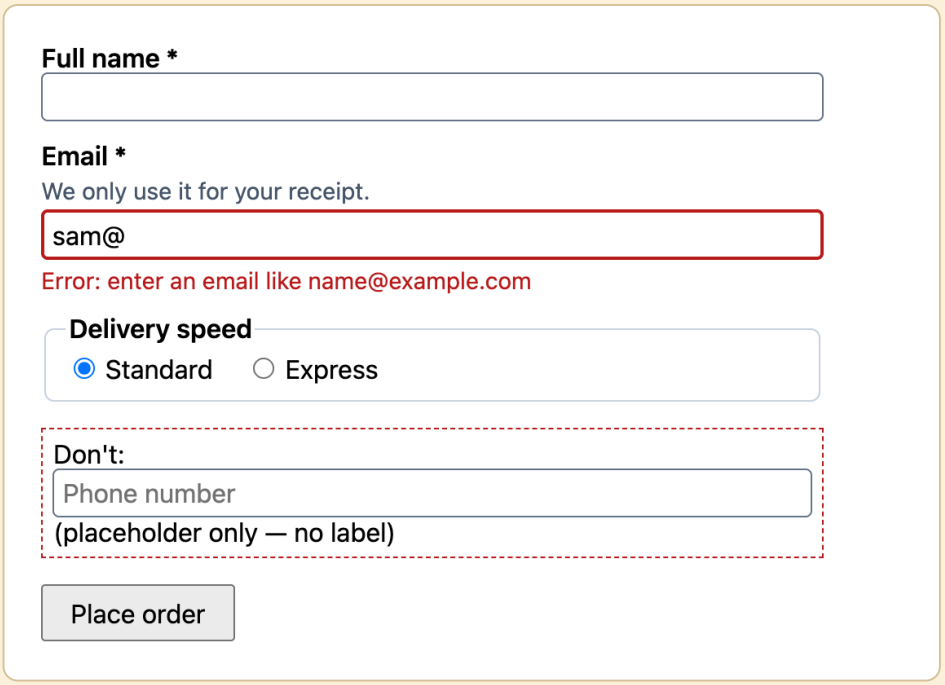


Figure 3.6 — `code/form-a11y.html`.

```
<label for="name">Full name <span aria-hidden="true">*</span></label>
<input id="name" type="text" name="name" autocomplete="name" required>

<label for="email">Email <span aria-hidden="true">*</span></label>
<p class="hint" id="email-hint">We only use it for your receipt.</p>
<input id="email" type="email" name="email" autocomplete="email" required
       aria-invalid="true" aria-describedby="email-hint email-err" value="sam@">
<p class="error" id="email-err">Error: enter an email like name@example.com</p>

<fieldset>
  <legend>Delivery speed</legend>
  <input type="radio" id="std" name="speed" checked><label for="std">Standard</label>
  <input type="radio" id="exp" name="speed"><label for="exp">Express</label>
</fieldset>

Don't: <input type="text" placeholder="Phone number">   <!-- placeholder only, no label -->
```

What Chrome exposes (`code/04_form_tree.mjs`):

```
RootWebArea "Accessible form"
  form
    textbox "Full name"  required=true
    textbox "Email"  required=true invalid="true" value="sam@" description="We only use it for your receipt. Error: enter
an email like name@example.com"
    group "Delivery speed"
      Legend
      radio "Standard"  checked=true
      radio "Express"  checked=false
    textbox "Phone number"
    button "Place order"

After typing into the placeholder-only field, the visible hint is gone: { value: '555', placeholderShown: false }
```

1. **Every field has a** `<label>`, tied by `for`/`id` or by wrapping the input. It becomes the name *and* enlarges the click target (clicking "Standard" checks the radio).
2. **Group related controls** (radios, checkboxes, address parts) in `<fieldset>` + `<legend>`; the legend names the group ("Delivery speed, grouping").
3. **Hints and errors** link via `aria-describedby`; `aria-invalid="true"` marks the error. Error text says what's wrong and how to fix it, not by colour alone (the word "Error:" and the thicker border — WCAG 1.4.1 Use of Color, 3.3.1 Error Identification, 3.3.3 Error Suggestion).
4. **Required**: native `required` (exposed as `required=true`); the visual `*` is `aria-hidden` because the state is already announced — explain the asterisk once at the top.
5. **Placeholders are not labels.** Chrome falls back to the placeholder as a name, but it vanishes on typing (see output), is usually low-contrast, and some AT ignores it.
6. `autocomplete` tokens (`name`, `email`, `street-address`, `one-time-code`…) let browsers and password managers fill fields — WCAG 1.3.5 (Identify Input Purpose), a big help for motor and memory impairments. WCAG 2.2 adds 3.3.7 Redundant Entry (don't make people retype information) and 3.3.8 Accessible Authentication (allow password managers and paste; no memory/puzzle tests without an alternative).
7. On submit with errors, move focus to an error summary or the first invalid field.

3.9 WCAG 2.2

What it is

The **Web Content Accessibility Guidelines** (WCAG, W3C) are the international standard laws and contracts point to. **WCAG 2.2** became a W3C Recommendation (final standard status) on 5 October 2023. It is backwards compatible: meeting 2.2 also meets 2.1 and 2.0.

- **4 principles — POUR**: **Perceivable**, **Operable**, **Understandable**, **Robust**.
- **13 guidelines** (e.g. 1.1 Text Alternatives, 2.1 Keyboard Accessible, 3.3 Input Assistance).
- **Success criteria** (SC) — testable statements numbered like `1.4.3`, each at a **level**:
  - **A** — minimum; without it some users are completely blocked.
  - **AA** — the usual target for laws and policies; "conform to WCAG 2.2 AA" = every A and AA criterion.

- **AAA** — enhanced; not recommended as a blanket site requirement because some criteria can't be met for all content.

Criteria you will meet most often

Principle	SC	Level	In plain words
Perceivable	1.1.1 Non-text Content	A	Images, icons and controls have text alternatives (§3.5)
	1.3.1 Info and Relationships	A	Visual structure is in the markup: headings, lists, labels, tables
	1.3.5 Identify Input Purpose	AA	Personal-data fields use `autocomplete`
	1.4.1 Use of Color	A	Colour is not the only signal
	1.4.3 Contrast (Minimum)	AA	Text ≥ 4.5:1 (large text ≥ 3:1)
	1.4.4 Resize Text	AA	Works at 200 % text size
	1.4.10 Reflow	AA	Usable at 320 CSS px wide (400 % zoom) without horizontal scrolling
	1.4.11 Non-text Contrast	AA	UI parts and meaningful graphics (input borders, focus rings, icons) ≥ 3:1
Operable	2.1.1 Keyboard	A	Everything works from a keyboard
	2.1.2 No Keyboard Trap	A	Focus can always move away
	2.4.1 Bypass Blocks	A	Skip links / landmarks
	2.4.2 Page Titled	A	Descriptive `<title>`
	2.4.3 Focus Order	A	Logical Tab order
	2.4.4 Link Purpose (In Context)	A	Link text makes sense
	2.4.6 Headings and Labels	AA	Headings and labels describe topic or purpose
	2.4.7 Focus Visible	AA	A visible focus indicator
Understandable	3.1.1 Language of Page	A	`<html lang>`
	3.3.1 Error Identification	A	Errors described in text
	3.3.2 Labels or Instructions	A	Inputs have labels/instructions
Robust	4.1.2 Name, Role, Value	A	Every UI component exposes correct name, role, state (§3.4)
	4.1.3 Status Messages	AA	Status messages announced without moving focus (live regions)

What's new in WCAG 2.2

Nine new success criteria:

SC	Level	Requirement
2.4.11 Focus Not Obscured (Minimum)	AA	A focused element isn't entirely hidden by sticky headers, cookie banners, etc.
2.4.12 Focus Not Obscured (Enhanced)	AAA	…not hidden at all
2.4.13 Focus Appearance	AAA	Focus indicator big enough with enough contrast
2.5.7 Dragging Movements	AA	Anything done by dragging also works with single clicks/taps
2.5.8 Target Size (Minimum)	AA	Pointer targets ≥ 24×24 CSS px (or enough spacing)
3.2.6 Consistent Help	A	Help links/contact info in the same place across pages
3.3.7 Redundant Entry	A	Don't ask for the same information twice in a process
3.3.8 Accessible Authentication (Minimum)	AA	No cognitive tests (remembering, transcribing, puzzles) to log in unless an alternative or aid exists — allow paste and password managers
3.3.9 Accessible Authentication (Enhanced)	AAA	Stricter (no object/image recognition exceptions)

One removal: **4.1.1 Parsing** is obsolete — browsers and AT now handle markup errors consistently (Chapter 1's parsing algorithm). Validation is still good practice (§2.10).

Colour contrast — the formula

Contrast ratio compares the **relative luminance** (perceived brightness, 0 = black to 1 = white) of text and background:

1. Convert each sRGB channel (the screen colour space's red/green/blue parts) from 0–255 to 0–1 and undo the gamma curve (screens store brightness on a curve; this recovers linear light): `c ≤ 0.04045 ? c/12.92 : ((c+0.055)/1.055)^2.4`.
2. `L = 0.2126 R + 0.7152 G + 0.0722 B` — green weighs most because eyes are most sensitive to it.
3. Ratio = `(L_lighter + 0.05) / (L_darker + 0.05)`, from 1:1 to 21:1 (black on white).

Level	Normal text	Large text (≥ 24 px, or ≥ 18.66 px bold)	UI components & graphics
AA	4.5:1	3:1	3:1 (1.4.11)
AAA	7:1	4.5:1	—

```
// code/contrast_lib.mjs
export function luminance(hex) {
  const [r, g, b] = hex.replace('#', '').match(/../g).map(h => parseInt(h, 16) / 255)
    .map(c => (c <= 0.04045 ? c / 12.92 : ((c + 0.055) / 1.055) ** 2.4));   // undo sRGB gamma
  return 0.2126 * r + 0.7152 * g + 0.0722 * b;                              // green looks brightest to the eye
}
export function contrast(fg, bg) {
  const [a, b] = [luminance(fg), luminance(bg)].sort((x, y) => y - x);
  return (a + 0.05) / (b + 0.05);
}
```

`code/05_contrast.mjs` output:

```
pair                    ratio   normal text   large text / UI
black on white          21.00:1 pass AA       pass AA | AAA ok
mid grey on white       4.54:1  pass AA       pass AA
one step lighter        4.48:1  FAIL          pass AA
light grey placeholder  2.85:1  FAIL          FAIL
white on blue-500       3.68:1  FAIL          pass AA
white on blue-700       6.70:1  pass AA       pass AA
error red on white      6.47:1  pass AA       pass AA
white on amber          2.15:1  FAIL          FAIL
```

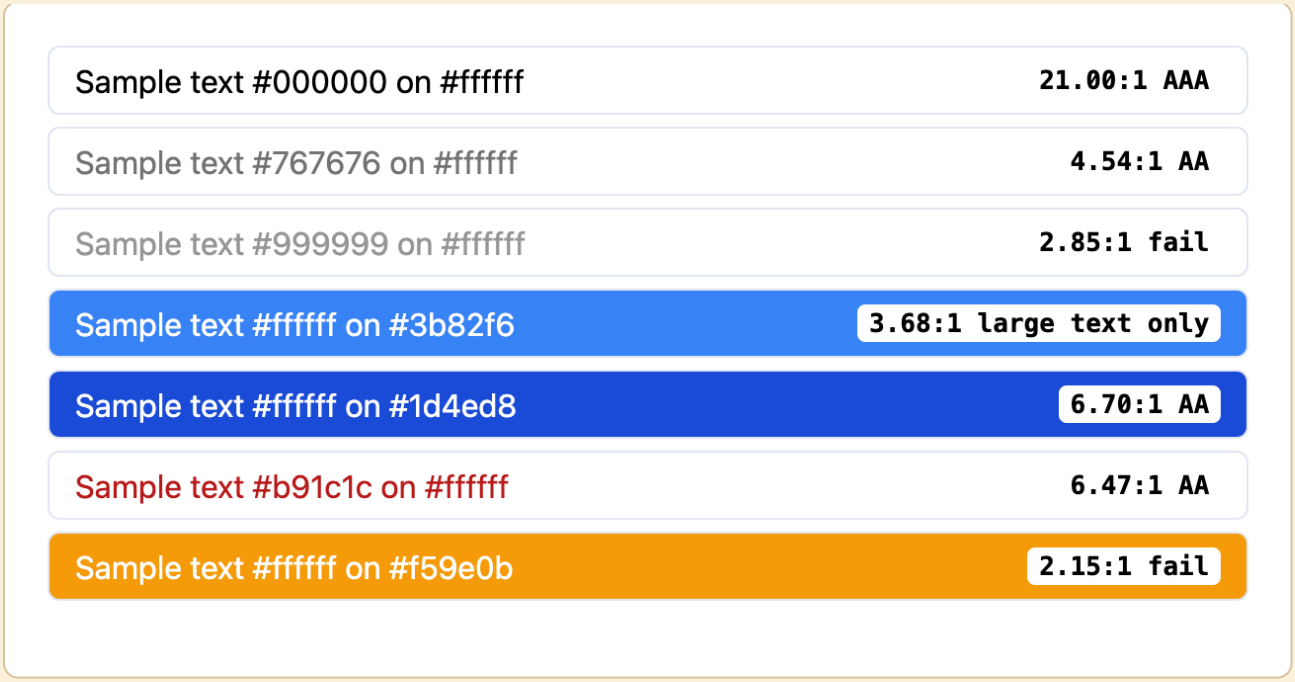


Figure 3.7 — `code/contrast.html` computes each ratio in the browser with the same formula.

Lessons: `#767676` is the lightest grey passing AA on white (4.54:1); one step lighter fails. White on mid-blue `#3b82f6` (a very common button colour) fails for normal text — this book's Chapter 1 page used it and switched to `#1d4ed8`. White on amber/yellow almost always fails. Thresholds are strict: 4.49:1 fails. DevTools' colour picker shows the ratio and AA/AAA lines live.

Self-check

1. Minimum contrast for 16 px body text at AA? → 4.5:1.
2. Name two criteria new in 2.2, and the one removed. → Target Size (Minimum) 2.5.8, Focus Not Obscured (Minimum) 2.4.11 (also Dragging Movements, Consistent Help, Redundant Entry, Accessible Authentication); 4.1.1 Parsing removed.

3.10 Testing tools

How it works — three layers

1. **Automated checkers** quickly find mechanical problems: missing alt/labels/`lang`, contrast, empty links/buttons, invalid ARIA. They can't judge whether alt text is *good*, focus order sensible, or a custom widget usable — estimates range from roughly a third of WCAG criteria to just over half of real-world issues by volume.
2. **Manual keyboard testing** — no tools needed.
3. **Screen-reader testing** — hear what users hear.

Tool	Type	Notes
axe DevTools / **axe-core** (Deque)	Extension; JS library for CI with Playwright, Cypress, Jest (`jest-axe`)	The engine behind many tools; designed for no false positives
Lighthouse	Built into Chrome DevTools	Accessibility score from a subset of axe rules; also performance/SEO (Chapter 14)
WAVE (WebAIM)	Extension / website	Icons drawn on the page — good for learning
DevTools Accessibility pane	Chrome/Firefox/Safari	Role, name, states; Firefox simulates colour-vision deficiencies and shows Tab order
W3C Nu checker / html-validate	Validators	Structural errors (§2.10)
eslint-plugin-jsx-a11y (lint rules), **Testing Library** `getByRole`	Dev-time	Catch issues in React code; role-based test queries double as accessibility tests

Screen reader	Platform	Cost	Start / key tips
NVDA	Windows	Free (open source)	`Insert` or `CapsLock` = NVDA key; `H` next heading, `D` next landmark, `Tab` next focusable
JAWS	Windows	Commercial	Similar single-key navigation; common in workplaces
VoiceOver	macOS, iOS	Built in	macOS: Cmd+F5 toggles; VO = Ctrl+Option; VO+U opens the rotor (headings, landmarks, links)
TalkBack	Android	Built in	Swipe right/left to move; double-tap to activate
Narrator	Windows	Built in	Basic testing

Test the common pairings: NVDA or JAWS with Chrome/Firefox/Edge, VoiceOver with Safari.

A 5-minute keyboard test

1. Click the address bar, then Tab. Is the first stop a working skip link?
2. Can you **see** focus at every step? Is it ever hidden behind a sticky header?
3. Does the order follow the visual layout?
4. Can you operate every control (Enter/Space/arrows), open and close every menu and dialog (Escape), and never get stuck?
5. Zoom to 200 % and 400 %: any overlap or horizontal scrolling?

A mini automated audit

`code/06_mini_audit.mjs` implements a handful of axe-style rules — covering all six WebAIM Million top failures — with jsdom and `dom-accessibility-api` (a JS implementation of the accessible-name spec, used by Testing Library): for each interactive element it calls `computeAccessibleName()` and `getRole()` and flags missing names, placeholder-only fields, positive `tabindex`, unfocusable `role=button`, and focusables inside `aria-hidden`; plus `html lang`, empty headings, `img` alt, broken `aria-describedby`/`labelledby` ids, orphan labels and inline-colour contrast. Run against a deliberately bad sign-up form:

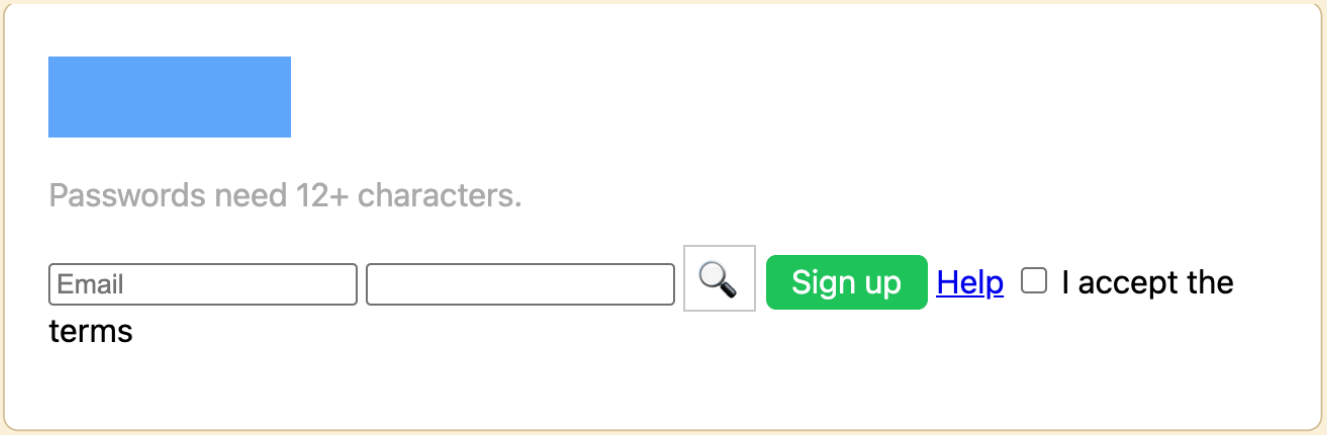


Figure 3.8 — `code/bad-a11y.html`. Looks usable — with a mouse and good eyesight.

```
bad-a11y.html: 12 issue(s)
rule                WCAG    element / problem
html-has-lang       3.1.1   <html> no lang attribute
empty-heading       BP      <h1> heading has no text
image-alt           1.1.1   <img> missing alt (use alt="" if decorative)
label               3.3.2   <input> only a placeholder ("Email"), no label
accessible-name     4.1.2   <input> password field has no accessible name
tabindex            2.4.3   <button> tabindex="3" breaks the natural focus order
focusable-role      2.1.1   <div> role="button" but not keyboard-focusable
aria-hidden-focus   4.1.2   <a> focusable element inside aria-hidden
accessible-name     4.1.2   <input> checkbox has no accessible name
aria-valid-ref      4.1.2   <input> references missing id "tos-text"
orphan-label        1.3.1   <label> "I accept the terms" is not tied to any control
color-contrast      1.4.3   <p> #aaaaaa on #ffffff is 2.32:1 (needs 4.5:1)

form-a11y.html: 1 issue(s)
rule                WCAG    element / problem
label               3.3.2   <input> only a placeholder ("Phone number"), no label
```

(`BP` = best practice, not a specific criterion.) The good form's only finding is its deliberate "Don't" example. The tool **cannot** know whether "🔍" is a good button name (a screen reader may say "magnifying glass tilted left"), whether alt texts are meaningful, or whether Tab order matches the layout — that's what the keyboard and screen-reader passes are for. In real projects use axe-core, not hand-written rules.

In the interview

30-second answer: "I layer it: axe or Lighthouse in development and CI catch missing labels, alt text and contrast; then a manual keyboard pass for focus order, visibility and traps; then a screen-reader pass — VoiceOver with Safari, NVDA with Chrome or Firefox — on key flows. Automation only finds part of the issues, so the manual steps aren't optional."

- *Can Lighthouse give 100 and the page still be inaccessible?* → Yes; it checks a subset of rules and nothing about meaning, order or usability.
- *How do you keep regressions out?* → axe-core in component/end-to-end tests, lint rules, role-based test queries, and accessibility in the definition of done.

Chapter 3 TL;DR

- **Semantic HTML is the foundation of accessibility**: native elements bring roles, names, states, focus and keyboard behaviour that assistive technology understands.
- **Landmarks**: top-level `header` → banner, `nav` → navigation (label multiples), `search`, one `main`, `aside` → complementary, top-level `footer` → contentinfo; `section` is a region only when named; `article` = self-contained.
- **Headings** form the navigable outline; no outline algorithm exists.
- **Accessibility tree** = role + name + description + states + value; names come from `aria-labelledby` → `aria-label` → native label/`alt` → content → `title`/placeholder.

- **Alt text**: purpose in context; `alt=""` for decoration; destination for linked images; complex images get a text equivalent.
- **ARIA**: first rule — use native HTML. ARIA changes semantics only, never behaviour. Use it for names, descriptions, states (`aria-expanded`, `aria-current`, `aria-invalid`), live regions (`role="status"`), and hiding decoration (`aria-hidden`).
- **Keyboard**: everything reachable and operable; DOM order = visual order; `tabindex` 0 / -1 only; `:focus-visible` rings with ≥ 3:1 contrast; skip link; manage focus for dialogs and route changes; targets ≥ 24×24 px.
- **Forms**: real `<label>`s, `fieldset`/`legend`, `aria-describedby` for hints and errors, `aria-invalid`, native `required`, `autocomplete`; placeholders are not labels.
- **WCAG 2.2 AA** is the usual target: POUR, 13 guidelines, A/AA/AAA; contrast 4.5:1 text, 3:1 large text and UI; nine new criteria, 4.1.1 removed.
- **Test in layers**: axe/Lighthouse/WAVE → keyboard pass → screen reader (NVDA, JAWS, VoiceOver, TalkBack).

Next: Chapter 4 — Images, Media and Embeds.

Chapter 4 — Images, Media and Embeds

Photos, logos, charts, podcasts, videos, maps and payment widgets (self-contained interactive components, often from another company) are "media": non-text content, usually in its own file. HTML's elements for them — `<img>`, `<picture>`, `<figure>`, `<audio>`, `<video>`, `<track>`, `<iframe>` (a window showing another page), `<svg>` (vector drawings) and `<canvas>` (a JavaScript drawing surface) — each carry accessibility, performance and security rules beginners miss.

Media is also most of a page's weight (download size): images are typically the largest share of bytes, and the biggest image above the fold (visible without scrolling) often decides how fast the page *feels* — the **Largest Contentful Paint** (LCP, Chapter 14). So this chapter is as much about "which file is downloaded, and when" as about markup.

In this chapter: 4.1 `<img>` · 4.2 Image formats (with real file sizes) · 4.3 Responsive images: `srcset`, `sizes`, `<picture>` · 4.4 Loading hints · 4.5 `<figure>` · 4.6 Audio and video, WebVTT captions · 4.7 `<iframe>` and `sandbox` · 4.8 SVG basics · 4.9 `<canvas>` · 4.10 Choosing the right element.

How the examples work. Every example is a real file in `code/`, with no network access: pictures, the audio beep and the video were generated by `code/make_assets.py` (Python + Pillow, an image library) and `code/make_video.mjs` (headless Chrome records an animated canvas). Screenshots are headless Chrome at 900 CSS px wide and device pixel ratio 2 (a "Retina"-style high-density screen; terms defined in §4.3). `format_sizes.mjs` and `srcset_pick.mjs` print the outputs shown.

4.1 The `<img>` element

What it is

`<img>` embeds a picture. It is a **void element** (no closing tag or content) and a **replaced element**: its box is replaced by an external resource (the image file) whose content CSS cannot style from inside.

Why it matters

- Images carry meaning (a chart, a product). Screen-reader users, people whose images failed on a slow connection, and search engines get only the `alt` text.
- Images without known dimensions make the page jump as they load (**layout shift**), hurting the Core Web Vital **CLS** (Core Web Vitals = Google's three headline page-experience metrics; CLS, Cumulative Layout Shift, scores how much content jumps).

How it works

Attribute	Required?	What it does
`src`	yes	Image URL (relative `media/a.jpg` or absolute `https://…`).
`alt`	yes, in practice	Text alternative: shown on failure, read by screen readers, indexed by search. `alt=""` = decorative.
`width`, `height`	strongly recommended	Intrinsic (natural) size in CSS pixels, **no units** (`width="800"`). Lets the browser reserve space before the file arrives.
`srcset`, `sizes`	optional	Candidate files for different screens (4.3).
`loading`, `decoding`, `fetchpriority`	optional	Loading hints (4.4).

1. The parser meets `<img>` and (unless lazy) starts downloading immediately — even before CSS is ready; the **preload scanner** (a fast look-ahead over raw HTML) finds images early.
2. From `width`/`height` the browser derives the **aspect ratio** (width ÷ height; its built-in stylesheet effectively applies `aspect-ratio: auto <width> / <height>`), so it reserves a correctly shaped box even when CSS makes the image fluid.
3. Bytes arrive, are decoded (decompressed to raw pixels) and painted into the box.
4. On failure the browser shows a broken-image indicator plus the `alt` text.

Standard CSS so images shrink with their container but never stretch:

```css
img { max-width: 100%; height: auto; }   /* fluid width, height follows the aspect ratio */
```

`height: auto` matters: without it the HTML `height="533"` keeps the image 533 px tall while the width shrinks, squashing it. The attributes don't stop an image being responsive — CSS overrides the displayed size; the attributes still supply the ratio.

```html
<!-- code/01_img_basics.html (excerpt) -->
<figure>
  <img src="media/hills-800.jpg" alt="Two green hills under a blue sky with a yellow sun"
       width="800" height="533">
  <figcaption>Figure 1 — JPEG, 800x533. width/height reserve the space before it loads.</figcaption>
</figure>
<img src="media/logo.png" alt="HC" width="80" height="80">
<img src="media/logo.svg" alt="" width="80" height="80">   <!-- decorative duplicate -->

<!-- three images whose file is missing -->
<img src="media/does-not-exist.jpg" alt="Sales chart: revenue up 12% in Q3" width="200" height="120">
<img src="media/does-not-exist.jpg" alt="image123.jpg" width="200" height="120">
<img src="media/does-not-exist.jpg" width="200" height="120">
```

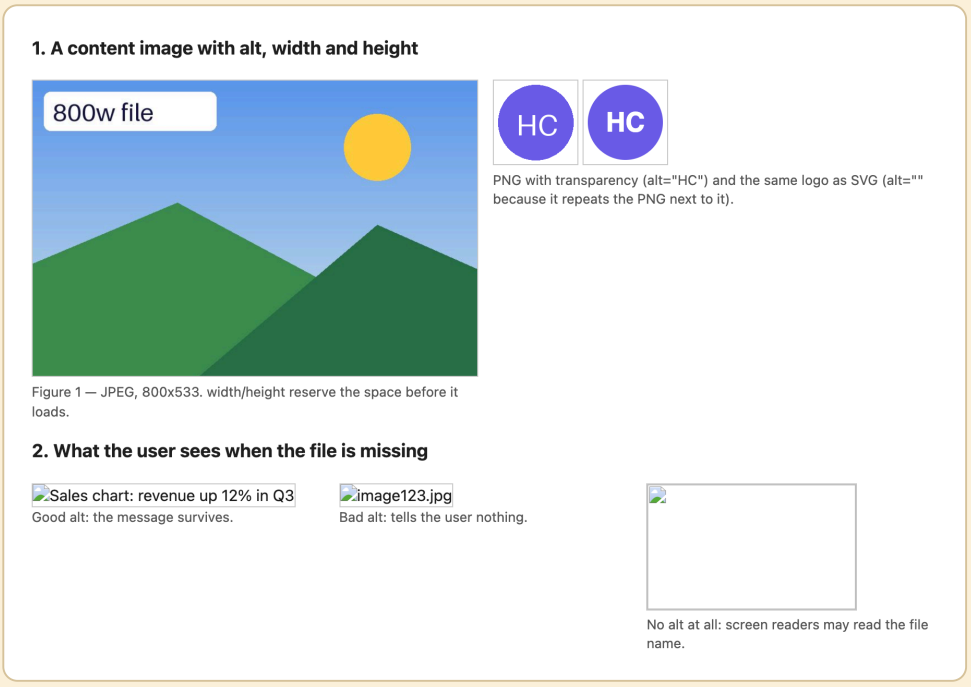


Figure 4.1 — When the file is missing, only the first broken image still communicates anything.

Writing alt text — a decision list

The full decision list is in §3.5. The essentials: describe the *information*, not the pixels (`alt="Revenue rose 12% in Q3"`, not `"bar chart"`); for a linked/button image describe the action or destination (magnifier in a search button → `alt="Search"`; a logo linking home → `alt="Acme home"`); decorative or repeating nearby text → `alt=""` (empty, not missing — missing makes many screen readers read the file name), or better, put decoration in CSS `background-image`; complex images get a short alt plus full text nearby; never start with "image of".

Common mistakes

- Omitting `width`/`height` → content jumps when the image loads.
- Units in the attributes (`width="800px"`) — they take plain integers.
- CSS `width` without `height: auto` → distorted images.
- Keyword-stuffed or file-name alt text.
- A 4000-px camera photo in a 300-px slot (fixed by 4.3).

In the interview

> **30-second answer:** "`<img>` is a void, replaced element. `src` and `alt` are required in practice; `alt` conveys the image's purpose and is empty for decoration. Always set `width` and `height` so the browser

knows the aspect ratio and reserves space, preventing layout shift, and use `max-width: 100%; height: auto` for fluid images."

- *`alt=""` vs no alt?* → Empty says "decorative, skip it"; missing is an error, and assistive tech may read the URL.
- *CSS background image or `<img>`?* → `<img>` for content (alt, indexable, responsive via srcset); backgrounds for decoration.

4.2 Image formats — JPEG, PNG, GIF, WebP, AVIF, SVG

What it is

Raster formats store a grid of pixels (JPEG, PNG, GIF, WebP, AVIF). **Vector** formats store drawing instructions ("circle here, radius 40"), so they scale without blur (SVG). **Lossy compression** discards detail the eye is unlikely to notice for much smaller files (JPEG, lossy WebP, AVIF); **lossless** keeps every pixel exactly (PNG, GIF, lossless WebP/AVIF).

Comparison table

Format	Type	Compression	Transparency	Animation	Best for	Browser support
JPEG (`.jpg`)	raster	lossy	no	no	photos, as the one universal format	everywhere
PNG	raster	lossless	yes (alpha channel: per-pixel see-through level)	no (APNG, animated PNG, exists)	screenshots, UI, logos with transparency, sharp text	everywhere
GIF	raster	lossless, 256 colours max	1-bit (on/off)	yes	legacy animations — prefer video or WebP/AVIF	everywhere
WebP	raster	lossy or lossless	yes	yes	general JPEG/PNG replacement	all modern browsers (Safari since 2020)
AVIF	raster	lossy or lossless	yes	yes	smallest photo files; HDR (high dynamic range)/wide colour	Baseline 2024 (widely available since July 2026)
SVG	vector	text (XML, a tag-based format like HTML)	yes	yes (CSS/SMIL, SVG's own animation tags)	icons, logos, illustrations, simple charts	everywhere

("Baseline" = works in current Chrome, Edge, Firefox and Safari — §1.1.)

Real numbers

`make_assets.py` saved the same 800×533 picture in four formats (JPEG q80, WebP q80, AVIF q60, optimised PNG) — once as a flat graphic, once with photo-like grain. `code/format_sizes.mjs` prints the sizes:

```
hills-800  (flat graphic)
  png       8.8 KB    100% of PNG
  jpg      19.1 KB    216% of PNG
  webp      7.4 KB     84% of PNG
  avif      4.3 KB     48% of PNG
photo-800  (photo-like grain)
  png     508.2 KB    100% of PNG
  jpg     106.8 KB     21% of PNG
  webp    106.3 KB     21% of PNG
  avif     73.7 KB     14% of PNG
```

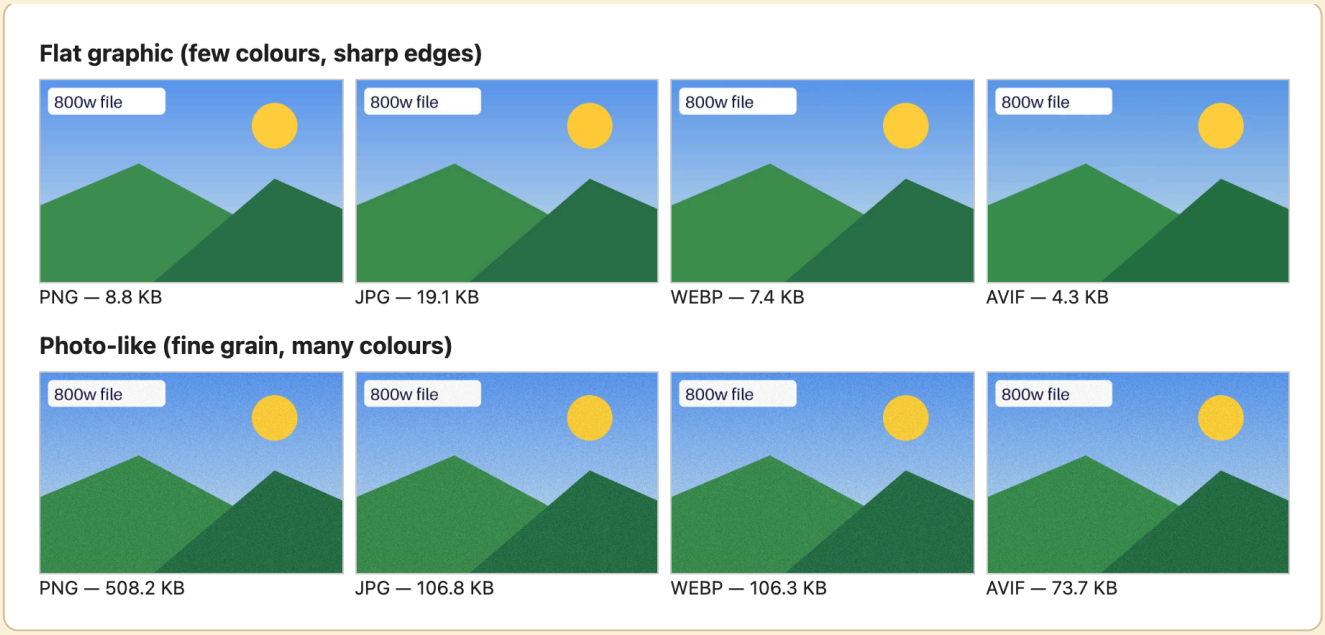


Figure 4.2 — Visually identical at this size; the byte counts are not.

1. **Content decides the winner.** For a flat graphic, lossless PNG (8.8 KB) beats JPEG (19.1 KB): lossless compression is very efficient on large same-colour areas, while JPEG (which compresses in 8×8-pixel blocks) wastes bytes on blocky artefacts around sharp edges. For a photo, PNG is 5× bigger than JPEG.
2. **AVIF was smallest in both cases**; WebP roughly equal or better than the older format. Savings vary per image and quality — measure your own.
3. Vector-like artwork as SVG is often smaller still and sharp at any size (4.8).

How to choose (quick rules)

1. Icon, logo, flat illustration → **SVG**.
2. Photo → **AVIF** (or WebP) with a **JPEG** fallback via `<picture>` (4.3).
3. Screenshot or graphic needing transparency / exact pixels → **PNG** or lossless WebP.
4. Short animation → a muted, looping `<video>` (MP4/WebM) is usually far smaller than a GIF (or animated WebP/AVIF).

Common mistakes

- Photos as PNG "for quality" — huge files, no visible benefit.
- Logos as JPEG — fuzzy edges, no transparency.
- AVIF with no fallback when old devices matter.
- Re-compressing a JPEG repeatedly (quality degrades each time).

Real-world examples

Image CDNs (content delivery networks that store copies worldwide and convert images on the fly — Cloudinary, imgix, Cloudflare Images, Next.js `<Image>`; System Design Fundamentals) read the browser's `Accept` header (the file types it understands) and serve AVIF or WebP when supported, else JPEG.

4.3 Responsive images — `srcset`, `sizes` and `<picture>`

The problem it solves

A 360-px phone doesn't need the 1600-px hero (top-of-page banner) a desktop does — but a phone with three physical pixels per CSS pixel *does* need more pixels than its CSS width suggests. One `src` can't fit everyone. Three needs:

1. **Resolution switching** — same picture, different sizes; the browser downloads the smallest file that's still sharp (`srcset` + `sizes` on `<img>`).
2. **Art direction** — a *different crop or picture* per layout (wide landscape on desktop, tight square on phones); you decide (`<picture>` + `<source media>`).

3. **Format switching** — AVIF/WebP where supported, JPEG elsewhere (`<picture>` + `<source type>`).

- **CSS pixel** — the unit CSS lengths use; roughly one pixel of a traditional 96-dpi screen.
- **Device pixel ratio (DPR)** — physical pixels per CSS pixel (`window.devicePixelRatio`): 1 on older monitors, 2 on Retina laptops, often 3 on phones.

How it works — density descriptors (`x`)

For a **fixed display size**, list files by density:

```
<img srcset="media/hills-400.jpg 1x, media/hills-800.jpg 2x"
     src="media/hills-400.jpg" alt="Hills at sunrise" width="200" height="133">
```

DPR 1 takes `1x`; DPR 2+ takes `2x`; `src` is the fallback for very old browsers.

How it works — width descriptors (`w`) + `sizes`

For a size that **changes with the layout**, give each file's real pixel width with `w`, and the displayed width with `sizes`:

```
<img srcset="hills-400.jpg 400w, hills-800.jpg 800w, hills-1600.jpg 1600w"
     sizes="(max-width: 600px) 100vw, (max-width: 1000px) 50vw, 400px"
     src="hills-800.jpg" alt="Hills at sunrise" width="800" height="533">
```

1. **Evaluate** `sizes` left to right: each entry is `(media condition) length` (a media condition tests the screen, e.g. `(max-width: 600px)` = viewport at most 600 px wide); the first match wins; the last, condition-less entry is the default. Result: the **slot width** in CSS px (`vw` = 1 % of the **viewport**, the visible window area).
2. **Why `sizes`?** The browser picks the image while still downloading the HTML, *before* CSS and layout exist — waiting would delay every image. `sizes` is your promise about the layout; without it, a `w` srcset assumes `100vw`.
3. **Each candidate's density** = file width ÷ slot width (`800w` in a 400-px slot = 2x).
4. **Pick** a candidate with density ≥ DPR — typically the smallest sharp-enough one. The spec lets browsers deviate (e.g. reuse a larger cached file, or save data on a slow connection).

`code/srcset_pick.mjs` models this rule (first matching `sizes` entry → slot; smallest candidate with `w / slot ≥ dpr`, else the largest):

```
sizes = "(max-width: 600px) 100vw, (max-width: 1000px) 50vw, 400px"
viewport  dpr  slot(CSS px)  needs(px)  chosen
     360  1x           360         360  hills-400.jpg
     360  2x           360         720  hills-800.jpg
     360  3x           360        1080  hills-1600.jpg
     768  1x           384         384  hills-400.jpg
     768  2x           384         768  hills-800.jpg
     768  3x           384        1152  hills-1600.jpg
    1280  1x           400         400  hills-400.jpg
    1280  2x           400         800  hills-800.jpg
    1280  3x           400        1200  hills-1600.jpg
```

The biggest file goes to the DPR-3 *phone*, not the desktop — physical pixels, not screen size, decide.

How it works — `<picture>`

`<picture>` wraps any number of `<source>`s and one `<img>`. The browser walks sources **in order** and uses the **first** whose `media` matches and whose `type` it supports; else the `<img>`. The `<img>` is always required: it's what is displayed, and it carries `alt`, `width`, `height`, `loading` for all sources. (An art-directed crop of a different shape can carry its own `width`/`height` on the `<source>`.)

```
<!-- code/03_responsive_images.html (excerpt) -->
<picture>
  <!-- art direction: narrow screens get a square crop -->
  <source media="(max-width: 600px)" srcset="media/hills-square-600.jpg">
  <!-- format switching: best format the browser can decode -->
  <source type="image/avif" srcset="media/hills-800.avif">
  <source type="image/webp" srcset="media/hills-800.webp">
  <img src="media/hills-800.jpg" alt="Hills at sunrise" width="800" height="533">
</picture>
```

The demo prints `img.currentSrc` — the file actually chosen (also visible in DevTools' Network panel):

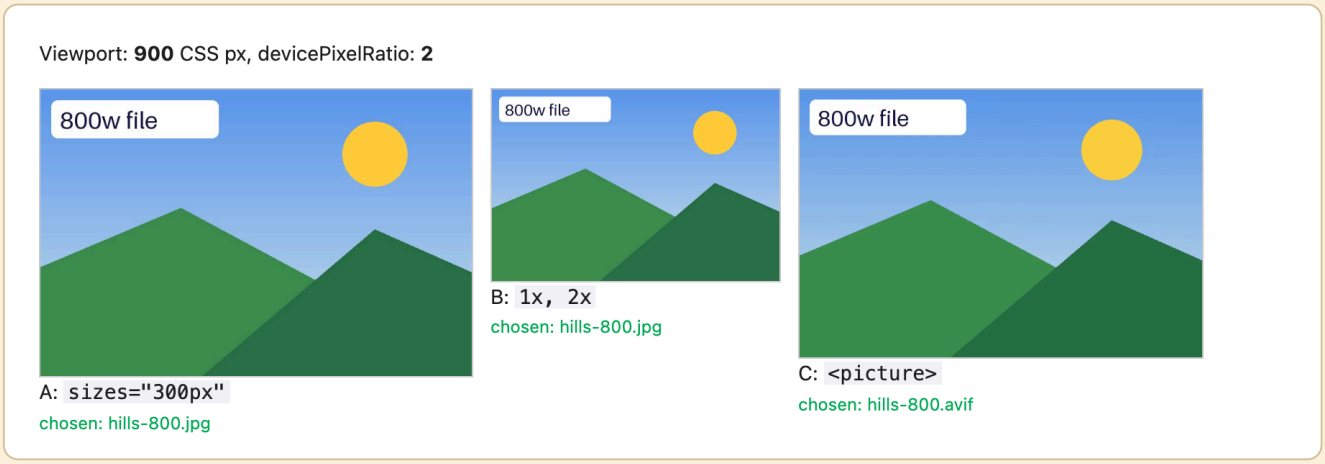


Figure 4.3 — 900 px wide, DPR 2. A: slot 300 px × 2 = 600 px needed → the 800w file. B: DPR 2 → the `2x` file. C: no media match, AVIF supported → AVIF.

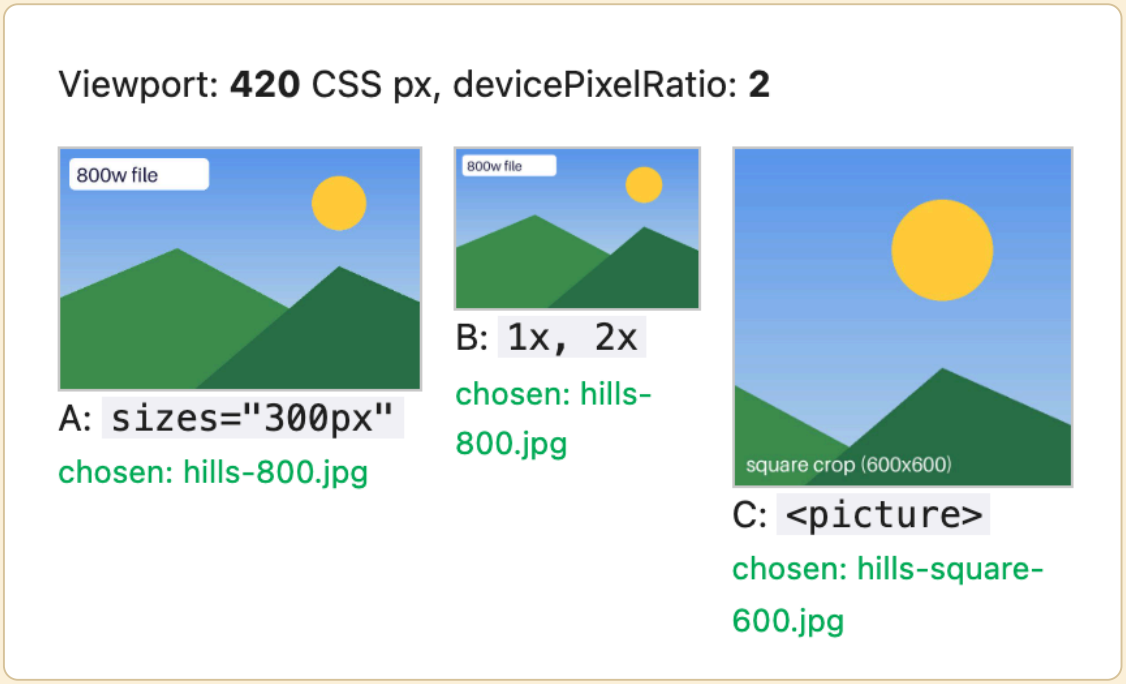


Figure 4.4 — At 420 px only `<picture>` (C) changed crop, because `(max-width: 600px)` now matches; A and B still pick by density.

Trade-offs

Need	Use	Who decides?
Fixed-size image, sharp on high-DPR screens	`srcset` with `1x, 2x`	browser
Fluid image, many screen sizes	`srcset` with `w` + `sizes`	browser
Different crop/composition per layout	`<picture>` + `<source media>`	you
Modern format with fallback	`<picture>` + `<source type>`	browser (by support)

Common mistakes

- **`w` descriptors without `sizes`** — assumed 100vw, so far-too-big files for small slots.
- `sizes` that lie (`sizes="100vw"` for a sidebar thumbnail).
- `alt` on `<source>` — it belongs on the `<img>`.
- `<picture>` just for resolution switching — plain `srcset` lets the browser choose smarter.
- Mixing `x` and `w` in one `srcset` (invalid).

When not to use it

Icons and SVGs don't need `srcset` (vectors are always sharp). For background images use CSS `image-set()` (Chapter 10).

In the interview

30-second answer: "`srcset` lists candidate files: `x` descriptors for fixed-size images, `w` descriptors plus `sizes` for fluid ones. `sizes` gives the display width before CSS loads; the browser divides each file's width by it to get density and picks one dense enough for the device pixel ratio. `<picture>` is for when *I* control the choice — art direction with `media`, format fallbacks with `type`; its `<img>` child is mandatory and carries alt and dimensions. If two sources match, the first wins."

Self-check

1. Viewport 768 px, DPR 2, `sizes="(max-width: 1000px) 50vw, 400px"` — slot and needed pixels? → 384 CSS px; 768 device px → the 800w file.

4.4 Loading hints — `loading`, `decoding`, `fetchpriority`

What they are

Hints that change *when* and *how urgently* an image is fetched or decoded — the page works without them, loads faster with them.

Attribute	Values	Effect
`loading`	`eager` (default), `lazy`	`lazy` defers fetching until near the viewport. Also works on `<iframe>`.
`fetchpriority`	`high`, `low`, `auto` (default)	Raises/lowers download priority relative to other images. Baseline since Oct 2024.
`decoding`	`sync`, `async`, `auto` (default)	Whether decoding may happen off the main rendering step (not holding up drawing the rest). Rarely needed.

Native lazy loading for images and iframes has been Baseline since December 2023 (Firefox 121 was last, for iframes; images worked everywhere earlier) — no JS library needed for the basic case.

How it works

1. Browsers check each lazy element's distance to the viewport with a generous margin (a screen or more), so images normally arrive before the user scrolls to them.
2. Lazy images do **not** delay the window `load` event (they may never be fetched).
3. Lazy loading happens only with JavaScript enabled — a privacy measure, since otherwise image requests would reveal scroll position even with JS off.
4. `fetchpriority="high"` moves an image up the download queue — for the one image that is the page's LCP element (e.g. the main product photo).

```
<!-- code/04_lazy_loading.html (excerpt) -->
<img id="hero" src="media/hills-400.jpg" alt="Hero: hills at sunrise" width="400" height="267"
     fetchpriority="high">                                  <!-- above the fold -->
<img id="lazy1" class="far" src="media/photo-800.jpg" alt="Grainy hills"
     width="400" height="267" loading="lazy">               <!-- 3000 px lower -->
<img id="lazy2" src="media/hills-1600.jpg" alt="Large hills"
     width="400" height="267" loading="lazy">
<script>
  addEventListener('load', () => {
    report.textContent = 'after the load event:\n' + [...document.images]
      .map(i => `${i.id.padEnd(6)} ${(i.getAttribute('loading') || '(none)').padEnd(6)} ${i.naturalWidth ? 'LOADED' : 'not
loaded yet'}`)
      .join('\n');
  });
</script>
```

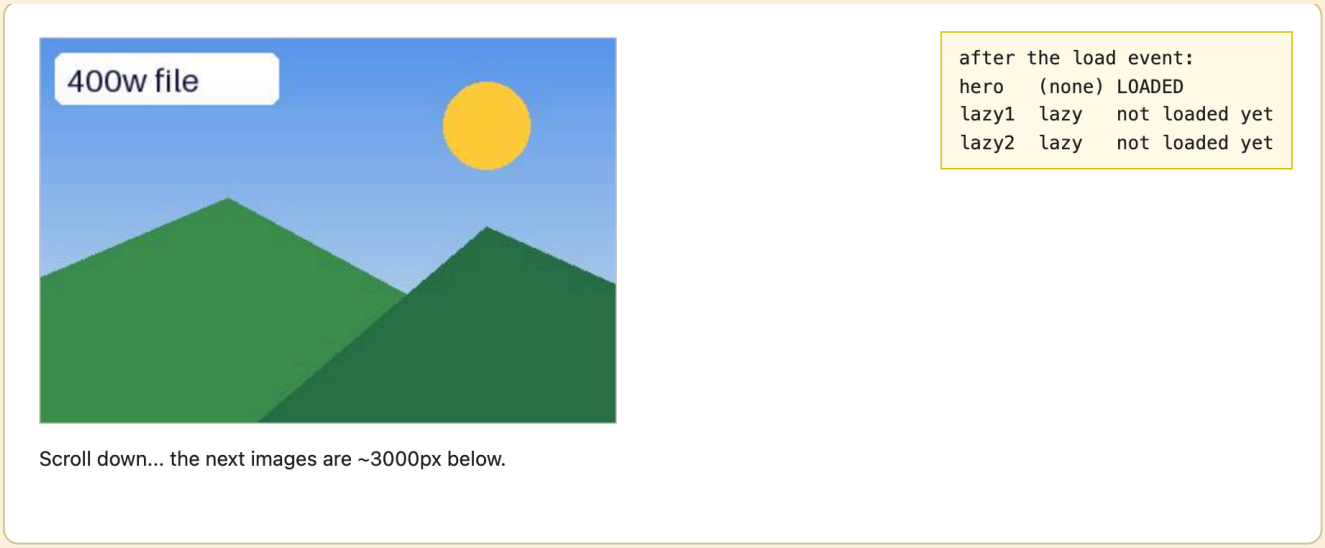


Figure 4.5 — At `load`, only the hero was downloaded; the two lazy images (~660 KB) were never fetched.

Common mistakes

- **Lazy-loading the hero / LCP image** — it waits for layout, delaying the most important paint. Never lazy-load above the fold.
- Lazy images without `width`/`height` — they start 0×0, the page jumps, and the browser misjudges which are near the viewport.
- `fetchpriority="high"` on many images: if everything is high, nothing is.

4.5 `<figure>` and `<figcaption>`

What it is

`<figure>` groups **self-contained content referenced from the main text** — image, diagram, code listing, quotation, table — and `<figcaption>` (first or last child) captions it; the caption becomes the figure's accessible name. Test: could it move to an appendix with the text saying "see Figure 3"? Then it's a figure. (Example markup in §4.1.)

Caption vs alt

The **alt** *replaces* the image for people who can't see it; the **caption** *adds* context for everyone. Don't copy one into the other (a screen reader reads it twice). If the caption fully describes the image, a short alt is fine.

Common mistakes

- Wrapping *every* image in `<figure>` (decorative images and icons aren't figures).
- A `<p>` or `<div class="caption">` instead of `<figcaption>` — the association is lost for assistive technology.

4.6 Audio and video — `<audio>`, `<video>`, `<source>`, `<track>`

What it is

`<audio>` and `<video>` play media natively — no plugins (add-ons like the old Flash player). Both are **media elements** sharing one JavaScript API: `play()`, `pause()`, `currentTime`, `volume`, events like `play`, `ended`, `timeupdate`.

How it works

1. The browser reads `<source>` children **in order** and picks the first whose `type` (MIME type such as `video/webm`, optionally with codecs — the compression methods inside) it can play. (Safari 17.4+ may pick the best supported one instead.) Or use one `src` attribute.

2. Other content inside the element (e.g. a download link) is **fallback** for browsers without the element at all — *not* for unsupported formats.
3. `<track>` attaches timed text files in **WebVTT** (`.vtt`, first line `WEBVTT`) — captions, subtitles, descriptions, chapters.
4. Without `controls` there is no UI; you'd build your own with the JS API.

Attribute	Element	Meaning
`controls`	both	Built-in play/pause/volume/seek UI.
`autoplay`	both	Try to start immediately. Browsers block autoplay **with sound**; muted autoplay is generally allowed.
`muted`	both	Start muted (needed for autoplay).
`loop`	both	Restart at the end.
`preload`	both	`none` (nothing until play), `metadata` (duration/dimensions only), `auto` (may download all).
`poster`	video	Image shown before playback.
`playsinline`	video	On iPhone, play in the page instead of full-screen.
`width` / `height`	video	Reserve space (like `<img>`).

`<track>` attributes: `kind` (`subtitles` default, `captions`, `descriptions`, `chapters`, `metadata`), `src`, `srclang`, `label` (shown in the captions menu), `default`.

Captions vs subtitles. *Subtitles* translate dialogue for hearing viewers who don't know the language. *Captions* serve viewers who can't hear: dialogue **and** meaningful sounds ("[door slams]").

```
WEBVTT

00:00:00.000 --> 00:00:01.400
The sun rises on the left.

00:00:01.400 --> 00:00:03.000
[soft wind] It drifts to the right.
```

```
<!-- code/05_audio_video.html (excerpt) -->
<audio controls preload="metadata">
  <source src="media/tone.wav" type="audio/wav">
  <p>Your browser cannot play this audio. <a href="media/tone.wav">Download the WAV</a>.</p>
</audio>

<video controls preload="none" poster="media/poster.jpg" width="640" height="360">
  <source src="media/clip.webm" type="video/webm">
  <track kind="captions" src="media/captions.vtt" srclang="en" label="English" default>
  <p>Your browser cannot play this video. <a href="media/clip.webm">Download it</a>.</p>
</video>
```

A second demo video is seeked to 2.0 s by script and shows the matching cue. Gotcha: browsers refuse `<track>` files on `file://` pages (text tracks must be same-origin or served with CORS — headers by which a server lets other origins read its files), so the demo adds the same cues (timed caption lines) with the TextTrack API — `video.addTextTrack('captions', 'English', 'en').addCue(new VTTCue(1.4, 3.0, '…'))`. Over `http://` the `<track>` works as written.

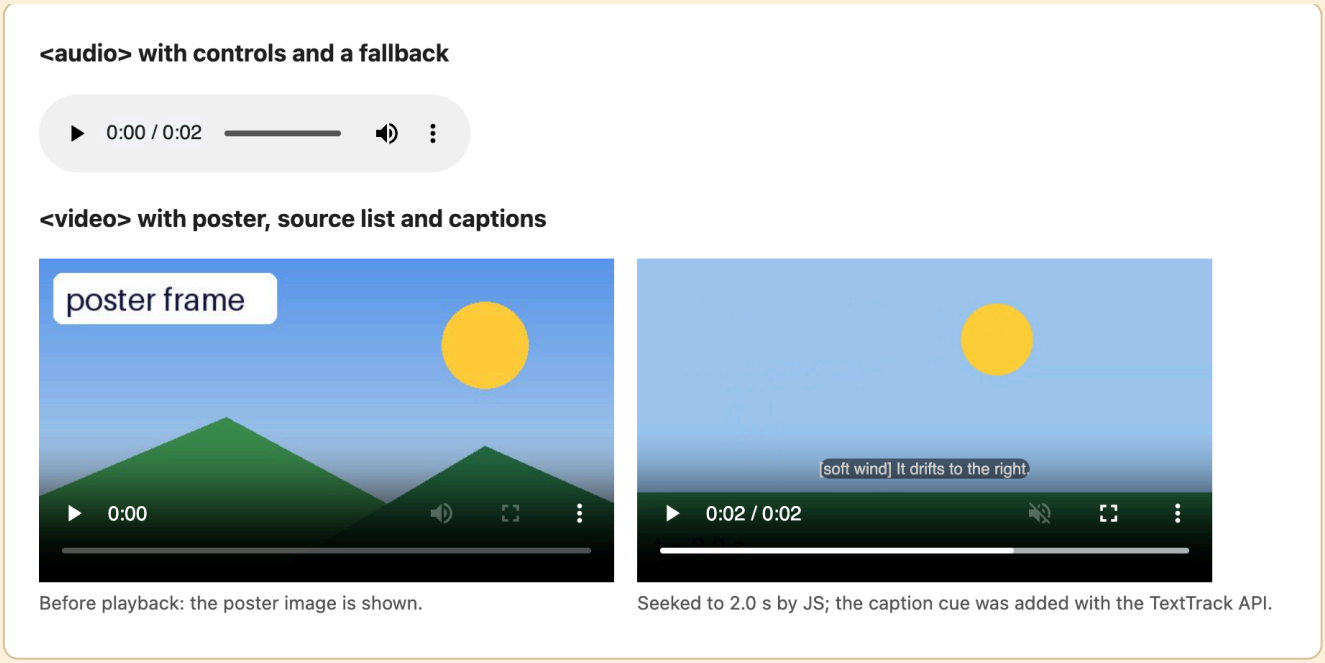


Figure 4.6 — Native controls, a poster before playback, and a caption cue at t = 2 s.

Formats in practice

A **container** (MP4, WebM) wraps separately compressed video and audio streams, each with its own **codec** (coder–decoder).

Container	Typical codecs	Support
MP4	H.264 video + AAC audio	every browser — the safest single choice
WebM	VP9 or AV1 video + Opus audio	Chrome, Edge, Firefox, Safari (14.1+ on macOS, 17.4+ on iOS)
MP4	AV1 or HEVC (H.265)	varies by browser and hardware — always provide a fallback
MP3 / AAC / Opus / WAV	audio	MP3 and AAC everywhere; WAV is uncompressed (large)

List the smallest well-supported format first, MP4/H.264 last as the universal fallback, and always declare `type` so the browser skips formats without downloading them.

Accessibility and UX checklist

- Captions for every video with speech (WCAG 2.2 SC 1.2.2); a transcript for audio-only (1.2.1).
- No autoplay with sound; anything moving for more than 5 seconds must be pausable (2.2.2).
- `poster` and dimensions on `<video>` so the page doesn't jump.
- Big sites stream with **adaptive bitrate** (HLS or DASH switch quality with bandwidth) via Media Source Extensions (an API letting JS feed video data into `<video>` piece by piece) — `<video src="file.mp4">` downloads one fixed-quality file. YouTube and Netflix work this way.

Common mistakes

- `autoplay` without `muted` (and `playsinline` on iOS) — browser autoplay policies block it.
- Expecting the fallback paragraph for an unsupported *format*.
- A 50 MB unoptimised "background video"; `preload="auto"` on a page of many videos.

Real-world examples

Social feeds replace GIFs with `<video autoplay muted loop playsinline>` — the MP4 is typically a fraction of the GIF size.

In the interview

30-second answer: "`<video>` / `<audio>` with `controls`, multiple `<source>`s with `type` so the browser picks a playable format, and `<track kind="captions">` with WebVTT for accessibility. Autoplay only works muted; add `playsinline` for iOS. For large video, use adaptive streaming (HLS/DASH), not one big file."

4.7 Embedding other pages — `<iframe>`

What it is

An `<iframe>` (inline frame) shows **another complete HTML document** in a rectangle of your page, with its own DOM, CSS, JS and **browsing context** (its own document, history, scripts). If it's from another **origin** (scheme + host + port), the **same-origin policy** stops the two pages reading each other's data.

Uses: YouTube/Vimeo players, maps, payment fields (Stripe, Adyen host card inputs in iframes so card numbers never touch your page), social posts, ads, code playgrounds.

Why it matters

You're putting someone else's code in your page — and iframes are how attackers do **clickjacking**: loading *your* site invisibly inside theirs to trick users into clicking. Both directions need controls.

How it works — key attributes

Attribute	Purpose
`src`	URL of the embedded document.
`srcdoc`	Inline HTML instead of `src` (previews, sandboxed user content).
`title`	**Required for accessibility**: announced so users know what the frame contains.
`width`, `height`	Size (default 300×150).
`loading="lazy"`	Defer off-screen frames — a big win for video and map embeds.
`sandbox`	Extra restrictions (below).
`allow`	Permissions policy: which powerful features the frame may use, e.g. `allow="fullscreen; camera; geolocation"`.
`referrerpolicy`	How much of your URL to send as `Referer`, e.g. `no-referrer`.

`sandbox`. An empty `sandbox` turns *everything* off — scripts, forms, pop-ups, plugins, top-level navigation (changing the whole tab's address) — and gives the content a unique **opaque origin** (so it can't read its real origin's cookies/storage). Add back only what's needed: `allow-scripts`, `allow-forms`, `allow-popups`, `allow-same-origin`, `allow-modals`, `allow-downloads`, `allow-top-navigation-by-user-activation`.

> Warning: `allow-scripts` + `allow-same-origin` on a document from **your own origin** lets its script reach into the parent and remove the sandbox — the sandbox becomes meaningless. Use that pair only for content from a different origin.

The demo loads one small document (via `srcdoc`) into three frames; its script writes "allowed" if it ran, then tries to read the parent's title:

```
<!-- code/06_iframe.html (excerpt) -->
<iframe class="widget" title="Widget without sandbox"></iframe>
<iframe class="widget" sandbox title="Widget, fully sandboxed"></iframe>
<iframe class="widget" sandbox="allow-scripts" title="Widget, scripts only"></iframe>

<!-- inside each frame -->
<script>
  document.querySelector('#js span').textContent = 'allowed (set by JS)';
  let parentTitle;
  try { parentTitle = window.parent.document.title; }
  catch (e) { parentTitle = 'blocked (' + e.name + ')'; }
</script>
```

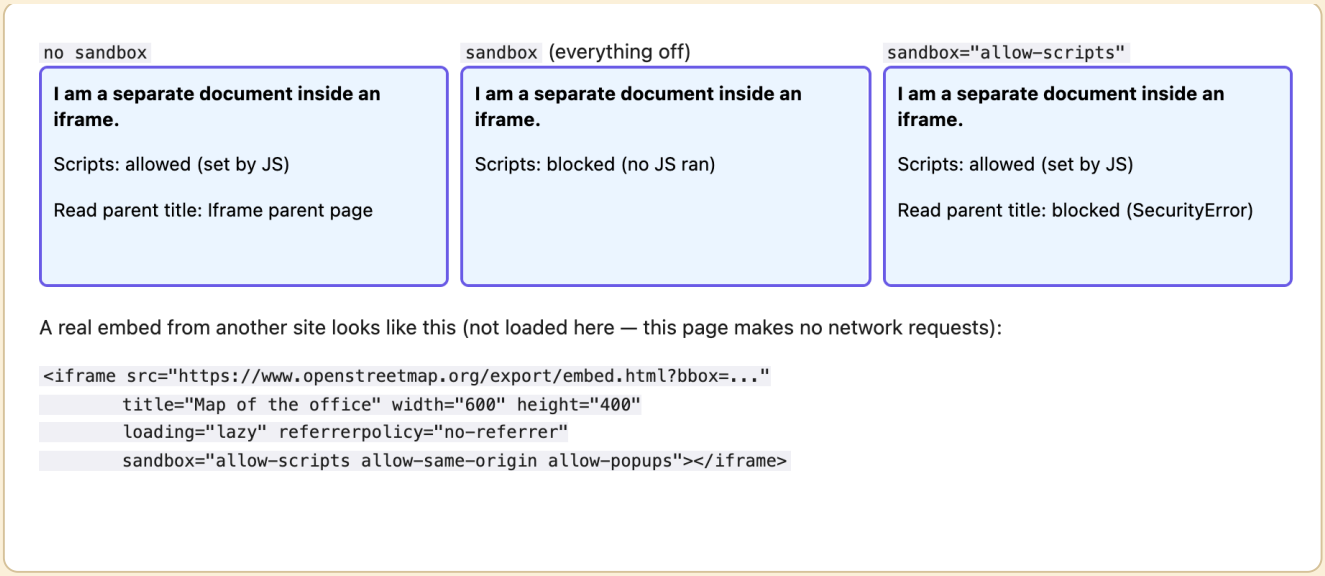


Figure 4.7 — `srcdoc` content shares the parent's origin, so unsandboxed it reads the parent. `sandbox` blocks scripts; `allow-scripts` lets them run in an opaque origin (no `allow-same-origin`), so touching the parent throws `SecurityError`.

The other direction: stopping others from framing *you*

Your server decides who may embed your pages via HTTP response headers:

- `Content-Security-Policy: frame-ancestors 'self'` (modern; can list allowed sites), or
- the older `X-Frame-Options: DENY` / `SAMEORIGIN`.

Banks and login pages send these against clickjacking — which is why some sites show an error inside an iframe.

`<embed>` and `<object>`

Old elements for plugin content (Flash, Java applets — both gone). Today mostly used for PDFs, where an `<iframe>` or plain link is usually better. Don't use them for new work.

Common mistakes

- No `title`.
- Heavy third-party embeds (video, maps, chat) loaded eagerly — each can pull megabytes of script. Use `loading="lazy"` or a "click to load" placeholder (**facade**).
- `sandbox="allow-scripts allow-same-origin"` on same-origin untrusted content.
- Iframes for your own page layout (a 1990s pattern) — breaks URLs, history, accessibility.

In the interview

> **30-second answer:** "An iframe embeds another document with its own browsing context; cross-origin frames are isolated by the same-origin policy. I always add a `title`, lazy-load off-screen embeds, restrict untrusted content with `sandbox` plus only the needed tokens, and use `allow` for permissions. To stop my site being framed — clickjacking — I send `Content-Security-Policy: frame-ancestors`."

- *Why do payment providers use iframes?* → The card field lives on their origin, so the merchant page (and any malicious script on it) can't read the card number — which also shrinks the merchant's PCI DSS scope (the card industry's security standard; less card data touched, fewer rules apply).

4.8 SVG basics

What it is

SVG (Scalable Vector Graphics) is an XML language for 2-D vector drawings: shapes, not pixels, so razor-sharp at any zoom and tiny for icons and logos. Use it **inline** (part of the DOM — CSS and JS can style and animate each shape), as a file via `<img src="x.svg">`, or in CSS (`background-image: url(x.svg)`).

How it works

1. **Coordinates.** `viewBox="minX minY width height"` defines internal coordinates: `viewBox="0 0 200 120"` → x 0→200, y 0→120, **y pointing down**. `width`/`height` (or CSS) set the displayed size; the viewBox scales to fit (preserving aspect ratio by default).
2. **Shapes.** `<rect x y width height rx>`, `<circle cx cy r>`, `<ellipse>`, `<line x1 y1 x2 y2>`, `<polyline>`/`<polygon points="x,y x,y …">`, `<text>`.
3. **Paths.** `<path d="…">`: `M x y` move, `L` line, `H`/`V` horizontal/vertical line, `C` cubic Bézier (a smooth curve shaped by two "pull" points), `Q` quadratic (one pull point), `T` smooth quadratic continuation, `A` arc, `Z` close. Upper-case = absolute, lower-case = relative.
4. **Painting.** `fill`, `stroke`, `stroke-width` — also CSS properties.
5. **Reuse.** Define once with `<symbol id="…" viewBox="…">`, place copies with `<use href="#id"/>`.
6. **`currentColor`.** `fill`/`stroke="currentColor"` takes the surrounding text colour, so one icon works on a green button and a red one (set CSS `color` on each).

```
<!-- code/07_svg.html (excerpt) -->
<svg width="200" height="120" viewBox="0 0 200 120" role="img" aria-labelledby="t1">
  <title id="t1">Basic SVG shapes</title>
  <rect x="10" y="10" width="60" height="40" rx="6" fill="#6c5ce7"/>
  <circle cx="110" cy="30" r="20" fill="#fdcb6e" stroke="#e17055" stroke-width="4"/>
  <line x1="150" y1="10" x2="190" y2="50" stroke="#2d3436" stroke-width="3"/>
  <polygon points="10,110 40,70 70,110" fill="#00b894"/>
  <path d="M90 80 Q 120 40 150 80 T 190 80" fill="none" stroke="#0984e3" stroke-width="3"/>
</svg>

<!-- same viewBox, half the display size: still sharp -->
<svg width="100" height="60" viewBox="0 0 200 120" aria-hidden="true">…</svg>

<!-- icon sprite + currentColor -->
<svg width="0" height="0" style="position:absolute">
  <symbol id="check" viewBox="0 0 24 24">
    <path d="M4 12l5 5L20 6" fill="none" stroke="currentColor" stroke-width="3"/></symbol>
</svg>
<button><svg class="icon" aria-hidden="true"><use href="#check"/></svg>Save</button>

<img src="media/logo.svg" alt="HC logo" width="60" height="60">
```

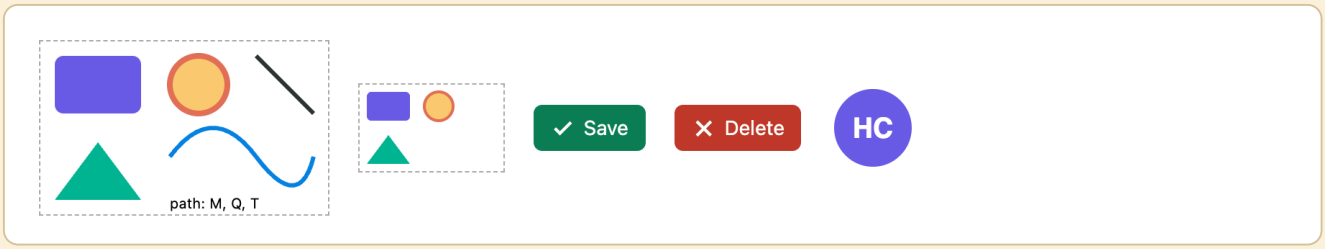


Figure 4.8 — Shapes and a path; the same drawing scaled down; one icon coloured per button via `currentColor`; an SVG file as `<img>`.

Inline SVG vs `<img src="x.svg">`

	Inline `<svg>`	`<img src="x.svg">` / CSS background
Style parts with page CSS / animate with JS	yes	no (a closed image)
`currentColor` follows the text colour	yes	no
Cached separately	no (part of the HTML)	yes
Scripts inside the SVG run	yes — never inline untrusted SVG	no (scripts and external resources disabled)
Accessibility	`role="img"` + `<title>`/`aria-label`, or `aria-hidden="true"` if decorative	`alt` like any image

Common mistakes

- No `viewBox` → the drawing gets cropped instead of scaling.
- Icon-only buttons whose icon is neither hidden nor accompanied by text → no accessible name.
- Inlining user-uploaded SVG — it can contain `<script>` (an **XSS**, cross-site scripting, risk). Serve it through `<img>` or sanitise it.
- Design-tool exports full of editor metadata — run them through an optimiser such as SVGO.

4.9 `<canvas>` — an introduction

What it is

`<canvas>` is a blank bitmap that JavaScript paints via a drawing API — the 2D context (`getContext('2d')`) or WebGL/WebGPU (GPU-accelerated, for 3-D). Afterwards it holds only **pixels**, not shapes: there's no DOM node for "bar 3", so to change something you redraw.

How it works

1. The `width`/`height` **attributes** size the drawing buffer (default 300×150). CSS sizes only stretch it on screen, blurring it.
2. On high-DPR screens, make the buffer `devicePixelRatio` times bigger, keep the CSS size, and `ctx.scale(dpr, dpr)` to keep drawing in CSS-pixel coordinates — one canvas pixel per physical pixel.
3. Draw with `fillRect`, `strokeRect`, `beginPath/moveTo/lineTo/arc/fill/stroke`, `fillText`, `drawImage`, after setting `fillStyle`, `strokeStyle`, `font`, `textAlign`.
4. Animate by clearing and redrawing in `requestAnimationFrame` (calls your code just before each screen refresh) — how `make_video.mjs` drew the 4.6 clip.

```
<!-- code/08_canvas.html (excerpt) -->
<canvas id="chart" width="420" height="220" role="img"
        aria-label="Bar chart of visits: Mon 12, Tue 19, Wed 7, Thu 15, Fri 22">
  Visits: Mon 12, Tue 19, Wed 7, Thu 15, Fri 22.
</canvas>
<script>
  const data = { Mon: 12, Tue: 19, Wed: 7, Thu: 15, Fri: 22 };
  const canvas = document.getElementById('chart');
  const dpr = window.devicePixelRatio || 1;                 // sharp on high-DPI screens
  canvas.style.width = canvas.width + 'px'; canvas.style.height = canvas.height + 'px';
  canvas.width *= dpr; canvas.height *= dpr;
  const ctx = canvas.getContext('2d');
  ctx.scale(dpr, dpr);

  const max = Math.max(...Object.values(data)), barW = 50, gap = 25, base = 190;
  ctx.font = '13px system-ui'; ctx.textAlign = 'center';
  Object.entries(data).forEach(([day, v], i) => {
    const x = 30 + i * (barW + gap), h = (v / max) * 150;
    ctx.fillStyle = '#6c5ce7'; ctx.fillRect(x, base - h, barW, h);
    ctx.fillStyle = '#222';
    ctx.fillText(v, x + barW / 2, base - h - 6);
    ctx.fillText(day, x + barW / 2, base + 18);
  });
  ctx.beginPath(); ctx.moveTo(20, base); ctx.lineTo(410, base); ctx.stroke();
</script>
```

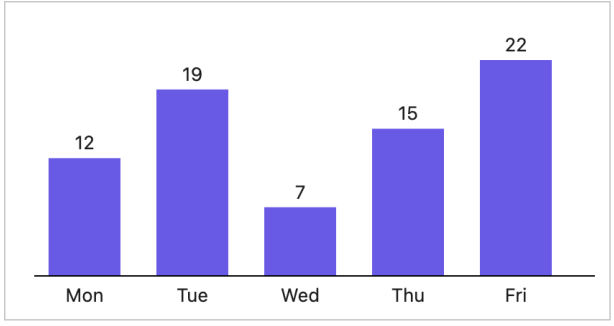


Figure 4.9 — Painted with the 2D context. The fallback text and `aria-label` give assistive technology the data it cannot "see".

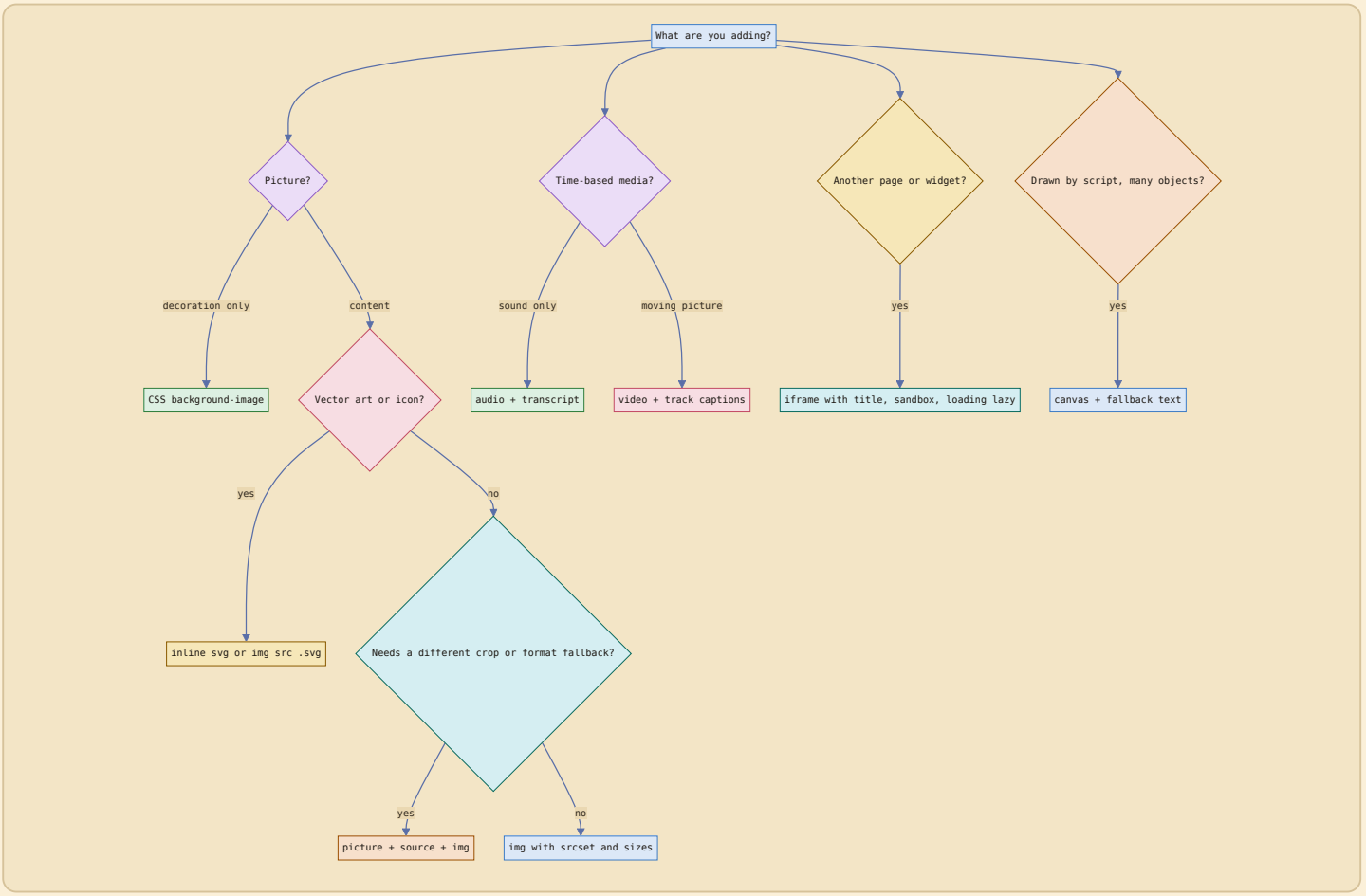
Canvas vs SVG

	SVG	Canvas
Model	retained (browser keeps every shape as a DOM node)	immediate (draw and forget; only pixels remain)
Scales without blur	yes	no — redraw at the new size
Events per shape (click a bar)	native	you do hit-testing (which shape is under the pointer)
Accessibility	text and titles are real content	none built in — add fallback text / `aria-label` / a data table
Performance sweet spot	up to a few thousand elements	many thousands of objects, pixel effects, games, real-time charts
Examples	icons, logos, D3 charts, a clickable 50-region map	games, image editors, Figma-style editors, big-data plots

Common mistakes

- Sizing with CSS only → blurry, stretched drawing.
- No fallback/accessible text for meaningful drawings.
- Text-heavy UI on canvas that would be simpler and more accessible as HTML.

4.10 Choosing the right element



Chapter interview round-up

1. **How do you make images responsive?** CSS `max-width:100%; height:auto`; `srcset` (`x` or `w` + `sizes`); `<picture>` for art direction/format fallback; `width`/`height` against layout shift.
2. **How do you improve image loading performance?** Right format (AVIF/WebP), right size (srcset), compression, lazy-load below the fold, `fetchpriority="high"` for the LCP image, dimensions to avoid CLS, a CDN.
3. **When is `alt=""` correct?** Decorative images and images whose information is already in adjacent text.
4. **How would you embed an untrusted HTML snippet?** An iframe with `sandbox` (no `allow-same-origin`), possibly via `srcdoc`, or sanitise it.
5. **SVG vs PNG for a logo?** SVG: sharp at any size, usually smaller, styleable; PNG only if too complex for vectors.
6. **Canvas vs SVG for a chart?** SVG for typical dashboards (accessible, per-element interaction); canvas for tens of thousands of points or real-time updates.

Chapter TL;DR

- `<img src alt width height>`: meaningful alt, empty alt for decoration, dimensions prevent layout shift, fluid via `max-width:100%; height:auto`.
- Formats: SVG for vectors; AVIF/WebP (JPEG fallback) for photos; PNG for pixel-exact graphics; video instead of GIF. Measure — content type changes the winner.
- `srcset` + honest `sizes` lets the browser choose by device pixels; `<picture>` lets *you* choose by layout (`media`) or format (`type`); `currentSrc` shows the winner.
- `loading="lazy"` below the fold, never on the hero; `fetchpriority="high"` for the LCP image.
- `<figure>`/`<figcaption>` for referenced, self-contained content; alt and caption complement, not duplicate.
- `<video>`/`<audio>` with `controls`, ordered `<source type>`s, WebVTT `<track>` captions; autoplay only when muted; stream big video adaptively.
- `<iframe>`: `title`, `loading="lazy"`, minimal `sandbox` tokens, `allow`; defend your own pages with `frame-ancestors`.
- SVG: `viewBox`, shapes and paths, `symbol`/`use`, `currentColor`; never inline untrusted SVG.
- `<canvas>`: script-painted pixels, buffer = attributes × `devicePixelRatio`, provide accessible fallback.

Next: Chapter 5 — Tables and Forms.

Chapter 5 — Tables and Forms

Tables and forms are where HTML *structure* matters most. A table is a set of relationships ("this 135 is North's revenue in Q2") that screen readers announce cell by cell. A form is a contract with a server about which named values get sent, how and when — plus a built-in validation engine (the browser's own checker that rejects bad input before sending) most developers under-use. Get them right and keyboard support, autofill (the browser filling saved details), mobile keyboards, validation messages and accessibility come almost free; get them wrong (a `<div>` "button", an unlabelled input, an error shown only in red) and no JavaScript fully repairs it.

In this chapter: 5.1 Table semantics · 5.2 How a form works (`action`, `method`, `name`, labels, buttons) · 5.3 Input types · 5.4 `select`, `datalist`, `textarea`, `fieldset`, `output`, `progress`, `meter` · 5.5 Built-in validation, `:user-invalid` and the Constraint Validation API · 5.6 `autocomplete` · 5.7 Accessible error messages.

Every example is a file in `code/`. HTML files were screenshotted in headless Chrome (900 CSS px wide); the Node scripts (`table_headers.mjs`, `form_data.mjs`, `constraint_api.mjs`) run under jsdom and print the outputs shown.

5.1 Table semantics

What it is

A **data table** is a grid where each **data cell** (`<td>`) is described by one or more **header cells** (`<th>`) — column header, row header or both. The elements exist so software can compute that relationship, the way your eyes go up to the column heading and left to the row heading.

Why it matters

A screen-reader user entering "135" hears "Q2, North, 135" — without header markup, just "135". Structure also lets browsers repeat `<thead>` on printed pages and CSS style header rows/columns with plain selectors (Chapter 6).

How it works — the elements

Element	Role
`<table>`	The table.
`<caption>`	The table's title and accessible name. First child of `<table>`.
`<colgroup>` / `<col>`	Describe columns; only `background`, `border`, `width`, `visibility: collapse` apply to them.
`<thead>`	Header row(s).
`<tbody>`	Body rows; several allowed to group rows; inserted automatically if omitted.
`<tfoot>`	Footer/summary rows (totals).
`<tr>`	A row.
`<th>`	Header cell. `scope="col"`, `"row"`, `"colgroup"` or `"rowgroup"` says which cells it describes.
`<td>`	Data cell.
`colspan` / `rowspan`	Span several columns/rows.
`headers="id1 id2"`	On a `<td>`: explicit ids of its header cells, for irregular tables where `scope` isn't enough.

Steps: (1) `<caption>` with units; (2) column headings in `<thead>` as `<th scope="col">`; (3) a row-naming first cell as `<th scope="row">`; (4) totals in `<tfoot>`; (5) two-level headings: `colspan`/`rowspan` with `scope="colgroup"`/`"rowgroup"`. `colgroup` refers to a column group, so declare groups with `<colgroup>`; `rowgroup` refers to the enclosing `<thead>`/`<tbody>`/`<tfoot>`. Truly irregular tables: `id` + `headers`; (6) CSS: `border-collapse: collapse`, right-aligned numbers, `font-variant-numeric: tabular-nums` (equal-width digits so columns line up).

```html
<!-- code/01_table.html (first table) -->
<table>
  <caption>Quarterly revenue by region (USD thousands)</caption>
  <colgroup><col><col><col class="highlight"><col></colgroup>
  <thead>
    <tr><th scope="col">Region</th><th scope="col">Q1</th><th scope="col">Q2</th><th scope="col">Q3</th></tr>
  </thead>
  <tbody>
    <tr><th scope="row">North</th><td class="num">120</td><td class="num">135</td><td class="num">150</td></tr>
    <tr><th scope="row">South</th><td class="num">98</td><td class="num">102</td><td class="num">97</td></tr>
    <tr><th scope="row">West</th><td class="num">143</td><td class="num">160</td><td class="num">171</td></tr>
  </tbody>
  <tfoot>
    <tr><th scope="row">Total</th><td>361</td><td>397</td><td>418</td></tr>
  </tfoot>
</table>
```

```css
table { border-collapse: collapse; }
caption { caption-side: top; text-align: left; font-weight: 700; }
th, td { border: 1px solid #c8c8d8; padding: 6px 10px; }
tbody tr:nth-child(even) td { background: #f8f8fc; }        /* zebra stripes */
td.num, tfoot td { text-align: right; font-variant-numeric: tabular-nums; }
col.highlight { background: #fff6d6; }                      /* whole Q2 column */
```

The second table has two header levels:

```html
<colgroup span="2"></colgroup>   <!-- Term + Course -->
<colgroup span="2"></colgroup>   <!-- the column group that "Students" spans -->
<thead>
  <tr><th rowspan="2" scope="col">Term</th><th rowspan="2" scope="col">Course</th>
      <th colspan="2" scope="colgroup">Students</th></tr>
  <tr><th scope="col">Enrolled</th><th scope="col">Completed</th></tr>
</thead>
<tbody>
  <tr><th rowspan="2" scope="rowgroup">Spring</th><th scope="row">HTML</th><td>40</td><td>35</td></tr>
  <tr><th scope="row">CSS</th><td>32</td><td>30</td></tr>
</tbody>
<tbody> … Autumn … </tbody>
```

Quarterly revenue by region (USD thousands)

Region	Q1	Q2	Q3
North	120	135	150
South	98	102	97
West	143	160	171
Total	**361**	**397**	**418**

Course enrolment (two-level headers)

Term	Course	Students	
		Enrolled	Completed
Spring	**HTML**	40	35
	CSS	32	30
Autumn	**HTML**	52	41
	CSS	45	44

Figure 5.1 — Two data tables. The highlighted column comes from a single `<col class="highlight">`.

What assistive technology computes

`code/table_headers.mjs` rebuilds each grid (accounting for spans) and, for every data cell, collects the `<th>`s above it with `scope` `col`/`colgroup` and to its left with `row`/`rowgroup` — a simplified form of the HTML standard's "assigning header cells" algorithm. Output (abridged):

```
Table: Quarterly revenue by region (USD thousands)
   120  <- Q1 / North
   135  <- Q2 / North
   150  <- Q3 / North
    98  <- Q1 / South
   ...
Table: Course enrolment (two-level headers)
    40  <- Students / Enrolled / Spring / HTML
    35  <- Students / Completed / Spring / HTML
    32  <- Students / Enrolled / Spring / CSS
    30  <- Students / Completed / Spring / CSS
    52  <- Students / Enrolled / Autumn / HTML
    41  <- Students / Completed / Autumn / HTML
    45  <- Students / Enrolled / Autumn / CSS
    44  <- Students / Completed / Autumn / CSS
```

That is roughly what a screen reader speaks entering each cell.

Responsive tables

Keep the table a table and scroll it inside a labelled, focusable region (so keyboard users can scroll):

```
<div role="region" aria-labelledby="cap1" tabindex="0" style="overflow-x: auto">
  <table><caption id="cap1">…</caption>…</table>
</div>
```

Changing `display` on table elements (e.g. `display: block` to stack cells) has historically made some browsers drop table semantics — test with a screen reader if you do.

When not to use it

Never for page layout — use Grid or Flexbox (Chapter 8, Chapter 9). On an inherited layout table, `role="presentation"` tells assistive technology to ignore its table semantics.

Common mistakes

- `<td><b>…</b></td>` instead of `<th>`.
- No `<caption>` (or an unconnected heading outside the table).
- The obsolete `summary` attribute — explain in the caption or nearby text.
- Empty spacing cells; merged cells "because it looks nicer" (each merge makes navigation harder).
- `<table>` for card lists or form layout.

In the interview

30-second answer: "Tables are for tabular data only. I give them a `<caption>`, use `<thead>/<tbody>/<tfoot>`, mark headers with `<th>` and `scope` (`col`/`row`, or `colgroup`/`rowgroup` for spanning headers), and fall back to `id`/`headers` for irregular tables, so screen readers announce the right headers per cell. On small screens I wrap the table in a scrollable, focusable region rather than changing its display."

- *When do you need `headers`?* → When a cell's headers aren't regularly in its row/column, e.g. irregular multi-level tables.
- *`border-collapse: collapse` vs `separate`?* → Collapse merges adjacent borders into one line; separate (default) draws each cell's border with `border-spacing` between.

5.2 How a form works — `action`, `method`, `name`

What it is

A `<form>` collects input and **submits** it: it builds name/value pairs from its controls (`<input>`, `<select>`, `<textarea>`, `<button>`; `<output>` is never submitted) and sends them as an HTTP request (Chapter 1).

How it works — submission step by step

1. The user presses a submit button, or **Enter** in a text field (*implicit submission*).

2. The browser runs **constraint validation** (5.5); if anything is invalid it stops, shows a message and focuses the first invalid control.
3. A `submit` event fires; JS can `event.preventDefault()` and handle it itself (e.g. with `fetch`).
4. The browser builds the **form data set** from *successful controls*:
   - the control has a `name` (no name → not sent);
   - it is not **disabled** (disabled → not sent; `readonly` → sent);
   - checkboxes and radios only if **checked** (value defaults to `"on"` without `value`);
   - `<select multiple>` sends one pair per selected option;
   - only the submit button actually used is included (if named).
5. It encodes the data using `enctype` (e.g. space → `+`, `@` → `%40`) and sends it to `action` using `method`:
   - `method="get"` (default): data in the URL query string — `/signup?name=Ada+Lovelace&email=ada%40example.com`. Bookmarkable, cacheable and meant to be idempotent (repeating it has the same effect as once — System Design Fundamentals) — right for searches and filters, wrong for passwords or anything that changes data.
   - `method="post"`: data in the request body. For anything that creates/changes something or is private.
   - `method="dialog"`: inside a `<dialog>`, closes it instead of sending.
6. `enctype` (POST only): `application/x-www-form-urlencoded` (default), `multipart/form-data` (**required for file uploads**), `text/plain` (debugging only).
7. The browser navigates to the server's response (unless JS prevented it).

Also: `action` defaults to the current URL; `target` works like a link's; `novalidate` turns off step 2's built-in UI; a button can override the form with `formaction`, `formmethod`, `formenctype`, `formnovalidate`, `formtarget`; a control *outside* the form joins it with `form="formId"`.

Labels

Every control needs a **label**, connected explicitly or by wrapping:

```
<label for="name">Full name</label>          <!-- explicit: for = the input's id -->
<input id="name" name="name">

<label>Email <input type="email" name="email"></label>   <!-- implicit: wrap the input -->
```

A real `<label>` gives the accessible name (read on focus), a bigger click target, and lets voice-control users say "click Email". A `placeholder` is **not** a label: it disappears on typing, usually has poor contrast and isn't reliably announced (§3.8).

Buttons

`<button>` has three types, and **the default inside a form is `submit`**:

type	Does
`submit` (default)	Submits the form.
`reset`	Resets every control to its initial value (rarely wanted — clicked by accident).
`button`	Nothing by itself — for JavaScript actions.

A "Show password" or "Add row" button without `type="button"` submits the form when clicked — a classic bug (the page "refreshes").

```html
<!-- code/02_form_basics.html (excerpt) -->
<form action="02_form_basics.html" method="get">
  <label for="name">Full name</label>
  <input id="name" name="name" autocomplete="name" value="Ada Lovelace">
  <label>Email <input type="email" name="email" autocomplete="email" value="ada@example.com"></label>
  <label><input type="checkbox" name="newsletter" value="yes" checked> Send me the newsletter</label>
  <input type="hidden" name="source" value="chapter5">
  <button type="submit">Sign up</button>
  <button type="reset">Reset</button>
  <button type="button" id="preview">Preview data</button>
</form>
<script>
  const form = document.querySelector('form');
  const show = () => result.textContent = '?' + new URLSearchParams(new FormData(form));
  show();
  preview.addEventListener('click', show);
</script>
```

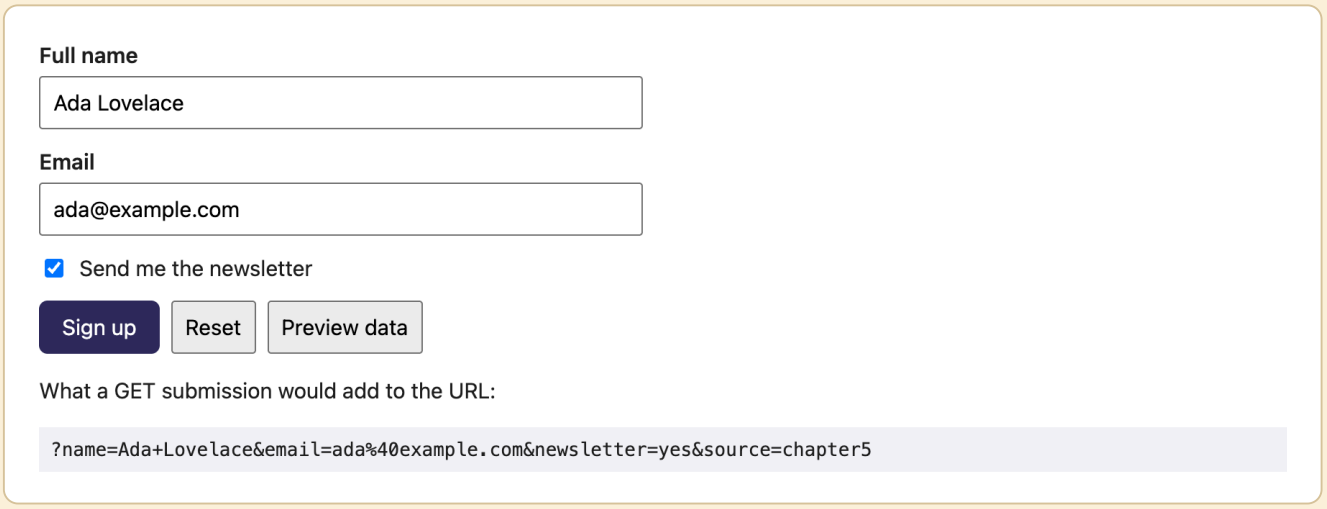


Figure 5.2 — `FormData` + `URLSearchParams` produce exactly the query string a GET submission would send.

Exactly what gets sent

`code/form_data.mjs` puts every tricky case in one form and prints `new FormData(form)`, `getAll("skills")` and the GET URL:

```html
<input name="name" value="Ada Lovelace">
<input name="email" type="email" value="ada@example.com">
<input name="nick">                                  <!-- empty text: still sent, as "" -->
<input type="checkbox" name="newsletter" value="yes" checked>
<input type="checkbox" name="terms" value="yes">       <!-- unchecked: NOT sent -->
<input type="radio" name="plan" value="free">
<input type="radio" name="plan" value="pro" checked>
<select name="skills" multiple> <option selected>HTML</option><option selected>CSS</option><option>JS</option> </select>
<input name="coupon" value="SAVE10" disabled>          <!-- disabled: NOT sent -->
<input name="country" value="IN" readonly>             <!-- readonly: sent -->
<input value="no name attribute">                      <!-- no name: NOT sent -->
<input type="hidden" name="source" value="ch5">
<textarea name="bio">Hi &amp; bye</textarea>
```

Output:

```
FormData entries:
  name       = "Ada Lovelace"
  email      = "ada@example.com"
  nick       = ""
  newsletter = "yes"
  plan       = "pro"
  skills     = "HTML"
  skills     = "CSS"
  country    = "IN"
  source     = "ch5"
  bio        = "Hi & bye"

data.getAll("skills") = [ 'HTML', 'CSS' ]

GET URL:
  http://localhost/signup?
name=Ada+Lovelace&email=ada%40example.com&nick=&newsletter=yes&plan=pro&skills=HTML&skills=CSS&country=IN&source=ch5&bio=Hi+%26+bye
```

(`http://localhost` is jsdom's pretend address.) The unchecked checkbox, disabled field and unnamed input are absent — so server-side, **"checkbox missing" means "unchecked"**; code expecting `terms=no` never sees it.

Security notes

- Client-side anything (validation, `readonly`, hidden fields, `disabled`) is **not security** — anyone can edit the request. The server must validate everything again.
- Never send passwords or personal data with GET — URLs land in history, server logs and `Referer` headers.
- POST forms that change data need server-side **CSRF** protection (cross-site request forgery: another site secretly makes your browser submit a form to a site you're logged into); defences are secret per-form tokens or `SameSite` cookies, which the browser won't send on requests started by other sites (JavaScript book).

In the interview

30-second answer: "A form submits its *successful controls* — named, enabled, and for checkboxes/radios, checked — to `action` using `method`. GET puts data in the URL (searches, filters); POST in the body (changes, private data); file uploads need `multipart/form-data`. Every control needs a `<label>`; buttons default to `type=submit`, so non-submit buttons need `type=button`. Server-side validation is mandatory."

- *What isn't submitted?* → Unnamed, disabled, unchecked controls, unselected options, buttons other than the one used, `<output>`.
- *Disabled vs readonly?* → Disabled: not focusable, not submitted, not validated. Readonly: focusable, submitted, not editable, not validated.
- *How do you submit via JS without reloading?* → Listen for `submit`, `preventDefault()`, send `new FormData(form)` with `fetch`. To trigger submission from code use `form.requestSubmit()` — it runs validation and fires `submit`; `form.submit()` skips both.

5.3 Input types

What it is

`type` changes the control's UI, validation and — on phones — the on-screen keyboard. Unknown types fall back to `text`, so a specific type is never worse.

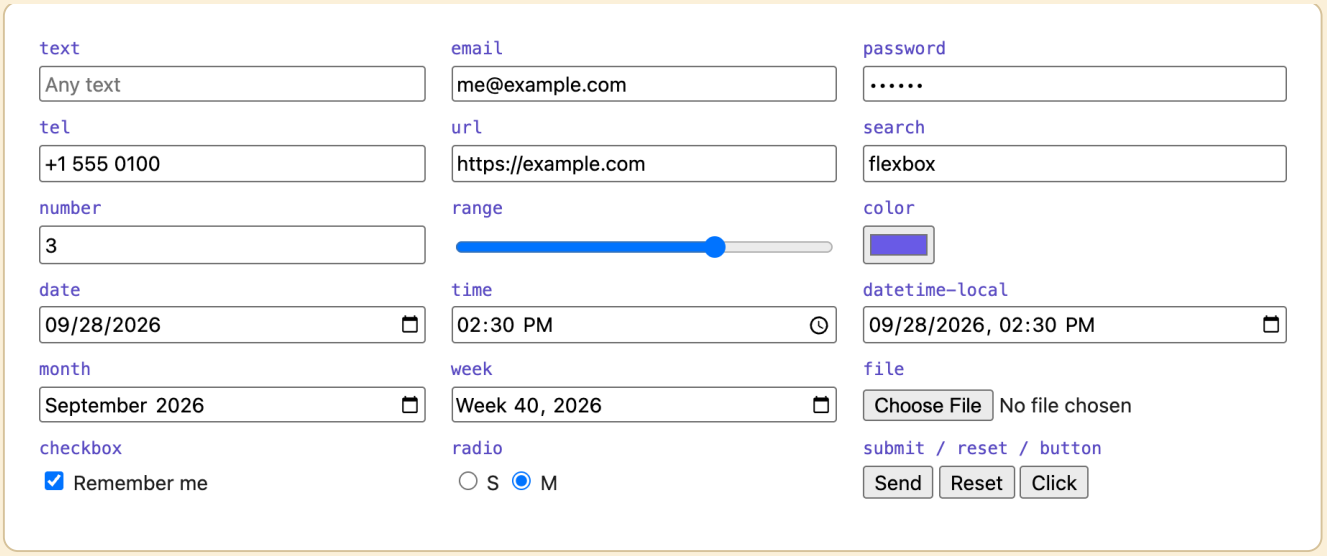


Figure 5.3 — Chrome on macOS (`code/03_input_types.html`); other browsers draw them differently.

Type	Use for	Built-in validation	Mobile keyboard / UI
`text`	any single-line text	—	normal
`email`	email address	`typeMismatch` if not `x@y` shaped; `multiple` allows a comma list	`@` key visible
`password`	secrets	—	masked; password managers engage
`tel`	phone numbers	**none** (formats vary worldwide — add `pattern` if needed)	phone keypad
`url`	web addresses	`typeMismatch` without a scheme like `https:`	`/` and `.com` keys
`search`	search boxes	—	"search" enter key; may show a clear button
`number`	real quantities (count, age)	`min`, `max`, `step`, `badInput`	numeric keypad, spin buttons
`range`	imprecise values (volume)	always a value within range	slider
`color`	colour pick	value always `#rrggbb`	colour picker
`date`, `time`, `datetime-local`	dates/times	`min`, `max`, `step`	native pickers; value `YYYY-MM-DD`, `HH:MM`, `YYYY-MM-DDTHH:MM`
`month`, `week`	months/weeks	same	**not supported in desktop Firefox and Safari** — text box
`checkbox`	independent yes/no	`required` = must be checked	—
`radio`	one choice from a group (same `name`)	`required` on any one = must pick one	arrow keys move within the group
`file`	uploads	`accept` filters the picker (not enforced); `multiple`	camera/photos on phones (`capture`)
`hidden`	data the user doesn't see	—	not rendered (trivially editable — not secret)
`submit`, `reset`, `button`, `image`	buttons (prefer `<button>`, which can contain HTML)	—	—

- **`type="number"` only for numbers you'd do maths on.** Card numbers, postcodes, IDs are *strings of digits*: `type="text" inputmode="numeric"` (maybe `pattern="[0-9]*"`). `number` drops leading zeros, accepts `e`, and scroll-wheels change it. A 6-digit one-time code: `type="text" inputmode="numeric" autocomplete="one-time-code"`.
- A date's **display** follows the user's locale (language-and-region, `09/28/2026` in the US), but its **value** is always ISO `2026-09-28`.

`inputmode` and `enterkeyhint`

`inputmode` picks the virtual keyboard **without** changing validation: `text`, `numeric`, `decimal`, `tel`, `email`, `url`, `search`, `none`. `enterkeyhint` labels the Enter key: `enter`, `done`, `go`, `next`, `previous`, `search`, `send`.

Other common input attributes

`placeholder` (hint, not label), `value`, `required`, `readonly`, `disabled`, `autofocus` (sparingly — can disorient screen-reader users), `size`, `list` (→ `<datalist>`), `multiple`, `accept`, `spellcheck`, `autocapitalize`.

5.4 Choosing and grouping — `select`, `datalist`, `textarea`, `fieldset`, `output`, `progress`, `meter`

```
<!-- code/04_select_textarea_fieldset.html (excerpt) -->
<label>Country
  <select name="country" autocomplete="country">
    <option value="">-- choose --</option>
    <optgroup label="Europe"><option value="DE">Germany</option><option value="FR">France</option></optgroup>
    <optgroup label="Asia"><option value="IN" selected>India</option><option value="JP">Japan</option></optgroup>
  </select>
</label>

<select name="skills" multiple size="4"> … </select>

<input name="browser" list="browsers" value="Fire">
<datalist id="browsers"><option value="Chrome"><option value="Firefox"><option value="Safari"><option value="Edge">
</datalist>

<textarea name="message" rows="4" maxlength="200">Hello!
Line two keeps its line break.</textarea>

<fieldset>
  <legend>Delivery speed</legend>
  <label><input type="radio" name="speed" value="standard" checked> Standard (free)</label>
  <label><input type="radio" name="speed" value="express"> Express (+$5)</label>
</fieldset>
<fieldset disabled> <legend>Gift options (disabled fieldset)</legend> … </fieldset>

<progress max="100" value="70">70%</progress>
<meter min="0" max="100" low="60" high="85" optimum="20" value="91">91%</meter>
<input type="number" id="qty" name="qty" value="3"> x $12 = <output name="total" for="qty">$36</output>
```

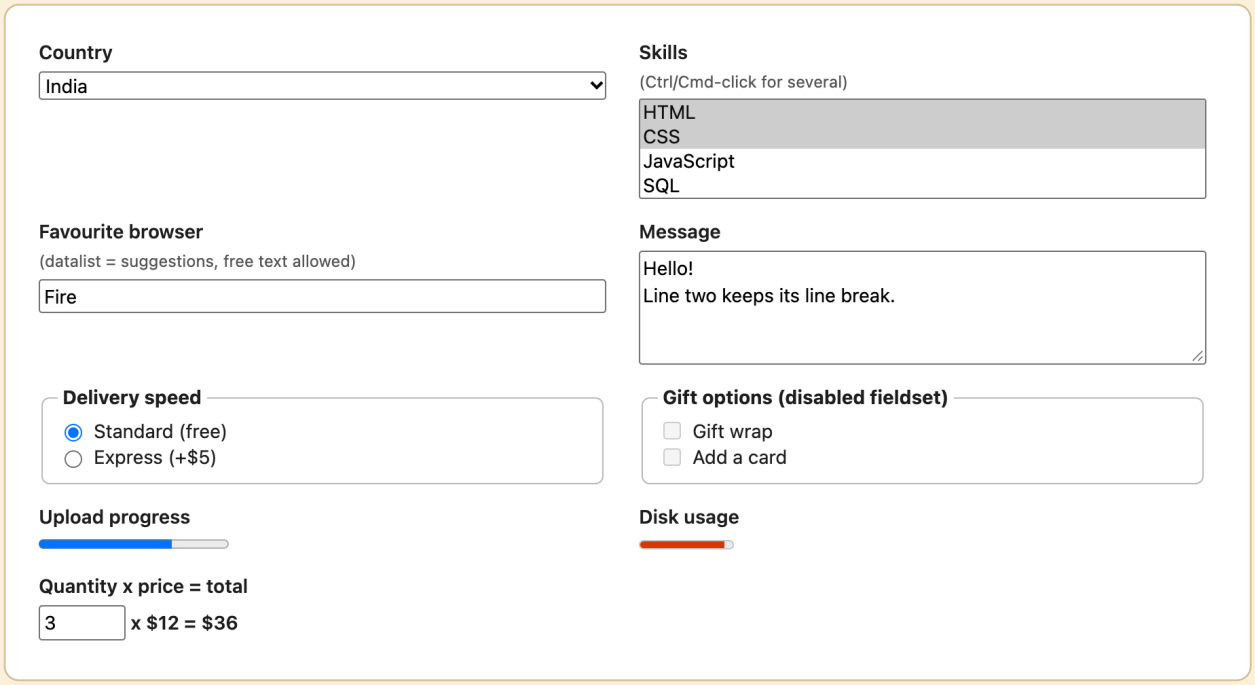


Figure 5.4 — The disabled fieldset disables every control inside; the meter is red because 91 is above `high` and far from `optimum`.

Element	What it is	Key details
`<select>`	dropdown of fixed options	Each `<option>` has a `value` (sent) and text (shown). A first option `value=""` + `required` forces a real choice. `<optgroup label>` groups options.
`<select multiple>`	list box, several choices	Hard to use (Ctrl/Cmd-click); for more than a few options, checkboxes are friendlier.
`<datalist>`	**suggestions** for an `<input list>`	User may still type anything — autocomplete, not restriction (a select restricts).
`<textarea>`	multi-line text	Initial value is its **content** (between the tags, not a `value` attribute); `rows`/`cols`; `maxlength`; line breaks preserved. CSS `resize` controls the drag handle.
`<fieldset>` + `<legend>`	groups related controls	Legend announced with each control ("Delivery speed, Standard, radio button"). Use for every radio and checkbox group (ARIA alternative: `role="radiogroup"`/`"group"` + `aria-labelledby`). `disabled` on the fieldset disables (and unsubmits) everything inside.
`<output>`	result of a calculation	`for` lists the inputs it depends on; an implicit live region, so updates can be announced. Not submitted.
`<progress>`	completion of a task	`value`/`max`; omit `value` for an indeterminate ("busy") bar.
`<meter>`	a measurement within a known range	`min`, `max`, `low`, `high`, `optimum` set the colour (good/average/bad). Disk usage, ratings, password strength — not task progress.

Common mistakes

- Radios without `<fieldset>`/`<legend>` → the question is never announced.
- Radios of one group with different `name`s → they don't exclude each other.
- `<select>` for 2–3 options (radios show all choices, one click fewer), or for 200 countries without type-ahead (a **combobox** — text box + filtered suggestions — may be better).
- `<progress>` for a static measurement (use `<meter>`).

5.5 Built-in validation and the Constraint Validation API

What it is

Browsers validate forms with no JavaScript: you declare **constraints** with attributes; on submit the browser checks them, blocks submission on failure, shows a message by the first failing control, and exposes the result to CSS and JS.

The constraint attributes → the validity flag they set

Attribute	Applies to	Fails when	validity flag
`required`	most inputs, select, textarea	empty / unchecked / nothing selected	`valueMissing`
`type="email"`, `type="url"`	input	not shaped like an email/URL	`typeMismatch`
`pattern="regex"`	text-like inputs	doesn't match the **whole** regex (regular expression; implicitly anchored `^…$`)	`patternMismatch`
`minlength` / `maxlength`	text-like, textarea	shorter / longer — **only checked after the user edits the value**	`tooShort` / `tooLong`
`min` / `max`	number, range, date/time	below / above	`rangeUnderflow` / `rangeOverflow`
`step`	number, range, date/time	not a multiple of `step` from the base (`min`, else initial `value`, else 0)	`stepMismatch`
(the browser)	number, date…	unparsable input	`badInput`
`setCustomValidity("msg")`	any	message non-empty	`customError`

Empty optional fields skip every check except `required` — an empty `pattern` field is valid. Controls that are `disabled`, `readonly`, `type="hidden"`, or inside a `<datalist>` are **barred from constraint validation** (`willValidate === false`) — so a required field in `<fieldset disabled>` never blocks submit.

Styling states: `:invalid` vs `:user-invalid`

`:invalid`/`:valid` (pseudo-classes — selectors matching a state) apply *immediately*, so required fields load red — hostile. `:user-invalid`/`:user-valid` match only **after the user interacts with the field or tries to submit**. Baseline 2023 (all major engines since November 2023; widely available since May 2026).

```
input:user-invalid { border-color: #c0392b; background: #fdeeee; }
input:user-valid   { border-color: #1e8449; }
```

The JavaScript API

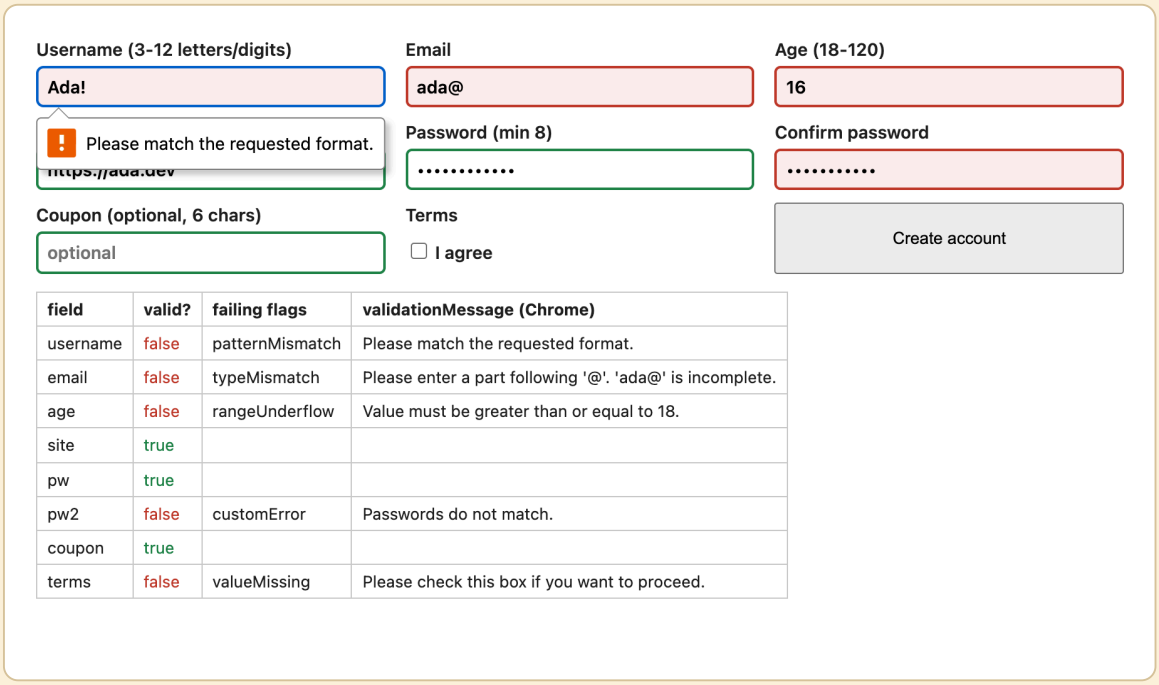
Each control has:

- `validity` — a `ValidityState` with the flags above plus `valid`.
- `validationMessage` — the browser's localised message (empty when valid).
- `willValidate` — whether it takes part in validation.
- `checkValidity()` — returns a boolean; fires `invalid` on failure.
- `reportValidity()` — same, plus shows the message bubble.
- `setCustomValidity(msg)` — your own rule; **clear it with** `''` when valid, or the field stays invalid forever.

Forms have `checkValidity()`, `reportValidity()` and `requestSubmit()`.

Chrome demo: a sign-up form with bad values, one custom rule (passwords must match), then `form.requestSubmit()` (exactly like clicking submit):

```html
<!-- code/05_validation.html (excerpt) -->
<input name="username" required pattern="[a-z0-9]{3,12}" value="Ada!">
<input name="email" type="email" required value="ada@">
<input name="age" type="number" min="18" max="120" value="16">
<input name="site" type="url" value="https://ada.dev">
<input name="pw" type="password" required minlength="8" value="correcthorse">
<input name="pw2" type="password" required value="correcthors">
<input name="coupon" pattern=".{6}" placeholder="optional">
<input name="terms" type="checkbox" required>
<script>
  function checkMatch() {
    form.pw2.setCustomValidity(form.pw2.value === form.pw.value ? '' : 'Passwords do not match.');
  }
  form.pw.addEventListener('input', checkMatch);
  form.pw2.addEventListener('input', checkMatch);
  checkMatch();
  form.requestSubmit();
</script>
```



field	valid?	failing flags	validationMessage (Chrome)
username	false	patternMismatch	Please match the requested format.
email	false	typeMismatch	Please enter a part following '@'. 'ada@' is incomplete.
age	false	rangeUnderflow	Value must be greater than or equal to 18.
site	true		
pw	true		
pw2	false	customError	Passwords do not match.
coupon	true		
terms	false	valueMissing	Please check this box if you want to proceed.

Figure 5.5 — Chrome blocked submission, focused the first invalid field and showed its bubble; after the attempt, `:user-invalid` (red) and `:user-valid` (green) apply. The empty optional coupon is valid.

The same API under jsdom (`code/constraint_api.mjs`) — seven inputs, then `d` is fixed, given a custom error and cleared, and `form.checkValidity()` is called with a capturing `invalid` listener (`invalid` doesn't bubble):

```html
<input id="a" required>
<input id="b" type="email" value="not-an-email">
<input id="c" type="number" min="1" max="5" value="9">
<input id="d" pattern="[A-Z]{3}" value="abc">
<input id="e" pattern="[A-Z]{3}" value="">
<input id="f" required disabled>
<input id="g" minlength="5" value="ab">
```

Output:

```
a: willValidate=true valid=false valueMissing
b: willValidate=true valid=false typeMismatch
c: willValidate=true valid=false rangeOverflow
d: willValidate=true valid=false patternMismatch
e: willValidate=true valid=true -
f: willValidate=false valid=true -
g: willValidate=true valid=true -

after d.value = "ABC":           d: willValidate=true valid=true -
after setCustomValidity("..."):  d: willValidate=true valid=false customError | That code is already taken.
after setCustomValidity(""):     d: willValidate=true valid=true -

form.checkValidity() = false | invalid events fired: 3
```

1. `e` — empty optional field with `pattern` is **valid**.
2. `f` — disabled required field is barred (`willValidate=false`); Chrome agrees.
3. `g` — `minlength` isn't enforced on a value set in HTML or by script; `tooShort` appears only after the user types.

Trade-offs: built-in bubbles vs custom messages

	Built-in UI	`novalidate` + your own messages (5.7)
Effort	zero	moderate
Message wording	generic, browser-controlled, browser-language	your words, specific ("Postcode must be 5 digits")
Styling	bubble can't be styled	full control
Shows all errors at once	no — one at a time	yes (error summary)
Accessibility	acceptable; bubbles vanish quickly	best, if done as in 5.7

Most production design systems (a company's shared UI component library) use `novalidate` with custom messages, **but keep the constraint attributes and API underneath** (`checkValidity()` still works with `novalidate`).

Common mistakes

- Not resetting `setCustomValidity('')` → never valid.
- `pattern` duplicating `type="email"`, or too strict for real names ("O'Brien", "José", single-letter surnames).
- Client validation as security; styling `:invalid` so forms load red.

In the interview

30-second answer: "HTML has declarative constraints — `required`, `type`, `pattern`, `min/max/step`, `minlength/maxlength` — checked on submit and exposed through `validity` flags, `validationMessage`, `checkValidity()` / `reportValidity()`, and `setCustomValidity()` for custom rules. I style with `:user-invalid` so errors show only after interaction. Client-side validation is UX; the server must re-validate."

- *`checkValidity` vs `reportValidity`?* → Both return a boolean and fire `invalid`; `reportValidity` also shows the message.
- *"Confirm password"?* → On input, `setCustomValidity(match ? '' : 'Passwords do not match.')`.

5.6 `autocomplete` — help the browser fill the form

What it is

`autocomplete` tells browsers and password managers **what a field means** via standard tokens so they fill it correctly. WCAG 2.2 SC 1.3.5 *Identify Input Purpose* (AA; Chapter 3) requires it on fields collecting information about the user.

Field	Token
Full name / given / family	`name`, `given-name`, `family-name`
Email, phone	`email`, `tel`
Username	`username`
Login password	`current-password`
Sign-up / change password	`new-password` (triggers strong-password suggestions)
SMS one-time code	`one-time-code`
Address	`street-address`, `address-line1`, `address-level2` (city), `postal-code`, `country` / `country-name`
Card	`cc-name`, `cc-number`, `cc-exp`, `cc-csc` (expiry, 3–4 digit security code)
Birthday	`bday`
Turn off (e.g. searching other people's names)	`off`

Tokens can take a section prefix (`shipping street-address`, `billing postal-code`).

Why it matters

Autofill turns a 2-minute checkout into seconds, reduces typos, and helps people with motor or memory disabilities. Password managers rely on `current-password` vs `new-password` to tell sign-in from sign-up (offering to generate/save). WCAG 2.2's 3.3.7 *Redundant Entry* (don't make people retype information from the same process) is helped by autofill and "same as shipping" checkboxes.

Common mistakes

- `autocomplete="off"` on logins "for security" — pushes users to weaker, reused passwords; browsers often ignore it for passwords anyway.
- Splitting phones or dates into many boxes autofill can't fill.
- Blocking paste in password fields (hurts password managers; against WCAG 2.2's 3.3.8 *Accessible Authentication*).

5.7 Accessible error messages

The problem it solves

Built-in bubbles appear one at a time and vanish; colour alone fails colour-blind users; an error far from its field isn't announced on focus. WCAG requires errors **identified in text** (3.3.1), **labels or instructions** (3.3.2), and **suggested** fixes when known (3.3.3).

How it works — the pattern

Popularised by the UK government's GOV.UK Design System:

1. `novalidate` on the form so you control the messages; still use constraint attributes and `checkValidity()`.
2. On submit, check every field. For each failure:
   - `aria-invalid="true"` (screen readers say "invalid entry");
   - a specific message in an element next to the field;
   - linked via `aria-describedby="help-id error-id"`, so it's read after the label on focus.
3. An **error summary** at the top: heading with the count + links to each field (`href="#fieldId"`).
4. **Move focus to the summary** (`tabindex="-1"`, focusable by script but not in the Tab order) so it's announced immediately.
5. Not colour alone: an icon or the word "Error", red text at sufficient contrast.
6. Remove `aria-invalid` and the message as soon as the field is fixed; keep the user's input.

```
<!-- code/06_accessible_errors.html (excerpt) -->
<form id="f" novalidate>
  <div id="summary" tabindex="-1" hidden>
    <h2>There are 2 problems with your details</h2>
    <ul></ul>
  </div>
  <div class="field">
    <label for="email">Email address</label>
    <span class="help" id="email-help">We'll send the receipt here.</span>
    <input id="email" name="email" type="email" required autocomplete="email"
           aria-describedby="email-help email-err" value="ada@">
    <span class="error" id="email-err"></span>
  </div>
  <div class="field">
    <label for="zip">Postcode <span class="help">(required)</span></label>
    <input id="zip" name="zip" required autocomplete="postal-code" inputmode="numeric"
           pattern="[0-9]{5}" aria-describedby="zip-err" value="">
    <span class="error" id="zip-err"></span>
  </div>
  …
</form>
<script>
  const messages = {
    email: v => v.valueMissing ? 'Enter your email address.' : 'Enter an email address like name@example.com.',
    zip:   v => v.valueMissing ? 'Enter your postcode.' : 'Postcode must be 5 digits.',
    name:  v => 'Enter the name shown on your card.',
  };
  form.addEventListener('submit', e => {
    e.preventDefault();
    const problems = [];
    for (const input of form.querySelectorAll('input')) {
      const err = document.getElementById(input.id + '-err');
      if (input.checkValidity()) { input.removeAttribute('aria-invalid'); err.textContent = ''; }
      else {
        const msg = messages[input.name](input.validity);
        input.setAttribute('aria-invalid', 'true');
        err.textContent = msg;
        problems.push(`<li><a href="#${input.id}">${msg}</a></li>`);
      }
    }
    summary.hidden = problems.length === 0;
    summary.querySelector('ul').innerHTML = problems.join('');
    if (problems.length) summary.focus();
  });
</script>
```

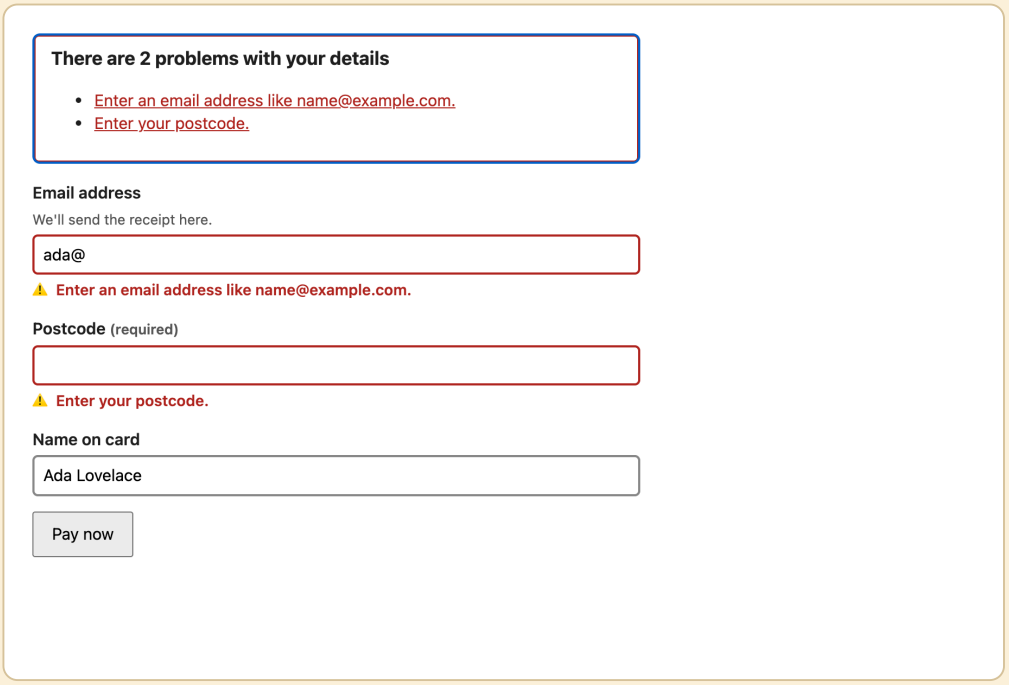


Figure 5.6 — After one failed submit: summary focused, each error has icon + text by its field, messages say how to fix.

Good error messages

Say what's wrong **and how to fix it** ("Postcode must be 5 digits", not "Invalid input"), in the label's words, without codes or blame. Validate on submit (optionally on `blur` — field loses focus — or, after the first submit, on `input`), not on every keystroke while typing.

Common mistakes

- Error text not programmatically associated (no `aria-describedby`); only a red border.
- Clearing the whole form on error.
- Disabling submit until valid — users can't discover *why*. Let them submit and explain.

In the interview

> **30-second answer:** "I use `novalidate` with the constraint API; on submit I mark each bad field with `aria-invalid`, show a specific text message beside it linked via `aria-describedby`, render an error summary with links at the top and move focus to it. Never colour alone, and keep the user's input."

Chapter TL;DR

- Tables are for data: `caption`, `thead`/`tbody`/`tfoot`, `th scope`, `headers` for complex cases; never for layout; scroll wide tables in a focusable region.
- A form sends **named, enabled, checked** controls to `action` via `method`; GET for reads, POST for changes, multipart for files.
- Every control needs a real `<label>`; group radios/checkboxes with `fieldset` + `legend`; buttons default to submit.
- Most specific `type`; `inputmode` for keyboards; `number` only for numeric quantities.
- Built-in validation: `required`, `type`, `pattern`, `min/max/step`, `minlength/maxlength` → `validity`; `setCustomValidity` (and clear it); `:user-invalid` for styling.
- `autocomplete` tokens on personal-data fields; don't block paste or autofill.
- Accessible errors: `aria-invalid`, text messages via `aria-describedby`, focused error summary, never colour alone — and always validate on the server.

Next: Chapter 6 — CSS Fundamentals.

Chapter 6 — CSS Fundamentals

HTML says *what* things are; **CSS (Cascading Style Sheets)** says how they look: colours, fonts, spacing, layout, animation. This chapter covers the machinery every later chapter relies on — how rules are written and attached, how **selectors** (patterns naming the elements a rule applies to) pick elements, and above all how the browser decides which competing rule wins. Most "CSS is broken" moments are misunderstandings of **specificity**, **the cascade** and **inheritance**; after this chapter you should predict any conflict's winner on paper — and prove it with the scripts here.

In this chapter: 6.1 Syntax and the three ways to add CSS · 6.2 Selectors (all 7 attribute forms, combinators) · 6.3 Pseudo-classes and pseudo-elements · 6.4 Specificity (by hand and by program) · 6.5 The cascade: origins, `!important`, inline, layers, order · 6.6 Inheritance and `inherit`/`initial`/`unset`/`revert` · 6.7 Units · 6.8 Colours · 6.9 Custom properties.

All examples are in `code/`. HTML pages were screenshotted in headless Chrome (900 CSS px wide); several print the values Chrome computed (via `getComputedStyle`, which returns an element's final style values). `specificity.mjs` and `cascade.mjs` are plain Node programs.

6.1 Syntax and adding CSS to a page

What it is

A stylesheet is a list of **rules**: a **selector** (which elements) + a **declaration block** (what to do). Each **declaration** is **property**: **value**;.

```
/*  selector      declaration block                    */
   h1          {  color: navy;   font-size: 2rem;  }
/*                property value  property  value      */
```

- **Comments** are `/* … */` (no `//` comments).
- **At-rules** start with `@`: `@media` (conditions), `@import` (load another sheet), `@font-face` (load a font), `@keyframes` (define an animation), `@layer`, `@supports` ("only if the browser supports this"), `@container` (conditions on a container's size).
- CSS is forgiving: an **invalid declaration is ignored** (not the whole rule), while an unknown selector in a list invalidates the whole rule. Skipping what it doesn't understand is what makes **progressive enhancement** work (a baseline for all browsers, extras newer ones use).
- Property names and keywords are case-insensitive; class names, IDs and attribute values are case-sensitive in HTML.

Three ways to add CSS

Method	How	Use for	Drawbacks
External	`<link rel="stylesheet" href="styles.css">` in `<head>`	real sites	one extra request (but cached and shared by every page)
Internal	`<style>…</style>` in `<head>`	single pages, demos, critical above-the-fold CSS (styles for what's visible before scrolling)	not cached separately, repeated per page
Inline	`style="color: red"` on an element	one-off values set by JavaScript, emails	beats selectors (hard to override), no media queries (Chapter 10) or pseudo-classes (§6.3), clutters HTML

`@import url(other.css);` works, but the browser discovers the second file only after downloading the first — a slow chain. Prefer several `<link>`s or a build tool that bundles files.

Stylesheets in `<head>` are **render-blocking**: no paint until they arrive — preventing a flash of unstyled content, but large CSS delays first paint (Chapter 14).

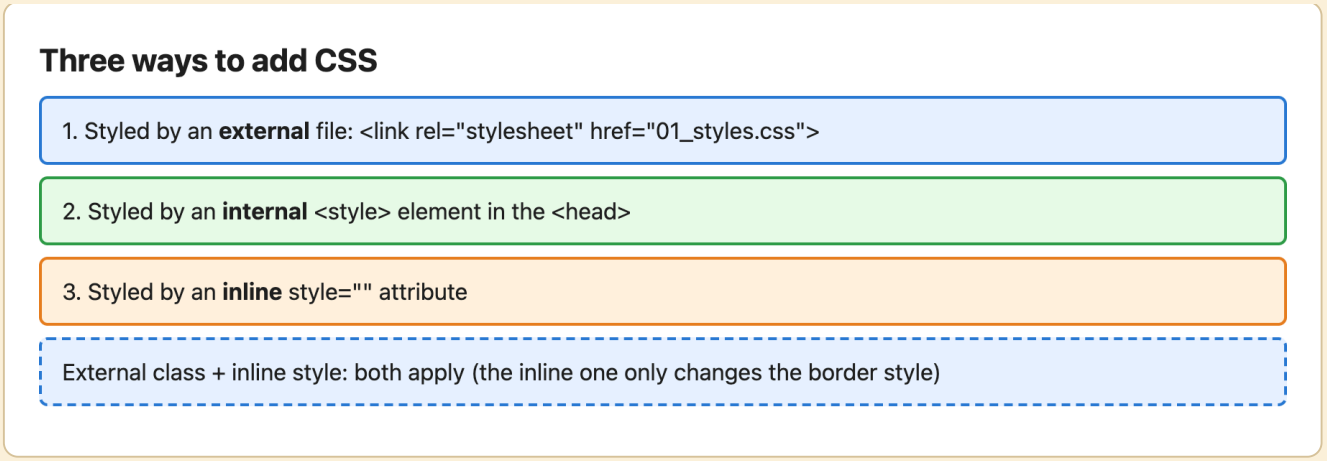


Figure 6.1 — `code/01_adding_css.html`: an external `01_styles.css`, an internal `<style>`, an inline `style`, and a box with an external class plus inline `border-style: dashed`. Styles from all sources combine; the last box takes colours from the external file and only its border style from the inline attribute.

6.2 Selectors

What it is

A **selector** is a pattern matched against elements in the DOM (Chapter 1) to decide where declarations apply.

Basic selectors

Selector	Example	Matches
Type (element)	`p`	every `<p>`
Class	`.card`	every element whose `class` list contains `card`
ID	`#main`	the element with `id="main"` (unique per page)
Universal	`*`	every element
Compound (no space)	`button.primary`	a `<button>` that also has class `primary`
Selector list	`h1, h2, h3`	any of them (one invalid selector drops the whole rule, except inside the "forgiving" `:is()` / `:where()`)

Attribute selectors — all seven forms

Selector	Matches when the attribute…	Example use
`[attr]`	exists (any value)	`a[title]`, `input[required]`
`[attr="v"]`	equals `v` exactly	`input[type="email"]`
`[attr~="v"]`	is a space-separated list containing the word `v`	`[data-tags~="new"]`
`[attr|="v"]`	is exactly `v` or starts with `v-`	`[lang|="en"]` matches `en`, `en-GB`
`[attr^="v"]`	starts with `v`	`a[href^="https"]`
`[attr$="v"]`	ends with `v`	`a[href$=".pdf"]`
`[attr*="v"]`	contains `v` anywhere	`a[href*="example"]`

Add `i` before `]` for case-insensitive comparison: `a[href$=".pdf" i]` also matches `CV.PDF`.

Combinators — relationships between elements

Combinator	Example	Matches
Descendant (space)	`nav a`	an `a` anywhere inside a `nav`, at any depth
Child `>`	`ul > li`	an `li` that is a **direct** child of a `ul`
Next sibling `+`	`h2 + p`	the `p` **immediately** after an `h2` (same parent)
Subsequent siblings `~`	`h2 ~ p`	**every** `p` after an `h2` (same parent)

So for a nested menu, `.menu li` matches items at every level, `.menu > li` only top-level ones. Browsers match **right to left**: for `nav a` they find every `a`, then look for a `nav` ancestor — so the rightmost part (the **key selector**) matters most for performance, though selector speed is rarely a real problem.

Figure 6.2 — `code/02_selectors.html`: one yellow highlight per selector. "div span" matches child and grandchild, "div > span" only the direct child; "h3 + p" one paragraph, "h3 ~ p" every later paragraph (skipping the span, never the one before).

Common mistakes

- `.nav .active` (descendant) when you meant `.nav.active` (same element, both classes).
- Styling by ID — extremely specific (6.4), painful to override.
- Long chains like `body div.container ul li a` — fragile and highly specific; prefer one meaningful class.
- Forgetting `+` and `~` only look at *later* siblings with the *same parent*.

6.3 Pseudo-classes and pseudo-elements

What they are

- A **pseudo-class** (one colon, `:hover`) selects elements in a **state or position** not expressed by an attribute — hovered, first child, checked.
- A **pseudo-element** (two colons, `::before`) selects a **part** of an element or creates a box not in the HTML — first letter, list bullet, generated content.

Legacy single-colon `:before`, `:after`, `:first-line`, `:first-letter` still work; new code uses `::`.

Pseudo-classes by family

Family	Pseudo-classes	Notes
User action	`:hover`, `:active`, `:focus`, `:focus-visible`, `:focus-within`	`:focus-visible` matches only when the browser thinks a focus ring is needed (keyboard use), so you can drop rings for mouse clicks without hurting keyboard users. Link order: `:link`, `:visited`, `:hover`, `:active` ("LVHA").
Links	`:link`, `:visited`, `:any-link`, `:target`	`:visited` is restricted to a few colour properties for privacy. `:target` = the element whose id is the URL `#fragment`.
Structural	`:first-child`, `:last-child`, `:only-child`, `:nth-child(An+B)`, `:nth-last-child()`, `:first-of-type`, `:nth-of-type()`, `:only-of-type`, `:empty`, `:root`	`An+B`: `odd` = `2n+1`, `even` = `2n`, `3n` every third, `3n+1` every third from the first, `-n+3` first three. `:nth-child(2 of .item)` counts only `.item` elements.
Form state	`:checked`, `:disabled`, `:enabled`, `:required`, `:optional`, `:valid`, `:invalid`, `:user-valid`, `:user-invalid`, `:in-range`, `:out-of-range`, `:placeholder-shown`, `:read-only`, `:default`, `:indeterminate`	See Chapter 5.
Logical	`:not(list)`, `:is(list)`, `:where(list)`, `:has(relative list)`	`:is()`/`:where()` shorten lists (`:is(h1,h2,h3) a`) and differ only in specificity (6.4). `:has()` matches an element **by what it contains or what follows it** — the "parent selector" (Baseline since December 2023, widely available since June 2026); e.g. `.group:has(:user-invalid)`.
Other	`:lang(en)`, `:dir(rtl)`, `:defined`, `:fullscreen`, `:modal`, `:popover-open`	

`li:nth-child(2)` = "an `li` that is its parent's 2nd child" (fails if the 2nd child isn't an `li`); `span:nth-of-type(2)` = "the 2nd `span` among its siblings", ignoring other types.

Pseudo-elements

Pseudo-element	Selects / creates
`::before`, `::after`	a generated child box at the start/end of the content. **Only exists if `content` is set** (`content: ""` for a decorative box). Replaced elements (content from outside CSS, e.g. `<img>` and most `<input>` types) don't render them — there's nowhere to insert the box.
`::first-letter`	first letter of a block (drop caps)
`::first-line`	the first **rendered** line — changes with width
`::marker`	a list item's bullet or number
`::placeholder`	an input's placeholder text
`::selection`	text the user has highlighted
`::file-selector-button`	the button of `<input type="file">`
`::backdrop`	the layer behind a modal `<dialog>` or full-screen element

Generated content (`::before`/`::after`) is decoration: screen readers may or may not read it and users can't select it — never put essential information there.

```css
/* code/03_pseudo.html (excerpt) */
#p-odd li:nth-child(odd)            { background: #ffe066; }
#p-of  li:nth-child(2 of .item)     { background: #ffe066; }
#p-not li:not(.skip)                { background: #ffe066; }
#p-has .card:has(img)               { background: #ffe066; }
#p-checked input:checked + label    { background: #ffe066; }

.quote::before { content: "\201C"; }            /* opening curly quote */
.quote::after  { content: "\201D"; }
.required::after { content: " *"; color: #c0392b; }
a.ext[href^="http"]::after { content: " \2197"; }  /* arrow after external links */
.dropcap::first-letter { font-size: 2.6em; float: left; line-height: 1; }
.firstline::first-line { font-weight: 700; }
ol.fancy li::marker { color: #e67e22; }
input.ph::placeholder { color: #9b59b6; font-style: italic; }
.sel::selection { background: #2d2a5a; color: #fff; }
```

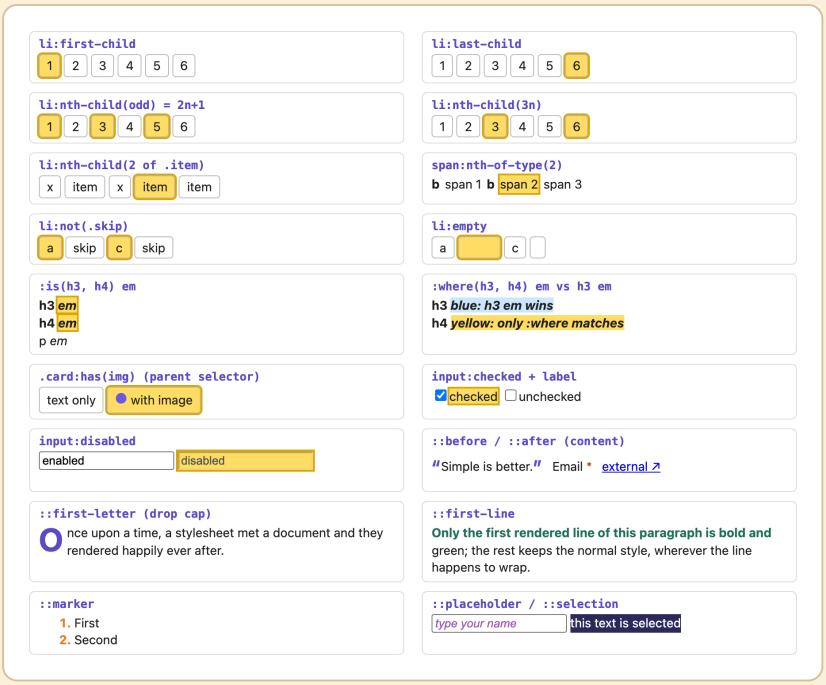


*Figure 6.3 — `code/03_pseudo.html`. In the `:empty` row the `<li>` containing only a space is **not** matched — whitespace counts as content in current browsers.*

Common mistakes

- `::before` with no `content` → nothing appears.
- `li:first-child` when the first child is a `<div>` → no match (use `:first-of-type`).
- `:focus { outline: none }` → keyboard users are lost; style `:focus-visible`.
- Real text (prices, labels) in `content:`.

In the interview

30-second answer: "Pseudo-classes select by state or position — `:hover`, `:focus-visible`, `:nth-child`, `:checked`, `:has()`. Pseudo-elements style a part or generate content — `::before`/`::after` need `content`, plus `::first-letter`, `::marker`, `::placeholder`, `::selection`. `:has()` lets a parent be styled by its contents (`.card:has(img)`, no JS), and `:is()`/`:where()` group selectors, differing only in specificity."

6.4 Specificity

What it is

When rules from the same origin and layer (§6.5) set the **same property on the same element**, the browser compares their selectors' **specificity** — how *targeted* each is. The more specific wins regardless of source order; only on a tie does the later rule win.

How it works — the score (A, B, C)

Column	Counts	Examples
A	ID selectors	`#main`
B	class selectors, attribute selectors, pseudo-classes	`.btn`, `[type="text"]`, `:hover`, `:nth-child(2)`
C	type selectors and pseudo-elements	`p`, `li`, `::before`
(nothing)	universal `*`, combinators ( ), `>`, `+`, `~`), `:where(…)`	

1. `:is(…)`, `:not(…)` and `:has(…)` add their **most specific argument** — even if a less specific argument matched.
2. `:where(…)` always adds **zero** — ideal for library defaults and resets.
3. `:nth-child(An+B of S)` = one pseudo-class **plus** the most specific selector in `S`.
4. Compare A, then B, then C. **No carrying** (like a three-digit score where digits never overflow): `(0,11,0)` loses to `(1,0,0)`.
5. Inline `style=""` isn't scored — it beats any selector; `!important` beats both (6.5).
6. Repeating a class (`.btn.btn`, (0,2,0)) really adds specificity — a deliberate override hack.

Worked scoring table

Selector	A (IDs)	B (classes, attrs, pseudo-classes)	C (types, pseudo-elements)	Score
`*`	–	–	–	(0,0,0)
`li`	–	–	`li`	(0,0,1)
`ul > li + li`	–	–	`ul` `li` `li`	(0,0,3)
`.nav`	–	`.nav`	–	(0,1,0)
`a:hover`	–	`:hover`	`a`	(0,1,1)
`input[type="email"]`	–	`[type="email"]`	`input`	(0,1,1)
`p::first-line`	–	–	`p` `::first-line`	(0,0,2)
`.nav a.active`	–	`.nav` `.active`	`a`	(0,2,1)
`#main .card p`	`#main`	`.card`	`p`	(1,1,1)
`#main #sidebar`	`#main` `#sidebar`	–	–	(2,0,0)
`:is(#a, .b, p) span`	`#a` (max arg)	–	`span`	(1,0,1)
`:where(#a, .b, p) span`	–	–	`span`	(0,0,1)
`button:not(.primary, [disabled])`	–	`.primary` (max arg — tie, either counts 1)	`button`	(0,1,1)
`.card:has(> img)`	–	`.card`	`img` (from the argument)	(0,1,1)
`li:nth-child(2n+1 of .item)`	–	`:nth-child` + `.item`	`li`	(0,2,1)
`html body div#app main.content article.post > p:first-child::first-letter`	`#app`	`.content` `.post` `:first-child`	`html` `body` `div` `main` `article` `p` `::first-letter`	(1,3,7)

The calculator

`code/specificity_lib.mjs` implements these rules in ~70 lines (a scanner walking the selector character by character, recursing into `:is()`/`:not()`/`:has()`; `compare = (x, y) => x[0] - y[0] || x[1] - y[1] || x[2] - y[2]`). `node specificity.mjs` scores every selector in the table above (plus `ul li` (0,0,2), legacy `p:first-line` (0,0,2), `#main` (1,0,0)) with no `MISMATCH` against the hand values, then ranks rules for one link:

```
Winner order for <nav id="top" class="nav"><a class="active">…</a></nav> (highest first):
  1. #top a                   (1,0,1)
  2. a:is(.active, #x)        (1,0,1)
  3. nav a.active             (0,1,2)
  4. .nav a                   (0,1,1)
  5. :where(#top) a.active    (0,1,1)
  6. a                        (0,0,1)
```

The two `(1,0,1)` selectors tie, so the **later** one in the stylesheet would win; `a:is(.active, #x)` scores an ID even though the link matched via `.active`.

Seeing it in the browser

Each box in `04_specificity.html` has competing rules; a script prints Chrome's computed colour:

```css
/* code/04_specificity.html (excerpt) */
#c1 p      { color: gray; }    /* (1,0,1) */
#c1 .lead  { color: green; }   /* (1,1,0) */
#c1 #intro { color: blue; }    /* (2,0,0)  <- wins */

#c3 .a.b.c.d.e.f.g.h.i.j.k { color: green; }  /* (1,11,0) */
#c3 #x                     { color: blue; }   /* (2,0,0)  <- wins: A is compared first */

#c6 :is(#nope, .z) { color: blue; }  /* (2,0,0) -- counts #nope although .z matched */
#c6 .z.z.z         { color: green; } /* (1,3,0) */
```

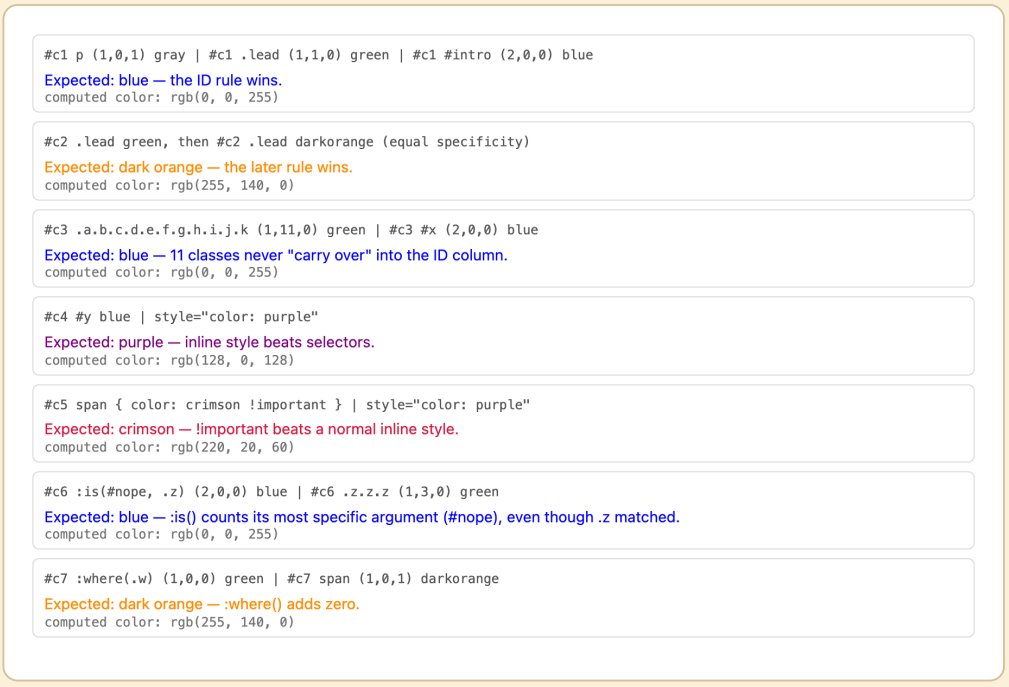


Figure 6.4 — Every prediction confirmed by `getComputedStyle`. The `#c1 …` case prefix adds `(1,0,0)` to every selector in a case without changing who wins.

Trade-offs — keeping specificity low

High specificity forces ever-higher specificity to override it — a "specificity war" ending in `!important` everywhere. Instead:

- Style with **single classes** (`.card-title`), as in BEM (`block__element--modifier` naming) or utility CSS (tiny one-purpose classes like `.mt-4`; Chapter 13).
- Avoid IDs in selectors (keep them for link targets and JS hooks).
- Wrap defaults in `:where()` so anything can override them.
- Use **cascade layers** (`@layer`) to order whole groups regardless of specificity (6.5).

In the interview

30-second answer: "Specificity is a three-part score — IDs, then classes/attributes/pseudo-classes, then types/pseudo-elements — compared column by column with no carrying, so one ID beats any number of

classes. `:is`, `:not` and `:has` take their most specific argument; `:where`, `*` and combinators are zero. Equal scores fall back to source order; inline styles and `!important` sit above specificity in the cascade."

- *Score of* `#nav ul li a:hover`*?* → (1,1,3). `.nav > li:first-child a:hover`*?* → (0,3,2).
- `div div div div p` *vs* `.text`*?* → `.text` (0,1,0) beats (0,0,5).
- `ul#menu > li.item:not(.disabled) a[href]`*?* → A `#menu`; B `.item`, `.disabled`, `[href]`; C `ul`, `li`, `a` → (1,3,3).
- *A reset anything can override?* → `:where(ul, ol) { margin: 0 }` (0,0,0), or an early `@layer`. (`:where(.x) .y` = (0,1,0).)

6.5 The cascade and origins

What it is

The **cascade** is the algorithm that picks one value when several declarations set the same property on the same element. Per CSS Cascading and Inheritance Level 5, criteria are compared **in order**, stopping at the first difference:

1. **Origin and importance** — where the declaration came from, and whether it's `!important`.
2. **Context** — shadow DOM encapsulation (a component's private, sealed-off DOM and styles; beyond this chapter).
3. **Element-attached styles** — a `style=""` attribute beats stylesheet rules.
4. **Cascade layers** — which `@layer` the rule is in.
5. **Specificity** (6.4).
6. **Order of appearance** — later wins.

Origins

- **User-agent** — the browser's built-in stylesheet (why `<h1>` is big and `<a>` blue).
- **User** — set by the person (accessibility settings, extensions).
- **Author** — yours.

Rank	Declarations (strongest first)
1	CSS transitions (Chapter 12), while running
2	`!important` user-agent
3	`!important` user
4	`!important` author
5	CSS animations, while running
6	normal author
7	normal user
8	normal user-agent

The `!important` reversal is deliberate: normally your CSS beats the user's, but a user's `!important` (e.g. large text for low vision) beats you.

Cascade layers (`@layer`)

Layers order **groups** of rules explicitly, so a low-priority group never wins by accident of specificity:

```
@layer reset, base, components;          /* order declared up front: later = stronger */
@layer reset      { #l1 p   { color: gray; } }
@layer components { .card p { color: teal; } }
p.note { color: navy; }                  /* unlayered: stronger than every layer */
```

Normal declarations: later layers beat earlier ones; **unlayered beats all layers**. `!important` flips it: earlier layers win, and layered `!important` beats unlayered `!important`. Baseline since 2022; layer architecture in Chapter 13.

A model of the cascade

`code/cascade.mjs` turns criteria 1, 3, 4, 5 and 6 into a sort key — origin rank (normal ua 1, user 2, author 3; important author 5, user 6, ua 7), inline flag, layer rank (unlayered = after the last layer; reversed for `!important`), specificity, source order — and sorts competing declarations for `<p id="intro" class="lead" style="color: purple">`. The key reads `origin.attached.layer.A.B.C.order`; bigger wins, left to right:

```
A. Specificity decides (same origin, no layers):
  WIN color: blue     #intro                [author]  key=3.0.0.1.0.0.1
      color: green    .lead                 [author]  key=3.0.0.0.1.0.2
      color: black    p                     [author]  key=3.0.0.0.0.1.0

B. Equal specificity -> the later rule wins:
  WIN color: orange   .lead                 [author]  key=3.0.0.0.1.0.1
      color: green    .lead                 [author]  key=3.0.0.0.1.0.0

C. Inline style beats any selector; !important beats inline:
  WIN color: red      p                     [author, !important]  key=5.0.0.0.0.1.2
      color: purple   style=""              [author]  key=3.1.0.0.0.0.1
      color: blue     #intro                [author]  key=3.0.0.1.0.0.0

D. Origins: the browser default loses to author CSS, but user !important beats everything else:
  WIN color: yellow   p                     [user, !important]  key=6.0.0.0.0.1.2
      color: blue     #intro                [author, !important]  key=5.0.0.1.0.0.1
      color: canvastext p                     [ua]  key=1.0.0.0.0.1.0

E. Layers (@layer reset, base, components): unlayered wins even with lower specificity:
  WIN color: navy     p                     [author]  key=3.0.3.0.0.1.2
      color: teal     .lead                 [author, @layer components]  key=3.0.2.0.1.0.1
      color: gray     #intro                [author, @layer reset]  key=3.0.0.1.0.0.0

F. ...but with !important the layer order flips: the earliest layer wins:
  WIN color: gray     p                     [author, !important, @layer reset]  key=5.0.3.0.0.1.0
      color: teal     #intro                [author, !important, @layer components]  key=5.0.1.1.0.0.1
      color: navy     #intro                [author, !important]  key=5.0.0.1.0.0.2
```

Case E is confirmed in the browser in Figure 6.5 (first box: navy).

`!important` — when (not) to use it

Legitimate: utility classes that must always apply (`.hidden { display: none !important }`), overriding third-party inline styles you can't edit, user stylesheets. Elsewhere it signals a specificity problem — and only another `!important` beats it.

Common mistakes

- "The last rule always wins" — only among equal origin, layer and specificity.
- Fighting a library with longer selectors instead of putting it in a lower `@layer`.
- Forgetting the browser's own stylesheet (margins on `body`, `h1`, `ul`…) — hence resets/normalize (normalize.css makes defaults consistent rather than removing them; Chapter 13).

In the interview

30-second answer: "The cascade compares origin and importance first — user-agent, user, author, with `!important` reversing that order — then inline style attributes, then cascade layers (later wins, unlayered beats layered, reversed for `!important`), then specificity, then source order. So specificity only matters within the same origin and layer."

- *Why can't my `#id` rule beat a Bootstrap class?* → It can unless the class uses `!important`; the real cause is often load order or a layer.
- *Where do you put a third-party library?* → `@import url(lib.css) layer(vendor);` or an early layer, so your rules win without specificity tricks.

6.6 Inheritance

What it is

Inherited properties pass their value from parent to child. Most **text** properties inherit (`color`, `font-*`, `line-height`, `letter-spacing`, `text-align`, `visibility`, `cursor`, `list-style`); most **box** properties don't (`margin`, `padding`, `border`, `background`, `width`, `display`, `position`). Each property's spec/MDN entry says "Inherited: yes/no".

Inheritance applies only when **no declaration** matched the element — any matching rule, however weak, beats an inherited value. So the browser's `a { color: blue }` beats `body { color: red }` for links.

The global keywords (valid for every property)

Keyword	Meaning
`inherit`	Use the parent's computed value (post-cascade, units resolved) — even for non-inherited properties.
`initial`	The property's **initial value from the spec** (`display: initial` is `inline`, `color: initial` is `canvastext` — usually black).
`unset`	`inherit` for inherited properties, `initial` for the rest — "as if no rule matched" (so `color: unset` under a red parent is red).
`revert`	Roll back to the **previous origin's** value — for author styles, usually the browser default (`display: revert` on a `<div>` gives `block`, not `inline`).
`revert-layer`	Roll back to the previous cascade layer's value.

`all: unset` (or `all: revert`) applies a keyword to every property — handy to strip a `<button>`'s default look.

```
/* code/05_cascade_inheritance.html (excerpt) */
.parent { color: #b03a2e; font-style: italic; border: 2px solid #b03a2e; padding: 6px; }

.kw .inherit { border: inherit; }            /* copies the parent's border */
.kw .initial { color: initial; }             /* spec default: black */
.kw .unset   { color: unset; border: unset; }/* color inherits, border resets */
.kw .block        { display: block; }
.kw .block.revert { display: revert; }       /* back to the browser default for <b>: inline */

.fonts .fixed { font: inherit; }             /* form controls don't inherit font by default */
```

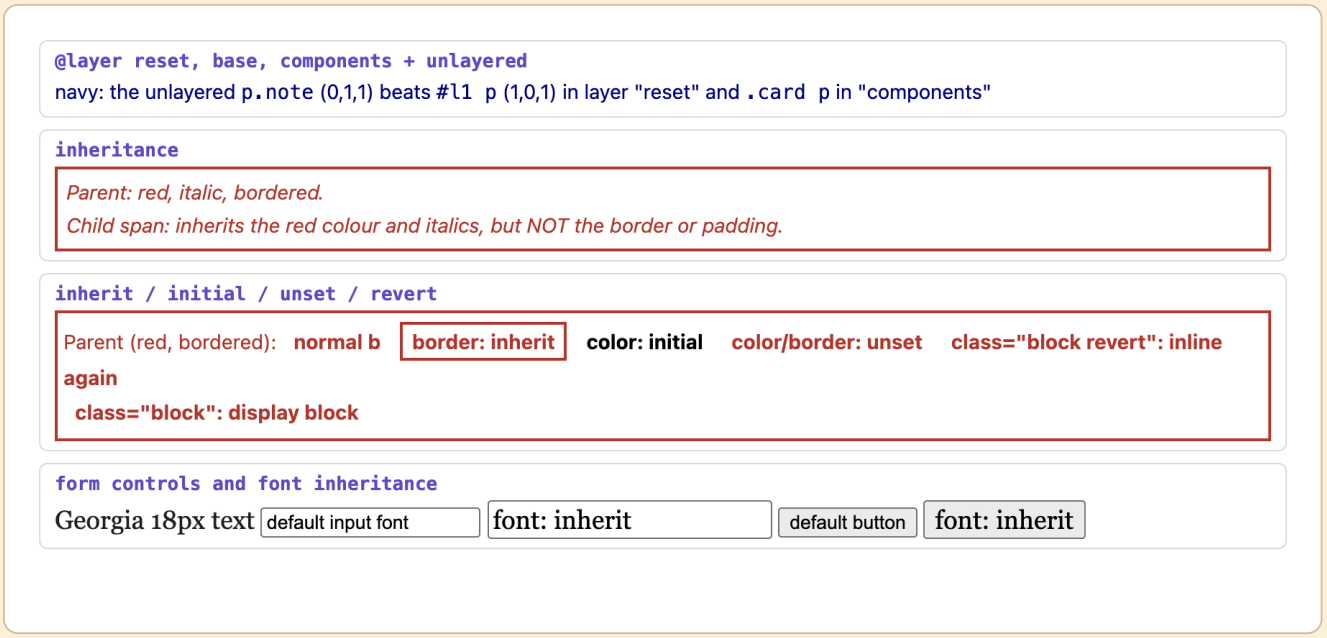


Figure 6.5 — Inheritance and the global keywords. Last row: the browser gives form controls their own font, so they ignore the page font until `font: inherit` — one of the first lines in most resets.

6.7 Units

What it is

Lengths are **absolute** (fixed) or **relative** (to the font, parent or viewport). Relative units make layouts adapt to user settings and screens.

Unit	Relative to	Typical use
`px`	the CSS pixel (1/96 inch at arm's length; not a device pixel)	borders, shadows, small fixed details
`em`	the element's own `font-size` — **except in `font-size` itself, where it's the parent's**	padding/margins scaling with the text, e.g. buttons
`rem`	the root (`<html>`) font-size — 16px by default	font sizes and spacing site-wide
`%`	per property: `width`, `padding`, `margin` (**including top/bottom**) → containing block's (usually the parent's content box, Chapter 7) **width**; `height` → its height; `font-size` → parent font-size	fluid widths
`vw` / `vh`	1% of the viewport width / height	full-screen sections, fluid type
`svh` / `lvh` / `dvh`	small / large / dynamic viewport height (accounts for mobile toolbars; Baseline 2022)	`min-height: 100dvh` hero sections
`vmin` / `vmax`	the smaller / larger of `vw`, `vh`	squares that fit any orientation
`ch`	width of the "0" glyph of the current font	line length, e.g. `max-width: 65ch` (45–75 characters is readable); input widths
`ex`, `lh`, `rlh`, `cqw` …	x-height, line height, container query width (Chapter 10)…	specialised
`cm`, `mm`, `in`, `pt`, `pc`	physical units (1in = 96px)	print stylesheets

```
/* code/06_units.html (excerpt) */
html { font-size: 16px; }                /* 1rem = 16px */
body { font-size: 14px; }
.em  li { font-size: 1.2em; }            /* 1.2 x parent -> compounds when nested */
.rem li { font-size: 1.2rem; }           /* always 1.2 x 16 = 19.2px */
.btn { font-size: 20px; padding: 0.5em 1em; }   /* em = own font-size */
.outer { width: 400px; }
.pct   { width: 50%; padding-top: 10%; }  /* both relative to the parent's WIDTH */
.vw { width: 50vw; }
.measure { max-width: 40ch; }
```

Figure 6.6 — Values computed by Chrome and printed by the page.

1. **em compounding:** body 14px → 14 × 1.2 = **16.8px** → × 1.2 = **20.16px** → × 1.2 = **24.192px**. With `rem` every level is 16 × 1.2 = **19.2px**.
2. **em in padding:** the button's own font-size is 20px → 0.5em = **10px**, 1em = **20px**. (Parent 20px, child `font-size: 1.5em; padding: 1em` → child is 30px, padding 30px.)
3. **%:** 50% of a 400px parent = **200px**; `padding-top: 10%` = 10% of the parent's *width* = **40px**.
4. **vw:** 50vw of a 900px viewport = **450px**.
5. **ch:** 40 × the "0" width in 14px system-ui ≈ **352.7px**.

Why `rem` for font sizes

If a user raises the browser default to 20px, `rem` and `em` sizes scale; `px` ignores it — but only if you **don't set the root font-size in `px`** (the demo does just for exact arithmetic). Common practice: `rem` for type and spacing, `em` for component spacing that tracks its own text, `px` for hairlines (1px borders), `%`/`fr` (grid fraction, Chapter 9)/`vw` for layout.

Common mistakes

- Nested `em` font sizes snowballing or shrinking.
- `height: 100vh` on mobile hiding content behind the toolbar — use `100dvh` (or `svh`).
- `width: 100vw` causing a horizontal scrollbar on desktop (`vw` includes the scrollbar) — use `100%`.
- Expecting `padding-top: 50%` to be half the parent's *height* — it's half the width (the old aspect-ratio-box trick, now replaced by `aspect-ratio`).
- `html { font-size: 62.5% }` tricks — harmless but confusing; plain 1rem = 16px math is fine.

In the interview

30-second answer: "`px` is fixed; `em` is relative to the element's font-size (the parent's when used in font-size, so it compounds); `rem` is relative to the root, respecting user settings without compounding; `%` depends on the property — widths and even vertical padding use the containing block's width; `vw`/`vh` are viewport percentages, with `dvh`/`svh` fixing mobile toolbars; `ch` is handy for line length."

6.8 Colours

What it is

Most notations describe the same **sRGB** colours (the standard range of ordinary screens); newer ones (`oklch()`, `oklab()`, `color()`) reach wider-gamut colours (a *gamut* is the set of colours a screen can show) on modern

displays.

Notation	Example	Notes
Named	`rebeccapurple`, `tomato`	~150 names; plus `transparent` and `currentColor`
Hex	`#663399`, short `#639`, with alpha `#66339980`, `#6398`	two hex digits per channel (00–ff); 4th pair = alpha (opacity) — so 50 % black is `#00000080` (0x80 = 128)
`rgb()`	`rgb(102 51 153)`, `rgb(102 51 153 / 50%)`	modern space-separated syntax with `/ alpha`; legacy `rgba(102, 51, 153, 0.5)` still works (`rgba` is now an alias)
`hsl()`	`hsl(270 50% 40%)`	hue angle (0 red, 120 green, 240 blue), saturation (vivid vs grey), lightness — easy to tweak by hand
`oklch()`	`oklch(44% 0.16 303)`	**L**ightness (0–100%), **C**hroma (colourfulness, ~0–0.4), **H**ue. Perceptually uniform: equal L *looks* equally light across hues. Baseline since 2023.
`color-mix()`	`color-mix(in oklch, #663399 50%, white)`	mixes two colours in a chosen colour space (which changes how the blend looks). Baseline since 2023.
`currentColor`	`border: 2px solid currentColor`	the element's computed `color` — keeps borders, SVG icons and text in sync

```
<!-- code/07_colors.html (excerpt) -->
<div class="sw" style="background: rebeccapurple">rebeccapurple</div>
<div class="sw" style="background: #663399">#663399</div>
<div class="sw" style="background: rgb(102 51 153)">rgb(102 51 153)</div>
<div class="sw" style="background: hsl(270 50% 40%)">hsl(270 50% 40%)</div>
<div class="sw" style="background: oklch(44.03% 0.1603 303.37)">oklch(…)</div>
…
<div class="sw" style="background: hsl(60 100% 50%)">hsl yellow</div>   <!-- … and 4 more hues -->
<div class="sw" style="background: oklch(70% 0.15 100)">oklch yellow</div>
…
<div class="sw" style="background: color-mix(in srgb, red, blue)">srgb red+blue</div>
<div class="sw" style="background: color-mix(in oklch, red, blue)">oklch red+blue</div>
<button class="btn">currentColor border</button>
```

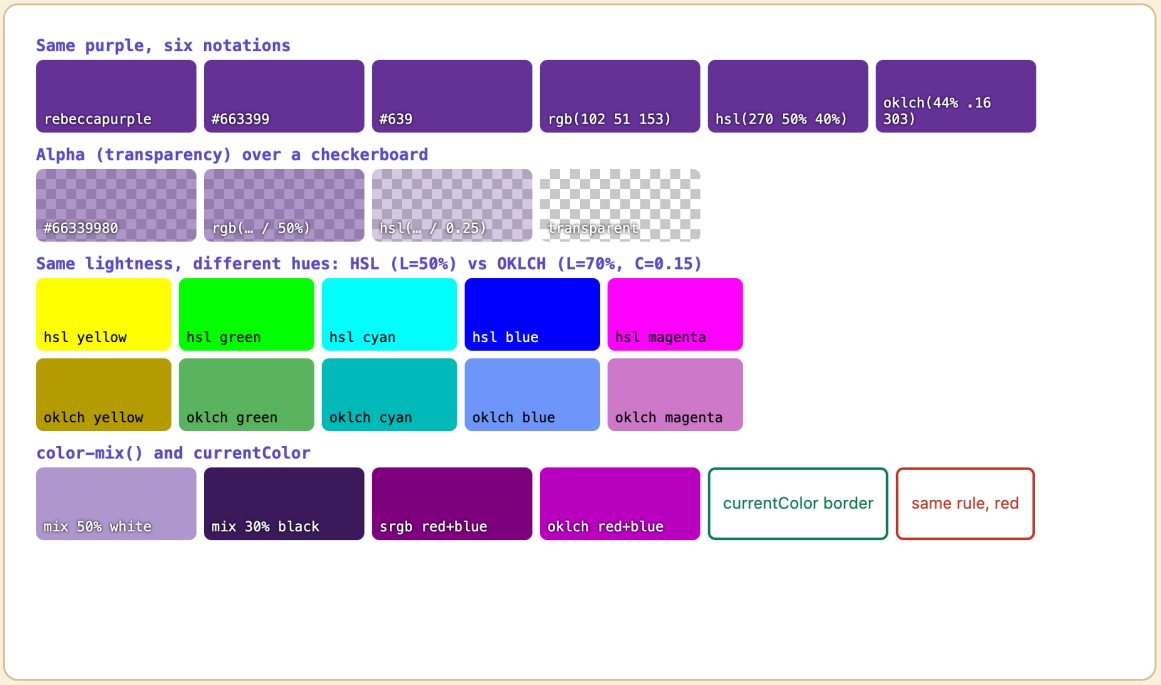


Figure 6.7 — The HSL row claims one lightness (50%) but yellow looks far brighter than blue; the OKLCH row really is evenly light. Mixing red and blue in sRGB gives a darker purple than in OKLCH.

Why OKLCH matters

Design systems generate palettes ("brand-100 … brand-900") by varying lightness. In HSL the same step gives visibly uneven results across hues, so text contrast becomes unpredictable; in OKLCH lightness is perceptual,

so the palette behaves consistently and one number lightens or darkens predictably. Contrast requirements (WCAG) are in Chapter 11.

Common mistakes

- Mixing syntaxes: `rgb(102, 51, 153 / 50%)` is invalid (commas *or* spaces with `/`).
- `opacity: 0.5` when you meant a translucent background — `opacity` fades text and children too; use an alpha colour.
- The same colour hard-coded in dozens of places instead of a custom property (6.9).

6.9 Custom properties (CSS variables)

What it is

A **custom property** is a self-named property starting with `--` (`--brand: #5b4bc4;`), read with `var(--brand)`. Unlike preprocessor variables (Sass's `$brand`, compiled away before publishing), custom properties are **live**: they cascade and inherit, can change per element, per media query or from JavaScript at runtime, and every use updates instantly.

How it works

1. **Declare** on a selector — usually `:root` (the `<html>` element) so it's inherited everywhere.
2. **Use** with `var(--name)` or `var(--name, fallback)`; the fallback applies only if the property is **undefined** (`var(--gap, 1rem)` with `--gap: 2rem` gives `2rem`).
3. They **inherit**, so redefining one on an element changes it for that subtree — the basis of theming (swapping a whole colour set, e.g. light/dark) and component variants.
4. Case-sensitive (`--Brand` ≠ `--brand`); any token sequence is allowed, checked only when substituted.
5. If the substituted value is invalid for the property (`color: var(--space)` where `--space: 12px`), the declaration is **invalid at computed-value time**: it behaves as `unset` — it does **not** fall back to an earlier `color` declaration.
6. JS reads with `getComputedStyle(el).getPropertyValue('--x')`, writes with `el.style.setProperty('--x', value)`.
7. Combine with `calc()`: `padding: calc(var(--space) * 2)`.

```
/* code/08_custom_properties.html (excerpt) */
:root { --brand: #5b4bc4; --radius: 8px; --space: 12px; --surface: #fff; --text: #222; }
[data-theme="dark"] { --surface: #1e1b3a; --text: #eceaff; --brand: #a99cff; }

.panel { background: var(--surface); color: var(--text); padding: var(--space); }
.card  { border: 2px solid var(--brand); border-radius: var(--radius); padding: var(--space); }
.card h3 { color: var(--brand); }
.card.danger   { --brand: #c0392b; }                       /* local override */
.card.fallback { border-color: var(--not-defined, orange); }
.card.big      { padding: calc(var(--space) * 2); }
.card.bad      { color: var(--space); }                    /* invalid -> inherits */
```

```
card.style.setProperty('--brand', '#0a7d55');                    // write
getComputedStyle(document.documentElement).getPropertyValue('--radius'); // read -> "8px"
```

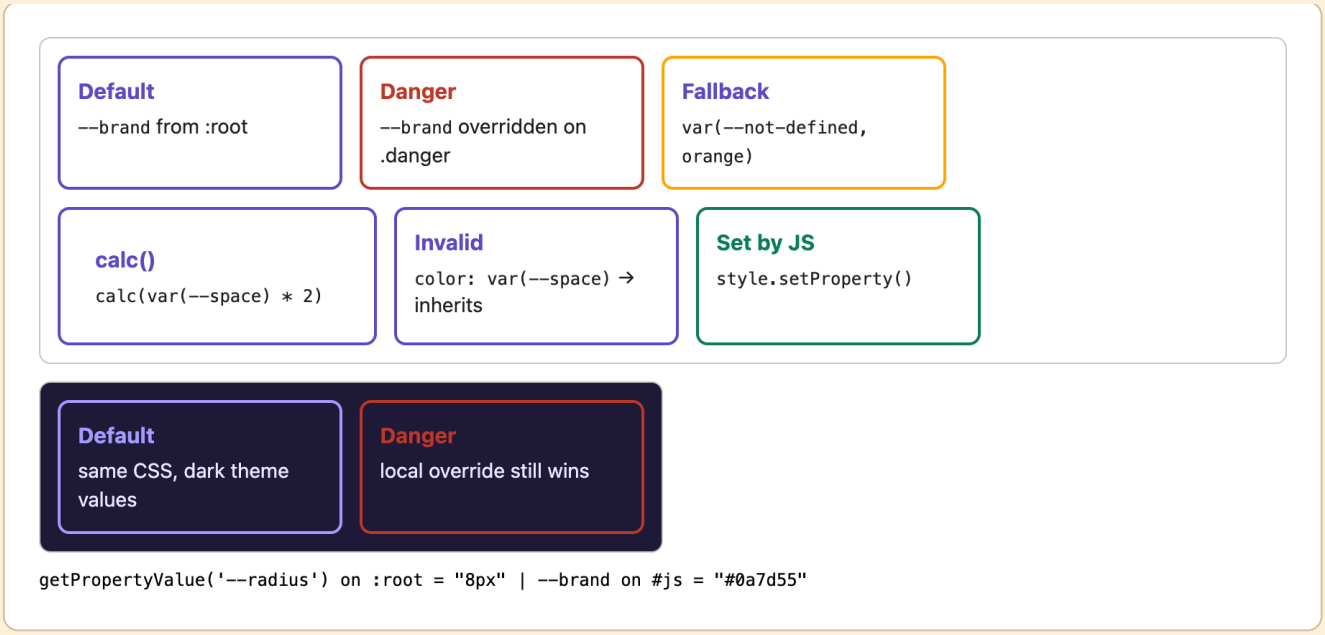


Figure 6.8 — One set of rules, many looks: the dark panel only redefines the variables; the danger card's local `--brand` wins in both themes.

Trade-offs

	Custom properties	Preprocessor variables (Sass)
When resolved	runtime, in the browser	build time — gone from the output CSS
Cascade/inheritance, per-element values	yes	no
Change from JS / media queries	yes	no
Use in selectors or media query conditions	no	yes
Type checking	none by default; `@property` registers a type and initial value (Baseline 2024), which also makes them animatable	compile errors

Common mistakes

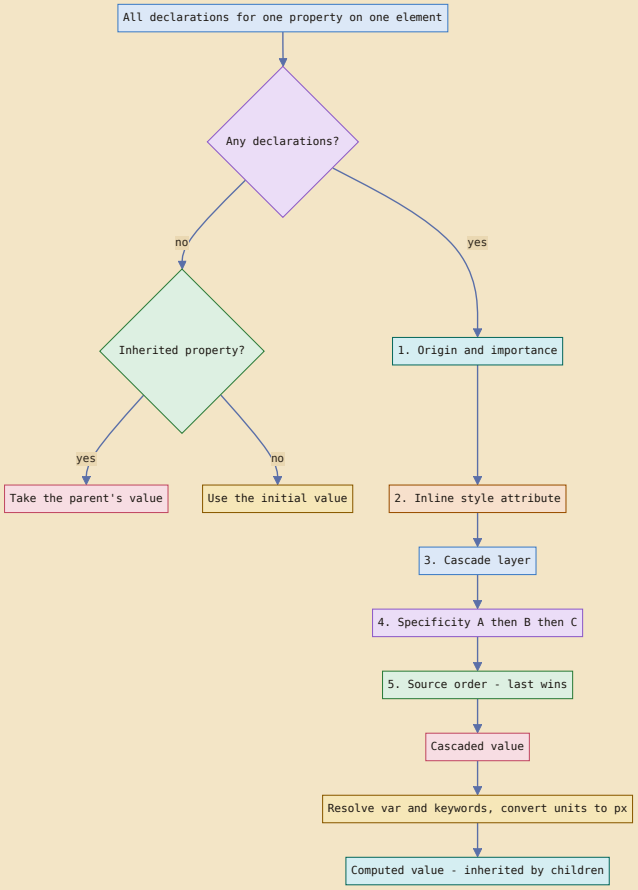
- Forgetting `--` or `var()`: `color: --brand` does nothing.
- Expecting the fallback when the variable *is* defined but unsuitable.
- Theme variables on a component instead of `:root` (inheritance flows down only).
- A variable in a media query condition (`@media (min-width: var(--bp))`) — not allowed.

In the interview

30-second answer: "Custom properties are author-defined properties like `--brand` used via `var(--brand, fallback)`. They cascade and inherit, so I define tokens (named design values: brand colour, spacing) on `:root`, override them on a theme attribute or component class, and change them from JS with `setProperty`. Unlike Sass variables they exist at runtime — which enables theming. An invalid substitution behaves like `unset`."

- *Dark mode with them?* → Colour tokens on `:root`, redefined under `@media (prefers-color-scheme: dark)` (matches a dark system setting) or `[data-theme="dark"]`; components use only tokens (Chapter 13).

6.10 How it all fits together



Chapter TL;DR

- Rule = selector + declarations; external stylesheets; inline only when necessary; invalid declarations are skipped, not fatal.
- Selectors: type, class, ID, `*`, 7 attribute forms (with `i` flag), combinators (space, `>`, `+`, `~`), lists. Prefer single classes.
- Pseudo-classes = states/positions (incl. `:is`, `:where`, `:not`, `:has`); pseudo-elements = parts (`::before` needs `content`).
- Specificity = (IDs, classes/attrs/pseudo-classes, types/pseudo-elements), no carrying; `:where` = 0, `:is/:not/:has` = max argument.
- Cascade: origin & importance → inline → layer → specificity → source order; `!important` reverses origins and layers. Use layers, not `!important`.
- Text properties inherit, box properties don't; any matching rule beats inheritance; `inherit`/`initial`/`unset`/`revert` (spec default vs inherit-or-initial vs previous origin).
- Units: `rem` for type, `em` for local scaling, `%` of the containing block (vertical padding uses width), `dvh` for mobile, `ch` for measure, `px` for hairlines.
- Colours: hex/`rgb`/`hsl`, `oklch` for uniform palettes, `/ alpha`, `currentColor`, `color-mix()`.
- Custom properties cascade and inherit — the foundation of themes.

Next: Chapter 7 — The Box Model and Layout Basics.

Chapter 7 — The Box Model and Layout Basics

Chapter 6 decided *which* declarations apply; this chapter decides *where the element ends up*. Everything on a page is a rectangle — a **box** — and layout sizes and places those boxes. Before Flexbox (Chapter 8, one row or column) and Grid (Chapter 9, rows *and* columns) make sense, you need the rules they build on: box sizing, default flow, `position`, overflow, and who is painted on top.

In this chapter: 7.1 The box model · 7.2 `box-sizing` · 7.3 Margin collapsing · 7.4 `display` · 7.5 Normal flow and formatting contexts (BFC) · 7.6 Positioning · 7.7 Overflow and ellipsis · 7.8 `z-index` and stacking contexts · 7.9 Floats (legacy).

Every example is a page in `code/` with a screenshot in `img/`; outlines and labels are drawn by the example CSS. The `.mjs` scripts open the pages in headless Chrome via `puppeteer-core` and print sizes measured with `getBoundingClientRect()` (an element's on-screen position and size) — every number here is measured. Run one with `node code/02_margin_collapse.mjs`.

> **Tip — outlines are your layout debugger.** An `outline` is drawn around a box but takes no space, so `* { outline: 1px solid rgb(255 0 0 / .3) }` shows every box without changing the layout (a `border` would).

7.1 The box model

What it is

The **CSS box model**: every element generates a rectangular box of four nested areas, inside out:

Area	What it is	Set with	Takes the element's background?
Content	Where text, images and child boxes go	`width`, `height` (and `min-`/`max-`)	Yes
Padding	Space between content and border	`padding`, `padding-top` …	Yes
Border	A line (or image) around the padding	`border`, `border-width`, `border-style`, `border-color`	Drawn on top of the background
Margin	Empty space *outside* the border that pushes other boxes away	`margin`, `margin-left` …	No — always transparent

Each area's outer edge has a spec name: **content box**, **padding box**, **border box**, **margin box**.

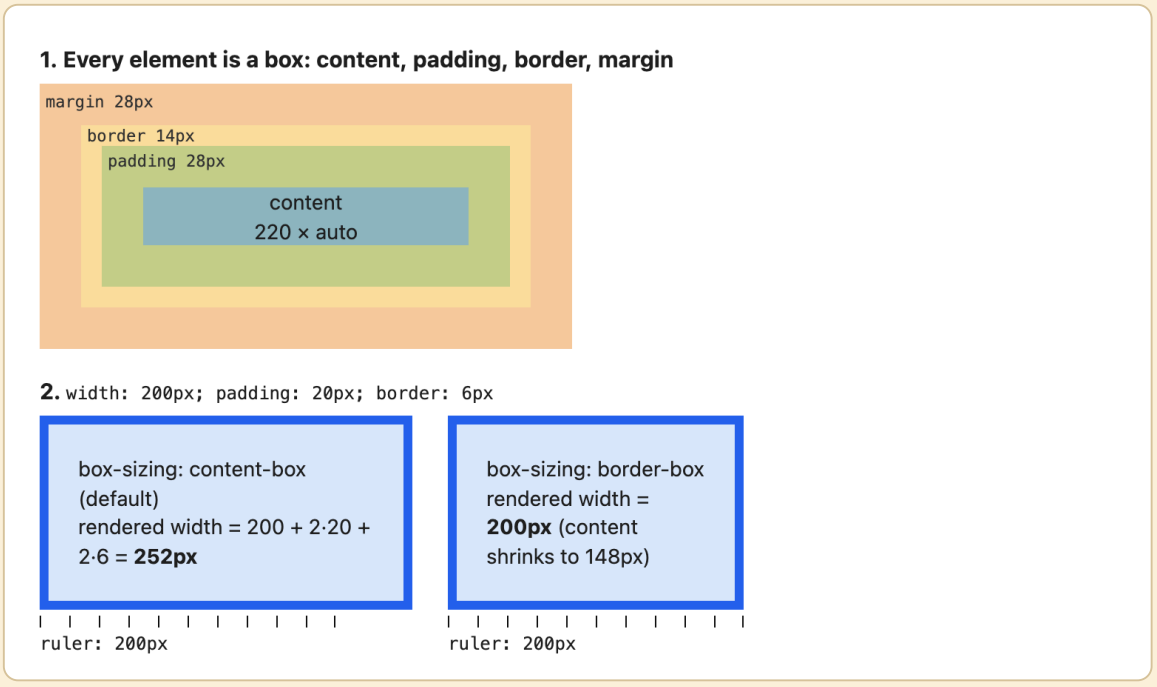


Figure 7.1 — `code/01_box_model.html`, coloured like the DevTools box-model panel (Elements → Computed); content and padding come from two background layers clipped to `content-box` and `padding-box`.

How the size is calculated

With the default `box-sizing: content-box`, `width`/`height` set the **content box only**:

```
rendered width  = width  + padding-left + padding-right + border-left + border-right
space taken     = rendered width + margin-left + margin-right
```

So `width: 200px; padding: 20px; border: 6px solid` = 200 + 2×20 + 2×6 = **252px** on screen (and `width: 300px; padding: 10px 20px; border: 2px` = **344px**) — the surprise `box-sizing` (7.2) fixes.

Shorthands — the clock rule

`margin`, `padding`, `border-width` take 1–4 values **clockwise from the top**:

Written	Top	Right	Bottom	Left
`margin: 10px`	10	10	10	10
`margin: 10px 20px`	10	20	10	20
`margin: 10px 20px 30px`	10	20	30	20
`margin: 10px 20px 30px 40px`	10	20	30	40

`border: 2px solid #333` sets width, style and colour on all sides; the style is required — `border: 2px #333` draws nothing (default style `none`). **Logical properties** like `margin-block` (top + bottom in horizontal text) and `padding-inline` follow the writing direction (Chapter 13).

Sizing rules worth knowing

- `width: auto` (default) on a block = "fill the **containing block** (the rectangle a box is sized and positioned against — normally the parent's content box; §7.6) minus my margins, borders and padding". *Not* `width: 100%`, which ignores margins and can overflow.
- `height: auto` = as tall as the content. A percentage height works only if the parent has a definite height; otherwise it acts as `auto`.
- **Percentages for padding and margin — even top/bottom — use the containing block's *width*** (so `padding-top: 10%` in a 600×200 parent is 60px). Hence the old `padding-top: 56.25%` 16:9 trick (today: `aspect-ratio: 16 / 9`).
- `min-width` / `max-width` clamp: `max-width: 65ch` for readable text; `min-height: 100vh` for a full-screen section that can still grow. On conflict, `min-*` wins.
- **Intrinsic sizes**: `min-content` (as narrow as the longest word allows), `max-content` (as wide as the content wants, no wrapping), `fit-content` (max-content, capped at the available space) — reused as flex and grid track sizes (a *track* is a grid row or column).
- **Margins can be negative**; **padding cannot**.
- `margin: auto` on a block with a set width splits the leftover horizontal space — the classic `margin: 0 auto` centring.

Border vs outline vs box-shadow

	Takes space?	Per side?	Follows `border-radius`?	Typical use
`border`	Yes	Yes	Yes	Visible frames, dividers
`outline`	No (drawn outside the border box)	No	Yes in current browsers	Focus rings (Chapter 3), debugging
`box-shadow`	No	No (but can be offset)	Yes	Elevation (a raised-card look), fake borders that don't shift layout

Common mistakes

- `width: 100%` *and* padding on a content-box element → overflows by padding + border.
- `height: 100%` expecting to fill the screen when the parent's height is `auto`.

- `border: 1px red` with no style → nothing shows.

7.2 `box-sizing`

What it is

`box-sizing` decides *which box* `width` and `height` describe:

Value	`width` means…	Padding and border…
`content-box` (default)	the content box	are **added** on top
`border-box`	the border box	are **taken out of** the width; content shrinks (`width: 250px; padding: 25px; border: 5px` → 190px content)

The problem it solves

Designers think in outer sizes ("the sidebar is 300px"). With content-box, 20px padding secretly makes it 340px and the two-column layout breaks; with border-box, 300px stays 300px. Measured by `code/01_box_sizing.mjs` for Figure 7.1's boxes (`width: 200px; padding: 20px; border: 6px`):

```
content-box  width property 200px  content 200px  on screen (border box) 252px
border-box   width property 200px  content 148px  on screen (border box) 200px
```

`withPage` in `code/measure.mjs` is a ten-line helper (launch Chrome, open the file, run a function in the page, close). The three DOM measurements you'll use most:

API	Returns
`el.getBoundingClientRect()`	Border box position and size on screen (includes transforms and fractional pixels such as 12.5)
`el.offsetWidth` / `offsetHeight`	Border box size, rounded to an integer, ignores transforms
`el.clientWidth` / `clientHeight`	Padding box size minus any scrollbar

The universal reset

Almost every modern stylesheet and CSS reset (Chapter 13) starts with:

```
*, *::before, *::after { box-sizing: border-box; }
```

`box-sizing` isn't inherited, so the rule targets every element and pseudo-element directly (`*` alone doesn't match pseudo-elements). A variant — `html { box-sizing: border-box }` + `*, *::before, *::after { box-sizing: inherit }` — lets a third-party widget opt its whole subtree back into content-box.

Browser defaults already give many form controls border-box (in Chrome, `<input>`, `<select>`, `<button>`, `<textarea>`), and browsers differ — another reason to set it everywhere so a `div` and an `input` with equal `width` line up.

When not to use it

No real downside. One edge: if padding + border exceed the width (`width: 20px; padding: 20px`), content can't go below zero, so the box grows beyond `width`.

In the interview

30-second answer: "Every element is a box of content, padding, border and margin. By default `width` sets only the content, so padding and border are added — 200 wide plus 20 padding plus 6 border is 252. With `box-sizing: border-box`, `width` includes padding and border, so the box stays 200 and the content shrinks.

> Most projects apply border-box to everything in the reset because it matches how designs are measured. Margin is never included in either model."

- *Is `box-sizing` inherited?* → No, hence the `*` selector (or the `inherit` pattern).

7.3 Margin collapsing

What it is

In normal block layout, when two **vertical** margins touch with nothing between, they **collapse** into one margin the size of the *larger*, not the sum. Horizontal margins never collapse. It was designed for text — paragraphs with `margin: 1em 0` should be 1em apart, not 2em — and is behind most "why is there a gap / where did my margin go?" questions.

How it works — the three situations

1. **Adjacent siblings**: one block's bottom margin and the next's top margin collapse.
2. **Parent and first/last child**: if no border, padding, inline content or clearance (space added by `clear`, §7.9) separates a parent from its first child, the child's top margin collapses *through* the parent — appearing **outside** it. Same for the last child's bottom margin (parent height `auto`).
3. **Empty blocks**: a block with no content, padding, border or height has its own top and bottom margins collapse together.

Margins meeting	Result
All positive (30, 20)	The largest: **30**
Positive and negative (30, −10)	Largest positive + most negative: **20**
All negative (−10, −25)	The most negative: **−25**

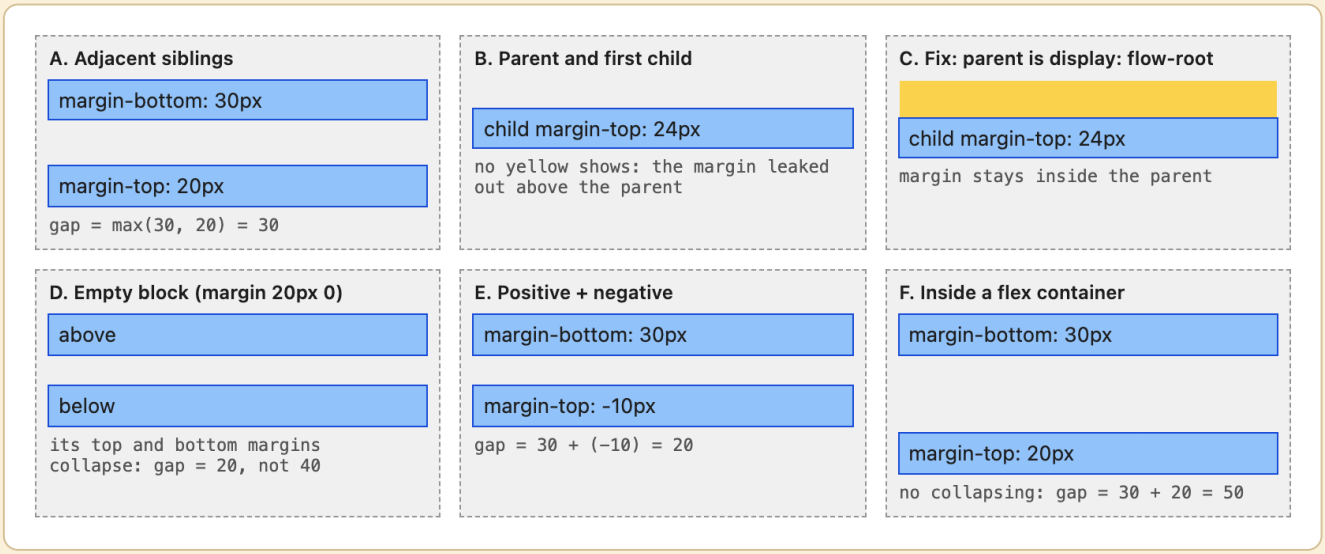


Figure 7.2 — `code/02_margin_collapse.html`. In case B no yellow shows: the 24px margin sits above the parent and the child fills it exactly.

Measured gaps (`code/02_margin_collapse.mjs`: gap = next box's top − previous box's bottom; inset = child top − parent top):

```
A siblings 30 / 20           -> 30px
B child top inside parent    -> 0px
C same with flow-root parent -> 24px
D across empty block 20/20   -> 20px
E 30 / -10                   -> 20px
F flex column 30 / 20        -> 50px
```

What stops margins collapsing

- Anything **between** them: parent padding or border (even `padding-top: 1px`), inline content, a `clear`ed float.

- The parent starting a new **block formatting context** (7.5): `display: flow-root`, `overflow` other than `visible`/`clip`, `inline-block`, floats, absolute positioning.
- **Flex and grid**: margins of flex/grid items never collapse (case F; 40 + 25 in a grid = 65). Hence vertical stacks built with `display: flex; flex-direction: column; gap` or `display: grid; gap` — `gap` never collapses.
- Floated and absolutely positioned boxes never collapse with anything.

Common mistakes

- A card's first heading `margin-top` moving the *whole card* (parent-child collapse). Fix: `display: flow-root` or padding on the card, or drop the child's top margin.
- `margin-bottom: 20px` on every item plus `margin-top: 20px` on the next, expecting 40px.
- "Fixing" collapse with `overflow: hidden` — works (new BFC) but clips shadows and focus rings. Prefer `flow-root`.

Real-world examples

Design systems (and the "Every Layout" book's *Stack*) avoid the topic by setting margins in **one direction only** — `.stack > * + * { margin-block-start: 1rem }` — or using `gap`.

In the interview

> **30-second answer:** "Vertical margins between blocks in normal flow collapse to the larger — 30 and 20 give 30, not 50. It happens between siblings, between a parent and its first or last child when nothing separates them, and on empty blocks; negative margins are added to the largest positive. It doesn't happen in flex or grid, for floats or absolutely positioned boxes, or when padding, a border or a new BFC like `display: flow-root` separates them."

7.4 `display`

What it is

`display` answers two questions:

- **Outer display type** — how the box joins its parent's layout: **block** (own line) or **inline** (within a line of text).
- **Inner display type** — how it lays out its *children*: **flow**, **flex**, **grid**, **table**…

Every keyword is a combination; since 2023 all major browsers (Baseline) accept the explicit **two-value syntax**:

Keyword	Two-value form	Outer	Children laid out as
`block`	`block flow`	block	normal flow
`inline`	`inline flow`	inline	normal flow (inline)
`inline-block`	`inline flow-root`	inline	flow, in its own formatting context
`flow-root`	`block flow-root`	block	flow, in its own formatting context
`flex`	`block flex`	block	flex (Ch. 8)
`inline-flex`	`inline flex`	inline	flex
`grid` / `inline-grid`	`block grid` / `inline grid`	block / inline	grid (Ch. 9)
`list-item`	`block flow list-item`	block	flow, plus a `::marker`
`table`, `table-row`, `table-cell`	—	—	table layout (what `<table>` uses)

Two keywords generate **no box of their own**:

- `display: none` — the element and subtree produce no boxes: no space, not painted, not in the accessibility tree, not focusable (unlike `visibility: hidden` / `opacity: 0`, which keep space — §1.7). For content screen readers still need, use a visually-hidden class (Chapter 3).
- `display: contents` — the element's own box disappears but its **children** lay out as if they were the grandparent's — handy to let a wrapper's items join a parent's flex/grid. Caution: browsers historically

dropped the element's semantics from the accessibility tree (a `<ul>` stopped being announced as a list); most engines fixed the common cases, but test with a screen reader and avoid it on buttons and interactive elements.

Block vs inline vs inline-block

Behaviour	`block`	`inline`	`inline-block`
Starts on a new line	Yes	No	No
Default width	Fills the container	Its content	Its content (shrink-to-fit)
`width` / `height`	Respected	**Ignored**	Respected
Horizontal padding/border/margin	Push neighbours	Push neighbours	Push neighbours
Vertical padding/border	Push neighbours	**Painted but don't move lines** (overlap)	Push neighbours
Vertical margin	Applies (may collapse)	**Ignored**	Applies (never collapses)
Default for	`div`, `p`, `h1`–`h6`, `ul`, `section`, `form`	`span`, `a`, `strong`, `em`, `code`	nothing by default; `button`, `input`, `img` behave similarly

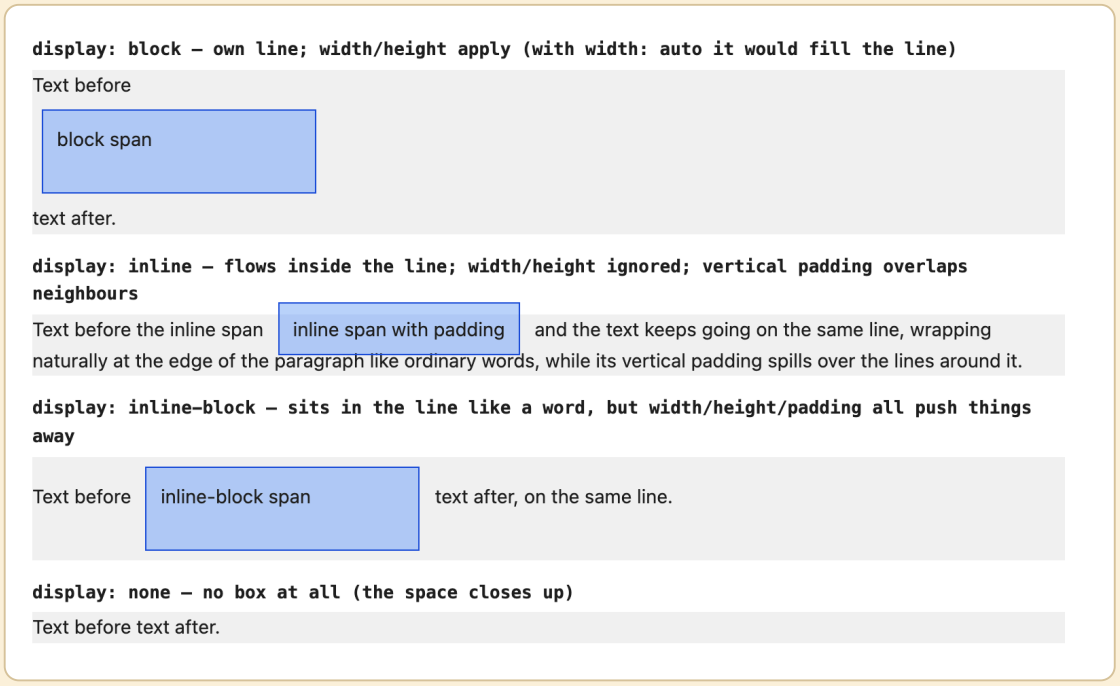


Figure 7.3 — `code/03_display.html`. Every span has `width: 180px; height: 40px; padding: 10px; margin: 8px`; only `display` differs.

```
block         rendered 200 x 60  height of its <p>: 121
inline        rendered 176 x 37  height of its <p>: 45
inline-block  rendered 200 x 60  height of its <p>: 76
none          rendered   0 x  0  height of its <p>: 22
```

Output of `code/03_display.mjs`. The inline span ignores `width` (176px is text + padding) and its paragraph stays two ordinary lines (45px) despite 10px vertical padding; the inline-block keeps 200×60 and the line grows.

Block/inline defaults are only the **user-agent stylesheet** (the browser's built-in CSS); HTML defines content models, not layout (Chapter 2) — a `<span>` with `display: block` is still phrasing content.

Replaced elements

`<img>`, `<video>`, `<iframe>`, `<input>` are **replaced elements**: content from outside CSS, an **intrinsic size**, inline-level yet accepting `width`/`height`. Classic surprise: an `<img>` in a `div` leaves a small gap below because, as an inline box, it sits on the text **baseline** leaving room for descenders (the tails of g, p, y). Fix: `img { display: block }` (common in resets) or `vertical-align: middle`.

Common mistakes

- `width` on an `<a>`/`<span>` with no effect → make it `inline-block` or `block` (an `<a>` that must be 44px tall within text: `inline-block` or `inline-flex`).
- `margin-top` on an inline link → ignored.
- `inline-block` rows of cards fighting the whitespace gap (spaces between tags render as a space). Use flex or grid with `gap`.

7.5 Normal flow and formatting contexts

What it is

Normal flow is the layout with no layout CSS:

1. **Block-level boxes** stack in the **block direction** (top to bottom in English), each on a new line, filling the width if `width: auto`.
2. **Inline-level boxes** (text, `span`, `a`, images) sit side by side in the **inline direction** and wrap into **line boxes** (one per line of text).
3. Vertical margins between blocks collapse (7.3).
4. A block's height grows to fit its content.

"Block/inline direction" replace "vertical/horizontal" because they rotate with the writing mode: in vertical Japanese text, blocks stack horizontally (right to left) and lines run top to bottom — hence names like `margin-block` and `inline-size`.

Formatting contexts

A **formatting context** is a region where one set of layout rules governs the children:

Context	Created by	Children laid out by
Block formatting context (BFC)	root `<html>`, `display: flow-root`, `inline-block`, floats, `position: absolute/fixed`, `overflow` other than `visible`/`clip`, table cells, flex/grid items, `contain: layout/paint` (a hint isolating an element's layout/painting), multi-column containers	block and inline flow
Inline formatting context (IFC)	a block containing only inline-level content	line boxes
Flex formatting context	`display: flex / inline-flex`	flexbox (Ch. 8)
Grid formatting context	`display: grid / inline-grid`	grid (Ch. 9)

A new **BFC** is a sealed room:

- child margins don't collapse through its edges (7.3),
- it **contains its floats** — its height includes them (7.9),
- it doesn't overlap outside floats; it sits beside them.

`display: flow-root` exists *only* to create a BFC with no side effects, replacing the old `overflow: hidden` and "clearfix" hacks (§7.9); supported in every current browser.

Centring in normal flow

Goal	Normal-flow technique
Centre inline content horizontally	`text-align: center` on the parent
Centre a block with a known or max width	`max-width: 40rem; margin-inline: auto`
Centre a block's children vertically	`align-content: center` on the block itself (plain blocks since 2024 — Baseline newly available)
Centre anything both ways	`display: grid; place-items: center` (Chapter 9)

7.6 Positioning

What it is

`position` moves a box away from its normal-flow spot using the **inset properties** `top`, `right`, `bottom`, `left` (shorthand `inset`, same clock rule as `margin`). A **positioned element** is any with `position` other than `static`; the rectangle its offsets and percentages refer to is its **containing block**.

Value	Stays in flow (keeps its space)?	Offsets measured from	Scrolls with the page?	Typical use
`static` (default)	Yes	— insets ignored	Yes	Everything
`relative`	**Yes** — original space kept; neighbours don't move	Its own normal position	Yes	Small nudges; anchor for absolute children
`absolute`	**No**	Nearest **positioned ancestor's** padding box (else the initial containing block)	Yes, with that ancestor	Badges, dropdowns, overlays in a card
`fixed`	No	The **viewport**	**No** — stays put	Cookie bars, chat buttons, old-style fixed headers
`sticky`	Yes	Its nearest **scrolling ancestor** (scrollport — the visible window of a scrollable box)	Scrolls until the threshold, then sticks	Table headers, section headings, sidebars

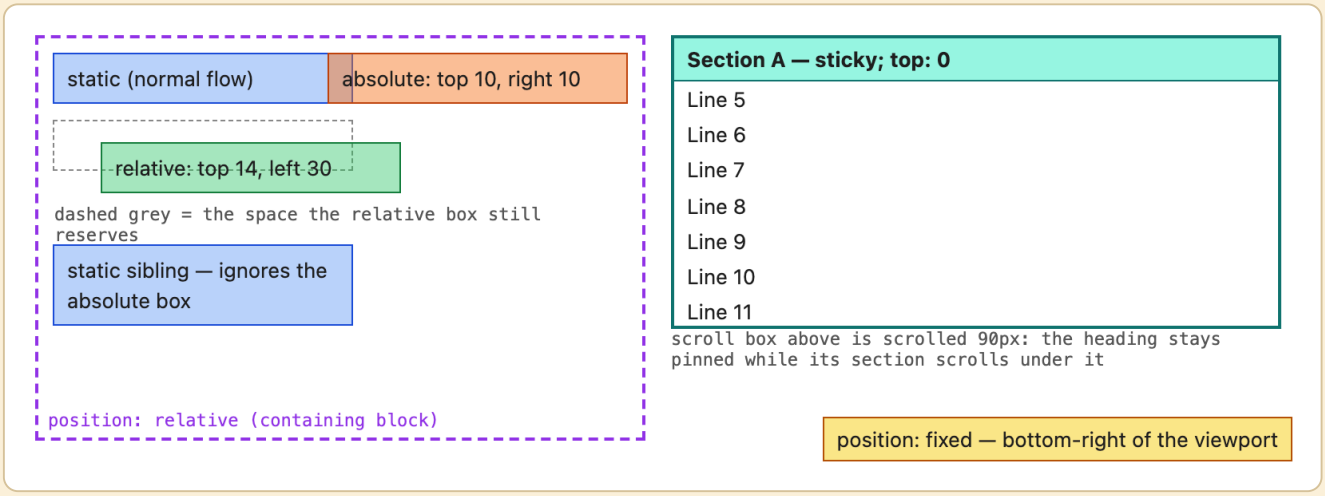


Figure 7.4 — `code/04_positioning.html`. The dashed grey rectangle is the space the relative box still reserves; the absolute box overlaps the static box because it left the flow.

```
.container { position: relative; }                    /* becomes the containing block for .abs */
.rel  { position: relative; top: 14px; left: 30px; }    /* nudged; space kept */
.abs  { position: absolute; top: 10px; right: 10px; }   /* pinned to the container's corner */
.scroller { height: 180px; overflow: auto; }
.scroller h4 { position: sticky; top: 0; }              /* sticks to the scroll box's top */
.fixed { position: fixed; right: 12px; bottom: 12px; }  /* glued to the viewport */
```

Measured by `code/04_positioning.mjs`, including a 500px page scroll:

```
relative : moved 30px right, 14px down from its normal spot
absolute : 10px from container's right padding edge, 10px from its top
sticky   : heading top - scroller top = 0px after scrolling 90px (stays pinned)
fixed    : before scroll right/bottom gap = 12/12px, after scrolling 500px = 12/12px (glued to the viewport)
```

How each one works

`relative` — laid out in flow, then shifted visually; neighbours still see the original position. Its main job today: being the **containing block** for absolute children, usually with no offsets.

`absolute`

1. Removed from flow — siblings close up.
2. The browser walks up to the first ancestor with `position` other than `static` (or with `transform`, `filter`, `perspective`, `contain: layout/paint`, or `will-change` of those); its **padding box** is the containing block — so `right: 0` in a parent with `padding: 20px` sits flush with the inner border edge, not 20px in. With none, the initial containing block (viewport-sized, anchored at the document top) is used.
3. Insets position the margin edges; with `width: auto` the box **shrinks to fit**.
4. Both `left` and `right` (or `inset: 0`) stretch it; `inset: 0; margin: auto` plus width and height centres it.

`fixed` — like absolute but against the **viewport**, so it doesn't scroll. Trap: an ancestor with `transform`, `filter`, `perspective`, `backdrop-filter`, `contain: paint` or `will-change: transform` becomes the containing block, and your "fixed" modal scrolls away. Hence modals rendered at the end of `<body>` (React "portals" render a component elsewhere in the DOM) — or better, native `<dialog>`, which uses the **top layer** (painted above the whole page) and escapes all this.

`sticky`

1. Behaves as `relative` until its scroll container scrolls it to the threshold (e.g. `top: 0`), then sticks.
2. It **never leaves its parent**: when the parent's bottom scrolls up, the sticky box is pushed away — "each heading sticks until the next section".
3. Needs at least one inset; without one it never sticks.
4. It sticks to the nearest **scroll container** (`overflow: auto/scroll/hidden`). A wrapper with `overflow: hidden` that isn't what actually scrolls captures it and it "doesn't work". `overflow: clip` (and `visible`) don't create scroll containers, so they're safe.

Trade-offs

Approach	Good for	Beware
Absolute positioning for layout	Overlays, badges, decorative bits	Overlap and clipped text as content grows or users zoom; not for main layout
Fixed headers/footers	Always-available controls	Covers content (use `scroll-padding-top` so same-page `#id` links don't hide under the header); eats small-screen space
Sticky	Headers that stay visible while scrolling	Silent failures from ancestor `overflow` or a parent exactly as tall as the sticky element

Common mistakes

- `position: absolute` with no positioned ancestor — jumps to the page corner. Add `position: relative` to the intended parent.
- `top: 50%; left: 50%` centring without `translate: -50% -50%` (offsets move the *top-left corner*). Prefer `inset: 0; margin: auto` or grid.
- Sticky in a parent only as tall as itself → no room to stick.
- `z-index` on a static element (7.8).

In the interview

> **30-second answer:** "`static` is normal flow. `relative` keeps the element's space and shifts it visually; mostly it's the reference for absolute children. `absolute` leaves the flow and positions against the nearest positioned ancestor's padding box. `fixed` positions against the viewport so it doesn't scroll — unless an ancestor has a transform or filter, which becomes the containing block. `sticky` is in flow until a threshold in its scroll container, then sticks, but only within its parent."

- *Sticky header not sticking?* → No `top`, an ancestor `overflow` that isn't the scroller, or a too-short parent.
- *`position: fixed` modal scrolling with the page?* → An ancestor has `transform`/`filter`/`will-change`; move it out, or use `<dialog>`.

7.7 Overflow

What it is

Overflow is content that doesn't fit its box — a fixed `height`, or a long word or wide image exceeding `width`. `overflow` (or `overflow-x`/`overflow-y`) decides what happens:

Value	Overflowing content	Scrollbars	Scrollable by script?	Creates a BFC / scroll container?
`visible` (default)	Spills out, paints over what follows	No	No	No
`hidden`	Clipped at the padding box	No	**Yes** (`el.scrollTop = …`, focus, anchor links)	Yes / yes
`clip`	Clipped	No	**No**	No / no
`scroll`	Clipped	**Always**, even if not needed (ugly on Windows)	Yes	Yes / yes
`auto`	Clipped	Only when needed	Yes	Yes / yes

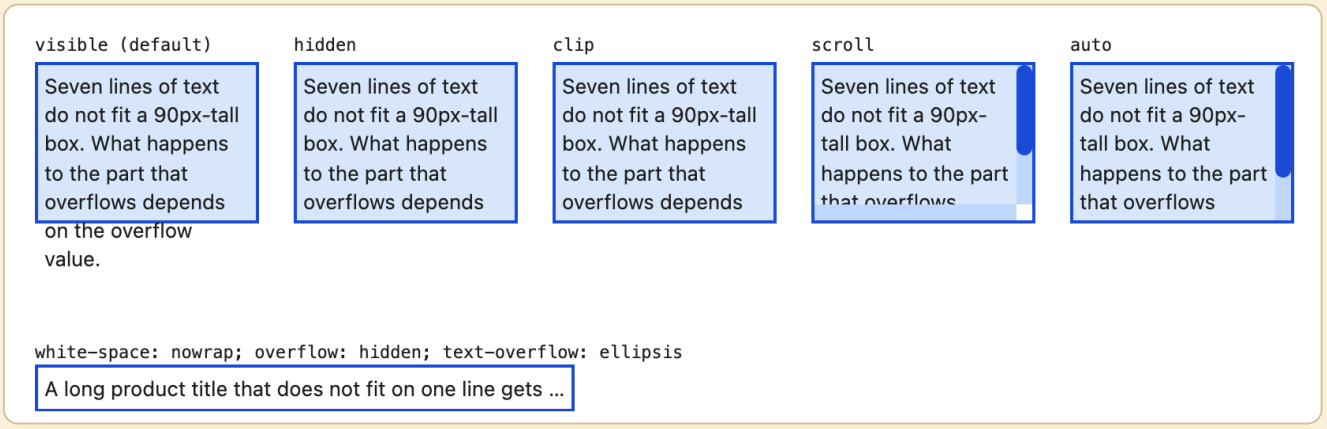


Figure 7.5 — `code/05_overflow.html`. Scrollbars are coloured with `::-webkit-scrollbar` for the screenshot; macOS normally uses overlay scrollbars that appear only while scrolling.

`code/05_overflow.mjs` sets `scrollTop = 20` on each box and reads it back:

```
visible  clientHeight 98  scrollHeight 131  scrollTop after setting 20 -> 0
hidden   clientHeight 98  scrollHeight 135  scrollTop after setting 20 -> 20
clip     clientHeight 98  scrollHeight 131  scrollTop after setting 20 -> 0
scroll   clientHeight 88  scrollHeight 135  scrollTop after setting 20 -> 20
auto     clientHeight 98  scrollHeight 135  scrollTop after setting 20 -> 20
```

`hidden` boxes *can* still be scrolled programmatically (a keyboard user tabbing to an inner link scrolls them too); `clip` cannot. `scroll` spent 10px on an unneeded horizontal scrollbar (clientHeight 88). Scroll containers count bottom padding in the scrollable area (135 vs 131). `overflow: clip` reached Baseline in 2022.

Useful companions

- **Single-line truncation** — all three required:

  ```
  .title { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
  ```

- **Multi-line truncation**: `display: -webkit-box; -webkit-box-orient: vertical; -webkit-line-clamp: 3; overflow: hidden;` — `-webkit-` prefixed (began as one engine's extension) but works everywhere.
- **Long words and URLs**: `overflow-wrap: anywhere` (or `break-word`).
- **Mixing axes**: if one axis is `visible` and the other isn't, `visible` computes to `auto` — use `clip` to clip one axis only.
- `scrollbar-gutter: stable` reserves scrollbar space so content doesn't jump.
- `overscroll-behavior: contain` stops scrolling in a modal or chat panel from "chaining" to the page behind once it reaches its end.

Common mistakes

- `overflow: hidden` to fix a float/margin problem, later clipping focus outlines and shadows.
- `overflow: hidden` on `body`/`html` to lock scrolling, breaking `position: sticky` below.
- Truncating text users need with no way to see the rest (add a tooltip or expand control; keep the full text in the DOM for screen readers).

7.8 `z-index` and stacking contexts

What it is

Overlapping boxes (negative margins, positioning, transforms, grid items sharing a cell) are drawn in a **stacking order** along the **z-axis** (pointing out of the screen). `z-index` sets an element's place on it, but only **within its stacking context** — a group painted as one unit. Think of each context as a sheet of paper: you can reorder drawings on a sheet, and reorder whole sheets, but a drawing can never slip between two other sheets, whatever its number.

How it works

1. `z-index` applies to **positioned** elements and to **flex and grid items**; on a static block it does nothing.
2. Higher `z-index` in the **same** context paints in front; ties paint in DOM order (later in front).
3. A numeric `z-index` (even `0`) on a positioned or flex/grid item **creates a new stacking context** (`auto` paints at the same level as `0` but creates none).
4. Descendants' z-indexes are compared only with each other; outside, the group has its root's z-index.

Painting order within one context (back to front):

1. The context root's background and borders
2. Descendants with **negative** `z-index`
3. In-flow, non-positioned block boxes
4. Floats
5. Inline content
6. Positioned descendants with `z-index: auto` or `0`, in DOM order
7. Positioned descendants with **positive** `z-index`, lowest first

So a child with `z-index: -1` inside parent A (`z-index: 5`) can never go behind A's own background — A's background is always the back of its context.

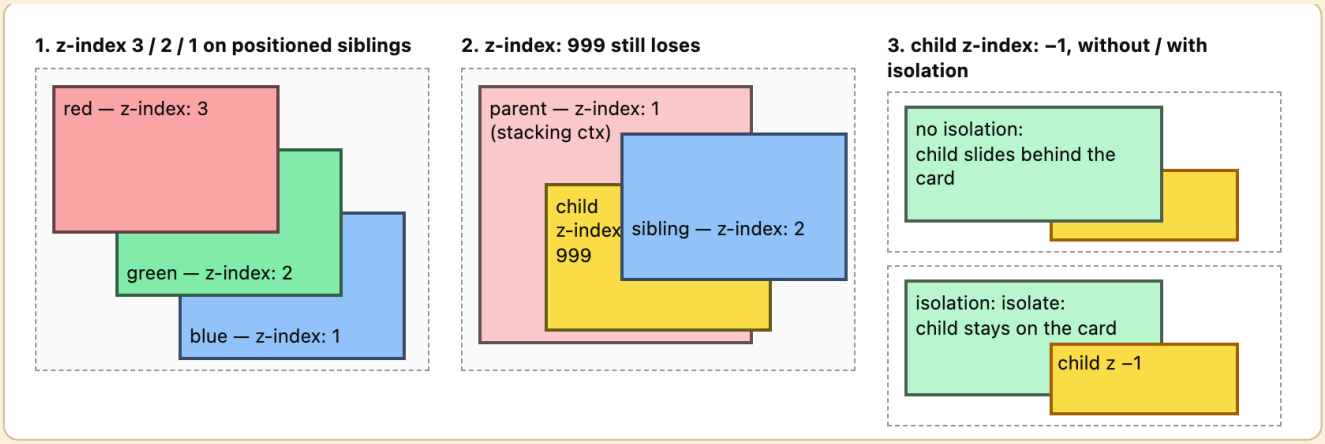


Figure 7.6 — `code/06_stacking.html`.

```
/* demo 2 — the classic trap */
.parent  { position: absolute; z-index: 1; }     /* creates a stacking context */
.child   { position: absolute; z-index: 999; }   /* 999 only counts INSIDE .parent */
.sibling { position: absolute; z-index: 2; }     /* 2 > 1, so it covers .parent and everything in it */

/* demo 3 — isolation: isolate makes a stacking context without z-index */
.card.iso { isolation: isolate; }                /* the child's z-index: -1 now stays above the card's background */
```

`code/06_stacking.mjs` asks `document.elementFromPoint(x, y)` ("which element would get a click here?") where boxes overlap:

```
demo 1  where red, green and blue overlap -> r1
demo 2  where child (999) and sibling (2) overlap -> b2
demo 3  inside the yellow child, no isolation -> card
demo 3  inside the yellow child, isolation: isolate -> u2
```

What creates a stacking context

- The root element (`<html>`)
- `position: relative` / `absolute` **with** `z-index` other than `auto`
- `position: fixed` or `sticky` (always)
- A flex or grid item with `z-index` other than `auto`
- `opacity` less than 1
- `transform`, `scale`, `rotate`, `translate`, `filter`, `backdrop-filter`, `perspective`, `clip-path`, `mask` (any value but `none`)
- `mix-blend-mode` other than `normal`
- `isolation: isolate`
- `will-change` naming any property above
- `contain: layout`, `paint`, `strict` or `content`; `container-type: size` or `inline-size`
- The **top layer** — an open modal `<dialog>`, a shown popover (`popover` attribute), a fullscreen element — paints above *everything*, whatever the z-indexes (so a modal beats a `z-index: 10000` header)

Why it matters

"`z-index: 99999` and the dropdown is still behind the header" is almost always a stacking-context problem: an ancestor (a `transform` for an animation, `opacity: .99`) created a context whose own level is below the header's.

How to debug and fix it

1. Walk up from the element that's "behind", looking for ancestors in the list above (Chrome/Edge DevTools → Layers panel, or Edge's 3D view, visualise contexts).
2. Compare the z-indexes of the **two ancestors that are siblings in a common context**, not the elements themselves.
3. Fix: raise that ancestor's z-index, remove the property that created the unwanted context, or move the overlay in the DOM (or into the top layer with `<dialog>` / `popover`).

4. Prevent: a small **z-index scale** in custom properties (`--z-dropdown: 100; --z-sticky: 200; --z-modal: 300;`) and `isolation: isolate` on components so internal z-indexes can't leak.

In the interview

30-second answer: "`z-index` only orders elements within the same stacking context, and only on positioned or flex/grid items. A z-index on a positioned element, opacity below 1, transforms, filters and `isolation: isolate` create a new context, and everything inside is flattened into it — so a child with `z-index: 999` can't escape a parent with `z-index: 1` that sits below a sibling with `z-index: 2`. To debug I find the ancestors that are siblings in a common context and compare those."

7.9 Floats (legacy)

What it is

`float: left` / `right` takes a box out of normal flow, pushes it to that side, and lets **inline content wrap around it** — designed for magazine-style images in articles. From roughly 2005 to 2015 floats were abused for whole page layouts with a pile of hacks; flexbox and grid replaced that. You still need floats to wrap text around a shape, and to read older code.

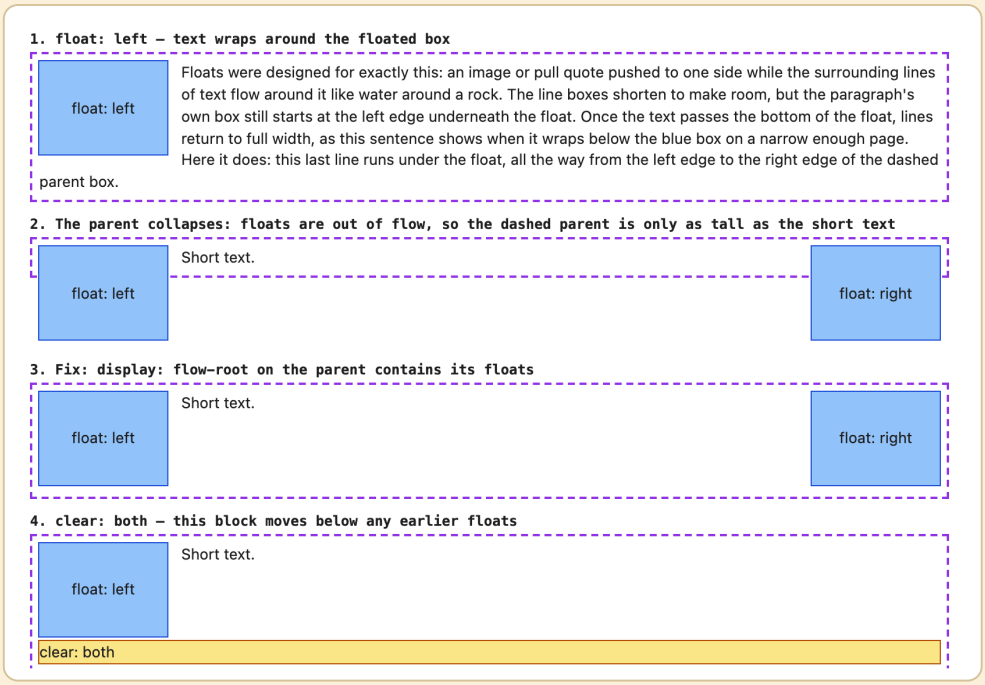


Figure 7.7 — `code/07_floats.html`.

How it works

1. The float moves as far left (or right) as it can, at the vertical position it would have had.
2. Following blocks act as if it isn't there, but their **line boxes shorten** to avoid it, so text flows around.
3. Floats are out of flow, so the parent's auto height ignores them — it **collapses** if everything inside floats (demo 2).
4. `clear: left | right | both` makes a later block start below earlier floats (demo 4).
5. A parent establishing a **BFC** — best with `display: flow-root` — contains its floats (demo 3).

Measured parent heights (`padding: 6px`, 2px border; floats 80px tall with a 4px bottom margin), from `code/07_floats.mjs`:

```
w1  display: block      parent height 129.0625px
w2  display: block      parent height 34.84375px
w3  display: flow-root  parent height 100px
w4  display: flow-root  parent height 118.84375px
```

`w2` is one line of text plus padding and border; `w3`, identical content, contains the floats: 80 + 4 + 2×6 + 2×2 = **100px**.

The clearfix (read it, don't write it)

Before `flow-root`, floats were contained with this pseudo-element hack, still seen in legacy and Bootstrap 3–era (2013) CSS:

```
.clearfix::after { content: ""; display: table; clear: both; }
```

When to use floats today

Need	Use
Wrap text around an image or pull quote	`float` (optionally `shape-outside: circle(50%)` to wrap around a shape)
Columns, card rows, nav bars, sidebars	Flexbox or grid
Push one item right in a row	Flexbox `margin-left: auto` (Chapter 8)

`float` is ignored on flex and grid items.

Chapter 7 TL;DR

- **Box model**: content → padding → border → margin. Margin is transparent and outside; background covers content + padding (and under the border).
- `box-sizing: border-box` makes `width` include padding and border; put it on `*, *::before, *::after`. Content-box: 200 + 2×20 + 2×6 = 252.
- **Percent padding/margin** (even vertical) use the containing block's **width**. `width: auto` ≠ `100%`.
- **Margin collapsing**: vertical margins in block flow merge to the largest (positive + most negative if mixed); between siblings, parent/first-last child, empty blocks; never in flex/grid, for floats/absolute, or across padding/border/new BFC.
- `display`: outer (block/inline) + inner (flow/flow-root/flex/grid). Inline ignores width/height and vertical margins; inline-block keeps them. `none` removes the box; `contents` removes only the element's own box (check accessibility).
- **Normal flow**: blocks stack in the block direction, inlines wrap into line boxes. A **BFC** (`display: flow-root`) contains floats and stops margin collapse through its edges.
- **Position**: `relative` = shifted, space kept; `absolute` = out of flow, nearest positioned ancestor's padding box; `fixed` = viewport (unless an ancestor has transform/filter); `sticky` = in flow until its threshold in the nearest scroll container, never leaves its parent.
- **Overflow**: visible / hidden (script-scrollable) / clip (not scrollable, no BFC) / scroll (always bars) / auto (bars when needed). Ellipsis needs `nowrap + hidden + text-overflow`.
- **Stacking**: `z-index` works on positioned and flex/grid items, only within its **stacking context**; contexts come from z-index on positioned elements, opacity < 1, transforms, filters, `isolation: isolate`, fixed/sticky and more. The top layer beats everything.
- **Floats**: for wrapping text around images; contain with `flow-root`, escape with `clear`; don't build layouts with them.

Next: Chapter 8 — Flexbox.

Chapter 8 — Flexbox

Normal flow (Chapter 7) stacks blocks and wraps text, nothing more. Side-by-side boxes, equal heights or vertical centring used to need floats, tables or negative-margin hacks. **Flexbox** (the CSS Flexible Box Layout module, a W3C Candidate Recommendation supported everywhere for about a decade) solves the **one-dimensional** case: lay items out in a **row or a column**, share out spare space, and align them — navbars, toolbars, button groups, cards, form rows, centring. For two-dimensional layouts (rows *and* columns) use Grid (Chapter 9).

In this chapter: 8.1 Container and items · 8.2 The two axes, direction and `order` · 8.3 `flex-grow` / `flex-shrink` / `flex-basis` (calculated by hand, verified in Chrome) · 8.4 The `flex` shorthand and `min-width: auto` · 8.5 Alignment and auto margins · 8.6 Wrapping and `gap` · 8.7 Common patterns · 8.8 Pitfalls, debugging and interview prep.

Every example is a page in `code/` with its screenshot in `img/`: containers have a **dashed purple border**, items a **solid blue outline**, and widths in **red** are measured by the page after layout. The sizing maths is implemented in `code/flex_algorithm.mjs` and checked against headless Chrome by `code/03_flex_measure.mjs`.

8.1 Flex container and flex items

What it is

`display: flex` makes an element a **flex container**; its **direct children** become **flex items**, laid out by flexbox instead of normal flow. Grandchildren are unaffected — each level decides for itself (so `display: flex` on a `<ul>` makes the `<li>`s items, not the `<a>`s inside them).

```
<nav class="toolbar">          <!-- flex container -->
  <a href="/">Home</a>         <!-- flex item -->
  <a href="/docs">Docs</a>     <!-- flex item -->
  <span>v2.1 <b>beta</b></span> <!-- flex item; the <b> inside is NOT an item -->
</nav>
```

```
.toolbar { display: flex; }          /* block-level container (takes a full line) */
.badge   { display: inline-flex; }   /* inline-level container (sits in text); also `inline flex` */
```

A bare run of text directly inside the container becomes an **anonymous flex item** (browser-created, no element, so not targetable by CSS).

Which properties go where

On the container	On the items
`display: flex | inline-flex`	`flex-grow`, `flex-shrink`, `flex-basis` (or shorthand `flex`)
`flex-direction`	`align-self`
`flex-wrap` (both via `flex-flow`)	`order`
`justify-content`	`margin` (including `auto`)
`align-items`	`min-width` / `min-height` (matters a lot, see 8.4)
`align-content`	
`gap`, `row-gap`, `column-gap`	

What changes for the children

- Items are "blockified": `display: block` / `inline` no longer matters (a `<span>` item accepts width and height).
- `float` and `clear` are ignored; `vertical-align` has no effect.
- Margins of adjacent items **do not collapse** (§7.3).
- An item with `z-index` creates a stacking context even without `position` (§7.8).
- By default items sit in a row, are content-width, may shrink but not grow, and **stretch** to the container's height.

8.2 The two axes, `flex-direction` and `order`

What it is

Flexbox never says "horizontal"/"vertical". It uses two **axes**:

- The **main axis** — the direction items are placed in, from **main-start** to **main-end**.
- The **cross axis** — perpendicular to it, **cross-start** / **cross-end**.

flex-direction	Main axis (English, left-to-right text)	Cross axis
`row` (default)	left → right (follows text direction; right → left in Arabic or Hebrew)	top → bottom
`row-reverse`	right → left	top → bottom
`column`	top → bottom	left → right
`column-reverse`	bottom → top	left → right

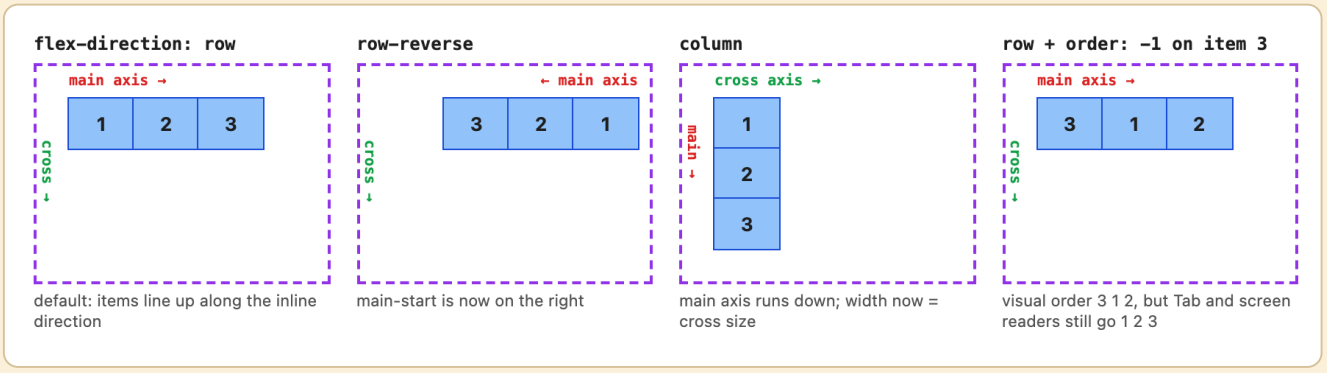


Figure 8.1 — `code/01_axes.html`. Red arrow = main axis, green = cross axis (drawn with `::before` / `::after`).

Why it matters

`justify-content` works on the main axis; `align-items`, `align-self` and `align-content` on the cross axis. Switch to `flex-direction: column` and `justify-content: center` centres *vertically*, while `align-items: center` now centres horizontally. If you remember one thing about flexbox, remember this.

`order`

`order: <integer>` (default `0`) changes **visual** position: ascending `order`, ties in source order (values 2, −1, 0, 0 → the −1 first, then the 0s, then 2). In the fourth demo `order: -1` moves item 3 to the front.

When not to use it (accessibility)

`order`, `row-reverse` and `column-reverse` change only what is *drawn*. **Tab order and screen-reader reading order still follow the DOM**, so keyboard focus jumps around — WCAG 2.2 SC 1.3.2 (Meaningful Sequence) and 2.4.3 (Focus Order) (Chapter 3).

- Change the HTML when the order means something; use `order` only for purely visual tweaks (an image before/after its text in alternating rows).
- The new CSS `reading-flow` property makes focus and reading order follow the visual flex/grid order, but as of 2026 only Chromium (Chrome/Edge 137+) ships it — not Baseline, so don't rely on it.

8.3 `flex-grow`, `flex-shrink` and `flex-basis`

What it is

Three sizing properties, read **along the main axis** (width in a row, height in a column):

Property	Meaning	Default
`flex-basis`	Starting size before free space is shared: a length (`200px`, `30%`), `0`, `content`, or `auto` (= `width`/`height` if set, else content size)	`auto`
`flex-grow`	How many *shares* of **positive free space** the item takes	`0` (don't grow)
`flex-shrink`	How strongly the item gives up space when there is **negative free space** (overflow), weighted by its basis	`1` (may shrink)

Free space = container's inner main size − Σ items' flex base sizes (plus margins, borders, padding) − gaps. Positive → grow; negative → shrink. (Like splitting a bill: leftover budget is shared by grow "shares"; an overspend is paid back in proportion to shrink × what each ordered.)

How it works — the algorithm, simplified

"Resolving the flexible lengths" (Flexbox Level 1, §9.7), for one line:

1. **Base sizes.** Each item's flex base size from `flex-basis`, clamped by `min-*`/`max-*` → its *hypothetical* size.
2. **Grow or shrink?** If Σ hypothetical sizes + gaps + margins/padding/borders < container, use `flex-grow`; otherwise `flex-shrink`.
3. **Freeze inflexible items** (factor 0) at their hypothetical size ("frozen" = final).
4. **Distribute** remaining free space among unfrozen items:
   - growing: `size = basis + free × grow / Σgrow`
   - shrinking: `size = basis − overflow × (shrink × basis) / Σ(shrink × basis)`
   - if unfrozen factors sum to **less than 1**, only that fraction of the free space is handed out.
5. **Clamp** each to `min-*`/`max-*`; if anything was clamped, **freeze** it there and redo step 4 with the remaining space.
6. Stop when every item is frozen.

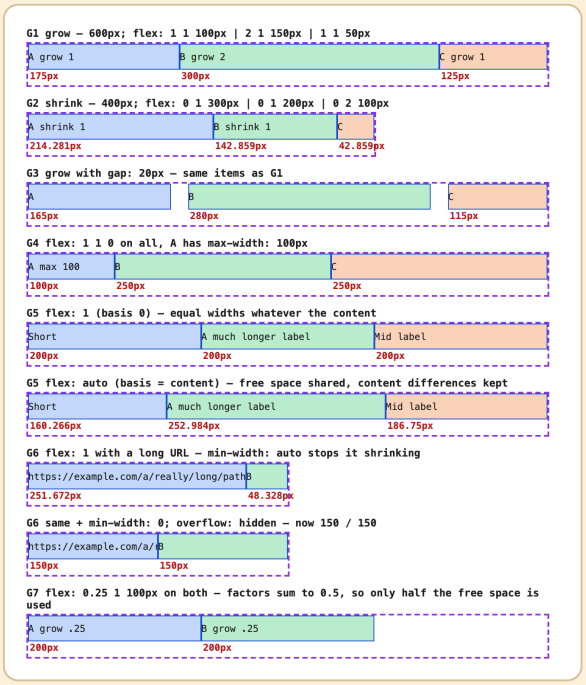


Figure 8.2 — `code/02_grow_shrink.html`. Items have no padding/border, so width = flex size. A script writes each `getBoundingClientRect()` width underneath (labels are absolutely positioned, so they don't affect the layout they report).

Worked example 1 — growing (G1)

Container 600px; items `flex: 1 1 100px`, `flex: 2 1 150px`, `flex: 1 1 50px`.

1. Bases 300 → free **+300** → grow.
2. Shares 1 + 2 + 1 = 4 → one share = 75px.
3. A = 100 + 75 = **175**; B = 150 + 150 = **300**; C = 50 + 75 = **125**. ✓ 600.

B is **not** twice A — `flex-grow` divides the *free space*, not final widths; grow factors become width ratios only when all bases are 0.

Worked example 2 — shrinking (G2)

Container 400px; items `flex: 0 1 300px`, `flex: 0 1 200px`, `flex: 0 2 100px`.

1. Bases 600 → free **−200** → shrink.
2. Scaled factors (shrink × basis): 300, 200, 200 → Σ **700**.
3. A loses 200 × 300/700 = 85.714 → **214.286**; B loses 57.143 → **142.857**; C loses 57.143 → **42.857**.

C has *twice* B's shrink factor yet loses the *same* amount, because its basis is half — weighting by size stops small items being crushed first.

Worked example 3 — gaps (G3)

G1 with `gap: 20px`: two gaps take 40px first → free 260 → share 65 → **165 / 280 / 115**.

Worked example 4 — a max-width freezes an item (G4)

Container 600px, all `flex: 1 1 0`, A has `max-width: 100px`.

1. Round 1: each gets 200; A violates its max → clamped to 100, **frozen**.
2. Round 2: 500 left for B and C → **250** each.

Result **100 / 250 / 250**, not 100/200/200 with 100px wasted — that's the step-5 loop.

Worked example 5 — factors below 1 (G7)

Container 600px, two items `flex: 0.25 1 100px`: free 400, factors sum 0.5 → only 200 handed out → **200 / 200**, 200px left empty (with `flex-grow: 1` each: 300 / 300).

Verified — the maths vs. Chrome

`code/02_flex_math.mjs` runs `flex_algorithm.mjs` with a per-round log (the core is the two distribution formulas above, then clamp-freeze-repeat). `code/03_flex_measure.mjs` then reads every item's `getBoundingClientRect().width` in headless Chrome and compares; for G5 and G6 it first measures content sizes — **min-content** (narrowest without overflow, roughly the longest word) and **max-content** (width with no line breaks) — on a hidden clone:

```
case   predicted (spec algorithm)          measured in Chrome                 diff
g1     175 / 300 / 125                     175 / 300 / 125                    0
g2     214.286 / 142.857 / 42.857          214.281 / 142.859 / 42.859         0.005
g3     165 / 280 / 115                     165 / 280 / 115                    0
g4     100 / 250 / 250                     100 / 250 / 250                    0
g5a    200 / 200 / 200                     200 / 200 / 200                    0
g5b    160.266 / 252.984 / 186.75          160.266 / 252.984 / 186.75         0
g6a    251.672 / 48.328                    251.672 / 48.328                   0
g6b    150 / 150                           150 / 150                          0
g7     200 / 200                           200 / 200                          0

content sizes used above (min-content / max-content):
  g5b 33.125 / 33.125  |  125.844 / 125.844  |  59.609 / 59.609
  g6a 251.672 / 251.672  |  6.625 / 6.625
```

Every case matches. The tiny G2 difference: Chrome stores layout positions in **1/64-pixel units** (214.2857… → 13714/64 = 214.28125), pushing rounding leftovers onto neighbours so the three still sum to exactly 400.

Common mistakes

- Expecting `flex-grow: 2` to be twice as wide as a `flex-grow: 1` sibling — true only with equal (typically 0) bases.
- Forgetting padding and border count toward the base: a `flex: 1` item with `padding: 0 40px` ends up 80px wider than an unpadded sibling.
- `flex-shrink: 0` on everything, then a row overflowing on phones.

Self-check

1. Container 500px, items `flex: 1 1 100px` and `flex: 3 1 100px`? → free 300, 4 shares of 75 → **175 and 325**.
2. Container 300px, items `flex: 0 1 200px` and `flex: 0 3 200px`? → scaled 200 and 600 (Σ 800); losses 25 and 75 → **175 and 125**.

8.4 The `flex` shorthand and minimum sizes

The shorthand

Prefer `flex` over the three longhands: it sets them together with sensible defaults for omitted parts (omitted `flex-shrink` → 1, omitted `flex-basis` → **0%**, not `auto`).

You write	Means (grow shrink basis)	Behaviour	Typical use
(nothing) / `flex: initial`	`0 1 auto`	Content-sized, may shrink, won't grow	Default
`flex: 1`	`1 1 0%`	Ignore content size, share **all** space equally	Equal columns, "fill the rest"
`flex: 2`	`2 1 0%`	Twice the share of a `flex: 1` sibling	Proportional columns
`flex: auto`	`1 1 auto`	Start from content size, share the **leftover** equally	Nav tabs that grow but keep content differences
`flex: none`	`0 0 auto`	Rigid: never grows or shrinks	Icons, avatars, buttons
`flex: 200px`	`1 1 200px`	Start at 200px, grow/shrink from there	Wrapping card rows
`flex: 0 0 200px`	exactly that	Fixed 200px	Sidebars

G5 shows `flex: 1` vs `flex: auto` with the same three labels: `flex: 1` gives 200/200/200 regardless of text; `flex: auto` starts from the text widths (33.1, 125.8, 59.6) and adds an equal 127.1px each → 160.3 / 253.0 / 186.8.

`min-width: auto` — the hidden minimum

Flex items have `min-width: auto` (`min-height: auto` in a column), which for flex items means **"never shrink below my min-content size"** — the longest word, URL or unshrinkable child. It protects text, but causes the most common flexbox bug: *"my `flex: 1` item overflows its container"*.

G6: two `flex: 1` items in 300px should be 150/150, but the first contains an unbreakable URL with min-content 251.672px, so the second gets only 48.328px. The fix:

```
.item { flex: 1; min-width: 0; }           /* allow shrinking below content size */
/* plus a way to deal with the overflowing content: */
.item { overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }  /* or overflow-wrap: anywhere */
```

Where it bites: a code block or table in a flex column, a long file name in a list row, a chart in a flex child, nested flex layouts with a wide `<pre>` (fix: `min-width: 0` on the item, `overflow: auto` on the `<pre>`). Making the item a scroll container (`overflow: hidden/auto/scroll`) also removes the automatic minimum; `overflow: clip` doesn't (not a scroll container).

8.5 Alignment

What it is

After sizing there may still be **free space** — on the main axis (items don't fill the line) and the cross axis (items shorter than the line). Alignment properties (from the CSS Box Alignment module, shared with Grid) decide where it goes.

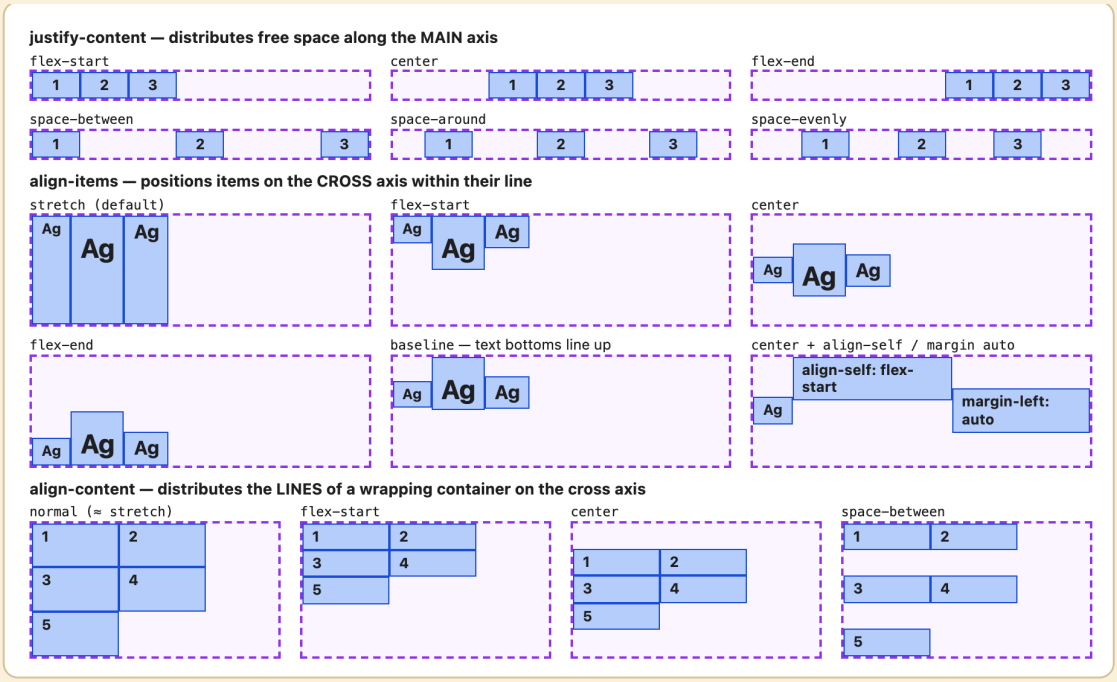


Figure 8.3 — `code/04_alignment.html`.

`justify-content` — main axis, on the container

Value	Where the free space goes
`flex-start` (effectively the default, `normal`)	All at the end; items packed at main-start
`flex-end`	All at the start
`center`	Half on each side
`space-between`	Between items only; first and last touch the edges
`space-around`	Equal space around each item, so edges get **half** a gap
`space-evenly`	Equal gaps everywhere, edges included

`start` / `end` follow the writing direction rather than the flex direction (matters only with `*-reverse`). `justify-content` does nothing if an item has `flex-grow` > 0 with no max-size — no free space remains.

`align-items` and `align-self` — cross axis, per line

Value	Effect on each item
`stretch` (default — `normal` behaves like it)	Items fill the line's cross size — **only if** their cross size (`height` in a row) is `auto` (a `height: 50px` item stays 50px)
`flex-start` / `start`	Top of the line (in a row)
`flex-end` / `end`	Bottom
`center`	Middle
`baseline`	Items shifted so each one's **first line of text** shares a baseline — for a label beside a big number or mixed font sizes

`align-items` sets the default; `align-self` overrides it per item. There is **no** `justify-self` in flexbox — the main axis is shared — use an **auto margin**.

Auto margins

`margin: auto` absorbs free space *before* `justify-content`, in either axis:

- `margin-left: auto` (or `margin-inline-start: auto`) pushes the item — and everything after — to the end ("logo and links left, Sign in far right").
- `margin-top: auto` on a column's last item pushes it to the bottom (a card's button).

- `margin: auto` on an only child centres it both ways.

`align-content` — distributing the lines

With `flex-wrap: wrap` there can be several **flex lines**; `align-content` distributes them on the cross axis (`flex-start`, `center`, `space-between`, `space-around`, `space-evenly`, `stretch`). Default `normal` behaves as `stretch`. **No effect on a single-line (`nowrap`) container.**

Common mistakes

- `align-items: center` to centre horizontally in a row (that's `justify-content`).
- A `height` on items, then wondering why `stretch` stopped.
- `align-content` on a non-wrapping container.
- `justify-content: center` on an overflowing row: content spills off *both* sides and the left part can't be scrolled to. `justify-content: safe center` falls back to start on overflow (Baseline since 2024 — Chrome 115, Firefox 63, Safari 17.6); or `margin: auto` on items, which never pushes content past the start.

8.6 Wrapping and `gap`

`flex-wrap`

Value	Behaviour
`nowrap` (default)	One line. Items shrink (if allowed), then **overflow**
`wrap`	New lines when the next item doesn't fit; lines stack along the cross axis
`wrap-reverse`	Same, but new lines go *above* (cross-start and cross-end swap)

`flex-flow: row wrap` = `flex-direction` + `flex-wrap`. Each line is sized **independently**: items are placed by *hypothetical* size (basis clamped by min/max) — as many as fit — then the 8.3 maths runs per line.

`gap`

`gap: <row-gap> <column-gap>` (or one value) puts fixed space **between** items and lines — never on the outer edges. Not a margin, never collapses, subtracted before free space is shared (G3). Baseline for flexbox since 2021 (before: item margins plus a negative container margin).

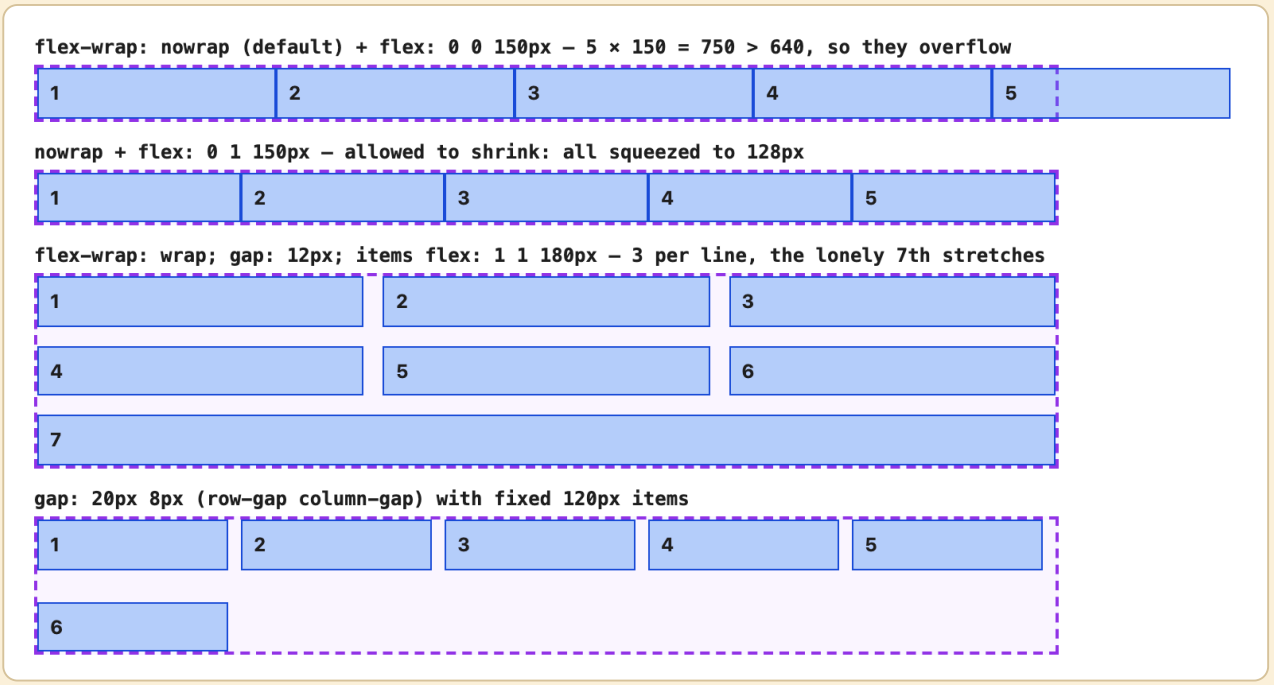


Figure 8.4 — `code/05_wrap_gap.html`.

`code/05_wrap_gap.mjs` groups cards by top edge to count items per line:

```
container 640px (flex: 1 1 180px, gap 12px):
  line 1: 3 item(s), widths 205.33, 205.34, 205.33
  line 2: 3 item(s), widths 205.33, 205.34, 205.33
  line 3: 1 item(s), widths 640
container 400px (flex: 1 1 180px, gap 12px):
  line 1: 2 item(s), widths 194, 194
  line 2: 2 item(s), widths 194, 194
  line 3: 2 item(s), widths 194, 194
  line 4: 1 item(s), widths 400
nowrap + flex: 0 1 150px in 640px: 128, 128, 128, 128, 128
```

By hand: 3 × 180 + 2 × 12 = 564 ≤ 640, but 4 × 180 + 3 × 12 = 756 > 640, so 3 fit; free 76 → +25.33 each → 205.33. The lone last card takes all 640px. Shrink row: overflow 750 − 640 = 110, equal weights → each loses 22 → 128.

The "last row" problem — and when to switch to grid

Wrapping flex is **content-out** (content drives sizes): each line shares its own free space, so items on a short last line grow wider than those above. Great for tags or buttons of different lengths; wrong for a uniform card grid. For equal wrapping columns use Grid's `repeat(auto-fill, minmax(180px, 1fr))` (Chapter 9), which is **layout-in**: tracks are defined first and items drop into them. (Flex workaround: `flex-grow: 0` with a fixed basis, at the cost of an uneven right edge.)

8.7 Common patterns

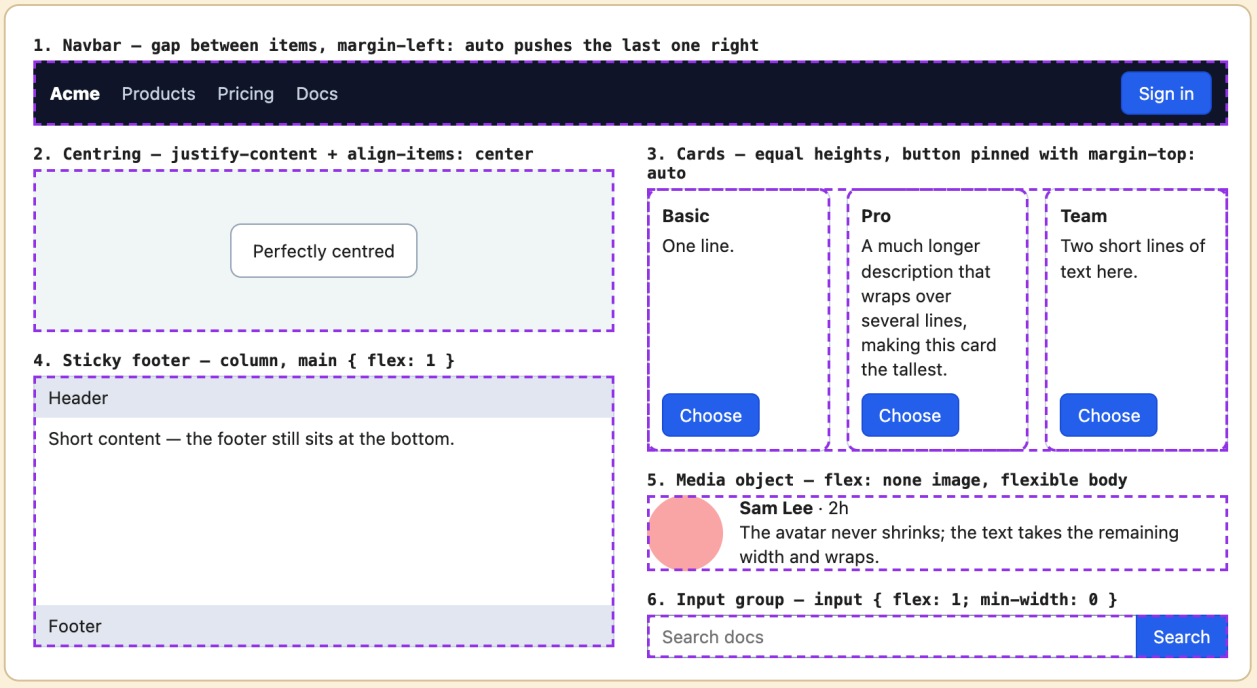


Figure 8.5 — `code/06_patterns.html`.

```css
/* 1. Navbar */
.nav       { display: flex; align-items: center; gap: 16px; }
.nav .push { margin-left: auto; }                    /* everything from here on goes right */

/* 2. Centre anything, both axes */
.center { display: flex; justify-content: center; align-items: center; }
/* (grid alternative: display: grid; place-items: center) */

/* 3. Equal-height cards with buttons aligned at the bottom */
.cards       { display: flex; gap: 12px; }            /* align-items: stretch -> equal heights */
.card        { flex: 1; display: flex; flex-direction: column; }
.card button { margin-top: auto; align-self: flex-start; }

/* 4. Sticky footer: footer at the bottom even when content is short */
body      { display: flex; flex-direction: column; min-height: 100dvh; }   /* demo uses 200px */
body main { flex: 1; }

/* 5. Media object: fixed image, flexible text */
.media         { display: flex; gap: 12px; align-items: flex-start; }
.media .avatar { flex: none; width: 56px; height: 56px; }
.media .body   { flex: 1; min-width: 0; }

/* 6. Input group */
.group       { display: flex; }
.group input { flex: 1; min-width: 0; }
```

Measured by `code/06_patterns.mjs`:

```
navbar : gap nav-right-edge minus button-right-edge = 12px (the 12px padding)
centre : space left/right 144.65625 / 144.65625, top/bottom 39.90625 / 39.90625
cards  : heights 193.5, 193.5, 193.5; button bottoms 308.6875, 308.6875, 308.6875
footer : page bottom 464.1875, footer bottom 463.1875 (only the 1px border below it)
media  : avatar width 56px (flex: none, never shrinks)
input  : input 359.875px + button 68.125px = form 428px
```

- **Navbar**: in production a real `<nav>` with a list; the `<ul>` is the flex container (Chapter 3). Add `flex-wrap: wrap` or a menu button for narrow screens (Chapter 10).
- **Centring**: works for unknown sizes on both axes. If the child can be taller than the container, prefer `margin: auto` on it so it can't overflow off the top.
- **Cards**: equal heights from `align-items: stretch`; column + `margin-top: auto` aligns buttons. Aligning *titles and paragraphs* across cards needs `subgrid` (Chapter 9).
- **Sticky footer** (footer at the bottom of short pages): `100dvh` = *dynamic* viewport height, accounting for mobile toolbars; `100vh` can be too tall on phones (Chapter 10).
- **Media object** (image beside text): `flex: none` keeps the avatar unsquashed; `min-width: 0` lets long names truncate.
- **Input group** (text box joined to a button): `<input>` has a default width from its `size` attribute, so without `min-width: 0` it may refuse to shrink.

Real-world examples

GitHub's repository header, most navbars, Bootstrap 5's `.d-flex`/`.justify-content-between` and Tailwind's `flex`, `items-center`, `justify-between`, `gap-4` (utility classes, one property each) map one-to-one to this chapter's properties.

8.8 Pitfalls, debugging, and interview prep

Pitfalls at a glance

Symptom	Cause	Fix
Item with `flex: 1` overflows / text doesn't truncate	`min-width: auto` = min-content	`min-width: 0` (and `overflow` / `text-overflow` on the content)
Image in a flex row is squashed	Images shrink like any item	`flex: none` or `flex-shrink: 0`
Image in a row becomes tall and distorted or overflows	`align-items: stretch` stretches it to the line (a 100×50 image in a 300px-tall row became 600×300 in Chrome)	`align-self: flex-start` (or `center`)
`align-items: stretch` has no effect	Item has an explicit `height`	Remove it or use `min-height`
`height: 100%` on an item's child doesn't work	Parent's height isn't definite (fixed in advance)	Make the item a flex container too, or `flex: 1` down the chain
`justify-content` does nothing	An item has `flex-grow`, so no free space remains	Remove the grow, or use `max-width`
Last row of wrapped cards is wider	Per-line sizing	Use grid
Tab order doesn't match what users see	`order` / `*-reverse`	Change the HTML instead
Unexpected space between items	`justify-content: space-*` or leftover `margin`	Inspect in DevTools

Debugging with DevTools

Chrome, Edge and Firefox show a **flex badge** beside flex containers in the Elements panel (F12); clicking it overlays container, items, free space and gaps. The **Flexbox editor** (icon by `display: flex` in Styles) clicks through `justify-content` / `align-items` values. Firefox's Layout panel explains *why* an item has its size: base size, growth/shrink, and any min/max clamp.

In the interview

30-second answer: "Flexbox is one-dimensional layout. `display: flex` makes direct children flex items, placed along a main axis set by `flex-direction`; the cross axis is perpendicular. `justify-content` distributes space on the main axis, `align-items` / `align-self` align on the cross axis, and `align-content` distributes wrapped lines. Sizes start from `flex-basis`; positive free space is shared by `flex-grow` ratios and overflow taken back in proportion to `flex-shrink` × basis, with min/max clamps. `flex: 1` means `1 1 0%`, so items share all the space equally. The classic bug is `min-width: auto`, which stops items shrinking below their content — fix with `min-width: 0`."

- *`flex: 1` vs `flex: auto`?* → Basis 0% vs auto: `flex: 1` ignores content size and splits everything; `flex: auto` keeps content differences and splits only the leftover (G5).
- *How is shrinking calculated?* → Overflow × (shrink × basis) / Σ(shrink × basis) — weighted by basis so small items don't vanish (G2: 214.29 / 142.86 / 42.86).
- *Flexbox or grid for a page layout?* → Grid for 2-D page structure and uniform card grids; flexbox inside components for content-sized rows. Often nested.
- *How do you centre a div?* → `display: flex; justify-content: center; align-items: center` on the parent, or `display: grid; place-items: center`.

Chapter 8 TL;DR

- `display: flex` → direct children become **flex items**; no margin collapsing, floats ignored, items blockified.

- **Main axis** = `flex-direction` (`row` default). `justify-content` → main axis. `align-items` / `align-self` / `align-content` → cross axis. Swap direction and the meanings swap.
- `order` and `*-reverse` change only visuals, not Tab or reading order (`reading-flow` is Chromium-only, not Baseline).
- **Sizing**: start from `flex-basis`; positive free space split by `flex-grow` shares (`basis + free × grow/Σgrow`); overflow taken back by `flex-shrink × basis` weights; clamp to min/max, freeze, repeat. Verified: 175/300/125 (grow), 214.29/142.86/42.86 (shrink), 100/250/250 (max-width freeze). Chrome stores sizes in 1/64 px.
- **Shorthand**: `flex: 1` = `1 1 0%`, `auto` = `1 1 auto`, `none` = `0 0 auto`, default `0 1 auto`, `flex: 200px` = `1 1 200px`.
- `min-width: auto` = min-content for flex items → overflowing `flex: 1` items; fix with `min-width: 0`.
- **Auto margins** absorb free space: `margin-left: auto` pushes right, `margin-top: auto` pushes down, `margin: auto` centres.
- **Wrap + gap**: lines are sized independently; `gap` only between items; for an even last row use grid.
- **Patterns**: navbar, centring, equal-height cards with bottom-aligned buttons, sticky footer (`min-height: 100dvh` + `main { flex: 1 }`), media object, input group.

Next: Chapter 9 — CSS Grid.

Chapter 9 — CSS Grid

Flexbox (Chapter 8) arranges items along **one** axis, sized by their content. **CSS Grid** defines **rows and columns at the same time**, then places items into the cells — or lets the browser place them. It is the tool for page structure, card galleries, dashboards, forms with aligned labels, and anything a designer drew on a grid. Grid has worked in every major browser since 2017 (Baseline widely available); `subgrid` is Baseline since September 2023.

In this chapter: 9.1 Grid vocabulary · 9.2 Defining tracks and the `fr` maths · 9.3 `repeat()`, `auto-fill` vs `auto-fit` (responsive grids without media queries) · 9.4 Placing items (lines, `span`, named lines, `grid-template-areas`, overlap) · 9.5 The implicit grid and auto-placement · 9.6 Alignment · 9.7 Subgrid · 9.8 Grid vs Flexbox · 9.9 Patterns · 9.10 Debugging, pitfalls and interview prep.

As in Chapters 7–8, every example is a page in `code/` with a screenshot in `img/`: containers have a **dashed purple border**, items a **solid blue outline**, **red badges** mark numbered lines, **red numbers** are track sizes read back from the browser. The `.mjs` scripts read `getComputedStyle(grid).gridTemplateColumns` — the **resolved** track sizes in pixels, the browser's own answer — and compare them with the hand calculation.

9.1 Grid vocabulary

What it is

`display: grid` (or `inline-grid`) makes a **grid container**; its direct children become **grid items**. The container defines an invisible grid (like a spreadsheet whose columns you size with maths):

Term	Meaning
Grid line	A dividing line between tracks, numbered from **1** at the start edge and backwards from **−1** (the last line) at the end edge. 3 columns → 4 column lines (`1fr 1fr 1fr` → 5).
Track	The space between two adjacent lines: a **column** or a **row**.
Cell	One row × one column — the smallest unit.
Grid area	Any rectangle of cells bounded by four lines (can be named).
Gutter / gap	Space between tracks (`gap`, `row-gap`, `column-gap`) — only *between* tracks, never at the edges.
Explicit grid	The tracks you declared with `grid-template-*`.
Implicit grid	Extra tracks the browser creates when items are placed outside the explicit grid or outnumber its cells.

Grid behaviours to know up front

Like flex items, grid items are blockified, `float` is ignored, margins don't collapse, and `z-index` works without `position` (Chapter 7). Grid is **layout-in** (container tracks first, items fall into them); flexbox is **content-out** (items' sizes drive layout) — the key to 9.8.

9.2 Defining tracks

What it is

`grid-template-columns` / `grid-template-rows` list each track's size, space-separated:

Value	Meaning
`200px`, `10rem`, `30%`	Fixed size (percent of the container)
`fr`	A **fraction of the leftover space** after fixed tracks and gaps
`auto`	As a maximum: content size, but stretches to fill leftover space when there are no `fr` tracks (with `align/justify-content: normal`). As a minimum: the item's automatic minimum (usually min-content, so content isn't crushed)
`min-content`	As narrow as the content can get (longest word)
`max-content`	As wide as the content wants, no wrapping
`fit-content(150px)`	`max-content`, capped at 150px (and at least `min-content`) — never grows beyond content or cap
`minmax(min, max)`	Any size between min and max, e.g. `minmax(200px, 1fr)`

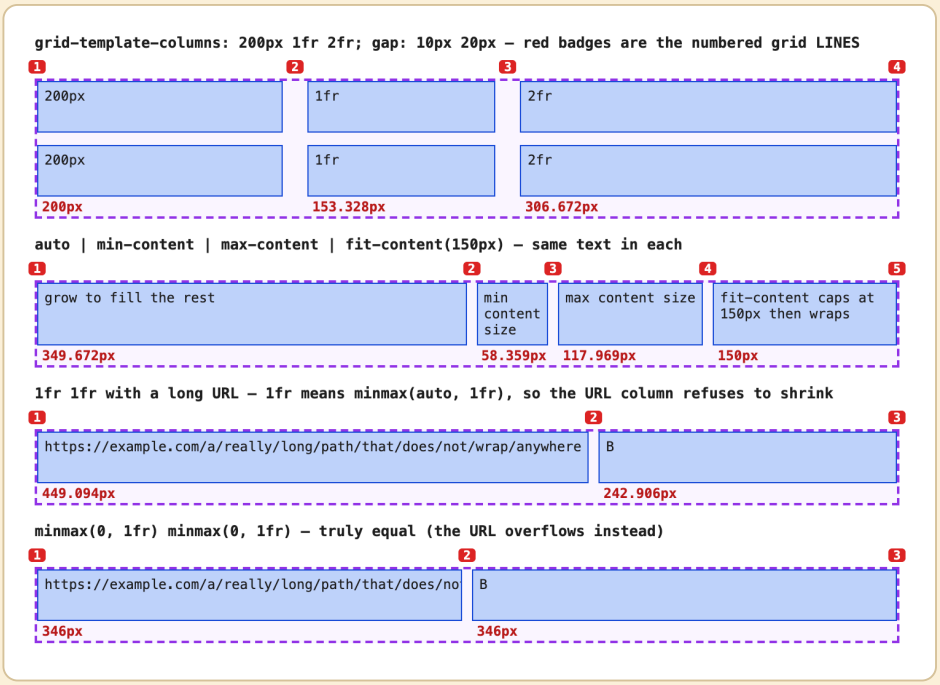


Figure 9.1 — `code/01_tracks.html`. Line numbers and sizes are drawn by a script from the resolved track list (absolutely positioned, so layout is unaffected).

How `fr` is calculated

1. Start with the container's inner size (700px).
2. Subtract every **fixed** track and gap: `200px 1fr 2fr` with `column-gap: 20px` → 700 − 200 − 2 × 20 = **460px**.
3. Divide by Σ`fr` (1 + 2 = 3): **1fr = 153.33px**, 2fr = 306.67px. (Container 1000px, `250px 1fr 1fr 1fr`, 10px gaps → (1000 − 250 − 30)/3 = 240px.)
4. A track whose share would fall below its minimum is fixed at that minimum and the rest re-split (like flexbox's freeze loop, §8.3).

`code/01_tracks.mjs` prints each declared value beside what Chrome resolved:

```
container content width: 700px
t1  200px 1fr 2fr                                   -> 200px 153.328px 306.672px
t2  auto min-content max-content fit-content(150px)  -> 349.672px 58.3594px 117.969px 150px
t3  1fr 1fr                                         -> 449.094px 242.906px
t4  minmax(0px, 1fr) minmax(0px, 1fr)               -> 346px 346px

by hand, t1: 1fr = 460 / 3 = 153.333px, 2fr = 306.667px (Chrome stores 1/64 px: 153.328125, 306.671875)
```

The tiny differences are Chrome's 1/64-pixel layout units, as in flexbox.

`fr` vs `%`

`%` tracks share the *whole* container and ignore gaps, so `50% 50%` plus a gap overflows (a horizontal scrollbar). `fr` shares only what's left after gaps.

The `1fr` minimum trap (t3 vs t4)

`1fr` is shorthand for **`minmax(auto, 1fr)`**, and the `auto` minimum means "never narrower than the item's min-content". In t3 a long unbreakable URL forces its column to 449px, leaving 242.9px — unequal. It's the grid twin of flexbox's `min-width: auto` (§8.4).

Fix: `minmax(0, 1fr)` (t4) lets the track shrink below its content — exactly 346px each — and handle the overflow with `overflow`, `overflow-wrap: anywhere` or `min-width: 0` on the item. For columns that **must** be equal (dashboards, code-and-preview splits): `repeat(3, minmax(0, 1fr))`.

Common mistakes

- `%` tracks with `gap` → horizontal scrollbar.
- Expecting `1fr 1fr` to be equal whatever the content.
- Forgetting rows default to `auto` — a row only exceeds its content height if sized, or if the container has a height and `fr` rows.

9.3 `repeat()`, `auto-fill` and `auto-fit`

What it is

`repeat(count, tracks)`: `repeat(4, 1fr)` = `1fr 1fr 1fr 1fr`; `repeat(2, 100px 1fr)` = `100px 1fr 100px 1fr`. The count can instead be **`auto-fill`** or **`auto-fit`** — "as many as fit". With `minmax()` this gives a responsive grid with **no media queries** (Chapter 10), the RAM pattern (**R**epeat, **A**uto, **M**inmax):

```
.gallery { display: grid; gap: 10px;
           grid-template-columns: repeat(auto-fill, minmax(120px, 1fr)); }
```

How it works

1. Tracks of the *minimum* size that fit: **`n = floor((width + gap) / (min + gap))`**.
2. n tracks are created; the `1fr` maximum stretches them to share the leftover.
3. Items auto-place; extras wrap onto new rows.
4. The difference shows only with **fewer items than tracks**:
    - `auto-fill` keeps the empty tracks — items keep their one-column width, space reserved on the right.
    - `auto-fit` **collapses** empty tracks (and their gaps) to 0, so items stretch to fill the row.

Auto-repeat needs a definite size (a fixed length or a `minmax` with a fixed minimum) to count tracks, so `repeat(auto-fill, 1fr)` is invalid.

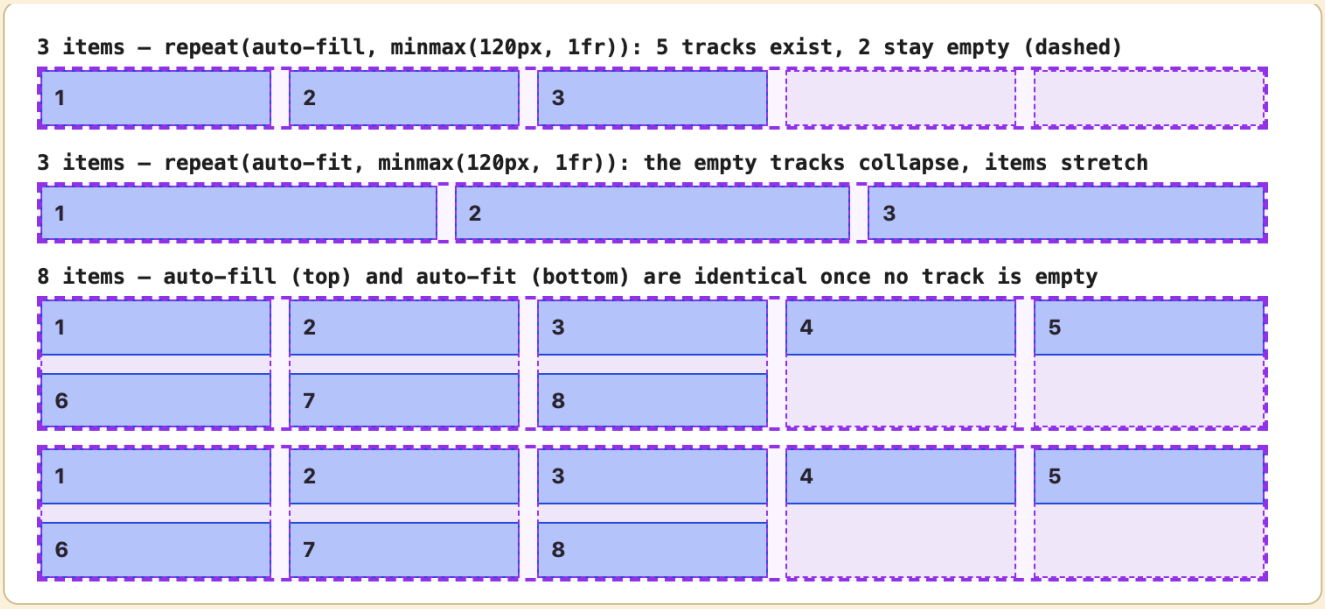


Figure 9.2 — `code/02_auto_fill_fit.html`. Every track, including empty ones, is outlined in dashed purple.

Output of `code/02_auto_fill_fit.mjs` (the four identical 400px lines merged):

```
container 700px, gap 10px, minmax(120px, 1fr):
  fill3  132px 132px 132px 132px 132px
  fit3   226.656px 226.672px 226.672px 0px 0px
  fill8  132px 132px 132px 132px 132px
  fit8   132px 132px 132px 132px 132px
container 400px, gap 10px, minmax(120px, 1fr):
  fill3, fit3, fill8, fit8  (all identical)  126.656px 126.672px 126.656px
by hand, 700px: n = floor(710 / 130) = 5, track = (700 - 40) / 5 = 132.000px
by hand, 400px: n = floor(410 / 130) = 3, track = (400 - 20) / 3 = 126.667px
by hand, auto-fit with 3 items at 700px: empty tracks and their gaps collapse -> (700 - 20) / 3 = 226.667px
```

In `fit3` the collapsed tracks appear as `0px`.

Which one to use

Want	Use
Cards keep a consistent size even with only one or two	`auto-fill`
A few items should stretch to fill the row	`auto-fit`

Guarding the minimum on narrow screens

If the container is narrower than the minimum (120px track in a 100px column), the single track overflows. Write `min(120px, 100%)`: `repeat(auto-fit, minmax(min(160px, 100%), 1fr))` (used in 9.9).

Self-check

1. Container 1000px, gap 20px, `repeat(auto-fill, minmax(200px, 1fr))`? → floor(1020/220) = 4 columns of (1000 − 60)/4 = **235px**. With 2 items and `auto-fit`? → (1000 − 20)/2 = **490px** each.

9.4 Placing items

By default each item takes the next free cell (9.5). To place one, name its **start and end lines** on each axis.

Line-based placement

Property	Example	Meaning
`grid-column-start` / `grid-column-end`	`1` / `3`	Start at column line 1, end at 3 (2 columns)
`grid-column` (shorthand)	`1 / 3`	Same
`span n`	`grid-column: span 2`	2 tracks wide, wherever auto-placement puts it
Negative lines	`grid-column: 1 / -1`	First to **last** line: full width
`grid-row`	`1 / span 2`	From row line 1, 2 rows tall
`grid-area` (4 values)	`3 / 2 / 4 / 5`	**row-start / column-start / row-end / column-end** (anticlockwise from the top, unlike `margin`)

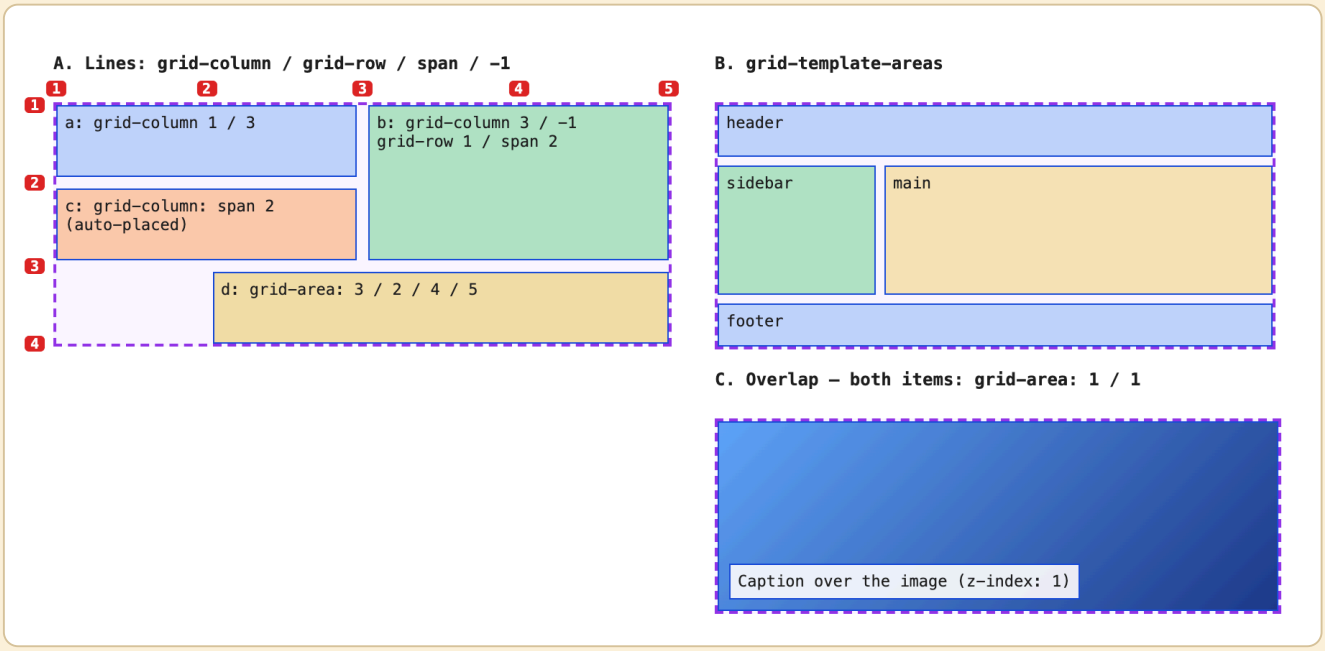


Figure 9.3 — `code/03_placement.html`.

```
#lines    { grid-template-columns: repeat(4, 1fr); grid-template-rows: repeat(3, 50px); gap: 8px; }
#lines .a { grid-column: 1 / 3; }
#lines .b { grid-column: 3 / -1; grid-row: 1 / span 2; }
#lines .c { grid-column: span 2; }            /* only a span: auto-placed into the first free spot */
#lines .d { grid-area: 3 / 2 / 4 / 5; }
```

`code/03_placement.mjs` maps each item back to line numbers:

```
a: columns 1 -> 3, rows 1 -> 2
b: columns 3 -> 5, rows 1 -> 3
c: columns 1 -> 3, rows 2 -> 3
d: columns 2 -> 5, rows 3 -> 4
```

`-1` resolved to line 5. Negative numbers count from the end of the **explicit** grid only, so `1 / -1` doesn't reach implicit columns.

Named lines

Name lines in square brackets and place by name — robust when numbers change:

```
.layout { grid-template-columns: [full-start] 1fr [content-start] minmax(0, 60rem) [content-end] 1fr [full-end]; }
.layout > *      { grid-column: content; }   /* "content" = content-start / content-end */
.layout > .bleed { grid-column: full; }      /* a full-width image or banner */
```

This **full-bleed** layout keeps text in a readable centre column while chosen elements run edge to edge.

`grid-template-areas`

The most readable page-layout tool: draw the layout as ASCII art, one string per row, and assign items by name.

```
#areas { grid-template-columns: 110px 1fr;
         grid-template-rows: 36px 90px 30px;
         grid-template-areas:
           "header  header"
           "sidebar main"
           "footer  footer"; }
.h { grid-area: header; } .s { grid-area: sidebar; } .m { grid-area: main; } .f { grid-area: footer; }
```

- Every string has the same number of cells.
- A name repeated in adjacent cells makes one area; each area must be a **rectangle** (`"a a" "a b"` is invalid — L-shaped).
- `.` (or `...`) marks an empty cell.
- Areas create named lines `header-start`/`header-end`, so `grid-column: header` works.
- Re-laying out at a breakpoint (a width where a media query switches layout) is just rewriting the strings (9.9).

Overlap

Items may share cells; later ones paint on top, and `z-index` changes that. Demo C puts an image and caption in `grid-area: 1 / 1` and aligns the caption bottom-left with `align-self: end; justify-self: start` — a hero banner with no `position: absolute`, so the container still grows with its content.

Accessibility note

As with flexbox `order`, placement doesn't change DOM order; keyboard and screen-reader users follow the source (§8.2). Keep source order logical.

9.5 The implicit grid and auto-placement

What it is

More items than explicit cells, or an item placed past the explicit grid, makes the browser add **implicit tracks** sized by `grid-auto-rows` / `grid-auto-columns` (default `auto` = content). E.g. `repeat(3, 1fr)` with 7 items and no rows → 3 implicit rows.

Auto-placement fills cells in order; `grid-auto-flow` sets how:

Value	Behaviour
`row` (default)	Fill each row left to right, then add rows
`column`	Fill each column top to bottom, then add columns
`row dense` / `column dense`	Also go **back** to fill earlier holes with later items that fit

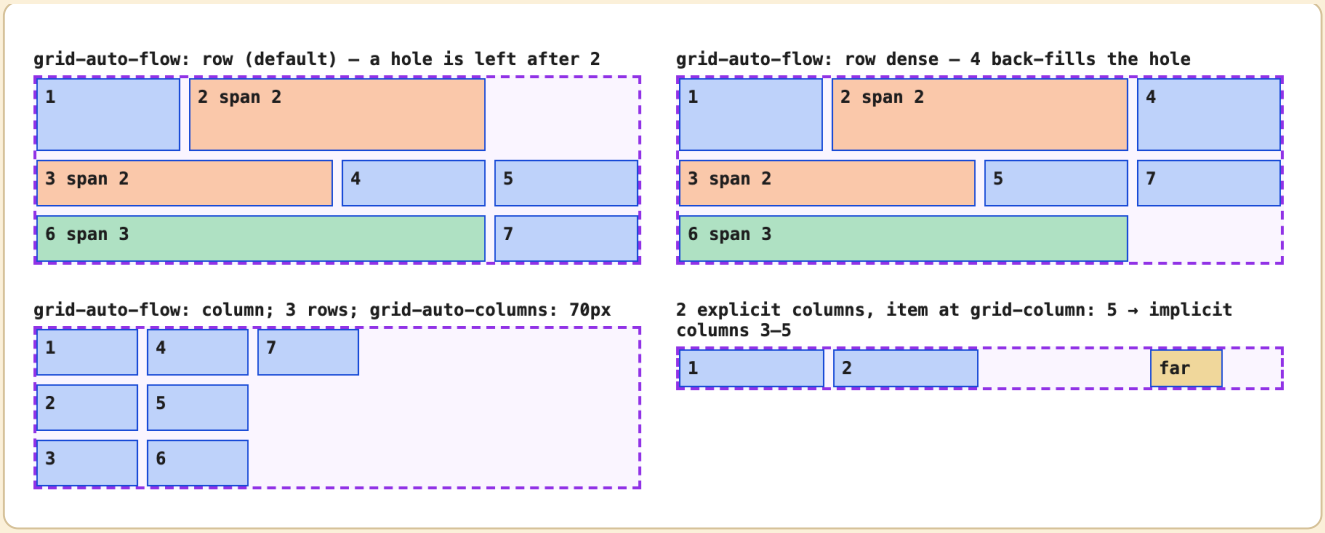


Figure 9.4 — `code/04_implicit.html`. Orange items span 2 columns, green spans 3.

```
.flow   { grid-template-columns: repeat(4, 1fr); grid-template-rows: 50px; grid-auto-rows: 32px; }
.dense  { grid-auto-flow: row dense; }
.by-col { grid-template-rows: repeat(3, 32px); grid-auto-flow: column; grid-auto-columns: 70px; }
.outside      { grid-template-columns: repeat(2, 100px); grid-auto-columns: 50px; }
.outside .far { grid-column: 5; }
```

Output of `code/04_implicit.mjs` (`r2c3` = row 2, column 3):

```
row      columns: 99px 99px 99px 99px                        rows: 50px 32px 32px
         placement: 1@r1c1 2@r1c2 3@r2c1 4@r2c3 5@r2c4 6@r3c1 7@r3c4
dense    columns: 99px 99px 99px 99px                        rows: 50px 32px 32px
         placement: 1@r1c1 2@r1c2 3@r2c1 4@r1c4 5@r2c3 6@r3c1 7@r2c4
bycol    columns: 70px 70px 70px                             rows: 32px 32px 32px
         placement: 1@r1c1 2@r2c1 3@r3c1 4@r1c2 5@r2c2 6@r3c2 7@r1c3
outside  columns: 100px 100px 50px 50px 50px                 rows: 26px
         placement: 1@r1c1 2@r1c2 far@r1c5
```

- **`row`**: item 3 needs two columns; only column 4 is free in row 1, so it drops to row 2, leaving **`r1c4` empty**. Row 1 is explicit (50px); rows 2–3 implicit (32px).
- **`dense`**: item 4 goes back to fill r1c4 — items now draw **out of source order**.
- **`bycol`**: three explicit rows, columns created as needed at 70px.
- **`outside`**: placing at column line 5 creates implicit columns 3–5 at 50px.

Useful settings

- `grid-auto-rows: minmax(100px, auto)` — at least 100px, growing with content; standard for card grids.
- `dense` suits galleries where order doesn't matter; avoid it where reading or tab order does.

9.6 Alignment

Grid uses the same Box Alignment properties as flexbox (§8.5), but **both axes have `justify-*` and `align-*`**, because items sit in 2-D cells:

- **inline axis** (row direction, left→right in English) → `justify-*`
- **block axis** (column direction, top→bottom) → `align-*`

Property	On	Aligns	Default
`justify-items` / `align-items`	container	every item **inside its cell**	`stretch` (unless the item has a set size or an aspect ratio, e.g. images → `start`)
`justify-self` / `align-self`	item	one item inside its cell (`place-self: center` centres just it)	inherits from `*-items`
`justify-content` / `align-content`	container	the **whole grid of tracks** in the container, when tracks are smaller than it	`normal` (tracks packed at start; `auto` tracks stretch)
`place-items`, `place-self`, `place-content`	—	shorthands: `align` value first, then `justify` (`place-items: end start` = align end, justify start)	—

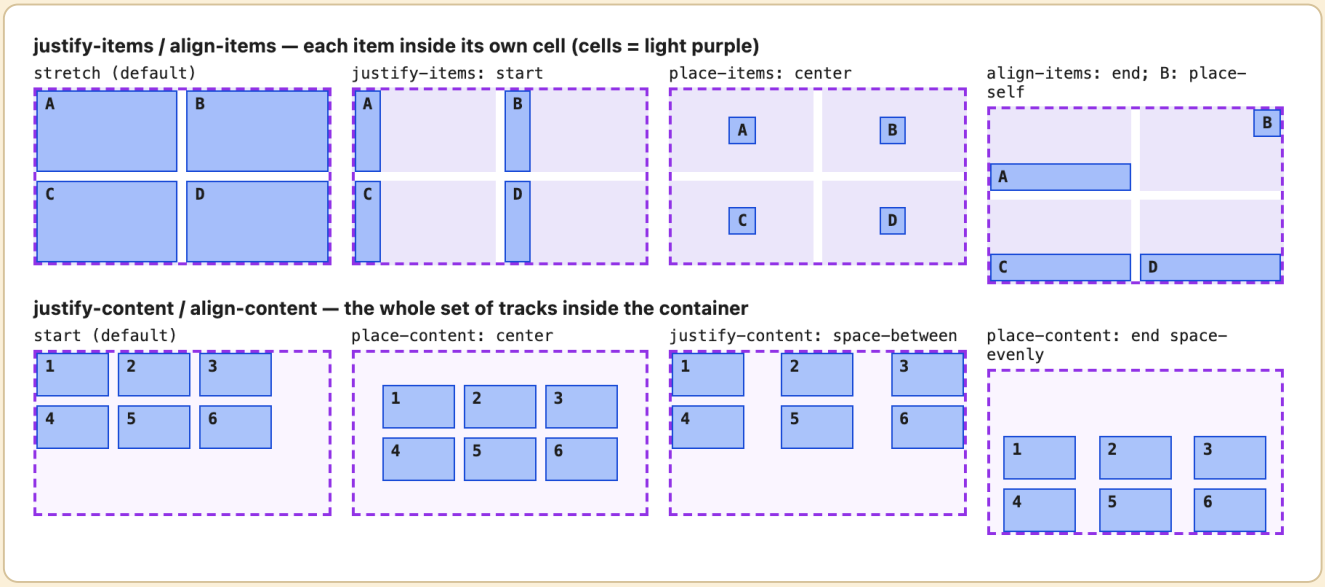


Figure 9.5 — `code/05_alignment.html`. Light-purple rectangles are the cells, showing each item's position inside its cell.

- `*-content` **only acts when there's free space** (fixed tracks smaller than the container); with `fr` tracks it does nothing.
- `space-between` / `-around` / `-evenly` in `*-content` **widen the effective gaps**; spanning items grow too.
- Auto margins work: `margin-left: auto` pushes an item to the end of its cell.

9.7 Subgrid

The problem it solves

A grid item that is itself a grid has its *own* tracks. Three pricing cards each lay out title, text and button, but card 2's longer title pushes its text lower — rows don't line up across cards. Before subgrid: fixed heights or JavaScript.

What it is

`grid-template-rows: subgrid` (or `-columns`) on a grid item spanning several parent tracks says: **don't create your own tracks on this axis — reuse the parent's tracks I sit on**. The card's children then join the parent's row sizing, so the tallest title in *any* card sets the title row for *all*. Defined in CSS Grid Level 2; **Baseline since September 2023** (Firefox 71, Safari 16, Chrome/Edge 117), widely available by 2026.

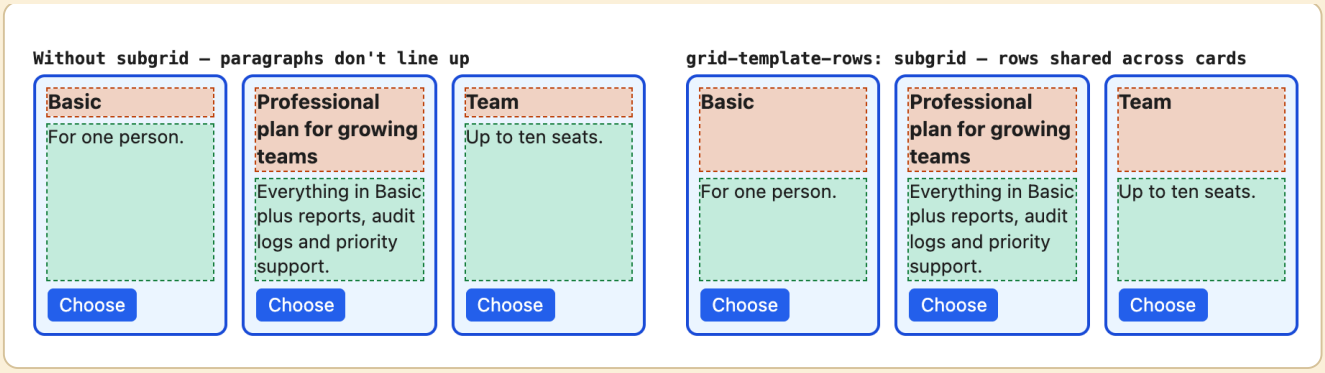


Figure 9.6 — `code/06_subgrid.html`. Orange dashed = title, green dashed = paragraph.

```
.sub      { display: grid; grid-template-columns: repeat(3, 1fr);
            grid-template-rows: auto auto auto; row-gap: 10px; }  /* the parent owns 3 rows */
.sub .card { display: grid; grid-row: span 3;                      /* each card covers all 3 */
            grid-template-rows: subgrid; row-gap: 6px; }           /* ...and reuses them */
```

Measured by `code/06_subgrid.mjs` (top edges of each paragraph and link):

```
plain  <p> tops: 76.9, 114.7, 76.9    link tops: 190.9, 190.9, 190.9
sub    <p> tops: 114.7, 114.7, 114.7  link tops: 190.9, 190.9, 190.9
```

Without subgrid the links happen to align (each card's middle row is `1fr` and cards stretch to equal height) but paragraphs don't; with subgrid every row is shared.

Details

- The subgrid item must **span** the parent tracks it shares (`grid-row: span 3`).
- It inherits the parent's gaps but may override them (`row-gap: 6px`).
- You can subgrid one axis and define your own tracks on the other.
- Parent line names are available inside.
- Uses: card rows, forms whose labels and inputs sit in separate `<fieldset>`s (Chapter 5), lists whose items have aligned columns (a `<ul>` of rows with `grid-template-columns: subgrid`).

9.8 Grid vs Flexbox

Question	Flexbox	Grid
Dimensions	One — a row **or** a column	Two — rows **and** columns
Who decides sizes?	**Content-out**: items' content and flex factors	**Layout-in**: the container's track definitions
Items in different rows line up in columns?	No — each flex line is independent	Yes — that's the point
Wrapping	Each line sizes itself; a lonely last item stretches	Items drop into the same columns; last row keeps the column width
Overlapping items	Needs negative margins or positioning	Place two items in the same cell
Gaps	`gap`	`gap`
Reordering	`order`, `*-reverse`	Line/area placement, `order`, `dense`
Typical jobs	Navbars, toolbars, button groups, tags, media objects, centring one thing, "push the last item right"	Page layouts, galleries, card grids, dashboards, form layouts, anything designed on a grid
Aligning content across siblings	Not possible	`subgrid`

Content-driven row of different-sized things → flexbox; things must line up both ways, or track sizes known up front → grid; usually both, nested (grid for the page and gallery, flexbox in each card's footer).

30-second answer: "Flexbox is one-dimensional and content-out: items share space by content and flex factors. Grid is two-dimensional and layout-in: I define rows and columns and place items, so things align across rows. Grid for page structure and card grids, flexbox for components like navbars and button groups; often nested."

9.9 Patterns

Holy grail layout (header, nav, main, aside, footer)

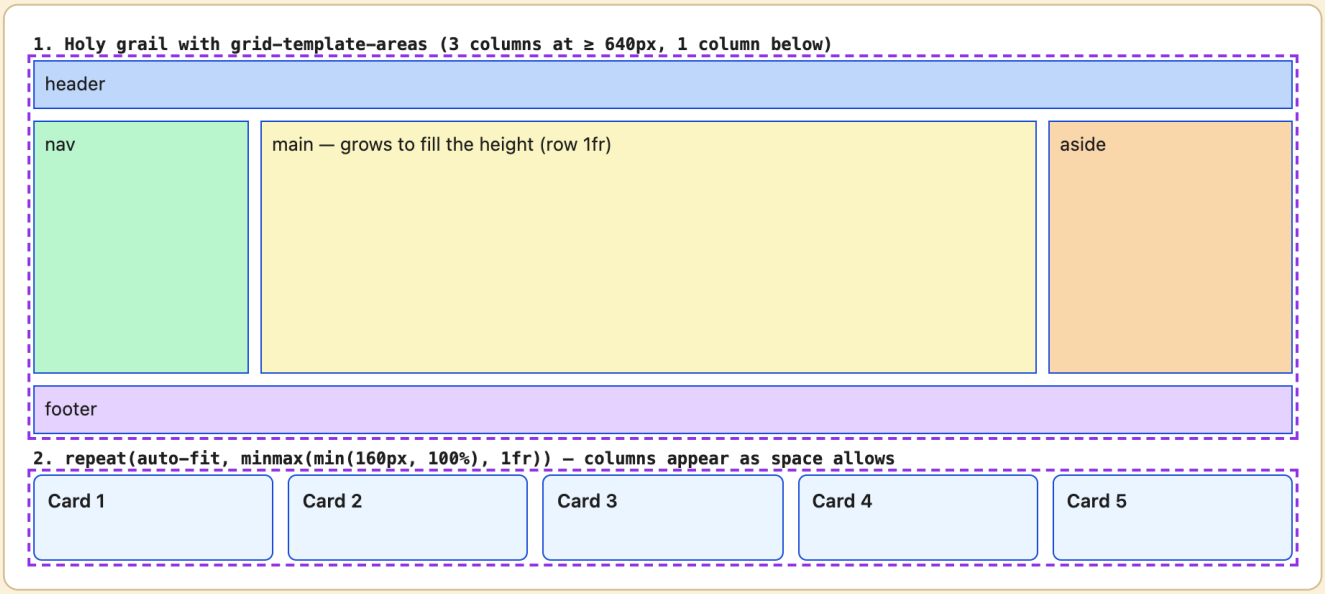
```
/* mobile first: one column, in source order */
.page { display: grid; gap: 8px; min-height: 100dvh;          /* demo uses 260px */
        grid-template-areas: "header" "nav" "main" "aside" "footer";
        grid-template-rows: auto auto 1fr auto auto; }
.page > header { grid-area: header; }  .page > nav    { grid-area: nav; }
.page > main   { grid-area: main; }    .page > aside  { grid-area: aside; }
.page > footer { grid-area: footer; }

/* three columns when there's room */
@media (min-width: 640px) {
  .page { grid-template-columns: 150px 1fr 170px;
          grid-template-rows: auto 1fr auto;
          grid-template-areas:
            "header header header"
            "nav    main   aside"
            "footer footer footer"; }
}
```

The `1fr` row gives `main` all spare height, so the footer sits at the screen bottom (the §8.7 sticky footer, done with grid). Media queries: Chapter 10.

Responsive card grid (RAM)

```
.cards { display: grid; gap: 10px;
         grid-template-columns: repeat(auto-fit, minmax(min(160px, 100%), 1fr)); }
```



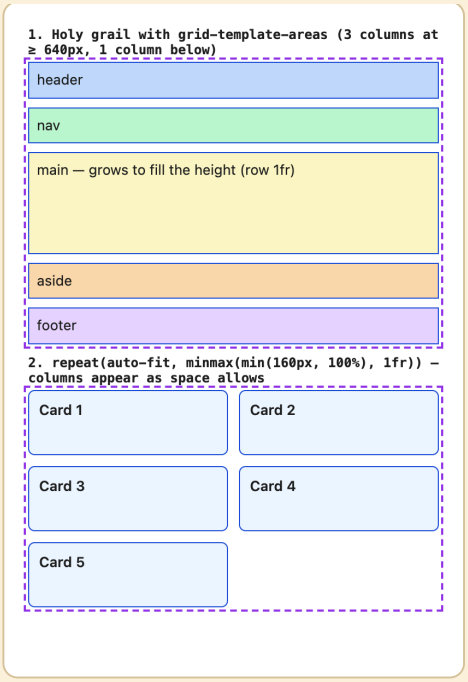


Figure 9.7 — `code/07_patterns.html` at 900px and 400px wide.

`code/07_patterns.mjs`:

```
viewport 900px
  holy grail: 150px 540px 170px
  cards     : 167.188px 167.203px 167.203px 167.203px 167.188px
viewport 400px
  holy grail: 376px
  cards     : 183px 183px
```

At 900px the container is 876px: 876 − 150 − 170 − 2 × 8 = 540 for `main`; floor(886/170) = 5 card columns. At 400px the media query no longer matches (one 376px column) and 2 card columns fit; card 5 keeps the column width on its own row — what flex-wrap couldn't do (§8.6).

Other patterns worth knowing

Pattern	Grid recipe
Centre anything	`display: grid; place-items: center`
Overlay / hero caption	Image and caption both `grid-area: 1 / 1`; align the caption with `align-self` / `justify-self`
Full-bleed article	Named lines `[full-start] 1fr [content-start] minmax(0, 60rem) [content-end] 1fr [full-end]` (9.4)
Form with aligned labels	`grid-template-columns: max-content 1fr`, one label + input per row
Sidebar that collapses	`grid-template-columns: fit-content(250px) minmax(0, 1fr)`
12-column design-system grid	`repeat(12, minmax(0, 1fr))` + `grid-column: span 4` on items

Masonry: not yet

Pinterest-style "masonry" (columns of uneven-height items) is being standardised in CSS Grid Level 3 as `display: grid-lanes`. As of 2026 it shipped in Safari (26.4) and is behind flags in Chrome and Firefox — not Baseline. Use CSS multi-column (`columns: 3`) or a JS library, and wrap experiments in `@supports (display: grid-lanes)` (a feature query).

9.10 Debugging, pitfalls and interview prep

Debugging

All major browsers show a **grid badge** beside grid containers in DevTools' Elements/Inspector panel (F12); toggling it overlays positive and negative line numbers, track sizes, gaps and area names. Firefox's grid inspector pioneered this and is still the most detailed.

Pitfalls at a glance

Symptom	Cause	Fix
`1fr` columns are unequal / grid overflows on mobile	`1fr` = `minmax(auto, 1fr)`; long words, `<pre>`, images or tables set the minimum	`minmax(0, 1fr)`; `min-width: 0` or `overflow` on the item (e.g. wrap a wide table in `overflow-x: auto`)
Horizontal scrollbar with `%` columns	`%` tracks ignore gaps	Use `fr`
Single auto-fill column overflows narrow screens	Minimum larger than the container	`minmax(min(200px, 100%), 1fr)`
Item lands in an unexpected extra row/column	Placed beyond the explicit grid → implicit tracks; or a misspelt area name (unknown names create implicit lines)	Check line numbers in DevTools; spell area names consistently
`grid-template-areas` ignored	Unequal cell counts or a non-rectangular area — the whole declaration is invalid	Fix the ASCII art
An image doesn't fill its cell	Items with an aspect ratio default to `start`, not `stretch` — in Chrome a 100×50 image in a 200×300 cell stays 100×50	`width: 100%; height: 100%; object-fit: cover` (Chapter 11)
`justify-content` does nothing	`fr` tracks leave no free space	Fixed tracks, or align items instead
Tab order jumps around	`dense`, placement or `order` moved items visually	Keep DOM order = visual order

In the interview

30-second answer: "Grid is two-dimensional layout. On the container I define tracks with `grid-template-columns` / `-rows` using lengths, `fr` for fractions of leftover space, `minmax` and `repeat` — `repeat(auto-fill, minmax(200px, 1fr))` gives a responsive grid with no media queries. Items are auto-placed, or placed by line numbers, spans or named `grid-template-areas`. Extra items create implicit tracks sized by `grid-auto-rows`. Alignment uses `justify-*` for the inline axis and `align-*` for the block axis, and `subgrid` lets nested grids share the parent's tracks so cards line up."

- *How is `fr` calculated?* → Container minus fixed tracks and gaps, divided by total `fr`; tracks below their minimum are fixed and the rest re-split. 700 − 200 − 40 = 460 → 1fr = 153.33.
- *A layout that reorders at a breakpoint?* → Named areas rewritten in a media query — keeping DOM order sensible for keyboard and screen-reader users.
- *What is subgrid for?* → Aligning the insides of sibling components (card titles, form labels) to shared tracks.

Chapter 9 TL;DR

- **Grid** = two-dimensional, layout-in. `display: grid` → direct children are items. Lines (1…n+1, −1 backwards), tracks, cells, areas, gaps; explicit vs implicit grid.
- **Tracks**: px/%/`fr`/`auto`/`min-content`/`max-content`/`fit-content()`/`minmax()`. **`fr` = share of space left after fixed tracks and gaps** (700 − 200 − 40 = 460 → 153.33 / 306.67, verified). `1fr` = `minmax(auto, 1fr)` → `minmax(0, 1fr)` for truly equal columns.

- **RAM**: `repeat(auto-fill|auto-fit, minmax(min(X, 100%), 1fr))`; n = floor((W + gap) / (X + gap)). auto-fill keeps empty tracks, auto-fit collapses them.
- **Placement**: `grid-column: 1 / 3`, `span 2`, `1 / -1` (full width), `grid-area: row-start / col-start / row-end / col-end`, named lines, `grid-template-areas` ASCII art (rectangles only, `.` = empty). Items can overlap in one cell.
- **Implicit grid**: `grid-auto-rows/columns` size extra tracks; `grid-auto-flow: row | column | dense` (dense breaks visual/DOM order).
- **Alignment**: `justify-*` = inline axis, `align-*` = block axis; `*-items`/`*-self` inside cells, `*-content` for the whole grid when tracks leave free space; `place-*` shorthands; `place-items: center` centres.
- **Subgrid** (Baseline since Sept 2023): `grid-template-rows: subgrid` on a spanning item reuses the parent's tracks so card internals align.
- **Grid vs Flexbox**: grid for page structure and uniform grids; flexbox for content-sized rows inside components; nest them.
- **Masonry** (`display: grid-lanes`) is Safari-only in 2026 — not Baseline.

Next: Chapter 10 — Responsive Design.

Chapter 10 — Responsive Design

The same page is opened on a 360px phone, a sideways tablet, a 1440px laptop and a 2560px monitor — by people who may have enlarged their default font, zoomed to 200 %, switched to dark mode, or asked the OS to reduce motion (animation can cause dizziness or nausea). **Responsive design** builds *one* page that works in all of those situations, instead of a separate "mobile site". Chapters 7–9 gave the layout tools; this chapter makes layouts **adapt**.

In this chapter: 10.1 What responsive design is · 10.2 The viewport meta tag and viewport units · 10.3 Media queries · 10.4 Mobile-first in practice · 10.5 User-preference queries · 10.6 Container queries · 10.7 Fluid typography with `clamp()` · 10.8 Responsive images recap and `image-set()` · 10.9 Intrinsic layouts · 10.10 Testing.

Every example is a page in `code/`. Screenshots in `img/` were made in headless Chrome by `code/make_figures.mjs`; the `NN_*.mjs` scripts print what the browser computed (matching queries, resolved grid columns, font sizes) using `code/measure.mjs` — as in Chapters 7–9, plus **viewport width**, **phone emulation** and **OS-preference emulation** (dark mode, high contrast…). Many screenshots are "contact sheets": one page at several widths, with the width in red above each.

10.1 What responsive design is

What it is

A **responsive** page rearranges and resizes itself to fit the **viewport** (the visible browser area) and the user's settings. Ethan Marcotte coined the term in 2010, with three ingredients:

Ingredient	Meaning	Today's tools
Fluid grids	Widths in relative units (%, `fr`, `vw`) instead of fixed pixels	Flexbox, Grid, `min()` / `max()` / `clamp()`
Flexible media	Images and videos shrink with their container	`max-width: 100%; height: auto`, `srcset`, `aspect-ratio`
Media queries	CSS that applies only under conditions ("at least 640px wide")	`@media`, and now `@container`

Why it matters

- Most web traffic is from phones; a page needing sideways scrolling or pinch-zoom loses readers.
- Google indexes the mobile version first ("mobile-first indexing"), so a broken mobile layout hurts discovery (Chapter 14).
- WCAG 2.2 requires content to **reflow** (rewrap into a narrow column without sideways scrolling) at 320 CSS px and survive 200 % text zoom (§10.10, Chapter 3).

Mobile-first vs desktop-first

	Mobile-first	Desktop-first
Base CSS (no query)	The phone layout	The desktop layout
Queries use	`min-width` / `width >=` — *add* features as space grows	`max-width` / `width <` — *undo* features as space shrinks
Small devices download/parse	only the rules they need, plus harmless overrides	desktop rules, then overrides to undo them
Design pressure	Forces deciding what matters most first	Tends to cram a big design into a small screen

Mobile-first is the default because the single-column phone layout is simplest — normal flow already stacks — and every larger layout is an enhancement (**progressive enhancement**: start with what works everywhere, then add).

Responsive vs adaptive: an **adaptive** site switches between a few fixed-width layouts; a **responsive** one is fluid *between* breakpoints — modern practice, with fewer breakpoints thanks to intrinsic layouts (10.9) and container queries (10.6).

10.2 The viewport meta tag and viewport units

What it is

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

This `<head>` line (Chapter 2) tells a mobile browser: "lay the page out at the device's width in CSS pixels, at 100 % zoom".

The problem it solves

When the iPhone launched (2007), sites were designed for ~1000px desktops, so mobile browsers **pretend** to be wide: without the tag they lay out in a **layout viewport** of about **980px**, then zoom out to fit. A responsive page then never sees its phone media queries fire.

A **CSS pixel** isn't a hardware pixel: at **device pixel ratio** (DPR) 3, each CSS pixel is 3×3 physical pixels; `width=device-width` = as many CSS pixels as the screen is wide (e.g. 390 on a typical modern phone).

Proof

`code/01_no_viewport.html` and `code/01_viewport.html` differ only by the tag. `code/01_viewport.mjs` opens both on an emulated 400px phone (Puppeteer's `isMobile: true` makes Chrome honour the tag like a real phone):

```
01_no_viewport.html  layout 980px, visible 980px, scale 0.408, font-size 16px
01_viewport.html     layout 400px, visible 400px, scale 1, font-size 16px
```

Without the tag the page is scaled by 400 / 980 ≈ 0.408 — "16px" text is really about **6.5px**.

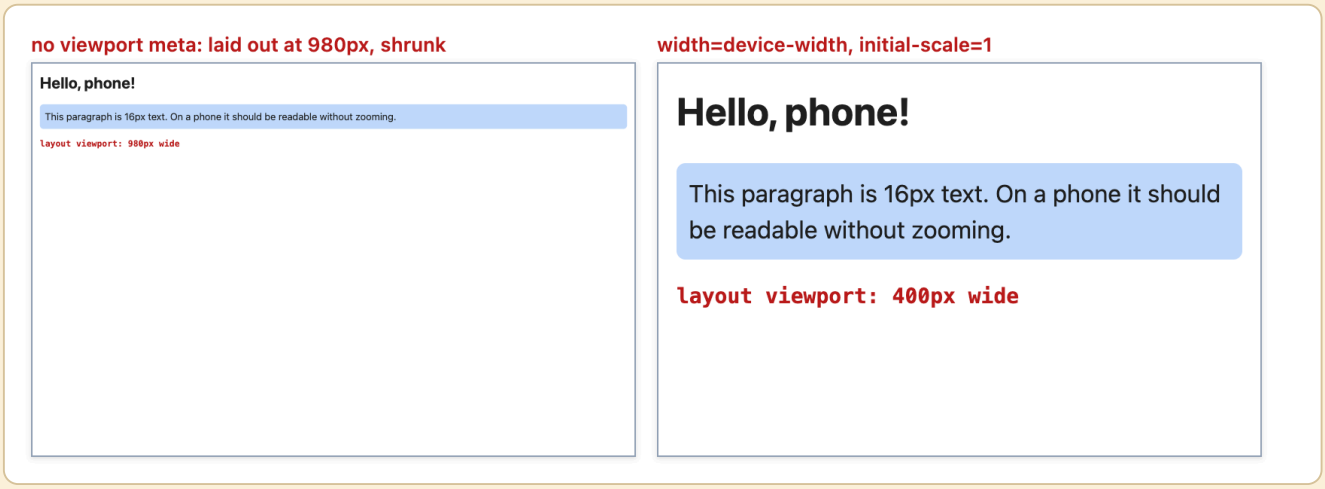


Figure 10.1 — The same page on a 400px phone, without and with the viewport meta tag.

The values

Key	Typical value	Notes
`width`	`device-width`	Layout viewport = screen width in CSS px. A number (`width=600`) is rarely right.
`initial-scale`	`1`	Start at 100 % zoom; also stops iOS changing zoom on rotation.
`maximum-scale`, `user-scalable=no`	**don't**	Blocking zoom fails WCAG 1.4.4 (Resize text); iOS Safari has ignored `user-scalable=no` since iOS 10 for this reason.
`viewport-fit`	`cover`	Notched phones (camera cut-out); then pad with `env(safe-area-inset-*)` (browser-supplied unsafe-edge sizes).

Viewport units

`vw` / `vh` are 1 % of the viewport (Chapter 6). On phones the height **changes** as the address bar slides, so CSS Values 4 added three variants (Baseline since December 2022):

Unit	1% of…	Behaviour on a phone
`svh` / `svw` (small)	the viewport with toolbars **shown**	never too tall; may leave a gap when toolbars hide
`lvh` / `lvw` (large)	the viewport with toolbars **hidden**	may be hidden behind toolbars at first
`dvh` / `dvw` (dynamic)	the viewport **right now**	follows the toolbars; can cause relayout while scrolling
`vh` / `vw` (classic)	in practice the **large** viewport on mobile	why `height: 100vh` "overflows" on phones

Use `min-height: 100svh` (always fits) or `100dvh` (tracks the toolbar) for full-screen heroes and the §8.7 sticky footer; `vw` for fluid type (10.7), never alone — it ignores zoom settings. On desktop all four heights are equal. Also: `vi` / `vb` (inline/block direction), `vmin` / `vmax`.

Common mistakes

- Forgetting the tag: every phone breakpoint silently fails.
- Disabling zoom "to feel like an app" — an accessibility failure.
- `width: 100vw` with a vertical scrollbar: on desktop `100vw` *includes* the scrollbar, so the element is ~15px too wide → sideways scrolling. Use `width: 100%`.

10.3 Media queries

What it is

A **media query** is a condition about the device or environment; rules inside `@media (condition) { … }` apply only while it's true, re-evaluated live (rotation, resize).

```
@media screen and (width >= 40em) { .cards { grid-template-columns: repeat(2, 1fr); } }
```

Media queries add no specificity; normal cascade rules apply (Chapter 6) — so mobile-first queries go **after** the base rules they override (equal specificity → later wins).

Where you can use them

Place	Example
CSS at-rule	`@media (width >= 40em) { … }`
`<link>`	`<link rel="stylesheet" href="print.css" media="print">` — still downloaded, but not render-blocking when it doesn't match
`@import`	`@import url(wide.css) (width >= 64em);`
`<source>` in `<picture>`	`<source media="(width >= 800px)" srcset="wide.jpg">` (Chapter 4)
JavaScript	`matchMedia('(width >= 64em)')`

Anatomy

1. **Media type** (optional): `all` (default), `screen`, `print`. Others (`tv`, `handheld`…) are deprecated.
2. **Media features** in parentheses: `(width >= 40em)`, `(orientation: landscape)`, `(hover: hover)`.
3. **Logic**: `and`; comma = **or**; `not` negates a whole query (or, in Level 4 syntax, a condition: `not (hover: hover)`); `only` is a legacy guard — harmless, unneeded.

Media features you'll actually use

Feature	Values / example	Use
`width` (+ `min-` / `max-`)	`(width >= 40em)`	Layout breakpoints (viewport width)
`height`	`(height < 500px)`	Short landscape phones: shrink sticky headers
`orientation`	`portrait` | `landscape`	Rarely needed — width says more
`aspect-ratio`	`(aspect-ratio >= 16/9)`	Full-screen media layouts
`hover` / `any-hover`	`hover` | `none`	Show hover-reveal UI only when the *primary* (or *any*) pointer can hover
`pointer` / `any-pointer`	`fine` | `coarse` | `none`	Larger tap targets for touch (`coarse`)
`resolution`	`(resolution >= 2x)`	High-DPR assets (prefer `srcset` / `image-set()`)
`prefers-color-scheme`	`light` | `dark`	Dark mode (10.5)
`prefers-reduced-motion`	`reduce` | `no-preference`	Tone down animation (10.5, Chapter 12)
`prefers-contrast`	`more` | `less` | `custom` | `no-preference`	Stronger borders/colours
`forced-colors`	`active` | `none`	Windows High Contrast / contrast themes
`print` (type)	`@media print`	Hide nav, show link URLs, black on white

Range syntax (Media Queries Level 4)

Comparison operators replace `min-` / `max-`. Baseline since March 2023, *widely available* since September 2025 (Chrome 104, Firefox 102, Safari 16.4).

Old	Range syntax
`(min-width: 640px)`	`(width >= 640px)`
`(max-width: 639.98px)`	`(width < 640px)`
`(min-width: 640px) and (max-width: 1023.98px)`	`(640px <= width < 1024px)`

Why `.98`? With `max-width: 639px` and `min-width: 640px`, a 639.5px window (possible with zoom or fractional DPR) matches **neither**; range syntax's `<` removes the gap.

`em` or `px` for breakpoints?

In media queries `em` / `rem` are always relative to the **browser's default font size** (normally 16px), *not* your `html { font-size }` (even `62.5%`). So `40em` = 640px by default, but 800px for a user with a 20px default — the layout goes single-column earlier, exactly what large-text users need. Both units work; `em` ties breakpoints to text size.

Choosing breakpoints

Let the **content** decide — widen the window until it looks wrong and add a breakpoint *there*; device lists go stale. Aim for two or three layout breakpoints plus intrinsic techniques (10.9) and container queries (10.6). Reference: Tailwind `sm 640 · md 768 · lg 1024 · xl 1280 · 2xl 1536`; Bootstrap 5 `576 · 768 · 992 · 1200 · 1400`.

Media queries in JavaScript

`window.matchMedia(query)` returns a `MediaQueryList` with `.matches` and a `change` event — for behaviour CSS can't express (swap a component, pause a carousel). Don't listen to `resize` and compare `innerWidth`; `change` fires only when the answer flips (`addListener()` is the deprecated older form).

```
const desktop = matchMedia('(width >= 64em)');
desktop.addEventListener('change', e => console.log(e.matches ? 'desktop' : 'not desktop'));
```

10.4's output shows it: resizing 400 → 900 → 1100 → 1300 → 700 fired exactly **two** events.

Common mistakes

- Mixing desktop-first `max-width` and mobile-first `min-width` queries — overlaps and gaps. Pick one direction.
- A media query **before** the base rule it should override.
- `(hover: hover)` alone to guess "desktop": touchscreen laptops and phones with mice exist.
- Breakpoints for specific devices ("iPhone 15") instead of content.

10.4 Mobile-first in practice

The page

`code/02_mobile_first.html` is a small travel page: base styles are the phone layout; two `min-width`-style queries add columns. The red badge (a `::after` whose `content` each query overrides) shows the active rule set.

```
/* Base = phone: single column; normal flow + grid gap does the stacking */
.layout { display: grid; gap: 14px; padding: 14px; }
.cards  { display: grid; gap: 10px; }

@media (width >= 40em) {                      /* 640px: tablets, small laptops */
  header { display: flex; justify-content: space-between; align-items: center; }
  .cards { grid-template-columns: repeat(2, 1fr); }
}
@media (width >= 64em) {                      /* 1024px: sidebar appears */
  .layout { grid-template-columns: 1fr 240px; max-width: 1200px; margin-inline: auto; }
  .cards  { grid-template-columns: repeat(3, 1fr); }
}
```

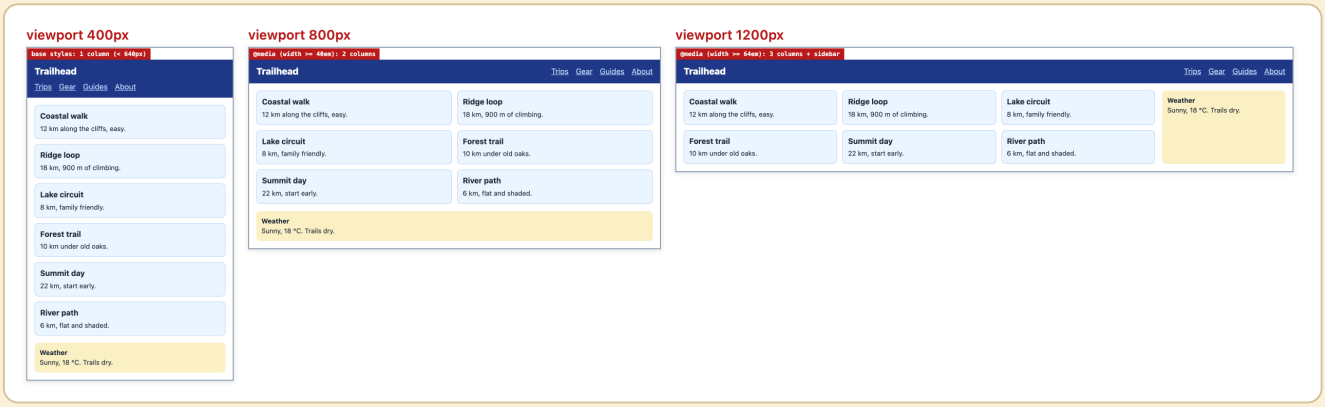


Figure 10.2 — One page, three widths. The HTML never changes; only which rules apply.

`code/02_media_queries.mjs` — matching queries and the resolved card grid:

```
 400px  (width >= 40em) no   (width >= 64em) no   cards: 372px
 800px  (width >= 40em) yes  (width >= 64em) no   cards: 381px 381px
1200px  (width >= 40em) yes  (width >= 64em) yes  cards: 299.328px 299.328px 299.344px
matchMedia events: [ 'change -> desktop at 1100px', 'change -> not desktop at 700px' ]
```

By hand at 1200px: 1200 − 2 × 14 padding = 1172; − 240 sidebar − 14 gap = 918 for `.cards`; − 2 × 10 gaps = 898; ÷ 3 = **299.33px**. ✓

Why it works

- **Normal flow** is already mobile-first: blocks stack, so the base CSS barely lays out.
- Each query **adds** a layout; nothing is "undone".
- **DOM order** (header, cards, aside) is the reading order at every width, so keyboard and screen-reader users get the same sequence (Chapter 3). Visual reordering with `order` or grid placement breaks that.

Responsive navigation (common pattern)

At phone width a long nav usually becomes a **menu button**: a real `<button aria-expanded="false" aria-controls="menu">` (telling screen readers whether the menu is open and what it controls) toggling the list with a few lines of JS, or the Popover API (`popover` attribute, Baseline since January 2025) for a no-JS disclosure. With few links, the demo's flex-wrap nav is simplest.

Responsive tables

Data tables can't reflow into one column without losing meaning. Wrap them in a scroll container — `<div class="table-wrap" tabindex="0" role="region" aria-label="Prices">` with `.table-wrap { overflow-x: auto; }`; WCAG allows two-dimensional scrolling for data tables (Chapter 5).

10.5 User-preference media queries

What it is

OS settings for dark theme, reduced motion or more contrast are exposed to CSS by the **user-preference media features** (Media Queries Level 5):

Feature	Baseline	Typical use
`prefers-color-scheme: dark`	widely available (2020)	Dark theme
`prefers-reduced-motion: reduce`	widely available (2020)	Remove or soften animation — Chapter 12
`prefers-contrast: more`	widely available (2022)	Solid borders, stronger colours
`forced-colors: active`	widely available (2022)	Windows contrast themes replace your colours: let system colours win, keep visible borders/focus on custom controls (system colours like `CanvasText`, `ButtonText`), don't rely on background images for meaning
`prefers-reduced-transparency`	**not** Baseline (Chromium only)	Remove frosted-glass effects
`prefers-reduced-data`	not supported anywhere yet	—

Dark mode — the essentials

1. **Declare support**: `<meta name="color-scheme" content="light dark">` and/or `:root { color-scheme: light dark; }`. The browser then themes its **own** UI — default background and text colour, form controls, scrollbars — even with no other dark CSS, and avoids a white flash. (`color-scheme`: Baseline 2022.)
2. **Colour tokens** (named values like `--bg`) as custom properties (§6.9), light values first.
3. **Override the tokens** in `@media (prefers-color-scheme: dark)`; components use only tokens.
4. One-off values: `light-dark(lightValue, darkValue)` picks by the element's **used colour scheme** (what `color-scheme` and the user's setting finally decide) — no media query, but `color-scheme: light dark` must be set. Baseline since May 2024 (newly available).

```
:root { color-scheme: light dark;
        --bg: #ffffff; --text: #1f2937; --card: #eff6ff; --accent: #1d4ed8; --border: #bfdbfe; }
@media (prefers-color-scheme: dark) {
  :root { --bg: #0f172a; --text: #e2e8f0; --card: #1e293b; --accent: #93c5fd; --border: #334155; }
}
@media (prefers-contrast: more) { :root { --border: currentColor; } }

body  { background: var(--bg); color: var(--text); }
.note { color: light-dark(#92400e, #fcd34d); }
```

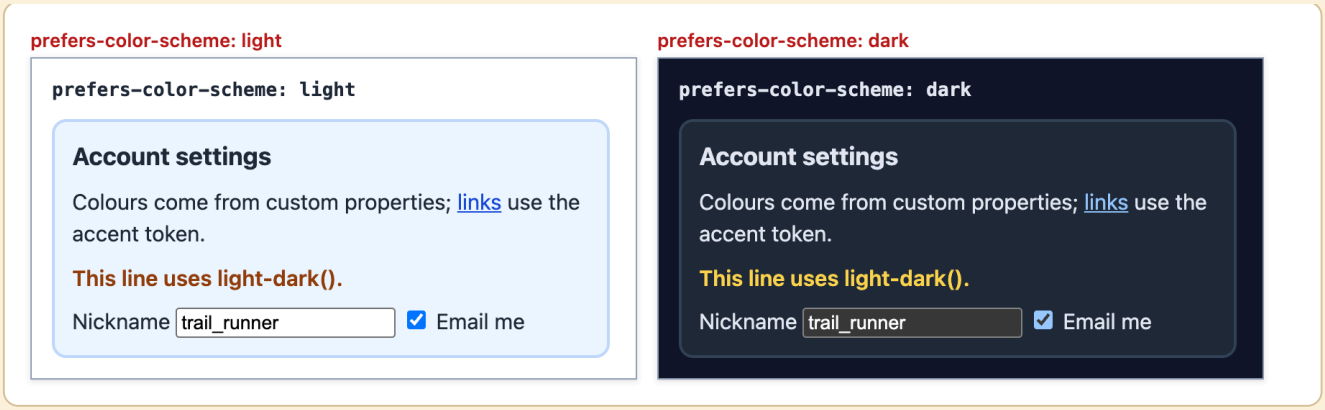


Figure 10.3 — `code/03_preferences.html` under emulated light and dark. Nobody styled the input and checkbox — `color-scheme` made the browser draw dark versions. Reproduce the dark shot with `node tools_web_shot.mjs chapters/10-responsive-design/code/03_preferences.html out.png 420 --dark`.

`code/03_preferences.mjs` reads computed colours under three emulated settings:

```
light (default)        bg rgb(255, 255, 255)  text rgb(31, 41, 55)  card border rgb(191, 219, 254)  note rgb(146, 64, 14)
<input> bg rgb(255, 255, 255)
dark                   bg rgb(15, 23, 42)  text rgb(226, 232, 240)  card border rgb(51, 65, 85)  note rgb(252, 211, 77)
<input> bg rgb(59, 59, 59)
light + more contrast  bg rgb(255, 255, 255)  text rgb(31, 41, 55)  card border rgb(31, 41, 55)  note rgb(146, 64, 14)
<input> bg rgb(255, 255, 255)
```

With more contrast the border became the text colour (`currentColor`); `rgb(59, 59, 59)` is Chrome's built-in dark control colour. Gotcha: headless Chrome follows the **computer's** theme when nothing is emulated, so on a dark-mode Mac a `color-scheme: light dark` page screenshots dark — `measure.mjs` emulates `light` explicitly.

Letting users override

A theme toggle sets `data-theme="dark"` on `<html>` and stores the choice; tokens are defined for `[data-theme="dark"]` too, and the media query applies only when no choice was made (`:root:not([data-theme="light"])`). Full architecture, with `color-scheme` per theme: Chapter 13.

Designing a dark theme well

- Don't invert: dark greys/navies (pure `#000` with pure white text causes halation — a glowing halo around letters), lighter accents, contrast re-checked ≥ 4.5:1 (Chapter 11).
- Shadows barely show on dark backgrounds — show elevation with slightly lighter surfaces.
- Dim very bright photos (`filter: brightness(.9)`) or supply dark versions with `<picture><source media="(prefers-color-scheme: dark)" …>`.

Reduced motion in one rule

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { animation-duration: 0.01ms !important; animation-iteration-count: 1 !important;
                           transition-duration: 0.01ms !important; scroll-behavior: auto !important; }
}
```

A common blunt reset; §12.9 covers the gentler "motion only when allowed" pattern and WCAG's requirements.

10.6 Container queries

What it is

A **container query** (`@container`) styles an element by the size of an **ancestor container** instead of the viewport. Baseline since February 2023, **widely available** since August 2025 (Chrome 105, Firefox 110, Safari 16).

The problem it solves

A product card sits in a wide main column, a narrow sidebar and a two-up grid — on one screen. A media query only knows the *viewport*, so it can't tell them apart; you used to hand-add modifier classes (`.card--compact`).

With container queries the card adapts to the space **it is given** ("how big is my box?", not "how big is the room?").

How it works

1. Mark an ancestor as a **size container**: `container-type: inline-size` (width only — the usual choice) or `size` (both axes; then the container can't get its height from content, so give it one). Shorthand `container: name / type` adds an optional **name**.
2. Write rules in `@container [name] (condition) { … }`; they apply to **descendants** of the matching container.
3. For each element the browser finds the **nearest ancestor** container (with that name, if given) and tests **its** content-box size.
4. A container can't style *itself* by its own query (circular), so the demo puts the container on a wrapper (`.card-slot`) and styles the `.product` inside.

```
.card-slot { container: card / inline-size; }       /* name "card", measured inline */

.product    { display: grid; gap: 8px; }             /* base: picture above text */
.product h3 { font-size: clamp(15px, 3cqi + 8px, 24px); }

@container card (width >= 400px) { .product { grid-template-columns: 160px 1fr; } }
@container card (width >= 600px) { .product { grid-template-columns: 240px 1fr; padding: 16px; } }
```

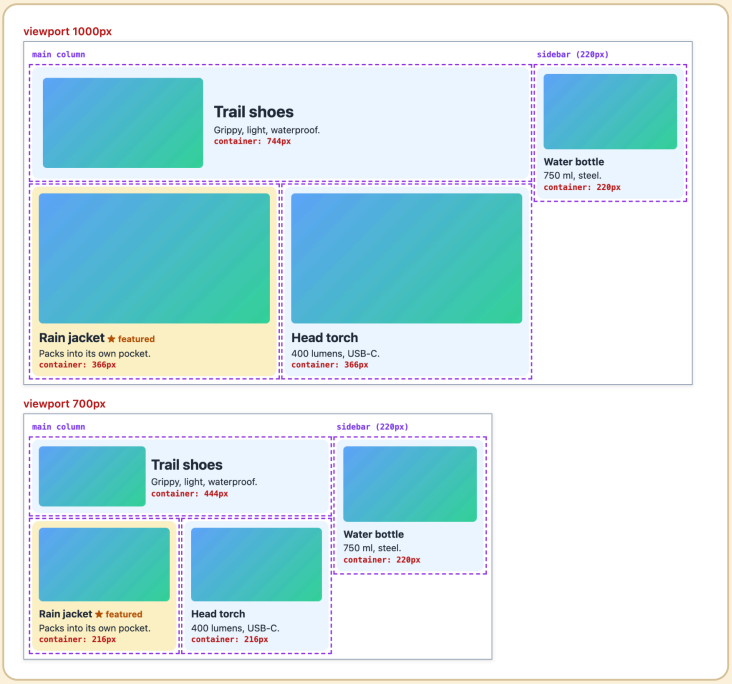


Figure 10.4 — `code/04_container_queries.html`. Dashed purple = containers; each card prints its container width. Same component, same viewport, three layouts.

Output of `code/04_container_queries.mjs`:

```
viewport 1000px
  Trail shoes   container 744px  columns 240px 456px    h3 24px      ::after none
  Rain jacket   container 366px  columns 346px          h3 18.98px   ::after " ★ featured"
  Head torch    container 366px  columns 346px          h3 18.98px   ::after none
  Water bottle  container 220px  columns 200px          h3 15px      ::after none
viewport 700px
  Trail shoes   container 444px  columns 160px 256px    h3 21.32px   ::after none
  Rain jacket   container 216px  columns 196px          h3 15px      ::after " ★ featured"
  Head torch    container 216px  columns 196px          h3 15px      ::after none
  Water bottle  container 220px  columns 200px          h3 15px      ::after none
```

The sidebar card (220px) is identical at both widths, while "Trail shoes" switches from the ≥600px to the ≥400px layout — impossible with a media query.

Container query units

Unit	1% of the query container's…
`cqw` / `cqh`	width / height
`cqi` / `cqb`	inline size / block size (writing-mode aware — prefer these; a 500px container → `10cqi` = 50px)
`cqmin` / `cqmax`	smaller / larger of the two

With no suitable container they fall back to **small viewport** units (`svw`/`svh`/`svi`…). Check: `clamp(15px, 3cqi + 8px, 24px)` in 366px = 3 × 3.66 + 8 = **18.98px** ✓; in 220px = 14.6 → clamped to **15px** ✓; in 744px = 30.3 → capped at **24px** ✓.

Style queries

`@container style(--featured: yes)` tests a **custom property's computed value** on the nearest container instead of its size; every element is a style container, so no `container-type` is needed. In the demo `.featured { --featured: yes; }` turns "Rain jacket" yellow with a star — a design-token switch with no modifier class. **Baseline since May 2026** (newly available: Chrome 111, Safari 18, Firefox 151), custom properties only (`style(display: flex)` is unsupported) — treat as an enhancement for now.

Other container queries (not Baseline)

`scroll-state()` queries ("is this sticky header stuck?", "is this carousel item snapped?") are Chromium-only (Chrome 133+).

Container queries vs media queries

Use a media query for…	Use a container query for…
Page-level layout (how many columns the page has)	Components that appear in different-sized slots
User preferences (`prefers-*`), print, pointer type	A card/widget's internal layout
Anything about the device	Text/spacing relative to the component (`cqi`)

They combine: media queries shape the page; container queries let each piece adapt to the space it got.

Common mistakes

- `container-type` on the element you want to restyle — it only restyles **descendants**.
- `container-type: size` without an explicit height: collapses to 0, because size containment (sizing the box as if empty) ignores its content.
- `inline-size` containment means the container's **width** can't depend on its children — inside a shrink-to-fit parent (a `flex: none` item, inline-block, float) it may collapse. Give it width from outside (grid track, block flow, `flex: 1`).
- An unnamed `@container (width > 400px)` matches the **nearest** size container of any name — name containers in design systems.

Fallback for older browsers

```
@supports not (container-type: inline-size) { .product { /* a sensible single layout */ } }
```

`@supports (feature: value)` is a **feature query**: rules apply only if the browser understands the declaration. Needed now only for old devices.

In the interview

30-second answer: "Media queries respond to the viewport; container queries respond to an ancestor I declared a container with `container-type: inline-size`. That makes components portable: the same card stacks in a sidebar and sits side-by-side in the main column on one screen. A container can't query itself, so I put it on a wrapper, and use `cqi` for sizes relative to it. Size queries are Baseline since 2023; style queries on custom properties became Baseline in 2026."

- *Why not* `container-type: size` *by default?* → Size containment stops its size depending on content; for height that usually collapses it to 0. `inline-size` only removes the width dependency.
- *Drop media queries?* → No: page layout and user preferences still belong to media queries.

10.7 Fluid typography with `clamp()`

What it is

Fluid typography makes font sizes (and often spacing) change **smoothly** with the viewport between a minimum and maximum, instead of jumping at breakpoints. The tool is `clamp(MIN, PREFERRED, MAX)` = `max(MIN, min(VAL, MAX))` — Baseline since 2020, with `min()` and `max()`.

How to derive the formula

Goal: **16px at a 320px viewport → 24px at 1200px**.

1. **Slope** = Δsize / Δviewport = (24 − 16) / (1200 − 320) = 8/880 = 0.00909 → **0.909vw** (1vw = 1 % of the viewport).
2. **Intercept** (size at 0px) = 16 − 0.00909 × 320 = 13.09px = **0.818rem**.
3. Preferred value `0.818rem + 0.909vw`, clamped to `1rem … 1.5rem`:

```
:root {
  --step-0: clamp(1rem, 0.818rem + 0.909vw, 1.5rem);      /* 16px → 24px  */
  --step-3: clamp(1.75rem, 1.296rem + 2.273vw, 3rem);     /* 28px → 48px  */
  --space:  clamp(0.75rem, 0.341rem + 2.045vw, 1.875rem); /* 12px → 30px  */
}
p  { font-size: var(--step-0); }
h1 { font-size: var(--step-3); }
```

(Another: 18px at 400px → 22px at 1400px: slope 4/1000 = 0.4vw, intercept 16.4px → `clamp(1.125rem, 1.025rem + 0.4vw, 1.375rem)`.)

`code/05_fluid_type.mjs` compares Chrome's computed sizes with the formula:

```
slope = 0.909vw, intercept = 13.091px = 0.818rem
  280px  p 16.00px (by hand 16.00)   h1 28.00px
  320px  p 16.00px (by hand 16.00)   h1 28.01px
  600px  p 18.54px (by hand 18.55)   h1 34.37px
  900px  p 21.27px (by hand 21.27)   h1 41.19px
 1200px  p 24.00px (by hand 24.00)   h1 48.00px
 1600px  p 24.00px (by hand 24.00)   h1 48.00px
default font 20px, viewport 1000px, rem + vw (ours)  -> 25.45px
default font 20px, viewport 1000px, vw only (bad)    -> 16px
```

The 0.01px differences come from rounding coefficients to three decimals.

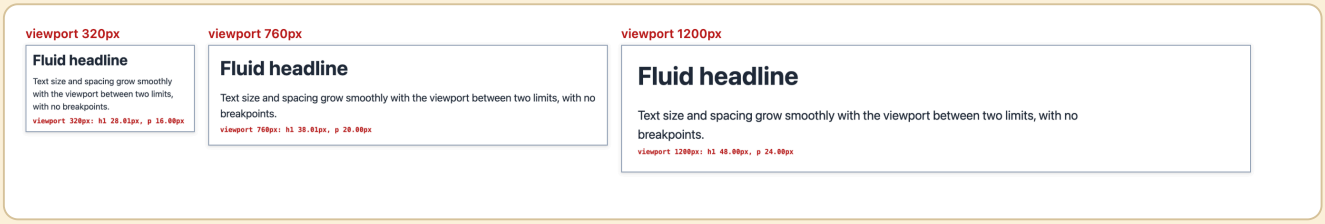


Figure 10.5 — `code/05_fluid_type.html` at three widths, printing its own computed sizes.

Why the `rem` part is essential (accessibility)

The last two lines emulate a user with a 20px **default font size**: `rem + vw` grows to 25.45px (0.818 × 20 + 9.09); pure `vw` stays **16px** — ignoring the setting and not growing with page zoom as users expect. WCAG 2.2 SC 1.4.4 (Resize text) needs text to scale to 200 %: keep a `rem` term in the preferred value, `MIN`/`MAX` in `rem`, and MAX/MIN at or below about 2.5 so zoom still enlarges text meaningfully. (`clamp(1rem, 2vw, 1.5rem)` works, but between its limits it ignores the user's setting.)

Type scales

A **modular scale** is font sizes related by one multiplier (each step = previous × ratio, e.g. 1.2 "minor third" on small screens, 1.333 "perfect fourth" on large). Fluid type interpolates each step linearly between its small- and large-screen size — what `--step-0 … --step-5` token sets from generators like Utopia do. The same works for spacing (`--space`).

Common mistakes

- `font-size: 4vw` alone — unreadable on phones, huge on monitors, ignores user settings.
- `clamp(16px, …, 24px)` — px limits ignore the user's default; prefer `rem`.
- Forgetting `line-height`: large headings need a smaller unitless line-height (a plain multiplier, 1.1–1.2), body ~1.5 (Chapter 11).

10.8 Responsive images recap (and `image-set()`)

Full coverage in §4.3: `img { max-width: 100%; height: auto }` plus `width`/`height` attributes (no layout shift); `srcset` with `w` + `sizes` (browser picks by slot width and DPR) or `1x`/`2x` (fixed size, sharper on high-DPR); `<picture>` + `<source media>` for **art direction** (a different crop per width) or `<source type>` for a newer format with fallback; `aspect-ratio` to hold a box's shape before content loads.

`image-set()` for CSS background images

`srcset` only works on `<img>`. For **background images** use `image-set()` — Baseline since September 2023, widely available since March 2026 (older Safari needed `-webkit-image-set()`):

```
.hero {
  background-image: url("media/hills-400.jpg");                 /* fallback */
  background-image: image-set("media/hills-400.jpg" 1x,
                              "media/hills-800.jpg" 2x);
  background-size: cover;
}
```

Candidates can also be chosen by format: `image-set("a.avif" type("image/avif"), "a.jpg" type("image/jpeg"))`. `code/06_image_set.mjs` loads `code/06_image_set.html` at three pixel ratios and logs the network:

```
devicePixelRatio 1: downloaded hills-400.jpg
devicePixelRatio 2: downloaded hills-800.jpg
devicePixelRatio 3: downloaded hills-800.jpg
```

Only **one** file per screen. A background image is decoration: meaningful pictures need `<img alt>` (the demo's `role="img"` + `aria-label` exist only because it demonstrates a decorative technique).

10.9 Intrinsic layouts — responsive without breakpoints

What it is

Intrinsic design (Jen Simmons' term) lets the browser's layout algorithms respond to available space themselves, using `min()`, `max()`, `clamp()`, `flex-wrap`, `auto-fit` and `minmax()`. Fewer breakpoints → fewer awkward in-between widths, and components that work in any container.

Three patterns

`code/07_intrinsic.html` — no `@media` anywhere:

```css
/* 1. Readable centred column: at most 60ch, and never touching the screen edges */
.wrapper { width: min(100% - 2rem, 60ch); margin-inline: auto; }

/* 2. Sidebar that wraps when the content would get narrower than 60% */
.with-sidebar          { display: flex; flex-wrap: wrap; gap: 10px; }
.with-sidebar > .side  { flex: 1 1 150px; }                    /* wants ~150px */
.with-sidebar > .content { flex: 999 1 0; min-inline-size: 60%; } /* takes the rest */

/* 3. Card grid: as many ≥140px columns as fit (Chapter 9 §9.3) */
.grid { display: grid; gap: 10px; grid-template-columns: repeat(auto-fit, minmax(min(140px, 100%), 1fr)); }
```

Pattern 2: the content's huge `flex-grow` (999) takes almost all free space, keeping the sidebar near 150px. When the line can't hold `150px + gap + 60%`, flex-wrap moves the sidebar to its own full-width line — a "breakpoint" chosen by content, working in any container.

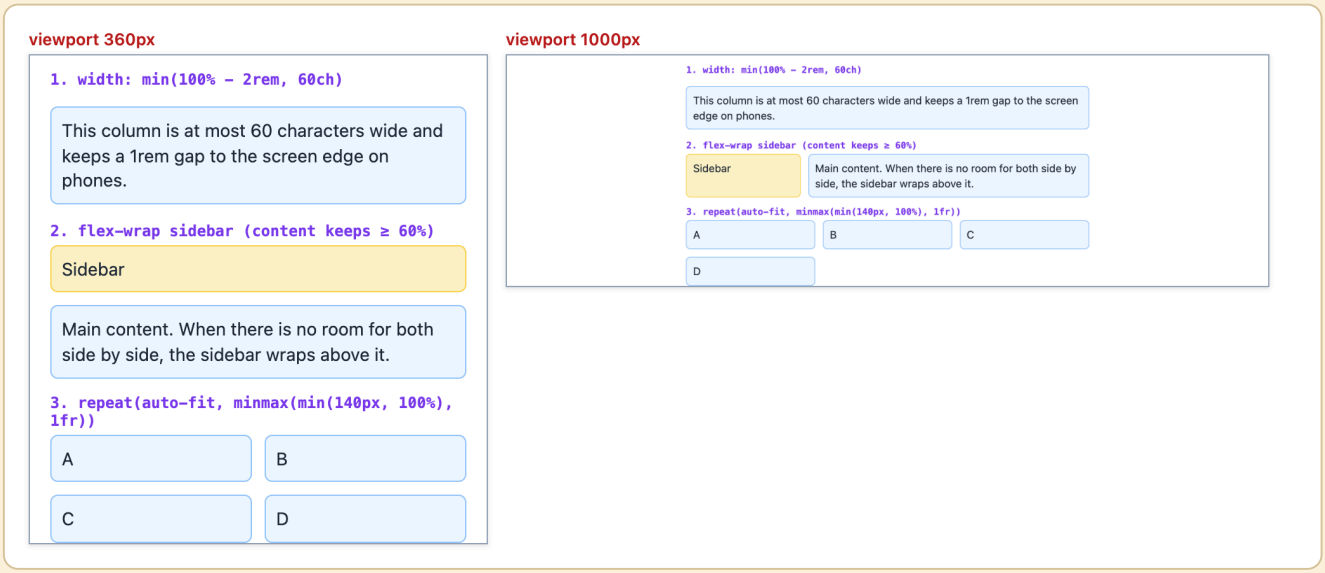


Figure 10.6 — The same page at 360px and 1000px.

`code/07_intrinsic.mjs`:

```
 360px  wrapper 328.0px | sidebar wrapped above content | grid 2 column(s)
 480px  wrapper 448.0px | sidebar beside content (150.3px + 287.7px) | grid 3 column(s)
1000px  wrapper 529.1px | sidebar beside content (150.3px + 368.8px) | grid 3 column(s)
```

By hand at 480px: wrapper = min(480 − 32, 60ch ≈ 529px) = 448px; the line needs 150 + 10 + 0.6 × 448 = 428.8 ≤ 448, so the sidebar stays beside; free 448 − 10 − 150 = 288 shared 1 : 999 → sidebar 150.3px ✓. At 360px the wrapper is 328px and the line would need 356.8px, so it wraps ✓.

Other intrinsic tools

- `flex-wrap: wrap` + `flex: 1 1 12rem` — a button row that becomes a column when needed (Chapter 8).
- `max-inline-size: 65ch` on text — the **measure** (line length) that reads best (Chapter 11).
- `aspect-ratio` placeholders + `object-fit: cover` (Chapter 11).

When not to use it

When the design genuinely changes structure (a sidebar becoming a bottom tab bar), a media query states that intent more clearly than a flex trick.

10.10 Testing responsive design

What to test

Check	How	Pass criterion
No sideways scrolling	Resize from 320px to 1920px+	`scrollWidth ≤ clientWidth` at every width (except data tables, maps, etc.)
Reflow (WCAG 2.2 SC 1.4.10, AA)	View at 320 CSS px wide (= 1280px at 400 % zoom)	All content and functions with vertical scrolling only
Resize text (SC 1.4.4, AA)	Browser zoom 200 %; also raise the default font size	No clipped or overlapping text
Text spacing (SC 1.4.12, AA)	Line-height 1.5, paragraph spacing 2em, letter 0.12em, word 0.16em	Nothing clipped — avoid fixed heights on text boxes
Target size (SC 2.5.8, AA, new in 2.2)	Inspect tap targets	At least 24 × 24 CSS px (or enough spacing)
Orientation (SC 1.3.4)	Rotate the device	Works in both, unless essential
Touch vs mouse	Real phone; `(hover: none)`	Nothing reachable *only* on hover
Preferences	Emulate dark, reduced motion, forced colours	Readable, no lost UI

Tools

- **DevTools device mode**: Chrome/Edge and Firefox Ctrl/Cmd+Shift+M; Safari Responsive Design Mode (Develop menu). Drag the width slowly to find breaking widths. Chrome's Rendering panel emulates `prefers-color-scheme`, `prefers-reduced-motion`, `forced-colors` and more; Elements shows a `container` badge for inspecting container queries.
- **Real devices**: emulators don't reproduce touch precision, the dynamic toolbar (`dvh`), the on-screen keyboard or real CPU speed. Test at least one Android phone and one iPhone via remote debugging (Chrome `chrome://inspect`, Safari Web Inspector).
- **Automated screenshots and checks** in CI with Puppeteer or Playwright, like this chapter's scripts.

An automated overflow test

`code/08_overflow_check.mjs` opens pages at 320/768/1280px, compares `scrollWidth` with the viewport's `clientWidth`, and lists elements whose `getBoundingClientRect().right` sticks out (or whose content overflows its box):

```
08_broken.html          320px  OVERFLOW (page is 575px wide): div.promo, img, p
```

Every other page/width combination (`02_mobile_first`, `08_fixed`, and `08_broken` at 768/1280px) reports `ok`.

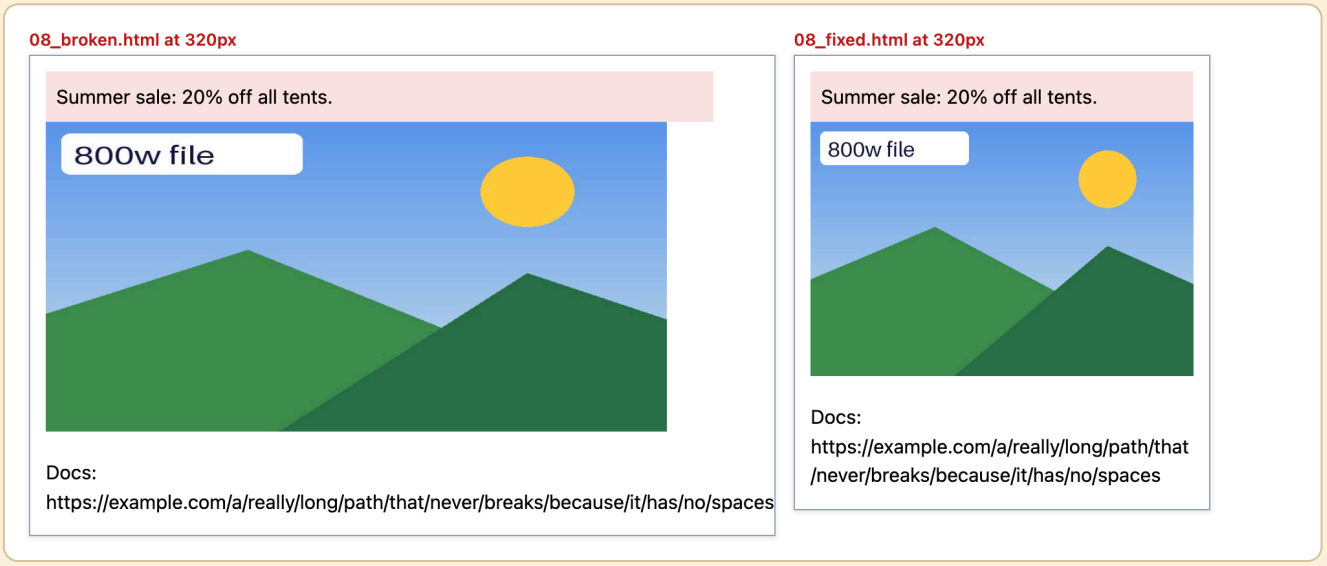


Figure 10.7 — The three most common causes of sideways scrolling on phones, and their one-line fixes.

Bug in `08_broken.html`	Fix in `08_fixed.html`
`.promo { width: 500px }`	`max-width: 500px` — fixed widths are the #1 cause
`<img width="480">` with no CSS	`img { max-width: 100%; height: auto; }`
Long URL with no break opportunities	`overflow-wrap: anywhere` (or `break-word`) on text containers

Other culprits: `100vw` widths (10.2), `1fr` grid columns that can't shrink below a long word (`minmax(0, 1fr)`, §9.10), flex items with `min-width: auto` (Chapter 8), `<pre>` blocks and wide tables (wrap in `overflow-x: auto`).

In the interview

30-second answer: "I build mobile-first: base CSS is the single-column phone layout and `min-width` queries add columns where the content needs them. The viewport meta tag makes phones lay out at device width. Intrinsic techniques — flex-wrap, `repeat(auto-fit, minmax())`, `clamp()` fluid type with a `rem` term — keep breakpoints few, and container queries handle components in different-sized slots. I respect `prefers-color-scheme` with `color-scheme`, and `prefers-reduced-motion`. Images use `srcset`/`sizes`. I test in device mode, on real phones, at 320px and 200 % zoom for WCAG reflow."

- *Why does `100vh` misbehave on mobile?* → It equals the large viewport (toolbars hidden), so with toolbars visible the bottom is cut off. Use `svh` or `dvh`.
- *How do you pick breakpoints?* → Where the content breaks, not device widths; `em` so they follow the user's font size.
- *How do you avoid horizontal scrolling?* → No fixed widths (`max-width`), flexible media, `overflow-wrap`, `minmax(0,1fr)`, and an automated check at 320px.
- *What can't DevTools device mode emulate well?* → Real touch, dynamic toolbars/`dvh`, the on-screen keyboard, real performance.

Chapter 10 TL;DR

- **Responsive** = fluid layout + flexible media + conditional CSS. Build **mobile-first**: base CSS is the phone layout; `(width >= …)` queries add columns.
- **`<meta name="viewport" content="width=device-width, initial-scale=1">`** — without it phones lay out at ~980px and shrink (scale 0.408 on a 400px phone). Never disable zoom.
- **Viewport units**: `svh` (toolbars shown), `lvh` (hidden), `dvh` (live); `vh` ≈ `lvh` on phones; `100vw` includes the scrollbar.
- **Media queries**: type + features + `and` / `,` / `not`. Range syntax `(640px <= width < 1024px)` is widely available and removes boundary gaps. `em` in queries = browser default font size. JS: `matchMedia(q)` + `change` event.
- **Preferences**: `prefers-color-scheme` + `color-scheme: light dark` (themes form controls too) + token overrides; `light-dark()` (Baseline 2024); `prefers-contrast`, `forced-colors`, `prefers-reduced-motion`.
- **Container queries** (widely available since Aug 2025): `container: name / inline-size` on a wrapper, `@container name (width >= 400px)` for descendants, `cqi` units. Style queries on custom properties: Baseline 2026. Scroll-state queries: Chromium only.
- **Fluid type**: `clamp(MIN, a·rem + b·vw, MAX)`; slope = Δsize/Δviewport, intercept = size − slope × viewport; keep a `rem` term so user settings and zoom work (vw-only stayed 16px with a 20px default).
- **Images**: `max-width: 100%`, `srcset`/`sizes`, `<picture>`; `image-set()` for backgrounds (one file per DPR).
- **Intrinsic layouts**: `min(100% - 2rem, 60ch)`, flex-wrap sidebar, `repeat(auto-fit, minmax(min(X,100%),1fr))` — breakpoints chosen by content.
- **Test**: 320px reflow, 200 % zoom, text spacing, 24px targets, real devices, dark/reduced motion, automated overflow checks.

Next: Chapter 11 — Typography, Colour and Visual Effects.

Chapter 11 — Typography, Colour and Visual Effects

Layout decides *where* boxes go; this chapter is how they *look*. Most of a page is text, so typography — fonts, sizes, line spacing, line length — does more for readability than any other styling. Then colour and contrast (which decide whether some readers can read at all), and the decorative toolkit: backgrounds and gradients, borders and radius, shadows, filters, blend modes (rules for mixing an element's colours with what's behind it), clipping, masks and image fitting — each with a performance or accessibility side that's easy to miss.

In this chapter: 11.1 Fonts, `@font-face`, variable fonts · 11.2 Loading fonts well (`font-display`, preload, subsetting, self-hosting) · 11.3 Text properties · 11.4 Colour and contrast, `color-mix()` and relative colours · 11.5 Backgrounds and gradients · 11.6 Borders, radius and outlines · 11.7 Shadows · 11.8 Filters, blend modes, clipping and masks · 11.9 `object-fit` and `object-position`.

As in Chapters 7–10, every example is a page in `code/`, screenshotted by `code/make_figures.mjs`; the `NN_*.mjs` scripts use `code/measure.mjs` (a copy of Chapter 10's helper) to read back what Chrome computed — font load status, contrast ratios, even individual **pixel colours** from screenshots to check gradient and blend-mode maths. The one web font, **Inter**, is in `code/media/` with its SIL Open Font License (free to use and redistribute).

11.1 Fonts

What it is

A **font family** is a designed set of letter shapes (**glyphs**), usually in several **weights** (thin → black) and **styles** (normal, italic):

```
body {
  font-family: "Inter", system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  font-size: 1rem;           /* 16px by default */
  font-weight: 400;          /* 100–900; normal = 400, bold = 700 */
  font-style: normal;        /* italic | oblique */
  line-height: 1.5;
}
/* shorthand: [style] [variant] [weight] [stretch] size[/line-height] family  (size and family required) */
h1 { font: italic 700 2rem/1.2 Georgia, serif; }
```

Font stacks and generic families

`font-family` is a **fallback list**: for every character the browser uses the first family that's available *and* has that glyph. Always end with a **generic family**, which always maps to something installed:

Generic	Looks like	Typical mapping
`serif`	strokes with small "feet"	Times New Roman
`sans-serif`	no feet	Helvetica / Arial
`monospace`	every character equally wide	Menlo, Consolas, Courier
`system-ui`	the OS UI font	San Francisco (Apple), Segoe UI (Windows), Roboto (Android)
`cursive`, `fantasy`	script / decorative	varies wildly — avoid for body text
`ui-serif`, `ui-sans-serif`, `ui-monospace`, `ui-rounded`	the OS's UI variants	only some platforms (Apple mainly)

A **system font stack** (`system-ui, …, sans-serif`) costs zero downloads and looks native — GitHub and many design systems use one. Quote family names with spaces; never quote generic names (`"serif"` means a font literally named "serif").

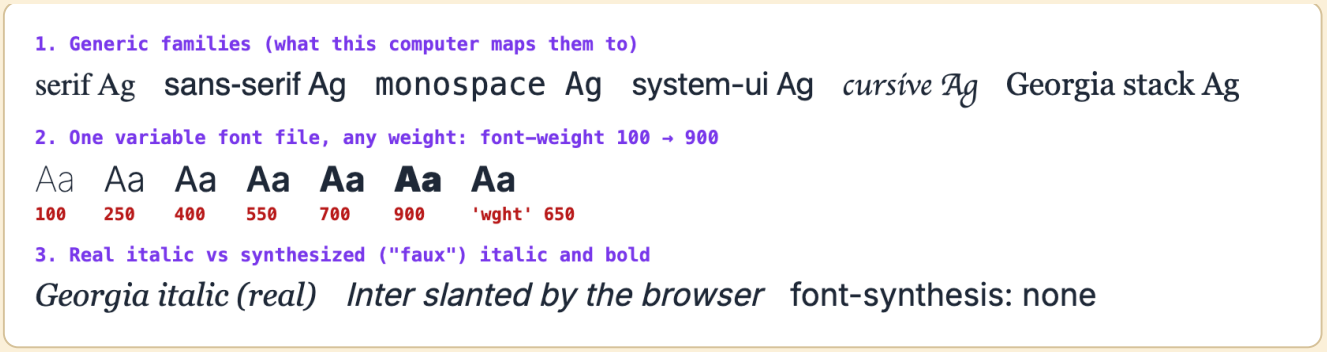


Figure 11.1 — `code/01_fonts.html`: generic families on this Mac, one variable font at seven weights, and real vs synthesized italic.

`@font-face`: using your own font

```
@font-face {
  font-family: "Inter Var";                                  /* the name YOU choose */
  src: url("media/inter-latin-wght-normal.woff2") format("woff2");
  font-weight: 100 900;                                      /* this file covers a range */
  font-style: normal;
  font-display: swap;                                        /* 11.2 */
}
.inter { font-family: "Inter Var", system-ui, sans-serif; }
```

1. `@font-face` only *declares* a font; nothing downloads yet.
2. When text needs that family (at a matching weight/style), the browser **downloads the file** — lazily.
3. Per weight/style request it picks the rule whose descriptors (`font-weight`, `font-style`, `font-stretch` inside `@font-face`) match best. With **static fonts** (one weight and style per file) you write one rule per file (Regular 400, Bold 700, Italic…) sharing one `font-family` name; `font-weight: 100 900` declares a variable font covering the range.
4. If no face matches a requested bold or italic, the browser **synthesizes** one (smears glyphs for bold — **faux bold** — slants for italic). `font-synthesis: none` turns this off (last sample in Figure 11.1 stays upright).

Font formats

Format	Use
WOFF2	The only one you need: Brotli-compressed, supported by every current browser
WOFF	Older compressed format; very old browsers only
TTF / OTF	Desktop formats; larger; avoid on the web
EOT, SVG fonts	Obsolete

Variable fonts

A **variable font** stores one or more **axes** of variation in one file, so every weight from 100 to 900 (not just nine steps) comes from one download:

Axis tag	Meaning	Prefer this property
`wght`	weight	`font-weight: 550`
`wdth`	width	`font-stretch: 85%`
`ital`	italic on/off	`font-style: italic`
`slnt`	slant angle	`font-style: oblique 10deg`
`opsz`	optical size (shapes tuned for small vs large text)	`font-optical-sizing: auto` (default)

`font-variation-settings: "wght" 650, "GRAD" 50` sets axes directly — needed for **custom axes** (uppercase tags like `GRAD`, grade) — but overrides the higher-level properties and doesn't compose through inheritance (a child setting one axis wipes its parent's), so use normal properties where they exist. Baseline widely available since 2018.

`code/01_fonts.mjs` confirms the one file serves the whole range with genuinely different outlines:

```
Inter Var weight=100 900 status=loaded display=swap
document.fonts.check("700 22px Inter Var"): true
"Hamburgefonstiv" at 40px, width by weight -> 100: 307.5px, 400: 325.2px, 700: 340.5px, 900: 353.5px
font-synthesis on the last sample: none
```

("Hamburgefonstiv" is a classic type-testing word.) Heavier weights are wider — real glyphs, not a smeared regular.

Trade-offs

Option	+	−
System font stack	0 bytes, instant, native feel	Looks different per OS; less brand identity
One variable web font	Any weight from one file (~50 KB here for Latin)	One larger file; loading to manage (11.2)
Several static web fonts	Each file small	Every extra weight/style is another request

Common mistakes

- No generic family at the end — a failed web font falls back to the browser default (often Times).
- `font-weight: 600` with only 400 and 700 files → the browser picks 700 (nearest heavier); `bold` on a family with no bold face → faux bold (provide the face or `font-synthesis: none`).
- Loading five weights "just in case".
- Quoting generic names (`"sans-serif"`).

11.2 Loading fonts well: `font-display`, preload, subsetting

The problem it solves

A web font is a separate download that usually starts **late** — once HTML and CSS are parsed and the browser knows the text needs it. Meanwhile:

- **FOIT** (Flash Of Invisible Text): text hidden until the font arrives — blank space on slow networks.
- **FOUT** (Flash Of Unstyled Text): fallback font shown, then **swapped** — the reflow may shift content (hurting **CLS**, Cumulative Layout Shift, a Core Web Vital — Chapter 14).

`font-display`

This descriptor (CSS Fonts Level 4; Baseline widely available) sets three periods:

1. **Block period** — text drawn invisibly while the font loads.
2. **Swap period** — text drawn in the fallback; if the font arrives, it swaps in.
3. **Failure period** — too late: keep the fallback for this page.

Value	Block period	Swap period	Use for
`auto`	browser decides (usually like `block`)		—
`block`	short (spec suggests 3 s)	infinite	icon fonts (characters are pictures), where fallback glyphs would be wrong
`swap`	extremely short (≈0–100 ms)	infinite	brand/heading fonts that must appear
`fallback`	extremely short	short (≈3 s)	body text: use the font only if it comes quickly
`optional`	extremely short (≈100 ms)	**none**	body text where stability matters most; used from cache on later pages

`code/02_font_display.html` declares one font three times (`block`, `swap`, `optional`, with a Georgia fallback so you can see which is used). `code/02_font_display.mjs` intercepts the network and delays every font by 1.5 s (timings machine-dependent):

```
t≈301ms  F-block=loading  F-swap=loading  F-optional=loading
t≈3503ms  F-block=loaded  F-swap=loaded  F-optional=loaded
  .block     drawn with: Menlo, Inter (web font)
  .swap      drawn with: Menlo, Inter (web font)
  .optional  drawn with: Georgia, Menlo
```

(Menlo is the red `<code>` labels' font.) The `optional` font **did** download, but its window had closed, so the paragraph stays Georgia for this view.

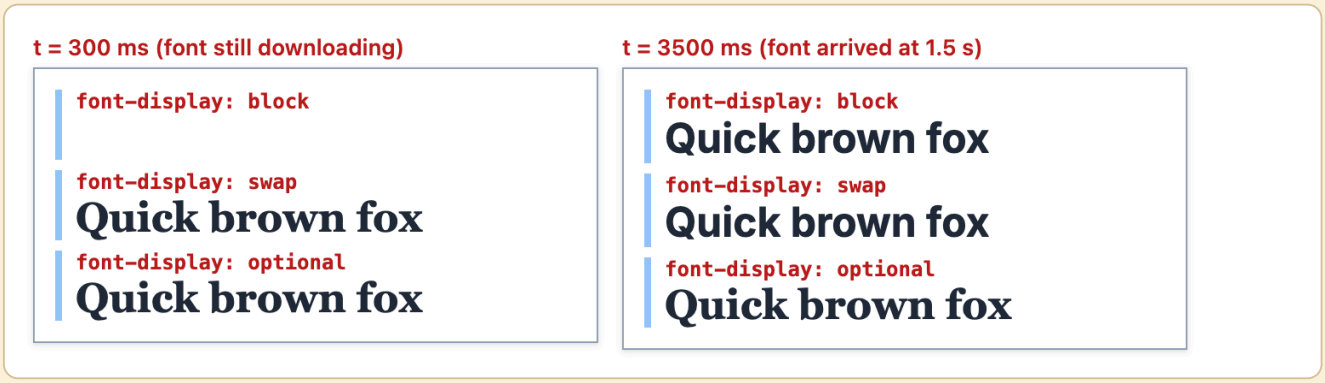


Figure 11.2 — FOIT vs FOUT vs "no swap", captured by delaying the font download.

Making fonts arrive sooner

Technique	How	Why
Preload the critical font	`<link rel="preload" href="/fonts/inter.woff2" as="font" type="font/woff2" crossorigin>`	Starts downloading during HTML parsing instead of after CSS. Font requests are always CORS-mode (Cross-Origin Resource Sharing), so `crossorigin` is required even same-origin — without it the preload doesn't match the real request and the font downloads twice. Preload only 1–2 fonts.
Self-host	Serve files from your own domain	No extra DNS/TLS connection (System Design Fundamentals); no third-party privacy issue (January 2022, Munich Regional Court — LG München I, case 3 O 17493/20 — held that loading Google-hosted fonts sent visitors' IP addresses to Google without consent, breaching the GDPR, and awarded the visitor €100)
Subset	Keep only needed characters; declare `unicode-range: U+0000-00FF, …`	The Inter file here is only the Latin subset (48 KB). A `unicode-range` face downloads only if the page uses one of its characters.
Match the fallback's metrics (letter sizes, line spacing)	`@font-face { font-family: "Inter Fallback"; src: local("Arial"); size-adjust: 107%; }`	The swap barely moves text, reducing CLS. `size-adjust` works in all current browsers (Safari since 17); `ascent-override`/`descent-override` not yet in Safari.
Fewer files	One variable font instead of 4–6 static ones	Fewer requests
Cache	Long `Cache-Control` max-age with hashed file names (fingerprinted, so new versions get new names)	Later pages are instant

From JavaScript

The **CSS Font Loading API** (`document.fonts`) exposes each face (`status: "unloaded" | "loading" | "loaded" | "error"`), `document.fonts.load('700 1em Inter')` to start a load, and `document.fonts.ready` (a promise), which the chapter scripts await before measuring.

In the interview

30-second answer: "Web fonts load late, so text either goes invisible (FOIT) or shows a fallback then swaps (FOUT), possibly shifting layout. I choose `font-display`: `swap` for brand text, `optional` or `fallback` for body text, `block` only for icon fonts. I self-host WOFF2, subset with `unicode-range`, preload the one or two critical files with `crossorigin`, use one variable font instead of many weights, and tune the fallback with `size-adjust` to reduce CLS."

- *When is `optional` better than `swap`?* → When layout stability matters more than showing the brand font on first view; it's cached and used on the next page.

11.3 Text properties

The essentials

Property	Example	Notes
`font-size`	`1.125rem`	`rem` so user settings work (Chapter 6); fluid with `clamp()` (§10.7)
`line-height`	`1.5`	**Unitless** (below). Body 1.4–1.6, headings 1.1–1.25
`letter-spacing`	`0.08em`	Small positive for ALL-CAPS labels; slightly negative for large headings
`word-spacing`	`0.1em`	Rarely needed
`text-align`	`start` | `end` | `center` | `justify`	`start`/`end` follow writing direction; avoid `justify` without hyphenation (rivers: gaps lining up down the paragraph)
`text-transform`	`uppercase` | `capitalize`	Keep meaningful capitals in the HTML; screen readers read the source
`text-indent`	`2em`	First-line indent (books)
`text-decoration`	`underline wavy red`	Longhands: `-line`, `-style` (solid/double/dotted/dashed/wavy), `-color`, `-thickness`
`text-underline-offset`	`0.2em`	Moves the underline away from descenders
`white-space`	`nowrap` | `pre` | `pre-wrap`	Space collapsing and wrapping
`overflow-wrap`	`anywhere` | `break-word`	Break a long word/URL only if it would overflow (§10.10)
`word-break`	`break-all`	Break between any letters — harsh; mainly CJK (Chinese, Japanese, Korean) or code
`hyphens`	`auto`	Dictionary hyphenation; needs a `lang` attribute; Baseline since 2023
`text-wrap`	`balance` | `pretty`	Better line breaking (below)
`font-variant-numeric`	`tabular-nums`	Equal-width digits so numbers align in tables and prices
`max-inline-size`	`65ch`	The **measure** — line length

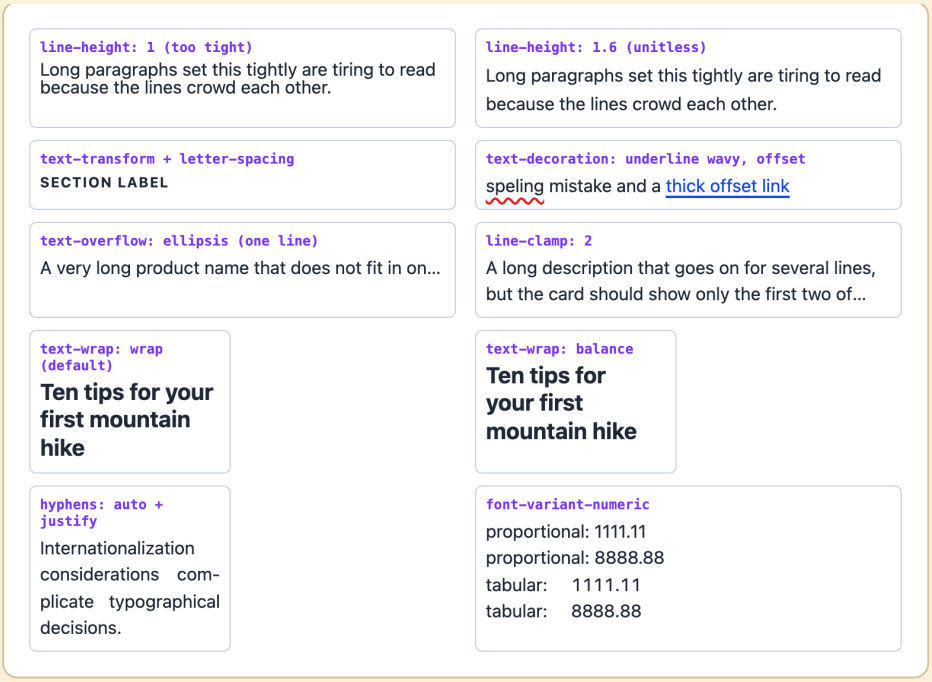


Figure 11.3 — `code/03_text.html`.

Why `line-height` should be unitless

`code/03_text.mjs` puts a 32px child in a 16px parent with `line-height: 1.5` vs `1.5em`:

```
line-height: 1.5   -> child (32px) line box 48px
line-height: 1.5em -> child (32px) computed 24px, line box 24px  (text taller than its line!)
proportional-nums width of "1" 6.73px, of "8" 9.36px
tabular-nums      width of "1" 9.22px, of "8" 9.22px
.caps letter-spacing computed: 0.96px (0.08em of 12px)
.balance text-wrap: balance
```

A unitless value is inherited as a **factor** and re-multiplied by each element's font size (32 × 1.5 = 48px). A length (`em`, `px`, `%`) is computed **once** on the parent (16 × 1.5 = 24px) and inherited as fixed pixels — bigger text overlaps its neighbours. The same output proves `tabular-nums`: every digit 9.22px wide.

Line length (the measure)

Comfortable reading needs roughly **45–75 characters per line**; `max-inline-size: 65ch` (`ch` ≈ width of "0") keeps paragraphs there at any width.

Better line breaking: `text-wrap`

- `text-wrap: balance` makes a short block's lines about equally long — no orphaned word under a long headline. Browsers only balance short blocks (it's expensive). **Baseline since May 2024.**
- `text-wrap: pretty` uses a slower paragraph algorithm, mainly avoiding a very short last line. Chrome and Safari 26, not Firefox — **not Baseline**; harmless as an enhancement.

Truncation

```
/* One line with "…" — all three are required */
.title { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }

/* N lines with "…" — the prefixed syntax is the one that works in every browser */
.desc { display: -webkit-box; -webkit-box-orient: vertical; -webkit-line-clamp: 2; overflow: hidden; }
```

Unprefixed `line-clamp` is specified but not implemented; the `-webkit-` version is interoperable. Truncation hides content from sighted users — keep the full text reachable (a `title` tooltip is **not** enough for touch and keyboard users).

Trap seen while building Figure 11.3: in a `1fr` grid column a `nowrap` line makes the **column** grow, so no ellipsis appears. Fix: `minmax(0, 1fr)` (§9.10) or `min-width: 0` on the item.

Common mistakes

- `line-height` in `px` on `body` — every heading inherits the same fixed spacing.
- Light grey, thin, small text — fails contrast (11.4).
- Centre-aligned long paragraphs; all-caps paragraphs.
- Removing link underlines in body text: colour mustn't be the only link cue (WCAG 1.4.1) unless link/text contrast is ≥ 3:1 *and* another cue appears on hover/focus.

11.4 Colour and contrast

§6.8 covered notations (hex, `rgb()`, `hsl()`, `oklch()`, alpha, `currentColor`). This section is about choosing readable colours and deriving palettes from one brand colour.

Contrast ratio — what it is

WCAG's **contrast ratio** is `(L1 + 0.05) / (L2 + 0.05)`, with L1/L2 the **relative luminance** (perceived brightness, 0 black to 1 white) of the lighter/darker colour — from **1:1** to **21:1**. The computation (undo sRGB gamma, weight green most, blue least) and its implementation are in §3.9; `code/contrast.mjs` is the same code. (The printed WCAG 2.0 formula uses a 0.03928 threshold; newer text corrects it to 0.04045 — identical results for 8-bit colours.)

The thresholds (WCAG 2.2)

4.5:1 normal text (1.4.3 AA); **3:1** large text (≥ 24px, or ≥ 18.66px bold); 7:1 / 4.5:1 for AAA (1.4.6); **3:1** for non-text UI against adjacent colours — icons, input borders, focus rings, chart lines (1.4.11 AA). Logos and purely decorative or disabled elements are exempt.

`code/04_contrast.mjs` — by the formula, then using colours the browser computed in `code/04_contrast.html`:

```
#000000 on #ffffff: 21.00:1  AAA
#767676 on #ffffff: 4.54:1  AA
#777777 on #ffffff: 4.48:1  large/UI only
#1d4ed8 on #ffffff: 6.70:1  AA
#ffffff on #3b82f6: 3.68:1  large/UI only
#e2e8f0 on #0f172a: 14.48:1  AAA
#fcd34d on #ffffff: 1.44:1  FAIL
#fff on var(--brand)                        -> rgb(255, 255, 255) on oklch(0.55 0.2 260)  = 5.02:1 AA
#fff on color-mix(in oklch, brand 70%, black) -> rgb(255, 255, 255) on oklch(0.385 0.14 260)  = 10.04:1 AAA
brand on oklch(from brand 95% 0.03 h)        -> oklch(0.55 0.2 260) on oklch(0.95 0.03 260)  = 4.32:1 large/UI only
#000 on oklch(from brand l c calc(h + 180))  -> rgb(0, 0, 0) on oklch(0.55 0.2 80)  = 4.25:1 large/UI only
```

(Four browser rows identical to formula rows are omitted.)

- **#767676** is the lightest AA grey on white; #777777 fails.
- White on a typical "blue-500" button (#3b82f6) is 3.68:1 — fine for large bold text, **not** normal labels; many design systems use a darker button shade for this reason.
- Yellow on white is nearly invisible (1.44:1).
- The complement (opposite hue) has the **same** OKLCH lightness (55%) as the brand, yet black on it gives 4.25:1 while white on the brand gives 5.02:1: OKLCH lightness is perceptual but *not* WCAG luminance — always check the actual pair.

Figure 11.4 — Contrast computed inside the page from the browser's own colours.

Deriving colours: `color-mix()` and relative colour syntax

Derive shades from one token instead of hard-coding them:

```
:root { --brand: oklch(55% 0.2 260); }
.darker     { background: color-mix(in oklch, var(--brand) 70%, black); }   /* 70% brand, 30% black */
.tint       { background: oklch(from var(--brand) 95% 0.03 h); }           /* keep hue, set L and C */
.complement { background: oklch(from var(--brand) l c calc(h + 180)); }    /* rotate the hue */
.faded      { background: rgb(from var(--brand) r g b / 50%); }            /* same colour, half opacity */
```

- `color-mix(in <space>, A p%, B)` — Baseline since 2023; mixing in `oklch`/`oklab` is more even than `srgb`.
- **Relative colour syntax** `<fn>(from <colour> …)` exposes the source's channels as keywords (`l c h`, `r g b`, `h s l`, `alpha`) to keep, replace or `calc()`. Baseline since September 2024 (newly available).

Other colour tools

- `currentColor` — the element's `color`; icons and borders that follow text.
- `accent-color: #1d4ed8` — tints checkboxes, radios, range sliders, progress bars. Not yet Baseline (Chrome for Android lacks it) but degrades gracefully.
- `color-scheme` and `light-dark()` — §10.5.

Accessibility beyond ratios

- **Never colour alone** (WCAG 1.4.1): add icon, text or pattern to "red = error"; underline links in text.
- About 1 in 12 men has a colour-vision deficiency, red/green the classic problem; DevTools (Rendering → Emulate vision deficiencies) simulates them.
- **APCA** (Accessible Perceptual Contrast Algorithm) is being explored for the future WCAG 3 — not a requirement today; audits and laws reference WCAG 2.2's ratio.

In the interview

30-second answer: "WCAG contrast is (L1 + 0.05)/(L2 + 0.05) using relative luminance; AA needs 4.5:1 for body text, 3:1 for large text and for UI like input borders and focus rings. I check real pairs with DevTools or a script — HSL or even OKLCH lightness doesn't guarantee it — and never rely on colour alone. For palettes I define a brand token in OKLCH and derive shades with `color-mix()` or relative colour syntax."

11.5 Backgrounds and gradients

The background properties

Property	Values	Notes
`background-color`	any colour	Painted under all image layers; always set one as a fallback under images
`background-image`	`url()`, gradients, `image-set()`	Comma-separated **layers**; the **first is on top**
`background-repeat`	`repeat` | `no-repeat` | `repeat-x` | `space` | `round`	Default `repeat`
`background-position`	`center`, `right 10px bottom 10px`, `30% 70%`	Where the image sits
`background-size`	`auto` | `cover` | `contain` | `200px 100px`	`cover` fills and crops; `contain` fits entirely
`background-origin`	`padding-box` | `border-box` | `content-box`	Box that positions the image
`background-clip`	`border-box` | `padding-box` | `content-box` | `text`	Box where painting stops
`background-attachment`	`scroll` | `fixed` | `local`	`fixed` is ignored on iOS, not Baseline, and costs a repaint every scroll
`background`	shorthand	`url(a.jpg) center / cover no-repeat, #e0f2fe` — `size` follows `position` after `/`

Gradients

A **gradient** is a browser-generated image — no download, crisp at any size, animatable only indirectly.

Function	Shape	Example
`linear-gradient()`	along a line	`linear-gradient(135deg, #f59e0b, #ef4444 50%, #7c3aed)`
`radial-gradient()`	circles/ellipses from a point	`radial-gradient(circle at 30% 30%, #fff 0 8%, #60a5fa 40%, #1e3a8a)`
`conic-gradient()`	around a point (like a clock hand)	`conic-gradient(#3b82f6 0 40%, #22c55e 0 70%, #f59e0b 0)` — a pie chart
`repeating-*-gradient()`	the pattern repeats	`repeating-linear-gradient(45deg, #fde68a 0 10px, #f59e0b 10px 20px)` — stripes

Colour stops are `colour [position] [position]`. Two stops at one position make a **hard edge** (`linear-gradient(to right, green 50%, grey 50%)` is a 50/50 split), and a `0` position after a larger one means "same as the previous stop" — how the conic pie gets sharp slices. Direction: `to right`, `to top left`, or an angle (`0deg` = up, `90deg` = right).

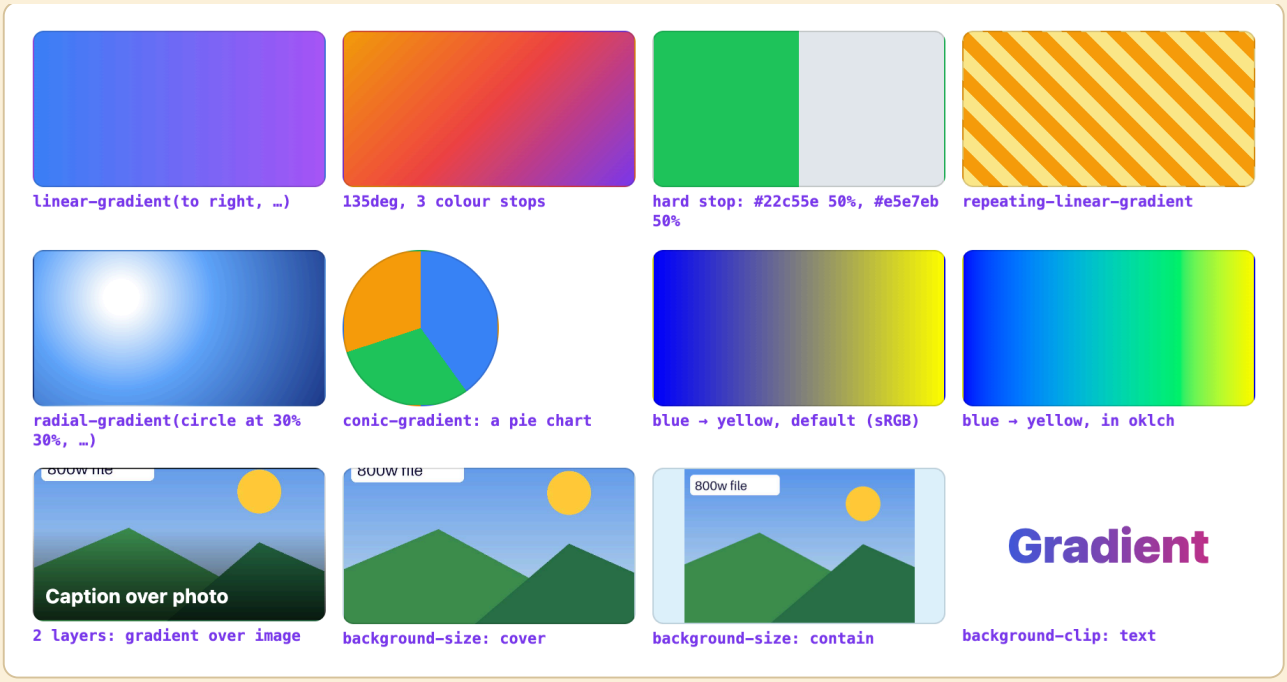


Figure 11.5 — `code/05_backgrounds.html`.

Interpolation colour space

Gradients blend in **sRGB** by default, so complementary colours pass through muddy grey. Since June 2024 (Baseline) you can pick the space: `linear-gradient(in oklch to right, blue, yellow)`. `code/05_backgrounds.mjs` reads screenshot pixels:

```
.srgb   at  50%: rgb(128, 128, 127)
.oklch  at  50%: rgb(0, 208, 188)
.hard   at  49%: rgb(34, 197, 94)
.hard   at  51%: rgb(229, 231, 235)
```

The sRGB midpoint of blue (0, 0, 255) and yellow (255, 255, 0) averages the channels to grey (128, 128, 127); in OKLCH the hue travels around the wheel through vivid teal. The `.hard` samples 2 % apart are the two pure stop colours.

Multiple backgrounds — a real use

Text over photos needs contrast: put a gradient layer **above** the image.

```css
.hero {
  background:
    linear-gradient(to top, rgb(0 0 0 / .75), transparent 60%),   /* layer 1: on top */
    url("hero.jpg") center / cover no-repeat;                     /* layer 2: below */
  color: #fff;
}
```

Gradient text

```css
.gradient-text { background: linear-gradient(90deg, #2563eb, #db2777);
                 -webkit-background-clip: text; background-clip: text; color: transparent; }
```

Write both lines: web-features (the dataset behind Baseline) still lists unprefixed `background-clip: text` as not Baseline. Check contrast at the gradient's lightest point and keep a solid `color` fallback in forced-colours mode (an OS high-contrast setting that replaces page colours).

Common mistakes

- The gradient layer **after** the image — it's underneath and invisible.
- No `background-color` under a photo: if it fails, white text sits on white.
- Meaningful images as CSS backgrounds — no `alt`, not announced; use `<img>` (Chapter 4).
- Huge background photos on mobile — use `image-set()` (§10.8).

11.6 Borders, radius and outlines

Borders

`border: <width> <style> <colour>` on all sides, per side (`border-left`) or per aspect (`border-width: 1px 2px`). **The style is required** — initial value `none`, so `border: 2px #333` shows nothing (Chapter 7). Styles: `solid`, `dashed`, `dotted`, `double`, `groove`, `ridge`, `inset`, `outset`, `hidden`, `none`. Borders take layout space. `border-image` slices an image into a border — rarely used.

`border-radius`

Rounds the corners of the **border box** (clipping the background with it):

Value	Result
`8px`	all four corners
`24px 0 24px 0`	top-left, top-right, bottom-right, bottom-left (clockwise)
`50%` on a square	a circle (`aspect-ratio: 1` keeps it square)
`999px`	a **pill**: an oversized radius is reduced to fit, so the ends are exact semicircles at any height
`50% / 30%`	**elliptical** corners: horizontal radii / vertical radii

Percentages use the box's own width (horizontal radius) and height (vertical). Overlapping adjacent radii are scaled down proportionally. (`corner-shape` for squircles and bevels is Chromium-only in 2026.)

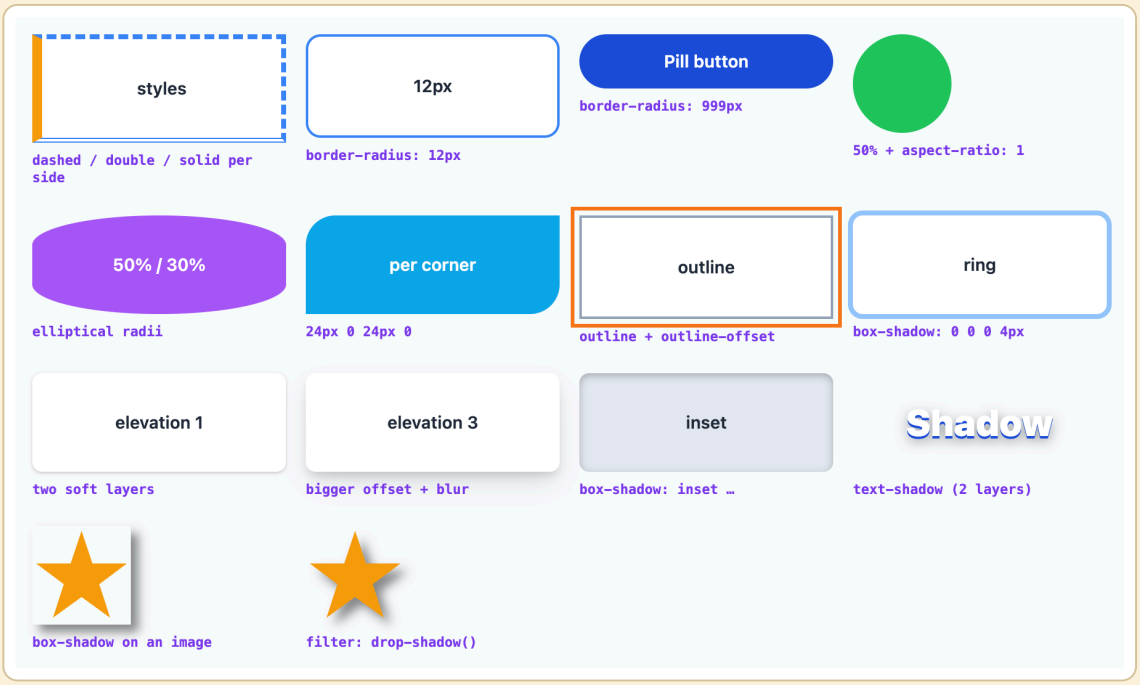


Figure 11.6 — `code/06_borders_shadows.html`.

Outline vs border vs ring

	`border`	`outline`	`box-shadow: 0 0 0 4px` ("ring")
Takes layout space	**yes**	no	no
Per-side control	yes	no	no
Follows `border-radius`	yes	yes (all current browsers since 2023)	yes
Offset	—	`outline-offset` (can be negative)	via extra layers
Shown in forced-colours mode	yes	**yes**	**no** (shadows are removed)
Main use	structure, dividers	**focus indicators**	decorative rings

`code/06_borders_shadows.mjs` confirms only borders change layout size (every box has `height: 80px`):

```
.styles   layout box 206x88  border-left 8px  outline none       shadow none
.outline  layout box 206x84  border-left 2px  outline 3px +4px   shadow none
.ring     layout box 206x80  border-left 0px  outline none       shadow rgb(147, 197, 253) 0px 0px 0px 4px
.elev3    layout box 206x80  border-left 0px  outline none       shadow rgba(0, 0, 0, 0.1) 0px 4px 6px -1px, rgba(0,
.ellipse  border-top-left-radius: 50% 30%
```

So focus styles use `outline` — `:focus-visible { outline: 3px solid #2563eb; outline-offset: 2px; }` doesn't shift layout and survives Windows contrast themes. Never remove it without a replacement (§3.7). A popular pattern: a transparent outline plus a box-shadow ring, so forced-colours mode still gets the outline.

11.7 Shadows

`box-shadow`

```
box-shadow: [inset] <offset-x> <offset-y> [<blur>] [<spread>] <colour>, … ;
```

Part	Effect
offset-x / offset-y	Moves the shadow right/down (negative = left/up) — implies light direction
blur	Softness; 0 = hard edge
spread	Grows (positive) or shrinks (negative) the shadow before blurring
`inset`	Drawn inside the box (pressed/sunken look)
multiple, comma-separated	First on top; layer a tight dark shadow and a large soft one for realistic depth

Elevation systems (Material Design and most component libraries) show how "high" a surface floats with increasingly large, soft, offset shadows:

```
.elev1 { box-shadow: 0 1px 2px rgb(0 0 0 / .12), 0 1px 3px rgb(0 0 0 / .08); }
.elev3 { box-shadow: 0 4px 6px -1px rgb(0 0 0 / .1), 0 10px 20px -5px rgb(0 0 0 / .15); }
```

The negative spread keeps the large soft shadow from peeking out at the sides. Shadows don't affect layout and paint outside the box (so a **parent's** `overflow: hidden` clips them).

`text-shadow`

`text-shadow: <x> <y> [<blur>] <colour>, …` — no spread, no inset. For legibility on busy images (a soft dark halo) or display effects; keep body text shadow-free.

`filter: drop-shadow()`

`box-shadow` follows the **box**; `filter: drop-shadow(x y blur colour)` follows the element's **alpha shape** (its non-transparent pixels) — the star in Figure 11.6 gets a star-shaped shadow, as would a transparent PNG logo. No spread or inset, and costlier to render.

Performance

Large blurred shadows are expensive to paint, and **animating** `box-shadow` repaints every frame. To animate elevation, put the bigger shadow on a pseudo-element and animate its `opacity` (Chapter 12).

11.8 Filters, blend modes, clipping and masks

Figure 11.7 — `code/07_filters_blend.html`.

`filter`

Applies image-processing functions to an element **and everything inside it**, in the order written:

Function	Example	Effect
`blur()`	`blur(3px)`	Gaussian (smooth, even) blur
`brightness()`	`brightness(1.4)`	< 1 darker, > 1 brighter
`contrast()`	`contrast(1.8)`	
`grayscale()`	`grayscale(1)`	0–1 (or %)
`hue-rotate()`	`hue-rotate(120deg)`	Rotates hues
`invert()`	`invert(1)`	
`opacity()`	`opacity(.5)`	Like the `opacity` property
`saturate()`	`saturate(3)`	
`sepia()`	`sepia(1)`	
`drop-shadow()`	11.7	
`url(#svgFilter)`		Any SVG filter

Baseline widely available since 2016. Uses: greyscale logos that colour on hover, dimming images in dark mode, disabled states.

`backdrop-filter`

Filters what's **behind** the element (which must be partly transparent) — the "frosted glass" of macOS/iOS panels: `background: rgb(255 255 255 / .3); backdrop-filter: blur(6px) saturate(1.4);`. Baseline since September 2024 (Safari 18 dropped the `-webkit-` requirement). Check text contrast against the *worst* possible background and provide a more opaque fallback for `prefers-reduced-transparency` where supported.

Blend modes

- `mix-blend-mode` — how the element's pixels combine with content **behind** it.
- `background-blend-mode` — how an element's **own background layers** combine; used for duotone images (recoloured in two tones).
- `isolation: isolate` — a new stacking context so blending stays inside a group (§7.8).

Per channel, values 0–1 (a = top, b = backdrop): `multiply` = a·b (always darker; white disappears), `screen` = 1 − (1−a)(1−b) (always lighter; black disappears), `overlay`, `darken` / `lighten`, `difference` = |a − b|, `luminosity` / `color` / `hue` / `saturation` (mix HSL-like components). `code/07_filters_blend.mjs` blends #ef4444 over sky blue and compares the screenshot pixel with the formula:

```
multiply   by hand rgb(94, 40, 63)  browser rgb(94, 40, 63)
screen     by hand rgb(245, 178, 240)  browser rgb(245, 178, 240)
difference by hand rgb(139, 82, 167)  browser rgb(139, 82, 167)
```

Clipping and masking

- `clip-path` hides everything outside a shape: `circle(40% at 50% 50%)`, `ellipse()`, `inset(10px round 8px)`, `polygon(0 0, 100% 0, 100% 75%, 0 100%)`, `path("…")`, or SVG `url(#clip)`. Hard-edged; clicks outside pass through. Baseline since 2021 (animatable between shapes with equal point counts).
- `mask-image` uses an image's **alpha** (or luminance) as a soft mask: `mask-image: linear-gradient(to right, #000 40%, transparent)` fades an element out. Unprefixed masks Baseline since December 2023.

Side effects to remember

`filter`, `backdrop-filter`, `mix-blend-mode`, `clip-path`, `mask` or `opacity < 1` **create a stacking context** (§7.8); a non-`none` `filter` or `backdrop-filter` also becomes the **containing block for** `position: fixed` **descendants** — a "fixed" header inside a filtered parent scrolls with it. Filters are GPU work; large blurred areas on low-end phones can drop frames.

11.9 `object-fit` and `object-position`

What it is

For **replaced elements** (content from an external source, not CSS) — `<img>`, `<video>`, `<canvas>`, `<iframe>` content — `object-fit` decides how the content fills the element's box when aspect ratios differ: the `background-size` of images. Baseline widely available (2020).

Value	Scale factor	Result
`fill` (default)	separate x and y	Stretched — **distorted**
`contain`	min(box/content) per axis	Whole image visible, **letterboxed** (empty bars)
`cover`	max(box/content) per axis	Box filled, overflow **cropped**
`none`	1	Natural size, centred, cropped
`scale-down`	the smaller of `none` and `contain`	Never enlarges

`object-position` (default `50% 50%`) chooses which part stays visible when cropping, or where a letterboxed image sits — like `background-position` (`0% 50%` keeps the left edge, vertically centred).

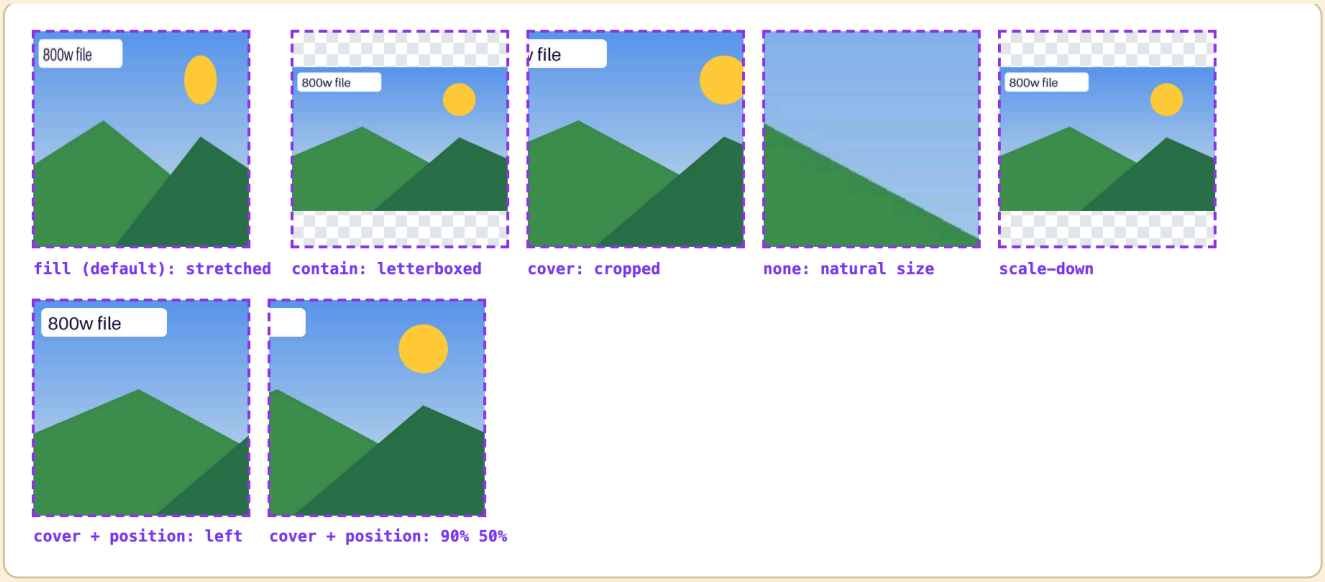


Figure 11.8 — `code/08_object_fit.html`. Dashed purple = the element's box; checkerboard = empty space.

`code/08_object_fit.mjs` prints each box's computed values, then the maths by hand from the natural size:

```
fill         box 150×150  natural 800×533  object-fit fill       object-position 50% 50%
...
cover-right  box 150×150  natural 800×533  object-fit cover      object-position 90% 50%

contain: scale = min(150/800, 150/533) = 0.1875 -> drawn 150×99.94 (bars above and below)
cover:   scale = max(150/800, 150/533) = 0.2814 -> drawn 225.14×150, 75.14px cropped horizontally
object-position 90% 50% on cover: image x offset = (150 - 225.14) × 0.9 = -67.63px
scale-down: the smaller of none (scale 1) and contain (0.1875) -> same as contain here
```

A percentage position aligns that percentage point of the image with the same point of the box, so offset = (box − image) × percentage — the `background-position` rule. (Example: 800×400 in 200×200 → `cover` scale 0.5 → 400×200, 200px cropped; `contain` scale 0.25 → 200×100, 50px bars.)

Real-world use

- **Card thumbnails / avatars** of unpredictable shape: `width: 100%; aspect-ratio: 4 / 3; object-fit: cover;` (+ `border-radius: 50%` for avatars).
- **Product photos** that mustn't be cropped: `object-fit: contain` on a neutral background.
- **Hero video**: `<video autoplay muted loop playsinline>` + `object-fit: cover` for a full-width banner.
- **Grid cells**: images don't stretch by default (§9.10); `width: 100%; height: 100%; object-fit: cover` fills them.
- When cropping could cut off the subject (faces), set `object-position` per image, e.g. inline from a CMS (content management system) focal point.

Common mistakes

- Only `width` on an `<img>` — the box needs both dimensions (via `height` or `aspect-ratio`) differing from the image's ratio for `object-fit` to matter.
- `object-fit` on a `<div>` — replaced elements only; use `background-size`.
- `cover` cropping important content (text in images).

Chapter 11 TL;DR

- **Fonts**: stacks end with a generic family (`system-ui` stack = 0 bytes); `@font-face` + **WOFF2**; one **variable font** (`font-weight: 100 900`) replaces many files; missing faces are **synthesized** unless `font-synthesis: none`.
- **Loading**: `font-display` `block` (invisible ≤3 s) / `swap` / `fallback` / `optional` (≈100 ms, then never); preload 1–2 fonts **with** `crossorigin`, self-host, subset with `unicode-range`, `size-adjust` the fallback.
- **Text**: `rem` sizes, **unitless** `line-height`, 45–75ch measure, `text-wrap: balance` (Baseline 2024), ellipsis = nowrap + hidden + `text-overflow`, `-webkit-line-clamp`, `tabular-nums`, `hyphens: auto` needs `lang`.
- **Contrast**: (L1+0.05)/(L2+0.05); AA 4.5:1 text, 3:1 large text and UI; #767676 is the lightest AA grey on white; never colour alone; derive shades with `color-mix(in oklch, …)` / `oklch(from var(--c) l c h)` and still

check each pair.

- **Backgrounds**: first layer on top; `cover`/`contain`; linear/radial/conic/repeating gradients, hard stops, `in oklch` avoids grey midpoints.
- **Borders/shadows**: border style required; `999px` pills, `a / b` ellipses; focus = `outline` (no layout, survives forced colours); layered `box-shadow` for elevation; `drop-shadow()` follows alpha; don't animate shadows directly.
- **Effects**: `filter`, `backdrop-filter` (Baseline 2024), blend modes (multiply = a·b, screen, difference), `isolation`, `clip-path`, `mask-image` — all create stacking contexts; filters also capture `position: fixed`.
- **`object-fit`**: fill / contain (min scale) / cover (max scale) / none / scale-down, + `object-position`.

Next: Chapter 12 — Transitions, Transforms and Animation.

Chapter 12 — Transitions, Transforms and Animation

Motion, used well, explains what just happened: a menu slides out of its button, a card grows into the detail view, a spinner says "working". Used badly it distracts, makes some people physically ill, and causes **jank** (visible stutter when the browser misses frames). CSS has three core tools — **transforms** (move, rotate, scale without touching layout), **transitions** (animate a change between states) and **keyframe animations** (a timeline of states) — plus newer ones for entering/leaving elements, **view transitions** and **scroll-driven** animation. This chapter shows how the browser computes every in-between frame, then covers performance and accessibility.

In this chapter: 12.1 Transitions · 12.2 Timing functions · 12.3 2D transforms · 12.4 3D transforms · 12.5 Keyframe animations · 12.6 The Web Animations API · 12.7 Performance · 12.8 Enter and exit animations (`@starting-style`, `allow-discrete`) · 12.9 Reduced motion and accessibility · 12.10 View transitions · 12.11 Scroll-driven animations.

How the figures and numbers were made. Every example is a page in `code/`. `code/make_figures.mjs` makes **filmstrips**: it pauses every animation with the Web Animations API (12.6; CSS transitions and animations appear as `CSSTransition`/`CSSAnimation` objects in `document.getAnimations()`), sets `currentTime` to an exact moment and screenshots — so each frame is exactly t = 0, 200, 400 … ms. The same trick in `code/anim.mjs` lets the `NN_*.mjs` scripts (Node + Puppeteer driving headless Chrome) read **computed values mid-animation** and compare them with the maths; measurement uses `code/measure.mjs` (Chapter 10's helper).

```
// code/anim.mjs — freeze every animation at the same moment
export const seek = (page, ms) => page.evaluate(ms => {
  for (const a of document.getAnimations()) { a.pause(); a.currentTime = ms; }
  return new Promise(r => requestAnimationFrame(() => requestAnimationFrame(r)));   // let the frame render
}, ms);
```

12.1 Transitions

What it is

A **transition** says: "when this property's value changes, move smoothly from old to new over this time". Something else changes the value (`:hover`/`:focus-visible`, a JS-added class, a media query); the transition only animates the change.

```
.button {
  background: #2563eb;
  transition: background-color 200ms ease-out, translate 150ms ease-out;
}
.button:hover { background: #1d4ed8; translate: 0 -2px; }
```

How it works

1. The browser computes the style before and after the change.
2. For each `transition-property` whose value changed **and can be interpolated** (sensible in-between values exist, as for 10px → 20px), it creates a transition.
3. Each **frame** (one screen redraw, typically 60 per second) it computes *progress* = timing-function(elapsed / duration) and sets old + (new − old) × progress.
4. If the value changes again mid-way, the transition is replaced by one starting from the **current** value; reversing a half-finished transition is **shortened** proportionally (un-hovering after 50ms doesn't take the full duration).

The properties

Longhand	Example	Meaning
`transition-property`	`opacity, translate` | `all`	What to animate (`all` = every animatable property — convenient but can animate things you didn't intend)
`transition-duration`	`200ms`	How long (default `0s` = no transition)
`transition-timing-function`	`ease-out`	Speed curve (12.2); default `ease`
`transition-delay`	`100ms`	Wait before starting (negative = start part-way)
`transition-behavior`	`allow-discrete`	Also transition **discrete** properties like `display` (12.8)
`transition`	`opacity 200ms ease-out 50ms`	Shorthand; first time = duration, second = delay; comma-separate several

What can be animated

- **Interpolable**: lengths, percentages, numbers, colours, transforms, shadows, gradient colour stops (not whole gradients in most browsers), `grid-template-columns` track sizes (Baseline since 2022), `clip-path` shapes with equal point counts…
- **Discrete** (`display`, `visibility`, `font-family`, `position`…) can't be in-betweened: normally they flip; with `transition-behavior: allow-discrete` they flip at 50 % — except `display`/`content-visibility`, which stay visible for the whole transition (12.8).
- `auto` isn't interpolable: `height: 0 → auto` jumps. Chromium's `interpolate-size: allow-keywords` / `calc-size()` fix it but are **not Baseline** (§1.1 defines Baseline); portable options: animate `grid-template-rows: 0fr → 1fr` on a wrapper, or measure the height in JS.

Events

`transitionrun`, `transitionstart` (after the delay), `transitionend` (once **per property** — two for `opacity .2s, translate .2s`), `transitioncancel` (interrupted). Filter by `event.propertyName` so a `background-color` `transitionend` doesn't close your menu early.

Common mistakes

- `transition` only in the `:hover` rule — hover-in animates, hover-out is instant. Put it on the base rule.
- `transition: all 300ms` where layout properties change — animates widths and paddings you never meant to (expensively, 12.7).
- Expecting a transition when an element is **added** or leaves `display: none` — no "before" style exists. `@starting-style` solves it (12.8).
- Too long: UI feedback feels right around **100–300ms**; larger movements up to ~500ms.

12.2 Timing functions (easing)

What it is

A **timing function** (*easing*) maps **time progress** (0 → 1) to **output progress** (usually 0 → 1). Linear motion looks mechanical; real objects accelerate and decelerate.

Function	Shape	Typical use
`linear`	straight line	spinners, progress, colour cycling
`ease` (default)	quick start, long gentle stop	general purpose
`ease-in`	slow start, fast end	things **leaving** the screen
`ease-out`	fast start, slow end	things **entering** or responding to input (snappy)
`ease-in-out`	slow–fast–slow	moving between two on-screen positions
`cubic-bezier(x1, y1, x2, y2)`	any smooth curve; y may leave 0–1 for overshoot	brand motion, "back" effects
`steps(n, jump-start | jump-end | jump-none | jump-both)`	n discrete jumps	sprite sheets (one image holding every frame), typewriter, ticking clocks
`linear(0, 0.36 30%, 1 45%, …)`	straight lines through listed points	bounces and springs (Baseline since December 2023)

Keywords are shortcuts: `ease` = `cubic-bezier(.25, .1, .25, 1)`, `ease-in` = `(.42, 0, 1, 1)`, `ease-out` = `(0, 0, .58, 1)`, `ease-in-out` = `(.42, 0, .58, 1)`; `step-start` = `steps(1, jump-start)`, `step-end` = `steps(1, jump-end)`.

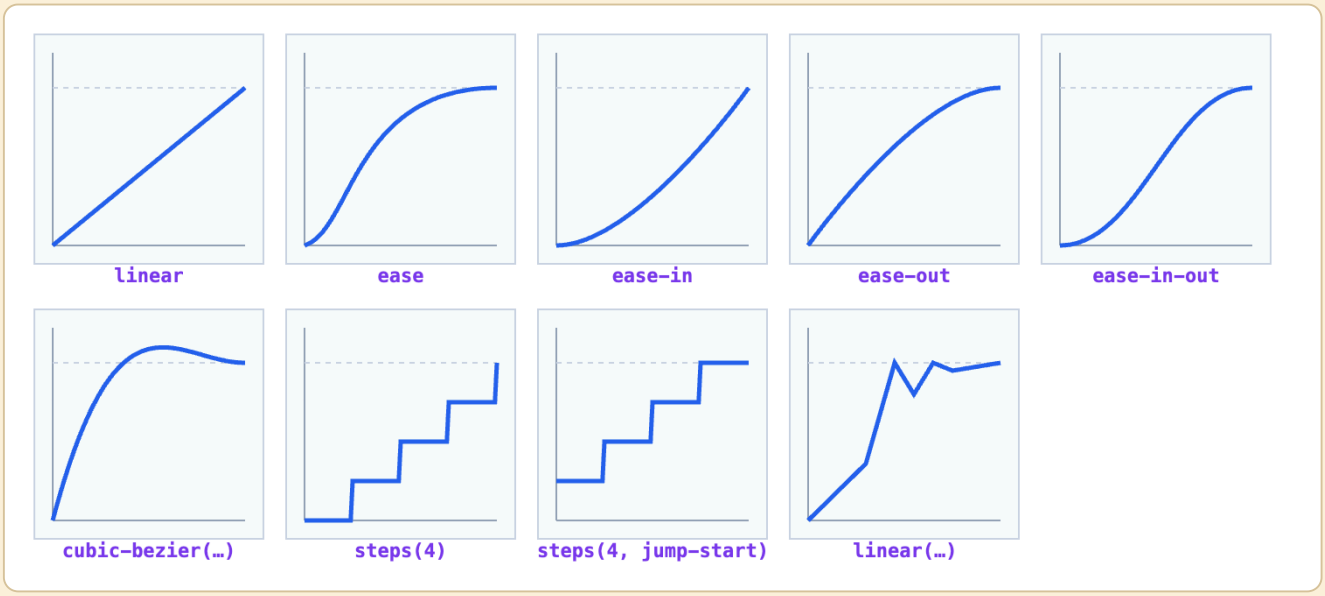


Figure 12.1 — `code/02_curves.html` animates a probe per easing, pauses it at 101 moments and reads the computed value — Chrome's real curves. Dashed line = 100 %.

How a cubic Bézier is computed

`cubic-bezier(x1, y1, x2, y2)` is a curve from (0, 0) to (1, 1) with two control points, both coordinates functions of a parameter *s*: `x(s) = 3·x1·s·(1−s)² + 3·x2·s²·(1−s) + s³`, likewise `y` with `y1`, `y2`. For time *t* the browser **solves x(s) = t** for *s* — x is time, so it must stay within 0–1 to give one answer; y is output and may overshoot — then returns y(s). `code/01_timing.mjs` does this by **bisection** (halve the range containing the answer until close enough) and compares with the browser.

The proof: a race

`code/01_timing.html` transitions eight balls 160px over 1000ms, each with a different easing:

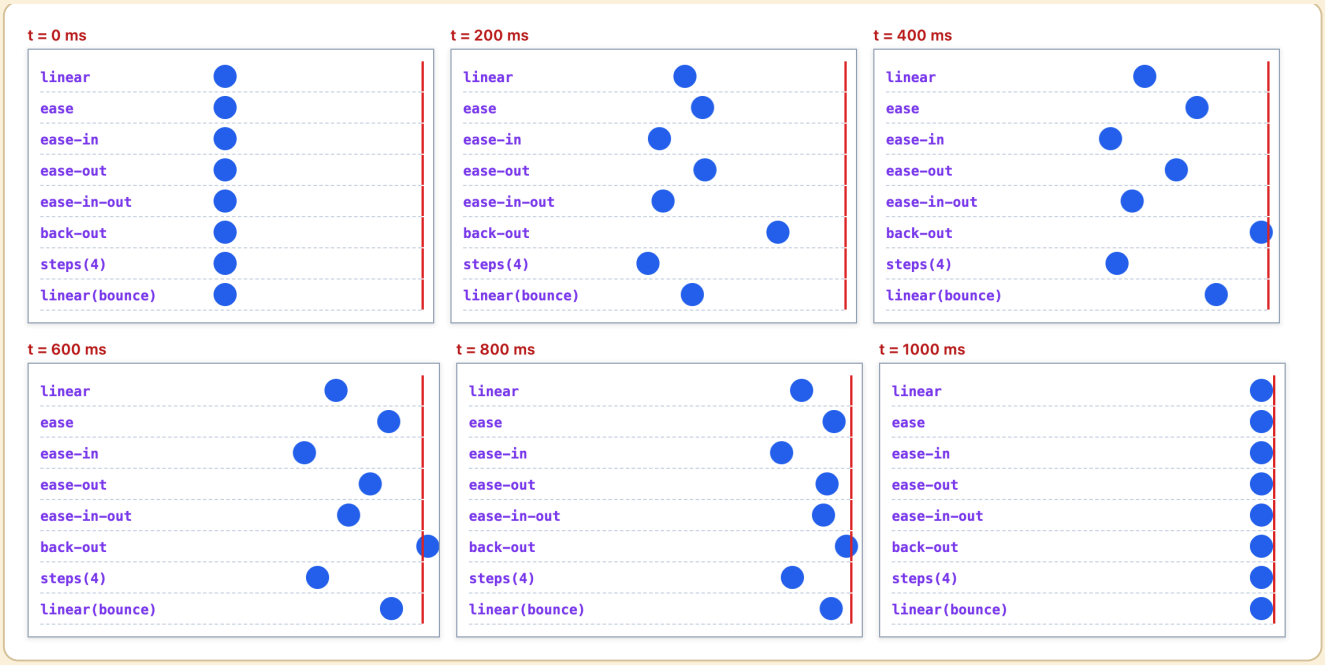


Figure 12.2 — Filmstrip made by pausing all eight transitions and seeking them to each time.

Browser values (the script prints a second "our maths" row under each; every one is identical to three decimals, so they're merged here):

```
progress (0 = start, 1 = end) at t =        0ms   200ms   400ms   600ms   800ms  1000ms
linear          browser       0.000   0.200   0.400   0.600   0.800   1.000
ease            browser       0.000   0.295   0.683   0.885   0.976   1.000
ease-in         browser       0.000   0.062   0.215   0.429   0.692   1.000
ease-out        browser       0.000   0.308   0.571   0.785   0.938   1.000
ease-in-out     browser       0.000   0.082   0.332   0.668   0.918   1.000
back-out        browser       0.000   0.703   1.030   1.097   1.042   1.000
steps(4)        browser       0.000   0.000   0.250   0.500   0.750   1.000
linear(bounce)  browser       0.000   0.240   0.787   0.900   0.960   1.000
```

- `ease` covers 68 % of the distance at 40 % of the time — why it feels quick.
- `back-out` = `cubic-bezier(.34, 1.56, .64, 1)`: y1 = 1.56 > 1, so progress overshoots (109.7 % at 600ms) and settles.
- `steps(4)` defaults to `jump-end`: the value changes at the *end* of each quarter, so at 200ms it's still 0 (`steps(2)` over 1s → 0.5 at 0.6s).
- `linear(0, 0.36 30%, 1 45%, 0.8 55%, …)`: interpolates linearly between points; at 600ms (between 55 % → 0.8 and 65 % → 1) it's 0.9.

Choosing

Entering/responding: `ease-out` (or a `cubic-bezier(.2, .8, .2, 1)`-style curve). Leaving: `ease-in`, shorter than the entrance. On-screen moves: `ease-in-out`. Continuous rotation: `linear`. Design systems define a few named easing/duration **tokens** (`--ease-out`, `--dur-fast: 150ms`) and use only those.

12.3 2D transforms

What it is

`transform` changes where an element is **painted** — moved, rotated, scaled, skewed — **without changing its layout box**: neighbours don't move (so a hover `translate: 0 -4px` never shifts the elements below), and the flow space stays reserved. That's what makes transforms ideal for animation.

Function	Example	Effect
`translate(x, y)` / `translateX()` / `translateY()`	`translate(40px, 20px)`	Move; `%` is relative to the element's **own** size — `top: 50%; left: 50%; translate: -50% -50%` centres an absolutely positioned element of unknown size
`rotate(angle)`	`rotate(30deg)`, `0.25turn`	Clockwise rotation
`scale(sx, sy)`	`scale(1.4, 0.6)`	Resize (1 = unchanged; negative flips)
`skew(ax, ay)` / `skewX()`	`skewX(-20deg)`	Slant
`matrix(a, b, c, d, e, f)`	—	The general form everything reduces to

`transform-origin` (default `50% 50%`, the centre) is the fixed point for rotate, scale and skew.

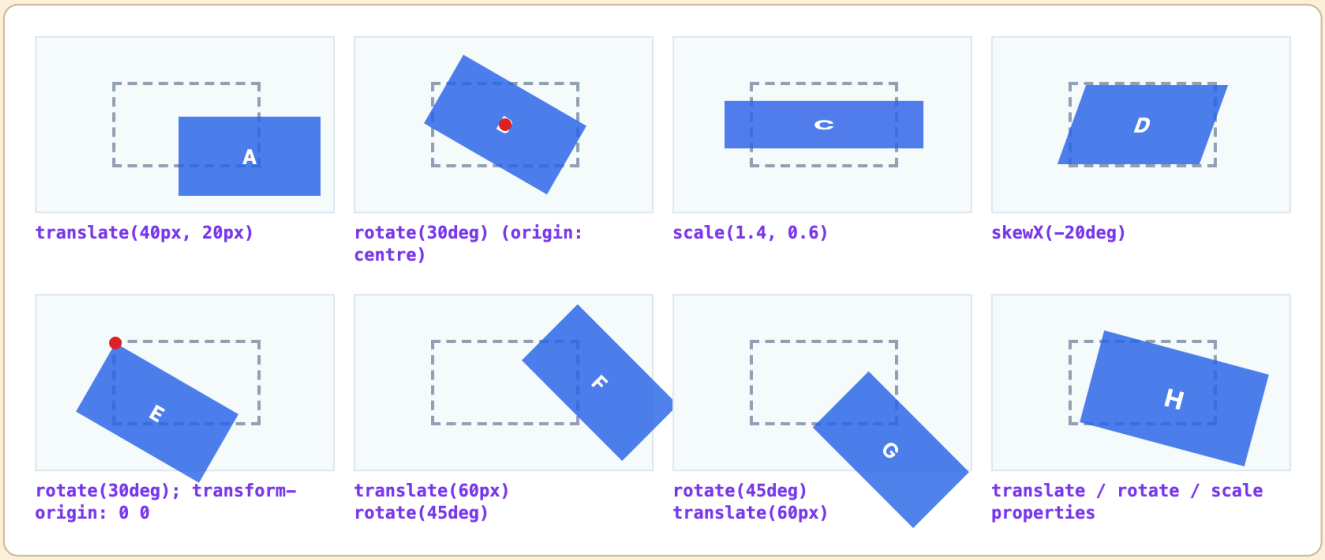


Figure 12.3 — `code/03_transforms.html`. Red dots mark the transform origin.

Order matters

`translate(60px, 0) rotate(45deg)` (F) and `rotate(45deg) translate(60px, 0)` (G) differ:

- Read **right to left** as operations on the element: F = rotate, then move right 60px.
- Or **left to right** as moving the **coordinate system**: G = rotate the axes 45°, then move 60px along the *rotated* x-axis — diagonally. (So `rotate(90deg) translateX(100px)` moves the element *down*.)

`code/03_transforms.mjs` prints the layout box (unchanged for all), the painted **bounding box** (smallest upright rectangle enclosing the drawn shape) and the resolved matrix:

```
A t1: layout 50,30 90x50 | painted bbox 90,50 90x50 | transform matrix(1, 0, 0, 1, 40, 20)
B t2: layout 50,30 90x50 | painted bbox 43.5,10.8 102.9x88.3 | transform matrix(0.866025, 0.5, -0.5, 0.866025, 0, 0)
C t3: layout 50,30 90x50 | painted bbox 32,40 126x30 | transform matrix(1.4, 0, 0, 0.6, 0, 0)
D t4: layout 50,30 90x50 | painted bbox 40.9,30 108.2x50 | transform matrix(1, 0, -0.36397, 1, 0, 0)
E t5: layout 50,30 90x50 | painted bbox 25,30 102.9x88.3 | transform matrix(0.866025, 0.5, -0.5, 0.866025, 0, 0)
F t6: layout 50,30 90x50 | painted bbox 105.5,5.5 99x99 | transform matrix(0.707107, 0.707107, -0.707107, 0.707107, 60, 0)
G t7: layout 50,30 90x50 | painted bbox 87.9,47.9 99x99 | transform matrix(0.707107, 0.707107, -0.707107, 0.707107,
42.4264, 42.4264)
H t8: layout 50,30 90x50 | painted bbox 55.1,22 119.8x85.9 | transform none
H individual properties: translate 20px 10px, rotate 15deg, scale 1.2 (transform stays none)
```

Check B: a 90×50 box rotated 30° has a bounding box 90·cos30° + 50·sin30° = 77.9 + 25 = **102.9** wide and 90·sin30° + 50·cos30° = 45 + 43.3 = **88.3** tall ✓. In `matrix(a, b, c, d, e, f)`, (a, b, c, d) = (cos θ, sin θ, −sin θ, cos θ) for a rotation and (e, f) is the translation: G's is 60·cos45° = **42.43** in x and y — the 60px move along the rotated axis ✓. B and E share a matrix; only `transform-origin` differs.

Individual transform properties

`translate`, `rotate` and `scale` are also **separate properties** (Baseline since 2022, widely available), applied in a fixed order — translate, rotate, scale, then `transform` — so order surprises can't happen, and each animates

independently (a hover `scale` doesn't clobber an animation's `translate`). They don't appear in `transform` (H shows `none`). This chapter's pages use them wherever possible.

Side effects

- A non-`none` transform creates a **stacking context** and becomes the **containing block** for `position: fixed` and `absolute` descendants (§7.6, §7.8).
- Transforms don't apply to non-replaced **inline** boxes (a plain `<span>`); make it `inline-block`.
- Scaled-up text can look blurry mid-animation on some GPUs; animate to a crisp final state.
- `getBoundingClientRect()` returns the **painted** box (transform included); `offsetWidth`/`offsetLeft` the layout box.

12.4 3D transforms

What it is

3D adds a **z-axis** toward the viewer: `translateZ()`, `rotateX()`, `rotateY()`, `rotate3d()`, `scale3d()`, `matrix3d()`, and `rotate: y 60deg`. To look 3D they need **perspective**.

Property	On	Meaning
`perspective: 600px`	the **parent** (scene)	Viewer's distance to the z = 0 plane; smaller = stronger distortion. All children share one vanishing point.
`perspective()` function	the element's own `transform`	Per-element perspective (each has its own vanishing point)
`perspective-origin`	the parent	Where the viewer's eye is (default centre)
`transform-style: preserve-3d`	a parent whose children stay in 3D	Otherwise children are flattened into its plane
`backface-visibility: hidden`	a face	Hide the element when its back faces the viewer

Figure 12.4 — `code/04_3d.html`.

The flip card, step by step

```
.scene { perspective: 600px; }                                  /* 1. give the scene depth */
.card  { position: relative; transform-style: preserve-3d;       /* 2. keep both faces in 3D */
         transition: rotate 600ms ease-in-out; }
.face  { position: absolute; inset: 0; backface-visibility: hidden; }   /* 3. hide a face pointing away */
.back  { rotate: y 180deg; }                                     /* 4. the back starts facing away */
.scene:hover .card, .card.flipped { rotate: y 180deg; }          /* 5. flip the whole card */
```

Past 90° the front points away (hidden) and the pre-rotated back faces the viewer — hence `backface-visibility: hidden` on both faces. In a real component flip on a **button** (toggle `.flipped`), not only hover, so keyboard and touch users can use it.

Things that flatten 3D

`overflow` other than `visible`, `opacity < 1`, `filter`, `clip-path`, `mask`, `mix-blend-mode`, `isolation: isolate` on the `preserve-3d` element render its children flat. If your cube collapses, look for one of these.

12.5 Keyframe animations

What it is

A **keyframe animation** runs a named sequence of states independent of any state change — it can start on load, loop, alternate and be delayed.

```
@keyframes bounce {
  0%, 100% { translate: 0 -12px; animation-timing-function: ease-in; }   /* falling: speed up   */
  50%      { translate: 0 12px;  animation-timing-function: ease-out; }  /* rising: slow down   */
}
.ball { animation: bounce 1000ms infinite; }

@keyframes spin { to { rotate: 1turn; } }        /* missing "from" = the element's own current value */
.spinner { animation: spin 1s linear infinite; }
```

The `animation-*` properties

Longhand	Values	Notes
`animation-name`	`bounce`	The `@keyframes` name
`animation-duration`	`1s`	One iteration
`animation-timing-function`	`ease` …	Applies **between each pair of keyframes**, not across the whole animation; overridable per keyframe (as in `bounce`)
`animation-delay`	`500ms`, `-250ms`	A **negative** delay starts part-way — handy for staggering spinners
`animation-iteration-count`	`3`, `infinite`, `0.5`	
`animation-direction`	`normal` | `reverse` | `alternate` | `alternate-reverse`	`alternate` plays every other iteration backwards
`animation-fill-mode`	`none` | `forwards` | `backwards` | `both`	What applies **before** (during the delay) and **after**
`animation-play-state`	`running` | `paused`	Pause/resume from CSS
`animation-composition`	`replace` | `add` | `accumulate`	Combining with the underlying value (Baseline 2023)
`animation-timeline`	`auto` | `scroll()` | `view()`	Drive by scroll instead of time (12.11)
`animation`	`bounce 1s ease-in-out 200ms infinite alternate both`	Shorthand; first time = duration, second = delay (`pulse 2s 500ms` → 500ms delay)

Keyframe rules: `from` = `0%`, `to` = `100%`; a missing 0 %/100 % uses the element's normal value; percentages can be grouped (`0%, 100%`); `!important` inside keyframes is ignored. While running, animation values beat normal declarations but lose to `!important` ones (§6.5).

The proof: values mid-animation

`code/05_keyframes.html` runs a bounce, a spinner, an alternating slide and four copies of a colour animation (`red → green`, 500ms, **`after a 500ms delay`**) with different fill modes; `code/05_keyframes.mjs` seeks them:

```
t=   0ms bounce 0px -12px spin 0deg     slide 0px       | fill none=grey (own style), forwards=grey (own style),
backwards=red, both=red
t= 250ms bounce 0px -4.43144px spin 90deg    slide 20.6659px | fill none=grey (own style), forwards=grey (own style),
backwards=red, both=red
t= 499ms bounce 0px 11.9179px spin 179.64deg slide 79.7241px | fill none=grey (own style), forwards=grey (own style),
backwards=red, both=red
t= 750ms bounce 0px -4.43144px spin 270deg   slide 139.334px | fill none=#796538, forwards=#796538, backwards=#796538,
both=#796538
t=1000ms bounce 0px -12px spin 0deg     slide 160px     | fill none=grey (own style), forwards=green, backwards=grey (own
style), both=green
t=1500ms bounce 0px 12px  spin 180deg   slide 80px      | fill none=grey (own style), forwards=green, backwards=grey (own
style), both=green
events on .fm.both (restarted): animationstart after ~500ms, animationend after ~1000ms
```

- **Bounce at 250ms**: the 0 % → 50 % segment lasts 500ms with `ease-in`; ease-in(0.5) = 0.315; −12 + 24 × 0.315 = **−4.43px** ✓. At 750ms the 50 % → 100 % segment uses `ease-out`: 12 − 24 × 0.685 = **−4.43px** ✓ — per-keyframe easing.
- **Slide** (`ease-in-out`, alternate): ease-in-out(0.25) = 0.129 × 160 = **20.67px** ✓; at 1500ms the second iteration runs backwards, half-way → **80px** ✓.
- **fill-mode** — the most misunderstood property (e.g. a fade-in that's visible during its delay needs `backwards` or `both`):

Mode	During the delay (0–500ms)	After the end (≥ 1000ms)
`none`	element's own style (grey)	own style (grey)
`forwards`	own style	**last keyframe** (green)
`backwards`	**first keyframe** (red)	own style
`both`	first keyframe	last keyframe

- At 750ms all four show **#796538**, the halfway mix of #dc2626 and #16a34a in sRGB (legacy `rgb`/hex colours interpolate in sRGB): (220 + 22)/2 = 121 = 0x79, (38 + 163)/2 ≈ 101 = 0x65, (38 + 74)/2 = 56 = 0x38.

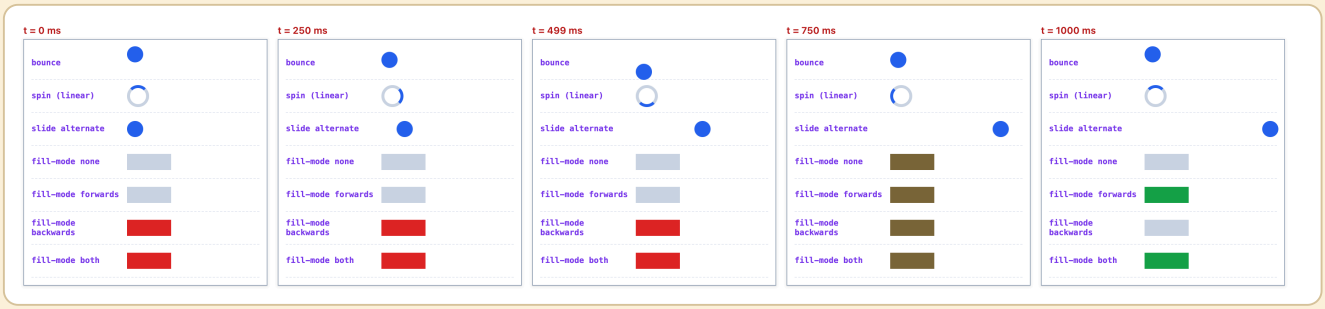


Figure 12.5 — Filmstrip of `code/05_keyframes.html`.

Events

`animationstart` (after the delay), `animationiteration` (each loop), `animationend`, `animationcancel` — the output's last line shows start at ~500ms and end at ~1000ms.

Transition or animation?

Use a **transition** when…	Use a **keyframe animation** when…
Animating between two states triggered by a change (hover, class, open/closed)	It must run by itself (on load, looping, attention-seeking)
Only a start and an end	You need intermediate steps
It should reverse naturally when the state flips back	You need iteration, direction, delay choreography

12.6 The Web Animations API (WAAPI)

What it is

The **Web Animations API** is JavaScript's interface to the same engine that runs CSS transitions and animations (Baseline widely available since 2020–2023). `element.animate(keyframes, options)` creates one; every animation, CSS or JS, is an `Animation` object you can pause, seek, speed up, reverse and `await`.

```
const anim = box.animate(
  [{ translate: '0 0', opacity: 1 }, { translate: '200px 0', opacity: 0.5 }],   // keyframes
  { duration: 400, easing: 'ease-out', fill: 'forwards' });                     // options
anim.pause(); anim.currentTime = 200;        // scrub
anim.playbackRate = 2; anim.play();          // double speed
await anim.finished;                         // a Promise
anim.reverse();
document.getAnimations();                    // every running animation, CSS ones included
```

Output of `code/11_waapi.mjs` (page: `code/11_waapi.html`):

```
created: playState=running, listed by document.getAnimations(): true
paused at 200ms: translate=136.929px, opacity=0.658
finished: currentTime=400ms (the remaining 200ms took ~100ms of real time at 2x), playState=finished
fill: forwards keeps the end: translate=200px
after reverse(): translate=none  (back at the start; fill: forwards doesn't hold the START value)
```

ease-out(0.5) = 0.685 → 200 × 0.685 = 136.9px; opacity 1 − 0.5 × 0.685 = 0.658 ✓.

When to use it

- Values known only at runtime (animating from a clicked element's position — FLIP, 12.7).
- Sequencing with promises (`await a.finished; b.play()`), scrubbing with a slider, syncing with media.
- Testing and screenshots — how this chapter's figures were made.

Prefer CSS for simple **declarative** states; prefer WAAPI over hand-written `requestAnimationFrame` loops, because WAAPI `transform`/`opacity` animations can run on the compositor (12.7) and are cancellable and inspectable in DevTools' Animations panel. `anim.commitStyles()` writes the current values into the `style` attribute — useful before `cancel()` to keep the end state without `fill: forwards` (fills keep an animation alive forever).

12.7 Performance: animate `transform` and `opacity`

How the browser renders a frame

Each frame re-runs the rendering pipeline — **style → layout → paint → composite** (§1.7) — from the first stage the changed property affects. Style, layout and paint run on the **main thread** (which also runs your JavaScript); compositing can run on a separate **compositor thread**, often on the GPU.

Property changed	Needs	Examples	Cost
Geometry	style → **layout** → paint → composite	`width`, `height`, `top`/`left`, `margin`, `padding`, `font-size`	Highest: the element *and possibly everything around it* is re-laid out
Appearance	style → **paint** → composite	`color`, `background`, `box-shadow`, `border-radius`, `outline`	Medium: pixels re-drawn
Compositor-only	(style) → **composite**	`transform` (and `translate`/`rotate`/`scale`), `opacity`, often `filter`	Lowest: the GPU moves/fades an already-painted layer, even while the main thread is busy

At 60 fps a frame has about **16.7ms**; on a 120 Hz screen, 8.3ms. Layout of a big page can take several ms alone, so layout animations drop frames (jank).

Measured

`code/06_performance.mjs` runs four 1-second animations of the same box in `code/06_performance.html` and reads Chrome's counters (`page.metrics()`; a "style recalc" is one run of the style stage):

```
move-left       layouts  60   style recalcs  61   layout time 1.8ms
grow-width      layouts  60   style recalcs  61   layout time 1.8ms
move-transform  layouts   1   style recalcs   5   layout time 0.0ms
fade            layouts   1   style recalcs   5   layout time 0.0ms
```

`left` / `width` ran layout on **every frame** (~60 in one second); `transform` and `opacity` needed **one** in total — the compositor animated them off the main thread. (Counts vary ±1 per run; layout time is tiny only because the page is tiny — on a real page each of those 60 layouts could cost milliseconds.)

Techniques

- **Move with `translate`, not `top` / `left`; resize with `scale`, not `width` / `height`; fade with `opacity`.**
- **Expensive paints**: animate the `opacity` of a pseudo-element that already has the big shadow, rather than `box-shadow`.
- **FLIP** (First, Last, Invert, Play) for layout changes (e.g. a card expanding from a grid into a detail view): measure the **F**irst position, apply the layout change, measure the **L**ast, **I**nvert with a transform so it *looks* unmoved, then **P**lay a transform animation back to none. Layout happens once; the motion is a cheap transform. View transitions (12.10) automate this.
- `will-change: transform` hints the browser to promote the element to its own **layer** (a separately painted image the compositor can move without repainting) ahead of time. Sparingly and temporarily: every layer costs GPU memory, and `will-change` creates a stacking context.
- `contain: layout paint` / `content-visibility` limit how far layout and paint spread (Chapter 14).
- **Measure**: DevTools Performance panel (purple Layout and green Paint blocks per frame), Rendering → "Paint flashing" and "Layer borders", and the Animations panel to slow animations down.

In the interview

30-second answer: "Each frame the browser does style, layout, paint and composite. Animating layout properties like `left` or `width` re-runs layout and paint every frame — I measured about 60 layouts per second for a `left` animation. `transform` and `opacity` can be handled by the compositor on the GPU, so I animate those — translate instead of top/left, scale instead of width — and use FLIP when a layout change must look animated. `will-change` sparingly, and I check the Performance panel for long frames."

- *Is `filter` compositor-only?* → Often in Chromium, but a blur can be expensive; test.
- *Why can a transform animation keep running while JS is busy?* → It runs on the compositor thread.
- *Downside of `will-change` everywhere?* → Too many layers → memory pressure and slower pages; also stacking contexts.

12.8 Enter and exit animations: `@starting-style` and `allow-discrete`

The problem it solves

Dialogs, popovers (floating panels opened with the `popover` attribute), toasts (brief self-dismissing notices) and menus usually go from `display: none` to visible, which breaks transitions twice:

1. **Entering**: an element that wasn't rendered has no "before" style — it just appears.
2. **Exiting**: `display` flips to `none` immediately, so the fade-out is never seen.

The tools

- `@starting-style { … }` defines the style to transition **from** when an element is first rendered (or leaves `display: none`). Baseline since August 2024.
- `transition-behavior: allow-discrete` (or the keyword in the shorthand) lets discrete properties join a transition; for `display`, the element stays displayed for the whole exit and flips to `none` at the end. `transition-behavior` is Baseline since August 2024, but transitioning `display` works in Chrome and Safari, not yet Firefox, and the `overlay` property (keeps a closing popover/dialog in the **top layer**, drawn above everything) is Chromium-only. Where unsupported the element simply closes without the fade — safe progressive enhancement.

```
#a { opacity: 0; translate: 0 -16px;                          /* closed look = exit end state */
     transition: opacity 300ms ease-out, translate 300ms ease-out,
                 display 300ms allow-discrete, overlay 300ms allow-discrete; }
#a:popover-open { opacity: 1; translate: 0 0; }               /* open look */
@starting-style { #a:popover-open { opacity: 0; translate: 0 -16px; } }   /* enter starts here */
```

`code/07_starting_style.html` has popover **A** (above) and **B** (same transition without `@starting-style` or `allow-discrete`). `code/07_starting_style.mjs` opens and closes both, freezing at 100ms:

```
opening, 100ms: A: display block opacity 0.49  transitions running: opacity+translate   |   B: display block opacity 1.00
transitions running: none
opened        : A: display block opacity 1.00  transitions running: none   |   B: display block opacity 1.00  transitions
running: none
closing, 100ms: A: display block opacity 0.51  transitions running: display+opacity+overlay+translate   |   B: display
none  opacity 0.00  transitions running: none
closed        : A: display none  opacity 0.00  transitions running: none   |   B: display none  opacity 0.00  transitions
running: none
```

B never animates (no starting style on open; `display: none` immediately on close). A fades in from the `@starting-style` values and, while closing, stays `display: block` with a running `display` transition until the fade ends.

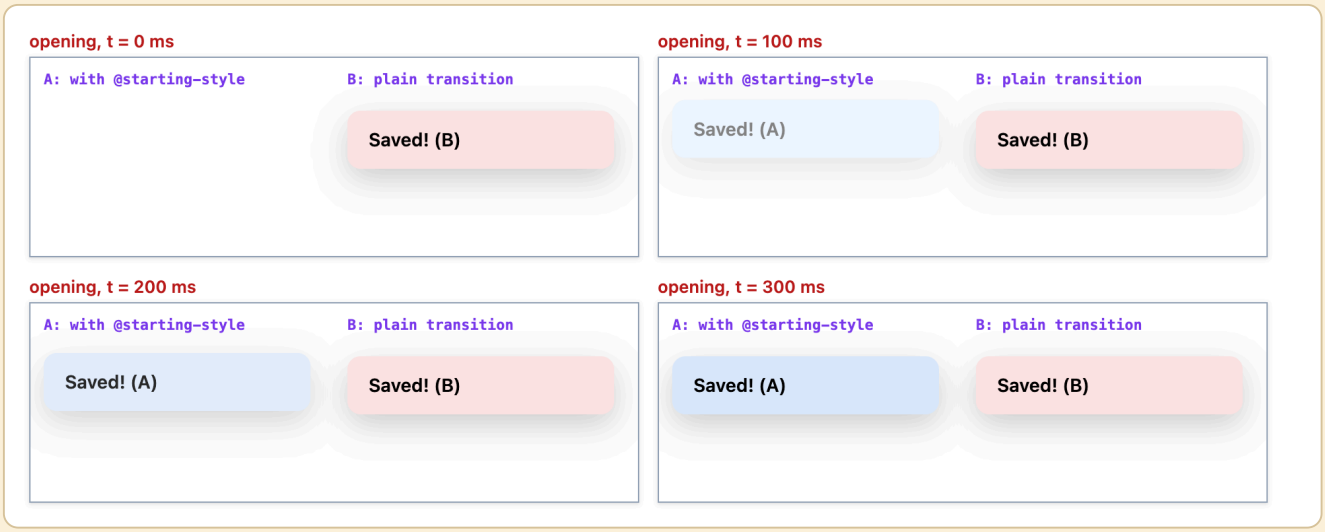


Figure 12.6 — A (blue) animates in; B (red) just appears.

Where you'll use it

`<dialog>` (`[open]` and its `::backdrop`, transitioned the same way), `[popover]` (`:popover-open`), JS-added elements (toasts, list items), `<details>` content, and anything toggled with `display` or `hidden`.

12.9 Reduced motion and accessibility

Why it matters

For people with **vestibular disorders** (inner-ear balance problems), large on-screen movement — parallax, zooming, spinning, sliding panels — can trigger dizziness, nausea and headaches; others find motion distracting (ADHD, cognitive disabilities). Every major OS has a setting (macOS/iOS "Reduce motion", Windows "Animation effects" off, Android "Remove animations"), exposed as `prefers-reduced-motion: reduce` (Baseline widely available).

The WCAG rules

(Levels: A minimum, AA the usual legal target, AAA strictest — §3.9.)

Criterion	Level	Requirement
2.2.2 Pause, Stop, Hide	A	Auto-starting moving, blinking or scrolling content lasting **more than 5 seconds** alongside other content must be pausable/stoppable (carousels, marquees, animated backgrounds, a 30s looping hero video)
2.3.1 Three Flashes or Below Threshold	A	Nothing flashes more than **3 times per second** (seizure risk)
2.3.3 Animation from Interactions	AAA	Interaction-triggered motion can be disabled unless essential

Two patterns

`code/08_reduced_motion.html`:

```
/* 1. Motion only when allowed: big movement is ADDED for users with no preference */
@keyframes slide-in { from { translate: -60px 0; opacity: 0; } }
@media (prefers-reduced-motion: no-preference) {
  .card { animation: slide-in 600ms ease-out both; }
  html  { scroll-behavior: smooth; }
}

/* 2. Replace, don't remove: a loading spinner must still say "busy" */
@keyframes pulse { 50% { opacity: .35; } }
.spinner { animation: spin 800ms linear infinite; }
@media (prefers-reduced-motion: reduce) { .spinner { animation: pulse 2s ease-in-out infinite; } }
```

`code/08_reduced_motion.mjs` emulates both settings:

```
prefers-reduced-motion: no-preference -> card animation: slide-in spinner: spin 0.8s   scroll-behavior: smooth   running animations: 2
prefers-reduced-motion: reduce        -> card animation: none     spinner: pulse 2s   scroll-behavior: auto   running animations: 1
```

Reduced motion ≠ no motion. Fades, colour changes and small scale changes are usually fine; large translation, zoom, rotation, **parallax** (layers moving at different scroll speeds) and **scroll-jacking** (overriding normal scroll speed or direction) cause harm. The blunt global reset in §10.5 (durations → `0.01ms`, not `0`, so `animationend` / `transitionend` still fire for code that waits on them) is a reasonable safety net, but designing each animation's reduced version is better. In JS, check `matchMedia('(prefers-reduced-motion: reduce)').matches` before WAAPI animations, smooth `scrollIntoView`, or autoplaying video.

Other rules of thumb

- Autoplaying carousels need a visible pause button (2.2.2) — and usually shouldn't autoplay.
- Don't put essential information only in motion.
- Keep focus behaviour intact: don't move focus into an element that's still invisible.

12.10 View transitions

What it is

A **view transition** animates between two **states of the page** — before and after a DOM update, or (cross-document) between pages. The browser **snapshots** the old state, lets you update the DOM, snapshots the new state, and animates — cross-fading the page and **morphing** each element with a `view-transition-name` (captured separately, it moves from its old size/position to the new one). It's FLIP (12.7) done for you.

Support (verified September 2026)

Feature	Status
Same-document: `document.startViewTransition()`, `view-transition-name`, the `::view-transition-*` pseudo-elements	**Baseline newly available since October 2025** (Chrome 111, Safari 18, Firefox 144)
`view-transition-class` (style many groups with one selector)	Baseline since October 2025
View transition **types** (`startViewTransition({ update, types: ['forward'] })`, `:active-view-transition-type()`)	Baseline since January 2026
Cross-document (MPA — multi-page application) view transitions: `@view-transition { navigation: auto; }`	**Not Baseline**: Chrome 126+, Safari 18.2+, not Firefox yet

How it works (same-document)

1. You call `document.startViewTransition(updateCallback)`.
2. The browser captures the **old** state: the page (`root`) and each element with a (unique) `view-transition-name`.
3. It runs your callback (may be async) to change the DOM, then captures the **new** state.
4. It builds a pseudo-element tree over the page:

```
::view-transition
└─ ::view-transition-group(item-2)        ← animates size and position old → new
   └─ ::view-transition-image-pair(item-2)
      ├─ ::view-transition-old(item-2)     ← old snapshot, fades out
      └─ ::view-transition-new(item-2)     ← new live view, fades in
```

5. It animates the tree with ordinary CSS animations (default 250ms `ease`), then removes it; the page underneath already shows the new state. Style `::view-transition-group(name)` (or `(*.class)`) to change the morph.

```
const update = () => items.classList.toggle('list');
toggle.addEventListener('click', () => {
  if (!document.startViewTransition) return update();     // no support: just update
  document.startViewTransition(update);
});
```

```
.item:nth-child(1) { view-transition-name: item-1; }       /* one unique name per morphing element */
.item { view-transition-class: item; }
::view-transition-group(*.item) { animation-duration: 500ms; animation-timing-function: ease-in-out; }
::view-transition-old(*.item), ::view-transition-new(*.item) { height: 100%; }   /* see below */
@media (prefers-reduced-motion: reduce) {
  ::view-transition-group(*), ::view-transition-old(*), ::view-transition-new(*) { animation: none !important; }
}
```

Figure 12.7 — `code/09_view_transition.html` going grid → list, frozen at five moments via `document.getAnimations()`.

`code/09_view_transition.mjs` waits for `transition.ready` (pseudo-elements exist), pauses everything and inspects it (abridged — item-2 and item-3 repeat item-1's animations):

```
20 animations while the transition runs:
  ::view-transition-group(item-1) -ua-view-transition-group-anim-item-1 500ms
  ::view-transition-group(root) -ua-view-transition-group-anim-root 250ms
  ::view-transition-new(item-1) -ua-mix-blend-mode-plus-lighter 500ms
  ::view-transition-new(item-1) -ua-view-transition-fade-in 500ms
  ::view-transition-old(item-1) -ua-view-transition-fade-out 500ms
  … (plus mix-blend-mode-plus-lighter on old(item-1), the root old/new pairs at 250ms, and item-2/item-3)
t=  0ms ::view-transition-group(item-2): width 100.0px, height 70.0px, transform matrix(1, 0, 0, 1, 120, 54)
t=250ms ::view-transition-group(item-2): width 215.0px, height 53.0px, transform matrix(1, 0, 0, 1, 65, 75)
t=500ms ::view-transition-group(item-2): width 330.0px, height 36.0px, transform matrix(1, 0, 0, 1, 10, 96)
```

`-ua-…` are Chrome's built-in keyframes. "Two" morphs from 100×70 at (120, 54) to a 330×36 row at (10, 96); at half-way (`ease-in-out`(0.5) = 0.5) every value is the average — 215 = (100 + 330)/2 ✓. `root` keeps 250ms because the rule only targeted `*.item`.

Aspect-ratio tip: old and new snapshots keep their own aspect ratios at the group's width, so a tall box becoming a short row looks smeared mid-way; `height: 100%` on `::view-transition-old/new` stretches both to the group (used in Figure 12.7).

Cross-document view transitions

Both pages opt in with `@view-transition { navigation: auto; }` and share `view-transition-name`s; a **same-origin** navigation then animates with no JavaScript. Chrome and Safari only in 2026 — others just navigate, so it's safe to add.

Pitfalls

- Duplicate `view-transition-name`s abort the transition.
- During a transition the overlay blocks clicks — keep it short.
- Snapshots are images: text stretches during a morph; cross-fade handles most cases.
- Keep the no-support path (`if (!document.startViewTransition) return update();`).

12.11 Scroll-driven animations

What it is

A **scroll-driven animation** replaces time with a **scroll timeline**: progress follows how far a container has scrolled (`scroll()`) or how far an element has travelled through the viewport (`view()`). Reading-progress bars, reveal-on-scroll and parallax without JS scroll listeners — and potentially off the main thread.

Support: not Baseline. Chrome 115+ and Safari 26+; Firefox only behind a flag (September 2026). Always wrap in `@supports (animation-timeline: scroll())` (a feature query) and make the page work without it.

```css
/* Reading progress bar */
@keyframes grow { from { scale: 0 1; } to { scale: 1 1; } }
.progress { transform-origin: 0 50%; animation: grow linear both; animation-timeline: scroll(root block); }

/* Reveal an element as it enters the viewport */
@keyframes fade-up { from { opacity: 0; translate: 0 40px; } to { opacity: 1; translate: 0 0; } }
.reveal { animation: fade-up linear both; animation-timeline: view(); animation-range: entry 0% entry 100%; }
```

Piece	Meaning
`scroll(<scroller> <axis>)`	Anonymous scroll timeline: `nearest` (default), `root` or `self`; axis `block` (default), `inline`, `x`, `y`
`view(<axis> <inset>)`	Timeline of the element's own visibility in its scrollport
`animation-range`	Which part of the timeline maps to 0–100 %: `entry`, `exit`, `contain`, `cover` + percentages
`scroll-timeline-name` / `view-timeline-name`	Named timelines, usable by other elements (with `timeline-scope`)
Duration	Ignored/`auto` — scroll distance replaces time; use `linear` easing so progress tracks scroll 1:1

`code/10_scroll_driven.mjs` scrolls `code/10_scroll_driven.html` (400px-tall viewport) and reads the progress bar and the second reveal box:

```
CSS.supports("animation-timeline: scroll()") = true
scrollY   0px (  0% of 540px): bar scale 0 1       #r2 top  562px (height 48) opacity 0.00 translate 0px 40px
scrollY 135px ( 25% of 540px): bar scale 0.25 1    #r2 top  427px (height 48) opacity 0.00 translate 0px 40px
scrollY 186px ( 34% of 540px): bar scale 0.344444 1 #r2 top  376px (height 48) opacity 0.50 translate 0px 20.1042px
scrollY 270px ( 50% of 540px): bar scale 0.5 1     #r2 top  292px (height 48) opacity 1.00 translate 0px
scrollY 540px (100% of 540px): bar scale 1         #r2 top   22px (height 48) opacity 1.00 translate 0px
```

The bar's scale equals the scroll fraction (186/540 = 0.344). The box's `entry` range runs from "top touches the viewport bottom" (top = 400px) to "fully inside" (400 − 48 = 352px); at 376px it's half-way → opacity ≈ 0.50, translate ≈ 20px ✓.

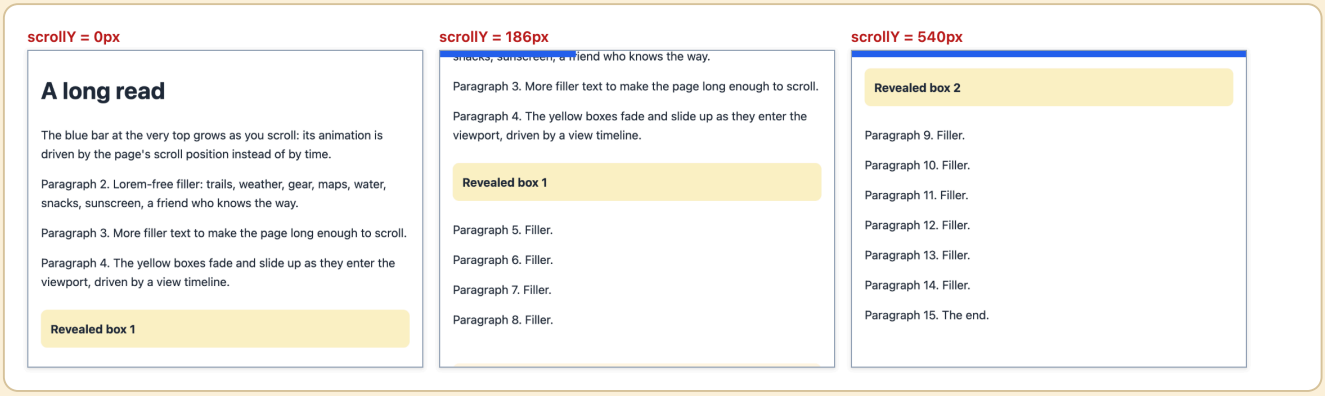


Figure 12.8 — The progress bar at three scroll positions.

Cautions

- Respect reduced motion: big parallax or slide-ins inside `@media (prefers-reduced-motion: no-preference)` (the demo disables the reveal under `reduce`).
- Don't hide content until it scrolls in if it might never animate (unsupported browser, print, find-in-page); the demo's `@supports` wrapper keeps content visible by default.
- Scroll-driven animations should *follow* scrolling, never replace it (no scroll-jacking).

Chapter 12 TL;DR

- **Transitions** animate a *change*: `transition: prop duration easing delay`, declared on the base rule; interpolable values only (not `auto`); `transitionend` per property; reversal is shortened.
- **Timing functions**: `ease` (default), `ease-in` (leaving), `ease-out` (entering), `ease-in-out`, `cubic-bezier()` (x ∈ 0–1, y may overshoot), `steps(n)` (jump-end by default), `linear(points)` (Baseline 2023). Our bisection solver matched Chrome to 3 decimals.
- **2D transforms** move the *painted* box, not the layout box (layout stayed 50,30 90×50 for all eight). Order matters; `transform-origin`; `%` translate = own size; individual `translate`/`rotate`/`scale` (Baseline 2022) apply in fixed order and animate independently. Transforms create stacking contexts and containing blocks.
- **3D**: `perspective` on the parent, `preserve-3d`, `backface-visibility: hidden` → flip cards; `overflow`, `opacity < 1`, `filter` flatten 3D.
- **Keyframes**: `@keyframes` + `animation: name duration easing delay count direction fill-mode`; easing per segment; **fill-mode** = what shows during the delay (backwards) and after the end (forwards); negative delays start mid-way.
- **WAAPI**: `el.animate()`, `pause()`, `currentTime`, `playbackRate`, `reverse()`, `await anim.finished`, `document.getAnimations()` (includes CSS animations).
- **Performance**: animate `transform` and `opacity` (≈1 layout per animation), not `left`/`width` (≈60 layouts/second, measured). FLIP for layout changes; `will-change` sparingly.
- **Enter/exit**: `@starting-style` (Baseline 2024) for the "from" style; `transition-behavior: allow-discrete` + `display`/`overlay` keep an element visible while animating out (display: Chrome/Safari; overlay: Chromium).
- **Reduced motion**: `@media (prefers-reduced-motion: no-preference)` to opt motion in; replace rather than remove; WCAG 2.2.2 (pause > 5s auto motion), 2.3.1 (≤ 3 flashes/s), 2.3.3 (AAA).

- **View transitions**: `document.startViewTransition(update)` + `view-transition-name` → snapshot, update, morph via `::view-transition-group/old/new`; same-document Baseline since Oct 2025; cross-document Chrome/Safari only.
- **Scroll-driven**: `animation-timeline: scroll()` / `view()` + `animation-range`; **not Baseline** (no Firefox) → `@supports`, content visible without it.

Next: Chapter 13 — Modern CSS and Architecture.

Chapter 13 — Modern CSS and Architecture

Until about 2022, CSS for a big site meant tools: Sass (a language compiled into plain CSS) for nesting and variables, strict naming conventions so styles didn't collide, and JavaScript for things like "style the parent when a child is checked". Browsers have caught up: nesting, `:has()`, cascade layers, `@scope`, logical properties and `light-dark()` are built in, and they change how you organise stylesheets. This chapter covers each feature, then the question every team faces — **how do you structure CSS so it stays maintainable as a project grows?** — comparing BEM, utility-first (Tailwind CSS), CSS Modules and CSS-in-JS, plus Sass, resets and a complete theming system.

In this chapter: 13.1 Native nesting · 13.2 `:is()` and `:where()` · 13.3 `:has()` · 13.4 Cascade layers · 13.5 `@scope` · 13.6 Logical properties · 13.7 Why CSS needs an architecture; BEM · 13.8 Utility-first and Tailwind CSS v4 · 13.9 CSS Modules · 13.10 CSS-in-JS · 13.11 Choosing an approach · 13.12 Sass and PostCSS · 13.13 Resets and normalize · 13.14 Theming and dark mode.

How the numbers and figures were made. Every example is a page in `code/`. Each `NN_*.mjs` script opens its page in headless Chrome 154 via `code/measure.mjs` (Chapter 10's helper) and prints what the browser computed — `getComputedStyle()`, bounding boxes and the CSSOM (the browser's object model of the stylesheet). The Tailwind example is a real Tailwind CSS v4.3 build; the CSS Modules example is compiled by Lightning CSS (13.12), the compiler many **bundlers** (build tools that combine a project's source files into a few optimised ones) use. `code/make_figures.mjs` takes the screenshots. Support data is from the **web-features** dataset (behind MDN's and caniuse's "Baseline" badges — §1.1: *newly available* = in all core browsers, *widely available* = 30 months later), checked September 2026. For older devices, check your analytics or provide a fallback.

13.1 Native CSS nesting

What it is

Nesting writes a rule inside another, so the inner selector is relative to the outer. Sass has had it since 2006; browsers do it natively: **Baseline newly available since December 2023** (Chrome 120, Firefox 117, Safari 17.2), **widely available since June 2026**.

Why it matters

Without nesting you repeat the parent selector for every child, hover state and media query, and related rules drift apart. Nesting keeps a component in one block — with no **build step** (a program transforming sources before serving), and DevTools shows the rule as written.

How it works

1. Inside a style rule you can write declarations **and** further rules.
2. A nested selector with no `&` is a **descendant** of the parent: `.title {}` inside `.card {}` = `.card .title` (write `> .title` for a child).
3. `&` (the **nesting selector**) stands for the parent's matched elements; its position sets the relationship: `&:hover` (same element), `& > p` (child), `.dark-page &` (parent *inside* `.dark-page`), `& + &` (an item right after another).
4. A nested selector may start with a **combinator** (space, `>`, `+`, `~`): `> p` means `& > p`.
5. Conditional at-rules (`@media`, `@supports`, `@container`, `@layer`, `@scope`) nest too; their declarations apply to the parent selector.
6. `&` behaves like `:is(<parent selector list>)` — which matters for specificity (§6.4).

`code/01_nesting.html`:

```
.card {
  padding: 16px;
  border: 2px solid #cbd5e1;

  .title { font-size: 20px; margin: 0; }          /* = .card .title (descendant)        */
  > p { margin: 4px 0 0; color: #334155; }         /* = .card > p (relative selector)    */
  &:hover { border-color: #2563eb; }               /* = .card:hover (no space)           */
  &.featured { border-color: #f59e0b; }            /* = .card.featured (same element)    */
  .dark-page & { background: #0f172a; }            /* = .dark-page .card (& at the end)  */
  @media (width >= 600px) { padding: 24px; }       /* nested at-rule: applies to .card   */
  &__body { color: red; }                          /* NOT Sass: no string joining        */
}

/* & behaves like :is() -> the parent list's HIGHEST specificity is used */
#main, .sidebar {
  .link { color: crimson; }                         /* = :is(#main, .sidebar) .link  (1,1,0) */
}
.sidebar .link.plain { color: green; }              /* (0,3,0) -- still loses!              */
```

`code/01_nesting.mjs` walks the CSSOM and reads computed styles at two viewport widths:

```
CSSOM tree (selectorText of each rule):
body
.card
  & .title
  & > p
  &:hover
  &.featured
  .dark-page &
  @media (width >= 600px)
    CSSNestedDeclarations
#main, .sidebar
  & .link
.sidebar .link.plain

Was the "&__body" rule kept? false

viewport 500px: .card padding 16px, border rgb(245, 158, 11), background rgb(15, 23, 42)
  .title font-size 20px   .card__body color rgb(0, 0, 0) (not red)   .sidebar .link color rgb(220, 20, 60)

viewport 700px: .card padding 24px, border rgb(245, 158, 11), background rgb(15, 23, 42)
  .title font-size 20px   .card__body color rgb(0, 0, 0) (not red)   .sidebar .link color rgb(220, 20, 60)
```

- The browser stores `.title` as `& .title` and `> p` as `& > p` — an implicit `&` and descendant space.
- The nested `@media` holds a `CSSNestedDeclarations` object (bare `padding: 24px` treated as `& { … }`): 16px at 500px, 24px at 700px.
- **`&__body` was thrown away.** Sass glues text into `.card__body`; native CSS works on selectors, not strings, so Chrome discards the rule. The #1 surprise when moving BEM-style (13.7) Sass to native nesting (same for `&-large`).
- The sidebar link is **crimson** although `.sidebar .link.plain` (0,3,0) looks stronger: the nested rule is `:is(#main, .sidebar) .link`, and `:is()` takes its most specific argument — `#main` — giving (1,1,0) even though the element matched via `.sidebar`.

Compiling nesting away for old browsers

To support pre-late-2023 browsers, a build tool can **lower** (flatten) nesting. `code/01_nesting_lower.mjs` asks Lightning CSS to target Chrome 100:

```
.card {
  padding: 16px;
}

.card .title {
  font-size: 20px;
}
/* ... .card:hover, .dark-page .card ... */

@media (min-width: 600px) {
  .card {
    padding: 24px;
  }
}

:is(#main, .sidebar) .link {
  color: #dc143c;
}
```

It writes `:is(#main, .sidebar) .link` — the browser's meaning (Sass writes `#main .link, .sidebar .link`, two selectors with *different* specificities) — and rewrote the range media query `(width >= 600px)` to `min-width`.

Common mistakes

- Porting Sass `&__title` or `&-large`.
- **Nesting too deep** — `.page .main .card .body .title a` nested is as fragile as flat. Keep to 2–3 levels: states, owned children, media/container queries.
- Forgetting a nested rule inherits the `:is()` specificity of the *whole* parent list.

In the interview

30-second answer: "CSS supports nesting natively — Baseline since late 2023. A nested selector is relative to its parent; `&` represents the parent and behaves like `:is()` of the parent list, so it takes the highest specificity in that list. Unlike Sass it doesn't concatenate strings, so `&__title` doesn't work. I use it for states, owned children and nested media queries, kept shallow; a build tool is needed only to lower it for pre-2023 browsers."

13.2 `:is()` and `:where()`

What it is

Both take a **selector list** and match an element matching any entry. They differ only in **specificity**: `:is()` counts as its **most specific argument** (`:is(p, .x, #y)` = (1,0,0) whichever matched); `:where()` always counts **zero** (`:where(#a) .b` = (0,1,0)). Baseline widely available since 2021. Chapter 6 covers the scoring; here they're architecture tools.

Why it matters

1. **Shorter selectors**: `:is(article, section, aside) :is(h1, h2, h3)` replaces nine.
2. **Forgiving parsing**: one invalid selector in a normal list drops the **whole rule**; inside `:is()`/`:where()` bad entries are just ignored — safe for new pseudo-classes.
3. **Specificity control**: `:where()` for defaults anything can override (resets, libraries); `:is()` to raise specificity.

Proof

`code/02_is_where.html`:

```
:is(article, section, aside) :is(h1, h2, h3) { color: rebeccapurple; }

:is(.promo, :unknown-thing) { font-weight: 700; }           /* forgiving   */
.promo, :unknown-thing { text-decoration: underline; }      /* NOT forgiving */

:where(.content) a  { color: gray; }        /* (0,0,1)                      */
a                   { color: teal; }        /* (0,0,1), later -> wins       */
:is(#nav, .content) b { color: gray; }      /* (1,0,1)                      */
.content b.hot      { color: teal; }        /* (0,2,1) -> loses             */
```

`code/02_is_where.mjs` reads computed styles and scores selectors with Chapter 6's calculator:

```
h2 inside article color      rgb(102, 51, 153) (rebeccapurple = rgb(102, 51, 153))
.promo font-weight           700   <- forgiving :is() kept the rule
.promo text-decoration-line  none   <- plain list with one bad selector was dropped
:where(.content) a vs a      rgb(0, 128, 128) (teal = rgb(0, 128, 128))
:is(#nav,.content) b vs .content b.hot rgb(128, 128, 128) (gray = rgb(128, 128, 128))

Rules the browser kept: body  |  :is(article, section, aside) :is(h1, h2, h3)  |  :is(.promo)  |  :where(.content) a  |  a
|  :is(#nav, .content) b  |  .content b.hot

Specificity (A,B,C):
  :is(article, section, aside) :is(h1, h2, h3)   (0,0,2)
  :where(.content) a                             (0,0,1)
  a                                              (0,0,1)
  :is(#nav, .content) b                          (1,0,1)
  .content b.hot                                 (0,2,1)
```

Chrome kept `:is(.promo)` (silently dropping `:unknown-thing`) while `.promo, :unknown-thing` is **missing**; and `:is(#nav, .content) b` wins with an ID's weight though no `#nav` exists.

Where each is used

Use	Pick	Example
Reset / library defaults that must be easy to override	`:where()`	`:where(ul, ol)[role="list"] { list-style: none }`
Shortening a long list without changing priorities much	`:is()`	`:is(h1,h2,h3):hover`
Using a new pseudo-class safely next to old ones	either (forgiving)	`:is(:focus-visible, .focus)`
Deliberately raising a selector's weight	`:is()` with an ID or class	`.btn:is(#_, .btn)` (a hack — prefer layers, 13.4)

Common mistakes

- `:is()` in a library's defaults: an `#app` argument makes everything inside hard to override. Use `:where()`.
- Pseudo-elements inside: `:is(::before)` never matches — not allowed in these lists.

13.3 `:has()` — the relational ("parent") selector

What it is

`A:has(B)` matches **A** if **B**, evaluated *relative to A*, matches something. With a descendant (`:has(img)`) it's the long-awaited **parent selector**; with `+`/`~` (`h3:has(+ .subtitle)`, `h2:has(+ p)`) a **previous-sibling** selector. Baseline newly available since **December 2023** (Firefox 121 last), widely available since June 2026. Basic syntax: §6.3.

The problem it solves

Selectors used to look only **down and forward**. "Grid layout if the card has an image", "highlight the label whose radio is checked", "lock scrolling while a dialog is open" needed JS adding classes; `:has()` does them in CSS, always in sync.

How it works

1. The argument is a **relative selector list**; each item implicitly starts at the `:has()` element (default combinator: descendant).
2. Specificity = most specific argument, like `:is()` (`.card:has(> img, #hero)` = (1,1,0)).
3. Re-evaluated whenever the DOM or state changes (checking a box, typing, opening a dialog).
4. Restrictions: no `:has()` inside `:has()`; no pseudo-element arguments.

`code/03_has.html` shows six patterns:

```
.card:has(img)                { display: grid; grid-template-columns: 64px 1fr; }  /* parent       */
.field:has(input:invalid)     { border-left-color: #dc2626; background: #fef2f2; } /* form state   */
h3:has(+ .subtitle)           { margin-bottom: 0; color: #7c3aed; }                /* prev sibling */
ul:has(> li:nth-child(4))     { columns: 2; }                                      /* quantity     */
label.opt:has(input:checked)  { background: #dbeafe; font-weight: 600; }           /* checked      */
html:has(dialog[open])        { overflow: hidden; }                                /* page-level   */
```

Figure 13.1 — Every box's styling comes from a `:has()` rule; no JavaScript adds classes.

`code/03_has.mjs` reads styles, changes state like a user (click "Yearly", fix the email, add a 4th item, open the dialog), and reads again:

```
.card with img display=grid, without img display=block
invalid field border=rgb(220, 38, 38), valid field border=rgb(203, 213, 225)
h3 followed by .subtitle margin-bottom=0px, lone h3 margin-bottom=8px
3-item list columns=auto, 5-item list columns=2
Monthly (checked) weight=600, Yearly weight=400
<html> overflow with dialog closed: visible

after: click "Yearly", fix the email, add a 4th <li>, showModal()
Monthly weight=400, Yearly weight=600
first field border=rgb(203, 213, 225)
first list columns=2
<html> overflow with dialog open: hidden

a:has(:has(b))  -> SyntaxError
p:has(::before)  -> SyntaxError
li:has(+ li)  -> valid
```

Every style followed the state change instantly; the two forbidden forms are rejected by the selector parser. (Exactly 3 items: `ul:has(> li:nth-child(3)):not(:has(> li:nth-child(4)))`.)

Trade-offs and performance

`:has()` makes the browser look "up" the tree on changes — more work, but engines optimise common cases. Keep it cheap: **anchor it** (`.card:has(img)`, not bare `:has(img)` or `body:has(.x .y .z)`), and avoid `*:has(...)` on huge,

constantly changing DOMs.

When not to use it

- For *behaviour* (disabling a button, moving focus) — JavaScript's job; `:has()` only changes appearance.
- Error styles at page load: `:invalid` matches before typing; use `.field:has(:user-invalid)` (Chapter 5).

Real-world examples

Selected table rows (`tr:has(input:checked)`), error-highlighting form groups, quantity queries (styles by child count), modal scroll locking, dimming siblings of a hovered item: `.list:has(.item:hover) .item:not(:hover) { opacity: .6 }`.

In the interview

> **30-second answer:** "`:has()` matches an element if the relative selector inside matches — `.card:has(img)` styles cards containing an image. It's a parent selector and, with `+`/`~`, a previous-sibling selector, and it reacts to state like `:checked` (`label:has(input:checked)`) or `:invalid`. Baseline since late 2023. I anchor it for performance and never use it for behaviour."

13.4 Cascade layers (`@layer`)

What it is

A **cascade layer** is a named bucket of rules. Declare the bucket order once, and a rule in a later layer beats one in an earlier layer **however specific either selector is** (like floors of a building). Baseline since March 2022, widely available since September 2024. §6.5 placed layers in the cascade; this section uses them to structure a project.

The problem it solves

Specificity wars: a library ships `#app .btn.btn-primary`; you write something more specific or `!important`; the next person escalates. Layers say, once, "vendor CSS is below my components, utilities above both".

How it works

1. `@layer reset, vendor, base, components, utilities;` — **first = weakest**. Put this statement first so order doesn't depend on file load order (a layer's position is fixed by its first appearance).
2. Add rules with `@layer name { … }` (blocks merge), or import a file into a layer: `@import url(x.css) layer(vendor);` (must precede all rules except `@charset` and `@layer` statements, or it's ignored).
3. The cascade compares **origin & importance → context → style attribute → layer → specificity → order** — layer **before** specificity.
4. **Unlayered** styles form an implicit final layer: they beat every named layer.
5. **`!important` reverses the order**: an earlier layer's `!important` beats a later one's, and layered `!important` beats unlayered `!important` — letting a reset lock something down.
6. `revert-layer` rolls a property back to what the earlier layers (or earlier origins) produce.
7. Layers nest: `@layer components { @layer dialog { … } }` → `components.dialog`.

`code/04_layers.html` (with `code/04_vendor.css` as a library):

```
@layer reset, vendor, base, components, utilities;
@import url("04_vendor.css") layer(vendor);     /* #app .btn.btn-primary { background: #6b7280; ... } */

@layer components { .btn { background: #2563eb; border-radius: 8px; }  .card a { color: #7c3aed; } }
@layer utilities  { .text-red { color: #dc2626; }  .no-radius { border-radius: 0 !important; } }
@layer reset      { .keep-radius { border-radius: 999px !important; } }
p { color: #065f46; }                                    /* unlayered */
@layer utilities  { .plain-link { color: revert-layer; } }
@layer components { @layer dialog { .dlg { color: #b45309; } } }
```

`code/04_layers.mjs` lists the layers from the CSSOM and reads each fight's winner:

```
statement: reset, vendor, base, components, utilities
import:    04_vendor.css layer(vendor)
block:     reset
block:     base
block:     components
block:     utilities
block:     components.dialog

#app .btn.btn-primary (vendor, 1,2,0) vs .btn (components, 0,1,0):
   background rgb(37, 99, 235) (#2563eb = rgb(37, 99, 235)), radius 8px, padding-left 20px (vendor's, nobody overrode it)
.card a (components)                    color rgb(124, 58, 237)
.text-red (utilities) vs .card a        color rgb(220, 38, 38)
.plain-link { color: revert-layer }     color rgb(124, 58, 237)  <- back to the components value
unlayered p vs base body                color rgb(6, 95, 70)
!important: utilities 0 vs reset 999px  radius 999px  <- earlier layer wins for !important
nested layer components.dialog          color rgb(180, 83, 9)
```

A one-class selector in `components` beat an ID selector in `vendor`; the vendor's padding survived because nothing higher set padding (layers settle *conflicts*, they don't delete rules); `revert-layer` fell back to the components colour; for `!important`, **reset** won.

A layer order for real projects

```
@layer reset, tokens, vendor, base, layout, components, utilities, overrides;
```

Layer	Holds	Why here
`reset`	the reset (13.13)	weakest: everything may override it
`tokens`	custom properties (`--brand`, 13.14)	declarations only, rarely conflicts
`vendor`	third-party CSS via `@import … layer(vendor)`	your base styles should beat it
`base`	element defaults (`body`, `a`, `h1`)	
`layout`	page grids, containers	
`components`	cards, buttons, dialogs	
`utilities`	single-purpose classes (`.visually-hidden`, `.text-center`)	must beat components
`overrides`	temporary fixes	explicit, easy to find and delete

Tailwind v4 emits `@layer theme, base, components, utilities;` (13.8), slotting neatly into this scheme.

Common mistakes

- **Your CSS unlayered while the framework is layered**: unlayered beats all layers — maybe intended, or maybe your stray rules become impossible to override with utilities.
- **Order by accident** — no up-front statement, so an early file creates layers in the wrong order.
- Expecting a later layer's `!important` to win — it's the reverse.

In the interview

30-second answer: "Cascade layers order groups of rules explicitly. The cascade compares layers before specificity, so a simple class in a later layer beats an ID selector in an earlier one. I declare the order once — reset, vendor, base, components, utilities — and import third-party CSS into a low layer, which is how I override a library without `!important`. Unlayered styles beat all layers, and `!important` inverts the order."

13.5 `@scope`

What it is

`@scope (<root>) to (<limit>) { … }` applies rules only to elements **inside** a scope root and, optionally, **not inside** a lower boundary — a **donut scope** (root = ring, limit = hole). Inside, `:scope` is the root element itself.

Status (September 2026): Baseline newly available since March 2026. Chromium shipped it first (Chrome 118, 2023); web-features counts it complete from Firefox 146 and Safari 26.4. Older browsers ignore the whole block, so give unscoped fallbacks for essentials.

The problem it solves

- **Leaking into content**: `.card img { border: … }` also hits images in user-generated content inside the card. BEM avoids this with long class names; `@scope … to (.content)` does it with a boundary.
- **Proximity**: with nested themes (`.light` in `.dark` in `.light`), `.dark a` and `.light a` tie on specificity, so the later one wins — wrong for the innermost box. In `@scope`, when specificity ties, the rule whose root is **nearest** (fewest steps up) wins, regardless of source order.

How it works

1. Match the scope root(s) (`.card`).
2. Match the limit(s) (`.content`) inside each root; everything at or below a limit is excluded.
3. Apply the rules in between; bare selectors like `img` act as descendants of `:scope`.
4. Specificity: the root selector adds **nothing** (unlike nesting — `@scope (#main)` adds nothing); `:scope` adds (0,1,0).
5. **Scoping proximity** is a new cascade step *after* specificity, *before* order of appearance.
6. `@scope { … }` **without a prelude** (no root/limit) inside a `<style>` scopes to that `<style>`'s parent — component-local styles in HTML.

`code/05_scope.html`:

```
@scope (.card) to (.content) {
  :scope { background: #f8fafc; }          /* the .card itself                */
  img    { border: 3px solid #2563eb; }    /* card's images, not content's    */
  p      { font-weight: 600; color: #1d4ed8; }
}
@scope (.dark)  { a { color: #fbbf24; } :scope { background: #1e293b; color: #e2e8f0; } }
@scope (.light) { a { color: #1d4ed8; } :scope { background: #fff; color: #0f172a; } }
```

```
<div id="inline">
  <style> @scope { :scope { outline: 2px dashed #16a34a; } b { color: #16a34a; } } </style>
  <b>styled by an inline, prelude-less @scope</b>
</div>
<b>same &amp;lt;b&amp;gt;, outside that div</b>
```

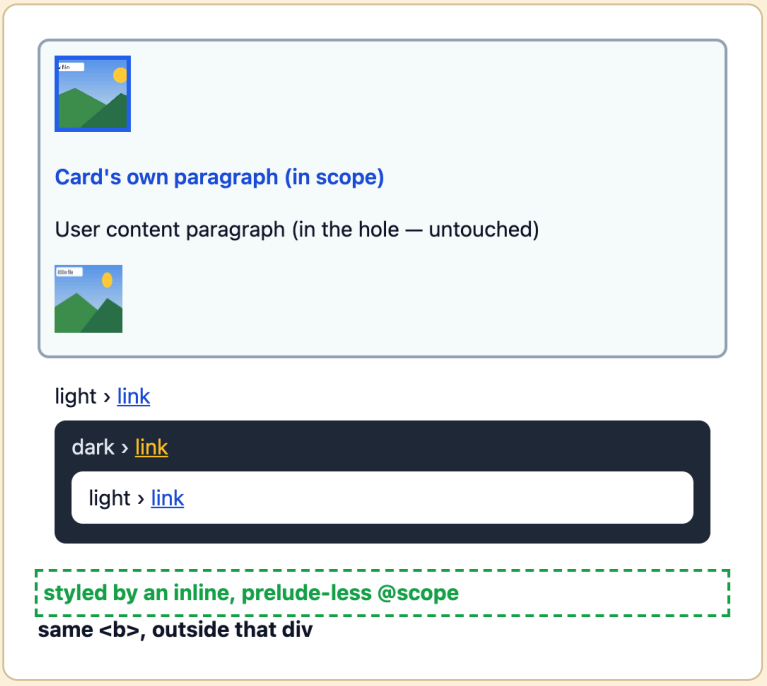


Figure 13.2 — Donut scope (top), proximity with nested themes (middle), prelude-less inline scope (bottom).

`code/05_scope.mjs`:

```
CSSScopeRule supported: true
start=.card end=.content
start=.dark end=null
start=.light end=null

.card background rgb(248, 250, 252)
card's own img border 3px   img inside .content border 0px
card's own p  rgb(29, 78, 216) 600
p in .content rgb(15, 23, 42) 400

links: light rgb(29, 78, 216)   dark rgb(251, 191, 36)   light-inside-dark rgb(29, 78, 216)
(amber = rgb(251, 191, 36), blue = rgb(29, 78, 216))

inline @scope: <b> inside rgb(22, 163, 74), <b> outside rgb(15, 23, 42)
```

The hole works (content's `img` and `p` untouched), and the innermost **light** link is blue because the nearest root decided. Without `@scope`, the later `.light a` would also colour the *dark* box's link blue.

`@scope` vs the alternatives

Need	Tool
Keep a component's styles from matching outside it	`@scope (.card)`, BEM names, CSS Modules
Stop styles at a nested region (slots, user content)	`@scope … to (…)` — only this does it natively
Nearest theme wins	`@scope` proximity (or custom properties, which inherit from the nearest ancestor anyway — 13.14)
Full isolation both ways (page CSS can't get in)	Shadow DOM (a hidden, separately styled DOM tree attached to an element; used by web components)

Common mistakes

- Expecting `@scope` to block **inheritance**: inherited properties (colour, font) still flow into the hole; only *selector matching* stops.
- Thinking the root adds specificity.

13.6 Logical properties

What it is

Logical properties describe boxes by the **flow of text**, not physical sides. The **inline** axis is the direction text runs in a line (left→right in English, right→left in Arabic, top→bottom in vertical Japanese); the **block** axis is how lines stack; `start`/`end` are each axis's beginning and end. So `margin-inline-start` is the margin **before the text starts** — left in English, right in Arabic. Baseline widely available since 2021.

Why it matters

Arabic, Hebrew, Persian and Urdu display right-to-left (`dir="rtl"`; RTL vs LTR). With physical properties (`padding-left`, `border-left`, `left: 0`) every side needs flipping in an RTL stylesheet; with logical properties **one stylesheet works for both**, and for vertical `writing-mode`s.

The mapping

Physical (horizontal LTR)	Logical
`width` / `height`	`inline-size` / `block-size` (also `min-` / `max-`)
`margin-left` / `margin-right`	`margin-inline-start` / `margin-inline-end`
`margin-top` / `margin-bottom`	`margin-block-start` / `margin-block-end`
`margin-left` + `margin-right`	`margin-inline: a b` (shorthand; `padding-inline: 1rem` = left + right padding)
`padding-*`, `border-*`	`padding-inline`, `padding-block-start`, `border-inline-start`, …
`top` / `right` / `bottom` / `left`	`inset-block-start` / `inset-inline-end` / `inset-block-end` / `inset-inline-start`
`border-top-left-radius`	`border-start-start-radius` (block-start, inline-start)
`text-align: left` / `right`	`text-align: start` / `end`
`float: left`	`float: inline-start`
`overflow-x` / `overflow-y`	`overflow-inline` / `overflow-block` (newer — check support)

Units too: `vi` / `vb` (viewport) and `cqi` / `cqb` (containers, §10.6).

Proof

`code/06_logical.html` applies one rule set to `dir="ltr"`, `dir="rtl"` and `writing-mode: vertical-rl` containers:

```
.note {
  inline-size: 220px;
  padding-inline-start: 36px;  padding-inline-end: 48px;  padding-block: 10px;
  border-inline-start: 6px solid #2563eb;
  margin-block-end: 12px;
  text-align: start;
}
.note .badge { position: absolute; inset-inline-end: 6px; inset-block-start: 6px; }
```

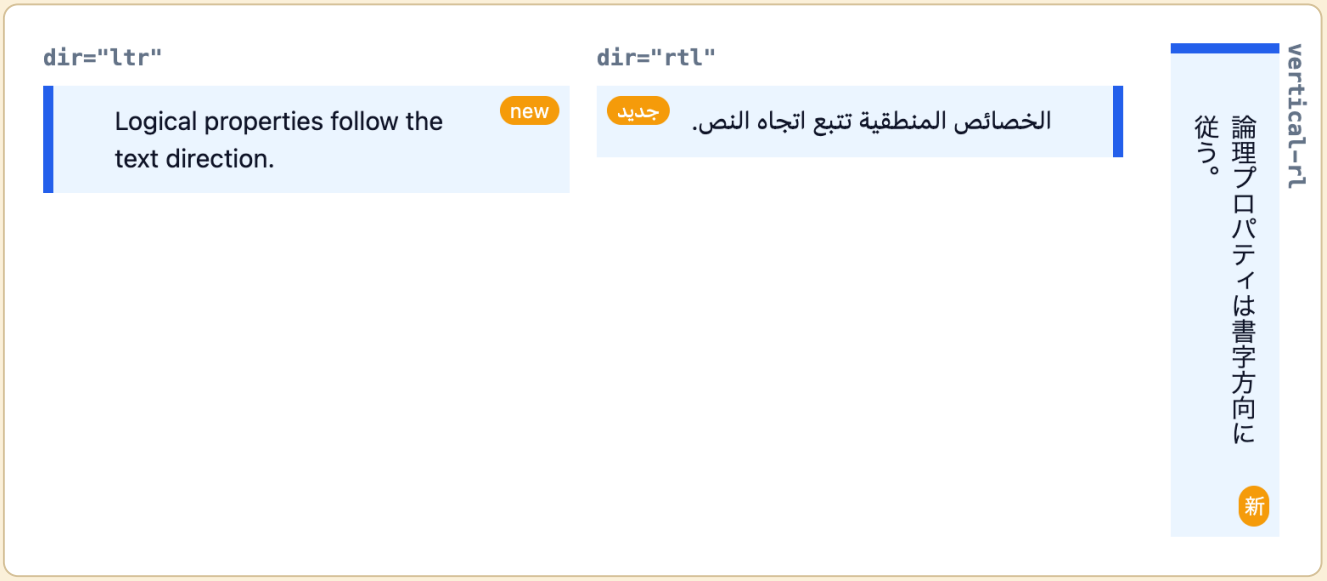


Figure 13.3 — Same CSS, three writing directions; the accent bar is always at the "start" of the text.

`code/06_logical.mjs` measures which **physical** side got each value:

```
box  size(px)   36px padding  48px padding  6px border  12px margin  badge corner
ltr  310x64     Left          Right         Left        Bottom       right/top
rtl  310x42     Right         Left          Right       Bottom       left/top
vrl  64x290     Top           Bottom        Top         Left         right/bottom
```

In vertical-rl, lines run top→bottom (inline axis vertical) and stack right→left, so inline-start = **top**, inline-end = **bottom**, block-end = **left**. LTR/RTL boxes are 310px **wide** (220 + 36 + 48 + 6); the vertical box (`inline-size: 200px`) is 290px **tall** (200 + 36 + 48 + 6). In RTL, `border-inline-end` draws on the left; in horizontal writing `block-size` is height.

When not to use it

- Genuinely physical things: a downward shadow, a fixed bottom-right "back to top" button, an arrow that points right because the image does.
- Don't mix `margin-left` and `margin-inline-start` on one element — the same side in LTR, so they fight.

Common mistakes

- `dir="rtl"` on `<html>` but hard-coded `left`/`right` in components.
- Directional icons (→, "next") must mirror in RTL: `:dir(rtl) .icon-next { scale: -1 1; }` (`:dir()` Baseline since 2023).

13.7 Why CSS needs an architecture — and BEM

The problems

CSS was designed for documents; at application scale its core ideas hurt:

Problem	What happens	Symptom
Global namespace (one shared pool of names)	Every selector can match anything on any page	Changing `.title` breaks a page you've never seen
Specificity & order dependence	Winners depend on selector weight and file order	Escalation: `!important`, `#id` hacks
No dead-code detection	You can't tell if a rule is still used	Stylesheets only grow ("append-only CSS")
Implicit coupling	Styles depend on DOM structure (`.nav ul li a`)	Moving markup breaks styles

Architectures are answers in two families: **conventions** humans follow (BEM, OOCSS, SMACSS, ITCSS, CUBE CSS) and **tooling** that enforces them (CSS Modules, CSS-in-JS, utility generators). Native features (layers, `@scope`, nesting) help both.

BEM — Block, Element, Modifier

BEM (Yandex, ~2009) gives every styled thing a unique, flat class:

- **Block** — a standalone component: `.card`
- **Element** — a part meaningful only inside its block: `.card__title` (double underscore)
- **Modifier** — a variant or state: `.card--featured`, `.btn__icon--large` (double hyphen)

`code/07_bem.html`:

```
.card { width: 170px; padding: 14px; border: 1px solid #cbd5e1; border-radius: 12px; }
.card__title  { margin: 0; font-size: 16px; }
.card__price  { margin: 4px 0 12px; color: #475569; }
.card__button { width: 100%; background: #2563eb; color: #fff; /* … */ }
.card--featured { border: 2px solid #f59e0b; background: #fffbeb; }
.card--featured .card__button { background: #d97706; }
```

```
<article class="card card--featured">
  <h3 class="card__title">Rain jacket</h3>
  <p class="card__price">$129</p>
  <button class="card__button">Add to cart</button>
</article>
```

Why it works: nearly every selector is one class → (0,1,0) everywhere, so weight and order stop mattering; names say where things belong; no dependence on tags or nesting depth (`.card h3` would couple to markup and raise specificity). **Rules:** no element-of-element (`card__body__title` → `card__title`); modifiers never alone (`class="card--featured"` without `card` is a bug); avoid styling by tag inside blocks.

Other conventions in one line each

- **OOCSS** (Object-Oriented CSS, Nicole Sullivan, 2008): separate *structure* from *skin* and *container* from *content* — reusable "objects" like `.media`.
- **SMACSS**: rules sorted into Base, Layout, Module, State (`.is-open`), Theme.
- **ITCSS** (Inverted Triangle CSS): files ordered generic/low-specificity → explicit/high (settings → tools → generic → elements → objects → components → utilities) — cascade layers (13.4) now express this natively.
- **CUBE CSS**: Composition, Utility, Block, Exception — lean on the cascade, add blocks only where needed, exceptions via data attributes.

Trade-offs

BEM needs discipline and code review (nothing enforces names), names get long, and there's still one global namespace — two teams can both invent `.card`. Tooling (13.9, 13.10) and `@scope` solve that last part.

13.8 Utility-first CSS and Tailwind CSS v4

What it is

Utility-first CSS provides many tiny single-purpose classes — `flex`, `p-4`, `text-slate-600`, `rounded-lg` — composed **in the HTML** instead of writing CSS per component. **Tailwind CSS** is the dominant framework; **v4** (January 2025; 4.3.3 here) moved configuration into CSS and builds on cascade layers, `@property` (registers a typed custom property the browser can animate and validate), `color-mix()` and custom properties for every **design token** (a named design decision — brand colour, spacing step, radius — defined once, reused everywhere).

Why teams choose it

- **CSS stops growing**: new components reuse utilities, so size tracks *distinct* utilities, not components.
- **No naming**, no file switching; deleting HTML deletes its styles (no dead CSS).
- **Constraints**: utilities come from a design scale, keeping UIs consistent.
- **Generated on demand**: the build emits only classes actually used.

How a v4 build works

1. Your CSS entry imports Tailwind and optionally defines tokens in `@theme`.
2. The build scans sources for strings that look like class names (it doesn't run your code — names must appear **complete**).
3. Each recognised class becomes a rule in `@layer utilities`, plus the theme variables it uses, the Preflight reset (13.13) in `@layer base`, and needed `@property` registrations.

`code/08_tailwind.css` and part of `code/08_tailwind.html`:

```
@import "tailwindcss" source(none);    /* don't auto-scan the folder ...        */
@source "./08_tailwind.html";           /* ... scan exactly this file             */

@theme {                                /* tokens -> CSS variables AND utilities */
  --color-brand: #2563eb;               /* enables bg-brand, text-brand, ...     */
  --color-brand-dark: #1d4ed8;
}
```

```
<article class="w-[170px] rounded-xl border border-slate-300 bg-white p-3.5">
  <h3 class="text-base font-bold">Trail shoes</h3>
  <p class="mt-1 mb-3 text-slate-600">$89</p>
  <button class="w-full rounded-lg bg-brand p-2 text-white hover:bg-brand-dark">Add to cart</button>
</article>
```

(v4 auto-detects sources, skipping `.gitignore`d files; `source(none)` + `@source` just makes this example deterministic.) `code/08_tailwind_build.mjs` runs the real CLI and inspects the output:

```
classes used in the HTML: 28
utility rules generated:   28
output: 7623 bytes (6513 minified)
top-level at-rules: layer properties | layer theme, base, components, utilities | layer theme  | layer base  | layer
utilities  | property --tw-border-style | property --tw-leading ...
theme variables emitted: --color-amber-50 --color-amber-500 --color-amber-600 --color-amber-700 --color-slate-300 --color-
slate-600 --color-slate-900 --color-white --color-brand --color-brand-dark

  .p-3\.5 {
    padding: calc(var(--spacing) * 3.5);
  }
  .bg-brand {
    background-color: var(--color-brand);
  }
  .w-\[170px\] {
    width: 170px;
  }
    .hover\:bg-brand-dark:hover {
      background-color: var(--color-brand-dark);
    }
```

- **Only what you use**: 28 classes → 28 rules, and only the 10 colour variables they reference (of hundreds). The whole stylesheet, reset included, is ~6.5 KB minified (whitespace stripped), before gzip/Brotli.
- **Layers**: `theme, base, components, utilities` — utilities are strongest, so a utility beats a component class without specificity games; your unlayered CSS beats all (13.4).
- **Spacing is a formula**: `p-3.5` = `calc(var(--spacing) * 3.5)`, `--spacing: 0.25rem` → 14px; change one variable to rescale the UI.
- **Arbitrary values** in brackets (`w-[170px]`) escape the scale.
- **Variants** (state/screen prefixes): `hover:` wraps the rule in `@media (hover: hover)` (in the full output) so touch devices avoid "sticky hover"; `md:` becomes `@media (width >= 48rem)`.

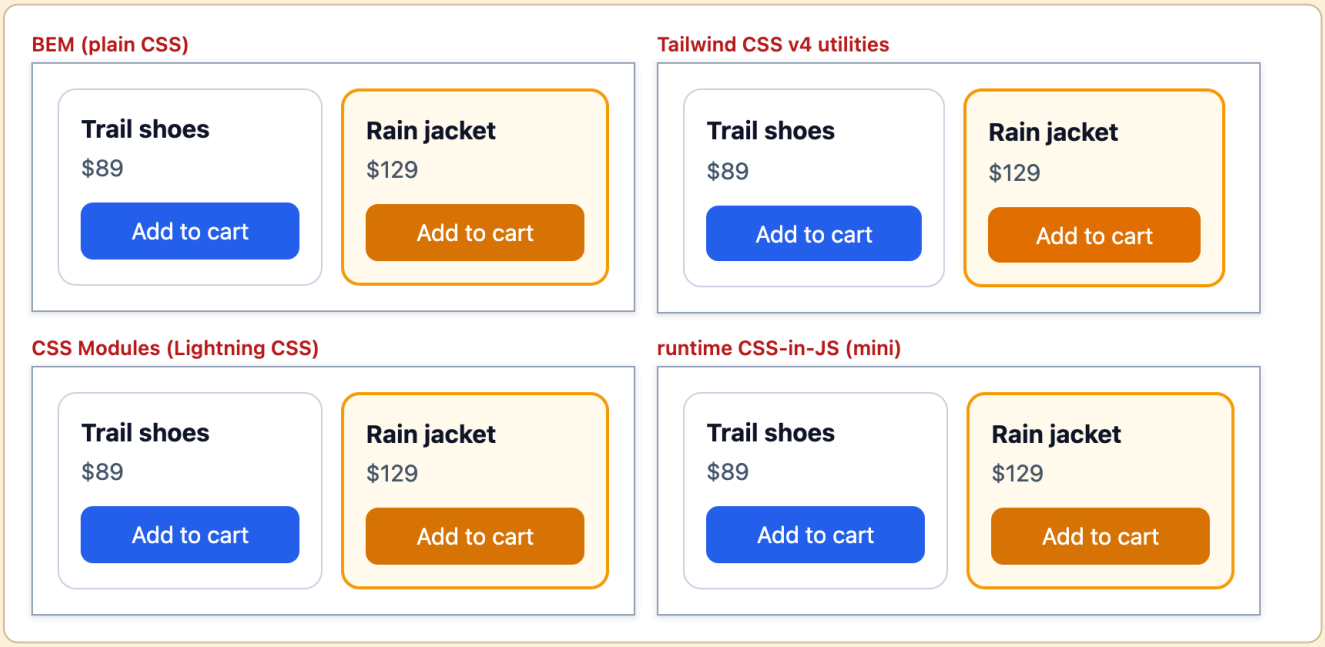


Figure 13.4 — The same two cards built four ways (13.7–13.10): identical pixels; different places styles live and how they're scoped.

Trade-offs

Pro	Con
Tiny, bounded CSS; no dead code	Long `class` attributes; HTML harder to scan
No naming, no specificity fights	Repetition unless you extract **components** (React/Vue components or template partials)
Design constraints built in	Learning the vocabulary; team must accept styling in markup
Easy to delete/refactor locally	Class names must be static strings (`bg-${color}-500` built at runtime won't be found)

Common mistakes

- **Dynamic class names** (`'text-' + color`, `'bg-' + c + '-500'`): never in the source in full, so no CSS. Map to full names: `{ red: 'text-red-600', blue: 'text-blue-600' }[color]`.
- Recreating BEM with `@apply` (copies utilities into your own rule) everywhere — losing both approaches' benefits.
- Forgetting focus styles: utilities don't add them (`focus-visible:ring-2` does).

In the interview

> **30-second answer:** "Utility-first CSS composes small single-purpose classes in the markup. Tailwind scans sources and generates only the utilities used, into cascade layers, so CSS stays bounded with no dead code or specificity war; v4 keeps configuration in CSS (`@theme`). The costs are verbose markup and a learning curve; I control repetition by extracting components, and class names must appear in full in the source."

13.9 CSS Modules

What it is

A **CSS Module** is a normal CSS file (`*.module.css` by convention) whose class names are **local by default**: the build swaps each class for a generated, collision-free name and gives JavaScript an object mapping your short names to the generated ones. You write `.title`; the page gets `.title_14He7q`; the component writes `className={styles.title}`. Built into Vite, Next.js, webpack's css-loader, Parcel and others.

How it works

1. Import from a component: `import styles from './Card.module.css'`.
2. At build time the compiler renames every class and `@keyframes` name by a pattern (file, class, a content hash).
3. It emits the renamed CSS into the bundle and returns the **name map** as the module's default export.
4. `composes: other` adds another class's generated name to the export, so the element gets both.
5. `:global(...)` opts a selector out of renaming.

`code/09_card.module.css`:

```
.card   { width: 170px; padding: 14px; border: 1px solid #cbd5e1; border-radius: 12px; background: #fff; }
.title  { margin: 0; font-size: 16px; }
.button { width: 100%; background: #2563eb; /* … */ }
.featured { composes: card; border: 2px solid #f59e0b; background: #fffbeb; }
.featured .button { background: #d97706; }
:global(body) { font: 15px/1.4 system-ui, sans-serif; margin: 16px; display: flex; gap: 12px; }
```

`code/09_css_modules.mjs` compiles it with Lightning CSS and renders the component (Figure 13.4, third panel):

```
import styles from "./card.module.css"  ->  styles = {
  button: 'button_14He7q',
  card: 'card_14He7q',
  featured: 'featured_14He7q card_14He7q',
  price: 'price_14He7q',
  title: 'title_14He7q'
}

Generated CSS (first rules):
.card_14He7q {
  background: #fff;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
  width: 170px;
  padding: 14px;
}

.title_14He7q {
  margin: 0;
  font-size: 16px;
}

wrote 09_css_modules.out.css and 09_css_modules.html
```

`.title` in another module gets a different hash, so they never collide; `styles.featured` holds **two** names because of `composes`.

Trade-offs

Pro	Con
Real CSS (all features, DevTools, cached as a static file)	Needs a build tool and JS imports
Zero runtime cost	Styling by props means toggling classes (`clsx(styles.btn, big && styles.big)`)
Scoping enforced by the tool, not discipline	Generated names unreadable in production (fine in dev with a readable pattern)
Unused module files are detectable (no import → dead)	Unused classes *within* a file aren't removed

> **Don't confuse with "CSS module scripts"**: `import sheet from './x.css' with { type: 'css' }` returns a `CSSStyleSheet` (for shadow DOM) — a different, newer browser feature, **not Baseline** (no Safari as of September 2026).

13.10 CSS-in-JS

What it is

CSS-in-JS writes styles in JavaScript beside the component, often computed from props (component inputs):

- **Runtime** libraries (styled-components, Emotion): in the browser, while rendering, they **serialise** the style object, hash it into a class name and inject a rule into a `<style>` element.
- **Zero-runtime / compile-time** libraries (vanilla-extract, Linaria, Panda CSS, StyleX): authored in JS/TS, but extracted to static CSS at build time, like CSS Modules.

How runtime CSS-in-JS works — built in 25 lines

`code/10_mini_css_in_js.js` is a toy of the mechanism:

```
function css(styles) {
  const key = JSON.stringify(styles);
  if (cache.has(key)) return cache.get(key);          // same styles -> same class, inject once
  const cls = 'css-' + hash(key);                     // name derived from content
  sheet.insertRule(`.${cls}{${toCss(styles)}}`);      // + nested '&:hover' rules
  cache.set(key, cls);
  return cls;
}
// a component: styles depend on props
el.className = css({ border: featured ? '2px solid #f59e0b' : '1px solid #cbd5e1', /* … */ });
```

`code/10_css_in_js.mjs` renders two cards in jsdom (the fourth panel of Figure 13.4 is the same code in Chrome, `code/10_css_in_js.html`):

```
card 1 classes: css-1i6k3fq css-1qt5iag css-1d6s3yc css-11at6p7
card 2 classes: css-kegznd css-1qt5iag css-1d6s3yc css-n2x86t

rules injected at runtime (8):
  .css-1i6k3fq { width: 170px; padding: 14px; border-radius: 12px; border: 1px solid rgb(203, 213, 225); backgro
  .css-1qt5iag { margin: 0px; font-size: 16px; }
  ...
  .css-kegznd { width: 170px; padding: 14px; border-radius: 12px; border: 2px solid rgb(245, 158, 11); backgroun
  .css-n2x86t { width: 100%; padding: 8px; border: 0px; border-radius: 8px; background: rgb(217, 119, 6); color
  .css-n2x86t:hover { filter: brightness(0.9); }

progress bar rendered at 0..100%: 101 extra rules
with width: var(--pct) instead: 1 class (css-1gyb3r) + style="--pct: 42%"
```

- Identical styles share a class (`css-1qt5iag` for both titles) — the cache deduplicates.
- Different props → **different classes**.
- **The cost**: a continuously changing value (a progress bar) creates a rule per value — 101 rules — and serialising, hashing and inserting happen **during render, on the main thread** (and inserted rules trigger style recalculation), hurting responsiveness. The standard fix is the native one: one static rule with a custom property (`width: var(--pct)`) set inline.

Trade-offs

Pro	Con
Styles colocated with components; prop-driven styles are natural	Runtime cost: JS bundle + style work during render (hurts the Core Web Vitals INP — responsiveness — and LCP — main-content speed, §14.3)
Scoped automatically; dead styles die with the component	**Server rendering** (SSR — HTML built on the server) needs extra setup to collect styles; poor fit for **React Server Components** (server-only components that ship no JS; runtime libraries have needed *client components*)
TypeScript types for style props (zero-runtime libs)	CSS can't be cached separately from JS (runtime libs)

The ecosystem has moved to zero-runtime tools, CSS Modules and Tailwind. In March 2025 the styled-components maintainer announced **maintenance mode** (bug fixes only) and advised against it for new projects, citing its poor fit with React Server Components. It's still maintained — v6.4 (April 2026) added RSC-friendly theming via custom properties and faster re-renders — so existing projects are safe, but the advice for new work stands.

13.11 Choosing an approach

	BEM (plain CSS)	Utility-first (Tailwind)	CSS Modules	Runtime CSS-in-JS	Zero-runtime CSS-in-JS
Scoping	Convention	N/A (no custom selectors)	Build tool (renamed classes)	Generated classes	Build tool
Build step	Optional	Required	Required	No (runtime)	Required
Runtime cost	None	None	None	**Yes**	None
Dynamic styles	Classes / custom props	Classes / custom props	Classes / custom props	Props → styles	Variants / custom props
Dead-code removal	Manual	Automatic	Per file	Automatic	Automatic
CSS size growth	Grows with features	Plateaus	Grows with features	Grows with features	Grows / atomic (one class per declaration, StyleX) plateaus
Works without a JS framework	Yes	Yes	Needs a bundler	No	Needs a bundler
Good fit	Content sites, CMS themes, small teams	Product UIs, design systems with fixed scale	React/Vue apps wanting "just CSS"	Legacy apps already using it	TS-heavy apps wanting typed styles

Practical defaults in 2026:

1. **Content site / no framework** → plain CSS with layers, nesting, BEM-like names or `@scope`, and tokens.
2. **React/Vue/Svelte app** → Tailwind *or* CSS Modules (Svelte and Vue also scope `<style>` blocks natively) + tokens as custom properties.
3. **Always**: reset and tokens in low layers, dynamic values as custom properties, and don't mix three approaches in one codebase.

In the interview

30-second answer: "CSS's global namespace and specificity cause most maintenance pain. BEM solves it by convention with flat single-class names; CSS Modules by renaming classes at build time; utility-first by generating single-purpose classes so you rarely write new CSS; CSS-in-JS by generating classes from JS. I'd pick CSS Modules or Tailwind for a component app, plain CSS with layers and BEM or `@scope` for a content site, and avoid runtime CSS-in-JS in new projects for its render-time cost and poor fit with server components."

- *Migrating a big legacy stylesheet?* → Wrap it in a low `@layer legacy`, write new components in a higher layer with the chosen approach, delete legacy rules as pages migrate.

13.12 Preprocessors (Sass) and PostCSS

What it is

A **preprocessor** is a language that compiles to CSS. **Sass** (SCSS syntax, a superset of CSS) dominates; Less and Stylus are older. **PostCSS** instead parses CSS into a tree and runs plugins on it (Autoprefixer adds **vendor prefixes** like `-webkit-` that older browsers needed; others lower new syntax). **Lightning CSS** is a fast all-in-one compiler written in Rust (lowering, prefixing, minifying, CSS Modules) — used in 13.1 and 13.9.

What Sass adds

`code/11_sass_example.scss` — **(not run: Sass isn't in this book's toolchain)**:

```scss
@use "sass:color";
@use "sass:math";
$brand: #2563eb;                       // compile-time variable
$sizes: (sm: 12px, md: 16px, lg: 20px);

@mixin focus-ring($c: $brand) { outline: 2px solid $c; outline-offset: 2px; }
%button-base { border: 0; font: inherit; }   // placeholder for @extend

.card {
  padding: math.div(24px, 2);          // -> 12px
  &__title { font-size: 20px; }        // -> .card__title (string joining)
  &--featured { border-color: color.adjust($brand, $lightness: -10%); }
}
.button {
  @extend %button-base;
  &:focus-visible { @include focus-ring; }
}
@each $name, $size in $sizes { .text-#{$name} { font-size: $size; } }   // generates 3 classes
```

Modern Sass: use the module system (`@use`/`@forward` — each file loaded once, names private unless shared); `@import` is **deprecated** since Dart Sass 1.80 (October 2024) and will be removed in Dart Sass 3.0; global functions like `darken()` are deprecated for `color.adjust()`/`color.scale()`; **Dart Sass** is the only maintained implementation (LibSass and node-sass are deprecated). A Sass `$brand` can't change per section at runtime — it's replaced at compile time.

What native CSS now covers

`code/11_native.html` rewrites the Sass ideas in plain CSS (`code/11_native.mjs` checks Chrome; `oklch(L C H)` = lightness, chroma, hue in a perceptually even space):

```css
:root { --brand: #2563eb; --radius: 8px; }
.card {
  padding: calc(24px / 2);
  .card-title { font-size: 20px; }
  &.is-featured { border-color: oklch(from var(--brand) calc(l - 0.1) c h); }  /* relative colour */
}
.button {
  background: var(--brand);
  &:hover { background: color-mix(in oklch, var(--brand), black 15%); }
}
.theme-green { --brand: #16a34a; }     /* runtime re-theming */
```

```
padding: calc(24px / 2)                -> 12px
border-color: oklch(from --brand l-0.1) -> oklch(0.44613 0.215194 262.885)
.button background (var(--brand))       -> rgb(37, 99, 235)
same rule under .theme-green            -> rgb(22, 163, 74)
hover: color-mix(in oklch, brand, black 15%) -> oklch(0.464211 0.182915 262.885)
```

Sass feature	Native CSS today	Status (Sept 2026)
`$variables`	Custom properties `--x` — **live**, inherit, change per element and at runtime (the green button)	Widely available
Nesting	Native nesting (13.1) — no `&__` joining	Widely available
`math.div`, arithmetic	`calc()`, `min()`, `max()`, `clamp()`, `round()`, `mod()`	`calc` etc. widely; `round()/mod()/rem()` newly available (2024)
`darken()` / `color.adjust`	`color-mix()` (widely) and relative colour syntax `oklch(from …)` (newly available, Sept 2024)	✓
Partials / `@use`	`@import` (blocks parallel loading — avoid in production) or bundler concatenation; `@import … layer()`	✓
`@mixin`	CSS `@mixin` / `@apply` spec still being designed	No browser
`@function`	Custom functions `@function --name()`	**Chromium only** (139+)
`@if` / `if()`	`if()` function	**Chromium only** (137+)
`@each` / loops generating classes	None — generate at build time (or use utilities)	—
`@extend`	None (rarely a good idea: it rewrites selectors unpredictably)	—

Verdict. For new projects native CSS covers most daily Sass use; keep Sass for mixins, loops or an existing Sass design system. A PostCSS/Lightning CSS step for prefixes and lowering is still common either way.

13.13 Resets and normalize

What it is

Every browser applies a **user-agent stylesheet**: 8px body margin, heading and paragraph margins, bullets and 40px list padding, form controls with their own small font. A **reset** removes or neutralises these. Three generations:

1. **Hard resets** (Eric Meyer, 2007/2011): zero almost everything, including list styles and heading sizes — you restyle it all.
2. **normalize.css** (2012): keep useful defaults but make browsers **consistent** and fix bugs.
3. **Modern minimal resets** (Andy Bell, Josh Comeau, Tailwind's Preflight): a dozen rules fixing today's real annoyances; browsers are consistent enough that normalizing matters less.

A modern reset, line by line

`code/12_reset.css` — in the lowest layer so anything can override it without specificity fights:

```
@layer reset {
  *, *::before, *::after { box-sizing: border-box; }          /* width includes padding + border */
  * { margin: 0; }                                             /* add spacing on purpose */
  html { -webkit-text-size-adjust: none; text-size-adjust: none; }  /* no iOS text inflation */
  body { min-height: 100dvh; line-height: 1.5; -webkit-font-smoothing: antialiased; }
  img, picture, video, canvas, svg { display: block; max-width: 100%; }
  input, button, textarea, select { font: inherit; }          /* controls don't inherit fonts by default */
  p, h1, h2, h3, h4, h5, h6 { overflow-wrap: break-word; }    /* long URLs can't overflow */
  h1, h2, h3, h4, h5, h6 { text-wrap: balance; }
  p { text-wrap: pretty; }
  :where(ul, ol)[role="list"] { list-style: none; padding: 0; }
  @media (prefers-reduced-motion: reduce) {
    *, *::before, *::after { animation-duration: 0.01ms !important; transition-duration: 0.01ms !important; scroll-
behavior: auto !important; }
  }
}
```

`code/12_reset.mjs` loads `code/12_reset.html` at 400px, measures, adds the reset, measures again:

```
property                                browser default          with reset
body margin                             8px                      0px
h1 margin-block                         21.44px 21.44px          0px 0px
h1 font-size                            32px                     32px
#box width (200px + padding + border)   250px                    200px
img display / rendered width            inline / 800px           block / 400px
button font                             13.3333px Arial          16px Times
ul list-style / padding-left            disc / 40px              none / 0px
p scrollWidth (overflow?)               606px of 384px           400px of 400px
```

Margins gone; `box-sizing` makes `width: 200px` really 200px (§7.2); the 800px image no longer overflows 400px; the button uses the page font (Times here since the demo sets none — normally your `body` font); the `role="list"` list lost its bullets; the long URL wraps (606px in a 384px box before). Heading sizes untouched — this reset only removes *surprises*.

- **`role="list"`**: Safari's VoiceOver stops announcing a list when `list-style: none` is applied; `role="list"` restores it, so the reset strips bullets only from lists carrying it.
- **`:where()`** keeps that rule low-specificity (13.2); the **`layer`** keeps *all* of it below your styles (13.4).
- **`text-wrap: pretty`** is not Baseline (no Firefox, September 2026) — harmless where unsupported. `text-size-adjust` is non-standard/partial, hence both spellings.
- **Never** put `outline: none` / `*:focus { outline: 0 }` in a reset — it removes the keyboard focus indicator (§3.7).

13.14 Theming and dark mode with custom properties

What it is

A **theme** is a set of values (colours, radii, fonts, spacing) applied consistently. Build it with **design tokens** as **custom properties** in three tiers, plus `color-scheme` (which light/dark modes the page supports) and `light-dark()` (picks one of two values by active mode). §10.5 introduced `prefers-color-scheme` and basic dark mode; §6.9 custom properties. This section combines them into an architecture.

The three tiers of tokens

Tier	Example	Who uses it
Primitive (reference) tokens	`--blue-600: #2563eb`	Only the semantic tokens
Semantic (system) tokens	`--color-accent`, `--color-surface`, `--color-text`	Components (a button references `--color-accent`)
Component tokens (optional)	`--card-bg: var(--color-surface)`	One component; lets one instance be tweaked

Components never mention a raw colour; dark mode or a new brand changes **only semantic tokens**.

How it works

1. `color-scheme: light dark` on `:root` declares support; the browser themes its own UI (controls, scrollbars, default background), and **`light-dark(a, b)` picks `a` or `b` by the element's used colour scheme** (so `light-dark(#fff, #000)` under a dark OS gives `#000`). Baseline newly available since May 2024 (colours only; image values newer still).
2. Each semantic token gets a `light-dark()` pair — no duplicated `@media` block.
3. A user choice **overrides** the OS by pinning `color-scheme`: `:root[data-theme="dark"] { color-scheme: dark }`. Because `light-dark()` follows `color-scheme`, every token flips without duplication.
4. A tiny **blocking script in `<head>`** reads the saved choice from `localStorage` (per-site key–value storage surviving reloads) and sets `data-theme` *before first paint*, so there's no flash of the wrong theme (a justified exception to "don't block rendering", §14.2). It goes on `<html>` because tokens live on `:root` and `<body>` doesn't exist yet.
5. Other axes (brand, density, high contrast) are attribute-scoped token overrides: `[data-brand="green"] { --color-accent: … }`. Custom properties inherit, so an override on any element re-themes its subtree.

`code/13_theming.html` (key parts):

```
<meta name="color-scheme" content="light dark">
<script>
  try { const t = localStorage.getItem('theme');
        if (t === 'light' || t === 'dark') document.documentElement.dataset.theme = t; } catch {}
</script>
```

```
@layer tokens, base, components;
@layer tokens {
  :root {
    --blue-600: #2563eb;  --blue-300: #93c5fd;  --slate-900: #0f172a; /* … primitives */
    color-scheme: light dark;
    --color-bg:      light-dark(var(--slate-50), var(--slate-900));
    --color-surface: light-dark(#fff, var(--slate-800));
    --color-text:    light-dark(var(--slate-900), var(--slate-200));
    --color-accent:  light-dark(var(--blue-600), var(--blue-300));
  }
  :root[data-theme="light"] { color-scheme: light; }
  :root[data-theme="dark"]  { color-scheme: dark; }
  [data-brand="green"] { --color-accent: light-dark(#15803d, #86efac); }
}
@layer components {
  .card a { color: var(--color-accent); }
  .btn { background: var(--color-accent); color: var(--color-surface); }
}
```

```
select.addEventListener('change', () => {
  if (select.value === 'system') delete root.dataset.theme; else root.dataset.theme = select.value;
  try { select.value === 'system' ? localStorage.removeItem('theme') : localStorage.setItem('theme', select.value); }
catch {}
});
```

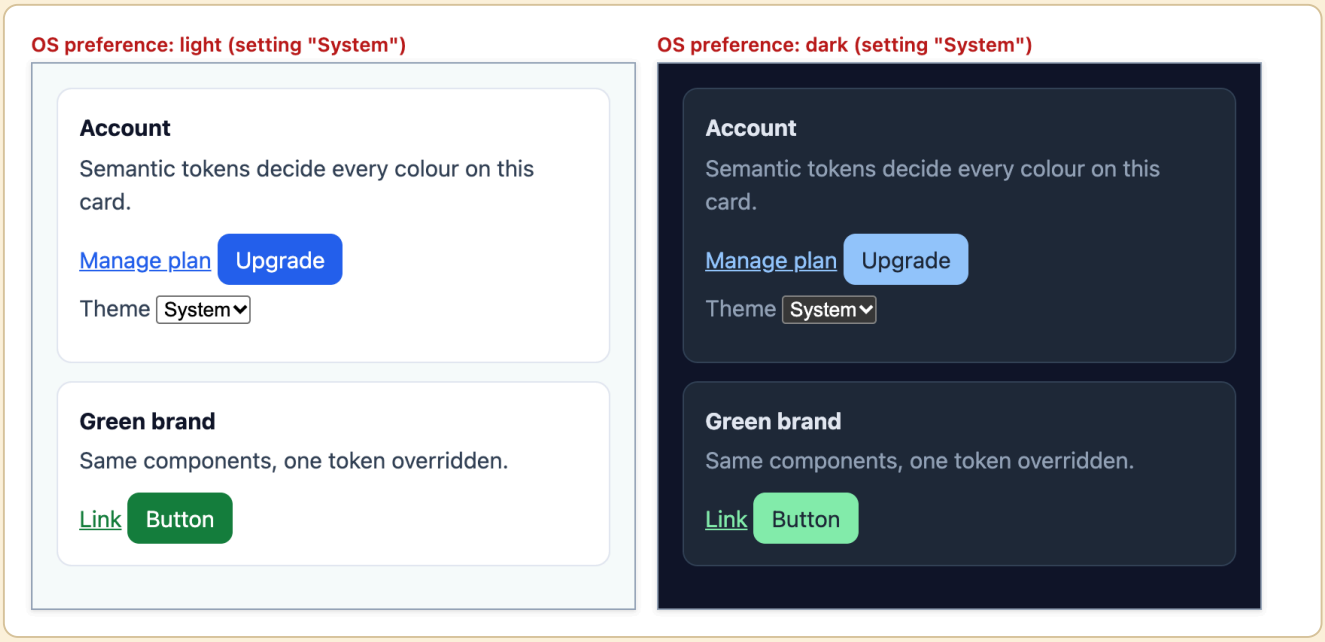


Figure 13.5 — Setting "System" under emulated light and dark OS preferences; the select box and page background are themed by `color-scheme` with no CSS of their own.

`code/13_theming.mjs` tests every OS preference × page setting, then reloads to test persistence:

```
OS pref  setting   page bg              text                accent (link)        green-brand link
light    system    rgb(248, 250, 252)   rgb(15, 23, 42)      rgb(37, 99, 235)     rgb(21, 128, 61)     saved=null
light    light     rgb(248, 250, 252)   rgb(15, 23, 42)      rgb(37, 99, 235)     rgb(21, 128, 61)     saved=light
light    dark      rgb(15, 23, 42)      rgb(226, 232, 240)   rgb(147, 197, 253)   rgb(134, 239, 172)   saved=dark
dark     system    rgb(15, 23, 42)      rgb(226, 232, 240)   rgb(147, 197, 253)   rgb(134, 239, 172)   saved=null
dark     light     rgb(248, 250, 252)   rgb(15, 23, 42)      rgb(37, 99, 235)     rgb(21, 128, 61)     saved=light
dark     dark      rgb(15, 23, 42)      rgb(226, 232, 240)   rgb(147, 197, 253)   rgb(134, 239, 172)   saved=dark

after reload: data-theme=dark  body bg=rgb(15, 23, 42)  <select> shows "dark"
```

"System" follows the OS, an explicit choice beats it, the brand override keeps its own light/dark pair, and after reload the head script applied "dark" before rendering.

Going further

- **Typed tokens with `@property`** (Baseline newly available since July 2024): `@property --accent { syntax: '<color>'; inherits: true; initial-value: #2563eb; }` lets the browser *interpolate* the token (animated theme changes) and rejects invalid values.
- **Fallback without `light-dark()`**: light tokens on `:root`, overridden in `@media (prefers-color-scheme: dark)` and `[data-theme=dark]` — the Chapter 10 pattern.
- **Contrast**: check every token pair (text on bg/surface, accent on surface) at ≥ 4.5:1 in *both* themes (§11.4).
- `contrast-color()` (returns black or white for readable text on a background) became Baseline in April 2026 — useful for user-chosen brand colours.

Common mistakes

- `light-dark()` without `color-scheme` → always the light value.
- Components using primitive tokens (`var(--blue-600)`) — dark mode then needs component changes.
- Setting the theme in a `useEffect` (React hook running after render) or `DOMContentLoaded` handler → paints light, then flashes dark.
- Storing the theme only in a cookie read by JS after load — same flash (unless rendered server-side from the cookie).

In the interview

30-second answer: "I define primitive tokens, then semantic tokens like `--color-surface` that components use. For dark mode I set `color-scheme: light dark` and give each semantic token a `light-dark()` pair; a user override pins `color-scheme` on the root via `data-theme`, saved in localStorage and applied by a tiny inline head script to avoid a flash. Extra themes are scoped token overrides, which work because custom properties inherit."

Chapter 13 TL;DR

- **Nesting** (Baseline 2023): nested selectors are relative to the parent; `&` = parent, like `:is()` (highest specificity of the parent list); no `&__` string joining (Chrome dropped that rule); nested `@media` works; Lightning CSS can lower it.
- **`:is()` / `:where()`**: "any of"; forgiving lists; `:is()` = max argument specificity, `:where()` = 0 → `:where()` for resets and library defaults.
- **`:has()`** (Baseline 2023): parent / previous-sibling / state selector (`.card:has(img)`, `h3:has(+ .subtitle)`, `label:has(:checked)`, `html:has(dialog[open])`); no nested `:has()`; anchor it.
- **`@layer`**: declare order first; later layers win regardless of specificity (a class in `components` beat `#app .btn.btn-primary` in `vendor`); unlayered beats layers; `!important` flips; `revert-layer`; `@import … layer()`.
- **`@scope`** (Baseline March 2026): `@scope (.card) to (.content)` donut scope; `:scope`; proximity breaks specificity ties; prelude-less `@scope` in inline `<style>`.
- **Logical properties**: `inline`/`block` + `start`/`end`; one stylesheet for LTR, RTL and vertical text.
- **Architecture**: global namespace + specificity + dead code are the problems. **BEM** (`block__element--modifier`), **Tailwind v4** (generated utilities in `@layer utilities`, only what's used — 28 classes → 28 rules, ~6.5 KB), **CSS Modules** (`.title` → `.title_14He7q`, `composes`), **CSS-in-JS** (runtime hash + inject during render, 101 rules for 101 progress values; prefer zero-runtime or custom properties).
- **Sass**: variables, mixins, loops, `&__` joining; `@use` not `@import`; native CSS covers variables, nesting, colour maths — not mixins/loops (`@function`/`if()` Chromium-only).
- **Reset**: small modern reset in `@layer reset`; never remove focus outlines.
- **Theming**: primitive → semantic → component tokens; `color-scheme` + `light-dark()` (Baseline 2024); `[data-theme]` pins `color-scheme`; persist in localStorage, apply in `<head>` before first paint.

Next: Chapter 14 — Performance, SEO, Interview Prep and Cheat Sheets.

Chapter 14 — Performance, SEO, Interview Prep and Cheat Sheets

A semantic, accessible, well-laid-out page still fails users if the main image takes four seconds, content jumps while loading, or every click freezes for half a second — and fails its owners if search engines can't understand it. This chapter covers **web performance** (critical rendering path, render-blocking resources, Core Web Vitals — measured on real pages in headless Chrome) and **SEO basics**, then turns the book into revision material: 47 interview questions linked to the chapters, one-page cheat sheets, and a glossary of more than 200 terms.

In this chapter: 14.1 The critical rendering path · 14.2 Render-blocking resources (measured: FCP 848 → 40 ms) · 14.3 Core Web Vitals (LCP, INP, CLS, measured with `PerformanceObserver`) · 14.4 Fixing each metric · 14.5 SEO basics (title, meta, canonical, JSON-LD) · 14.6 Interview questions · 14.7 Cheat sheets · 14.8 Glossary.

How the numbers were made. Performance only means something on a slow network, so `code/serve.mjs` fakes one without a server: headless Chrome requests `http://site.test/<file>`, and Puppeteer intercepts each request and answers from `code/` after an artificial delay ("the stylesheet takes 800 ms"). Absolute timings are machine-dependent but repeat within a few ms. Metrics are read with the browser's performance APIs (`PerformanceObserver` delivers timing records) — the same ones the `web-vitals` library and **real-user monitoring** (RUM: timings collected from actual visitors) use. Thresholds were checked on web.dev and SEO rules on Google Search Central (Google's documentation for site owners) in September 2026.

14.1 The critical rendering path

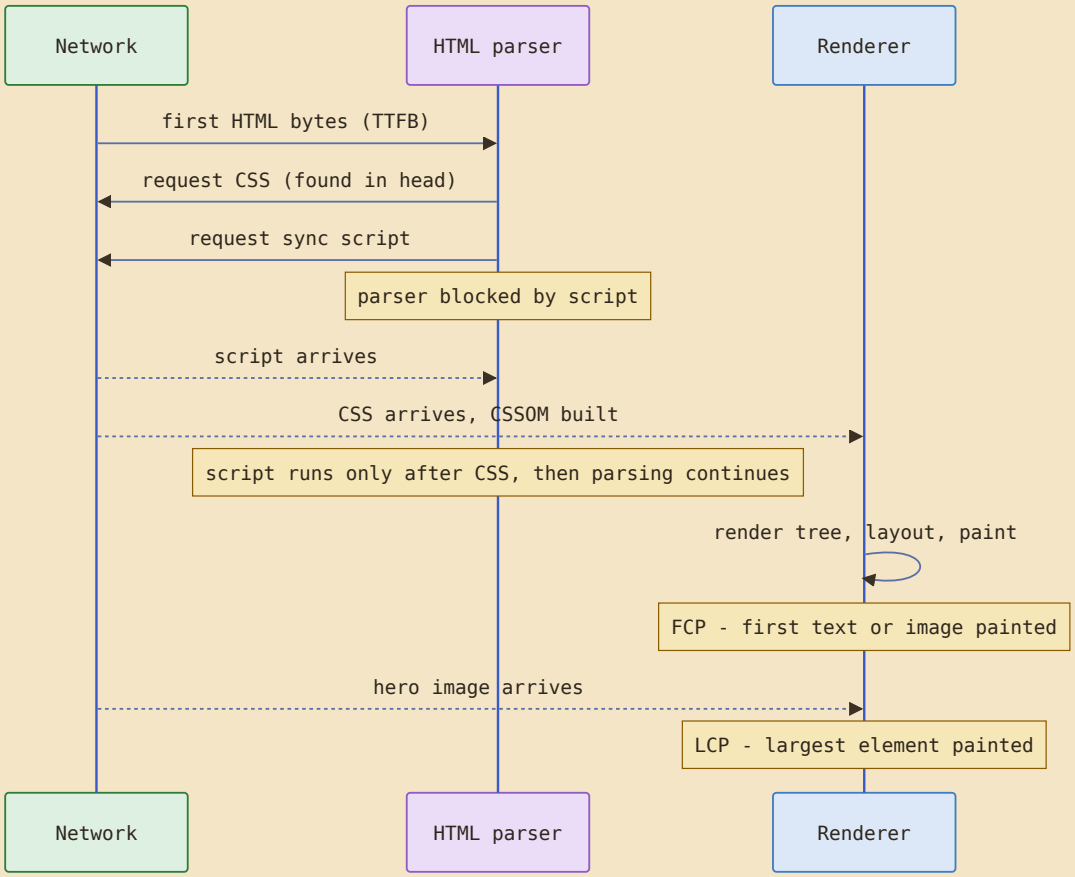
What it is

The **critical rendering path** (CRP) is the minimum work before the first pixels: HTML → **DOM**, CSS → **CSSOM**, both → **render tree** (displayed elements with final styles) → **layout** → **paint** → **composite** (§1.7 walks through each). Here we care about **what makes it slow**.

A **resource** (stylesheet, script, image, font) is **critical** if the first render must wait for it. By default:

- **CSS is render-blocking**: no paint until every `<head>` stylesheet is loaded and parsed — otherwise unstyled content would show and then jump (**FOUC**, flash of unstyled content, plus a reflow).
- **A classic `<script>` without `async`/`defer` is parser-blocking**: the HTML parser stops, downloads the script, *waits for pending stylesheets* (the script might read computed styles), runs it, then continues; nothing below it is in the DOM yet. (Render-blocking delays painting; parser-blocking stops the DOM being built.)
- **Images, fonts, `async`/`defer` scripts aren't render-blocking** — but can still delay what users feel (main image, text appearing).

How it works — the timeline and its metrics



Metric	Measures	Good (75th percentile — the value 75 % of real visits beat or match)
TTFB (Time To First Byte)	Server + network until the first HTML byte	≤ 0.8 s
FCP (First Contentful Paint)	First text/image painted	≤ 1.8 s
LCP (Largest Contentful Paint)	Main content painted (Core Web Vital, 14.3)	≤ 2.5 s
INP, **CLS**	Responsiveness and visual stability (Core Web Vitals)	≤ 200 ms, ≤ 0.1

The three levers

1. **Fewer critical resources** — inline small critical CSS, `defer` scripts, don't load what the first screen doesn't need.
2. **Fewer critical bytes** — minify and compress (gzip/Brotli/zstd), remove unused CSS/JS.
3. **Shorter critical path length** (round trips — request, wait, then the next) — make critical resources discoverable in the HTML, preconnect to other **origins** (scheme + host + port), avoid chains (CSS `@import`ing CSS, JS injecting the main image).

14.2 Render-blocking resources

The experiment

One article page with three `<head>`s; the stylesheet takes **800 ms**, the script **600 ms** (`code/01_render_blocking.mjs`).

`code/01_blocking.html` — the classic mistake:

```
<link rel="stylesheet" href="01_styles.css">   <!-- render-blocking -->
<script src="01_app.js"></script>              <!-- parser-blocking -->
```

`code/01_defer.html` — only the script fixed:

```
<link rel="stylesheet" href="01_styles.css">
<script src="01_app.js" defer></script>
```

`code/01_optimized.html` — critical CSS inlined, the rest loaded without blocking:

```
<style>/* only what the first screen needs */ body{…} header{…} main{…}</style>
<link rel="preload" href="01_styles.css" as="style" onload="this.onload = null; this.rel = 'stylesheet'">
<noscript><link rel="stylesheet" href="01_styles.css"></noscript>
<script src="01_app.js" defer></script>
```

Output:

```
01_blocking.html   FCP  848 ms   DOMContentLoaded  833 ms   load  834 ms   app.js ran at 833 ms
                   01_app.js blocking (done at 632 ms)  |  01_styles.css blocking (done at 833 ms)
01_defer.html      FCP  844 ms   DOMContentLoaded  833 ms   load  837 ms   app.js ran at 833 ms
                   01_app.js non-blocking (done at 631 ms)  |  01_styles.css blocking (done at 832 ms)
01_optimized.html  FCP   40 ms   DOMContentLoaded  631 ms   load  631 ms   app.js ran at 631 ms
                   01_app.js non-blocking (done at 630 ms)  |  01_styles.css non-blocking (done at 830 ms)
```

(`blocking`/`non-blocking` is Chrome's own classification from `PerformanceResourceTiming.renderBlockingStatus`.)

1. **Blocking**: nothing for ~850 ms; the script downloaded by 632 ms but **ran at 833 ms** — it waited for the stylesheet.
2. **Only `defer`**: FCP unchanged (848 → 844 ms) because the stylesheet was the bottleneck. Fixing the wrong resource buys nothing — find the *critical* one first. (`defer` still helped: the parser never stopped.)
3. **Optimized**: first paint at **40 ms**, ~20× sooner. The preloaded stylesheet arrived at 830 ms and applied itself (`onload` flips `rel` to `stylesheet` without ever blocking render); `load` fired at 631 ms without waiting for it. (`DOMContentLoaded` = HTML parsed and deferred scripts run; `load` = page and blocking resources loaded.)

The toolbox

Resource	Default	Better
Stylesheet	Render-blocking	Keep small; inline critical CSS (≈ first screen, a few KB); load the rest non-blocking; a non-matching `media` (e.g. `print`) makes it non-blocking (still downloaded, low priority); split per page
Classic script in `<head>`	Parser-blocking	`defer` (in order after parsing, before `DOMContentLoaded`) or `async` (on arrival, any order — independent scripts like analytics); `type="module"` is deferred by default
Web font	Text invisible until loaded (up to ~3 s)	`font-display: swap`/`optional`, `<link rel="preload" as="font" type="font/woff2" crossorigin>`, subset, self-host (§11.2)
Main image	Discovered when the parser reaches it	In the HTML, `fetchpriority="high"`, never `loading="lazy"` above the fold (§4.4)
Other origins (a CDN — content delivery network of nearby copy servers — font hosts)	DNS + TCP + TLS on first use — each at least one round trip (§1.3–1.4)	`<link rel="preconnect" href="https://cdn.example">` for the 1–3 most important
`@import` in CSS	Serial chain: CSS → discover → fetch CSS	`<link>` both files, or bundle

```
<!-- defer vs async -->
<script src="app.js" defer></script>       <!-- download in parallel, run after parsing, in order -->
<script src="analytics.js" async></script>  <!-- download in parallel, run ASAP, any order -->
<script type="module" src="main.js"></script>  <!-- deferred by default -->
```

Related features: `rel="preload"` (Baseline widely available) fetches a known-needed resource early; `rel="modulepreload"` for ES modules; `fetchpriority` (Baseline since October 2024) nudges priority; `blocking="render"` (deliberately render-blocking) is **not Baseline** (no Firefox); `<script type="speculationrules">` prerendering of the next page is **Chromium-only**. HTTP **103 Early Hints** lets a server send `Link: rel=preload` headers before the HTML is ready.

Common mistakes

- Preloading everything — preloads compete with truly critical requests. Preload only late-discovered critical resources (a font referenced from CSS, a hero image set in CSS or JS).
- No `crossorigin` on font preloads → downloaded twice.
- Scripts at the end of `<body>` *and* `defer` — fine, but `defer` in `<head>` starts the download earlier.
- Inlining **all** CSS — uncacheable across pages, bloats every HTML response.

In the interview

> **30-second answer:** "CSS in the head is render-blocking and classic scripts are parser-blocking; a script also waits for pending CSS. So I keep critical CSS small or inline it, load the rest without blocking, `defer` scripts (or `async` for independent ones), make the LCP image discoverable in the HTML with `fetchpriority=high`, preconnect to key origins, and avoid chains like CSS `@import`."

- *`async` vs `defer`?* → Both download in parallel; `defer` runs after parsing, in order; `async` runs on arrival, any order, possibly mid-parse.
- *Where should `<script>` go?* → In `<head>` with `defer` (or `type="module"`), so the download starts early without blocking the parser.

14.3 Core Web Vitals: LCP, INP, CLS

What it is

Core Web Vitals are Google's three user-experience metrics for loading, interactivity and visual stability, measured on **real visits** ("field data", vs "lab data" from test runs). A page passes when the **75th percentile** of visits (mobile and desktop separately) is "good".

Metric	Question it answers	Good	Needs improvement	Poor
LCP — Largest Contentful Paint	When did the main content appear?	≤ 2.5 s	2.5–4.0 s	> 4.0 s
INP — Interaction to Next Paint	How quickly does the page respond to clicks, taps and key presses?	≤ 200 ms	200–500 ms	> 500 ms
CLS — Cumulative Layout Shift	How much does content jump unexpectedly?	≤ 0.1	0.1–0.25	> 0.25

INP **replaced First Input Delay (FID)** in **March 2024**. FID measured only the *delay before the first* interaction's handler started; INP measures the *whole* time to the next painted frame, for (almost) *all* interactions.

How each is computed

LCP

1. Candidates: `<img>`, `<image>` inside SVG, `<video>` (poster or first frame), elements with CSS `background-image: url()`, and text-containing block elements.
2. Each larger painted candidate emits a new `largest-contentful-paint` entry; the **last** before the user's first input (click, tap, key press, scroll) is the LCP.
3. It splits into **TTFB** → **resource load delay** (until the image request starts) → **resource load duration** → **element render delay**.

INP

1. For each discrete **interaction** (click/tap = pointerdown + pointerup + click; key press = keydown + keyup; hover and scroll don't count), **latency** = **input delay** (the main thread is busy before your handlers can

run) + **processing time** (your handlers) + **presentation delay** (style, layout, paint to the next frame).
2. The page's INP is the **worst** interaction latency — or, with many interactions, a high percentile (one outlier ignored per 50).
3. The Event Timing API reports it rounded to 8 ms; entries shorter than `durationThreshold` (default 104 ms, minimum 16 ms) aren't reported.

CLS

1. Each time a *visible* element moves between frames without user cause, a **layout shift** is scored **impact fraction** (share of viewport affected) × **distance fraction** (distance moved, relative to the viewport).
2. Shifts within 500 ms of input are excluded (`hadRecentInput`) — expected movement.
3. Shifts group into **session windows** (gaps < 1 s, window ≤ 5 s); CLS = the **largest window's sum**.

The experiment

Two versions of a product-guide page on a simulated phone (412×800) and slow network (CSS 300 ms, JS 800 ms, hero image 1800 ms):

	02_bad (**html**)	**02_fixed** (**html**)
Script	Sync in `<head>`	`defer`
Hero image	Injected by JS after `DOMContentLoaded`, `loading="lazy"`, no `width`/`height`	In the HTML, `width="800" height="533"`, `fetchpriority="high"`
Promo banner (appears after 1 s)	Inserted above the content into an empty `<div>`	Inserted into a slot with `min-height` reserved
Click handler	300 ms of synchronous work, then updates the UI	Updates the UI first, **yields** (pauses so the browser can paint and handle input), then does the 300 ms in 50 ms chunks

The INP fix:

```
const yieldToMain = () => globalThis.scheduler?.yield ? scheduler.yield() : new Promise(r => setTimeout(r, 0));

button.addEventListener('click', async () => {
  status.textContent = 'Saved ✓';           // 1. visible feedback first
  await yieldToMain();                        // 2. let the browser paint
  for (let i = 0; i < 6; i++) {               // 3. the same work, in small chunks
    busyFor(50);
    await yieldToMain();
  }
});
```

`code/02_vitals.mjs` installs three `PerformanceObserver`s (types `largest-contentful-paint`, `layout-shift`, and `event` with `durationThreshold: 16`, all `buffered`) before any page script, loads each page, clicks the button like a user, and computes the metrics as defined above (CLS = largest session window; INP = worst interaction, max duration per `interactionId`):

```
02_bad.html
  LCP 2648 ms  (element <img>, candidates: <p> @844, <img> @2648)  -> needs improvement
  CLS 0.195  (shifts: 0.069 @1833ms, 0.126 @2638ms)  -> needs improvement
  INP 304 ms  (slow events in the click: pointerdown, pointerup, click)  -> needs improvement
02_fixed.html
  LCP 1844 ms  (element <img>, candidates: <p> @340, <img> @1844)  -> good
  CLS 0.000  (shifts: none)  -> good
  INP < 16 ms  (slow events in the click: none)  -> good
```

(Timings vary by a few ms per run; the fixed INP is sometimes 16–24 ms.)

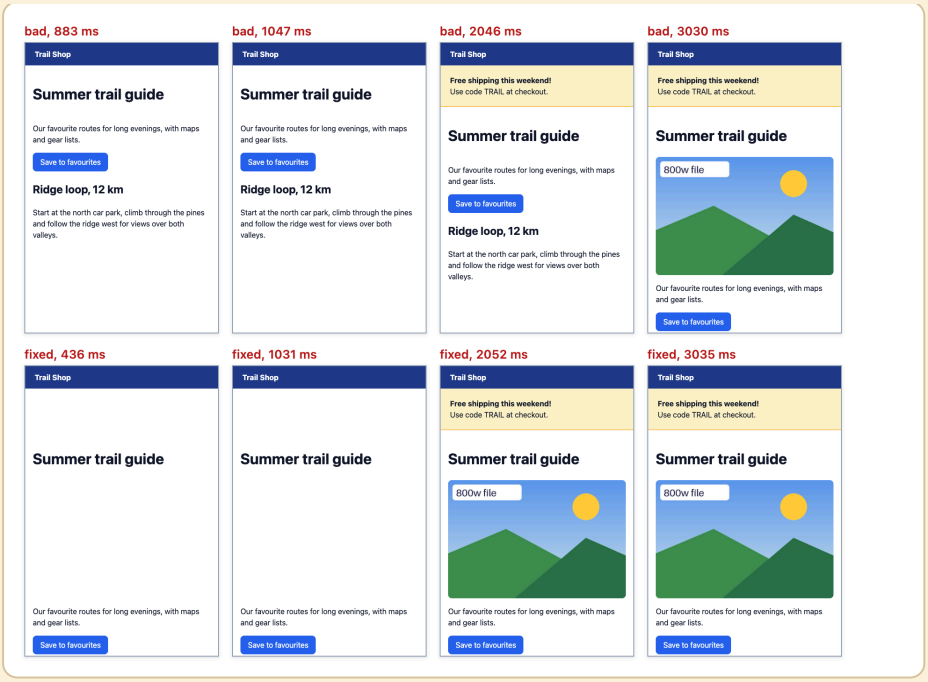


Figure 14.1 — `code/make_figures.mjs`: screenshots requested at ~0.4, 1, 2 and 3 s; labels show when Chrome delivered each frame (the bad page can't be captured before its first paint at ~880 ms).

- **LCP 2648 → 1844 ms.** The bad page's image request couldn't start until the blocking script (800 ms) ran and injected it — ~850 ms **resource load delay** before an 1800 ms download. In the fixed page the **preload scanner** (look-ahead parser) finds the `<img>` at once, so LCP ≈ download time. The remaining 1.8 s is **load duration** — next fix: a smaller image (`srcset`, AVIF/WebP, §4.3).
- **CLS 0.195 → 0.** Two shifts 0.8 s apart fell in one session window (0.069 + 0.126). `width`/`height` give the image an `aspect-ratio` (its box is reserved, so loading moves nothing) and `min-height` holds the banner slot.
- **INP 304 ms → under 16 ms.** All three click events are one interaction, and the 300 ms loop delayed the next paint for all of them. After the fix feedback paints immediately; the same work runs in chunks between frames.

> **Two measurement gotchas.** (1) An early fixed page still shifted 15.7 px because its `height` said 500 for an 800×**533** image — wrong dimensions are almost as bad as none. (2) Headless Chrome reported no Event Timing entries for a button outside its real (smaller) window, so `code/serve.mjs` uses `--window-size=1200,1400`. Lab tooling has quirks — cross-check with field data.

Field data vs lab data

	Field (real users, "RUM")	Lab (synthetic)
Sources	Chrome UX Report (CrUX, 28-day rolling, public), Search Console's Core Web Vitals report, your own RUM with the `web-vitals` library	Lighthouse (in Chrome DevTools), PageSpeed Insights' lab section, WebPageTest, DevTools Performance panel, scripts like this chapter's
Strength	What users actually experience; what Google uses	Repeatable, debuggable, before release
INP	Measured	Needs a scripted interaction; Lighthouse's navigation mode reports **Total Blocking Time (TBT)** as a proxy

API support (web-features, September 2026): the LCP and Event Timing APIs are **Baseline newly available since December 2025** (Safari 26.2 added them). The **Layout Instability API** behind CLS is **Chromium-only**, so CLS field data comes from Chrome users. The metrics apply to all users; only measurement coverage differs.

In the interview

> **30-second answer:** "Core Web Vitals are LCP for loading — good is ≤ 2.5 s; INP for responsiveness — ≤ 200 ms; CLS for visual stability — ≤ 0.1, all at the 75th percentile of real visits. INP replaced FID in March 2024. I measure field data with CrUX or web-vitals and debug in the lab with Lighthouse and

> DevTools. Typical fixes: make the LCP image discoverable and high-priority, break up long tasks and yield to the main thread, and reserve space for images, ads and late content."

- *Why can't Lighthouse show INP on page load?* → INP needs real interactions; lab tools use TBT as a proxy or a scripted user flow.
- *Is a layout shift after a click counted?* → Not within 500 ms of the input (`hadRecentInput`).
- *Does a font swap count as CLS?* → Yes, if text reflows and moves content; match fallback metrics (`size-adjust`) or use `font-display: optional`.

14.4 Fixing each metric (and the other basics)

LCP checklist

1. **Find the LCP element** (DevTools Performance panel or `largest-contentful-paint` entries) and its four sub-parts (TTFB, load delay, load duration, render delay).
2. **TTFB**: caching (CDN, HTTP cache headers), no redirects (each costs a round trip), server-side rendering or static generation (HTML built at deploy time) for content pages.
3. **Load delay**: image in the HTML (not CSS/JS); `fetchpriority="high"`; never lazy; preload if it must be referenced from CSS.
4. **Load duration**: right-sized responsive images (`srcset`/`sizes`), AVIF/WebP, a CDN.
5. **Render delay**: remove render-blocking CSS/JS (14.2); don't hide content until JS runs (A/B-testing snippets, client-only rendering).

INP checklist

1. **Input delay**: break up **long tasks** (> 50 ms of uninterrupted main-thread work) during load; defer third-party scripts (ads, chat, analytics); avoid heavy timers colliding with input.
2. **Processing**: only the visible update in the handler; `await scheduler.yield()` (or `setTimeout`) before the rest; debounce input handlers (react when typing pauses); heavy work in a Web Worker (background JS thread).
3. **Presentation**: small DOM; no layout thrashing; `content-visibility: auto` for long off-screen sections (Baseline since September 2025); animate `transform`/`opacity` (§12.7).

CLS checklist

1. Always `width` **and** `height` (correct values) on images and videos, or CSS `aspect-ratio`.
2. **Reserve space** for ads, embeds, banners and late components (`min-height`, skeleton placeholders), or overlay them (`position: fixed`) so they don't push content.
3. Never insert content **above** existing content unless the user asked.
4. Fonts: `font-display: optional`, or tune the fallback with `size-adjust`/`ascent-override`.
5. Animate with `transform`, not `top`/`height` (transforms don't cause layout shifts).
6. Stay eligible for the **back/forward cache** (bfcache) — instant, shift-free back navigations (avoid `unload` handlers and `Cache-Control: no-store` where possible).

Other performance basics

Area	Practice
Transfer	Brotli/gzip/zstd, HTTP/2 or HTTP/3, long cache lifetimes for fingerprinted (content-hashed) files (`app.3f9a1c.css`, `Cache-Control: max-age=31536000, immutable`) — §1.5
Images	Formats and responsive images (§4.2); `loading="lazy"` below the fold
CSS	Remove unused CSS; avoid huge selectors on huge DOMs; `contain`/`content-visibility` for independent regions
JavaScript	Ship less; code-split per route (load pieces on demand); `defer`; audit third parties
Budgets	Performance budgets (e.g. LCP < 2.5 s on a mid-range phone, JS < 200 KB compressed) checked in CI with Lighthouse CI

14.5 SEO basics

What it is

Search engine optimization (SEO) makes pages easy for search engines to **crawl** (fetch), **render** (Google runs JavaScript in an up-to-date Chromium), **index** (store and understand) and **rank**, and attractive to click. Content quality and relevance matter most — HTML can't make a thin page rank — but broken HTML *can* keep a good page out of the index. This is the HTML side, following Google Search Central.

The essentials, in the order a crawler meets them

Item	HTML	Notes
Crawl permission	`robots.txt` (rules for crawler bots) at the site root	A `Disallow` rule blocks **crawling**, not indexing: a blocked URL can still be indexed from links. To keep a page out, allow crawling and use `noindex`
Index permission	`<meta name="robots" content="noindex">` (or `X-Robots-Tag` header)	The classic accident: a staging `noindex` shipped to production
Title	`<title>Specific page title | Brand</title>`	The result headline (Google may rewrite it). Unique per page; no hard limit, but truncated by width — ~50–60 characters display fully
Description	`<meta name="description" content="…">`	Often the snippet; not a ranking signal; unique, specific; truncated around 150–160 characters
Canonical	`<link rel="canonical" href="https://…">`	Preferred URL among duplicates (tracking parameters like `?utm_source=`, `http`/`https`, `www`). A strong **hint**, not a directive (Google usually follows it). Absolute URLs recommended
Language	`<html lang="en">`; `<link rel="alternate" hreflang="de" href="…">`	`hreflang` links translations (include a self-reference and optionally `x-default`)
Mobile	`<meta name="viewport" content="width=device-width, initial-scale=1">`	Google indexes the **mobile** version (mobile-first indexing)
Keywords meta	`<meta name="keywords">`	**Ignored** by Google Search
Social previews	`og:title`, `og:image`, … (Open Graph)	Link previews in social apps; not a Google ranking signal
Sitemap	`sitemap.xml` (XML list of the site's URLs), listed in robots.txt / Search Console	Helps discovery of new or deep pages

Semantic HTML is SEO

Search engines read structure like assistive technology does (§3.1):

- **Headings** describe the outline; one clear `<h1>` is a helpful convention (Google says multiple `<h1>`s aren't a problem); don't skip levels.
- **Links** must be real `<a href>` (Google follows `href`, not `onclick`) with **descriptive text** — "Ridge loop route and map", not "click here".
- **Images**: `alt` text for image search and accessibility; descriptive file names.
- **Landmarks and** `<article>`/`<time>` identify the main content and dates.
- Content must be in the HTML or reliably rendered; content needing a click to load may never be seen.

Structured data (JSON-LD)

Structured data describes the page with the **schema.org** vocabulary (types like Article, Product, Person and their properties) in machine-readable form, making it eligible for **rich results** (breadcrumbs, product price and rating, recipe cards, article dates). Google supports JSON-LD (JSON for Linked Data), Microdata and RDFa (attribute-based), and **recommends JSON-LD**: a `<script type="application/ld+json">` block separate from the visible markup.

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Article",
  "headline": "Summer Trail Guide: 5 Evening Hikes Near Denver",
  "image": ["https://www.trailshop.example/img/hills-1200.jpg"],
  "datePublished": "2026-06-01T08:00:00-06:00",
  "dateModified": "2026-06-15T09:30:00-06:00",
  "author": [{ "@type": "Person", "name": "Maya Ortiz", "url": "https://www.trailshop.example/team/maya" }]
}
</script>
```

Rules: describe what's **visible** on the page; include each rich-result type's **required** properties (Article has none; the above are Google's recommended set); valid JSON (double quotes, no trailing commas). Structured data makes a page *eligible* — it **doesn't guarantee** rich results and isn't a direct ranking boost. Google restricts or retires types over time (FAQ and HowTo results were cut back in 2023), so check the current Search Gallery. Test with the **Rich Results Test** and **Schema Markup Validator**.

The experiment

`code/03_seo_audit.mjs` is a small jsdom on-page checker (parsing HTML like a crawler's first pass). It audits `code/03_seo_bad.html` — `div` soup with an accidental `noindex`, a relative canonical, two `<h1>`s, no `alt`, "Click here" and JSON-LD with a trailing comma — and `code/03_seo_good.html`:

```
03_seo_bad.html
  FAIL  doctype present (standards mode)
  FAIL  <html lang> = null
  FAIL  <title> 4 chars: "Home"
  FAIL  meta description missing
  FAIL  meta viewport (mobile-friendly)
  FAIL  robots: "noindex"
  FAIL  canonical: /guide (absolute URL recommended)
  NOTE  meta keywords present — ignored by Google, harmless but useless
  FAIL  one <h1> (found 2) — a clarity convention, not a Google rule
  FAIL  heading levels 1→1→4 (skips a level)
  FAIL  <main> landmark
  FAIL  images without alt: 1
  FAIL  vague link text: "Click here"
  FAIL  JSON-LD does not parse: Expected double-quoted property name in JSON at position 95 (line 2 column 95)
  score: 0/13

03_seo_good.html
  PASS  doctype present (standards mode)
  PASS  <html lang> = "en"
  PASS  <title> 60 chars: "Summer Trail Guide: 5 Evening Hikes Near Denver | Trail Shop"
  PASS  meta description 136 chars
  PASS  meta viewport (mobile-friendly)
  PASS  robots: "(none = index, follow)"
  PASS  canonical: https://www.trailshop.example/guides/summer-trails (absolute URL recommended)
  ...   (headings, <main>, alt text and link text all pass)
  PASS  JSON-LD Article: recommended properties present
  PASS  JSON-LD BreadcrumbList: recommended properties present
  score: 14/14
```

The most damaging line in the bad page is `noindex` — every other fix is wasted while it's there. The trailing comma is the second classic: the whole JSON-LD block is silently ignored.

Performance and SEO

Core Web Vitals are part of Google's page-experience signals: good vitals help, but relevance dominates — a fast page with poor content won't outrank a slower, better answer. Treat performance as a user-experience goal, SEO a side benefit.

Common mistakes

- Client-only rendering of content pages with empty initial HTML (rendering may be delayed; other crawlers and social previews may not run JS). Prefer server rendering or static generation.
- Blocking CSS/JS in `robots.txt` → Google can't render the page properly.
- The same `<title>`/description everywhere; every page canonical to the home page.
- Links built with `<span onclick>` or `<a>` without `href`.

In the interview

> **30-second answer:** "For SEO the HTML needs a unique, descriptive title and meta description, a canonical URL, `lang` and viewport, no accidental `noindex`, semantic structure — headings, landmarks, real links with descriptive text, alt text — and JSON-LD structured data using schema.org to become eligible for rich results. Content should be server-rendered or reliably renderable, and good Core Web Vitals help as part of page experience."

14.6 Interview questions

Forty-seven common front-end interview questions with short answers and links to the full explanation. Cover the answer, say yours aloud, then check.

HTML and accessibility

1. **What does `<!doctype html>` do?** → Standards mode (without it: quirks mode with legacy layout rules). §2.2
2. **Why is `<meta charset="utf-8">` first in `<head>`?** → It must be within the first 1024 bytes so the parser decodes the rest correctly. §2.3
3. **Block vs inline elements?** → Block boxes start on a new line and take the available width; inline boxes flow within text. It's CSS `display`, not the element "type". §7.4
4. **`<section>` vs `<article>` vs `<div>`?** → `<article>` is self-contained (could be syndicated on its own); `<section>` is a thematic group with a heading; `<div>` has no meaning. §3.2
5. **What are landmarks and why do they matter?** → `header`, `nav`, `main`, `aside`, `footer` (and named `form`/`section`) map to ARIA landmark roles screen-reader users jump between. §3.2
6. **What is the first rule of ARIA?** → Don't use ARIA if a native element does the job; no ARIA is better than bad ARIA. §3.6
7. **How do you write good alt text?** → Describe the image's purpose in context; `alt=""` for decorative images; never omit `alt`. §3.5
8. **`<button>` vs `<a>`?** → Links navigate (have `href`); buttons perform actions. Both are natively keyboard-accessible; a `div` with `onclick` is not. §3.7
9. **How do you label a form field accessibly?** → `<label for="id">` (or wrap the input); group radios with `fieldset`/`legend`; link errors with `aria-describedby`. §3.8, §5.7
10. **GET vs POST forms?** → GET puts fields in the URL (bookmarkable, for searches); POST sends them in the request body (changes, sensitive data). §5.2
11. **How do `srcset` and `sizes` work?** → `srcset` lists candidate files with widths; `sizes` gives the displayed width; the browser picks the smallest adequate file for the screen's density. §4.3
12. **When do you use `<picture>`?** → Art direction (different crops per breakpoint) or format fallbacks (AVIF → WebP → JPEG). §4.3
13. **What does WCAG AA require for text contrast?** → 4.5:1 for normal text, 3:1 for large text and UI components. §3.9

CSS fundamentals

14. **How is specificity calculated?** → (IDs, classes/attributes/pseudo-classes, types/pseudo-elements), compared left to right; inline styles beat selectors; `!important` changes the origin ordering. §6.4
15. **What decides which declaration wins?** → Origin and importance, context, inline style, cascade layer, specificity, (for `@scope` rules) scope proximity, then order of appearance. §6.5
16. **Pseudo-class vs pseudo-element?** → `:hover` selects an element in a state; `::before` creates/selects a part of an element. §6.3
17. **Which properties inherit?** → Mostly text properties (colour, font, line-height); box properties (margin, border, width) don't. `inherit`, `initial`, `unset`, `revert` control it. §6.6
18. **`em` vs `rem`?** → `em` is relative to the element's own font size (the parent's for `font-size`); `rem` to the root's. Use `rem` for predictable scaling. §6.7
19. **Custom properties vs Sass variables?** → Live, inherited values that can change per element and at runtime; Sass variables disappear at compile time. §6.9, §13.12
20. **`:is()` vs `:where()`?** → Same matching; `:is()` takes its most specific argument's specificity, `:where()` is zero. §13.2

21. **What does `:has()` enable?** → Selecting an element by what it contains or what follows it — a parent/previous-sibling selector. §13.3
22. **How do cascade layers help?** → They order groups of rules before specificity is considered, ending specificity wars with third-party CSS. §13.4

Box model and layout

23. **Explain the box model and `box-sizing`.** → Content + padding + border + margin; `content-box` sizes the content only, `border-box` includes padding and border. §7.1, §7.2
24. **What is margin collapsing?** → Adjacent vertical margins of blocks in normal flow combine into the larger; not in flex/grid or across a BFC boundary (Q27). §7.3
25. **Explain `position` values.** → `static` (normal), `relative` (offset, keeps its space), `absolute` (out of flow, nearest positioned ancestor), `fixed` (viewport), `sticky` (relative until a threshold, then sticks within its container). §7.6
26. **Why doesn't my `z-index` work?** → It applies only to positioned (or flex/grid) items and is compared within one stacking context; a parent with `opacity < 1`, `transform` etc. creates a new one. §7.8
27. **What is a block formatting context (BFC)?** → An independent layout region: contains floats, prevents margin collapse with children; created by `display: flow-root`, flex/grid items, `overflow` other than `visible`/`clip`, etc. §7.5
28. **What does `flex: 1` mean?** → `flex: 1 1 0%`: grow equally from a zero basis, allowed to shrink. §8.4
29. **Why does a flex item with long text overflow instead of shrinking?** → Its automatic minimum is `min-width: auto` (its content); set `min-width: 0` (a scroll container via `overflow: hidden`/`auto` also works; `overflow: clip` doesn't). §8.4
30. **How do you center something horizontally and vertically?** → `display: grid; place-items: center` (or flex with `justify-content` and `align-items: center`). §8.7
31. **Flexbox or Grid?** → Flexbox for one-dimensional content-driven layout; Grid for two-dimensional, layout-driven structure. §9.8
32. **`auto-fill` vs `auto-fit`?** → Both create as many tracks as fit; `auto-fit` collapses empty ones so items stretch. §9.3
33. **What is subgrid for?** → Letting nested items align to the parent grid's tracks (card titles aligned across cards). §9.7

Responsive, visual and motion

34. **What does the viewport meta tag do?** → Tells mobile browsers to lay out at device width instead of a ~980px virtual desktop. §10.2
35. **Mobile-first — what and why?** → Base styles for small screens, `min-width` queries add complexity; simpler CSS, faster phones. §10.4
36. **Media queries vs container queries?** → Media queries respond to the viewport; container queries to a component's container — reusable components. §10.6
37. **How does `clamp()` give fluid typography?** → `clamp(min, preferred, max)` with a viewport-relative preferred value (plus `rem` so zoom still works). §10.7
38. **How do you load a web font without invisible text?** → `font-display: swap` (or `optional`), preload the WOFF2, subset it. §11.2
39. **Which properties are cheap to animate and why?** → `transform` and `opacity`: the compositor animates them without layout or paint. §12.7
40. **How do you respect users who get motion sickness?** → Wrap non-essential motion in `@media (prefers-reduced-motion: no-preference)` or reduce it under `reduce`. §12.9

Architecture, performance and SEO

41. **BEM vs CSS Modules vs Tailwind?** → Convention (flat unique class names) vs build-time renamed classes vs generated single-purpose utilities; all fight the global namespace. §13.11
42. **Dark mode with a user toggle?** → Semantic tokens with `light-dark()`, `color-scheme: light dark`, a `data-theme` override pinning `color-scheme`, saved in localStorage and applied before first paint. §13.14
43. **What happens when you type a URL and press Enter?** → DNS, TCP/TLS, HTTP request/response, parsing, DOM/CSSOM, render tree, layout, paint, composite. §1.6
44. **What are render-blocking resources?** → Stylesheets (block paint) and synchronous scripts (block parsing); fix with critical CSS, `defer`/`async`. §14.2
45. **Core Web Vitals and their thresholds?** → LCP ≤ 2.5 s, INP ≤ 200 ms, CLS ≤ 0.1 at p75. §14.3

46. **How do you prevent layout shift from images?** → `width`/`height` attributes (correct ratio) or `aspect-ratio`. §14.4
47. **What HTML matters for SEO?** → Title, description, canonical, lang, viewport, no stray `noindex`, headings, real links, alt, JSON-LD. §14.5

14.7 Cheat sheets

Selectors

Selector	Matches	Specificity
`*`	every element	(0,0,0)
`p`	type	(0,0,1)
`.card`	class	(0,1,0)
`#main`	id	(1,0,0)
`[type="email"]`, `[href^="https"]`, `[href$=".pdf"]`, `[class*="icon"]`, `[lang|="en"]`	attribute: equals / starts / ends / contains / language prefix	(0,1,0)
`A B`	descendant	sum
`A > B`	child	sum
`A + B`	next sibling	sum
`A ~ B`	any later sibling	sum
`:hover :focus :focus-visible :focus-within :active :visited`	user action state	(0,1,0) each
`:checked :disabled :required :invalid :user-invalid :placeholder-shown`	form state	(0,1,0)
`:first-child :last-child :nth-child(2n+1) :nth-of-type(3) :only-child :empty`	structure	(0,1,0)
`:nth-child(2 of .item)`	nth among matches	(0,1,0) + argument
`:not(X) :is(X) :has(X)`	logic	most specific argument
`:where(X)`	logic	(0,0,0)
`::before ::after ::marker ::placeholder ::selection ::first-line ::backdrop`	parts	(0,0,1)
`&` (nesting)	parent rule's selector	as `:is(parent)`

Details: §6.2, §6.3, §13.1–13.3.

Specificity and the cascade

- Score = (A = IDs, B = classes + attributes + pseudo-classes, C = types + pseudo-elements); compare A, then B, then C. `*` and combinators add nothing.
- The cascade, strongest criterion first (UA = user agent, i.e. the browser's own default styles): **origin & `!important`** (transition > `!important` UA > `!important` user > `!important` author > animation > author > user > UA) → **context** (shadow DOM) → **inline `style`** → **layers** (later wins; unlayered last; reversed for `!important`) → **specificity** → **scope proximity** (`@scope`) → **order of appearance**.
- Keywords: `inherit` (parent's value), `initial` (spec default — the value the CSS specification defines), `unset` (inherit if inherited property, else initial), `revert` (browser/user value), `revert-layer` (previous layers' value).

Worked examples: `#nav .item a` = (1,1,1); `ul li:nth-child(2)` = (0,1,2); `a:not(.btn, #x)` = (1,0,1); `:where(#x) .y` = (0,1,0). §6.4

Box model

```
margin  →  border  →  padding  →  content
content-box:  width = content          total = width + padding + border (+ margin)
border-box:   width = content + padding + border
```

- Vertical margins between blocks in normal flow **collapse** (largest wins); not in flex/grid, not through padding/border, not inside a BFC. §7.3
- Percentage padding/margin are relative to the **containing block's inline size** — its width in horizontal text, even for top/bottom.
- `display`: `block`, `inline`, `inline-block`, `flex`, `inline-flex`, `grid`, `flow-root` (new BFC), `contents` (box removed, children stay — careful with accessibility), `none` (removed from layout and accessibility tree).
- Positioning: `relative` → offset; `absolute` → nearest positioned ancestor; `fixed` → viewport (or a transformed ancestor); `sticky` → scroll threshold. `inset: 0` = `top/right/bottom/left: 0`.

Flexbox

Container	Values (default first)
`display`	`flex` | `inline-flex`
`flex-direction`	`row` | `row-reverse` | `column` | `column-reverse`
`flex-wrap`	`nowrap` | `wrap` | `wrap-reverse`
`flex-flow`	shorthand: direction + wrap
`justify-content` (main axis)	`flex-start` / `normal` | `center` | `flex-end` | `space-between` | `space-around` | `space-evenly`
`align-items` (cross axis, per line)	`stretch` / `normal` | `flex-start` | `center` | `flex-end` | `baseline`
`align-content` (lines, when wrapping)	`normal` | `flex-start` | `center` | `space-between` | `stretch` …
`gap`, `row-gap`, `column-gap`	spacing between items

Item	Values
`flex-grow`	`0` — share of free space
`flex-shrink`	`1` — share of overflow to absorb (weighted by basis)
`flex-basis`	`auto` — starting size before grow/shrink
`flex`	`initial` = `0 1 auto`; `auto` = `1 1 auto`; `none` = `0 0 auto`; `1` = `1 1 0%`
`align-self`	overrides `align-items` for one item
`order`	`0` — visual order only (keyboard/reading order unchanged)
`min-width: 0`	lets items shrink below their content (`overflow: clip` doesn't; `hidden`/`auto` do)

Chapter 8

Grid

Container	Example
`grid-template-columns` / `-rows`	`200px 1fr 2fr`, `repeat(3, 1fr)`, `repeat(auto-fit, minmax(16rem, 1fr))`
`grid-template-areas`	`"head head" "side main" "foot foot"`
`grid-auto-rows` / `-columns`	size of implicit tracks: `minmax(100px, auto)`
`grid-auto-flow`	`row` | `column` | `dense`
`gap`	`1rem 2rem` (row column)
`justify-items` / `align-items` / `place-items`	align items inside their cells
`justify-content` / `align-content`	align the whole grid inside the container

Item	Example
`grid-column` / `grid-row`	`1 / 3`, `2 / span 2`, `1 / -1` (full width)
`grid-area`	`main` (named area) or `1 / 1 / 3 / 3`
`justify-self` / `align-self` / `place-self`	per-item alignment
`grid-template-columns: subgrid`	reuse the parent's tracks

Track sizes: `px`/`rem`, `%`, `fr` (share of free space), `auto`, `min-content`, `max-content`, `fit-content(300px)`, `minmax(min, max)`. Chapter 9

Units

Unit	Relative to	Typical use
`px`	CSS pixel (1/96 in; scales with zoom and device pixel ratio)	borders, small fixed details
`rem`	root font size (16px default)	font sizes, spacing
`em`	own font size (parent's for `font-size`)	padding that scales with text
`%`	containing block (varies by property)	fluid widths
`ch` / `ex` / `lh`	width of "0" / x-height / line height	`max-width: 65ch` for readable lines
`vw` / `vh`	1% of viewport width / height	full-screen sections
`svh` / `lvh` / `dvh`	small / large / dynamic viewport height (mobile toolbars)	`min-height: 100dvh`
`vi` / `vb`	viewport inline / block size	writing-mode-aware
`cqi` / `cqw` / `cqb`	container query inline / width / block	component-relative type
`fr`	free space in a grid	grid tracks
`deg` `turn` `ms` `s`	angles, time	transforms, transitions

§6.7, §10.2

Responsive patterns

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

```css
/* mobile-first breakpoints (range syntax) */
.layout { display: grid; gap: 1rem; }
@media (width >= 48rem) { .layout { grid-template-columns: 16rem 1fr; } }

/* intrinsic grid, no breakpoints */
.cards { display: grid; grid-template-columns: repeat(auto-fit, minmax(min(16rem, 100%), 1fr)); gap: 1rem; }

/* component responds to its container */
.card-wrap { container-type: inline-size; }
@container (inline-size >= 30rem) { .card { display: flex; } }

/* fluid type and space */
h1 { font-size: clamp(1.75rem, 1.2rem + 2.5vw, 3rem); }

/* user preferences */
@media (prefers-color-scheme: dark) { /* or use light-dark() tokens */ }
@media (prefers-reduced-motion: reduce) { * { animation: none; } }

/* images */
img { max-width: 100%; height: auto; }
```

```html
<img src="a-800.jpg" srcset="a-400.jpg 400w, a-800.jpg 800w, a-1600.jpg 1600w"
     sizes="(width >= 48rem) 50vw, 100vw" width="800" height="533" alt="…">
```

Chapter 10

HTML head starter

```html
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Page title | Site</title>
  <meta name="description" content="What this page offers, in one or two sentences.">
  <link rel="canonical" href="https://example.com/page">
  <meta name="color-scheme" content="light dark">
  <link rel="preload" href="/fonts/inter.woff2" as="font" type="font/woff2" crossorigin>
  <link rel="stylesheet" href="/css/site.3f9a1c.css">
  <script src="/js/app.8b2e0d.js" defer></script>
  <link rel="icon" href="/favicon.svg" type="image/svg+xml">
</head>
```

14.8 Glossary

Alphabetical; each term links to the section that explains it.

- `!important` — Flag that moves a declaration to a higher-priority "important" origin level. §6.5
- `@container` — Container query rule. §10.6
- `@font-face` — Declares a web font and its files. §11.1
- `@import` — Loads another stylesheet (can add a layer). §6.1
- `@keyframes` — Named animation timeline. §12.5
- `@layer` — Cascade layer. §13.4
- `@media` — Media query rule. §10.3
- `@property` — Registers a typed custom property. §13.14
- `@scope` — Limits rules to a subtree, optionally with a lower bound. §13.5
- `@starting-style` — Style to transition *from* when an element first renders. §12.8
- `@supports` — Feature query. §6.1
- **Absolute positioning** — Out of flow, placed relative to the nearest positioned ancestor. §7.6
- **Absolute URL** — URL with scheme and host. §1.2
- **Accessibility tree** — The browser's tree of roles, names and states for assistive tech. §3.4
- **Accessible name** — What a screen reader announces for an element. §3.4
- `alt` text — Text alternative for an image. §3.5
- **ARIA** — Attributes that add roles/states/properties for assistive technology. §3.6

- `aria-describedby` — Links an element to its description (e.g. an error). §5.7
- `aspect-ratio` — Preferred width/height ratio of a box; `<img width height>` sets it automatically. §4.1
- `async` (script) — Download in parallel, run as soon as available. §14.2
- **Attribute** — Name/value pair on an element. §2.1
- **Attribute selector** — `[name="value"]` and variants. §6.2
- `autocomplete` — Hints the browser what data a field wants. §5.6
- **AVIF** — Modern image format with strong compression. §4.2
- **Baseline** — web-features status: works in all major browsers (newly / widely available). §1.1
- **BEM** — Block__Element--Modifier naming convention. §13.7
- **bfcache** — Back/forward cache: instant back navigations. §14.4
- **Block axis** — Direction in which blocks/lines stack. §13.6
- **Block formatting context (BFC)** — Independent layout region for block boxes. §7.5
- **Block-level element** — Starts on a new line and fills the width. §7.4
- `border-box` — `box-sizing` in which width includes padding and border. §7.2
- **Box model** — Content, padding, border, margin. §7.1
- **Breakpoint** — Width at which a layout changes. §10.3
- **Canonical URL** — Preferred URL among duplicates (`rel="canonical"`). §14.5
- **Canvas** — Scriptable bitmap drawing surface. §4.9
- **Cascade** — Algorithm that picks the winning declaration. §6.5
- **Cascade layer** — Named, ordered group of rules. §13.4
- **Character reference** — `&`, `©` — escapes in HTML. §2.9
- `clamp()` — Value clamped between a min and max. §10.7
- **CLS** — Cumulative Layout Shift — visual stability metric. §14.3
- `color-mix()` — Mixes two colours in a colour space. §11.4
- `color-scheme` — Declares supported light/dark schemes. §10.5
- **Combinator** — Relationship between selectors (space, `>`, `+`, `~`). §6.2
- **Composite** — Final step: combining layers into the screen image. §1.7
- **Compositor-only properties** — `transform`, `opacity` — animate without layout/paint. §12.7
- **Computed value** — Value after cascade and inheritance, before layout. §6.10
- **Constraint Validation API** — JS API behind built-in form validation. §5.5
- **Container query** — Styles based on a container's size. §10.6
- **Containing block** — Box that percentages and positions are resolved against. §7.6
- `content-visibility` — Skips rendering off-screen content. §14.4
- **Core Web Vitals** — LCP, INP and CLS. §14.3
- **Critical CSS** — CSS needed for the first screen, often inlined. §14.2
- **Critical rendering path** — Work required before the first render. §14.1
- **CrUX** — Chrome UX Report — public field data. §14.3
- **CSS Modules** — Build-time locally scoped class names. §13.9
- **CSS-in-JS** — Styles authored in JavaScript. §13.10
- **CSSOM** — CSS Object Model — parsed stylesheets. §1.7
- `cubic-bezier()` — Custom easing curve. §12.2
- **Custom property** — `--name` value that inherits and can change at runtime. §6.9
- `defer` (script) — Download in parallel, run in order after parsing. §14.2
- **Design token** — Named design value (colour, space) stored as a variable. §13.14
- **Device pixel ratio** — Physical pixels per CSS pixel. §4.3
- **DevTools** — Browser developer tools. §1.8
- `<dialog>` — Native modal/non-modal dialog element (focus handling built in). §3.7
- `display` — Outer/inner display type of a box. §7.4
- **DNS** — Resolves domain names to IP addresses. §1.3
- **Doctype** — `<!doctype html>` — standards mode. §2.2
- **DOM** — Document Object Model — tree of the parsed HTML. §1.7
- **Donut scope** — `@scope (root) to (limit)` excluding an inner region. §13.5
- `dvh` — Dynamic viewport height unit. §10.2
- **Easing** — Speed curve of an animation (timing function). §12.2

- **Element** — Start tag, content, end tag. §2.1
- `em` — Unit relative to font size. §6.7
- **Event Timing API** — Reports input event latencies (INP). §14.3
- **FCP** — First Contentful Paint. §14.1
- `fetchpriority` — Hint to raise/lower a resource's priority. §4.4
- **Field data** — Metrics from real users. §14.3
- `<fieldset>` / `<legend>` — Group of related controls and its caption. §5.4
- `<figure>` / `<figcaption>` — Self-contained content with caption. §4.5
- `animation-fill-mode` — Whether keyframe styles apply before the animation starts / after it ends. §12.5
- `flex` shorthand — grow, shrink, basis. §8.4
- `flex-basis` — Item's starting main size. §8.3
- **Flex container / item** — Element with `display: flex` / its children. §8.1
- **Float** — Legacy layout: box shifted left/right, text wraps. §7.9
- **FLIP** — First-Last-Invert-Play animation technique. §12.7
- **Focus indicator** — Visible outline showing keyboard focus. §3.7
- `:focus-visible` — Focus that should show an indicator (keyboard). §3.7
- `font-display` — How text renders while a web font loads. §11.2
- **FOUC** — Flash of unstyled content. §14.1
- `fr` — Fraction of free space in a grid. §9.2
- `gap` — Space between flex/grid items. §8.6
- **Gradient** — `linear-/radial-/conic-gradient()` images. §11.5
- **Grid area** — Named or line-bounded region. §9.4
- **Grid line / track / cell** — Dividing lines / rows-columns / single slot. §9.1
- `:has()` — Relational ("parent") pseudo-class. §13.3
- **Heading levels** — `h1`–`h6` outline. §3.3
- `hreflang` — Links language versions of a page. §14.5
- **HSL / OKLCH** — Colour notations (hue-based, perceptual). §6.8
- **HTTP** — Request/response protocol of the web. §1.5
- **HTTP/2, HTTP/3** — Multiplexed HTTP versions (HTTP/3 over QUIC). §1.4
- `<iframe>` — Embedded browsing context. §4.7
- **Implicit grid** — Tracks created automatically for extra items. §9.5
- **INP** — Interaction to Next Paint — responsiveness metric. §14.3
- **Inheritance** — Children taking a parent's computed value. §6.6
- **Inline axis** — Direction text runs within a line. §13.6
- **Inline element** — Flows within a line of text. §7.4
- **Input types** — `email`, `number`, `date`, …. §5.3
- **Intrinsic layout** — Responsive without breakpoints (`auto-fit`, `min()`). §10.9
- `:is()` — Matches any selector in a list (max specificity). §13.2
- **JSON-LD** — JSON format for structured data. §14.5
- **Keyboard trap** — Focus that can't leave a component (WCAG failure). §3.7
- **Lab data** — Metrics from synthetic tests. §14.3
- **Landmark** — Region role (`banner`, `navigation`, `main`, …). §3.2
- **Layout (reflow)** — Computing box sizes and positions. §1.7
- **Layout shift** — Unexpected movement of visible content. §14.3
- **Lazy loading** — `loading="lazy"`: fetch when near the viewport. §4.4
- **LCP** — Largest Contentful Paint — loading metric. §14.3
- `light-dark()` — Picks a light or dark value by colour scheme. §13.14
- **Lighthouse** — Lab auditing tool (performance, a11y, SEO). §14.3
- **Link (`<a>`)** — Hyperlink; `href`, `target`, `rel`. §2.8
- **Lists** — `ul`, `ol`, `dl`. §2.7
- **Logical properties** — Flow-relative properties (`margin-inline-start`). §13.6
- **Long task** — Main-thread task over 50 ms. §14.4
- **Margin collapsing** — Adjacent vertical margins combine. §7.3
- **Media query** — Condition on the device/viewport/preferences. §10.3

- **Meta description** — Page summary for search snippets. §14.5
- `min-content` / `max-content` — Intrinsic sizes. §9.2
- `minmax()` — Track size range. §9.2
- **Mobile-first** — Base styles for small screens, enhance upward. §10.4
- **Nesting (CSS)** — Rules inside rules; `&`. §13.1
- `noindex` — Robots directive keeping a page out of the index. §14.5
- **normalize.css** — Stylesheet making browser defaults consistent. §13.13
- **Normal flow** — Default layout of block and inline boxes. §7.5
- `object-fit` — How replaced content fills its box. §11.9
- **Open Graph** — `og:` meta tags for social previews. §14.5
- `order` — Visual order of flex/grid items. §8.2
- **Origin (cascade)** — User-agent, user or author stylesheet. §6.5
- **Outline** — Line outside the border that doesn't take space. §11.6
- `overflow` — What happens to content that doesn't fit. §7.7
- **Paint** — Filling in pixels for boxes. §1.7
- `PerformanceObserver` — API to receive performance entries. §14.3
- `<picture>` — Art direction and format fallbacks. §4.3
- `place-items` — Shorthand for `align-items` + `justify-items`. §9.6
- **Positioned element** — `position` other than `static`. §7.6
- **PostCSS** — Plugin-based CSS transformer. §13.12
- `preconnect` — Early connection to another origin. §14.2
- `prefers-color-scheme` — Media feature for light/dark preference. §10.5
- `prefers-reduced-motion` — Media feature for reduced motion. §12.9
- `preload` — Fetch a known-critical resource early. §14.2
- **Preload scanner** — Look-ahead parser that finds resources early. §14.3
- **Preprocessor** — Language compiled to CSS (Sass, Less). §13.12
- **Pseudo-class** — `:state` selector (`:hover`, `:checked`). §6.3
- **Pseudo-element** — Selects or creates a part of an element (`::before`, `::marker`). §6.3
- **Quirks mode** — Legacy rendering mode without a doctype. §2.2
- `rel` — Link relationship (`noopener`, `canonical`, `preload`). §2.8
- **Relative colour syntax** — Derive a colour from another's channels: `oklch(from var(--c) l c calc(h + 180))`. §11.4
- `rem` — Unit relative to root font size. §6.7
- **Render tree** — Styled boxes that will be displayed. §1.7
- **Render-blocking** — Delays the first paint (stylesheets). §14.2
- `repeat()` — Repeats track definitions. §9.3
- **Reset (CSS)** — Rules neutralising browser defaults. §13.13
- **Responsive design** — Layouts that adapt to any screen and setting. §10.1
- `revert-layer` — Roll back to earlier layers' value. §13.4
- **Rich results** — Enhanced search results from structured data. §14.5
- `robots.txt` — Crawl rules for bots. §14.5
- **Role** — ARIA/semantic purpose of an element. §3.6
- **Sass** — The most used CSS preprocessor. §13.12
- `sandbox` — Restricts an iframe's capabilities. §4.7
- **schema.org** — Vocabulary for structured data. §14.5
- **Screen reader** — Software that speaks or brailles the UI. §3.1
- `<search>` — Landmark element for search/filter forms (widely available since April 2026). §3.2
- **Scroll-driven animation** — Animation progress linked to scroll. §12.11
- **Selector** — Pattern choosing elements to style. §6.2
- **Semantic HTML** — Elements chosen for meaning. §3.1
- **SEO** — Search engine optimization. §14.5
- **Session window (CLS)** — Group of layout shifts < 1 s apart, ≤ 5 s. §14.3
- **Shorthand property** — Sets several longhands at once (`margin`, `flex`, `inset`). §7.1
- **Sitemap** — XML list of URLs for crawlers. §14.5

- `sizes` — Displayed image width per condition. §4.3
- **Specificity** — Weight of a selector (A,B,C). §6.4
- `srcset` — Candidate image files. §4.3
- **Stacking context** — Group painted together along z. §7.8
- **Sticky positioning** — Relative until a scroll threshold, then fixed within its container. §7.6
- **Structured data** — Machine-readable page description. §14.5
- **Style query** — `@container style(--x: y)` — query a container's custom property (Baseline since May 2026). §10.6
- **Subgrid** — Nested grid reusing parent tracks. §9.7
- **SVG** — Vector image format written in XML. §4.8
- **Table semantics** — `caption`, `thead`, `th scope`. §5.1
- **Tailwind CSS** — Utility-first CSS framework. §13.8
- **TBT** — Total Blocking Time — lab proxy for responsiveness. §14.3
- **Text-level semantics** — `em`, `strong`, `code`, `abbr`, `time`, …. §2.6
- **TLS** — Encryption layer of HTTPS. §1.4
- `<track>` — Captions/subtitles for media. §4.6
- **Transform** — Moves/rotates/scales the painted box. §12.3
- **Transition** — Animates a property change. §12.1
- **TTFB** — Time To First Byte. §14.1
- **URL** — Address: scheme, host, path, query, fragment. §1.2
- `:user-invalid` — Invalid after the user interacted. §5.5
- **User-agent stylesheet** — The browser's default CSS. §13.13
- **Utility-first CSS** — Composing single-purpose classes. §13.8
- **Validation (HTML)** — Checking markup against the spec. §2.10
- **View transition** — Animated morph between DOM states. §12.10
- **Viewport** — Visible area of the page. §10.2
- **Viewport meta tag** — Sets the layout viewport on mobile. §10.2
- **WAAPI** — Web Animations API. §12.6
- **WCAG** — Web Content Accessibility Guidelines (2.2). §3.9
- **Web font** — Font downloaded with the page (`@font-face`, WOFF2). §11.1
- **WebP** — Image format, typically smaller than JPEG/PNG at similar quality. §4.2
- `:where()` — Like `:is()` with zero specificity. §13.2
- `will-change` — Hint that a property will animate. §12.7
- **Writing mode** — Horizontal or vertical text flow. §13.6
- `z-index` — Stacking order of positioned boxes. §7.8

Chapter 14 TL;DR

- **Critical rendering path**: HTML → DOM, CSS → CSSOM, render tree, layout, paint, composite. CSS is **render-blocking**; classic scripts are **parser-blocking** and also wait for pending CSS.
- **Render-blocking fixes** (measured, CSS 800 ms / JS 600 ms): all-blocking FCP 848 ms; `defer` only → 844 ms (CSS was the bottleneck); inline critical CSS + non-blocking stylesheet + `defer` → **40 ms**.
- **Core Web Vitals** (p75 of real visits): **LCP ≤ 2.5 s**, **INP ≤ 200 ms** (replaced FID in March 2024), **CLS ≤ 0.1**. LCP = last largest paint before input; INP = worst interaction's input delay + processing + presentation; CLS = largest session window of unexpected shifts.
- **Measured**: bad page LCP 2648 ms / CLS 0.195 / INP 304 ms → fixed page 1844 ms / 0 / < 16 ms, by putting the hero `<img>` in the HTML with correct `width`/`height` and `fetchpriority="high"`, reserving the banner's space, deferring the script, and yielding in the click handler.
- **Fix lists**: LCP (TTFB, discovery, size, render delay); INP (no long tasks, yield, small DOM); CLS (dimensions, reserved space, nothing inserted above, font metrics).
- **SEO**: title, meta description, canonical (hint, absolute), `lang`, viewport, no accidental `noindex`, `robots.txt` blocks crawling not indexing, semantic headings/links/alt, **JSON-LD** schema.org structured data (eligibility, not a guarantee), keywords meta ignored.

- **Revision**: 47 interview questions (§14.6), cheat sheets for selectors, specificity, box model, flexbox, grid, units and responsive patterns (§14.7), and a 212-term glossary (§14.8).

That's the end of the HTML & CSS Handbook. Next steps: the JavaScript Handbook for behaviour and the DOM (start with its Chapter 1, or jump to Chapter 13 — The DOM and Events if you already know the language), then the React Handbook for component-based UIs. Back to the front page of this book.

Chapter 15 — Interview Question Bank

Every HTML and CSS question that keeps showing up in front-end interviews, in one place: the classic h5bp list, the GreatFrontEnd and InterviewBit quizzes, popular CSS question guides, and the "build this layout" tasks from machine-coding rounds (timed rounds where you build a small UI live). Chapter 14 §14.6 already has 47 short answers; this bank includes all of them, links back to them, and adds the many questions that list does not have.

How to use this bank

- **Wording.** Each question is phrased the way interviewers usually phrase it, so you recognise it instantly.
- **★ = asked very often.** The question appears in several of the source lists *and* in reported interviews. If you are short on time, drill only the ★ questions (about a third of the bank).
- **Answers.** Each answer is a short version you can say out loud in 20–40 seconds: 2–6 sentences, or a small code snippet and a sentence or two. Terms are defined the first time they appear.
- **→ *More:*** links take you to the full explanation (with real browser output) in this book. A question with no link is not covered elsewhere in the book, so its answer here is complete on its own. "§14.6 Qn" means the question is also in Chapter 14's short list.
- **How to drill.** Cover the answer, say yours out loud (as if to an interviewer), then uncover and compare. Mark the ones you missed, follow the link, and come back two days later. For the "build this" questions, type the code from memory into a blank `.html` file and open it in a browser.
- **Versions.** Browser-support statements match the rest of the book (checked September 2026, current browsers Chrome 154, Safari 26, Firefox 15x). "Baseline" is the label for a feature that works in all major browsers (Chrome, Edge, Firefox, Safari); "newly available" means it only recently reached the last of them.

HTML documents, the `<head>` and elements

Q1. ★ What does a doctype do?

`<!doctype html>` is the first line of the file. It tells the browser to render the page in **standards mode** (follow the current CSS specifications). Without it, browsers fall back to **quirks mode**, which imitates old browser bugs — for example, tables do not inherit the page's font size, percentage heights and `line-height` are calculated differently, and sloppy CSS such as hex colours without `#` or lengths without units is accepted. (The old IE box-model quirk — `width` including padding and border — survives in modern browsers only for a few special cases such as table cells' height.) It is not an HTML element and is case-insensitive. HTML5's doctype is deliberately short because it exists only to trigger standards mode.

Follow-up: Can anything come before the doctype? Only a byte-order mark (BOM, an invisible marker some editors add) and comments/whitespace are tolerated; a stray visible character (any text) above it flips the page into quirks mode, and in old Internet Explorer even a comment or XML prolog before it did.

→ *More:* §2.2 Document structure and the doctype, §14.6 Q1

Q2. What are the different kinds of doctypes, and which one should you use?

HTML 4.01 and XHTML 1.0 had Strict, Transitional and Frameset doctypes that pointed to a DTD (Document Type Definition, a formal grammar file). Some of the long ones trigger **almost-standards mode** (standards mode except that images in table cells keep the old line-height gap). Today you always write `<!doctype html>`: it gives full standards mode in every browser.

→ *More:* §2.2 Document structure and the doctype

Q3. What is the difference between standards mode and quirks mode — how can you tell which one a page is in?

Standards mode follows the specs; quirks mode emulates old browser behaviour (box sizing, table and inline layout differences). Run `document.compatMode` in the console: `"CSS1Compat"` is standards mode, `"BackCompat"` is quirks mode. A missing or malformed doctype, or anything before it, can drop the page into quirks mode.

→ *More:* §2.2 Document structure and the doctype

Q4. What is the purpose of the `<head>` and `<body>` elements?

`<head>` holds **metadata** — information about the page that is not shown in it: the character encoding, `<title>`, viewport settings, links to stylesheets, scripts, icons, and SEO/social tags. `<body>` holds everything the

user sees and interacts with. The browser creates both even if you leave the tags out, but writing them makes the structure clear.

→ *More:* §2.2 Document structure and the doctype, §2.3 The `<head>`: metadata that matters

Q5. ⋆ Why should `<meta charset="utf-8">` be the first thing in `<head>`?

The browser must know the **character encoding** (how bytes map to characters) before it decodes the text. The declaration has to appear within the first 1024 bytes; if it comes later, the browser may guess, and characters like "é" or "₹" can turn into garbage (mojibake), or the parser may have to restart. UTF-8 covers every language and is the only encoding you should use.

Follow-up: What if the server's `Content-Type` header says a different charset? The HTTP header wins over the `<meta>` tag, so fix the server configuration as well.

→ *More:* §2.3 The `<head>`: metadata that matters, §14.6 Q2

Q6. ⋆ Which tags belong in a minimal, production-ready `<head>`?

```
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Page title – Site</title>
<meta name="description" content="One-sentence summary for search results">
<link rel="canonical" href="https://example.com/page">
<link rel="icon" href="/favicon.svg">
<link rel="stylesheet" href="/app.css">
<script src="/app.js" defer></script>
```

Plus `lang` on `<html>`, and optionally Open Graph tags (`og:title`, `og:image`) for link previews.

→ *More:* §2.3 The `<head>`: metadata that matters, §14.7 Cheat sheets

Q7. What does `<meta name="viewport" content="width=device-width, initial-scale=1">` do?

Mobile browsers otherwise lay a page out on a virtual screen about 980 px wide and shrink it, so your media queries never fire and text is tiny. This tag sets the layout width to the device's width in CSS pixels and the starting zoom to 100%. Never add `user-scalable=no` or `maximum-scale=1`: blocking zoom fails WCAG.

→ *More:* §10.2 The viewport meta tag and viewport units, §14.6 Q34

Q8. ⋆ What are void (empty) elements in HTML?

Elements that cannot have content and so have no closing tag: `<img>`, `<br>`, `<hr>`, `<input>`, `<meta>`, `<link>`, `<source>`, `<track>`, `<wbr>`, `<area>`, `<base>`, `<col>`, `<embed>`. The trailing slash in `<br />` is allowed but ignored in HTML. Writing `<div />` does **not** close a div — the parser treats it as an opening tag.

Follow-up: Can a custom element be void? No — `<my-icon />` is parsed as an opening tag only; custom elements always need a closing tag.

→ *More:* §2.1 Elements, tags and attributes

Q9. Are HTML tags and elements the same thing?

No. A **tag** is the text markup (`<p>`, `</p>`); an **element** is the whole thing — start tag, content, end tag — and becomes a node (an object) in the DOM (Document Object Model, the in-memory tree the browser builds from the HTML). Some elements have optional tags (the browser inserts `<tbody>` or `</li>` for you), so an element can exist without its tags being written.

→ *More:* §2.1 Elements, tags and attributes

Q10. ⋆ What are `data-*` attributes good for, and when should you avoid them?

They store custom data on an element that has no standard attribute: `<li data-id="42" data-status="done">`. JavaScript reads them via `el.dataset.id` (dashes become camelCase: `data-user-id` → `dataset.userId`), and CSS can select on them (`[data-status="done"]`). Avoid them for anything that has a real attribute or ARIA state (use `disabled`, `aria-expanded`), for large data (keep it in JS), and for secrets — they are visible in the page source.

Follow-up: How do you style based on a data attribute? `[data-state="open"] .panel { display: block; }` — but for states that assistive technology should know about, prefer the ARIA attribute (`[aria-expanded="true"]`) and style that, so the visual and announced states can't drift apart.

→ *More:* §2.1 Elements, tags and attributes

Q11. What is the difference between the `id` and `class` attributes?

`id` must be unique in the document; it is a target for fragment links (`#section`), `<label for>`, ARIA references and `getElementById`. `class` can be shared by many elements and an element can have many classes; it

is the normal hook for CSS. In CSS an ID selector has much higher specificity than a class, which is one reason style guides avoid IDs for styling.

→ *More:* §2.1 Elements, tags and attributes, §6.4 Specificity

Q12. What are global attributes? Name a few.

Attributes allowed on every element: `id`, `class`, `style`, `title`, `lang`, `dir`, `hidden`, `tabindex`, `data-*`, `contenteditable`, `draggable`, `inert`, `popover`, `translate`, `spellcheck`, `accesskey`, plus all ARIA `role` / `aria-*` attributes and event handler attributes (`onclick`).

→ *More:* §2.1 Elements, tags and attributes

Q13. ⋆ `<b>` vs `<strong>`, `<i>` vs `<em>`?

`<strong>` means strong importance or urgency; `<em>` means stress emphasis that changes the sentence's meaning ("I *never* said that"). `<b>` and `<i>` carry no importance: `<b>` draws attention (keywords, product names), `<i>` marks an alternate voice (foreign words, technical terms, ship names). All four look bold/italic by default — choose by meaning, then style with CSS.

Follow-up: Is `<b>` deprecated? No — it was redefined in HTML5 with a specific meaning (drawing attention without extra importance) and is still valid.

→ *More:* §2.6 Text-level semantics

Q14. What are character references (HTML entities), and when do you need them?

A way to write characters that would otherwise be parsed as markup or are hard to type: `<` (<), `>` (>), `&` (&), `"` ("), ` ` (non-breaking space), or a numeric code like `₹` (₹). With UTF-8 you only *need* them for `<` and `&` in text and for quotes inside attribute values.

→ *More:* §2.9 Character references and comments

Q15. ⋆ What is the difference between `<script>`, `<script async>` and `<script defer>`?

- Plain `<script src>`: HTML parsing **stops** while the file downloads and runs.
- `async`: downloads in parallel and runs **as soon as it arrives**, pausing parsing then; order between async scripts is not guaranteed. Good for independent scripts like analytics.
- `defer`: downloads in parallel and runs **after parsing finishes**, before `DOMContentLoaded`, **in document order**. Best default for your app code. Module scripts (`type="module"`) are deferred by default. `async` / `defer` do nothing on inline scripts (except `async` on modules).

Follow-up: Which one runs first, a `defer` script or `DOMContentLoaded` handlers? Deferred scripts run first, then `DOMContentLoaded` fires. `async` scripts may run before or after `DOMContentLoaded`.

→ *More:* §14.2 Render-blocking resources, §14.6 Q44

Q16. ⋆ Why is it a good idea to put CSS `<link>`s in the `<head>` and JS `<script>`s just before `</body>`? Any exceptions?

CSS in the head lets the browser fetch it early and paint the page styled once, avoiding a **flash of unstyled content** (FOUC). Scripts at the end used to stop them from blocking parsing of the content above. Today the better answer is `<script defer>` in the head: it downloads early *and* does not block parsing. Exceptions: tiny scripts that must run before first paint (the dark-mode theme script, feature detection) go inline in the head; async analytics can go anywhere.

Follow-up: Does a stylesheet block JavaScript too? Yes: a synchronous script that comes after a stylesheet waits for that stylesheet, because the script might read computed styles.

→ *More:* §14.2 Render-blocking resources, §13.14 Theming and dark mode with custom properties

Q17. What is progressive rendering?

Showing useful content as early as possible instead of waiting for the whole page: streaming HTML so the browser parses and paints the top first, inlining critical CSS, deferring scripts, lazy-loading below-the-fold images, and (in apps) server-side rendering with streaming. The aim is a fast First Contentful Paint and Largest Contentful Paint.

→ *More:* §14.1 The critical rendering path, §14.4 Fixing each metric (and the other basics)

Q18. What are `preload`, `prefetch`, `preconnect` and `dns-prefetch`, and when do you use each?

They are `<link rel>` **resource hints**:

- `preload`: fetch a resource *this page* needs soon at high priority (a hero image, a font discovered late in CSS). Needs `as="..."`.
- `prefetch`: low-priority fetch of something the *next* navigation will probably need.

- `preconnect`: open the DNS + TCP + TLS connection to another origin early (a font CDN).
- `dns-prefetch`: only the DNS lookup; a cheap fallback. Overusing `preload` steals bandwidth from more important files.

→ *More:* §11.2 Loading fonts well: `font-display`, preload, subsetting, §14.4 Fixing each metric (and the other basics)

Q19. What do `rel="noopener"` and `rel="noreferrer"` do, and why do they matter for `target="_blank"`?

A page opened in a new tab used to get `window.opener`, a handle to your page, and could redirect it to a phishing copy ("reverse tabnabbing"). `noopener` removes that handle; `noreferrer` also hides the `Referer` header (and implies `noopener`). Modern browsers apply `noopener` to `target="_blank"` links by default, but writing it is still common practice.

→ *More:* §2.8 Links

Q20. `<link>` vs `<a>`?

`<a href>` is a hyperlink the user clicks to navigate; it goes in the body. `<link>` is metadata in the `<head>` that relates the document to another resource — a stylesheet, an icon, a canonical URL, a preload — and is never clicked.

→ *More:* §2.3 The `<head>`: metadata that matters, §2.8 Links

Q21. What is the `<base>` element, and what problems can it cause?

`<base href="https://cdn.example.com/">` changes the base URL for *every* relative URL in the page, and `<base target>` sets the default link target. It surprises people: in-page links like `href="#top"` resolve against the base and navigate away, and single-page-app routers can break. There can be only one, and it must come before any URL in the head. Use it rarely.

Q22. What is Subresource Integrity (SRI)?

An `integrity` attribute containing a hash of the expected file: `<script src="https://cdn…/lib.js" integrity="sha384-…" crossorigin="anonymous">`. If the CDN (Content Delivery Network, servers that host files close to users) serves a modified file, the hash does not match and the browser refuses to run it. It protects you from a compromised third-party host.

Q23. How do you serve a page with content in multiple languages?

Set the page language on `<html lang="en">` and mark passages in another language with `lang` on that element (`<span lang="fr">`). Serve one URL per language (`/en/`, `/fr/`), list alternates with `<link rel="alternate" hreflang="fr" href="…">`, and let users switch language explicitly — use `Accept-Language` only as a first guess. Screen readers use `lang` to switch pronunciation; browsers use it for hyphenation, fonts and translation.

→ *More:* §2.2 Document structure and the doctype, §14.5 SEO basics

Q24. What must you be careful about when designing for multilingual sites?

Text length varies (German can be 30% longer than English), so avoid fixed-width buttons; right-to-left languages (Arabic, Hebrew) need `dir="rtl"` and logical CSS properties (`margin-inline-start` instead of `margin-left`); don't put text inside images; don't build sentences by concatenating strings (word order differs); date, number and currency formats differ; fonts must cover the scripts; and colours or icons can carry different meanings.

→ *More:* §13.6 Logical properties

Q25. What are the building blocks of "HTML5" as an open web platform?

"HTML5" became shorthand for the modern platform: semantic elements (`<header>`, `<nav>`, `<main>`, `<article>`), native media (`<video>`, `<audio>`), graphics (`<canvas>`, inline SVG), new form input types and built-in validation, storage (`localStorage`, IndexedDB), offline (service workers), real-time communication (WebSockets, server-sent events), Web Workers, and device APIs (geolocation). HTML itself is now a "Living Standard" maintained by WHATWG, with no version numbers.

→ *More:* §1.1 The web in one picture

Q26. Have you used HTML templating languages? What are they for?

Templating languages (Handlebars, Nunjucks, EJS, Pug, Jinja, Thymeleaf, JSX in React) generate HTML from data plus a template with loops, conditions and partials (reusable fragments). Good answers mention that they must **escape** output by default (turn `<` into `<`) to prevent XSS — cross-site scripting, where an attacker's text gets run as script. The native `<template>` element is different: it holds inert HTML that JavaScript clones at runtime.

Q27. How do you validate HTML, and why bother?

Run it through the W3C Nu HTML Checker (validator.w3.org/nu) or an editor/CI linter (html-validate). Browsers silently fix invalid markup, but each one may fix it differently, and invalid nesting (a `<div>` inside a `<p>`, a button inside a link) often breaks layout, scripts or accessibility.

→ *More:* §2.10 Validation

Semantic HTML and SEO

Q28. ⋆ What is semantic HTML, and why does it matter?

Using elements for their meaning — `<nav>` for navigation, `<button>` for actions, `<h2>` for a section heading — rather than generic `<div>`s styled to look right. It matters because browsers give semantic elements built-in behaviour (keyboard support, focus), assistive technology (software such as screen readers) relies on the meaning, search engines understand the page better, and the code is easier to read.

Follow-up: Give an example where semantics changed behaviour for you: e.g. replacing a clickable `div` with `<button>` fixed keyboard access, or using `<nav>` and `<main>` let screen-reader users jump between regions.

→ *More:* §3.1 Why semantics and accessibility, §3.2 Landmarks

Q29. ⋆ `<section>` vs `<article>` vs `<div>`?

`<article>` is self-contained content that would make sense on its own (a blog post, a product card, a comment). `<section>` is a thematic group of content that normally has a heading. `<div>` has no meaning; use it purely as a styling/layout wrapper.

Follow-up: Can an `<article>` contain `<section>`s, and vice versa? Yes, both ways: a long article split into sections, or a section listing several articles (blog teasers).

→ *More:* §3.2 Landmarks, §14.6 Q4

Q30. ⋆ What are landmarks, and why do they matter?

Landmarks are page regions that assistive technology lists so users can jump straight to them: `<header>` (banner, when not inside article/section), `<nav>` (navigation), `<main>` (main), `<aside>` (complementary), `<footer>` (contentinfo), `<search>`, and `<form>`/`<section>` when they have an accessible name. Have exactly one visible `<main>`.

Follow-up: Do you still need `role="main"` on `<main>`? No — every modern browser maps the element to the role; adding the role is redundant.

→ *More:* §3.2 Landmarks, §14.6 Q5

Q31. What is the relationship between `<header>` and `<h1>`? Can a page have several `<header>`s or `<h1>`s?

`<header>` is a container for introductory content (logo, nav, a heading); `<h1>` is a heading. A page can have many `<header>`s (one per article or section). Several `<h1>`s are valid, but the "document outline" algorithm that would rank them by nesting was never implemented by browsers, so use one `<h1>` per page and real levels (`h2`, `h3`) for structure.

→ *More:* §2.4 Headings, §3.3 Headings and the outline

Q32. Why must headings not skip levels, and why should you not pick a heading for its size?

Screen-reader users navigate by heading level to understand the page structure; jumping from `<h2>` to `<h4>` suggests missing content. Choose the level by structure, then set the size with CSS.

→ *More:* §2.4 Headings, §3.3 Headings and the outline

Q33. `<figure>` vs `<img>`?

`<img>` is an image. `<figure>` is a self-contained unit referenced from the main text — an image, chart, code listing or quote — optionally with a `<figcaption>`. The caption is visible to everyone; `alt` is the replacement text when the image cannot be seen. They do different jobs, so a figure image still needs `alt`.

→ *More:* §4.5 `<figure>` and `<figcaption>`

Q34. ⋆ Which HTML matters for SEO?

SEO (Search Engine Optimisation) is making pages easy for search engines to find, understand and rank. The HTML that matters: a unique `<title>` and `<meta name="description">`, `<link rel="canonical">`, `lang`, the viewport tag, no stray `noindex`, one clear `<h1>` and logical headings, real `<a href>` links (crawlers don't click buttons), descriptive `alt` text, semantic structure, and structured data (JSON-LD). Speed (Core Web Vitals) also counts.

Follow-up: Does the `keywords` meta tag help SEO? No — Google has ignored it for many years. Content quality, links and page experience matter.

→ *More:* §14.5 SEO basics, §14.6 Q47

Q35. What is a canonical URL?

`<link rel="canonical" href="…">` tells search engines which URL is the main version when the same content is reachable at several URLs (with tracking parameters, `http` vs `https`, `/page` vs `/page/`). It is a strong hint, not a command; it consolidates ranking signals onto one URL.

→ *More:* §14.5 SEO basics

Q36. `robots.txt` `Disallow` vs `<meta name="robots" content="noindex">`?

`robots.txt` controls **crawling** (fetching). `noindex` controls **indexing** (showing in results). A disallowed URL can still appear in results if other sites link to it — and Google can't see a `noindex` on a page it isn't allowed to fetch. To keep a page out of results, allow crawling and use `noindex`.

→ *More:* §14.5 SEO basics

Q37. What is structured data (JSON-LD, microdata), and why use it?

Machine-readable facts about the page using the schema.org vocabulary — product price and rating, article author and date, breadcrumbs, FAQs. Search engines can then show **rich results** (stars, prices). Google recommends JSON-LD: a `<script type="application/ld+json">` block, separate from the visible markup. **Microdata** (`itemscope`, `itemprop` attributes woven into the HTML) is the older alternative.

→ *More:* §14.5 SEO basics

Q38. How do client-side rendered single-page apps affect SEO, and what do you do about it?

In a **single-page app** (SPA) the server sends an almost-empty HTML shell and JavaScript builds the page. Google can run JavaScript, but rendering is delayed and other crawlers and link-preview bots often don't run it at all. Fixes: server-side rendering (SSR) or static generation so the HTML arrives complete, real `<a href>` links, unique titles and canonicals per route, and correct status codes (a real 404, not a 200 "not found" page).

→ *More:* §14.5 SEO basics

Accessibility (ARIA, keyboard and focus, WCAG)

Q39. What is web accessibility, and who benefits?

Accessibility (a11y) means people with disabilities can perceive, operate and understand the site: blind users with screen readers (software that reads the page aloud), people who use only a keyboard or switch devices, users with low vision who zoom to 400%, colour-blind users, deaf users who need captions, and people with cognitive or vestibular (motion-sensitivity) conditions. Everyone else benefits too — captions in a noisy train, keyboard shortcuts, clear forms — and in many countries it is a legal requirement.

→ *More:* §3.1 Why semantics and accessibility

Q40. ⋆ What is ARIA, and what is the first rule of ARIA?

ARIA (Accessible Rich Internet Applications) is a set of attributes — `role`, `aria-*` states and properties — that tell assistive technology what a custom widget is (`role="tab"`), what state it is in (`aria-expanded="true"`) and how it relates to others (`aria-controls`). It changes only what is *announced*, not behaviour: it adds no keyboard support or styling. First rule: if a native element does the job (`<button>`, `<input type="checkbox">`, `<dialog>`), use it instead. No ARIA is better than bad ARIA.

Follow-up: Name a common ARIA mistake. Adding `role="button"` to a `div` without making it focusable or keyboard-operable; `aria-hidden="true"` on a focusable element (it can be tabbed to but is announced as nothing); or `aria-label` on a `div`/`span` with no role, where most screen readers ignore it.

→ *More:* §3.6 ARIA, §14.6 Q6

Q41. What is the accessibility tree?

A parallel tree the browser builds from the DOM for assistive technology. Each node has a **role** (button, link, heading), an **accessible name** (what is read out), a **state** (checked, expanded, disabled) and a description. Elements hidden with `display: none`, `hidden` or `aria-hidden="true"` are left out. You can inspect it in DevTools' Accessibility pane.

→ *More:* §3.4 The accessibility tree

Q42. ⋆ `aria-label` vs `aria-labelledby` vs `aria-describedby` vs `<label>` — how is the accessible name computed?

In priority order: `aria-labelledby` (points to other elements' IDs, their text becomes the name) → `aria-label` (a string on the element) → native labelling (`<label for>`, `alt`, `<caption>`, `<legend>`) → the element's text content (for buttons, links) → `title` as a last resort. `aria-describedby` adds a secondary **description** (hints, error messages) read after the name. Prefer visible text and `<label>` so sighted and non-sighted users get the same words.

Follow-up: What does `aria-label` on a link with visible text do? It replaces the visible text as the accessible name, so voice-control users saying the visible words may not be able to activate it (WCAG 2.5.3 Label in Name). Keep the visible text at the start of the accessible name.

→ *More:* §3.4 The accessibility tree, §3.8 Accessible forms

Q43. How do you make a custom component (for example, a `<div>` "button") accessible — and why is `<button>` better?

A `div` with `onclick` is not focusable, not announced as a button and doesn't react to Enter/Space. To fix it you would need `role="button"`, `tabindex="0"`, key handlers for Enter and Space, a disabled state and focus styles. `<button type="button">` gives all of that for free, which is why the answer is "use the native element".

→ *More:* §3.7 Keyboard and focus

Q44. ⋆ `<button>` vs `<a>` — when do you use which?

Links **navigate** (they have an `href`, open in a new tab, show in history, are activated by Enter). Buttons **perform an action** on the page (submit, open a menu, toggle). A link that looks like a button is fine; a button used for navigation, or an `<a>` without `href` used as a button, breaks keyboard behaviour and confuses screen-reader users.

Follow-up: What about `<a href="#" onclick>`? It works with a mouse but announces as a link, adds a history entry or jumps to the top, and isn't activated by Space — use a `<button>`.

→ *More:* §2.8 Links, §14.6 Q8

Q45. How does `tabindex` work? Why avoid positive values?

`tabindex="0"` adds an element to the natural Tab order (at its DOM position); `tabindex="-1"` makes it focusable by script (`el.focus()`) but not by Tab — useful for headings you move focus to, or dialog containers. Positive values (`tabindex="5"`) jump the element ahead of everything else, creating a confusing order that is hard to maintain — avoid them.

→ *More:* §3.7 Keyboard and focus

Q46. ⋆ How do you manage focus in a modal dialog?

On open: move focus into the dialog (the first control or the dialog itself); keep Tab inside it (a **focus trap**); make the page behind it inert (unreachable). Escape closes it. On close: return focus to the button that opened it. Native `<dialog>` opened with `showModal()` does almost all of this: the rest of the page becomes inert, Escape closes, and it renders in the top layer.

```
<button onclick="dlg.showModal()">Edit</button>
<dialog id="dlg" aria-labelledby="t"><h2 id="t">Edit profile</h2>
  <form method="dialog"><button>Close</button></form></dialog>
```

Follow-up: Why is `aria-modal="true"` not enough? It only tells screen readers the rest is inert; it doesn't trap keyboard focus or block clicks. `showModal()` or the `inert` attribute on the background actually does.

→ *More:* §3.7 Keyboard and focus

Q47. ⋆ What is a visible focus indicator, and why should you never write `outline: none` on its own?

The ring around the focused control shows keyboard users where they are; WCAG requires it to be visible. Removing it without a replacement makes the site unusable by keyboard. Use `:focus-visible`, which shows the ring for keyboard focus but not for mouse clicks:

```
:focus-visible { outline: 3px solid #1d4ed8; outline-offset: 2px; }
```

→ *More:* §3.7 Keyboard and focus, §11.6 Borders, radius and outlines

Q48. What is a skip link?

The first focusable element on the page — "Skip to main content", linking to `#main` — usually visually hidden until it receives focus. It lets keyboard and screen-reader users jump past the repeated header and navigation on every page.

→ *More:* §3.7 Keyboard and focus

Q49. ⋆ What are the ways to hide content, and what does each hide it from?

Technique	Visible?	Screen reader?	Takes space?
`display: none` / `hidden` attribute	No	No	No
`visibility: hidden`	No	No	Yes
`opacity: 0`	No	**Yes** (still focusable)	Yes
`aria-hidden="true"`	Yes	No	Yes
`.visually-hidden` class	No	**Yes**	No (1px)
`inert`	Yes	No (and unfocusable)	Yes

Follow-up: Which one would you use for a closed dropdown panel? `hidden` / `display: none`, so its links are not focusable while closed. For an animated panel, `visibility: hidden` after the transition (or `inert`) keeps focus out.

→ *More:* §3.4 The accessibility tree, §3.6 ARIA

Q50. ⋆ How do you hide content visually but keep it available to screen readers?

A "visually hidden" (or `sr-only`) utility clips the element to 1×1 px, so it is not seen but still in the accessibility tree:

```
.visually-hidden {
  position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px;
  overflow: hidden; clip-path: inset(50%); white-space: nowrap; border: 0;
}
```

Use it for icon-button labels, skip links (until focused) and extra table context. Don't use `display: none` — that hides it from everyone.

→ *More:* §3.6 ARIA

Q51. ⋆ How do you write good alt text?

Describe the image's *purpose in this context*, not its pixels: a logo link gets the destination ("Acme home"), a chart gets its key message, a photo in an article gets what the reader needs to know. Decorative images get `alt=""` (empty, so screen readers skip them). Never omit `alt` — the screen reader may read the file name. Don't start with "image of".

Follow-up: What is the alt for an image that is a link with no other text? The link's destination or purpose ("Search", "Home"), because it becomes the link's accessible name.

→ *More:* §3.5 Alt text, §4.1 The `<img>` element, §14.6 Q7

Q52. What is WCAG? What do the levels A, AA and AAA mean?

WCAG (Web Content Accessibility Guidelines, currently 2.2) is the W3C's standard, organised under four principles — **POUR**: Perceivable, Operable, Understandable, Robust. Each success criterion has a level: A (minimum), AA (the usual legal and contractual target), AAA (enhanced, not required for whole sites).

→ *More:* §3.9 WCAG 2.2

Q53. ⋆ What contrast ratio does WCAG AA require?

4.5:1 for normal text, 3:1 for large text (≥ 24 px, or ≥ 18.66 px bold), and 3:1 for UI components and meaningful graphics (borders of inputs, focus rings, icons). AAA raises text to 7:1 and 4.5:1. The ratio is computed from the relative luminance of the two colours: (L1 + 0.05) / (L2 + 0.05).

Follow-up: Does contrast apply to placeholder text and disabled controls? Placeholder text must meet 4.5:1 if it conveys information; disabled (inactive) controls are exempt.

→ *More:* §3.9 WCAG 2.2, §11.4 Colour and contrast, §14.6 Q13

Q54. What is new in WCAG 2.2?

Nine new criteria (six at level A/AA), including **Focus Not Obscured (Minimum)**, AA (a sticky header must not fully cover the focused element), **Target Size (Minimum)**, AA (pointer targets at least 24×24 CSS px, or enough spacing that a 24 px circle around each doesn't overlap another), **Dragging Movements** (offer a non-drag alternative), **Consistent Help**, **Redundant Entry** (don't ask for the same info twice) and **Accessible**

Authentication (Minimum), AA (no memory or puzzle tests like retyping a code without allowing paste or password managers); the other three are AAA. 4.1.1 Parsing was removed.

→ *More:* §3.9 WCAG 2.2

Q55. What is an ARIA live region?

An element whose changes screen readers announce without moving focus — for toasts, "3 results found", form-save confirmations. `aria-live="polite"` (or `role="status"`) waits until the user is idle; `aria-live="assertive"` (or `role="alert"`) interrupts. The region must already exist in the DOM *before* you change its text, or the change may not be announced.

→ *More:* §3.6 ARIA, §5.7 Accessible error messages

Q56. How do you build an accessible accordion, tabs or dropdown menu?

Accordion: each header is a `<button aria-expanded aria-controls>` inside a heading (or use native `<details>/<summary>`). Tabs: `role="tablist"`, `role="tab"` with `aria-selected`, `role="tabpanel"`, arrow keys move between tabs. Disclosure menu: a `<button aria-expanded>` that toggles a list of links. Follow the WAI-ARIA Authoring Practices Guide (APG) keyboard patterns — and prefer native elements where they exist.

→ *More:* §3.6 ARIA, §10.4 Mobile-first in practice

Q57. How do you test accessibility?

Three layers: **automated** tools (axe DevTools, Lighthouse, WAVE) catch roughly a third of issues — missing alt, contrast, labels; a **manual keyboard test** (Tab through everything: can you reach, see and operate every control? any traps?); and a **screen-reader pass** (VoiceOver, NVDA) plus zoom to 200–400% and reduced motion. Real users with disabilities are the final test.

→ *More:* §3.10 Testing tools

Q58. What is `prefers-reduced-motion`, and why respect it?

A media feature that is `reduce` when the user has asked the OS to minimise animation. Large motion (parallax, zooms, auto-playing carousels) can cause nausea or dizziness for people with vestibular disorders. Wrap non-essential motion in `@media (prefers-reduced-motion: no-preference) { … }` or tone it down under `reduce`.

→ *More:* §12.9 Reduced motion and accessibility, §14.6 Q40

Forms and validation

Q59. How does a form submit? What do `action`, `method` and `name` do?

On submit the browser collects every enabled control that has a `name`, builds `name=value` pairs, and sends them to the `action` URL using `method` (GET or POST). A field without a `name` is **not sent**. `enctype` sets the encoding: `application/x-www-form-urlencoded` (default) or `multipart/form-data` (required for file uploads).

→ *More:* §5.2 How a form works — `action`, `method`, `name`

Q60. ⋆ GET vs POST in forms?

GET puts the data in the URL query string — bookmarkable, cacheable, shows in history and server logs; use it for searches and filters. POST sends the data in the request body — use it for anything that changes data, for passwords and other sensitive data, and for large or file uploads.

Follow-up: Is POST more secure than GET? Only slightly: the data isn't in the URL, logs or history, but it's still readable on the wire unless you use HTTPS, which is what actually protects it.

→ *More:* §5.2 How a form works — `action`, `method`, `name`, §14.6 Q10

Q61. ⋆ How do you label form fields accessibly?

Every input needs a `<label for="id">` (or the input wrapped inside the label) — clicking the label focuses the input, and it becomes the accessible name. Placeholders are not labels (they disappear and have low contrast). Group related radios or checkboxes in `<fieldset>` with a `<legend>`. Link hints and errors with `aria-describedby`.

Follow-up: Is `aria-label` an acceptable substitute for `<label>`? Only when there truly is no visible label (a search field with a visible magnifying-glass button, for example). A visible label helps everyone, including people with cognitive disabilities.

→ *More:* §3.8 Accessible forms, §14.6 Q9

Q62. What do `<fieldset>` and `<legend>` do?

`<fieldset>` groups related controls; `<legend>` is its caption, read before each control's label ("Delivery speed, Express, radio button"). A disabled fieldset disables every control inside it.

→ *More:* §5.4 Choosing and grouping — `select`, `datalist`, `textarea`, `fieldset`, `output`, `progress`, `meter`

Q63. Name the HTML5 input types, and why use them instead of `type="text"`.

`email`, `url`, `tel`, `number`, `range`, `date`, `time`, `datetime-local`, `month`, `week`, `color`, `search`, `password`, `file`, `checkbox`, `radio`, `hidden`. They give the right mobile keyboard, built-in validation (`email` checks the format) and native pickers. `inputmode` changes only the keyboard (`inputmode="numeric"` for a PIN, which shouldn't be `type="number"`).

→ *More:* §5.3 Input types

Q64. ★ How does built-in (constraint) validation work?

Attributes declare rules — `required`, `type="email"`, `min` / `max`, `minlength` / `maxlength`, `pattern`, `step` — and the browser blocks submission and shows a message when one fails. In JavaScript, `input.validity` gives flags (`valueMissing`, `typeMismatch`, `patternMismatch` …), `checkValidity()` / `reportValidity()` test them, and `setCustomValidity("message")` adds your own error (pass `""` to clear it). `novalidate` on the form turns the browser UI off so you can show your own messages while keeping the API.

Follow-up: Why might `pattern` not work on `type="number"`? `pattern` only applies to text-like inputs (`text`, `email`, `tel`, `url`, `search`, `password`). For numbers use `min`, `max` and `step`.

→ *More:* §5.5 Built-in validation and the Constraint Validation API

Q65. Is client-side validation enough?

No. It is a convenience for the user; anyone can bypass it (DevTools, curl, disabling JS). The server must validate and sanitise everything again — client-side checks are UX, server-side checks are security.

→ *More:* §5.2 How a form works — `action`, `method`, `name`, §5.5 Built-in validation and the Constraint Validation API

Q66. `:invalid` vs `:user-invalid`?

`:invalid` matches as soon as the value breaks a rule — so a `required` field is red before the user has typed anything. `:user-invalid` (Baseline since 2023) matches only after the user has interacted with the field or tried to submit, which is the behaviour you normally want for error styling.

→ *More:* §5.5 Built-in validation and the Constraint Validation API

Q67. `readonly` vs `disabled`?

`readonly`: the user can focus and select the value but not change it, and the value **is submitted**. `disabled`: not focusable, greyed out, and the value is **not submitted**; it also skips validation. Use `aria-disabled="true"` when you want a control to stay focusable (so screen-reader users can discover it) but not work.

→ *More:* §5.3 Input types

Q68. `<select>` vs `<datalist>`?

`<select>` forces a choice from a fixed list. `<datalist>` attached to an `<input list="id">` offers suggestions while still allowing free text (an autocomplete). Use radios instead of a select when there are only a few options — all are visible at once.

→ *More:* §5.4 Choosing and grouping — `select`, `datalist`, `textarea`, `fieldset`, `output`, `progress`, `meter`

Q69. `<progress>` vs `<meter>`; what is `<output>`?

`<progress>` shows completion of a task (upload 40%), and has an indeterminate state with no value. `<meter>` shows a scalar measurement within a known range (disk 80% full, password strength), with `low` / `high` / `optimum` thresholds. `<output>` holds the result of a calculation or user action and is announced as a live region in most screen readers.

→ *More:* §5.4 Choosing and grouping — `select`, `datalist`, `textarea`, `fieldset`, `output`, `progress`, `meter`

Q70. What does the `autocomplete` attribute do?

It tells the browser and password manager what a field contains — `autocomplete="email"`, `"given-name"`, `"street-address"`, `"one-time-code"`, `"current-password"`, `"new-password"`. Correct values let users fill forms in one tap and are required by WCAG 1.3.5 (Identify Input Purpose).

→ *More:* §5.6 `autocomplete` — help the browser fill the form

Q71. How do you show accessible error messages?

Put the error text next to the field, link it with `aria-describedby`, set `aria-invalid="true"`, and on submit move focus to the first invalid field (or to an error summary at the top that links to each field). Don't rely on colour alone — add an icon or text. Say what went wrong and how to fix it.

→ *More:* §5.7 Accessible error messages

Q72. What is the difference between `<button>`, `<button type="button">` and `<input type="submit">`?

A `<button>` inside a form defaults to `type="submit"`, so a "Show password" button without `type="button"` submits the form — a classic bug. `<input type="submit">` can only contain plain text (its `value`); `<button>` can contain HTML such as an icon. `type="reset"` clears the form and is rarely wanted.

→ *More:* §5.2 How a form works — `action`, `method`, `name`

Images, media and embeds

Q73. ⋆ How do `srcset` and `sizes` work? Explain how the browser picks an image.

`srcset` lists candidate files with their intrinsic widths (`hero-800.jpg 800w, hero-1600.jpg 1600w`). `sizes` tells the browser how wide the image will be displayed (`(min-width: 60rem) 50vw, 100vw`), because it has to choose *before* CSS is loaded. The browser takes the first matching `sizes` entry, multiplies by the device pixel ratio (e.g. 400 px × 2 = 800 px needed), and picks the smallest candidate that is wide enough (it may also consider cached files or data-saver mode). With only density descriptors (`1x, 2x`) no `sizes` is needed.

Follow-up: Why can't the browser just use CSS to decide? Image requests start during HTML parsing (by the preload scanner, which looks ahead in the HTML for files to fetch) before layout is known — `sizes` gives it the layout hint up front. `sizes="auto"` (Chromium and Firefox so far, not Safari — not Baseline) works for lazy-loaded images, whose layout is known by the time they are fetched.

→ *More:* §4.3 Responsive images — `srcset`, `sizes` and `<picture>`, §14.6 Q11

Q74. ⋆ When do you use `<picture>`?

For **art direction** (a different crop or image at different breakpoints) and **format fallbacks** (AVIF, then WebP, then JPEG). The browser takes the first `<source>` whose `media` and `type` match; the inner `<img>` is required — it provides `alt`, sizing and the fallback.

```
<picture>
  <source type="image/avif" srcset="hero.avif">
  <source type="image/webp" srcset="hero.webp">
  <img src="hero.jpg" alt="Team at the summit" width="1200" height="800">
</picture>
```

→ *More:* §4.3 Responsive images — `srcset`, `sizes` and `<picture>`, §14.6 Q12

Q75. Which image format would you choose: JPEG, PNG, GIF, WebP, AVIF or SVG?

Photos: AVIF or WebP (much smaller than JPEG), with a JPEG fallback if needed. Screenshots or images needing lossless quality or transparency: PNG or lossless WebP. Logos, icons and illustrations: SVG (vector, so sharp at any size and tiny). Animations: a muted looping `<video>` (MP4/WebM) instead of a huge GIF.

→ *More:* §4.2 Image formats — JPEG, PNG, GIF, WebP, AVIF, SVG

Q76. Raster vs vector images?

Raster images (JPEG, PNG, WebP, AVIF) are grids of pixels: they blur when scaled up and file size grows with dimensions. Vector images (SVG) are shapes described mathematically: crisp at any size, styleable with CSS, but unsuitable for photos.

→ *More:* §4.2 Image formats — JPEG, PNG, GIF, WebP, AVIF, SVG, §4.8 SVG basics

Q77. ⋆ Why set `width` and `height` on images?

From the two attributes the browser computes the aspect ratio and reserves the right amount of space before the image downloads, so the text below doesn't jump when it arrives. That prevents layout shift (CLS). With `img { max-width: 100%; height: auto; }` the image still scales responsively.

Follow-up: What if the image is responsive with `width: 100%`? Keep the attributes anyway; with `height: auto` in CSS the browser uses their ratio for the reserved space.

→ *More:* §4.1 The `<img>` element, §14.6 Q46

Q78. How does `loading="lazy"` work, and when should you avoid it?

It defers loading an image or iframe until it gets near the viewport, saving bandwidth. Avoid it on the **LCP image** and anything above the fold — lazy-loading the hero image delays the page's main paint. For the hero use `fetchpriority="high"` instead. `decoding="async"` lets the browser decode the image off the critical path.

→ *More:* §4.4 Loading hints — `loading`, `decoding`, `fetchpriority`

Q79. Have you worked with retina (high-DPI) graphics? What techniques do you use?

High-DPI screens have a **device pixel ratio** (DPR) of 2 or 3: several physical pixels per CSS pixel, so a 1x image looks blurry. Techniques: `srcset` with `2x` or `w` descriptors, SVG for icons and logos, `image-set()` for CSS backgrounds, and icon fonts or CSS where possible. Don't send 3x images to everyone — let the browser choose.

→ *More:* §4.3 Responsive images — `srcset`, `sizes` and `<picture>`, §10.8 Responsive images recap (and `image-set()`)

Q80. ⋆ Canvas vs SVG?

SVG is **retained-mode**: shapes are DOM elements you can style, animate with CSS, attach events to, and that screen readers can read; it stays sharp at any size, but thousands of nodes get slow. `<canvas>` is **immediate-mode**: you draw pixels with JavaScript and the browser keeps no objects, so it handles many moving objects (games, big charts) quickly but is not accessible by default and must be redrawn for resizing or high-DPI.

Follow-up: Which would you pick for an interactive chart with 50 bars? SVG (accessible, styleable, crisp). For a live chart with 100,000 points, canvas (or WebGL).

→ *More:* §4.9 `<canvas>` — an introduction

Q81. Inline SVG vs `<img src="icon.svg">`?

Inline SVG becomes part of the DOM: you can style it with CSS (`fill: currentColor` to match text colour) and script it, but it isn't cached separately and bloats the HTML. `<img>` SVG is cached and simple but can't be styled from the page. For icon sets, an SVG sprite with `<use href="#icon">` is a middle ground.

→ *More:* §4.8 SVG basics

Q82. Are you familiar with styling SVG?

Yes: inline SVG elements accept CSS properties such as `fill`, `stroke`, `stroke-width`, `stroke-dasharray` (for line-drawing animations), `opacity` and transforms. `fill: currentColor` makes an icon inherit the text colour. Accessible SVG uses `role="img"` and a `<title>` (or `aria-label`); decorative SVG gets `aria-hidden="true"`.

→ *More:* §4.8 SVG basics

Q83. How do you include audio and video, and make them accessible?

`<video controls width height poster>` with one or more `<source type>` children, and `<track kind="captions" srclang="en" src="captions.vtt">` for captions (WebVTT files). Provide a transcript for audio. Browsers generally allow autoplay only when muted (`autoplay muted playsinline`), and moving content that plays for more than 5 seconds needs a pause control (WCAG 2.2.2).

→ *More:* §4.6 Audio and video — `<audio>`, `<video>`, `<source>`, `<track>`

Q84. What is an `<iframe>`, and how do you embed one safely?

It embeds another HTML page inside yours. Use `sandbox` (turns everything off; then allow only what's needed, e.g. `sandbox="allow-scripts"`), `allow` for permissions (camera, fullscreen), `referrerpolicy`, `loading="lazy"` and a `title` for screen readers. To stop *others* from framing your site (clickjacking), send `Content-Security-Policy: frame-ancestors 'self'`.

→ *More:* §4.7 Embedding other pages — `<iframe>`

Q85. How do two iframes, or a page and its iframe, communicate?

Same-origin frames can access each other's `window` directly. Cross-origin frames must use `postMessage`: `frame.contentWindow.postMessage(data, "https://child.example")`, and the receiver listens for `message` events and **checks** `event.origin` before trusting the data.

Browser storage and platform APIs

Q86. ⋆ Describe the difference between a cookie, `sessionStorage` and `localStorage`.

	Cookie	`localStorage`	`sessionStorage`
Sent to server	Yes, with every matching request	No	No
Size	~4 KB each	~5 MB per origin	~5 MB per origin
Lifetime	Until `Expires` / `Max-Age`, or session	Until cleared	Until the tab closes
Scope	Domain + path	Origin (scheme + host + port)	Origin **and** tab
JS access	`document.cookie` unless `HttpOnly`	Yes	Yes

Cookies are for server-readable state (sessions); Web Storage is synchronous, strings only, and for small client-side preferences.

Follow-up: Is `localStorage` shared between tabs? Yes, between all tabs of the same origin (and a `storage` event fires in the other tabs). `sessionStorage` is per tab. Both are blocked or partitioned in some private-browsing and third-party contexts.

→ *More:* §1.5 HTTP basics — requests and responses

Q87. Where should you store an authentication token?

Anything readable by JavaScript (`localStorage`, non-HttpOnly cookies) can be stolen by an XSS attack. The common recommendation is an `HttpOnly; Secure; SameSite=Lax` (or `Strict`) cookie — JavaScript can't read it, it's only sent over HTTPS, and `SameSite` blocks most CSRF (cross-site request forgery, where another site makes the browser send an authenticated request). Pair it with CSRF tokens for state-changing requests if needed.

Q88. What are the cookie attributes `HttpOnly`, `Secure`, `SameSite`, `Domain`, `Path`?

`HttpOnly`: hidden from JavaScript. `Secure`: HTTPS only. `SameSite=Strict|Lax|None`: whether the cookie is sent on cross-site requests (`Lax`, the default in Chromium, sends it on top-level link navigations; `None` requires `Secure`). `Domain`/`Path`: which URLs receive it. `Expires`/`Max-Age`: lifetime.

→ *More:* §1.5 HTTP basics — requests and responses

Q89. When would you use IndexedDB instead of `localStorage`?

`localStorage` is synchronous (it blocks the main thread), stores only strings and is small. IndexedDB is an asynchronous, transactional database in the browser that stores structured data and files (Blobs), with indexes and much larger quotas — use it for offline data, caches or large datasets. The Cache API (used with service workers) stores HTTP responses.

Q90. What are Web Components?

A set of standards for reusable, encapsulated custom elements: **Custom Elements** (`customElements.define('my-card', class extends HTMLElement {…})`), **Shadow DOM** (a private DOM subtree whose styles don't leak in or out), and **`<template>`/`<slot>`** (inert markup to clone, and placeholders for the user's content). They work in any framework or none.

Q91. What is the Shadow DOM, and how do you style it?

A hidden DOM tree attached to an element (`attachShadow({mode: 'open'})`). Page CSS selectors don't reach inside, and its styles don't leak out — but **inherited properties** (colour, font) and **custom properties** do pass through. Style it from outside via CSS variables, `::part()` (parts the component exposes) and `::slotted()`/`:host` from inside.

Q92. What are Web Workers and service workers?

A **Web Worker** runs JavaScript on a background thread (no DOM access), talking to the page via `postMessage` — for heavy computation that would freeze the UI. A **service worker** is a special worker that sits between the page and the network: it can intercept requests, serve cached responses (offline support), and receive push messages. It requires HTTPS.

Q93. What are server-sent events and WebSockets?

Server-sent events (`EventSource`): a one-way stream of text messages from server to browser over plain HTTP, with automatic reconnection — good for notifications and live feeds. **WebSockets**: a full two-way connection for low-latency messages in both directions — chat, games, collaborative editing.

Selectors, specificity, the cascade and inheritance

Q94. What are the three ways to add CSS to a page, and which is preferred?

An external stylesheet (`<link rel="stylesheet">`, cached and shared across pages — the default), an internal `<style>` block (good for critical CSS or one-off pages), and the inline `style` attribute (highest priority among normal declarations, hard to override and to maintain, no media queries or pseudo-classes). `@import` inside CSS also works but delays loading because the browser discovers the file late.

→ *More:* §6.1 Syntax and adding CSS to a page

Q95. ⋆ What is CSS selector specificity, and how is it calculated?

Specificity is a score that decides between conflicting declarations from the same origin and layer. Count three columns **(A, B, C)**: A = ID selectors, B = classes, attribute selectors and pseudo-classes, C = type (element) selectors and pseudo-elements. Compare A first, then B, then C — so `#nav` (1,0,0) beats

`.a.b.c.d.e.f.g.h.i.j.k` (0,11,0). The universal selector `*` and combinators add nothing. Inline styles beat any selector; `!important` works at a different level (origin), not as extra specificity.

Follow-up: What specificity does `style="…"` have? It's not part of the A-B-C score; inline styles are a separate cascade step above all selectors. Only `!important` declarations (or animations, for animated values) beat it.

→ *More:* §6.4 Specificity, §14.6 Q14

Q96. What is the specificity of `:is()`, `:not()`, `:has()` and `:where()`?

`:is()`, `:not()` and `:has()` take the specificity of their **most specific argument** — `:is(#a, .b)` counts as an ID even when `.b` is what matched. `:where()` always has **zero** specificity, which makes it ideal for defaults and resets that should be easy to override.

→ *More:* §6.4 Specificity, §13.2 `:is()` and `:where()`, §14.6 Q20

Q97. ⋆ What decides which declaration wins? Explain the cascade.

The cascade sorts competing declarations by, in order: **origin and importance** (user-agent, user, author styles; `!important` reverses the order), **context** (shadow DOM), **inline style**, **cascade layer**, **specificity**, **scope proximity** (for `@scope`), and finally **order of appearance** (the last one wins). Only if one step ties does the next one count.

Follow-up: Where do animations and transitions fit in the cascade? Declarations from running CSS animations override all normal declarations; running transitions override even `!important` ones.

→ *More:* §6.5 The cascade and origins, §14.6 Q15

Q98. When should you use `!important`?

Rarely: for utility classes that must always win (`.hidden { display: none !important }`), for overriding third-party inline styles you can't change, and in user stylesheets for accessibility. Using it to win specificity fights creates an escalating war that only more `!important` can win. Fix the cause instead: lower specificity, use cascade layers. Note that `!important` in an earlier (lower) layer beats `!important` in a later layer.

→ *More:* §6.5 The cascade and origins, §13.4 Cascade layers (`@layer`)

Q99. ⋆ Pseudo-class vs pseudo-element?

A **pseudo-class** (one colon) selects an element in a particular state or position: `:hover`, `:focus-visible`, `:checked`, `:nth-child(2)`, `:has()`. A **pseudo-element** (two colons) selects or creates a *part* of an element that isn't in the DOM: `::before`, `::after`, `::first-line`, `::placeholder`, `::marker`, `::selection`, `::backdrop`.

Follow-up: Why the double colon? CSS3 introduced `::` to distinguish pseudo-elements; browsers still accept the old single colon for `:before`, `:after`, `:first-line` and `:first-letter` for compatibility.

→ *More:* §6.3 Pseudo-classes and pseudo-elements, §14.6 Q16

Q100. What are `::before` and `::after` used for? Why doesn't mine show up?

They insert a generated box as the first/last child of the element's content — decorative icons, quotes, tooltips' arrows, clearfixes, overlays. They only render if `content` is set (`content: ""` at minimum). Replaced elements such as `<img>`, `<video>` and `<iframe>` don't render them, and most form controls don't either — the exception is a checkbox or radio with `appearance: none`, which current browsers do decorate (the toggle switch in Q230 relies on it). Text in `content` is read by most screen readers, so don't put essential content in it.

→ *More:* §6.3 Pseudo-classes and pseudo-elements

Q101. ⋆ `:nth-child()` vs `:nth-of-type()`?

`:nth-child(2)` matches an element that is the 2nd child of its parent, *of any type*, and also matches the rest of the selector. `:nth-of-type(2)` counts only siblings of the same element type. For `<div><h2/><p/><p/></div>`, `p:nth-child(2)` is the first `<p>`; `p:nth-of-type(2)` is the second `<p>`. Modern `:nth-child(2 of .item)` counts only elements matching `.item`.

Follow-up: Write a selector for every third item starting with the second: `li:nth-child(3n + 2)`. For the last two items: `li:nth-last-child(-n + 2)`.

→ *More:* §6.3 Pseudo-classes and pseudo-elements

Q102. What do the combinators ` `, `>`, `+` and `~` mean?

Descendant (`nav a` — any depth), child (`ul > li` — direct children only), next sibling (`h2 + p` — the paragraph immediately after an h2), subsequent sibling (`h2 ~ p` — every later sibling paragraph).

→ *More:* §6.2 Selectors

Q103. Name the attribute selectors.

`[href]` (has it), `[type="email"]` (exactly), `[class~="card"]` (word in a space-separated list), `[lang|="en"]` (`en` or `en-…`), `[href^="https"]` (starts with), `[href$=".pdf"]` (ends with), `[href*="example"]` (contains); add `i` before `]` for case-insensitive matching.

→ *More:* §6.2 Selectors

Q104. ⋆ Explain how a browser determines which elements match a selector. Why does it read right to left?

Browsers match selectors **from right to left**: for `.nav ul li a`, they start with each `<a>` (the **key selector**) and walk up the ancestors to check the rest. Most elements fail the rightmost part immediately, so this is fast. That's why a very generic key selector with a long chain (`.sidebar *`) does more work. In practice modern engines are fast enough that selector performance rarely matters compared with layout costs.

Follow-up: So is `.nav a` faster than `a`? Practically irrelevant. Write selectors for readability and low specificity; worry about layout and paint costs instead.

→ *More:* §6.2 Selectors

Q105. ⋆ Which CSS properties inherit, and how do you control inheritance?

Mostly text-related properties inherit: `color`, `font-*`, `line-height`, `letter-spacing`, `text-align`, `visibility`, `cursor`, and custom properties. Box properties don't: `margin`, `padding`, `border`, `width`, `background`, `display`. The keywords `inherit` (take the parent's value), `initial` (the property's spec default), `unset` (inherit if inherited, else initial) `revert` (roll back to the previous origin — usually the browser's default stylesheet) and `revert-layer` (roll back to the previous cascade layer) work on every property; `all: unset` resets everything.

→ *More:* §6.6 Inheritance, §14.6 Q17

Q106. ⋆ What is the difference between "resetting" and "normalizing" CSS? Which would you choose?

A **reset** removes browser defaults (margins, list styles, heading sizes) so you start from zero; a **normalize** keeps useful defaults but makes them consistent across browsers and fixes bugs. Today most teams use a small **modern reset**: `box-sizing: border-box` everywhere, zero body margin, responsive media (`img { max-width: 100%; display: block }`), `font: inherit` on form controls — keeping semantics-friendly defaults like list bullets where wanted.

Follow-up: What's in your usual reset? `box-sizing: border-box`, `margin: 0` on body/headings/paragraphs, `img, svg, video { display: block; max-width: 100% }`, `input, button, textarea, select { font: inherit }`, `line-height: 1.5` on body, and `overflow-wrap: break-word` on text.

→ *More:* §13.13 Resets and normalize

Q107. How would you approach fixing browser-specific styling issues?

Reproduce it and identify the engine and version; check MDN / caniuse / Baseline for support; use `@supports` for feature detection and a fallback; use Autoprefixer (a PostCSS tool) instead of hand-written vendor prefixes; use a reset/normalize; and as a last resort, isolate a targeted fix with a comment and a removal date. Avoid user-agent sniffing (guessing the browser from its name string).

→ *More:* §13.12 Preprocessors (Sass) and PostCSS

Q108. How do you serve your pages for feature-constrained (older) browsers?

Progressive enhancement: build a working baseline in plain HTML and CSS, then layer on features where they're supported. Tools: `@supports (display: grid) { … }`, properties browsers ignore if unknown (write the fallback first, the modern value second), Autoprefixer and Browserslist, polyfills only where needed, and testing on real old devices. **Graceful degradation** is the reverse: build for modern browsers and make sure it still works acceptably when features are missing.

→ *More:* §10.6 Container queries, §13.12 Preprocessors (Sass) and PostCSS

Q109. Can you give an example of an `@media` rule other than `screen`?

`@media print { nav, .ads { display: none } a::after { content: " (" attr(href) ")" } }` for printing. Others: `@media (prefers-color-scheme: dark)`, `(prefers-reduced-motion: reduce)`, `(hover: hover) and (pointer: fine)` for mouse users, `(orientation: landscape)`, `(forced-colors: active)` for Windows high-contrast mode.

→ *More:* §10.3 Media queries, §10.5 User-preference media queries

---

Box model, `display` and units

Q110. ⋆ Explain the CSS box model.

Every element generates a rectangular box made of four layers, from inside out: **content** (text, images), **padding** (space inside the border, takes the background), **border**, and **margin** (transparent space outside, between boxes). By default `width` and `height` set only the content box, so a `width: 200px` box with `20px` padding and a `5px` border is 250 px wide.

Follow-up: What is the margin of a box doing to its background? Nothing — backgrounds paint under content and padding (and border by default, see `background-clip`), never under the margin.

→ *More:* §7.1 The box model, §14.6 Q23

Q111. ⋆ What does `* { box-sizing: border-box; }` do, and what are its advantages?

It makes `width` and `height` include padding and border, so `width: 200px` stays 200 px no matter what padding you add. Layout maths becomes predictable (`width: 50%` plus padding no longer overflows), which is why almost every reset includes it — usually as `*, *::before, *::after { box-sizing: border-box; }`. The other value, `content-box`, is the default.

Follow-up: Does `border-box` include the margin? No. Margins are always outside the box's size.

→ *More:* §7.2 `box-sizing`, §14.6 Q23

Q112. ⋆ What is the CSS `display` property? Give some examples.

It sets the box's **outer** role (how it takes part in its parent's layout: `block` or `inline`) and its **inner** layout for its children (`flow`, `flex`, `grid`, `table`). Common values: `block`, `inline`, `inline-block`, `flex`, `inline-flex`, `grid`, `inline-grid`, `flow-root`, `contents` (the box disappears, children stay), `none`, `table`, `list-item`.

→ *More:* §7.4 `display`

Q113. ⋆ Block vs inline vs inline-block?

Block (`div`, `p`, `h1`): starts on a new line, fills the available width, respects `width`/`height` and all margins. **Inline** (`span`, `a`, `strong`): flows within text, ignores `width`/`height`, and vertical margins/padding don't push other lines away. **Inline-block**: flows in the line like text but respects `width`, `height`, padding and margins like a block — useful for buttons and badges.

Follow-up: Can you set `padding` on an inline element? Yes, and it's painted, but vertical padding doesn't push neighbouring lines apart, so it can overlap the lines above and below.

→ *More:* §7.4 `display`, §14.6 Q3

Q114. Can you change an inline element into a block-level element?

Yes — `display` is CSS, independent of the HTML element. `a { display: block }` makes the whole link area clickable. But the HTML content model (which elements may contain which) doesn't change: `<span>` still can't legally contain a `<div>` in the markup. (Since HTML5, `<a>` may wrap block content.)

→ *More:* §7.4 `display`

Q115. ⋆ `display: none` vs `visibility: hidden` vs `opacity: 0`?

`display: none` removes the box: no space, not rendered, hidden from screen readers, not focusable. `visibility: hidden` keeps the space but hides it (also from screen readers and focus); a child can override it with `visibility: visible`. `opacity: 0` keeps space and stays interactive — clickable, focusable and announced. `opacity` animates smoothly and `visibility` flips discretely at the end of a transition; `display: none` can now be part of a transition too with `transition-behavior: allow-discrete` plus `@starting-style` (see Q182).

Follow-up: Which of the three still receives clicks? Only `opacity: 0`. Add `pointer-events: none` if it shouldn't, and remember it's still announced and focusable.

→ *More:* §3.4 The accessibility tree, §12.1 Transitions

Q116. What is a replaced element?

An element whose content comes from outside CSS — `<img>`, `<video>`, `<iframe>`, `<input>`, `<canvas>` — so it has an **intrinsic size** (and often an aspect ratio). CSS sizes the box but doesn't lay out the inside; `object-fit` controls how the content fills it, and `::before`/`::after` don't apply.

→ *More:* §7.4 `display`, §11.9 `object-fit` and `object-position`

Q117. Why is there a small gap under my image, or between inline-block elements?

Under an image: images are inline by default and sit on the text **baseline**, leaving room for letter descenders (g, y). Fix with `img { display: block }` or `vertical-align: middle`. Between inline-blocks: the white space (newlines) in the HTML is rendered as a space character. Fix by using flexbox or grid with `gap` instead.

→ *More:* §7.4 `display`

Q118. ⋆ Explain `px`, `em`, `rem`, `%`, `vw`/`vh` — when do you use each?

`px`: an absolute CSS pixel (1/96 inch at arm's length; not a device pixel) — borders, shadows. `em`: relative to the element's own `font-size` (for the `font-size` property itself, the parent's) — padding that should scale with the text. `rem`: relative to the root (`<html>`) font size — font sizes and spacing that respect the user's browser setting. `%`: relative to the parent (width-based for padding and margin, even vertical ones). `vw`/`vh`: 1% of the viewport's width/height — full-screen sections, fluid type.

Follow-up: What does `1px` mean on a phone with DPR 3? One CSS pixel is drawn with 3×3 device pixels, so CSS sizes look the same physical size on all screens.

→ *More:* §6.7 Units, §14.6 Q18

Q119. `em` vs `rem` — what is the compounding problem?

Nested `em` font sizes multiply: a list item with `font-size: 1.2em` inside another one becomes 1.44 × the base, and so on. `rem` always refers to the root, so there is no compounding. Use `rem` for font sizes, `em` for things that should scale with the local text size (icon size, button padding, media query breakpoints).

→ *More:* §6.7 Units

Q120. Why should font sizes use `rem` rather than `px`?

Users can set a larger default font size in the browser; `rem` scales with it, `px` ignores it (page zoom scales both, but the font-size setting doesn't). Fixed `px` text can fail WCAG 1.4.4 (resize text) for users who rely on that setting.

→ *More:* §6.7 Units

Q121. `100vh` on mobile is too tall — why, and what are `svh`, `lvh` and `dvh`?

On mobile the browser's toolbar shows and hides as you scroll; `vh` uses the **largest** viewport (toolbar hidden), so a `100vh` section is cut off behind the toolbar. `svh` = small viewport (toolbars shown), `lvh` = large, `dvh` = dynamic (changes as the toolbar moves, which can cause jumps). Use `min-height: 100svh` (or `100dvh`) for full-screen sections.

→ *More:* §10.2 The viewport meta tag and viewport units

Q122. Why should `line-height` be unitless?

A unitless `line-height: 1.5` is inherited as the *ratio*, so each child recomputes it from its own font size. `line-height: 1.5em` or `24px` is computed once on the parent and inherited as a fixed length, so a larger heading inside gets cramped, overlapping lines.

→ *More:* §11.3 Text properties

Q123. What is the difference between `border`, `outline` and `box-shadow`?

A border is part of the box model — it takes space and changes the size. An outline and a box-shadow are drawn outside the box without affecting layout. Outlines follow `border-radius` in modern browsers and are visible in Windows high-contrast (forced-colours) mode, which is why they are the right tool for focus rings; box-shadows disappear in that mode.

→ *More:* §7.1 The box model, §11.6 Borders, radius and outlines

Q124. What are `min()`, `max()` and `clamp()`?

Math functions that pick a value at render time: `width: min(100%, 60rem)` (never wider than 60rem, never overflow), `max(1rem, 2vw)`, and `clamp(MIN, PREFERRED, MAX)` — the preferred value bounded on both sides, e.g. `font-size: clamp(1rem, 0.8rem + 1vw, 1.5rem)`. They cut out many media queries.

→ *More:* §10.7 Fluid typography with `clamp()`, §14.6 Q37

Positioning, `z-index`, stacking contexts, floats and BFC

Q125. ⋆ Explain `static`, `relative`, `absolute`, `fixed` and `sticky` positioning.

`static` (the default): normal flow, offsets (`top`, `left`…) ignored. `relative`: stays in the flow and keeps its space, but is shifted visually by offsets; also becomes the containing block for absolute children. `absolute`: removed from the flow, placed relative to the nearest **positioned** ancestor (any value except `static`) or the initial containing block. `fixed`: removed from the flow, placed relative to the viewport — stays during scroll. `sticky`: behaves like `relative` until the scroll position crosses a threshold (`top: 0`), then sticks within its parent's box.

Follow-up: Does `position: relative` with no offsets do anything? Visually nothing, but it creates a containing block for absolute children and lets `z-index` apply.

→ *More:* §7.6 Positioning, §14.6 Q25

Q126. What is the containing block for an absolutely positioned element?

The padding box of the nearest ancestor whose `position` isn't `static` — or that has a `transform`, `filter`, `perspective`, `contain: layout/paint`, or `will-change` for those. If there is none, the **initial containing block** (the viewport-sized box at the document's start). That's why the usual pattern is `position: relative` on the parent, `position: absolute` on the child.

→ *More:* §7.6 Positioning

Q127. ⋆ Why doesn't my `position: sticky` work?

Common causes: no threshold set (`top: 0` is required); an ancestor has `overflow: hidden`, `auto` or `scroll`, which makes *it* the scrolling container to stick within; the parent is no taller than the sticky element, so there's no room to stick; or in a flex/grid container the item is stretched to full height (add `align-self: start`).

→ *More:* §7.6 Positioning

Q128. Why is my `position: fixed` element scrolling with the page (or positioned wrongly)?

An ancestor has a `transform`, `filter`, `perspective`, `backdrop-filter`, `contain` or `will-change: transform`. Those make the ancestor the containing block for fixed descendants, so "fixed" becomes relative to that box. Move the element outside that ancestor, remove the property, or use the top layer (`<dialog>`, `popover`).

→ *More:* §7.6 Positioning

Q129. ⋆ Is there any reason you'd use `translate()` instead of absolute positioning, or vice versa?

`translate()` moves an element visually **without** changing layout: its original space is kept, nothing reflows, and the change can be done on the GPU compositor — smooth for animation. Changing `top`/`left` triggers layout each frame. Use absolute positioning to *place* something in layout (a badge in a corner, taken out of the flow); use `transform: translate()` to *move or animate* it. Note that a transform creates a stacking context and a containing block for fixed children.

→ *More:* §12.3 2D transforms, §12.7 Performance: animate `transform` and `opacity`

Q130. ⋆ Describe `z-index` and how a stacking context is formed.

`z-index` sets the stacking order of positioned elements (and flex/grid items) along the z-axis. It only compares elements **within the same stacking context** — a group painted as one layer. A new stacking context is created by: the root element; positioned elements with a `z-index` other than `auto`; `position: fixed` or `sticky`; flex/grid items with a `z-index`; `opacity` below 1; `transform`, `filter`, `backdrop-filter`, `perspective`, `clip-path`, `mask`; `mix-blend-mode` other than `normal`; `isolation: isolate`; `contain: paint`/`layout`; `will-change` for any of those; and elements in the top layer.

Follow-up: Does `z-index` work on a flex item without `position`? Yes — flex and grid items accept `z-index` even when they're `position: static`.

→ *More:* §7.8 `z-index` and stacking contexts

Q131. ⋆ I set `z-index: 9999` on a modal and it still appears behind the header. Why?

The modal is inside a parent that forms its own stacking context (for example, a parent with `transform`, `opacity < 1` or `position: relative; z-index: 1`). Everything inside that parent is stacked as one unit, so 9999 only beats its siblings inside the parent; the parent as a whole sits below the header. Fix: raise the *ancestor's* z-index, remove the property that created the context, render the modal at the end of `<body>` (a "portal"), or use `<dialog>`/`popover`, which go into the **top layer** above every stacking context.

Follow-up: How do you debug it quickly? In DevTools, walk up the modal's ancestors and look for `transform`, `opacity`, `filter`, `will-change`, `isolation`, or `position` + `z-index`; the Layers panel and the 3D view also show stacking contexts.

→ *More:* §7.8 `z-index` and stacking contexts, §14.6 Q26

Q132. Without any `z-index`, in what order are elements painted?

Within one stacking context, back to front: the context's own background and borders; children with negative `z-index`; in-flow block boxes; floats; inline content; positioned descendants with `z-index: auto` or `0` in DOM order; then positive `z-index` children. So a later positioned element covers an earlier one, and any positioned element covers non-positioned ones.

→ *More:* §7.8 `z-index` and stacking contexts

Q133. ⋆ Describe floats and how they work.

`float: left|right` takes a box out of normal flow and pushes it to the side of its container; following **inline content (text) wraps around it**, but block boxes behave as if it weren't there (their backgrounds slide underneath). The parent doesn't grow to contain floated children ("collapses"). Floats were used for whole page layouts before flexbox and grid; today they are for what they were designed for — wrapping text around an image (optionally with `shape-outside`).

Follow-up: What does `shape-outside` add? It lets text wrap around a non-rectangular shape — `shape-outside: circle(50%)` on a floated round image.

→ *More:* §7.9 Floats (legacy)

Q134. ⋆ What are the various clearing techniques, and which is appropriate when?

`clear: both` on an element after the floats moves it below them. To make a parent contain its floats: `display: flow-root` on the parent (the modern, side-effect-free choice — it creates a BFC); `overflow: auto`/`hidden` (also a BFC but may clip shadows or show scrollbars); or the legacy **clearfix** — a `::after` pseudo-element with `content: ""; display: table; clear: both;`. In new code use `flow-root` — or don't use floats for layout at all.

→ *More:* §7.9 Floats (legacy)

Q135. ⋆ What is a block formatting context (BFC), and how is one created?

A BFC is an independent layout region in which block boxes are laid out vertically, isolated from the outside. Inside it: floats are contained (the parent grows around them), margins of children don't collapse through it, and the BFC box doesn't overlap floats outside it. Created by: the root `<html>`, `display: flow-root`, floats, absolute/fixed positioning, `inline-block`, table cells, flex and grid **items**, `overflow` other than `visible`/`clip`, `contain: layout`/`paint`, and multi-column containers.

Follow-up: Give a concrete reason you created a BFC recently: to stop a child's top margin leaking out of its parent card, or to stop text from wrapping under a floated image (making the text column a BFC).

→ *More:* §7.5 Normal flow and formatting contexts, §14.6 Q27

Q136. ⋆ What is margin collapsing? When does it happen and how do you stop it?

Vertical margins of block boxes in normal flow can combine into one margin equal to the larger of them. With a negative margin involved, the largest positive and the most negative are added (30px and −10px → 20px); if both are negative, the most negative wins. It happens between **adjacent siblings** (20px + 30px → 30px), between a **parent and its first/last child** when nothing separates them (the child's top margin "leaks out" of the parent), and through **empty blocks**. It never happens horizontally, nor in flex or grid containers. Stop it with padding or a border on the parent, `display: flow-root`, or by using flex/grid with `gap`.

Follow-up: Why does a heading's top margin push the whole card down instead of creating space inside it? The heading's margin collapsed through the card's top edge. Give the card padding or `display: flow-root`.

→ *More:* §7.3 Margin collapsing, §14.6 Q24

Q137. What is `overflow`, and what is the difference between `hidden`, `clip`, `auto` and `scroll`?

It controls content that doesn't fit its box. `visible` (default) lets it spill out. `hidden` clips it but the box is still a scroll container (scrollable by script, creates a BFC, breaks `position: sticky` inside). `clip` clips without creating a scroll container (sticky still works). `auto` shows scrollbars only when needed; `scroll` always reserves them.

→ *More:* §7.7 Overflow

Q138. How do you truncate text with an ellipsis — single line and multi-line?

```
.one-line { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
.three-lines { display: -webkit-box; -webkit-box-orient: vertical;
               -webkit-line-clamp: 3; line-clamp: 3; overflow: hidden; }
```

The single-line version needs a constrained width (and `min-width: 0` on a flex item). The multi-line version uses the prefixed `-webkit-line-clamp`, which works in all major browsers.

→ *More:* §11.3 Text properties

Flexbox

Q139. ⋆ Explain flexbox and its main properties.

Flexbox is a **one-dimensional** layout mode: `display: flex` on a container lays its direct children (flex items) out along a **main axis** (a row by default) and aligns them on the **cross axis**. Container properties: `flex-direction`, `flex-wrap`, `justify-content` (main axis), `align-items` and `align-content` (cross axis), `gap`. Item properties: `flex-grow`, `flex-shrink`, `flex-basis` (shorthand `flex`), `align-self`, `order`.

→ *More:* §8.1 Flex container and flex items, §8.2 The two axes, `flex-direction` and `order`

Q140. ⋆ What does `flex: 1` mean? What about `flex: auto`, `flex: none` and `flex: 0 0 200px`?

`flex` is shorthand for grow, shrink, basis. `flex: 1` = `1 1 0%`: start from zero and share *all* space equally — equal-width columns. `flex: auto` = `1 1 auto`: start from content size and share the *leftover* space, so items with more content end up wider. `flex: none` = `0 0 auto`: rigid, content-sized. `flex: 0 0 200px`: exactly 200 px, never grows or shrinks.

Follow-up: Why might `flex: 1` columns still be unequal? Because of the automatic minimum size: an item with a long unbreakable word can't shrink below it. Add `min-width: 0`.

→ *More:* §8.4 The `flex` shorthand and minimum sizes, §14.6 Q28

Q141. Explain `flex-grow`, `flex-shrink` and `flex-basis`.

`flex-basis` is the item's starting size on the main axis (`auto` = use `width`/content). If there is free space left, it's shared in proportion to `flex-grow`. If items overflow, the overflow is removed in proportion to `flex-shrink` **multiplied by each item's basis** (so bigger items shrink more). Items never shrink below their minimum size (see the next question).

→ *More:* §8.3 `flex-grow`, `flex-shrink` and `flex-basis`

Q142. ⋆ Why does a flex item with long text (or a long URL, or a `<pre>`) overflow instead of shrinking?

Flex items have `min-width: auto` by default, which means "don't shrink below my content's minimum width" — the longest word or the unbreakable string. Set `min-width: 0` (or `min-height: 0` in a column) on the item, and add `overflow-wrap: anywhere` or `text-overflow: ellipsis` if the text itself must break. Grid items have the same rule, which is why `1fr` columns can blow out too.

Follow-up: Does `overflow: hidden` on the item also fix it? Yes — a scroll container's automatic minimum size is 0. `overflow: clip` does not, because it doesn't create a scroll container.

→ *More:* §8.4 The `flex` shorthand and minimum sizes, §14.6 Q29

Q143. `justify-content` vs `align-items` vs `align-content`?

`justify-content` distributes items along the **main** axis (`flex-start`, `center`, `space-between`, `space-around`, `space-evenly`). `align-items` aligns items on the **cross** axis within their line (`stretch` — the default, `center`, `flex-start`, `baseline`). `align-content` distributes the **lines** themselves when wrapping produces several lines; it has no effect on a single-line container.

→ *More:* §8.5 Alignment

Q144. Why doesn't `justify-content: center` work in my `flex-direction: column` container?

Changing the direction swaps the axes: in a column, `justify-content` acts vertically and `align-items` horizontally. Also, vertical centering needs the container to have a height taller than its content.

→ *More:* §8.2 The two axes, `flex-direction` and `order`, §8.5 Alignment

Q145. How do you push one flex item to the far end (for example, a "Log out" link to the right)?

Give it `margin-left: auto` (or `margin-inline-start: auto`). In a flex container, leftover space is handed to `auto` margins before `justify-content` runs, so that margin grows as large as possible and shoves the item — and everything after it — to the end. No wrapper needed.

→ *More:* §8.5 Alignment

Q146. What does `gap` do, and why is it better than margins between items?

`gap` (`row-gap`, `column-gap`) adds space only *between* flex or grid items — never at the outer edges — so you don't need `:last-child { margin: 0 }` hacks, and it works correctly when items wrap. Supported in flexbox in all browsers since 2021.

→ *More:* §8.6 Wrapping and `gap`

Q147. Why is using `order` (or `flex-direction: row-reverse`) an accessibility concern?

They change only the *visual* order. Keyboard Tab order and screen-reader reading order still follow the DOM, so focus jumps around the screen unpredictably — WCAG 1.3.2 (Meaningful Sequence) and 2.4.3 (Focus Order). Change the DOM order instead when the order matters.

→ *More:* §8.2 The two axes, `flex-direction` and `order`

Q148. Why does the last row of my wrapping flex layout look wrong?

With `flex-wrap: wrap` and `flex-grow`, each line is sized independently, so a lonely last item stretches to full width (or `space-between` spreads two items to the far edges). Flexbox has no columns to align to. If items must line up in columns, use grid (`repeat(auto-fill, minmax(…))`).

→ *More:* §8.6 Wrapping and `gap`

CSS Grid

Q149. ⋆ Explain CSS Grid's main concepts.

Grid is a **two-dimensional** layout: `display: grid` defines **tracks** (rows and columns, with `grid-template-columns` / `grid-template-rows`), separated by numbered **lines**; items are placed into **cells** or **areas** spanning several cells. `gap` adds gutters. Items can be placed explicitly (`grid-column: 1 / 3`, `grid-area: header`) or flow automatically (auto-placement).

→ *More:* §9.1 Grid vocabulary, §9.2 Defining tracks

Q150. What is the `fr` unit? How is it different from `%`?

`fr` is a fraction of the **free space left after** fixed tracks and gaps are subtracted: `200px 1fr 2fr` gives the second column a third and the third column two thirds of what remains. `%` is relative to the whole container and ignores gaps, so `50% 50%` plus a `gap` overflows, while `1fr 1fr` doesn't.

Follow-up: What does `fr` do when the content is bigger than the share? The track grows to its minimum content size (see the `1fr` overflow question), so tracks become unequal unless you use `minmax(0, 1fr)`.

→ *More:* §9.2 Defining tracks

Q151. Why does my `1fr` column overflow when it contains a long word or a wide image?

`1fr` really means `minmax(auto, 1fr)` — the minimum is the content's minimum size, so the column refuses to shrink below the longest unbreakable item. Use `minmax(0, 1fr)` (or `min-width: 0` on the item) to allow it to shrink.

→ *More:* §9.2 Defining tracks

Q152. ⋆ `auto-fill` vs `auto-fit`?

Both create as many columns as fit in `repeat(auto-fill|auto-fit, minmax(200px, 1fr))`. With `auto-fill`, leftover empty tracks are kept, so a few items stay at their minimum width with empty space after them. With `auto-fit`, empty tracks collapse to zero and the `1fr` lets the existing items stretch to fill the row. They only look different when there are fewer items than columns.

Follow-up: Which is better for a product grid that is sometimes nearly empty (one or two results)? `auto-fill`: items keep a consistent card width. `auto-fit` would stretch a single result across the whole row. (A lone item in the last row of a *full* grid looks the same with both, because the tracks already exist.)

→ *More:* §9.3 `repeat()`, `auto-fill` and `auto-fit`, §14.6 Q32

Q153. How do you build a responsive card grid with no media queries?

```
.cards {
  display: grid; gap: 1rem;
  grid-template-columns: repeat(auto-fill, minmax(min(16rem, 100%), 1fr));
}
```

The browser fits as many 16rem columns as it can and shares the leftover space; `min(16rem, 100%)` stops a single column from overflowing on very narrow screens. This "RAM" pattern (Repeat, Auto, Minmax) is one of the most common interview tasks.

→ *More:* §9.9 Patterns, §9.3 `repeat()`, `auto-fill` and `auto-fit`

Q154. What is `grid-template-areas`?

A way to name regions with an ASCII-art map and place items by name:

```
.page { display: grid; grid-template-columns: 200px 1fr;
  grid-template-areas: "head head" "nav main" "foot foot"; }
header { grid-area: head; } nav { grid-area: nav; } main { grid-area: main; } footer { grid-area: foot; }
```

Changing the layout at a breakpoint is just redefining the map. Each area must be a rectangle; `.` marks an empty cell.

→ *More:* §9.4 Placing items

Q155. What are the explicit and implicit grid?

The **explicit grid** is the tracks you defined with `grid-template-*`. When items are placed outside it, or there are more items than cells, the browser adds **implicit** tracks, sized by `grid-auto-rows` / `grid-auto-columns` (default `auto`). `grid-auto-flow: dense` lets auto-placement backfill holes (but changes visual order).

→ *More:* §9.5 The implicit grid and auto-placement

Q156. What is subgrid, and what problem does it solve?

`grid-template-columns: subgrid` (or rows) on a grid item makes its own children use the **parent's** tracks instead of new ones. Classic use: a row of cards where each card's title, body and button line up across cards even though their content lengths differ. Baseline since September 2023.

→ *More:* §9.7 Subgrid, §14.6 Q33

Q157. ⋆ What is the difference between Flexbox and Grid? When would you use one over the other?

Flexbox is **one-dimensional** and **content-out**: items size themselves from their content, and you control one row (or column) at a time — navbars, toolbars, a row of tags, centring one thing. Grid is **two-dimensional** and **layout-in**: you define the rows and columns, then place content into them — page layouts, card galleries, forms with aligned labels. They combine: a grid page whose header uses flex.

Follow-up: Can grid do what flexbox does? Mostly, but flexbox is simpler when items should size to their content and wrap naturally (tags, buttons); grid is better when items must line up in both directions.

→ *More:* §9.8 Grid vs Flexbox, §14.6 Q31

Q158. Have you used a grid system (Bootstrap's 12 columns, etc.)? What do you prefer now?

Framework grid systems used floats, then flexbox, with classes like `col-md-6`. Native CSS Grid makes most of them unnecessary: `grid-template-columns: repeat(12, 1fr)` gives a 12-column grid in one line, and `auto-fit/minmax` patterns remove many breakpoints. Design systems still often keep a column grid for consistent spacing.

→ *More:* §9.9 Patterns

Responsive design: media queries, container queries and fluid sizing

Q159. What is responsive design?

One codebase whose layout adapts to any screen size and input method, using fluid layouts (flex, grid, percentages), flexible media (`max-width: 100%`, `srcset`), and media or container queries. The core ideas came from Ethan Marcotte (2010): fluid grids, flexible images and media queries.

→ *More:* §10.1 What responsive design is

Q160. ⋆ What is the difference between responsive and mobile-first — and between fixed, fluid, adaptive and responsive layouts?

Mobile-first is a *strategy for writing* responsive CSS: base styles target small screens, and `min-width` media queries add layout for larger ones (simpler CSS, phones download less override CSS). **Fixed** layouts have a set pixel width; **fluid** use percentages and stretch; **adaptive** serve a few fixed designs chosen at breakpoints; **responsive** are fluid *and* use breakpoints to rearrange.

→ *More:* §10.1 What responsive design is, §10.4 Mobile-first in practice, §14.6 Q35

Q161. How do media queries work? How do you choose breakpoints?

`@media (min-width: 48em) { … }` applies rules when the condition is true (here, viewport at least 48em wide). Media Queries Level 4 adds range syntax: `@media (48em <= width < 64em)`. Choose breakpoints where **your content** breaks (a line gets too long, a nav no longer fits), not at specific device sizes. `em` breakpoints respond to the user's font-size setting.

→ *More:* §10.3 Media queries

Q162. ⋆ What are container queries, and how do they differ from media queries?

Media queries react to the **viewport**. Container queries react to the size of an **ancestor container**, so a component adapts to the space it's given — the same card is horizontal in a wide main column and stacked in a narrow sidebar.

```
.card-wrap { container-type: inline-size; }
@container (min-width: 30rem) { .card { display: grid; grid-template-columns: 10rem 1fr; } }
```

Size queries are Baseline since 2023; container units like `cqi` (1% of the container's inline size) go with them.

Follow-up: Why does a container need `container-type`? Querying a size requires the container's size not to depend on its children (size containment), otherwise there'd be a loop. `inline-size` contains only the width, so height can still grow with content.

→ *More:* §10.6 Container queries, §14.6 Q36

Q163. What are style queries?

`@container style(--variant: compact) { … }` applies styles when the container's custom property has a given value — a way to theme or switch component variants from a parent. Style queries on custom properties became Baseline in 2026.

→ *More:* §10.6 Container queries

Q164. How does `clamp()` create fluid typography?

`font-size: clamp(1rem, 0.75rem + 1.2vw, 1.75rem)` scales smoothly with the viewport between a minimum and a maximum, with no breakpoints. Keep a `rem` part in the middle value: pure `vw` font sizes don't grow when the user zooms, which fails WCAG 1.4.4.

→ *More:* §10.7 Fluid typography with `clamp()`, §14.6 Q37

Q165. How do you make images and videos responsive?

`img, video { max-width: 100%; height: auto; }` so they shrink to fit their container; `srcset`/`sizes` or `<picture>` so the browser downloads an appropriately sized file; `aspect-ratio` plus `object-fit: cover` for fixed-ratio frames; `image-set()` for CSS backgrounds; and for embeds (YouTube iframes) `width: 100%; aspect-ratio: 16 / 9`.

→ *More:* §10.8 Responsive images recap (and `image-set()`), §4.3 Responsive images — `srcset`, `sizes` and `<picture>`

Q166. What are user-preference media queries?

Media features that reflect the user's OS or browser settings: `prefers-color-scheme` (light/dark), `prefers-reduced-motion`, `prefers-contrast`, `forced-colors` (Windows high contrast), `prefers-reduced-transparency`. Honour them to respect users' needs.

→ *More:* §10.5 User-preference media queries

Q167. ⋆ How do you implement dark mode that respects the user's preference but allows a manual override?

Define semantic colour tokens (`--surface`, `--text`) and give them light and dark values — with `color-scheme: light dark` plus `light-dark(#fff, #111)`, or with `@media (prefers-color-scheme: dark)`. A toggle sets `data-theme="dark|light"` on `<html>` (which pins `color-scheme`) and saves the choice in `localStorage`; a tiny blocking script in `<head>` applies the saved theme before first paint to avoid a white flash.

Follow-up: Why not just invert colours with `filter: invert(1)`? It inverts images and brand colours too, can't respect contrast design, and costs paint performance. Design dark tokens deliberately (lighter, desaturated accents; surfaces not pure black).

→ *More:* §13.14 Theming and dark mode with custom properties, §10.5 User-preference media queries, §14.6 Q42

Q168. How do you test responsive design?

DevTools device mode at many widths (drag through every width, not just presets), real phones (touch, mobile Safari quirks), 200% and 400% zoom and the 320 CSS px reflow test (WCAG 1.4.10 — no horizontal scrolling), larger default font size, landscape orientation, and an automated check that `document.documentElement.scrollWidth` doesn't exceed the viewport.

→ *More:* §10.10 Testing responsive design

Q169. ⋆ My page has horizontal scrolling on mobile. How do you find and fix it?

Find the culprit: in DevTools run `[...document.querySelectorAll('body *')].filter(e => e.getBoundingClientRect().right > document.documentElement.clientWidth)` (lists the elements that stick out past the right edge) or add `* { outline: 1px solid red }`. Usual causes: fixed widths (`width: 600px`), `100vw` (which includes the scrollbar width), images without `max-width: 100%`, long URLs or words (`overflow-wrap: anywhere`), grid/flex items with `min-width: auto` (use `minmax(0, 1fr)` or `min-width: 0`), and negative margins. Don't hide it with `overflow-x: hidden` on `body` — that can break `position: sticky`.

→ *More:* §10.10 Testing responsive design, §9.10 Debugging, pitfalls and interview prep

Q170. What is the `aspect-ratio` property?

`aspect-ratio: 16 / 9` makes a box keep a width-to-height ratio as its width changes — for video embeds, thumbnails and avatars (`aspect-ratio: 1`). It replaced the padding-top percentage hack, and on images it

reserves space to avoid layout shift.

→ *More:* §4.1 The `<img>` element, §11.9 `object-fit` and `object-position`

Centering techniques

Q171. ⋆ How do you center a `<div>` horizontally and vertically?

The two modern answers:

```
.parent { display: grid; place-items: center; min-height: 100svh; }
/* or */
.parent { display: flex; justify-content: center; align-items: center; }
```

The parent needs a height for vertical centering to be visible.

Follow-up: Why not `position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%)`? It works, but the element leaves the flow (the parent won't size around it) and can land on half pixels, blurring text. Grid or flex is simpler.

→ *More:* §8.7 Common patterns, §14.6 Q30

Q172. Name other ways to center something — and when each fits.

- Horizontally, a block with a width: `margin-inline: auto` (`margin: 0 auto`).
- Inline content or text: `text-align: center` on the parent.
- Positioned: `position: absolute; inset: 0; margin: auto;` with a set width and height (or `width: fit-content; height: fit-content` for unknown sizes), or `top: 50%; left: 50%; transform: translate(-50%, -50%)` (can blur on half pixels).
- One child in a flex container: `margin: auto` on the child.
- Vertically in a single line of text: `line-height` equal to the height (legacy).
- In normal block flow (no flex/grid): `align-content: center` on a block container works in all major browsers since 2024.

→ *More:* §7.5 Normal flow and formatting contexts, §8.5 Alignment, §9.6 Alignment

Q173. How do you center a modal that should scroll when its content is taller than the viewport?

Use `<dialog>` (which is centered by default) or a fixed overlay with grid centering, and cap the height: `max-height: calc(100svh - 2rem); overflow: auto;` on the dialog. With `align-items: center` and overflowing content, the top can be cut off unreachable; `margin: auto` on the child (or `align-items: safe center`) avoids that, because auto margins never push content past the top edge.

→ *More:* §8.5 Alignment, §3.7 Keyboard and focus

Transitions, transforms, animations and visual effects

Q174. ⋆ Transitions vs animations — what's the difference and when do you use each?

A **transition** animates a property between two states when its value changes (on `:hover`, a class toggle): `transition: transform 200ms ease-out`. It needs a trigger and has only a start and end. A **keyframe animation** (`@keyframes` + `animation`) runs on its own, can loop, alternate, and have many intermediate steps. Use transitions for state changes in UI; animations for loaders, attention cues and multi-step sequences.

Follow-up: What events can you listen for? `transitionend`, `transitionstart`, `transitionrun`, `transitioncancel`, and `animationstart`, `animationiteration`, `animationend`, `animationcancel`.

→ *More:* §12.1 Transitions, §12.5 Keyframe animations

Q175. Transforms vs transitions — and how do they affect performance?

A **transform** changes how a box is drawn — `translate`, `rotate`, `scale`, `skew` — without affecting layout. A **transition** is *how* a change is animated over time. Transforms are cheap to animate because they skip layout and paint; transitioning layout properties (`width`, `top`, `margin`) forces layout on every frame.

→ *More:* §12.3 2D transforms, §12.7 Performance: animate `transform` and `opacity`

Q176. ⋆ Which properties are cheap to animate, and why?

`transform` and `opacity` (and `filter` in many cases). When the element is on its own compositor layer, the browser can change them in the **composite** step on the GPU (graphics processor), without recalculating layout or repainting, and even when the main thread is busy. Animating `width`, `height`, `top`, `left`, `margin` triggers layout + paint every frame; `background-color` and `box-shadow` trigger paint.

Follow-up: Why can `opacity` still be expensive? If the element isn't promoted to its own layer, changing it repaints; and a very large layer (full-screen blur) costs memory and GPU fill time.

→ *More:* §12.7 Performance: animate `transform` and `opacity`, §14.6 Q39

Q177. ⋆ My animation is janky. How do you optimise it?

Record it in DevTools' Performance panel and look for long layout/paint tasks and dropped frames. Animate only `transform` / `opacity` (e.g. `translate` instead of `left`, `scale` instead of `width`); use `will-change: transform` sparingly, just before the animation, to promote a layer; avoid layout thrashing in JS (reading `offsetHeight` right after writing styles in a loop); use `requestAnimationFrame` or the Web Animations API instead of `setInterval`; reduce the painted area and expensive effects like big blurs.

→ *More:* §12.7 Performance: animate `transform` and `opacity`

Q178. What does `will-change` do, and why not put it everywhere?

It hints that a property will change soon so the browser can prepare (usually promoting the element to its own GPU layer). Each layer costs memory, and too many layers slow everything down; it also creates a stacking context and a containing block. Add it shortly before an animation and remove it afterwards — or only on elements that animate constantly.

→ *More:* §12.7 Performance: animate `transform` and `opacity`

Q179. What are timing functions (easing)?

They control the speed curve of a transition or animation: `linear`, `ease` (default), `ease-in` (slow start — for things leaving), `ease-out` (slow end — for things entering, feels responsive), `ease-in-out`, `cubic-bezier(x1, y1, x2, y2)` for custom curves, `steps(n)` for frame-by-frame sprites, and `linear()` with many points to approximate springs and bounces.

→ *More:* §12.2 Timing functions (easing)

Q180. Does the order of transform functions matter?

Yes. Transforms apply right to left relative to the element's coordinate system, so `translateX(100px) rotate(45deg)` and `rotate(45deg) translateX(100px)` end in different places (the second moves along the rotated axis). The individual properties `translate`, `rotate`, `scale` are always applied in that fixed order and can be animated independently.

→ *More:* §12.3 2D transforms

Q181. How do you build a 3D flip card?

A container with `perspective: 1000px`; an inner element with `transform-style: preserve-3d` and `transition: transform .6s`; two faces stacked with `position: absolute` and `backface-visibility: hidden`, the back pre-rotated `rotateY(180deg)`; on hover or a class, rotate the inner element `rotateY(180deg)`. `overflow: hidden` or `opacity` / `filter` on the inner element flatten the 3D, so avoid them there.

→ *More:* §12.4 3D transforms

Q182. Why can't you transition from `display: none`, and how do you animate an element in and out today?

`display` is a discrete property: the element doesn't exist in the "from" state, so there's nothing to transition from. Modern CSS fixes this with `@starting-style { … }` (the style to transition *from* when the element first appears) plus `transition-behavior: allow-discrete` (keeps the element displayed until the exit transition finishes). Works well for `<dialog>`, `popover` and elements added by JS. Both are Baseline since August 2024, but the *exit* animation of `display` (and the `overlay` property that keeps a closing dialog in the top layer) is not in Firefox yet — there the element simply disappears without the fade, which is a harmless fallback.

→ *More:* §12.8 Enter and exit animations: `@starting-style` and `allow-discrete`

Q183. What are view transitions and scroll-driven animations?

View transitions (`document.startViewTransition(() => updateDOM())`) animate between two DOM states — the browser snapshots old and new and cross-fades or morphs elements with the same `view-transition-name`. Same-document ones are Baseline since October 2025; cross-document (between pages, opted in with `@view-transition { navigation: auto }`) is Chrome and Safari only. **Scroll-driven animations** (`animation-timeline: scroll()` or `view()`) tie animation progress to scroll position instead of time — reading-progress bars, reveal-on-scroll — without JavaScript; not yet Baseline.

→ *More:* §12.10 View transitions, §12.11 Scroll-driven animations

Q184. CSS animations or JavaScript animations?

CSS (transitions/keyframes) for simple, declarative state changes — the browser can run `transform`/`opacity` off the main thread. The Web Animations API (`el.animate(keyframes, options)`) gives the same engine with JS control: pause, reverse, seek, promises when finished. Libraries (GSAP, Motion) for complex timelines, physics and SVG morphing. `requestAnimationFrame` for canvas or custom per-frame logic.

→ *More:* §12.6 The Web Animations API (WAAPI)

Q185. How would you implement a web design that uses non-standard fonts?

Self-host WOFF2 files with `@font-face`, set `font-display: swap` (show fallback text immediately, swap when the font arrives) or `optional` (use the font only if it arrives very fast), `preload` the one or two most important files, subset to the characters you need, consider a variable font instead of many weights, and tune the fallback with `size-adjust`/`ascent-override` to reduce layout shift. Always give a full fallback stack ending in a generic family. Two terms interviewers use: **FOIT** (flash of invisible text — text hidden while the font loads, the default for up to ~3 s) and **FOUT** (flash of unstyled text — fallback first, then a swap that can shift layout); `font-display` chooses between them.

→ *More:* §11.1 Fonts, §11.2 Loading fonts well: `font-display`, preload, subsetting, §14.6 Q38

Q186. How do `object-fit` and `object-position` work?

They control how a replaced element's content (an image or video) fills its box when the ratio differs: `cover` fills and crops, `contain` fits entirely with letterboxing, `fill` (default) stretches, `none` keeps natural size, `scale-down` picks the smaller of `none` and `contain`. `object-position` chooses which part stays visible (`top center`).

→ *More:* §11.9 `object-fit` and `object-position`

Q187. How do you make a gradient border, a glassmorphism card, or gradient text?

Gradient border: two backgrounds — `background: linear-gradient(#fff,#fff) padding-box, linear-gradient(90deg,#f0f,#0ff) border-box; border: 3px solid transparent;`. Glass: semi-transparent background + `backdrop-filter: blur(12px)` (check text contrast). Gradient text: `background: linear-gradient(…); background-clip: text; -webkit-background-clip: text; color: transparent;` (keep a fallback colour).

→ *More:* §11.5 Backgrounds and gradients, §11.8 Filters, blend modes, clipping and masks

---

Custom properties and modern CSS

Q188. How do CSS custom properties (variables) work?

Declared with two dashes — `:root { --brand: #1d4ed8; }` — and read with `var(--brand, fallback)`. They follow the cascade and **inherit**, so you can redefine them on any element or in a media query and all descendants update; JavaScript can change them at runtime (`el.style.setProperty('--x', '10px')`). An invalid value at use time makes the property behave as `unset` ("invalid at computed-value time"), not fall back to an earlier declaration.

Follow-up: How do you set a variable from JavaScript and read it back? `el.style.setProperty('--gap', '2rem')` and `getComputedStyle(el).getPropertyValue('--gap')`.

→ *More:* §6.9 Custom properties (CSS variables)

Q189. ⋆ What are the advantages of CSS variables over Sass variables?

Sass variables are replaced at **compile time** and disappear — they can't change per element, per media query (without duplication) or at runtime. CSS custom properties are **live** in the browser: they cascade, inherit, respond to media/container queries and JavaScript, and power theming (dark mode is just redefining tokens). Sass still helps with loops and mixins.

→ *More:* §6.9 Custom properties (CSS variables), §13.12 Preprocessors (Sass) and PostCSS, §14.6 Q19

Q190. What does `@property` add to custom properties?

`@property --angle { syntax: '<angle>'; inherits: false; initial-value: 0deg; }` registers a *typed* custom property. Because the browser now knows it's an angle (not just text), it can **animate** it — e.g. a rotating conic-gradient border — and validate it. Baseline since July 2024.

→ *More:* §13.14 Theming and dark mode with custom properties

Q191. ⋆ What is the `:has()` selector, and what problems does it solve?

`:has()` selects an element based on what it contains or what follows it — the long-wanted "parent selector": `.card:has(img)` (cards with an image), `form:has(:user-invalid) .submit` (style the button when any field is invalid), `h2:has(+ p)` (an h2 followed by a paragraph — a previous-sibling selector), `body:has(dialog[open])` (lock scrolling while a dialog is open). It removes a lot of JavaScript class-toggling. Baseline since December 2023.

Follow-up: Is `:has()` slow? Browsers optimise common cases well; avoid very broad anchors like `body:has(.x)` that must be re-checked on every DOM change in big documents, but normal use is fine.

→ *More:* §13.3 `:has()` — the relational ("parent") selector, §14.6 Q21

Q192. ⋆ `:is()` vs `:where()`?

Both take a selector list and match any of them, shortening long selectors (`:is(h1, h2, h3) a`). The difference is specificity: `:is()` takes its most specific argument's, `:where()` is always zero — use `:where()` in resets and library defaults so users can override them with a single class. Both are "forgiving": one invalid selector in the list doesn't kill the whole rule.

→ *More:* §13.2 `:is()` and `:where()`, §14.6 Q20

Q193. ⋆ What are cascade layers (`@layer`), and how do they help manage CSS at scale?

`@layer reset, vendor, base, components, utilities;` declares named layers in priority order (later wins). The cascade compares **layers before specificity**, so a simple `.btn` in `components` beats `#app .vendor .btn` in `vendor`. They end specificity wars with third-party CSS and make the order of your codebase explicit. Unlayered styles beat all layered ones; for `!important` the layer order reverses. Baseline since 2022.

Follow-up: What happens to styles not in any layer? They form an implicit final layer that beats all named layers for normal declarations — so wrapping third-party CSS in a low layer lets all your unlayered styles win.

→ *More:* §13.4 Cascade layers (`@layer`), §14.6 Q22

Q194. Is CSS nesting native now? How does it differ from Sass nesting?

Yes — native nesting is Baseline since late 2023: `.card { & h2 { … } &:hover { … } }`. Differences from Sass: nested selectors behave like they're wrapped in `:is()` (so specificity follows `:is()` rules), and you can't concatenate strings to build class names (`&__title` works in Sass, not in CSS). Don't nest deeply — it increases specificity and couples CSS to the DOM.

→ *More:* §13.1 Native CSS nesting

Q195. What is `@scope`?

`@scope (.card) to (.card-content) { img { … } }` limits rules to a subtree, with an optional lower boundary ("donut scope") so styles don't leak into nested components. When two scoped rules conflict, the one whose scope root is **closer** wins (scope proximity). Baseline newly available since March 2026.

→ *More:* §13.5 `@scope`

Q196. ⋆ What are CSS logical properties, and why are they important?

Properties that use flow-relative directions instead of physical ones: `margin-inline-start` instead of `margin-left`, `padding-block` instead of top/bottom, `inset-inline-end`, `inline-size` instead of `width`. They adapt automatically to right-to-left languages and vertical writing modes, so one stylesheet works for Arabic and English.

→ *More:* §13.6 Logical properties

Q197. What is `@supports`?

A feature query: `@supports (display: grid) { … }` or `@supports selector(:has(a)) { … }` applies rules only if the browser supports the property-value pair or selector. It's the CSS tool for progressive enhancement; `not`, `and` and `or` combine conditions.

→ *More:* §10.6 Container queries

Q198. What is CSS anchor positioning?

A native way to tether one element to another — a tooltip or dropdown positioned next to its button without JavaScript measurement. The anchor gets `anchor-name: --btn`; the floating element gets `position: absolute; position-anchor: --btn; position-area: bottom;` (or `top: anchor(bottom)`), and `position-try-fallbacks: flip-block` flips it when there's no room. Its core (`anchor-name`, `anchor()`, `position-area`, `position-try-fallbacks`) became Baseline newly available in January 2026 when Firefox 147 shipped it; a few newer extras (such as some `position-visibility` values) are still Safari-only, so check before using them. It pairs well with the `popover` attribute.

CSS architecture: preprocessors, BEM, utility-first and CSS-in-JS

Q199. ⋆ What are the advantages and disadvantages of CSS preprocessors (Sass, Less)?

Pros: variables, nesting, mixins (reusable blocks with parameters), functions and loops, partials combined into one file, colour maths. Cons: a build step, compiled output can get bloated (deep nesting, `@extend`), and debugging needs source maps. Much of what made Sass essential is now native (custom properties, nesting, `color-mix()`, `@layer`); many teams now use **PostCSS** (a plugin pipeline — Autoprefixer, minification) and plain modern CSS.

→ *More:* §13.12 Preprocessors (Sass) and PostCSS

Q200. What is BEM?

A naming convention — **Block, Element, Modifier**: `.card` (block), `.card__title` (element of the block), `.card--featured` (variant). Every selector is a single flat class, so specificity stays low and equal, names say where styles belong, and components don't leak into each other. The downside is long class names and discipline needed across a team.

→ *More:* §13.7 Why CSS needs an architecture — and BEM

Q201. What is utility-first CSS (Tailwind)? Pros and cons?

You style with many small single-purpose classes in the HTML (`flex items-center gap-4 p-6 rounded-xl`). Tailwind v4 scans your source files and generates only the classes you use. Pros: no naming, no dead CSS, small final stylesheet, consistent design tokens, fast iteration. Cons: long class lists in markup, a learning curve, repeated class lists unless you extract components, and the HTML mixes structure and presentation. Responsive variants use prefixes (`md:grid-cols-3`), mobile-first.

Follow-up: How do you avoid repeating long class lists? Extract a component in your framework (a `Button` component), not `@apply` everywhere — Tailwind itself recommends components first.

→ *More:* §13.8 Utility-first CSS and Tailwind CSS v4

Q202. What are CSS Modules?

A build-time technique: each `.module.css` file's class names are rewritten to unique names (`.title` → `.Card_title__x7a2`) and imported into JavaScript as an object (`styles.title`). You get locally scoped class names with plain CSS and zero runtime cost.

→ *More:* §13.9 CSS Modules

Q203. What is CSS-in-JS? Pros and cons?

Writing styles in JavaScript, co-located with components (styled-components, Emotion, or zero-runtime tools like vanilla-extract, Linaria, StyleX). Pros: scoping, dynamic styles from props, dead-code elimination with the component. Cons of *runtime* libraries: they generate and inject CSS while rendering, adding JavaScript and slowing rendering, and they complicate server components and streaming SSR. The trend is toward zero-runtime or native CSS.

→ *More:* §13.10 CSS-in-JS

Q204. ⋆ BEM vs CSS Modules vs Tailwind vs CSS-in-JS — how do you choose?

All four solve CSS's **global namespace** problem in different ways: a convention (BEM), build-time unique names (CSS Modules), generated utilities (Tailwind), or styles owned by JS components (CSS-in-JS). Choose by team and stack: plain CSS + BEM + `@layer` for small sites or server-rendered pages; CSS Modules or Tailwind for React apps; zero-runtime CSS-in-JS if you need typed, themeable component libraries.

→ *More:* §13.11 Choosing an approach, §14.6 Q41

Q205. What existing CSS frameworks have you used? How would you improve them?

Talk about one honestly: Bootstrap (ready components, consistent but every site looks similar and you ship unused CSS), Tailwind (utilities), component libraries (MUI, Chakra, shadcn/ui). Improvements interviewers like to hear: ship design tokens as custom properties, use `@layer` so overrides are easy, use logical properties for RTL, tree-shake unused CSS, and meet WCAG out of the box.

→ *More:* §13.11 Choosing an approach

Q206. What are design tokens?

Named, platform-independent design decisions — colours, spacing, font sizes, radii, shadows, durations. Usually in three tiers: **primitive** (`--blue-600`), **semantic** (`--color-action`), **component** (`--button-bg`). Components use only semantic/component tokens, so theming or rebranding means changing the mapping, not the components.

→ *More:* §13.14 Theming and dark mode with custom properties

Q207. How do you organise CSS in a large codebase so it stays maintainable?

A layer order (`@layer reset, base, components, utilities`), low and flat specificity (single classes, `:where()` for defaults), component-scoped styles (BEM, CSS Modules or `@scope`), design tokens as custom properties, no IDs or `!important` for styling, stylelint in CI, and deleting dead CSS when components are removed. Make the cascade predictable rather than fighting it.

→ *More:* §13.4 Cascade layers (`@layer`), §13.7 Why CSS needs an architecture — and BEM

Performance and the browser rendering pipeline

Q208. ⋆ What happens when you type a URL and press Enter?

The browser checks caches, resolves the domain to an IP address (DNS), opens a TCP connection and a TLS handshake (HTTPS) — or QUIC for HTTP/3 — sends the HTTP request, and receives the response. It then parses HTML into the DOM, fetches CSS/JS/images, builds the CSSOM from the CSS, combines them into the render tree, computes layout, paints pixels and composites layers onto the screen; JavaScript runs along the way and can change things.

Follow-up: What would you say about caching at each step? Browser memory/disk cache and service worker before the network; DNS caches in the OS and resolver; HTTP caching headers (`Cache-Control`, `ETag`) decide whether the response is reused or revalidated.

→ *More:* §1.6 "What happens when you type a URL and press Enter?", §14.6 Q43

Q209. ⋆ Explain the browser rendering pipeline (critical rendering path).

1. **Parse HTML → DOM**. 2. **Parse CSS → CSSOM** (CSS Object Model). 3. **Style**: match rules to each element, computing styles (the visible elements form the **render tree**). 4. **Layout** (reflow): compute every box's size and position. 5. **Paint**: draw text, colours, borders and images into layers. 6. **Composite**: combine layers in the right order on the GPU. The **critical rendering path** is the minimum work needed for the first paint, and you speed it up by reducing render-blocking CSS/JS.

Follow-up: What does JavaScript do to this pipeline? A parser-blocking script stops DOM construction; JS that changes styles or the DOM re-runs style, layout and paint for the affected part.

→ *More:* §1.7 The browser rendering pipeline, §14.1 The critical rendering path

Q210. ⋆ What is the difference between layout (reflow), painting and compositing?

Layout calculates geometry — triggered by changes to size or position (`width`, `font-size`, adding elements); it is the most expensive because it can cascade through the page. **Paint** fills in pixels — triggered by visual changes that don't move anything (`color`, `background`, `box-shadow`). **Compositing** only reassembles already-painted layers — `transform`, `opacity` — and is the cheapest.

Follow-up: Give one property for each: `width` → layout, `color` → paint, `transform` → composite only.

→ *More:* §1.7 The browser rendering pipeline, §12.7 Performance: animate `transform` and `opacity`

Q211. ⋆ What causes reflow and repaint, and how do you reduce them?

Reflow: changing geometry, adding/removing DOM nodes, changing fonts or window size, and **reading** layout values (`offsetWidth`, `getBoundingClientRect()`) after a write, which forces a synchronous layout. Reduce it by batching DOM reads before writes, changing classes instead of many inline styles, animating `transform`/`opacity`, using `contain: layout paint` / `content-visibility: auto` to limit the affected area, and avoiding **layout thrashing** — alternating writes and reads (`el.style.width = …; h += el.offsetHeight`) in a loop, which forces a fresh layout on every iteration; read everything first, then write.

→ *More:* §1.7 The browser rendering pipeline, §12.7 Performance: animate `transform` and `opacity`

Q212. What are render-blocking resources, and how do you eliminate them?

CSS in the `<head>` blocks rendering (the browser won't paint until it has the CSSOM, to avoid FOUC). **Synchronous scripts** block HTML parsing (and wait for preceding CSS). Fixes: inline the critical CSS and load the rest without blocking; split CSS by media (`media="print"` doesn't block); `defer` or `async` scripts; remove unused CSS/JS; avoid `@import` chains.

→ *More:* §14.2 Render-blocking resources, §14.6 Q44

Q213. What is critical CSS?

The minimal CSS needed to render the above-the-fold (initially visible) content, inlined in a `<style>` in the `<head>` so first paint doesn't wait for a network request; the full stylesheet loads afterwards. Tools like

Critters/Beasties extract it automatically. Trade-off: inlined CSS isn't cached across pages, so keep it small (≈ 14 KB, roughly what fits in the first round trip).

→ *More:* §14.2 Render-blocking resources

Q214. ⋆ What are the Core Web Vitals and their thresholds?

Google's user-experience metrics, measured at the 75th percentile of real visits: **LCP** (Largest Contentful Paint — when the biggest image or text block is drawn) ≤ 2.5 s; **INP** (Interaction to Next Paint — the delay from a click/tap/keypress to the next visual update, across the visit) ≤ 200 ms; **CLS** (Cumulative Layout Shift — how much visible content unexpectedly moves) ≤ 0.1. INP replaced FID in March 2024.

Follow-up: Why did INP replace FID? FID measured only the input delay of the first interaction; INP measures the full latency (delay + processing + rendering) of the slowest interactions across the whole visit, which matches what users feel.

→ *More:* §14.3 Core Web Vitals: LCP, INP, CLS, §14.6 Q45

Q215. How do you improve LCP, INP and CLS?

LCP: fast server response (TTFB) and CDN; make the LCP image discoverable in the HTML (not CSS or JS), `fetchpriority="high"`, never lazy-load it, compress it (AVIF/WebP, right size); remove render-blocking CSS/JS. **INP**: break up long JavaScript tasks (> 50 ms), yield to the main thread, do less work in event handlers, avoid huge DOMs. **CLS**: `width`/`height` or `aspect-ratio` on media, reserve space for ads and embeds, don't insert content above existing content, use `font-display` with metric-matched fallbacks, animate with `transform`.

→ *More:* §14.4 Fixing each metric (and the other basics)

Q216. Lab data vs field data?

Lab data (Lighthouse, DevTools) is measured in one simulated environment — reproducible, good for debugging, but it can't measure real INP. **Field data** (CrUX, the Chrome User Experience Report, or your own RUM — real-user monitoring with the `web-vitals` library) comes from actual visitors on their devices and networks; Google uses field data for ranking.

→ *More:* §14.3 Core Web Vitals: LCP, INP, CLS

Q217. ⋆ What are some "gotchas" for writing efficient CSS?

Big, unused stylesheets (download and parse cost); `@import` chains; expensive effects on large areas (big `box-shadow`, `filter: blur`, `backdrop-filter`) especially during scroll or animation; animating layout properties; very deep, overqualified selectors (`div.nav ul li a`) mainly hurt maintainability; universal selectors with costly properties; and huge numbers of DOM nodes. Use `content-visibility: auto` to skip rendering off-screen sections.

→ *More:* §14.4 Fixing each metric (and the other basics), §12.7 Performance: animate `transform` and `opacity`

Q218. ⋆ What are some ways to improve scrolling performance?

Use passive event listeners for `touchstart`/`wheel` (`{ passive: true }`); throttle scroll handlers or replace them with `IntersectionObserver` (lazy loading, reveal effects) and scroll-driven animations; avoid `background-attachment: fixed` and big repaints; promote animated elements with `transform`; use `content-visibility: auto` for long pages; virtualise very long lists (render only visible rows).

→ *More:* §12.7 Performance: animate `transform` and `opacity`, §14.4 Fixing each metric (and the other basics)

Q219. What tools would you use to find a performance problem?

Chrome DevTools: **Performance** panel (flame chart of scripting, style, layout, paint; long tasks), **Network** panel (waterfall, blocking requests, sizes), **Lighthouse** (lab audit with suggestions), the **Rendering** tab (paint flashing, layout-shift regions, layer borders), **Coverage** (unused CSS/JS); WebPageTest for real devices and networks; and field data from CrUX / PageSpeed Insights or RUM.

→ *More:* §1.8 Browser DevTools, §14.3 Core Web Vitals: LCP, INP, CLS

Q220. How do you optimise asset loading on a page overall?

Fewer and smaller bytes: minify and compress (Brotli/gzip), modern image formats with the right sizes, subset WOFF2 fonts, remove unused CSS/JS, code-split JS. Earlier: CDN, HTTP/2 or 3, `preconnect`/`preload` for critical resources, critical CSS. Smarter: `defer`/`async`, lazy-load below-the-fold images and iframes, long `Cache-Control` lifetimes with fingerprinted (hashed) file names.

→ *More:* §14.4 Fixing each metric (and the other basics), §1.5 HTTP basics — requests and responses

---

"Build this layout" — common live-coding tasks

Interviewers often hand you a screenshot or a sentence and watch you build it in 10–30 minutes. Say what element you choose and why, start with semantic HTML, then layout, then polish (focus, responsive, reduced motion).

Q221. ⋆ Build the holy grail layout: header, footer, a fluid main column and two sidebars; footer at the bottom even when content is short.

```
body { min-height: 100svh; margin: 0; display: grid;
  grid-template: "head head head" auto "nav main ads" 1fr "foot foot foot" auto
               / 12rem minmax(0, 1fr) 12rem; }
header { grid-area: head } nav { grid-area: nav } main { grid-area: main }
aside { grid-area: ads } footer { grid-area: foot }
@media (width < 48rem) {
  body { grid-template: "head" auto "nav" auto "main" 1fr "ads" auto "foot" auto / 1fr; }
}
```

The `1fr` row (the main row) absorbs extra height, which keeps the footer at the bottom. In `grid-template` each row size follows its own string, so in the one-column version `main` must carry the `1fr` — writing `"foot" 1fr` would make the *footer* row swallow all the spare height.

Follow-up: Why `minmax(0, 1fr)` for the main column? So long content (code blocks, tables) can't force the column wider than the space and cause horizontal scrolling.

→ *More:* §9.9 Patterns

Q222. ⋆ Build a sticky footer (footer at the bottom of the viewport on short pages, after the content on long ones).

```
body { min-height: 100svh; display: flex; flex-direction: column; margin: 0; }
main { flex: 1; }
```

Or grid: `body { display: grid; grid-template-rows: auto 1fr auto; min-height: 100svh; }`. Don't use `position: fixed` — that covers content on long pages.

→ *More:* §8.7 Common patterns

Q223. ⋆ Build a responsive navbar: logo on the left, links on the right; on mobile, a menu button toggles the links.

HTML: `<header><a class="logo" href="/">Acme</a><nav aria-label="Main"><button aria-expanded="false" aria-controls="menu">Menu</button><ul id="menu">…</ul></nav></header>`.

```
header { display: flex; align-items: center; justify-content: space-between; gap: 1rem; }
#menu { display: flex; gap: 1rem; list-style: none; }
nav button { display: none; }
@media (width < 40rem) {
  nav button { display: block; }
  #menu { display: none; flex-direction: column; }
  nav:has(button[aria-expanded="true"]) #menu { display: flex; }
}
```

A few lines of JS flip `aria-expanded`.

Follow-up: Why not a CSS-only checkbox hack for the menu? It works without JS but announces as a checkbox, not a menu button, and can't manage `aria-expanded`; a real `<button>` with a few lines of JS is more accessible.

→ *More:* §10.4 Mobile-first in practice, §8.7 Common patterns

Q224. Build a responsive card grid: 1 column on phones, as many as fit on wider screens, equal-height cards with the button at the bottom.

```
.grid { display: grid; gap: 1.5rem;
  grid-template-columns: repeat(auto-fill, minmax(min(18rem, 100%), 1fr)); }
.card { display: flex; flex-direction: column; }
.card .actions { margin-top: auto; }
.card img { width: 100%; aspect-ratio: 16 / 9; object-fit: cover; }
```

Grid stretches the cards in each row to the same height; `margin-top: auto` pushes the button down. Use `subgrid` if titles must align across cards.

→ *More:* §9.9 Patterns, §9.7 Subgrid

Q225. Build a two-column layout with a fixed-width sidebar and a fluid content area (sidebar stacks on mobile).

```
.layout { display: grid; grid-template-columns: 16rem minmax(0, 1fr); gap: 2rem; }
@media (width < 48rem) { .layout { grid-template-columns: 1fr; } }
```

A breakpoint-free alternative ("sidebar" pattern): `display: flex; flex-wrap: wrap;` with the sidebar `flex: 1 1 16rem` and the content `flex: 999 1 30rem` — the content wraps under when it would get narrower than 30rem.

→ *More:* §10.9 Intrinsic layouts — responsive without breakpoints

Q226. ⋆ Build an accessible modal dialog.

Use `<dialog>` with `showModal()`: focus moves in, the rest of the page is inert, Escape closes, and it's in the top layer. Add a heading referenced by `aria-labelledby`, a close button, `::backdrop` styling, and return focus to the trigger on close (the browser does this for `<dialog>`). Lock background scroll with `body:has(dialog[open]) { overflow: hidden; }`.

```
dialog { max-width: min(32rem, 100% - 2rem); max-height: calc(100svh - 2rem); overflow: auto; }
dialog::backdrop { background: rgb(0 0 0 / .5); }
```

Follow-up: What if you can't use `<dialog>`? Render the modal at the end of `<body>`, set `inert` on the rest of the page, move focus in, handle Escape, and restore focus on close — exactly what `<dialog>` does for you.

→ *More:* §3.7 Keyboard and focus, §12.8 Enter and exit animations: `@starting-style` and `allow-discrete`

Q227. Build a tooltip with CSS only.

```
.tip { position: relative; }
.tip [role="tooltip"] { position: absolute; bottom: calc(100% + 8px); left: 50%;
  translate: -50% 0; white-space: nowrap; opacity: 0; pointer-events: none;
  transition: opacity .15s; }
.tip:is(:hover, :focus-within) [role="tooltip"] { opacity: 1; }
```

Make it show on keyboard focus too (`:focus-within`), link it with `aria-describedby`, keep it hoverable and dismissable with Escape (WCAG 1.4.13). Mention that anchor positioning plus `popover` now handles flipping at screen edges natively.

→ *More:* §7.6 Positioning

Q228. Build a dropdown menu.

A `<button aria-expanded aria-controls>` toggles a positioned list (`position: absolute; top: 100%` inside a `position: relative` wrapper), with links inside. Close on Escape and on outside click, return focus to the button. Native options: `<details><summary>`, or the `popover` attribute (`<button popovertarget="m">` + `<div id="m" popover>`), which handles light-dismiss and the top layer without JS.

→ *More:* §3.6 ARIA, §12.8 Enter and exit animations: `@starting-style` and `allow-discrete`

Q229. Build tabs or an accordion with minimal JavaScript.

Accordion: native `<details name="faq"><summary>Question</summary>Answer</details>` — giving several `<details>` the same `name` makes them exclusive (only one open). Tabs: `role="tablist"` of `<button role="tab" aria-selected aria-controls>`, panels with `role="tabpanel"` and `hidden` on inactive ones; arrow keys move between tabs, Tab moves into the panel.

→ *More:* §3.6 ARIA

Q230. Build a CSS-only toggle switch from a checkbox.

```
.switch input { appearance: none; inline-size: 3rem; block-size: 1.6rem;
  border-radius: 1rem; background: #999; position: relative; cursor: pointer; }
.switch input::before { content: ""; position: absolute; inset: .2rem auto .2rem .2rem;
  aspect-ratio: 1; border-radius: 50%; background: #fff; transition: translate .2s; }
.switch input:checked { background: #1d4ed8; }
.switch input:checked::before { translate: 1.4rem 0; }
```

Keep the real `<input type="checkbox" role="switch">` and a `<label>` so it stays keyboard- and screen-reader-accessible; keep `:focus-visible` styles.

→ *More:* §6.3 Pseudo-classes and pseudo-elements, §5.3 Input types

Q231. Build a loading spinner and a skeleton loader.

```
.spinner { inline-size: 2rem; aspect-ratio: 1; border-radius: 50%;
  border: 4px solid #ddd; border-top-color: #1d4ed8; animation: spin 1s linear infinite; }
@keyframes spin { to { rotate: 1turn; } }
.skeleton { background: linear-gradient(90deg, #eee 25%, #f5f5f5 50%, #eee 75%) 0 0 / 200% 100%;
  animation: shimmer 1.5s infinite; }
@keyframes shimmer { to { background-position: -200% 0; } }
```

Add `role="status"` with visually hidden "Loading…" text, and slow or stop the motion under `prefers-reduced-motion: reduce`.

→ *More:* §12.5 Keyframe animations, §12.9 Reduced motion and accessibility

Q232. Make a full-bleed hero section with text over an image, readable at all sizes.

```
.hero { display: grid; min-height: 70svh; }
.hero > * { grid-area: 1 / 1; }              /* stack image and text in one cell */
.hero img { width: 100%; height: 100%; object-fit: cover; }
.hero .text { align-self: end; padding: 2rem;
  background: linear-gradient(transparent, rgb(0 0 0 / .7)); color: #fff; }
```

Grid stacking avoids absolute positioning; the dark gradient helps keep text contrast at 4.5:1 or better over any photo (check the lightest part of the image). The hero image is the likely LCP element: give it `fetchpriority="high"` and don't lazy-load it.

→ *More:* §9.4 Placing items, §11.9 `object-fit` and `object-position`

Q233. Build a responsive data table that works on small screens.

Keep a real `<table>` with `<caption>` and `<th scope>`. Wrap it in a container with `overflow-x: auto` (plus `tabindex="0"`, `role="region"` and an `aria-label` so keyboard users can scroll it), and optionally make the first column `position: sticky; left: 0`. Turning rows into stacked cards with `display: block` on cells can remove table semantics in some browsers — test with a screen reader.

→ *More:* §5.1 Table semantics, §10.4 Mobile-first in practice

Q234. Style a custom checkbox or radio button accessibly.

Keep the native input (so keyboard, focus and screen readers work), remove its default look with `appearance: none`, size it, draw the checked state with `::before` or a background, and use `:checked`, `:focus-visible`, `:disabled` for states. The simplest option: `accent-color: #1d4ed8` restyles native checkboxes, radios, range sliders and progress bars in one line.

→ *More:* §5.3 Input types, §6.3 Pseudo-classes and pseudo-elements

---

Scenarios, troubleshooting and senior-level questions

Q235. A colleague's CSS isn't applying. How do you debug it?

Open DevTools → Elements → Styles on the element: is the rule listed at all (if not: selector doesn't match, file not loaded — check Network, or a syntax error above it)? Is it **crossed out** (overridden — the Computed tab shows which rule wins and why: specificity, layer, order, `!important`)? Is there a warning icon (invalid value or a property that doesn't apply, like `width` on an inline element or `z-index` on a static one)? Also check media/container query conditions and inherited vs initial values.

→ *More:* §1.8 Browser DevTools, §6.10 How it all fits together

Q236. Why doesn't `height: 100%` work on my element?

Percentage heights resolve against the parent's **height**, and if the parent's height depends on its content (`auto`), the percentage is treated as `auto`. Give every ancestor a definite height, or better: use `min-height: 100svh` on the page container, flex/grid stretching (`align-items: stretch` is the default), or `height: 100%` inside a grid/flex item, which does have a definite size.

→ *More:* §7.1 The box model

Q237. My sticky navbar works on desktop but not on mobile Safari (or not at all). What could be wrong?

Check the usual sticky killers: an ancestor with `overflow: hidden/auto` (often added on `body` or a wrapper to stop horizontal scrolling — `overflow-x: clip` is the safe alternative), missing `top: 0`, a parent no taller than the nav, `align-self: stretch` inside flex/grid. Older Safari needed `-webkit-sticky`. Also check the `100vh`/toolbar behaviour if the nav sits inside a full-height container.

→ *More:* §7.6 Positioning, §7.7 Overflow

Q238. ⋆ The text in my flex row overflows instead of wrapping (or my grid breaks on mobile with horizontal scrolling). What's wrong?

Automatic minimum sizes: flex items have `min-width: auto` and `1fr` grid tracks have an `auto` minimum, so long words, URLs, `<pre>` blocks, tables or wide images refuse to shrink. Fix with `min-width: 0` on the flex item, `minmax(0, 1fr)` for grid tracks, `overflow-wrap: anywhere` for text, and `max-width: 100%` for media. Also check for fixed pixel widths and `minmax(300px, 1fr)` on screens narrower than 300px (use `min(300px, 100%)`).

→ *More:* §8.4 The `flex` shorthand and minimum sizes, §9.2 Defining tracks

Q239. An image inside a link adds strange extra height, or a button and an input next to each other don't line up. Why?

Inline-level boxes sit on the text baseline and include the line's descender space and the parent's `line-height`. Set `display: block` on the image, or align form controls with flexbox (`align-items: center`) and `font: inherit` so the input and button share font metrics.

→ *More:* §7.4 `display`

Q240. A page shows a flash of unstyled content (FOUC) or the dark theme flashes white first. How do you fix it?

FOUC: stylesheets loaded late (by JavaScript, at the end of the body, or via `@import` chains) — put `<link rel="stylesheet">` in the head, or inline critical CSS. Dark-mode flash: the theme is applied after first paint — run a tiny inline script in `<head>` that reads the saved theme and sets `data-theme` before the body renders, and set `color-scheme` so the browser's default background is dark too.

→ *More:* §14.2 Render-blocking resources, §13.14 Theming and dark mode with custom properties

Q241. How do you make a site work well in Windows high-contrast (forced-colors) mode?

In forced-colours mode the OS replaces your colours with a small user palette, and removes backgrounds and box-shadows. Use real borders or outlines (transparent ones become visible) for focus rings and button edges, use SVG icons with `fill: currentColor`, don't convey state by background colour alone, and test with `@media (forced-colors: active)` (using system colours like `CanvasText`, `Highlight` if you need fixes).

→ *More:* §11.6 Borders, radius and outlines, §10.5 User-preference media queries

Q242. How would you make an existing page accessible — where do you start?

Run an automated audit (axe/Lighthouse) for quick wins, then a keyboard-only pass (focus visible, order logical, no traps, everything operable), then fix semantics: landmarks, one `<h1>` and a heading hierarchy, real buttons and links, labels on all inputs, alt text, sufficient contrast, `lang`, and error messages tied to fields. Then a screen-reader pass and zoom/reflow testing. Prioritise blockers (can't complete a key task) over cosmetic issues.

→ *More:* §3.10 Testing tools, §3.9 WCAG 2.2

Q243. How do you reduce CSS bundle size in a large app?

Measure with the DevTools Coverage tab; remove dead CSS (PurgeCSS, Tailwind's source scanning, CSS Modules tree-shaken with components); split CSS per route or component; replace heavy frameworks with what you use; minify with Lightning CSS or cssnano; prefer modern CSS (fewer prefixes and hacks) with a Browserslist target; and inline only the critical part.

→ *More:* §13.8 Utility-first CSS and Tailwind CSS v4, §14.4 Fixing each metric (and the other basics)

Q244. How would you set up theming for multiple brands in one component library?

Components consume only semantic custom properties (`--color-action`, `--radius-control`). Each brand is a set of token values applied with a class or `data-brand` attribute on a root element (`[data-brand="acme"] { --color-action: oklch(55% .2 260); }`), shipped in their own `@layer tokens`. Dark mode is another axis of the same tokens (`light-dark()`). Generate token files from one source (e.g. Style Dictionary) for web, iOS and Android.

→ *More:* §13.14 Theming and dark mode with custom properties

Q245. How do the HTML, CSS and JavaScript of a page work together, and what is the DOM vs the CSSOM?

HTML gives structure and content (parsed into the **DOM**, a tree of node objects JavaScript can read and change). CSS gives presentation (parsed into the **CSSOM**, the tree of style rules). The browser combines DOM and CSSOM to render. JavaScript adds behaviour: it can modify both trees, which triggers style, layout and paint again. Scripts can also block parsing, and a script that reads styles must wait for pending CSS — which is why loading order matters.

→ *More:* §1.7 The browser rendering pipeline, §14.1 The critical rendering path

Q246. How do you safely insert user content into the page?

Use `textContent` (or framework templating that escapes by default), never `innerHTML` with untrusted strings — that allows XSS. If you must render user HTML, sanitise it (DOMPurify, or the newer `setHTML()` Sanitizer API where supported). Add a Content Security Policy (CSP, an HTTP header restricting where scripts may load from) as a second line of defence.

Q247. What is clickjacking, and how do you prevent it?

An attacker loads your site in an invisible `<iframe>` over a decoy page, tricking users into clicking your buttons. Prevent it by forbidding framing: `Content-Security-Policy: frame-ancestors 'self'` (modern) or `X-Frame-Options: DENY` (legacy).

→ *More:* §4.7 Embedding other pages — `<iframe>`

Q248. What's your process for turning a design (Figma) into HTML and CSS?

Identify the content structure first and write semantic HTML; extract design tokens (colours, type scale, spacing) into custom properties; identify reusable components; choose layout per region (grid for the page, flex for components); build mobile-first with fluid sizing (`clamp`, `min()`), adding breakpoints only where the content needs them; then states (hover, focus, disabled, error, empty, loading), accessibility checks and cross-browser testing.

→ *More:* §10.4 Mobile-first in practice, §13.14 Theming and dark mode with custom properties

Q249. ⋆ What's new in CSS that you're excited about (or have used recently)?

Pick two or three you can explain: container queries (components that adapt to their space), `:has()` (parent selection, less JS), cascade layers (no more specificity wars), native nesting, `@scope`, subgrid, `@starting-style` and view transitions (smooth UI without libraries), anchor positioning with `popover` (tooltips without JS), `oklch()`/`color-mix()`/`light-dark()` for theming, and scroll-driven animations.

→ *More:* §13.3 `:has()` — the relational ("parent") selector, §13.4 Cascade layers (`@layer`), §10.6 Container queries

Q250. What is the `popover` attribute?

`<div id="menu" popover>` plus `<button popovertarget="menu">` gives a toggleable panel with no JavaScript: it renders in the top layer (above every `z-index`), closes on Escape or an outside click ("light dismiss"), and the browser handles focus return. `:popover-open` styles the open state. Baseline since January 2025.

→ *More:* §12.8 Enter and exit animations: `@starting-style` and `allow-discrete`, §7.8 `z-index` and stacking contexts

---

Sources of the question lists

Question wording and frequency only; all answers above are written in my own words.

- h5bp — *Front-end Developer Interview Questions*: HTML Questions, CSS Questions, Performance Questions, General Questions (github.com/h5bp/Front-end-Developer-Interview-Questions)
- GreatFrontEnd — *CSS Interview Questions Guide for 2026* and the HTML/CSS quiz question list (greatfrontend.com)
- GreatFrontEnd — *Machine Coding Interview Questions for Freshers (2026 Guide)* (for the "build this" tasks)
- InterviewBit — *HTML Interview Questions* (HTML, HTML5, Advanced HTML and HTML/CSS/JavaScript sections)
- golinuxcloud — *CSS Interview Questions and Answers*; vskills — *Top 100 CSS Interview Questions*
- DEV Community — *Most frequently asked front-end interview questions*
- web.dev — *New to the web platform* monthly posts (to check browser support for anchor positioning, popover, view transitions)
- This book's own Chapter 14 §14.6 (all 47 questions are included above and linked)